

Understanding Intelligence Through Robotics

**From Geometry and Control to Learning, World Models, and Safe
Autonomy**

Chien-Hsiang Yeh

8 September 2026

Table of contents

Preface	xv
Unified Notation and Conventions	xvi
I Motion, Mechanics, and Control	1
1 Linear Algebra Foundations	2
Why linear algebra matters	2
Prerequisites and assumptions	2
Prerequisites	2
Assumptions	2
Dependency map	3
1.1 Vectors: quantities with magnitude and direction	3
1.1.1 Geometric vector versus coordinate column	3
1.1.2 Addition and scalar multiplication	4
1.1.3 Length and unit direction	4
1.1.4 Dot product and angle	5
1.1.5 Cross product and useful identities	7
1.1.6 Scalar triple product	8
1.1.7 Linear combinations, span, and basis	9
Quick test A: vectors and bases	10
Answers and hints	10
1.2 Matrices: linear maps and basis changes	11
1.2.1 Matrix dimensions	11
1.2.2 Identity, transpose, null space, rank, and inverse	12
1.2.3 Multiplication order	16
Quick test B: matrices, rank, and inverses	17
Answers and hints	17
1.3 Determinants and signed volume	17
1.3.1 What the determinant measures	17
1.3.2 Algebraic definition	19
1.3.3 Why determinants multiply	22
1.3.4 How an invertible linear map transforms a cross product	28
Quick test C: determinants	29
Answers and hints	29
1.4 Orthogonal matrices	30

Quick test D: orthogonal matrices	31
Answers and hints	31
Cumulative chapter test	31
Cumulative answers and hints	32
References	33
2 Geometry and Coordinate Frames	34
Why coordinate frames matter	34
Prerequisites and assumptions	34
Prerequisites	34
Assumptions	35
Dependency map	35
2.1 Cross product and rigid-body velocity	35
2.1.1 Example: velocity of a point on a rotating rigid body	36
Quick test A: cross products and rigid-body velocity	39
Answers and hints	39
2.2 Coordinate frames	39
2.2.1 What defines a frame?	39
2.2.2 Orientation convention	40
2.2.3 Points require translation	44
2.3 Worked frame-change example	45
2.3.1 Interactive frame-rotation graph	47
2.4 Differential-drive interpretation	48
2.5 Failure modes and engineering checks	49
2.5.1 Missing frame labels	49
2.5.2 Reversed transformation	49
2.5.3 Mixed units	49
2.5.4 Point-vector confusion	49
2.5.5 Invalid numerical rotation	49
Quick test B: coordinate frames and rotations	50
Answers and hints	50
Cumulative chapter test	51
Cumulative answers and hints	52
References	55
3 Differential-Drive Kinematics and Numerical Integration: Model and Derivation	56
Why this chapter matters	56
Prerequisites and assumptions	56
3.1 Planar pose and its derivative	58
3.1.1 Deriving the planar model	59
3.1.2 The no-sideways-motion constraint	59
Quick test A: planar kinematics	61
3.2 Differential-drive wheel kinematics	62
3.2.1 Relating wheel speeds to body motion	63
3.2.2 Characteristic motions	66
Quick test B: wheel-to-body mapping	68

3.3	From velocity to a finite pose change	69
3.3.1	Exact straight motion	69
3.3.2	Exact constant-curvature motion	70
	Quick test C: exact finite motion	72
3.4	Numerical integration with Forward Euler	72
3.4.1	Derivation from a Taylor expansion	73
3.4.2	Local and accumulated error	73
3.4.3	Worked comparison with the exact update	74
3.4.4	Choosing and checking an integration method	75
	Quick test D: Forward Euler	75
3.5	Engineering implications and failure modes	76
	Cumulative chapter test	76
	Cumulative answers and hints	77
	References	78
4	Differential-Drive Kinematics and Numerical Integration: Implementation	79
	Purpose and scope	79
4.1	Wheel-to-body velocity mapping	79
4.1.1	Software contract	80
4.1.2	Simplified reference snippet	80
4.1.3	Invariants	81
4.2	Body-to-wheel velocity mapping	81
4.2.1	Software contract	81
4.2.2	Implementation mapping	81
4.2.3	Invariants and required tests	82
4.3	Exact constant-input pose integration	83
4.3.1	Software contract	83
4.3.2	Numerically stable implementation mapping	84
4.3.3	Invariants and required tests	85
4.4	Forward Euler pose integration	85
4.4.1	Software contract	85
4.4.2	Implementation mapping	86
4.4.3	Exact and approximate cases	86
4.4.4	Invariants and required tests	86
5	ROS 2 Package Development Workflow	87
	Goal	87
5.1	Source the ROS 2 underlay	87
5.2	Create a workspace	88
5.3	Create a Python package	88
5.4	Inspect the generated package	89
5.5	Complete the package metadata	90
5.6	Resolve dependencies and run fast tests	91
5.7	Build and install the package	92
5.7.1	What <code>--symlink-install</code> changes	93
5.8	Source the overlay and run the package	93

5.9	Run tests through <code>colcon</code>	94
5.10	Repeat the daily development loop	94
	Common failure checks	95
	References	95
6	Planar Rigid-Body Motion: $SE(2)$ and Twists	96
	Why this chapter matters	96
	Prerequisites and assumptions	96
6.1	Pose coordinates and rigid transformations	96
6.1.1	A pose is a relative geometric relationship	96
6.1.2	Planar rotations and $SO(2)$	97
6.1.3	Transforming point coordinates by rotation and translation	98
6.1.4	Homogeneous transformations and $SE(2)$	99
6.1.5	Composing transformations	100
6.1.6	Inverting a transformation	101
6.1.7	Pose coordinates versus the pose itself	102
6.1.8	Worked example	103
	Quick test A: pose and $SE(2)$	103
	Answers and hints	104
6.2	Passive coordinate changes and active rigid motions	104
6.2.1	Passive coordinate change	105
6.2.2	Active rigid-body motion	105
6.2.3	One matrix form, three interpretations	106
6.2.4	Composition order	107
6.2.5	Why composition in $SE(2)$ is not commutative	107
6.2.6	Interpretation failures and engineering checks	108
	Quick test B: interpretation and composition	109
	Answers and hints	109
6.3	Time-varying rotations and rigid-body point velocity	110
6.3.1	Translational and angular velocity	110
6.3.2	Deriving the rotation-matrix derivative	111
6.3.3	Why $\dot{\mathbf{R}}_{wb}$ is not a rotation matrix	112
6.3.4	Velocity of a point fixed to the body	117
6.3.5	Worked example	119
6.3.6	Assumptions, failure modes, and checks	120
	Quick test C: rotation derivatives and point velocity	120
	Answers and hints	121
6.4	Planar twists and $\mathfrak{se}(2)$	121
6.4.1	Twist coordinates	121
6.4.2	The hat operator and $\mathfrak{se}(2)$	122
6.4.3	Body twist	126
6.4.4	Spatial twist expressed in the world frame	129
6.4.5	Relating body and spatial twists	132
6.4.6	Differential-drive body twist	134
6.4.7	Worked example: the two twist representations	135

6.4.8	Assumptions, failure modes, and checks	136
Quick test D:	planar twists	137
	Answers and hints	137
Cumulative chapter review		138
	Answers and hints	138
References		139
7	Degrees of Freedom and Spatial Motion: $SO(3)$ and $SE(3)$	140
	Why this chapter matters	140
	Prerequisites and assumptions	140
7.1	Degrees of freedom and configuration space	140
7.1.1	Configuration is a complete geometric description	140
7.1.2	Definition of degree of freedom	143
7.1.3	Why a free planar rigid body has three degrees of freedom	146
7.1.4	Why a free spatial rigid body has six degrees of freedom	147
7.1.5	Constraints, actuators, state variables, and degrees of freedom	147
7.1.6	Worked examples	148
7.1.7	Assumptions, limitations, and checks	149
Quick test A:	configuration and degrees of freedom	149
	Answers and hints	150
7.2	Spatial orientation and $SO(3)$	150
7.2.1	Constructing a spatial rotation matrix	150
7.2.2	Rotation constraints and the definition of $SO(3)$	152
7.2.3	Example: a positive quarter-turn about the world z_w -axis	153
Quick test B:	spatial orientation and $SO(3)$	154
	Answers and hints	154
7.3	Coordinate-axis rotations and composition	154
7.3.1	Constructing coordinate-axis rotation matrices	154
7.3.2	Composition order: the rightmost rotation acts first	157
7.3.3	Fixed-frame and body-frame increments	158
Quick test C:	coordinate-axis rotations and composition	160
	Answers and hints	160
7.4	Angular velocity as the rate of orientation change	161
7.4.1	From a finite turn to an instantaneous rate	161
7.4.2	Cross-product matrices and the orientation derivative	163
Quick test D:	angular velocity	169
	Answers and hints	170
7.5	Finite rotations from axis-angle coordinates	170
7.5.1	The problem: converting a rotation rate into a finite turn	170
7.5.2	The matrix exponential	172
7.5.3	Deriving Rodrigues' rotation formula	175
7.5.4	Euler's rotation theorem: every spatial rotation has an axis	177
7.5.5	Check: rotation about the world z_w -axis	183
7.5.6	Applying a constant angular velocity over a time interval	184

Quick test E: axis–angle coordinates and the exponential map	184
Answers and hints	185
7.6 Spatial pose and $SE(3)$	186
7.6.1 From a point transformation to a homogeneous matrix	186
7.6.2 Composing and inverting spatial transformations	188
7.6.3 Worked example: composition and inverse round trip	190
Quick test F: spatial pose and $SE(3)$	193
Answers and hints	194
7.7 Embedding planar poses in $SE(3)$	195
7.7.1 Recovering and checking a planar pose	199
Quick test G: planar poses inside $SE(3)$	200
Answers and hints	200
7.8 Unit quaternions and planar yaw	201
7.8.1 Why introduce unit quaternions	202
7.8.2 The Hamilton product and quaternion rotation	202
7.8.3 Why the half-angle formula produces the full rotation	205
7.8.4 Composing active rotations	207
7.8.5 Planar yaw and ROS 2 component order	208
7.8.6 Equivalent signs and numerical normalization	209
Quick test H: unit quaternions and planar yaw	210
Answers and hints	210
7.9 Spatial rigid-body velocity and twists	212
7.9.1 Velocity of a body-fixed point	212
7.9.2 Body twist and $\mathfrak{se}(3)$	214
7.9.3 Spatial twist expressed in the world frame	220
7.9.4 Relating body and spatial twists through the adjoint	224
Quick test I: spatial rigid-body velocity and twists	229
Answers and hints	229
7.10 Representation limitations and numerical checks	232
7.10.1 Equivalent information does not make representations interchange- able	232
7.10.2 Checking rotation matrices, transformations, and quaternions	232
7.10.3 Checking twists and their units	234
7.10.4 Round-trip checks and implementation consequences	234
Cumulative chapter review	236
Answers and hints	237
References	240
8 Screw Motion, the $SE(3)$ Exponential, and Spatial Pose Integration	241
Prerequisites and assumptions	241
8.1 Robot configurations, poses, and joint coordinates	241
Configuration and pose for links and joints	241
From free-body coordinates to joint coordinates	245
8.2 Screw axes and one-degree-of-freedom joint motion	248
8.2.1 The physical problem: one joint, one allowed motion	248

8.2.2	An oriented line in a declared frame	248
8.2.3	Revolute motion about the line	249
8.2.4	Helical motion and screw pitch	251
8.2.5	Prismatic motion along a direction	254
8.2.6	The normalisation rule and the declared component order	254
8.2.7	Worked checks	256
8.2.8	Contrasting the geometric and motion quantities	257
	Quick test A: screw axes and joint motion	258
8.3	Rigid motion from the $SE(3)$ exponential	259
8.3.1	From a constant twist to a finite pose	259
8.3.2	Closed forms for rotational and translational screws	261
8.3.3	Recovering screw coordinates with the $SE(3)$ logarithm	266
	Quick test B: the $SE(3)$ exponential and logarithm	274
8.3.4	From constrained motion to finite exponential coordinates	278
8.4	Constant-twist pose integration	280
8.4.1	Space-twist pose updates	281
8.4.2	Body-twist pose updates	283
8.4.3	Multiplication side and physical interpretation	285
	Quick test C: constant-twist pose integration	288
8.5	Numerical branches and validation	289
8.5.1	What logarithm branches mean	290
8.5.2	Small angular displacements	292
8.5.3	Recovering the principal logarithm, including rotations near π	300
8.5.4	Validation, deterministic evaluation, and evidence	307
8.5.5	Worked spatial cases	311
	Quick test D: numerical branches and validation	318
	Motion workflow and symbol summary	320
	The three workflows	320
	From ${}^a\eta$ to T_Δ and back	321
	Separating geometry from motion amount	322
	Symbol and purpose reference	323
	Cumulative chapter review	324
	Questions	324
	Answers and derivation hints	325
	Appendix: Chasles–Mozzi theorem and proof	331
	Theorem	331
	Proof	332
	References	338
9	Spatial Geometry Kernel Implementation	339
9.1	From C++ source to a reusable ROS 2 library	339
9.1.1	The roles of C++, CMake, ament, and colcon	339
9.1.2	What compile and link mean	340
9.1.3	The CMake library target	341
9.1.4	The test is another compiled target	343

9.1.5	What <code>colcon build</code> does	343
9.1.6	Install: create the consumer-facing file layout	344
9.1.7	Export: teach another package how to use the installation	345
9.2	Validated rotations and free-vector action	346
9.2.1	Mathematical contract	346
9.2.2	The complete class design	347
9.2.3	Numerical policy and error types	348
9.2.4	Complete <code>from_matrix</code> implementation	349
9.2.5	Complete <code>rotate_vector</code> implementation	351
9.2.6	Analytic example and focused tests	351
9.2.7	Deliberate limits	352
9.3	Validated rigid transformations and point action	352
9.3.1	The <code>Transform3</code> class	353
9.3.2	Complete <code>from_matrix</code> implementation	354
9.3.3	Complete <code>transform_point</code> implementation	355
9.3.4	Analytic example and focused tests	356
9.4	Rotation and transformation composition and inversion	357
9.4.1	Governing group operations	357
9.4.2	Public operations and matrix reconstruction	358
9.4.3	Complete <code>Rotation3</code> operations	359
9.4.4	Complete <code>Transform3</code> operations	359
9.4.5	Analytic fixtures and focused tests	360
9.4.6	Deliberate limits	361
9.5	Linear-first $SO(3)$ and $SE(3)$ hat and vee maps	361
9.5.1	Governing equations and component order	361
9.5.2	Public functions and the fixed component-order alias	364
9.5.3	Implementing the $SO(3)$ maps	364
9.5.4	Implementing the linear-first $SE(3)$ maps	365
9.5.5	Concrete round trip	367
9.5.6	Validation and focused evidence	367
9.5.7	Deliberate limits	368
9.6	Stable $SO(3)$ exponential from a rotation vector	368
9.6.1	Input, output, frame, and units	368
9.6.2	Rodrigues' formula and the two coefficients	369
9.6.3	Public factory and numerical policy	369
9.6.4	Validation and overflow-safe angle calculation	370
9.6.5	Exact-zero, small-angle, and nominal branches	370
9.6.6	Constructing the proper rotation	371
9.6.7	Concrete usage	372
9.6.8	Verification properties	373
9.6.9	Deliberate limits	373
9.7	Principal $SO(3)$ logarithm to a rotation vector	373
9.7.1	Input, output, frame, and units	374
9.7.2	Returning the coordinates and the formula branch	374
9.7.3	Recovering the principal angle	375

9.7.4	Identity, small-angle, and nominal formulas	376
9.7.5	Recovering the axis near π	377
9.7.6	Concrete half-turn and principal-branch examples	379
9.7.7	Validation and failure behaviour	380
9.7.8	Verification properties	380
9.7.9	Deliberate limits	381
9.8	Algorithm summaries for the $SO(3)$ exponential and principal logarithm	381
9.8.1	Exponential: rotation vector to rotation matrix	381
9.8.2	Principal logarithm: rotation matrix to rotation vector	382
9.8.3	Production checks omitted from the mathematical pseudocode	384
9.9	Why the kernel uses modelled motion coordinates	384
9.9.1	A normalised screw describes one motion direction	385
9.9.2	A twist describes the instantaneous motion	385
9.9.3	Exponential coordinates describe one finite motion increment	386
9.10	Linear-first $SE(3)$ exponential from finite coordinates	387
9.10.1	Why the left Jacobian produces the translation	388
9.10.2	Public factory and complete implementation	389
9.10.3	Equation-to-code map	392
9.10.4	Finite-pitch screw example	393
9.10.5	Validation and focused evidence	395
9.10.6	Deliberate limits	395
9.11	Principal linear-first $SE(3)$ logarithm to finite coordinates	396
9.11.1	Input, output, frame, and units	396
9.11.2	Why the linear block is recovered rather than copied	397
9.11.3	Identity rotation and exact translation preservation	398
9.11.4	Nonzero rotation and the inverse left Jacobian	399
9.11.5	Equation-to-code map	400
9.11.6	A representable near- π boundary	401
9.11.7	Finite-pitch quarter-turn example	402
9.11.8	Validation and focused evidence	403
9.11.9	Deliberate limits	404
9.12	Exact constant-twist pose integration	404
9.12.1	Input, output, frame, and units	404
9.12.2	Exact updates from the pose-rate equations	405
9.12.3	From velocity to a finite $SE(3)$ increment	406
9.12.4	Equation-to-code map	407
9.12.5	Equivalent body and space quarter-circle example	408
9.12.6	Validation and deliberate limits	418
9.13	Linear-first $SE(3)$ adjoint	419
9.13.1	Input, output, frame, and units	419
9.13.2	The conjugation identity checks the convention	420
9.13.3	Public method and implementation	421
9.13.4	Usage and analytic result	422
9.13.5	Validation and focused evidence	423
9.13.6	Deliberate limits	424

9.14	Planar pose embedding and extraction	424
9.14.1	Purpose, inputs, outputs, frames, and units	424
9.15	Normalized planar-yaw quaternion conversion	428
9.15.1	Purpose, convention, and mathematical result	428
9.15.2	C++ representation and implementation	429
9.15.3	Focused verification and deliberate limit	430
	References	430
10	General Robot Kinematics and Jacobians	431
	Prerequisites and assumptions	431
	From one joint to a complete robot	431
10.1	Configuration coordinates and forward kinematics	432
10.1.1	From joint values to a robot configuration	432
10.1.2	Task variables and the forward map	434
10.1.3	The total derivative and multivariable chain rule	438
	Quick test A: coordinates, forward maps, and total derivatives	444
10.2	Serial-chain pose through products of exponentials	445
10.2.1	Home configuration and joint screw axes	446
10.2.2	Space-form product of exponentials	450
10.2.3	Body-form product of exponentials	457
10.2.4	Equivalence with a transform chain	463
	Quick test B: serial-chain products of exponentials	469
10.3	Velocity kinematics and robot Jacobians	472
10.3.1	Task Jacobians	472
10.3.2	Space Jacobians	472
10.3.3	Body Jacobians	472
10.3.4	The body–space adjoint relation	472
10.4	Rank, singularities, and conditioning	472
10.4.1	Task-specific rank loss	472
10.4.2	Singular values and numerical conditioning	472
10.4.3	Full-twist and reduced-task Jacobians	472
10.5	Modelling limits and implementation checks	472
10.5.1	Supported serial-chain model	472
10.5.2	Explicit exclusions	472
10.5.3	Analytic and finite-difference checks	472
	Cumulative chapter review	472
	Answers and hints	472
	References	472
11	General Robot Kinematics and Jacobians Implementation	474
	Prerequisites	474
11.1	Typed joint-coordinate limits	474
11.1.1	Representing an optional bound	475
11.1.2	Validating before construction	476
11.1.3	Using an interval	479
11.1.4	Verification meaning	479

11.2	Revolute and prismatic joints	479
11.2.1	Storing geometry and typed limits	480
11.2.2	Normalizing the positive direction	482
11.2.3	Constructing a revolute screw	483
11.2.4	Constructing a prismatic screw	485
11.2.5	Verification meaning	486
11.3	Serial-chain model	486
11.3.1	Owning the home pose and ordered joints	487
11.3.2	Validating construction without duplicating nested checks	488
11.3.3	Verification meaning	490
11.4	Space-form forward kinematics	490
11.4.1	Public input and output	491
11.4.2	Accessing either joint kind	492
11.4.3	Evaluating the ordered product	494
11.4.4	Worked example: spatial two-revolute-joint motion	498
11.4.5	RViz implementation example: displaying the spatial 2R arm	501
11.4.6	How the classes and functions connect	514
II	Probability and Estimation	518
12	Finite Markov Chains	519
	Prerequisites	519
	Why model an uncontrolled state sequence?	519
12.1	A finite stochastic state sequence	520
12.2	The first-order Markov property	521
12.3	Time-homogeneous transition probabilities	522
12.4	Propagating the state distribution	523
12.5	Trajectory probabilities and absorbing states	524
	Quick test: finite Markov chains	525
	Answers and hints	526
	References	526
III	Optimisation, Planning, and Decisions	528
13	Finite Markov Decision Processes	529
	Prerequisites	529
	Why add control to stochastic dynamics?	529
13.1	Actions and feasible action sets	529
13.2	The controlled Markov property	530
13.3	Controlled transition probabilities	531
13.4	Uncontrolled and controlled dynamics are different objects	532
	Quick test: actions and controlled transitions	533

13.5	Rewards and joint one-step dynamics	534
	Quick test: rewards and joint dynamics	538
13.6	Policies and policy-induced dynamics	539
13.6.1	The policy-induced transition matrix	542
13.6.2	Policy-induced joint outcomes and rewards	544
	Quick test: policies and induced dynamics	547
13.7	Episode endings: termination and truncation	548
13.7.1	A terminal state ends the modelled task	548
13.7.2	An absorbing extension represents the stopped episode	549
13.7.3	Truncation stops the record, not the task	549
13.7.4	A time limit can belong inside or outside the task	550
	Quick test: termination and truncation	550
13.8	The finite MDP and its task settings	551
13.8.1	The finite MDP specifies controlled one-step outcomes	551
13.8.2	Initial conditions, policies, and task endings add distinct information	552
13.8.3	Cumulative reward gives discounting a purpose	552
	Quick test: the finite MDP and task settings	554
	Cumulative chapter test	555
	Answers and hints	556
	References	557
14	Finite-MDP Model Implementation	558
	Prerequisites	558
14.1	Identifying states and actions	558
14.1.1	Mathematical domains and ordering	558
14.1.2	Class declarations	559
14.1.3	Concrete use and conversion checks	560
14.2	Storing a joint-outcome distribution	560
14.2.1	Mathematical records and their representation	560
14.2.2	The distribution class	561
14.2.3	Construction and ownership	562
14.2.4	Recognising repeated events	563
14.2.5	Reading records without copying them	565
14.3	Kahan compensation for floating-point sums	565
14.3.1	Why ordinary addition loses information	565
14.3.2	The four operations in the implementation	566
14.3.3	A complete binary64 trace	567
14.3.4	Validation remains a tolerance test	568
14.4	Marginal probabilities and expected reward	568
14.4.1	One accumulator per next state	569
14.4.2	One accumulator for the expected reward	570
14.4.3	Finite inputs do not guarantee a finite computed reward	571
14.5	Constructing the complete finite MDP	572
14.5.1	Attaching a distribution to its current pair	573
14.5.2	Task actions and the absorbing extension	573

14.5.3	The model class	574
14.5.4	Completing and ordering the model rows	575
14.5.5	Generating terminal rows	577
14.5.6	Establishing the row order	577
14.5.7	Complete-action and bookkeeping queries	578
14.6	Finding a distribution and checking terminal membership	578
14.6.1	Searching for a state–action row	579
14.6.2	Accessing an optional borrowed distribution	581
14.6.3	Reading the terminal declaration	582
14.7	Model usage	583
14.7.1	Construction from individual events	583
14.7.2	Queries produce calculated values	584
14.8	How the classes and functions connect	585
14.8.1	Constructing distributions and model rows	585
14.8.2	Looking up and evaluating a distribution	586
14.8.3	Inspecting the model itself	587
14.9	Analytic examples	588
14.9.1	A task with uncertain progress	588
14.9.2	Calculating the expected results	589
14.9.3	Constructing the model	589
14.9.4	Reading the model	590
	References	591

Preface

This book is my collection of learning notes for understanding intelligence, especially intelligence embodied in agents that perceive their environment, predict what may happen, choose actions, and act in the physical world. Writing the notes is part of the learning process: the aim is to make each idea precise enough that I can explain it, derive it, implement it, and test it.

My main algorithmic interests are **reinforcement learning** and **world models**. Reinforcement learning studies how an agent can improve its decisions through interaction and feedback. A world model is an internal model that predicts how the environment or system may evolve, allowing an agent to evaluate possible actions before or while taking them. These algorithms depend on more fundamental ideas from mathematics, dynamics, control, estimation, planning, perception, and software engineering, so the notes build those foundations first.

Unified Notation and Conventions

This note is the canonical notation registry for the project. Chapters may introduce local symbols, but they must not redefine symbols listed here. When a textbook uses different notation, translate it into these conventions.

Core conventions

1. Scalars use italic lowercase letters, vectors use bold lowercase letters, and matrices use bold uppercase letters.
2. Vectors are column vectors unless stated otherwise.
3. SI units are used in calculations. Angles are expressed in radians.
4. Coordinate frames are right-handed. A leading left superscript identifies the frame in which coordinates are expressed.
5. Continuous time uses t ; sampled time uses the integer index k .
6. A wide hat over a twist, as in $\hat{\xi}$, denotes the declared Lie-algebra hat operator and is not an estimate.
7. Bold symbols and scalar symbols with the same letter are distinct; for example, $\mathbf{R}$ is a rotation matrix whereas R may be a signed turning radius.
8. A symbol may have a different local meaning only in a clearly separated mathematical domain where its type and meaning are explicitly redeclared; for example, m may denote a matrix dimension in a generic linear-algebra statement and robot mass in a dynamics model.

Part I: Motion, Mechanics, and Control

Notation introduced in Chapter 1

Symbol	Meaning, units, and convention
a, α	Scalar quantity.
m, n, i, j	Integer dimensions or indices declared locally.
$\mathbb{R}$	Set of real numbers.

Symbol	Meaning, units, and convention
$\mathbb{R}^n, \mathbb{R}^{m \times n}$	Real coordinate columns and real matrices with dimensions shown by the superscripts.
$\mathbb{R}_{\geq 0}, \mathbb{R}_{>0}$	Nonnegative and strictly positive real numbers.
$\mathbf{v} \in \mathbb{R}^n$	Column vector with n components.
$\mathbf{A} \in \mathbb{R}^{m \times n}$	Matrix with m rows and n columns.
$\mathbf{I}_n$	$n \times n$ identity matrix.
$\mathbf{0}_n$	Zero vector with n components.
$(\cdot)^\top$	Transpose.
$(\cdot)^{-1}$	Inverse, when it exists.
$(\cdot)^{-\top}$	Inverse transpose, $(\mathbf{A}^{-1})^\top = (\mathbf{A}^\top)^{-1}$ for nonsingular $\mathbf{A}$.
$\ \mathbf{v}\ _2$	Euclidean norm of $\mathbf{v}$, with the same units as $\mathbf{v}$.
$\det(\mathbf{A})$	Determinant of a square matrix.
$\text{rank}(\mathbf{A})$	Dimension of the column space of $\mathbf{A}$.
$\text{nullity}(\mathbf{A})$	Dimension of the null space of $\mathbf{A}$.
$\text{span}(\mathbf{v}_1, \dots, \mathbf{v}_k)$	Set of all linear combinations of the listed vectors.
$\text{Col}(\mathbf{A}), \mathcal{N}(\mathbf{A})$	Column space and null space of $\mathbf{A}$, which are subspaces of the output and input spaces.
$\mathbf{u} \times \mathbf{v}$	Three-dimensional cross product of vectors expressed in the same right-handed orthonormal frame; its units are the product of the input units.
$O(n), SO(n)$	Orthogonal group and special orthogonal group of real $n \times n$ matrices: $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_n$, and $SO(n)$ additionally requires $\det \mathbf{Q} = 1$.

Notation introduced in Chapter 2

Symbol	Meaning, units, and convention
$\{a\}$	Right-handed coordinate frame named a .
O_a	Origin of frame $\{a\}$.
$\mathbf{e}_{x,a}, \mathbf{e}_{y,a}, \mathbf{e}_{z,a}$	Unit basis vectors of frame $\{a\}$.
${}^a \mathbf{v}$	Coordinates of geometric vector $\mathbf{v}$ expressed in frame $\{a\}$.
${}^a \mathbf{p}_P$	Position of point P relative to O_a , expressed in frame $\{a\}$, in m.

Symbol	Meaning, units, and convention
${}^a\mathbf{r}_{OP}$	Relative-position vector from point O to point P , expressed in frame $\{a\}$, in m.
$\mathbf{R}_{ab}$	Rotation in $SO(2)$ or $SO(3)$ mapping coordinates from frame $\{b\}$ to frame $\{a\}$.
${}^a\mathbf{p}_b$	Position of origin O_b relative to O_a , expressed in frame $\{a\}$, in m.
$\boldsymbol{\omega}$	Three-dimensional angular-velocity vector in rad s^{-1} .

Notation introduced in Chapter 3

Symbol	Meaning, units, and convention
t	Continuous time in s.
$k \in \mathbb{N}_0$	Sample index, where $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.
t_k	Time at sample k , in s.
$\Delta t = t_{k+1} - t_k$	Sampling or numerical-integration interval in s.
$\dot{\mathbf{x}}, \ddot{\mathbf{x}}$	First and second derivatives with respect to time, with units of the differentiated quantity per s and per s^2 .
$\{w\}, \{b\}$	World and robot body frames.
O_w, O_b	Origins of the world and body frames.
$\mathbf{q} = [x \ y \ \theta]^\top$	Planar robot pose coordinates in the world frame, with component units m, m, rad.
θ	Counter-clockwise heading of $\{b\}$ relative to $\{w\}$, in rad.
$\mathbf{u} = [v \ \omega]^\top$	Planar body-velocity input with component units m s^{-1} , rad s^{-1} .
v, ω	Signed body-forward speed in m s^{-1} and counter-clockwise yaw rate in rad s^{-1} .
ϕ_L, ϕ_R	Left and right wheel angles in rad.
$\dot{\phi}_L, \dot{\phi}_R$	Left and right wheel angular speeds in rad s^{-1} .
v_L, v_R	Signed longitudinal ground speeds of the left and right wheels in m s^{-1} .
r, L	Effective wheel radius and track width between wheel contact lines, in m.
C	Instantaneous centre of curvature for planar motion.
ρ	Nonnegative radial distance in the declared local geometric context, in m.

Symbol	Meaning, units, and convention
R	Signed turning radius in m, positive for an ICC on the robot's left and distinct from rotation matrix $\mathbf{R}$.
$\Delta\ell, \Delta\theta$	Signed arc-length change in m and heading change in rad over an interval.
$\mathbf{q}_k^E$	Forward Euler approximation of the pose at sample k , with the same component units as $\mathbf{q}$.
$E(\Delta t)$	Declared numerical-integration error magnitude at step size Δt .
e_p	One-step position-error magnitude in the declared integration comparison, in m.
$O(h^p)$	Quantity bounded in magnitude by a constant times h^p as $h \rightarrow 0$.
$\text{sinc}(z)$	Unnormalised sinc function, $\sin z/z$ for $z \neq 0$ and 1 for $z = 0$.

Notation introduced in Chapter 6

Symbol	Meaning, units, and convention
${}^a\bar{\mathbf{p}}_P$	Homogeneous point coordinate obtained by appending 1 to ${}^a\mathbf{p}_P$; position entries are in m and the final entry is dimensionless.
$SO(2)$	Special orthogonal group of planar rotation matrices satisfying $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_2$ and $\det \mathbf{R} = 1$.
$SE(2)$	Special Euclidean group representing planar rotation and translation together.
$\mathbf{T}_{ab}$	Homogeneous transform in $SE(2)$ mapping coordinates from frame $\{b\}$ to frame $\{a\}$.
$T_{\mathbf{I}_2}SO(2)$	Tangent space of $SO(2)$ at $\mathbf{I}_2$: the linear space of rotation-curve derivatives through the identity.
$T_{\mathbf{I}_3}SE(2)$	Tangent space of $SE(2)$ at $\mathbf{I}_3$: the linear space of transformation-curve derivatives through the identity.
$\mathbf{S}_2$	Planar skew generator $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$.
$\mathfrak{so}(2)$	Lie algebra and identity tangent space of $SO(2)$, comprising 2×2 real skew-symmetric matrices.

Symbol	Meaning, units, and convention
$\mathfrak{se}(2)$	Lie algebra and identity tangent space of $SE(2)$, comprising structured 3×3 infinitesimal planar rigid-motion matrices.
$\xi = [\boldsymbol{\nu}^\top \omega]^\top$	Planar twist coordinate column with linear part $\boldsymbol{\nu} \in \mathbb{R}^2$ in m s^{-1} and angular part $\omega \in \mathbb{R}$ in rad s^{-1} .
$\hat{\xi}$	Hat-operator matrix representing ξ in $\mathfrak{se}(2)$; its upper-left block is in s^{-1} and its translation column is in m s^{-1} .
$(\cdot)^\vee$	Vee operator, inverse of the declared hat operator, returning the twist coordinate column.
ξ_b, ξ_w	Body and spatial twist coordinates for the same instantaneous motion, using body axes/body origin and world axes/world origin respectively.
$\boldsymbol{\nu}_b, \boldsymbol{\nu}_w$	Linear parts of the body and spatial twists in m s^{-1} .
Ad_T	Adjoint mapping that changes twist coordinates according to rigid transform T ; its dimensions and mixed units follow the declared twist convention.
ℓ	Characteristic length in m used to compare linear and angular twist components.

Notation introduced in Chapter 7

Symbol	Meaning, units, and convention
$\mathcal{B}$	Set of labelled material points belonging to the rigid body.
$\chi : \mathcal{B} \rightarrow \mathbb{R}^d$	Placement map assigning a world position in m to every material point in a d -dimensional model.
$\mathcal{C}$	Configuration space: the set of all configurations allowed by the model; it may be curved and need not be a vector space.
$\chi \in \mathcal{C}$	One complete physical configuration represented as an admissible placement map.
$\mathbf{q}$	Chosen column of generalised coordinates describing a configuration; its component units depend on the system.
n_{dof}	Number of configuration degrees of freedom.

Symbol	Meaning, units, and convention
n_v, n_c	Numbers of scalar variables and independent regular equality constraints in a local count.
$\mathbf{z} \in \mathbb{R}^{n_v}$	Ambient coordinate column used to express equality constraints; its component units depend on the system.
$\mathbf{g} : \mathbb{R}^{n_v} \rightarrow \mathbb{R}^{n_c}$	Equality-constraint mapping, with valid coordinates satisfying $\mathbf{g}(\mathbf{z}) = \mathbf{0}_{n_c}$.
$\mathbf{J}_g(\mathbf{z})$	$n_c \times n_v$ constraint Jacobian with entries $\partial g_i / \partial z_j$; row units depend on the corresponding constraint and coordinate.
$SO(3)$	Special orthogonal group of spatial rotation matrices satisfying $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_3$ and $\det \mathbf{R} = 1$.
$\mathbf{R}_x(\theta), \mathbf{R}_y(\theta), \mathbf{R}_z(\theta)$	Active right-hand-rule rotations in $SO(3)$ about the declared coordinate axes.
$\mathbf{Q}_w, \mathbf{Q}_b$	Finite orientation increments in $SO(3)$ expressed along world or body axes; premultiply or postmultiply accordingly.
$\boldsymbol{\omega}_w, \boldsymbol{\omega}_b$	Same world-relative angular velocity resolved along world or body axes, in rad s^{-1} .
$SE(3)$	Special Euclidean group representing spatial rotation and translation together.
$\mathbf{T}_{wb} \in SE(3)$	Spatial pose of frame $\{b\}$ relative to frame $\{w\}$ and transform from body to world point coordinates; its rotation block is dimensionless and translation block is in m.
$[\mathbf{v}]_\times$	Cross-product matrix satisfying $[\mathbf{v}]_\times \mathbf{y} = \mathbf{v} \times \mathbf{y}$, with the same units as $\mathbf{v}$.
$\mathfrak{so}(3)$	Lie algebra and identity tangent space of $SO(3)$, comprising 3×3 real skew-symmetric matrices.
$\mathbf{a}_w, \alpha, \boldsymbol{\varphi}_w = \alpha \mathbf{a}_w$	Unit rotation axis, signed angle, and rotation-vector coordinates expressed along world axes; the angle and rotation vector are labelled in rad.
$\exp(\mathbf{A})$	Matrix exponential $\sum_{j=0}^{\infty} \mathbf{A}^j / j!$.
$\mathbf{q} = (q_w, \mathbf{q}_v)$	Quaternion written as an ordered scalar–vector pair.
$\otimes$	Hamilton product of two quaternions, with order following rotation-composition order.
$\mathbf{q}^* = (q_w, -\mathbf{q}_v)$	Quaternion conjugate; the inverse when $\mathbf{q}$ has unit norm.
$\mathfrak{se}(3)$	Lie algebra and identity tangent space of $SE(3)$, comprising structured 4×4 infinitesimal spatial rigid-motion matrices.

Symbol	Meaning, units, and convention
$\xi = [\nu^\top \omega^\top]^\top$	Spatial twist coordinate column in the project's linear-first order: three entries in m s^{-1} followed by three in rad s^{-1} .
$\ \mathbf{M}\ _F$	Frobenius norm of a real matrix, with units following its entries.
$e_{\text{orth}}, e_{\text{det}}, e_q$	Orthogonality, determinant, and unit-quaternion residuals.
$\ \xi\ _\ell$	Application-dependent twist comparison magnitude in m s^{-1} after choosing characteristic length ℓ .
$\Omega_k = [\omega_{b,k}]_\times$	Body-expressed angular-velocity matrix at sample k , in s^{-1} .
$\tilde{\mathbf{R}}_{k+1}$	Forward Euler candidate for the next orientation matrix; it is not necessarily in $SO(3)$.

Notation introduced in Chapter 8

Symbol	Meaning, units, and convention
$I, L_i, \mathcal{B}_i, \bigsqcup_{i \in I} \mathcal{B}_i$	Finite link-index set, link i , its material-point set, and the disjoint union of all link point sets.
$c_b : \mathcal{B}_b \rightarrow \mathbb{R}^3$	Fixed body-coordinate assignment mapping each material point of link b to its coordinates in frame $\{b\}$, in m .
$A_{\mathbf{T}_{wb}} : \mathbb{R}^3 \rightarrow \mathbb{R}^3$	Affine point-coordinate map induced by pose matrix $\mathbf{T}_{wb}$, with input and output in m .
$\chi_{\mathbf{T}_{wb}} = A_{\mathbf{T}_{wb}} \circ c_b$	Rigid-link configuration induced by a pose and the one-to-one pose-to-configuration correspondence.
$\Phi_b : SE(3) \rightarrow \mathcal{C}_b$	
$\mathcal{Q}, N, n, j$	Generalised-coordinate domain, number of joints, number of coordinates, and joint index; $\mathcal{Q}$ may contain mixed-unit coordinates.
${}^a\mathcal{L}, {}^a\mathbf{p}_\ell, {}^a\mathbf{s}, \lambda$	Oriented axis line, a point on it in m , its unit positive direction, and signed distance λ in m .
$\theta_j, \dot{\theta}_j, d_j, \dot{d}_j$	Revolute angle and rate in rad and rad s^{-1} , and prismatic displacement and rate in m and m s^{-1} , for joint j .
$\mathbf{F}_e : \mathcal{Q} \rightarrow SE(3)$	Forward-kinematics map returning the pose of selected frame $\{e\}$ relative to space frame $\{s\}$; rotation is dimensionless and translation is in m .

Symbol	Meaning, units, and convention
h	Signed screw pitch in m rad^{-1} : axial travel per positive radian of rotation.
${}^a\mathbf{S}_R, {}^a\mathbf{S}_H, {}^a\mathbf{S}_P$	Normalised revolute, finite-pitch helical, and prismatic screw-axis coordinate columns expressed in frame $\{a\}$, using linear-first six-entry order and case-dependent mixed units.
${}^a\mathbf{S} = [({}^a\mathbf{s}_v)^\top ({}^a\mathbf{s}_\omega)^\top]^\top$	Normalised screw-axis coordinates with linear and angular coefficient blocks; $\mathbf{s}_v$ is in m rad^{-1} for rotational screws and dimensionless for prismatic screws, while $\mathbf{s}_\omega$ is dimensionless.
$\sigma, \dot{\sigma}$	Signed motion amount along a normalised screw and its rate: rad and rad s^{-1} for rotational screws, or m and m s^{-1} for prismatic screws.
$\widehat{a}\mathbf{S}$	Hat-operator matrix representation of a screw axis in $\mathfrak{se}(3)$, with mixed units following ${}^a\mathbf{S}$.
${}^a\boldsymbol{\eta} = [({}^a\boldsymbol{\rho})^\top ({}^a\boldsymbol{\phi})^\top]^\top$	Finite exponential-coordinate column with linear block $\boldsymbol{\rho}$ in m and rotation-vector block $\boldsymbol{\phi}$ labelled in rad .
$\mathbf{T}_\Delta$	Finite rigid-displacement transformation in $SE(3)$ produced by a screw exponential or finite twist increment; translation is in m .
$\boldsymbol{\Omega} = [{}^a\mathbf{s}_\omega]_\times, \mathbf{G}(\theta)$	Unit-axis cross-product matrix and normalised-screw translation map; $\mathbf{G}$ maps m rad^{-1} to m .
$\text{Log}(\mathbf{T}), \log_{\text{sel}}, \mathcal{D}_{\text{sel}}, \log_p$	Set of all real rigid-motion logarithms, a selected single-valued branch, its domain, and the chapter's principal branch; logarithms lie in $\mathfrak{se}(3)$ with finite mixed-unit blocks.
$\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_\times$	Cross-product matrix of the finite rotation-vector block, with entries labelled in rad .
$A(\theta), B(\theta), C(\theta), D(\theta)$	Continuously extended scalar coefficients used in the rotation, left-Jacobian, and inverse-left-Jacobian formulas.
$\mathbf{J}({}^a\boldsymbol{\phi})$	Dimensionless 3×3 left Jacobian of $SO(3)$ mapping $\boldsymbol{\rho}$ to the finite translation column.
$c_\theta, {}^a\mathbf{u}, s_\theta, \mathbf{Q}$	Trace-derived cosine, skew-difference column, sine magnitude, and symmetric axis outer-product matrix used by the principal rotation logarithm.
$\theta_{\text{series}}, \varepsilon_\pi, \varepsilon_{\text{zero}}, \varepsilon_{p,0}$	Series-selection, near- π , angular-zero, and translation-zero thresholds; the first three are in rad and $\varepsilon_{p,0}$ is in m .

Symbol	Meaning, units, and convention
$\ \mathbf{v}\ _\infty$	Infinity norm: the largest absolute component of a finite column, with the same units as $\mathbf{v}$.
$\tilde{\mathbf{T}}, e_{\text{row}}$	Candidate homogeneous transformation and its bottom-row residual before validation; candidate translation is in m and the residual is dimensionless.
$e_{R,\text{rt}}, e_{p,\text{rt}}$	Rotation and translation residuals for an exponential/logarithm round trip, dimensionless and in m respectively.

Part I

Motion, Mechanics, and Control

1 Linear Algebra Foundations

Why linear algebra matters

Autonomous robots represent directions, velocities, forces, sensor measurements, model parameters, and control commands with vectors. Matrices transform those vectors, combine coordinate descriptions, solve systems of equations, and expose whether information has been preserved or lost. Linear algebra provides the language needed to reason about these operations before they are used in geometry, kinematics, estimation, optimisation, and control.

This chapter develops that language from first principles. It distinguishes geometric vectors from coordinate columns, explains matrices as linear maps, connects null spaces and rank to invertibility, defines determinants as signed volume scaling, and derives the properties of orthogonal matrices used later for rotations.

Prerequisites and assumptions

Prerequisites

The reader should be comfortable with:

- arithmetic and elementary algebra;
- Cartesian coordinates;
- solving simple equations;
- sine and cosine; and
- the project notation conventions.

Assumptions

- Scalars are real numbers unless stated otherwise.
- Vectors are written as columns.
- Matrix and vector dimensions are declared when symbols are introduced.
- Coordinate bases used for Euclidean lengths and angles are orthonormal.
- Physical units are carried by the represented quantity; abstract basis and transformation matrices are dimensionless unless stated otherwise.

Dependency map

```

Vectors and linear combinations
  |
  v
Bases and coordinate columns
  |
  v
Matrices as linear maps
  |
  v
Null spaces, rank, and inverses
  |
  v
Determinants and orthogonal matrices
  |
  v
Coordinate frames, rotations, kinematics, estimation, and control

```

1.1 Vectors: quantities with magnitude and direction

1.1.1 Geometric vector versus coordinate column

A geometric vector is independent of any coordinate frame. A coordinate column is the numerical description of that vector in a chosen basis.

Let $\{a\}$ denote a right-handed coordinate frame with orthonormal basis vectors $\mathbf{e}_{x,a}$, $\mathbf{e}_{y,a}$, and $\mathbf{e}_{z,a}$. For a geometric vector $\mathbf{v}$, the symbol ${}^a\mathbf{v}$ denotes its coordinate column expressed in frame $\{a\}$. In three-dimensional space,

$${}^a\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}.$$

The entries mean that

$$\mathbf{v} = v_x \mathbf{e}_{x,a} + v_y \mathbf{e}_{y,a} + v_z \mathbf{e}_{z,a}.$$

The vector $\mathbf{v}$ is the physical or geometric object, whereas the column ${}^a\mathbf{v}$ depends on frame $\{a\}$. Omitting the frame label is safe only when the surrounding context makes the frame unambiguous.

1.1.2 Addition and scalar multiplication

For vectors expressed in the same frame,

$${}^a\mathbf{u} + {}^a\mathbf{v} = \begin{bmatrix} u_x + v_x \\ u_y + v_y \\ u_z + v_z \end{bmatrix}.$$

For a scalar α ,

$$\alpha {}^a\mathbf{v} = \begin{bmatrix} \alpha v_x \\ \alpha v_y \\ \alpha v_z \end{bmatrix}.$$

The algebra is valid only when the quantities are physically compatible. For example, adding two positions expressed in different frames is not meaningful until one has been transformed. Adding a velocity in metres per second to a position in metres is dimensionally invalid even if both are three-element columns.

1.1.3 Length and unit direction

The Euclidean norm is

$$\|{}^a\mathbf{v}\|_2 = \sqrt{v_x^2 + v_y^2 + v_z^2}.$$

If $\mathbf{v} \neq \mathbf{0}$, let ${}^a\mathbf{e}_v$ denote its unit direction expressed in frame $\{a\}$:

$${}^a\mathbf{e}_v = \frac{{}^a\mathbf{v}}{\|{}^a\mathbf{v}\|_2}.$$

The norm has the same physical units as the vector, while the unit vector is dimensionless because the units cancel.

1.1.4 Dot product and angle

Let two vectors be expressed as coordinate columns in the same orthonormal frame:

$${}^a\mathbf{u} = \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}, \quad {}^a\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}.$$

The superscript $\top$ denotes the **transpose** operation. It turns the column ${}^a\mathbf{u} \in \mathbb{R}^{3 \times 1}$ into the row

$$({}^a\mathbf{u})^\top = {}^a\mathbf{u}^\top = [u_x \ u_y \ u_z] \in \mathbb{R}^{1 \times 3}.$$

Here, $\top$ is an operation on the coordinate column; it is not a power. Multiplying this row by the column ${}^a\mathbf{v}$ produces a scalar:

$${}^a\mathbf{u}^\top {}^a\mathbf{v} = [u_x \ u_y \ u_z] \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = u_x v_x + u_y v_y + u_z v_z.$$

This scalar is the **dot product**, or Euclidean inner product, of the two vectors:

$$\mathbf{u} \cdot \mathbf{v} := {}^a\mathbf{u}^\top {}^a\mathbf{v}.$$

The norm introduced in Section 1.1.3 can equivalently be defined from the dot product of a vector with itself:

$$\begin{aligned} \|{}^a\mathbf{u}\|_2 &:= \sqrt{{}^a\mathbf{u}^\top {}^a\mathbf{u}} \\ &= \sqrt{u_x^2 + u_y^2 + u_z^2}. \end{aligned}$$

Equivalently, the squared norm is

$$\|{}^a\mathbf{u}\|_2^2 = {}^a\mathbf{u}^\top {}^a\mathbf{u}.$$

Thus, dotting a vector with itself gives its squared length, and taking the nonnegative square root gives its length. Because frame $\{a\}$ is orthonormal, this coordinate norm equals the geometric length $\|\mathbf{u}\|_2$.

The coordinate formula above has a geometric interpretation. To establish it, we first derive the law of cosines.

1.1.4.1 Law of cosines

Consider a triangle with two sides of lengths r and s and included angle θ . Place its vertices at

$$\mathbf{p}_0 = (0, 0), \quad \mathbf{p}_1 = (r, 0), \quad \mathbf{p}_2 = (s \cos \theta, s \sin \theta).$$

Let c be the length of the side from $\mathbf{p}_1$ to $\mathbf{p}_2$. The horizontal and vertical displacements along that side are $s \cos \theta - r$ and $s \sin \theta$. By the Pythagorean theorem,

$$\begin{aligned} c^2 &= (s \cos \theta - r)^2 + (s \sin \theta)^2 \\ &= s^2 \cos^2 \theta - 2rs \cos \theta + r^2 + s^2 \sin^2 \theta \\ &= r^2 + s^2 (\cos^2 \theta + \sin^2 \theta) - 2rs \cos \theta \\ &= r^2 + s^2 - 2rs \cos \theta. \end{aligned}$$

Therefore, the law of cosines is

$$\boxed{c^2 = r^2 + s^2 - 2rs \cos \theta}.$$

1.1.4.2 Deriving the dot-product angle formula

For nonzero vectors $\mathbf{u}$ and $\mathbf{v}$, place their tails at the same point and let θ be the angle between them. The vectors form a triangle whose side lengths are

$$r = \|\mathbf{u}\|_2, \quad s = \|\mathbf{v}\|_2, \quad c = \|\mathbf{u} - \mathbf{v}\|_2.$$

The law of cosines therefore gives

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\|\mathbf{u}\|_2\|\mathbf{v}\|_2 \cos \theta.$$

Because frame $\{a\}$ is orthonormal, Euclidean lengths are preserved by its coordinate representation. Expanding the same squared length using coordinate columns gives

$$\begin{aligned} \|\mathbf{u} - \mathbf{v}\|_2^2 &= ({}^a\mathbf{u} - {}^a\mathbf{v})^\top ({}^a\mathbf{u} - {}^a\mathbf{v}) \\ &= \|{}^a\mathbf{u}\|_2^2 + \|{}^a\mathbf{v}\|_2^2 - 2 {}^a\mathbf{u}^\top {}^a\mathbf{v}. \end{aligned}$$

Equating the two expressions for $\|\mathbf{u} - \mathbf{v}\|_2^2$ and cancelling their common terms gives

$${}^a\mathbf{u}^\top {}^a\mathbf{v} = \|\mathbf{u}\|_2\|\mathbf{v}\|_2 \cos \theta,$$

which is the dot-product angle formula. Solving it for the angle gives

$$\theta = \cos^{-1} \left(\frac{a_{\mathbf{u}}^T a_{\mathbf{v}}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \right),$$

provided neither vector is zero.

The dot product is useful for projection, alignment, and checking orthogonality. If

$$a_{\mathbf{u}}^T a_{\mathbf{v}} = 0,$$

then nonzero $\mathbf{u}$ and $\mathbf{v}$ are perpendicular.

1.1.5 Cross product and useful identities

The dot product returns a scalar that measures alignment. Three-dimensional geometry also needs an operation that constructs a vector perpendicular to two given directions. Let $\mathbf{u} = [u_1 \ u_2 \ u_3]^T \in \mathbb{R}^3$ and $\mathbf{v} = [v_1 \ v_2 \ v_3]^T \in \mathbb{R}^3$ be coordinate columns expressed in the same right-handed orthonormal frame. Their **cross product** is the coordinate column

$$\mathbf{u} \times \mathbf{v} := \begin{bmatrix} u_2 v_3 - u_3 v_2 \\ u_3 v_1 - u_1 v_3 \\ u_1 v_2 - u_2 v_1 \end{bmatrix} \in \mathbb{R}^3.$$

The output is perpendicular to both inputs because direct substitution gives

$$\mathbf{u}^T (\mathbf{u} \times \mathbf{v}) = 0, \quad \mathbf{v}^T (\mathbf{u} \times \mathbf{v}) = 0.$$

Its direction follows the right-hand rule. Expanding its squared norm gives

$$\begin{aligned} \|\mathbf{u} \times \mathbf{v}\|_2^2 &= (u_2 v_3 - u_3 v_2)^2 + (u_3 v_1 - u_1 v_3)^2 + (u_1 v_2 - u_2 v_1)^2 \\ &= \|\mathbf{u}\|_2^2 \|\mathbf{v}\|_2^2 - (\mathbf{u}^T \mathbf{v})^2 \\ &= \|\mathbf{u}\|_2^2 \|\mathbf{v}\|_2^2 \sin^2 \theta, \end{aligned}$$

where $\theta \in [0, \pi]$ is the angle between the nonzero inputs and the last equality uses $\mathbf{u}^T \mathbf{v} = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos \theta$. Taking the nonnegative square root gives

$$\|\mathbf{u} \times \mathbf{v}\|_2 = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \sin \theta.$$

If the input components have physical units, the cross-product components have the product of those units. The coordinate definition also gives the useful identities

$$\mathbf{u} \times \mathbf{v} = -\mathbf{v} \times \mathbf{u}, \quad \mathbf{u} \times \mathbf{u} = \mathbf{0}_3.$$

Cross products are not associative: $\mathbf{a} \times (\mathbf{b} \times \mathbf{c})$ and $(\mathbf{a} \times \mathbf{b}) \times \mathbf{c}$ are generally different. For $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$ expressed in the same right-handed orthonormal frame, the **vector triple-product identity** is

$$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a}^\top \mathbf{c}) - \mathbf{c}(\mathbf{a}^\top \mathbf{b}).$$

To verify the identity from the cross-product definition, set $\mathbf{d} := \mathbf{b} \times \mathbf{c}$. Substituting the components of $\mathbf{d}$ into $\mathbf{a} \times \mathbf{d}$ and regrouping by the components of $\mathbf{b}$ and $\mathbf{c}$ gives

$$\begin{aligned} \mathbf{a} \times \mathbf{d} &= \begin{bmatrix} a_2(b_1c_2 - b_2c_1) - a_3(b_3c_1 - b_1c_3) \\ a_3(b_2c_3 - b_3c_2) - a_1(b_1c_2 - b_2c_1) \\ a_1(b_3c_1 - b_1c_3) - a_2(b_2c_3 - b_3c_2) \end{bmatrix} \\ &= \begin{bmatrix} b_1(\mathbf{a}^\top \mathbf{c}) - c_1(\mathbf{a}^\top \mathbf{b}) \\ b_2(\mathbf{a}^\top \mathbf{c}) - c_2(\mathbf{a}^\top \mathbf{b}) \\ b_3(\mathbf{a}^\top \mathbf{c}) - c_3(\mathbf{a}^\top \mathbf{b}) \end{bmatrix} \\ &= \mathbf{b}(\mathbf{a}^\top \mathbf{c}) - \mathbf{c}(\mathbf{a}^\top \mathbf{b}). \end{aligned}$$

A case used later for rotations sets $\mathbf{b} = \mathbf{a}$ and $\mathbf{c} = \mathbf{y}$:

$$\mathbf{a} \times (\mathbf{a} \times \mathbf{y}) = \mathbf{a}(\mathbf{a}^\top \mathbf{y}) - (\mathbf{a}^\top \mathbf{a})\mathbf{y}.$$

If $\mathbf{a}$ is a unit vector, then $\mathbf{a}^\top \mathbf{a} = 1$.

1.1.6 Scalar triple product

A cross product supplies a direction perpendicular to two vectors and a magnitude equal to their parallelogram area. Taking its dot product with a third vector converts this area vector into a signed volume. For $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$ expressed in the same right-handed orthonormal frame, the **scalar triple product** is

$$\mathbf{a}^\top (\mathbf{b} \times \mathbf{c}) \in \mathbb{R}.$$

The word *scalar* records that the outer dot product returns one real number, while *triple* records that the expression uses three vectors. Substituting the cross-product components gives the explicit formula

$$\begin{aligned} \mathbf{a}^T(\mathbf{b} \times \mathbf{c}) &= a_1(b_2c_3 - b_3c_2) + a_2(b_3c_1 - b_1c_3) \\ &\quad + a_3(b_1c_2 - b_2c_1). \end{aligned}$$

Geometrically, $\|\mathbf{b} \times \mathbf{c}\|_2$ is the area of the parallelogram spanned by $\mathbf{b}$ and $\mathbf{c}$. If $\mathbf{b} \times \mathbf{c} \neq \mathbf{0}_3$, define its unit normal by $\mathbf{n} := (\mathbf{b} \times \mathbf{c}) / \|\mathbf{b} \times \mathbf{c}\|_2$. Then

$$\mathbf{a}^T(\mathbf{b} \times \mathbf{c}) = \underbrace{\mathbf{a}^T \mathbf{n}}_{\text{signed height}} \underbrace{\|\mathbf{b} \times \mathbf{c}\|_2}_{\text{base area}}.$$

Its absolute value is therefore the volume of the parallelepiped spanned by the three vectors. Its sign states whether the ordered directions $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ have positive or negative orientation, and it is zero when the three vectors are coplanar. If the input vectors carry physical units, the scalar triple product carries the product of all three units.

Direct component expansion also gives the cyclic identities

$$\boxed{\mathbf{a}^T(\mathbf{b} \times \mathbf{c}) = \mathbf{b}^T(\mathbf{c} \times \mathbf{a}) = \mathbf{c}^T(\mathbf{a} \times \mathbf{b})}.$$

Cyclically moving the three vectors does not change the value, but swapping any two vectors reverses its sign. This scalar-valued expression differs from the vector triple product in Section 1.1.5, whose output is a vector. The determinant discussion in Section 1.3 will show that the scalar triple product is also the determinant of the matrix whose columns are $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ in that order.

1.1.7 Linear combinations, span, and basis

Let $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$ and let $\alpha_1, \dots, \alpha_k \in \mathbb{R}$. A **linear combination** of these vectors is any vector of the form

$$\alpha_1 \mathbf{v}_1 + \dots + \alpha_k \mathbf{v}_k.$$

The **span** of $\mathbf{v}_1, \dots, \mathbf{v}_k$ is the set of every linear combination that can be formed from them:

$$\text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\} := \left\{ \sum_{j=1}^k \alpha_j \mathbf{v}_j : \alpha_1, \dots, \alpha_k \in \mathbb{R} \right\}.$$

The vectors are **linearly independent** if

$$\alpha_1 \mathbf{v}_1 + \dots + \alpha_k \mathbf{v}_k = \mathbf{0}_n$$

implies $\alpha_1 = \cdots = \alpha_k = 0$, where $\mathbf{0}_n \in \mathbb{R}^n$ is the zero vector. If a nonzero set of coefficients produces $\mathbf{0}_n$, the vectors are linearly dependent.

A **basis** of $\mathbb{R}^n$ is a set of n linearly independent vectors that spans $\mathbb{R}^n$. If $\mathbf{e}_1, \dots, \mathbf{e}_n$ is a basis, every $\mathbf{x} \in \mathbb{R}^n$ has a unique expansion

$$\mathbf{x} = x_1 \mathbf{e}_1 + \cdots + x_n \mathbf{e}_n.$$

The scalar coefficients $x_1, \dots, x_n \in \mathbb{R}$ form the coordinate column of $\mathbf{x}$ in that basis. Uniqueness is essential: if the basis vectors were dependent, the same vector could have more than one coefficient description.

The **standard basis** of $\mathbb{R}^n$ is $\mathbf{e}_1, \dots, \mathbf{e}_n$, where $\mathbf{e}_i$ has entry 1 in position i and entry 0 in every other position. In the standard basis,

$$\mathbf{x} = x_1 \mathbf{e}_1 + \cdots + x_n \mathbf{e}_n \iff \mathbf{x} = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}^\top.$$

Quick test A: vectors and bases

1. **Recall:** What is the difference between a geometric vector $\mathbf{v}$ and a coordinate column ${}^a\mathbf{v}$?
2. **Explanation:** Why must coordinate columns be associated with a basis?
3. **Derivation:** Use the law of cosines and an algebraic expansion of $\|\mathbf{u} - \mathbf{v}\|_2^2$ to derive ${}^a\mathbf{u}^\top {}^a\mathbf{v} = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos \theta$.
4. **Application:** Find the norm and unit direction of $\mathbf{x} = \begin{bmatrix} 3 & 4 \end{bmatrix}^\top$.
5. **Limitations:** Why is the angle formula undefined when either vector is zero?
6. **Basis:** Explain why two parallel nonzero vectors cannot form a basis of $\mathbb{R}^2$.
7. **Cross product:** For a unit vector $\mathbf{a} \in \mathbb{R}^3$, use the vector triple-product identity to simplify $\mathbf{a} \times (\mathbf{a} \times \mathbf{y})$.
8. **Scalar triple product:** Evaluate $\mathbf{e}_1^\top (\mathbf{e}_2 \times \mathbf{e}_3)$, then predict the value after swapping $\mathbf{e}_2$ and $\mathbf{e}_3$. What geometric change causes the sign change?

Answers and hints

1. $\mathbf{v}$ is the coordinate-independent geometric object; ${}^a\mathbf{v}$ is its numerical representation in basis or frame $\{a\}$.
2. The entries are coefficients multiplying particular basis vectors. Without the basis, the numbers do not determine physical or geometric directions.
3. Equate the law-of-cosines expression for $\|\mathbf{u} - \mathbf{v}\|_2^2$ with its dot-product expansion and cancel the common norm terms.
4. $\|\mathbf{x}\|_2 = 5$ and the unit direction is $\begin{bmatrix} 0.6 & 0.8 \end{bmatrix}^\top$.

5. The denominator $\|\mathbf{u}\|_2\|\mathbf{v}\|_2$ becomes zero, and a zero vector has no direction.
6. One vector is a scalar multiple of the other, so their span is only a line and their coefficient representation is not unique.
7. $\mathbf{a} \times (\mathbf{a} \times \mathbf{y}) = \mathbf{a}(\mathbf{a}^\top \mathbf{y}) - \mathbf{y}$ because $\mathbf{a}^\top \mathbf{a} = 1$.
8. Since $\mathbf{e}_2 \times \mathbf{e}_3 = \mathbf{e}_1$, the first value is 1. Swapping the two cross-product inputs gives $\mathbf{e}_3 \times \mathbf{e}_2 = -\mathbf{e}_1$, so the second value is -1 . The swap reverses the orientation of the ordered directions while preserving the volume magnitude.

1.2 Matrices: linear maps and basis changes

1.2.1 Matrix dimensions

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ maps an n -element coordinate vector to an m -element coordinate vector:

$$\mathbf{y} = \mathbf{A}\mathbf{x}$$

where $\mathbf{y} \in \mathbb{R}^m$ and $\mathbf{x} \in \mathbb{R}^n$.

If

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

then

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 \\ a_{21}x_1 + a_{22}x_2 \end{bmatrix}.$$

Equivalently, if $\mathbf{a}_1$ and $\mathbf{a}_2$ are the columns of $\mathbf{A}$,

$$\mathbf{A}\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2.$$

This column interpretation is central to rotation matrices: their columns are the axes of one frame expressed in another frame.

1.2.2 Identity, transpose, null space, rank, and inverse

Let $\mathbf{I}_n$ denote the $n \times n$ identity matrix. It leaves compatible coordinates unchanged:

$$\mathbf{I}_n \mathbf{x} = \mathbf{x}.$$

The transpose swaps rows and columns:

$$(\mathbf{A}^\top)_{ij} = a_{ji}.$$

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$. Its **column space**, denoted $\text{Col}(\mathbf{A})$, is the set of all linear combinations of its columns. The **rank**, denoted $\text{rank}(\mathbf{A})$, is the number of linearly independent columns, equivalently the dimension of $\text{Col}(\mathbf{A})$.

The **null space** of $\mathbf{A}$ is

$$\mathcal{N}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}_m\},$$

where $\mathbf{0}_m \in \mathbb{R}^m$ is the zero vector. The null space contains all input directions that the mapping sends to zero.

1.2.2.1 Rank-nullity theorem

The **nullity** of $\mathbf{A}$ is the dimension of its null space:

$$\text{nullity}(\mathbf{A}) := \dim \mathcal{N}(\mathbf{A}).$$

For every matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, the **rank-nullity theorem** states

$$\boxed{\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n}.$$

The number n is the dimension of the input space $\mathbb{R}^n$. The theorem says that its independent input directions divide into two counts: $\text{nullity}(\mathbf{A})$ directions that the mapping sends to zero and $\text{rank}(\mathbf{A})$ directions that produce independent output directions.

To prove the theorem, let

$$k := \text{nullity}(\mathbf{A}),$$

and choose a basis $\mathbf{n}_1, \dots, \mathbf{n}_k \in \mathbb{R}^n$ for $\mathcal{N}(\mathbf{A})$. Because these vectors are linearly independent, they can be extended to a basis of the whole input space by repeatedly adding vectors outside their current span. Write the resulting basis as

$$\mathbf{n}_1, \dots, \mathbf{n}_k, \mathbf{v}_1, \dots, \mathbf{v}_{n-k}.$$

The output vectors

$$\mathbf{A}\mathbf{v}_1, \dots, \mathbf{A}\mathbf{v}_{n-k}$$

span $\text{Col}(\mathbf{A})$. To see this, express any input $\mathbf{x} \in \mathbb{R}^n$ in the chosen input basis:

$$\mathbf{x} = \sum_{i=1}^k a_i \mathbf{n}_i + \sum_{j=1}^{n-k} b_j \mathbf{v}_j.$$

Because every $\mathbf{n}_i$ belongs to the null space, $\mathbf{A}\mathbf{n}_i = \mathbf{0}_m$. Applying $\mathbf{A}$ therefore gives

$$\mathbf{A}\mathbf{x} = \sum_{j=1}^{n-k} b_j \mathbf{A}\mathbf{v}_j,$$

so every possible output is a linear combination of the listed output vectors.

These output vectors are also linearly independent. If

$$\sum_{j=1}^{n-k} c_j \mathbf{A}\mathbf{v}_j = \mathbf{0}_m,$$

then

$$\mathbf{A} \left(\sum_{j=1}^{n-k} c_j \mathbf{v}_j \right) = \mathbf{0}_m.$$

Thus, $\sum_j c_j \mathbf{v}_j$ belongs to $\mathcal{N}(\mathbf{A})$ and can also be written as a linear combination of $\mathbf{n}_1, \dots, \mathbf{n}_k$. The complete list $\mathbf{n}_1, \dots, \mathbf{n}_k, \mathbf{v}_1, \dots, \mathbf{v}_{n-k}$ is a basis and is therefore linearly independent, so this is possible only when every $c_j = 0$. Hence, the $n - k$ vectors $\mathbf{A}\mathbf{v}_j$ form a basis of $\text{Col}(\mathbf{A})$, and

$$\text{rank}(\mathbf{A}) = n - k.$$

Since $k = \text{nullity}(\mathbf{A})$,

$$\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = (n - k) + k = n,$$

which proves the theorem.

For example, let

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{2 \times 3}.$$

The first two columns are independent and the third is their sum, so $\text{rank}(\mathbf{A}) = 2$. Solving $\mathbf{Ax} = \mathbf{0}_2$ gives

$$\mathcal{N}(\mathbf{A}) = \left\{ \alpha \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} \mid \alpha \in \mathbb{R} \right\},$$

so $\text{nullity}(\mathbf{A}) = 1$. The input dimension is therefore partitioned as

$$\underbrace{3}_{\text{input dimension}} = \underbrace{2}_{\text{rank}} + \underbrace{1}_{\text{nullity}}.$$

1.2.2.2 Injective mappings and unique recovery

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ define the linear map

$$T_{\mathbf{A}} : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad T_{\mathbf{A}}(\mathbf{x}) = \mathbf{Ax}.$$

The map $T_{\mathbf{A}}$ is **injective**, or **one-to-one**, if equal outputs can occur only when the inputs are equal. Formally, for every $\mathbf{x}^{(1)}, \mathbf{x}^{(2)} \in \mathbb{R}^n$, where the parenthesised superscripts (1) and (2) label two inputs and are not exponents,

$$\mathbf{Ax}^{(1)} = \mathbf{Ax}^{(2)} \implies \mathbf{x}^{(1)} = \mathbf{x}^{(2)}.$$

Equivalently, distinct inputs must produce distinct outputs:

$$\mathbf{x}^{(1)} \neq \mathbf{x}^{(2)} \implies \mathbf{Ax}^{(1)} \neq \mathbf{Ax}^{(2)}.$$

The map is **not injective** if at least one pair of distinct inputs produces the same output:

$$\exists \mathbf{x}^{(1)}, \mathbf{x}^{(2)} \in \mathbb{R}^n : \mathbf{x}^{(1)} \neq \mathbf{x}^{(2)}, \quad \mathbf{Ax}^{(1)} = \mathbf{Ax}^{(2)}.$$

To connect injectivity to the null space, suppose two inputs have the same output. Subtracting the outputs gives

$$\begin{aligned}\mathbf{Ax}^{(1)} &= \mathbf{Ax}^{(2)} \\ \implies \mathbf{A}(\mathbf{x}^{(1)} - \mathbf{x}^{(2)}) &= \mathbf{0}_m.\end{aligned}$$

If $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}_n\}$, then $\mathbf{x}^{(1)} - \mathbf{x}^{(2)} = \mathbf{0}_n$, so the inputs must be equal. Conversely, if a nonzero vector $\mathbf{z} \in \mathcal{N}(\mathbf{A})$ exists, then for any $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{A}(\mathbf{x} + \mathbf{z}) = \mathbf{Ax} + \mathbf{Az} = \mathbf{Ax}.$$

Because $\mathbf{z} \neq \mathbf{0}_n$, the inputs $\mathbf{x}$ and $\mathbf{x} + \mathbf{z}$ are different but produce the same output. Therefore,

$$\boxed{T_{\mathbf{A}} \text{ is injective} \iff \mathcal{N}(\mathbf{A}) = \{\mathbf{0}_n\} \iff \mathbf{A} \text{ has full column rank}}.$$

For a square matrix, these conditions are also equivalent to nonsingularity and invertibility. If the map is injective, its output uniquely identifies its input; if it is not injective, some input information cannot be recovered from the output.

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a square matrix with columns $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^n$, and let $\mathbf{x} = [x_1 \ \cdots \ x_n]^T \in \mathbb{R}^n$. Matrix-vector multiplication forms the linear combination

$$\mathbf{Ax} = x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n = \sum_{j=1}^n x_j\mathbf{a}_j.$$

For every matrix, $\mathbf{x} = \mathbf{0}$ implies $\mathbf{Ax} = \mathbf{0}$. The square matrix $\mathbf{A}$ is **nonsingular** when the converse is also true:

$$\mathbf{Ax} = \mathbf{0} \iff \mathbf{x} = \mathbf{0}.$$

Equivalently,

$$\mathbf{x} \neq \mathbf{0} \implies \mathbf{Ax} \neq \mathbf{0}.$$

Because $\mathbf{Ax} = \sum_{j=1}^n x_j\mathbf{a}_j$, nonsingularity means that

$$x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n = \mathbf{0}$$

only when $x_1 = \cdots = x_n = 0$. This is precisely the definition that the columns $\mathbf{a}_1, \dots, \mathbf{a}_n$ are linearly independent.

A square matrix that is not nonsingular is called **singular**. For a singular matrix, at least one nonzero $\mathbf{x}$ satisfies $\mathbf{Ax} = \mathbf{0}$, so the mapping loses information about that input direction.

More generally, if $\mathbf{A} \in \mathbb{R}^{m \times n}$, then its columns are $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$ and $\mathbf{Ax} = \sum_{j=1}^n x_j \mathbf{a}_j$. The condition $\mathbf{Ax} = \mathbf{0}_m \Rightarrow \mathbf{x} = \mathbf{0}_n$ means that $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}_n\}$ and that all n columns are linearly independent. In that case $\text{rank}(\mathbf{A}) = n$, which is called **full column rank**. The term *nonsingular* is normally reserved for the square case $m = n$.

Only a nonsingular square matrix has an inverse $\mathbf{A}^{-1} \in \mathbb{R}^{n \times n}$. The inverse reverses the action of $\mathbf{A}$ and satisfies

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{AA}^{-1} = \mathbf{I}_n.$$

1.2.3 Multiplication order

Let n be a positive integer. Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{R}^{n \times n}$ be three matrices, and let $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{R}^n$ be coordinate vectors. Define the intermediate vector $\mathbf{z}$ and output vector $\mathbf{y}$ by

$$\mathbf{z} = \mathbf{Bx} \quad \text{and} \quad \mathbf{y} = \mathbf{Az}.$$

Substituting the expression for $\mathbf{z}$ into the expression for $\mathbf{y}$ gives

$$\mathbf{y} = \mathbf{A}(\mathbf{Bx}) = (\mathbf{AB})\mathbf{x}.$$

Thus, the rightmost matrix acts first: $\mathbf{B}$ maps $\mathbf{x}$ to $\mathbf{z}$, and then $\mathbf{A}$ maps $\mathbf{z}$ to $\mathbf{y}$.

The third matrix $\mathbf{C}$ allows three transformations to be composed. Matrix multiplication is **associative**, meaning that changing the grouping does not change the product:

$$(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC}).$$

Both sides are matrices in $\mathbb{R}^{n \times n}$. However, matrix multiplication is generally **not commutative**, meaning that changing the order can change the result:

$$\mathbf{AB} \neq \mathbf{BA}.$$

This order becomes critical when several coordinate transformations are composed.

Quick test B: matrices, rank, and inverses

1. **Recall:** For $\mathbf{A} \in \mathbb{R}^{m \times n}$, what are the dimensions of an input $\mathbf{x}$ and output $\mathbf{y} = \mathbf{Ax}$?
2. **Explanation:** What does column j of $\mathbf{A}$ tell us about the mapping?
3. **Derivation:** Explain why $\mathbf{Ax} = \mathbf{0}$ has only the zero solution exactly when the columns of $\mathbf{A}$ are linearly independent.
4. **Application:** Determine the null space and rank of $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$.
5. **Limitations:** Why can a nonsquare matrix not have a two-sided inverse of the same kind as a nonsingular square matrix?
6. **Composition:** If $\mathbf{z} = \mathbf{Bx}$ and $\mathbf{y} = \mathbf{Az}$, which matrix acts first in $\mathbf{y} = \mathbf{ABx}$?
7. **Injectivity:** Define an injective matrix mapping and explain why it is equivalent to $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$.
8. **Rank-nullity:** State the rank-nullity theorem for $\mathbf{A} \in \mathbb{R}^{m \times n}$ and verify it for the matrix in Question 4.

Answers and hints

1. $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^m$.
2. It is the output produced by applying $\mathbf{A}$ to the corresponding standard basis input; general outputs are linear combinations of the columns.
3. The equation is the linear combination $x_1 \mathbf{a}_1 + \cdots + x_n \mathbf{a}_n = \mathbf{0}$. Only zero coefficients solve it precisely when the columns are independent.
4. The second column is twice the first, so the rank is 1. Solving $x_1 + 2x_2 = 0$ gives $\mathcal{N}(\mathbf{A}) = \{t[-2 \ 1]^T : t \in \mathbb{R}\}$.
5. A two-sided inverse would have to undo mappings in both domain and codomain, which requires equal dimensions and no lost directions.
6. $\mathbf{B}$ acts first, followed by $\mathbf{A}$.
7. A map is injective if $\mathbf{Ax}^{(1)} = \mathbf{Ax}^{(2)}$ implies $\mathbf{x}^{(1)} = \mathbf{x}^{(2)}$. If $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$, subtracting equal outputs gives $\mathbf{A}(\mathbf{x}^{(1)} - \mathbf{x}^{(2)}) = \mathbf{0}$, so $\mathbf{x}^{(1)} - \mathbf{x}^{(2)} = \mathbf{0}$. Conversely, a nonzero null vector $\mathbf{z}$ makes $\mathbf{x}$ and $\mathbf{x} + \mathbf{z}$ distinct inputs with the same output.
8. The theorem is $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$. The matrix in Question 4 has two input columns, rank one, and a one-dimensional null space, so $1 + 1 = 2$.

1.3 Determinants and signed volume

1.3.1 What the determinant measures

The determinant is defined for a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and returns a scalar:

$$\det : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}.$$

Geometrically, $|\det(\mathbf{A})|$ is the factor by which the linear map $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ scales n -dimensional volume. The sign records whether the ordered basis orientation is preserved or reversed.

$$\text{For } \mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in \mathbb{R}^{2 \times 2},$$

$$\det(\mathbf{A}) = ad - bc.$$

Its magnitude is the area of the parallelogram formed by the columns of $\mathbf{A}$. For $\mathbf{A} = [\mathbf{a}_1 \ \mathbf{a}_2 \ \mathbf{a}_3] \in \mathbb{R}^{3 \times 3}$, the determinant is the scalar triple product defined in Section 1.1.6:

$$\det(\mathbf{A}) = \mathbf{a}_1^\top (\mathbf{a}_2 \times \mathbf{a}_3).$$

Its magnitude is the volume of the parallelepiped formed by the columns.

1.3.1.1 Determinant sign, the right-hand rule, and reflection

The columns of $\mathbf{A} = [\mathbf{a}_1 \ \cdots \ \mathbf{a}_n]$ are the images of the standard basis vectors because $\mathbf{A}\mathbf{e}_i = \mathbf{a}_i$. The determinant sign therefore compares the orientation of the ordered transformed columns with the orientation of the ordered standard basis.

In three dimensions, the standard basis is right-handed:

$$\mathbf{e}_1 \times \mathbf{e}_2 = \mathbf{e}_3.$$

For $\mathbf{A} = [\mathbf{a}_1 \ \mathbf{a}_2 \ \mathbf{a}_3] \in \mathbb{R}^{3 \times 3}$, the determinant can be written as

$$\det(\mathbf{A}) = (\mathbf{a}_1 \times \mathbf{a}_2)^\top \mathbf{a}_3.$$

The cross product $\mathbf{a}_1 \times \mathbf{a}_2$ points in the right-hand-rule normal direction determined by the first two columns. Consequently:

- If $\det(\mathbf{A}) > 0$, then $\mathbf{a}_3$ has a positive component in that right-hand-rule direction. The ordered columns $(\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3)$ are right-handed, so the transformation preserves orientation.
- If $\det(\mathbf{A}) < 0$, then $\mathbf{a}_3$ points to the opposite side of the plane spanned by $\mathbf{a}_1$ and $\mathbf{a}_2$. The ordered columns are left-handed, so the transformation reverses orientation.
- If $\det(\mathbf{A}) = 0$, then the three columns are coplanar or otherwise linearly dependent. The three-dimensional volume collapses to zero, so a full three-dimensional orientation is no longer defined.

A reflection reverses orientation. For example, reflection across the y - z plane is represented by

$$\mathbf{S} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \det(\mathbf{S}) = -1.$$

This transformation sends $\mathbf{e}_1$ to $-\mathbf{e}_1$ while leaving $\mathbf{e}_2$ and $\mathbf{e}_3$ unchanged. The transformed basis is left-handed because

$$(-\mathbf{e}_1) \times \mathbf{e}_2 = -\mathbf{e}_3,$$

whereas its third basis vector is $+\mathbf{e}_3$. Thus, the negative determinant detects the orientation reversal introduced by the reflection.

A matrix with negative determinant need not be a pure reflection: it may also stretch, scale, or shear. The negative sign means that its total action contains an orientation-reversing, or reflection-like, component. By contrast, a proper rotation preserves right-handedness and has determinant $+1$.

1.3.2 Algebraic definition

The determinant is the unique scalar-valued function of the columns of a square matrix with three defining properties:

1. **Multilinearity:** it is linear in each column while the other columns are held fixed.
2. **Alternation:** swapping two columns changes the sign; consequently, a matrix with repeated columns has determinant zero.
3. **Normalisation:** $\det(\mathbf{I}_n) = 1$.

1.3.2.1 Multilinearity in one column

Write the columns of $\mathbf{A} \in \mathbb{R}^{n \times n}$ as $\mathbf{A} = [\mathbf{a}_1 \ \cdots \ \mathbf{a}_n]$, where each $\mathbf{a}_i \in \mathbb{R}^n$. Choose a column index $j \in \{1, \dots, n\}$ and hold every column except column j fixed. For any vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and scalars $\alpha, \beta \in \mathbb{R}$, linearity in column j means

$$\begin{aligned} \det(\mathbf{a}_1, \dots, \mathbf{a}_{j-1}, \alpha\mathbf{u} + \beta\mathbf{v}, \mathbf{a}_{j+1}, \dots, \mathbf{a}_n) \\ = \alpha \det(\mathbf{a}_1, \dots, \mathbf{a}_{j-1}, \mathbf{u}, \mathbf{a}_{j+1}, \dots, \mathbf{a}_n) \\ + \beta \det(\mathbf{a}_1, \dots, \mathbf{a}_{j-1}, \mathbf{v}, \mathbf{a}_{j+1}, \dots, \mathbf{a}_n). \end{aligned}$$

Thus, a sum placed in one column can be separated into a sum of determinants, and a scalar multiplying one column can be moved outside the determinant. The word **multilinear**

means that this linearity property holds separately for every column. It does not mean that the determinant is linear in the whole matrix at once; in general,

$$\det(\mathbf{A} + \mathbf{B}) \neq \det(\mathbf{A}) + \det(\mathbf{B}).$$

For a two-dimensional check, let $\mathbf{u} = [u_1 \ u_2]^\top$, $\mathbf{v} = [v_1 \ v_2]^\top$, and $\mathbf{w} = [w_1 \ w_2]^\top$. Linearity in the first column follows directly from

$$\begin{aligned} \det \begin{bmatrix} \alpha u_1 + \beta v_1 & w_1 \\ \alpha u_2 + \beta v_2 & w_2 \end{bmatrix} &= (\alpha u_1 + \beta v_1)w_2 - w_1(\alpha u_2 + \beta v_2) \\ &= \alpha(u_1 w_2 - w_1 u_2) + \beta(v_1 w_2 - w_1 v_2) \\ &= \alpha \det[\mathbf{u} \ \mathbf{w}] + \beta \det[\mathbf{v} \ \mathbf{w}]. \end{aligned}$$

1.3.2.2 Alternation and repeated columns

Let $\mathbf{A}'$ be obtained from $\mathbf{A}$ by swapping columns i and j , where $i \neq j$. Alternation means

$$\det(\mathbf{A}') = -\det(\mathbf{A}).$$

A single swap reverses the orientation of the ordered columns, so it reverses the sign of the signed volume without changing its magnitude.

Now suppose columns i and j are equal to the same vector $\mathbf{c} \in \mathbb{R}^n$. Let the determinant of this matrix be D . Swapping the two equal columns leaves the matrix numerically unchanged, so its determinant is still D . Alternation simultaneously requires the determinant to become $-D$. Therefore,

$$D = -D \implies 2D = 0 \implies D = 0.$$

Hence, every matrix with two identical columns has determinant zero. Geometrically, two identical column directions cannot span an n -dimensional parallelepiped, so its n -dimensional volume collapses to zero.

Equivalently, if $\mathbf{A} = [a_{ij}] \in \mathbb{R}^{n \times n}$, then

$$\det(\mathbf{A}) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n a_{\sigma(j),j}.$$

Here, S_n is the set of all permutations of the indices $1, \dots, n$. The sign $\text{sgn}(\sigma)$ equals $+1$ when the permutation can be produced by an even number of pairwise swaps and -1 when it requires an odd number.

If the columns of $\mathbf{A}$ are linearly dependent, one column can be written as a linear combination of the others. Multilinearity and alternation then force $\det(\mathbf{A}) = 0$. Conversely, $\det(\mathbf{A}) = 0$ means that the signed volume has collapsed, so the columns do not span $\mathbb{R}^n$. Therefore,

$$\boxed{\det(\mathbf{A}) \neq 0 \iff \mathbf{A} \text{ is nonsingular}}.$$

1.3.2.3 Example: a singular matrix loses one input component

Consider the linear map $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ from $\mathbb{R}^2$ to $\mathbb{R}^2$, where

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Its determinant is

$$\det(\mathbf{A}) = 0.$$

For an input coordinate column

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2,$$

the output is

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} x_1 \\ 0 \end{bmatrix} \in \mathbb{R}^2.$$

The output retains x_1 but completely removes x_2 . For example,

$$\mathbf{A} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}, \quad \mathbf{A} \begin{bmatrix} 3 \\ 7 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}.$$

The two inputs are different, but their outputs are identical. From the output $[3 \ 0]^\top$, we can recover $x_1 = 3$, but we cannot determine whether x_2 was 1, 7, or any other real number. In fact,

$$\mathcal{N}(\mathbf{A}) = \left\{ t \begin{bmatrix} 0 \\ 1 \end{bmatrix} : t \in \mathbb{R} \right\},$$

so the entire input direction $[0 \ 1]^T$ is lost.

Geometrically, $\mathbf{A}$ maps the entire two-dimensional plane onto the one-dimensional x -axis:

$$\mathbb{R}^2 \longrightarrow \left\{ \begin{bmatrix} y_1 \\ 0 \end{bmatrix} : y_1 \in \mathbb{R} \right\}.$$

Every two-dimensional region is flattened into a line segment, whose two-dimensional area is zero. This agrees with $\det(\mathbf{A}) = 0$.

Recall that **not injective** means that there exist at least two distinct inputs $\mathbf{x}^{(1)}, \mathbf{x}^{(2)} \in \mathbb{R}^n$ with the same output:

$$\mathbf{x}^{(1)} \neq \mathbf{x}^{(2)}, \quad \mathbf{A}\mathbf{x}^{(1)} = \mathbf{A}\mathbf{x}^{(2)}.$$

The statement does not mean that every pair of inputs has the same output; it means that at least one distinct pair cannot be distinguished from the output.

If $\mathbf{A}$ is singular, then there exists $\mathbf{z} \in \mathcal{N}(\mathbf{A})$ such that $\mathbf{A}\mathbf{z} = \mathbf{0}$. This implies $\mathbf{A}(\mathbf{x} + \mathbf{z}) = \mathbf{A}\mathbf{x}$ for all $\mathbf{x} \in \mathbb{R}^n$. Hence, a singular matrix is not injective and different inputs can produce the same output.

For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the general equivalences are

$\begin{aligned} \det(\mathbf{A}) = 0 &\iff \mathbf{A} \text{ is singular} \\ &\iff \mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\} \\ &\iff \mathbf{A} \text{ is not injective} \\ &\iff \text{different inputs can produce the same output} \\ &\iff \text{the input cannot be uniquely recovered.} \end{aligned}$
--

1.3.3 Why determinants multiply

Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$. The product $\mathbf{AB}$ represents the composition

$$\mathbf{x} \mapsto \mathbf{B}\mathbf{x} \mapsto \mathbf{AB}\mathbf{x}.$$

Geometrically, $\mathbf{B}$ first scales signed volume by $\det(\mathbf{B})$, and $\mathbf{A}$ then scales the result by $\det(\mathbf{A})$. This suggests

$$\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B}).$$

A formal proof follows from the defining properties of the determinant. Hold $\mathbf{A}$ fixed and define

$$F : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}, \quad F(\mathbf{B}) := \det(\mathbf{AB}).$$

Write $\mathbf{B} = [\mathbf{b}_1 \ \cdots \ \mathbf{b}_n]$, where each $\mathbf{b}_j \in \mathbb{R}^n$. Then

$$\mathbf{AB} = [\mathbf{Ab}_1 \ \cdots \ \mathbf{Ab}_n].$$

To check linearity in column j , let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and $\alpha, \beta \in \mathbb{R}$. Because multiplication by $\mathbf{A}$ is linear,

$$\mathbf{A}(\alpha\mathbf{u} + \beta\mathbf{v}) = \alpha\mathbf{Au} + \beta\mathbf{Av}.$$

The determinant is linear in column j , so

$$\begin{aligned} F(\mathbf{b}_1, \dots, \alpha\mathbf{u} + \beta\mathbf{v}, \dots, \mathbf{b}_n) \\ = \alpha F(\mathbf{b}_1, \dots, \mathbf{u}, \dots, \mathbf{b}_n) + \beta F(\mathbf{b}_1, \dots, \mathbf{v}, \dots, \mathbf{b}_n). \end{aligned}$$

Thus, F is multilinear in the columns of $\mathbf{B}$. If two columns of $\mathbf{B}$ are equal, their images under $\mathbf{A}$ are equal, so $\mathbf{AB}$ has repeated columns and $F(\mathbf{B}) = 0$. Swapping two columns of $\mathbf{B}$ swaps the corresponding columns of $\mathbf{AB}$ and changes the determinant sign.

Hence, F is an alternating multilinear scalar-valued function of the columns of $\mathbf{B}$. We claim that

$$F(\mathbf{B}) = c \det(\mathbf{B}), \quad c := F(\mathbf{e}_1, \dots, \mathbf{e}_n).$$

The following expansion proves the claim.

Let $\mathbf{e}_1, \dots, \mathbf{e}_n$ be the standard basis of $\mathbb{R}^n$. Because it is a basis, every column $\mathbf{b}_j$ of $\mathbf{B}$ has the unique expansion

$$\mathbf{b}_j = \sum_{i=1}^n b_{ij} \mathbf{e}_i.$$

Using multilinearity once for each of the n columns gives

$$F(\mathbf{B}) = \sum_{i_1=1}^n \cdots \sum_{i_n=1}^n \left(\prod_{j=1}^n b_{i_j, j} \right) F(\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_n}).$$

This sum initially contains n^n terms. If two indices are equal, such as $i_p = i_{q'}$, then the corresponding argument list contains the same basis vector twice. Alternation makes that term zero:

$$F(\dots, \mathbf{e}_{i_p}, \dots, \mathbf{e}_{i_p}, \dots) = 0.$$

A term can therefore survive only when $i_1, \dots, i_n$ are all different. Because every i_j belongs to $\{1, \dots, n\}$, the surviving index list must contain every index exactly once. Thus, it has the form

$$(i_1, \dots, i_n) = (\sigma(1), \dots, \sigma(n))$$

for some permutation $\sigma \in S_n$.

Alternation determines the value of every surviving basis term relative to the ordered basis term. Reordering $(\mathbf{e}_{\sigma(1)}, \dots, \mathbf{e}_{\sigma(n)})$ into $(\mathbf{e}_1, \dots, \mathbf{e}_n)$ requires an even or odd number of swaps, so

$$F(\mathbf{e}_{\sigma(1)}, \dots, \mathbf{e}_{\sigma(n)}) = \text{sgn}(\sigma) F(\mathbf{e}_1, \dots, \mathbf{e}_n).$$

Substituting this result into the expansion for $F(\mathbf{B})$ leaves only the permutation terms:

$$F(\mathbf{B}) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n b_{\sigma(j), j} F(\mathbf{e}_1, \dots, \mathbf{e}_n)$$

The scalar appearing in the claim is

$$c := F(\mathbf{e}_1, \dots, \mathbf{e}_n).$$

The columns of $\mathbf{I}_n$ are the ordered standard basis vectors, so the same reference value can be written as

$$c = F(\mathbf{e}_1, \dots, \mathbf{e}_n) = F(\mathbf{I}_n).$$

Substituting the surviving terms into the expansion gives

$$\begin{aligned} F(\mathbf{B}) &= c \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n b_{\sigma(j), j} \\ &= c \det(\mathbf{B}). \end{aligned}$$

A two-dimensional example makes the one-free-value idea explicit. In this example, $F(\mathbf{u}, \mathbf{v})$ is shorthand for $F([\mathbf{u} \ \mathbf{v}])$, where $\mathbf{u}$ and $\mathbf{v}$ are the two columns of the input matrix. Let $\mathbf{u} = u_1\mathbf{e}_1 + u_2\mathbf{e}_2$ and $\mathbf{v} = v_1\mathbf{e}_1 + v_2\mathbf{e}_2$. Using multilinearity in both columns gives

$$\begin{aligned} F(\mathbf{u}, \mathbf{v}) &= u_1v_1F(\mathbf{e}_1, \mathbf{e}_1) + u_1v_2F(\mathbf{e}_1, \mathbf{e}_2) \\ &\quad + u_2v_1F(\mathbf{e}_2, \mathbf{e}_1) + u_2v_2F(\mathbf{e}_2, \mathbf{e}_2). \end{aligned}$$

The repeated-basis terms are zero, $F(\mathbf{e}_1, \mathbf{e}_2) = c$, and $F(\mathbf{e}_2, \mathbf{e}_1) = -c$. Therefore,

$$\begin{aligned} F(\mathbf{u}, \mathbf{v}) &= u_1v_2c - u_2v_1c \\ &= c(u_1v_2 - u_2v_1) \\ &= c \det[\mathbf{u} \ \mathbf{v}]. \end{aligned}$$

After c is known, this equation fixes $F(\mathbf{u}, \mathbf{v})$ for every pair $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$.

For the particular function $F(\mathbf{B}) = \det(\mathbf{AB})$,

$$\begin{aligned} c &= F(\mathbf{I}_n) \\ &= \det(\mathbf{AI}_n) \\ &= \det(\mathbf{A}). \end{aligned}$$

Consequently,

$$\boxed{\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})}.$$

1.3.3.1 Why transposition does not change the determinant

The related identity

$$\boxed{\det(\mathbf{A}^\top) = \det(\mathbf{A})}$$

does not follow from multiplicativity alone. It follows directly from the permutation definition of the determinant.

Let $\mathbf{A} = [a_{ij}] \in \mathbb{R}^{n \times n}$. Because $(\mathbf{A}^\top)_{ij} = a_{ji}$,

$$\begin{aligned}
\det(\mathbf{A}^\top) &= \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{j=1}^n (\mathbf{A}^\top)_{\sigma(j),j} \\
&= \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{j=1}^n a_{j,\sigma(j)}.
\end{aligned}$$

A permutation $\sigma \in S_n$ is a bijection from the index set $\{1, \dots, n\}$ to itself. Because it is bijective, it has a unique **inverse permutation**, denoted by σ^{-1} , that reverses the mapping performed by σ . Define

$$\tau := \sigma^{-1}.$$

The defining identities are

$$\tau(\sigma(j)) = j, \quad \sigma(\tau(k)) = k.$$

Equivalently,

$$k = \sigma(j) \iff j = \tau(k).$$

For example, suppose $n = 3$ and

$$\sigma(1) = 2, \quad \sigma(2) = 3, \quad \sigma(3) = 1.$$

The inverse permutation reverses these assignments:

$$\tau(1) = 3, \quad \tau(2) = 1, \quad \tau(3) = 2.$$

For instance, $\sigma(1) = 2$ is reversed by $\tau(2) = 1$.

As σ runs through all permutations in S_n , $\tau = \sigma^{-1}$ also runs through all permutations in S_n . This is because inversion pairs every σ with exactly one τ , and the pairing is reversible: $\sigma = \tau^{-1}$. Therefore, replacing the summation index σ by τ neither loses nor duplicates a permutation.

The inverse permutation also has the same sign as the original permutation. Suppose σ can be produced by r pairwise swaps. To undo σ , perform those same r swaps in reverse order. Thus, σ^{-1} uses the same number of swaps as σ . They therefore have the same even-or-odd parity, so

$$\operatorname{sgn}(\tau) = \operatorname{sgn}(\sigma^{-1}) = \operatorname{sgn}(\sigma).$$

Now consider the product

$$\prod_{j=1}^n a_{j,\sigma(j)}.$$

Introduce the new index

$$k := \sigma(j).$$

Because σ is a permutation, as j visits $1, \dots, n$, the index $k = \sigma(j)$ also visits every value $1, \dots, n$ exactly once. From $\tau = \sigma^{-1}$, the relation $k = \sigma(j)$ implies $j = \tau(k)$. Each factor can therefore be rewritten as

$$a_{j,\sigma(j)} = a_{\tau(k),k}.$$

Consequently,

$$\prod_{j=1}^n a_{j,\sigma(j)} = \prod_{k=1}^n a_{\tau(k),k}.$$

The two products contain the same scalar factors, possibly in a different order. Scalar multiplication is commutative, so their order does not affect the product. In the three-dimensional example above,

$$\begin{aligned} \prod_{j=1}^3 a_{j,\sigma(j)} &= a_{1,2} a_{2,3} a_{3,1}, \\ \prod_{k=1}^3 a_{\tau(k),k} &= a_{3,1} a_{1,2} a_{2,3}. \end{aligned}$$

These products are equal because they contain exactly the same three factors.

Therefore,

$$\begin{aligned} \det(\mathbf{A}^\top) &= \sum_{\tau \in S_n} \operatorname{sgn}(\tau) \prod_{k=1}^n a_{\tau(k),k} \\ &= \det(\mathbf{A}). \end{aligned}$$

Thus, exchanging the rows and columns changes how the same determinant terms are indexed but does not change their values or signs.

1.3.3.2 Determinant of an inverse

For nonsingular $\mathbf{A}$, multiplicativity does give the inverse identity. Taking determinants of $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$ yields

$$\begin{aligned} 1 &= \det(\mathbf{I}_n) \\ &= \det(\mathbf{A}^{-1}\mathbf{A}) \\ &= \det(\mathbf{A}^{-1}) \det(\mathbf{A}). \end{aligned}$$

Since $\det(\mathbf{A}) \neq 0$,

$$\boxed{\det(\mathbf{A}^{-1}) = \frac{1}{\det(\mathbf{A})}}.$$

1.3.4 How an invertible linear map transforms a cross product

The cross product depends on lengths, angles, and the handedness of the coordinate system. A general linear map may stretch those lengths, change the angles, or reverse the handedness, so it does not usually pass through a cross product unchanged.

Let $\mathbf{A} \in \mathbb{R}^{3 \times 3}$ be nonsingular, and let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ be expressed in the same right-handed orthonormal coordinates. The transformed cross product is

$$\boxed{(\mathbf{A}\mathbf{a}) \times (\mathbf{A}\mathbf{b}) = \det(\mathbf{A})\mathbf{A}^{-\top}(\mathbf{a} \times \mathbf{b})},$$

where $\mathbf{A}^{-\top} := (\mathbf{A}^{-1})^{\top} = (\mathbf{A}^{\top})^{-1}$ is the **inverse transpose** of $\mathbf{A}$. Nonsingularity is required because $\mathbf{A}^{-1}$ must exist.

To prove the identity, compare the dot product of both sides with an arbitrary test column $\mathbf{c} \in \mathbb{R}^3$. The scalar-triple-product formula from Section 1.1.6 gives

$$\begin{aligned} \mathbf{c}^{\top} [(\mathbf{A}\mathbf{a}) \times (\mathbf{A}\mathbf{b})] &= \det [\mathbf{c} \quad \mathbf{A}\mathbf{a} \quad \mathbf{A}\mathbf{b}] \\ &= \det (\mathbf{A} [\mathbf{A}^{-1}\mathbf{c} \quad \mathbf{a} \quad \mathbf{b}]) \\ &= \det(\mathbf{A}) \det [\mathbf{A}^{-1}\mathbf{c} \quad \mathbf{a} \quad \mathbf{b}] \\ &= \det(\mathbf{A})(\mathbf{A}^{-1}\mathbf{c})^{\top}(\mathbf{a} \times \mathbf{b}) \\ &= \mathbf{c}^{\top} [\det(\mathbf{A})\mathbf{A}^{-\top}(\mathbf{a} \times \mathbf{b})]. \end{aligned}$$

The second equality uses $\mathbf{c} = \mathbf{A}(\mathbf{A}^{-1}\mathbf{c})$ to factor $\mathbf{A}$ from all three columns. The third uses determinant multiplicativity. The final equality uses $(\mathbf{A}^{-1}\mathbf{c})^T = \mathbf{c}^T\mathbf{A}^{-T}$. Because both candidate vectors have the same dot product with every $\mathbf{c}$, they are equal.

For an orthogonal matrix $\mathbf{Q} \in \mathbb{R}^{3 \times 3}$, $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}_3$ implies $\mathbf{Q}^{-T} = \mathbf{Q}$. The general identity then becomes

$$(\mathbf{Q}\mathbf{a}) \times (\mathbf{Q}\mathbf{b}) = \det(\mathbf{Q})\mathbf{Q}(\mathbf{a} \times \mathbf{b}).$$

If $\det(\mathbf{Q}) = 1$, then $\mathbf{Q}$ is a proper rotation and preserves the cross-product direction. If $\det(\mathbf{Q}) = -1$, then $\mathbf{Q}$ contains a reflection and reverses that direction. This determinant sign is why orthogonality alone is insufficient for a rotation to preserve cross products.

Quick test C: determinants

1. **Recall:** What does $|\det(\mathbf{A})|$ measure geometrically?
2. **Explanation:** What does the sign of a determinant record?
3. **Derivation:** Explain why repeated columns force the determinant to be zero.
4. **Application:** Compute $\det \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$ and interpret its magnitude.
5. **Limitations:** Why is the determinant not defined in the same way for a rectangular matrix?
6. **Composition:** Explain geometrically why $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$.
7. **Reindexing:** If $\sigma(1) = 2$, $\sigma(2) = 3$, and $\sigma(3) = 1$, find $\tau = \sigma^{-1}$ and verify $\prod_{j=1}^3 a_{j,\sigma(j)} = \prod_{k=1}^3 a_{\tau(k),k}$.
8. **Cross products under linear maps:** State how a nonsingular $\mathbf{A} \in \mathbb{R}^{3 \times 3}$ transforms a cross product. Why does a proper rotation preserve the cross product while a reflection reverses it?

Answers and hints

1. The factor by which the associated linear map scales n -dimensional volume.
2. A positive determinant preserves the ordered right-handed orientation; a negative determinant reverses it to a left-handed orientation and therefore indicates a reflection component; a zero determinant collapses the volume, so no full-dimensional orientation remains.
3. Swapping identical columns leaves the matrix unchanged but must negate the determinant, so the determinant equals its own negative and is zero.
4. The determinant is $2(4) - 1(3) = 5$, so areas are scaled by a factor of 5 and orientation is preserved.

5. A rectangular map changes dimension, so it does not map an n -dimensional volume to another volume in the same n -dimensional space.
6. $\mathbf{B}$ scales signed volume first and $\mathbf{A}$ scales the resulting signed volume second, so the factors multiply.
7. The inverse is $\tau(1) = 3$, $\tau(2) = 1$, and $\tau(3) = 2$. The left product is $a_{1,2}a_{2,3}a_{3,1}$, while the reindexed product is $a_{3,1}a_{1,2}a_{2,3}$; these are equal because scalar factors commute.
8. $(\mathbf{A}\mathbf{a}) \times (\mathbf{A}\mathbf{b}) = \det(\mathbf{A})\mathbf{A}^{-\top}(\mathbf{a} \times \mathbf{b})$. For orthogonal $\mathbf{Q}$, $\mathbf{Q}^{-\top} = \mathbf{Q}$. A proper rotation has determinant $+1$ and therefore preserves the cross product, whereas an orthogonal reflection has determinant -1 and introduces a minus sign.

1.4 Orthogonal matrices

Let $\mathbf{Q} \in \mathbb{R}^{n \times n}$. The matrix $\mathbf{Q}$ is **orthogonal** if

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_n.$$

The entries of $\mathbf{Q}^\top \mathbf{Q}$ are pairwise dot products between columns of $\mathbf{Q}$. Therefore, this equation is equivalent to saying that the columns form an orthonormal basis of $\mathbb{R}^n$.

Because $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_n$, the transpose satisfies the defining equation for the inverse:

$$\boxed{\mathbf{Q}^{-1} = \mathbf{Q}^\top}.$$

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$,

$$(\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top \mathbf{Q}^\top \mathbf{Q} \mathbf{y} = \mathbf{x}^\top \mathbf{y}.$$

Thus, orthogonal matrices preserve dot products, lengths, and angles.

Taking determinants of $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_n$ gives

$$\det(\mathbf{Q})^2 = 1,$$

so

$$\det(\mathbf{Q}) = +1 \quad \text{or} \quad \det(\mathbf{Q}) = -1.$$

The **orthogonal group** is

$$O(n) := \{ \mathbf{Q} \in \mathbb{R}^{n \times n} : \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_n \}.$$

The **special orthogonal group** is

$$SO(n) := \{\mathbf{Q} \in O(n) : \det(\mathbf{Q}) = 1\}.$$

Matrices in $SO(n)$ preserve lengths, angles, and orientation. Orthogonal matrices with determinant -1 preserve lengths and angles but reverse orientation through a reflection. Rotation matrices used for right-handed coordinate frames belong to $SO(2)$ or $SO(3)$.

Quick test D: orthogonal matrices

1. **Recall:** What equation defines an orthogonal matrix?
2. **Explanation:** Why is the inverse of an orthogonal matrix its transpose?
3. **Derivation:** Show that $\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$.
4. **Application:** Determine whether $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ belongs to $SO(2)$.
5. **Limitations:** Why is $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$ insufficient by itself to identify a proper rotation?

Answers and hints

1. $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$.
2. The transpose already satisfies the defining inverse equation.
3. Square both norms and use $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$.
4. It is orthogonal but has determinant -1 , so it is a reflection in $O(2)$ and does not belong to $SO(2)$.
5. Orthogonality permits determinant -1 reflections as well as determinant $+1$ rotations.

Cumulative chapter test

1. Define a basis and explain why coordinate columns are unique in a basis.
2. Given $\mathbf{A} \in \mathbb{R}^{3 \times 2}$ and $\mathbf{x} \in \mathbb{R}^2$, state the dimension of $\mathbf{A}\mathbf{x}$ and interpret it using the columns of $\mathbf{A}$.
3. Explain the relationship between $\mathcal{N}(\mathbf{A})$, independent columns, and full column rank.
4. Prove that a nonsingular square matrix cannot map a nonzero vector to zero.
5. Define the determinant using multilinearity, alternation, and normalisation.
6. Explain why $\det(\mathbf{A}) = 0$ indicates lost information.
7. Derive $\det(\mathbf{A}^{-1}) = 1/\det(\mathbf{A})$.

8. Distinguish $O(3)$ from $SO(3)$ and explain which set contains rigid-body rotation matrices.
9. State the rank-nullity theorem and explain what its two terms count.
10. State the vector triple-product identity and explain why the order and parentheses in a nested cross product matter.

Cumulative answers and hints

1. A basis is a linearly independent spanning set. Independence prevents multiple coefficient descriptions, while spanning guarantees that a description exists.
2. $\mathbf{Ax} \in \mathbb{R}^3$ and equals $x_1\mathbf{a}_1 + x_2\mathbf{a}_2$.
3. $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$ exactly when the columns are independent; for n columns this gives $\text{rank}(\mathbf{A}) = n$.
4. Nonsingularity is equivalent to $\mathbf{Ax} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$; the contrapositive is $\mathbf{x} \neq \mathbf{0} \Rightarrow \mathbf{Ax} \neq \mathbf{0}$.
5. The determinant is the unique scalar function that is linear in each column, changes sign under a column swap, and equals 1 on $\mathbf{I}_n$.
6. For $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\det(\mathbf{A}) = 0$ means that $\mathbf{A}$ is singular, so its null space contains a nonzero vector $\mathbf{z} \in \mathbb{R}^n$ satisfying $\mathbf{Az} = \mathbf{0}$. For any $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{A}(\mathbf{x} + \mathbf{z}) = \mathbf{Ax} + \mathbf{Az} = \mathbf{Ax}.$$

Therefore, the distinct inputs $\mathbf{x}$ and $\mathbf{x} + \mathbf{z}$ produce the same output. The output cannot reveal the input component in the null-space direction $\mathbf{z}$, so the input cannot be uniquely recovered. This is the meaning of **lost information** here: an independent input direction or degree of freedom has disappeared, not that Shannon information has been calculated. Geometrically, the map collapses n -dimensional volume into a lower-dimensional set, which is why its volume-scaling factor is zero.

7. Take determinants of $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$ and use multiplicativity.
8. $O(3)$ contains all orthogonal 3×3 matrices with determinant ± 1 ; $SO(3)$ contains the determinant-1 members and therefore the proper rigid-body rotations.
9. For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$. Rank counts independent output directions produced by the map, while nullity counts independent input directions sent to zero.
10. $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a}^\top \mathbf{c}) - \mathbf{c}(\mathbf{a}^\top \mathbf{b})$. The cross product is not associative, so changing the parentheses generally changes the result.

References

- Richard Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Appendix A, especially the discussion of orthonormal matrices, rank, singular values, and matrix factorisations.
- Kevin M. Lynch and Frank C. Park, *Modern Robotics*, Chapter 3, especially Section 3.2.1 on rotation matrices and the distinction between orthogonality and right-handed orientation.

2 Geometry and Coordinate Frames

Why coordinate frames matter

A robot represents positions, directions, velocities, measurements, and commands with numbers. Those numbers acquire physical meaning only when their coordinate frame and units are known. A controller that combines quantities expressed in incompatible frames can perform a mathematically valid calculation and produce a physically incorrect command.

Coordinate-frame methods allow us to:

- represent a robot's pose relative to a world frame;
- distinguish world-frame and body-frame quantities;
- transform sensor measurements into a shared frame;
- express target and tracking errors consistently;
- convert body-relative motion into world-relative motion; and
- detect invalid transformations before they reach a controller.

The central rule is simple: interpret every coordinate column together with the frame in which it is expressed.

Prerequisites and assumptions

Prerequisites

The reader should understand the material in Chapter 1, especially:

- geometric vectors and coordinate columns;
- dot products, norms, and orthogonality;
- matrices as linear maps and compositions;
- linear independence, rank, and inverses;
- determinants and signed orientation; and
- orthogonal matrices.

The reader should also be comfortable with sine, cosine, and angles measured in radians.

Assumptions

- Physical space is modelled as Euclidean space.
- Robot links and sensor mounts are treated as rigid bodies.
- Coordinate frames are right-handed and use orthonormal basis vectors.
- Length is measured in metres, time in seconds, and angles in radians.
- Coordinate vectors are written as columns.
- Numerical examples ignore measurement noise unless stated otherwise.

These idealisations isolate the geometry. A physical robot also introduces measurement noise, calibration error, structural compliance, and model mismatch.

Dependency map

```

Linear algebra foundations
  |
  v
Cross products and rigid-body velocity
  |
  v
Coordinate frames and units
  |
  v
Rotation matrices
  |
  v
Point transformations
  |
  v
Differential-drive motion and feedback control

```

2.1 Cross product and rigid-body velocity

Let $\mathbf{u}$ and $\mathbf{v}$ be geometric vectors with coordinate columns ${}^a\mathbf{u}, {}^a\mathbf{v} \in \mathbb{R}^3$ expressed in the same right-handed orthonormal frame $\{a\}$. Their cross product, expressed in frame $\{a\}$, is defined by

$${}^a\mathbf{u} \times {}^a\mathbf{v} = \begin{bmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{bmatrix} \in \mathbb{R}^3.$$

Its direction follows the right-hand rule. If both vectors are nonzero, let $\theta \in [0, \pi]$ be the smaller angle between them. The cross-product magnitude is

$$\|{}^a\mathbf{u} \times {}^a\mathbf{v}\|_2 = \|{}^a\mathbf{u}\|_2 \|{}^a\mathbf{v}\|_2 \sin \theta.$$

The cross product is perpendicular to both input vectors because

$${}^a\mathbf{u}^\top ({}^a\mathbf{u} \times {}^a\mathbf{v}) = 0, \quad {}^a\mathbf{v}^\top ({}^a\mathbf{u} \times {}^a\mathbf{v}) = 0.$$

2.1.1 Example: velocity of a point on a rotating rigid body

Let O and P be two points fixed to the same rigid body. Point O is the chosen reference point; it need not be stationary, and it need not be the body's centre. The vector ${}^a\mathbf{r}_{OP}$ runs from O to P . Although this vector is fixed relative to the body, its coordinates generally change as the body rotates. Express every quantity in the same orthonormal frame $\{a\}$:

- ${}^a\boldsymbol{\omega}$ as the body's angular-velocity vector, measured in rad s^{-1} ;
- ${}^a\mathbf{r}_{OP}$ as the position of P relative to O , measured in m;
- ${}^a\mathbf{v}_O$ as the linear velocity of O , measured in m s^{-1} ; and
- ${}^a\mathbf{v}_P$ as the linear velocity of P , measured in m s^{-1} .

At any instant, the motion of P can be understood as the sum of two simultaneous contributions:

1. **Translation with O .** If the body were not rotating, P would share the reference-point velocity ${}^a\mathbf{v}_O$.
2. **Motion relative to O caused by rotation.** Rotation adds a velocity whose direction and magnitude depend on ${}^a\boldsymbol{\omega}$ and ${}^a\mathbf{r}_{OP}$.

The angular-velocity vector points along the instantaneous rotation axis according to the right-hand rule, and its magnitude is the angular speed. Define the rotational contribution to the velocity of P by

$${}^a\mathbf{v}_{P/O} := {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}.$$

The cross product determines both the direction and the magnitude of this contribution. In particular,

$${}^a\boldsymbol{\omega}^\top {}^a\mathbf{v}_{P/O} = 0, \quad {}^a\mathbf{r}_{OP}^\top {}^a\mathbf{v}_{P/O} = 0.$$

Therefore, ${}^a\mathbf{v}_{P/O}$ is perpendicular to the rotation axis and to ${}^a\mathbf{r}_{OP}$. It points tangent to the instantaneous circular path generated by the rotation. Adding the translational and rotational contributions gives the rigid-body velocity relation

$${}^a\mathbf{v}_P = {}^a\mathbf{v}_O + {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}.$$

2.1.1.1 Velocity-vector construction

The construction in Figure 2.1 shows this addition for one instant:

- The green arrow ${}^a\mathbf{r}_{OP}$ locates P relative to O .
- The magenta arrow is the reference-point velocity ${}^a\mathbf{v}_O$. It is deliberately not parallel to ${}^a\mathbf{r}_{OP}$.
- The orange arrow is the rotational contribution ${}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP} = {}^a\mathbf{v}_{P/O}$. The right-angle marker shows that it is perpendicular to ${}^a\mathbf{r}_{OP}$.
- The dashed arrows are translated copies of the two velocity contributions. Moving a vector without rotating or resizing it does not change the vector, so the copies can be arranged head to tail.
- The blue arrow joins the beginning of the first contribution to the end of the second. It is therefore the resultant velocity ${}^a\mathbf{v}_P$.

In this planar view, ${}^a\boldsymbol{\omega}$ points along $+z_a$, out of the page. The right-hand rule then gives the displayed tangential direction for ${}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}$. Notice that only the rotational contribution must be perpendicular to ${}^a\mathbf{r}_{OP}$. The total velocity ${}^a\mathbf{v}_P$ generally is not, because ${}^a\mathbf{v}_O$ may point in any direction.

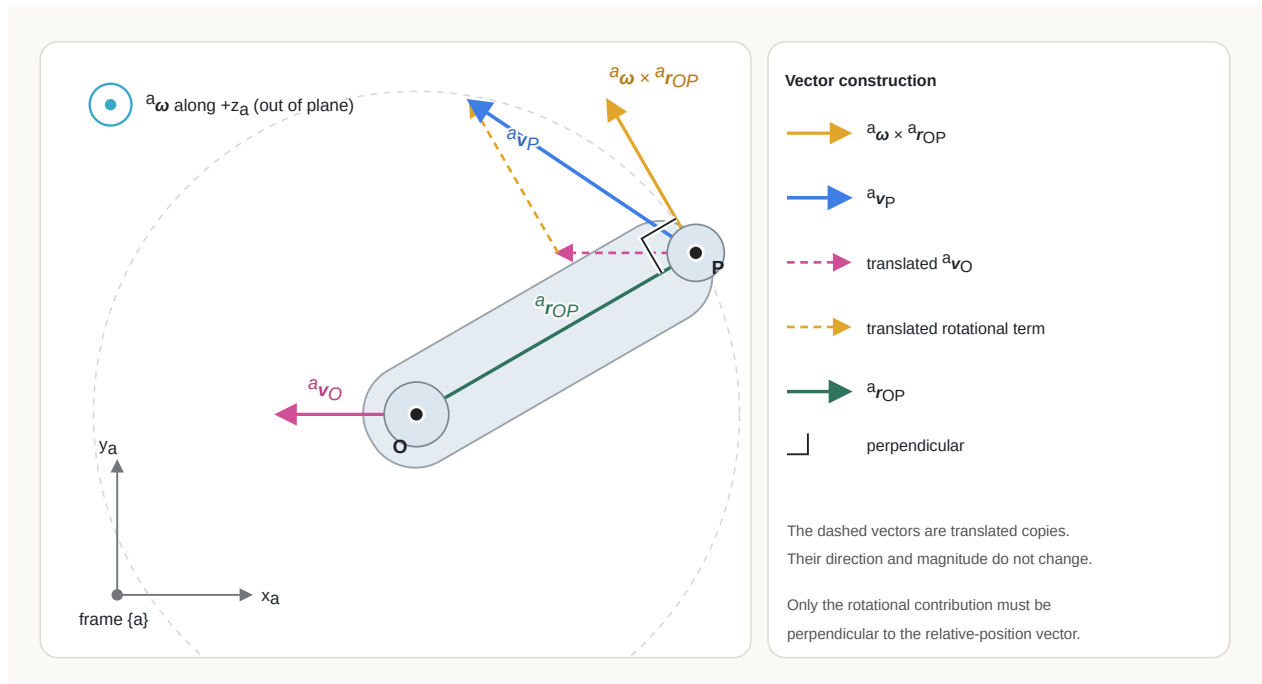


Figure 2.1: Velocity construction for two points on a rigid body. The rotational contribution is perpendicular to the position vector, while the reference-point velocity is independent of that perpendicular relationship.

For pure rotation about a fixed axis through O , the reference point is stationary, so ${}^a\mathbf{v}_O = \mathbf{0}$. Therefore,

$$\boxed{{}^a\mathbf{v}_P = {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}}.$$

If ϕ is the angle between ${}^a\boldsymbol{\omega}$ and ${}^a\mathbf{r}_{OP}$, then the relative speed caused by rotation is

$$\|{}^a\mathbf{v}_{P/O}\|_2 = \|{}^a\boldsymbol{\omega}\|_2 \|{}^a\mathbf{r}_{OP}\|_2 \sin \phi = \omega r_{\perp},$$

where $r_{\perp}$ is the perpendicular distance from P to the rotation axis. A point on the axis has $r_{\perp} = 0$ and therefore has no relative linear velocity due to rotation. Under the pure-rotation assumption, ${}^a\mathbf{v}_P = {}^a\mathbf{v}_{P/O}$.

For planar counter-clockwise rotation about the positive z -axis,

$${}^a\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix}, \quad {}^a\mathbf{r}_{OP} = \begin{bmatrix} x \\ y \\ 0 \end{bmatrix}.$$

The point velocity is

$${}^a\mathbf{v}_P = {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP} = \begin{bmatrix} -\omega y \\ \omega x \\ 0 \end{bmatrix}.$$

For example, if $\omega = 2 \text{ rad s}^{-1}$ and ${}^a\mathbf{r}_{OP} = [0.5 \ 0 \ 0]^T \text{ m}$, then

$${}^a\mathbf{v}_P = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

The point lies on the positive x -axis, while its instantaneous velocity points along the positive y -axis, tangent to its circular path.

Quick test A: cross products and rigid-body velocity

1. **Recall:** What rule determines the direction of $\mathbf{u} \times \mathbf{v}$?
2. **Explanation:** Why is $\mathbf{u} \times \mathbf{v}$ perpendicular to both input vectors?
3. **Derivation:** Substitute the component formula for ${}^a\mathbf{u} \times {}^a\mathbf{v}$ into ${}^a\mathbf{u}^T ({}^a\mathbf{u} \times {}^a\mathbf{v})$ and show that the terms cancel.
4. **Application:** Compute $[1 \ 2 \ 0]^T \times [0 \ 1 \ 0]^T$.
5. **Limitations:** What happens to the cross product when the input vectors are parallel, and why does that prevent it from defining a unique normal direction?
6. **Transfer:** At one instant, a rigid body has ${}^a\boldsymbol{\omega} = [0 \ 0 \ 2]^T \text{ rad s}^{-1}$, ${}^a\mathbf{r}_{OP} = [1 \ 0 \ 0]^T \text{ m}$, and ${}^a\mathbf{v}_O = [0.5 \ 0.5 \ 0]^T \text{ m s}^{-1}$. Find ${}^a\mathbf{v}_{P/O}$ and ${}^a\mathbf{v}_P$. Which velocity is perpendicular to ${}^a\mathbf{r}_{OP}$?

Answers and hints

1. The right-hand rule, applied in the order from $\mathbf{u}$ to $\mathbf{v}$.
2. Its dot product with either input is zero.
3. Expanding gives $u_x(u_y v_z - u_z v_y) + u_y(u_z v_x - u_x v_z) + u_z(u_x v_y - u_y v_x) = 0$ after pairwise cancellation.
4. The result is $[0 \ 0 \ 1]^T$.
5. Parallel vectors have $\sin \theta = 0$, so their cross product is the zero vector. A zero vector has no direction and therefore cannot specify a unique normal.
6. ${}^a\mathbf{v}_{P/O} = [0 \ 2 \ 0]^T \text{ m s}^{-1}$ and ${}^a\mathbf{v}_P = [0.5 \ 2.5 \ 0]^T \text{ m s}^{-1}$. The rotational contribution is perpendicular to ${}^a\mathbf{r}_{OP}$; the total velocity is not.

2.2 Coordinate frames

2.2.1 What defines a frame?

A three-dimensional coordinate frame $\{a\}$ consists of:

1. an origin O_a , which is the physical point from which positions in frame $\{a\}$ are measured; and
2. three right-handed orthonormal basis vectors $\mathbf{e}_{x,a}$, $\mathbf{e}_{y,a}$, and $\mathbf{e}_{z,a}$, which define the positive x_a , y_a , and z_a -axis directions.

The subscript a associates the origin and basis vectors with frame $\{a\}$. Orthonormal means that each basis vector has unit length and that distinct basis vectors are perpendicular. Right-handed means that $\mathbf{e}_{x,a} \times \mathbf{e}_{y,a} = \mathbf{e}_{z,a}$.

A free vector depends only on the basis orientation used to describe it. A point also depends on the frame origin. This distinction explains why changing the coordinate frame

of a free vector requires only a rotation, whereas changing the coordinates of a point generally requires a rotation and a translation.

2.2.2 Orientation convention

Let $\{a\}$ and $\{b\}$ be three-dimensional right-handed orthonormal frames. Define $\mathbf{R}_{ab} \in \mathbb{R}^{3 \times 3}$ as the rotation matrix that maps coordinate columns from frame $\{b\}$ to frame $\{a\}$. Its columns are the basis vectors of $\{b\}$ expressed in frame $\{a\}$:

$$\mathbf{R}_{ab} = \begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} & {}^a\mathbf{e}_{z,b} \end{bmatrix},$$

where each column ${}^a\mathbf{e}_{i,b} \in \mathbb{R}^3$ is an axis of frame $\{b\}$ written using the basis of frame $\{a\}$, for $i \in \{x, y, z\}$.

The following notation distinguishes a geometric vector from its coordinate representations:

Symbol	Meaning
$\mathbf{v}$	A geometric vector, independent of the coordinate frame used to describe it.
$\mathbf{e}_{x,b}$	The geometric unit vector along the positive x_b -axis of frame $\{b\}$.
$[\mathbf{v}]_b = {}^b\mathbf{v} \in \mathbb{R}^3$	The numerical coordinate column of $\mathbf{v}$ in frame $\{b\}$.
$[\mathbf{v}]_a = {}^a\mathbf{v} \in \mathbb{R}^3$	The numerical coordinate column of $\mathbf{v}$ in frame $\{a\}$.
$[\mathbf{e}_{x,b}]_a = {}^a\mathbf{e}_{x,b} \in \mathbb{R}^3$	The coordinate column of frame $\{b\}$'s geometric x -axis, expressed in frame $\{a\}$.

The bracket operator $[\cdot]_a$ means “take the coordinate column of the enclosed geometric vector in frame $\{a\}$.” The geometric vector $\mathbf{v}$ can be expanded using either frame's basis:

$$\mathbf{v} = v_x^{(b)} \mathbf{e}_{x,b} + v_y^{(b)} \mathbf{e}_{y,b} + v_z^{(b)} \mathbf{e}_{z,b} = v_x^{(a)} \mathbf{e}_{x,a} + v_y^{(a)} \mathbf{e}_{y,a} + v_z^{(a)} \mathbf{e}_{z,a}.$$

Here, $v_i^{(b)} \in \mathbb{R}$ and $v_i^{(a)} \in \mathbb{R}$ are the scalar components of $\mathbf{v}$ along axis $i \in \{x, y, z\}$ of frames $\{b\}$ and $\{a\}$, respectively. The corresponding coordinate columns are

$${}^b\mathbf{v} = [\mathbf{v}]_b = \begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \\ v_z^{(b)} \end{bmatrix}, \quad {}^a\mathbf{v} = [\mathbf{v}]_a = \begin{bmatrix} v_x^{(a)} \\ v_y^{(a)} \\ v_z^{(a)} \end{bmatrix}.$$

Apply the coordinate operator $[\cdot]_a$ to the expansion of $\mathbf{v}$ in the frame- $\{b\}$ basis. Because taking coordinates is a linear operation,

$$[\mathbf{v}]_a = v_x^{(b)}[\mathbf{e}_{x,b}]_a + v_y^{(b)}[\mathbf{e}_{y,b}]_a + v_z^{(b)}[\mathbf{e}_{z,b}]_a.$$

Using the equivalent superscript notation gives

$${}^a\mathbf{v} = v_x^{(b)} {}^a\mathbf{e}_{x,b} + v_y^{(b)} {}^a\mathbf{e}_{y,b} + v_z^{(b)} {}^a\mathbf{e}_{z,b}.$$

Writing this linear combination as matrix-vector multiplication gives

$${}^a\mathbf{v} = \underbrace{\begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} & {}^a\mathbf{e}_{z,b} \end{bmatrix}}_{\mathbf{R}_{ab}} \underbrace{\begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \\ v_z^{(b)} \end{bmatrix}}_{{}^b\mathbf{v}}.$$

Therefore,

$$\boxed{{}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}}.$$

The subscript order can be read as:

$\mathbf{R}_{ab}$ maps coordinates **from frame $\{b\}$ to frame $\{a\}$** .

The [linear algebra foundations chapter](#) develops orthogonal matrices, determinants, and inverses in general. Applying those results to the orthonormal, right-handed columns of $\mathbf{R}_{ab}$ gives:

$$\boxed{\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \mathbf{I}_3, \quad \det(\mathbf{R}_{ab}) = 1, \quad \mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top = \mathbf{R}_{ba}}.$$

The following derivations explain each property.

2.2.2.1 Orthonormal columns

Let $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbb{R}^3$ denote the columns of $\mathbf{R}_{ab}$:

$$\mathbf{R}_{ab} = [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{r}_3],$$

where $\mathbf{r}_1 = {}^a\mathbf{e}_{x,b}$, $\mathbf{r}_2 = {}^a\mathbf{e}_{y,b}$, and $\mathbf{r}_3 = {}^a\mathbf{e}_{z,b}$. Multiplying the transpose by the original matrix forms every pairwise dot product between these columns:

$$\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \begin{bmatrix} \mathbf{r}_1^\top \mathbf{r}_1 & \mathbf{r}_1^\top \mathbf{r}_2 & \mathbf{r}_1^\top \mathbf{r}_3 \\ \mathbf{r}_2^\top \mathbf{r}_1 & \mathbf{r}_2^\top \mathbf{r}_2 & \mathbf{r}_2^\top \mathbf{r}_3 \\ \mathbf{r}_3^\top \mathbf{r}_1 & \mathbf{r}_3^\top \mathbf{r}_2 & \mathbf{r}_3^\top \mathbf{r}_3 \end{bmatrix}.$$

Because the frame axes are orthonormal, every column has unit length,

$$\mathbf{r}_i^\top \mathbf{r}_i = 1,$$

and distinct columns are perpendicular,

$$\mathbf{r}_i^\top \mathbf{r}_j = 0 \quad \text{for } i \neq j,$$

where $i, j \in \{1, 2, 3\}$ are column indices. Therefore,

$$\boxed{\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I}_3.}$$

This identity means that $\mathbf{R}_{ab}$ preserves dot products and therefore preserves vector lengths and angles.

2.2.2.2 Inverse and reverse coordinate mapping

The inverse $\mathbf{R}_{ab}^{-1} \in \mathbb{R}^{3 \times 3}$ is defined by

$$\mathbf{R}_{ab}^{-1} \mathbf{R}_{ab} = \mathbf{I}_3.$$

Because $\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \mathbf{I}_3$, the transpose satisfies the defining property of the inverse. Hence,

$$\boxed{\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^{\top}}.$$

The coordinate transformation from frame $\{b\}$ to frame $\{a\}$ is

$${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}.$$

Multiplying by the inverse gives

$${}^b\mathbf{v} = \mathbf{R}_{ab}^{-1} {}^a\mathbf{v}.$$

Define $\mathbf{R}_{ba} \in \mathbb{R}^{3 \times 3}$ as the matrix that maps coordinates in the reverse direction, from frame $\{a\}$ to frame $\{b\}$. By definition,

$${}^b\mathbf{v} = \mathbf{R}_{ba} {}^a\mathbf{v}.$$

The two reverse mappings must be the same, so

$$\boxed{\mathbf{R}_{ba} = \mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^{\top}}.$$

2.2.2.3 Determinant and orientation

The notation $\det(\mathbf{R})$ denotes the **determinant** of a square matrix $\mathbf{R}$. Geometrically, $|\det(\mathbf{R})|$ is the volume-scaling factor of the corresponding linear transformation. For an orthogonal matrix $\mathbf{R}$, taking determinants of $\mathbf{R}^{\top}\mathbf{R} = \mathbf{I}_3$ gives

$$\det(\mathbf{R}^{\top}\mathbf{R}) = \det(\mathbf{I}_3),$$

and therefore by $\det(\mathbf{R}^{\top}\mathbf{R}) = \det(\mathbf{R}^{\top})\det(\mathbf{R})$ and $\det(\mathbf{R}^{\top}) = \det(\mathbf{R})$ gives

$$\det(\mathbf{R})^2 = 1.$$

Thus, an orthogonal matrix can have determinant $+1$ or -1 . Here, **orientation** means the handedness of the ordered coordinate axes. A right-handed frame satisfies

$$\mathbf{e}_x \times \mathbf{e}_y = \mathbf{e}_z.$$

The determinant sign has the following interpretation:

- $\det(\mathbf{R}) = +1$ preserves handedness, so a right-handed ordered basis remains right-handed.

- $\det(\mathbf{R}) = -1$ reverses handedness, so the transformation contains a reflection and produces a left-handed ordered basis from a right-handed one.

For example, consider

$$\mathbf{M} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The matrix $\mathbf{M} \in \mathbb{R}^{3 \times 3}$ reflects the x -coordinate. Its columns remain orthonormal, so $\mathbf{M}^T \mathbf{M} = \mathbf{I}_3$, but $\det(\mathbf{M}) = -1$. It reverses handedness and is therefore a reflection rather than a rigid-body rotation.

Because frames $\{a\}$ and $\{b\}$ are both right-handed, their rotation matrix must satisfy

$$\boxed{\mathbf{R}_{ab}^T \mathbf{R}_{ab} = \mathbf{I}_3 \quad \text{and} \quad \det(\mathbf{R}_{ab}) = 1}.$$

The determinant sign does not indicate whether a rotation is clockwise or counter-clockwise. It indicates whether the transformation preserves or reverses the handedness of space.

2.2.3 Points require translation

Let O_a and O_b be the origins of frames $\{a\}$ and $\{b\}$. Define ${}^a\mathbf{p}_b \in \mathbb{R}^3$ as the position of O_b relative to O_a , expressed in frame $\{a\}$ and measured in metres.

For a physical point P , let ${}^b\mathbf{p}_P \in \mathbb{R}^3$ be the position of P relative to O_b , expressed in frame $\{b\}$, and let ${}^a\mathbf{p}_P \in \mathbb{R}^3$ be the position of P relative to O_a , expressed in frame $\{a\}$. Both position columns are measured in metres. Their relationship is

$${}^a\mathbf{p}_P = \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b.$$

The rotation $\mathbf{R}_{ab} {}^b\mathbf{p}_P$ changes the basis used to express the displacement from O_b to P . The translation ${}^a\mathbf{p}_b$ then accounts for the displacement from O_a to O_b . A homogeneous transformation combines these two operations into one matrix equation.

2.3 Worked frame-change example

Consider two planar frames $\{a\}$ and $\{b\}$ whose axes differ by an angle $\theta \in \mathbb{R}$, measured in radians. A positive θ means that frame $\{b\}$ is rotated counter-clockwise relative to frame $\{a\}$. Define $\mathbf{R}_{ab}(\theta) \in \mathbb{R}^{2 \times 2}$ as the rotation matrix that maps two-dimensional coordinate columns from frame $\{b\}$ to frame $\{a\}$:

$${}^a\mathbf{v} = \mathbf{R}_{ab}(\theta) {}^b\mathbf{v},$$

where $\mathbf{v}$ is a geometric free vector and ${}^a\mathbf{v}, {}^b\mathbf{v} \in \mathbb{R}^2$ are its coordinate columns in frames $\{a\}$ and $\{b\}$.

Let $\mathbf{e}_{x,a}$ and $\mathbf{e}_{y,a}$ be the geometric unit axes of frame $\{a\}$, and let $\mathbf{e}_{x,b}$ and $\mathbf{e}_{y,b}$ be the geometric unit axes of frame $\{b\}$. By definition, the columns of $\mathbf{R}_{ab}$ are the axes of frame $\{b\}$ expressed in frame $\{a\}$:

$$\mathbf{R}_{ab} = \begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} \end{bmatrix}.$$

The x_b -axis makes angle θ with the x_a -axis. Its projection onto $\mathbf{e}_{x,a}$ is $\cos \theta$, and its projection onto $\mathbf{e}_{y,a}$ is $\sin \theta$. Therefore,

$${}^a\mathbf{e}_{x,b} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}.$$

Because the planar frames are right-handed and orthonormal, the y_b -axis is obtained by rotating the x_b -axis counter-clockwise by $\pi/2$. Its angle measured from the x_a -axis is therefore $\theta + \pi/2$, so

$${}^a\mathbf{e}_{y,b} = \begin{bmatrix} \cos(\theta + \pi/2) \\ \sin(\theta + \pi/2) \end{bmatrix}.$$

Using

$$\cos(\theta + \pi/2) = -\sin \theta, \quad \sin(\theta + \pi/2) = \cos \theta,$$

gives

$${}^a\mathbf{e}_{y,b} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}.$$

Placing these two coordinate columns side by side produces

$$\mathbf{R}_{ab}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

The column construction also explains why this matrix maps coordinates from frame $\{b\}$ to frame $\{a\}$. If

$${}^b\mathbf{v} = \begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \end{bmatrix},$$

where $v_x^{(b)}, v_y^{(b)} \in \mathbb{R}$ are the components of $\mathbf{v}$ along the x_b - and y_b -axes, then

$$\begin{aligned} {}^a\mathbf{v} &= v_x^{(b)} {}^a\mathbf{e}_{x,b} + v_y^{(b)} {}^a\mathbf{e}_{y,b} \\ &= \begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} \end{bmatrix} \begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \end{bmatrix} \\ &= \mathbf{R}_{ab}(\theta) {}^b\mathbf{v}. \end{aligned}$$

For the worked example, $\theta = \pi/2$ rad. Substitution gives

$$\mathbf{R}_{ab}(\pi/2) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

The first column $[0 \ 1]^\top$ says that the positive x_b -axis points along the positive y_a -axis. The second column $[-1 \ 0]^\top$ says that the positive y_b -axis points along the negative x_a -axis.

Let ${}^b\mathbf{v} \in \mathbb{R}^2$ be the coordinate column of a free vector $\mathbf{v}$, expressed in frame $\{b\}$ and measured in metres:

$${}^b\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \text{ m.}$$

The coordinate column of the same geometric vector in frame $\{a\}$ is ${}^a\mathbf{v} \in \mathbb{R}^2$:

$${}^a\mathbf{v} = \mathbf{R}_{ab}(\theta) {}^b\mathbf{v} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix} \text{ m.}$$

The coordinates changed, but the physical length did not:

$$\|{}^b\mathbf{v}\|_2 = \sqrt{2^2 + 1^2} = \sqrt{5} = \sqrt{(-1)^2 + 2^2} = \|{}^a\mathbf{v}\|_2.$$

2.3.1 Interactive frame-rotation graph

Fix the frame- $\{b\}$ coordinate column

$${}^b\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

For each angle $\theta \in [-\pi, \pi]$, the corresponding frame- $\{a\}$ coordinate column is

$${}^a\mathbf{v}(\theta) = \mathbf{R}_{ab}(\theta) {}^b\mathbf{v} = \begin{bmatrix} 2 \cos \theta - \sin \theta \\ 2 \sin \theta + \cos \theta \end{bmatrix}.$$

In the HTML book, move the angle slider to rotate frame $\{b\}$ relative to frame $\{a\}$. The graph keeps ${}^b\mathbf{v} = [2 \ 1]^\top$ fixed, so the geometric vector rotates with frame $\{b\}$ and its coordinates ${}^a\mathbf{v}(\theta)$ update continuously.

The PDF shows the default slider state $\theta = \pi/2$ rad. Use the HTML book to vary θ interactively.

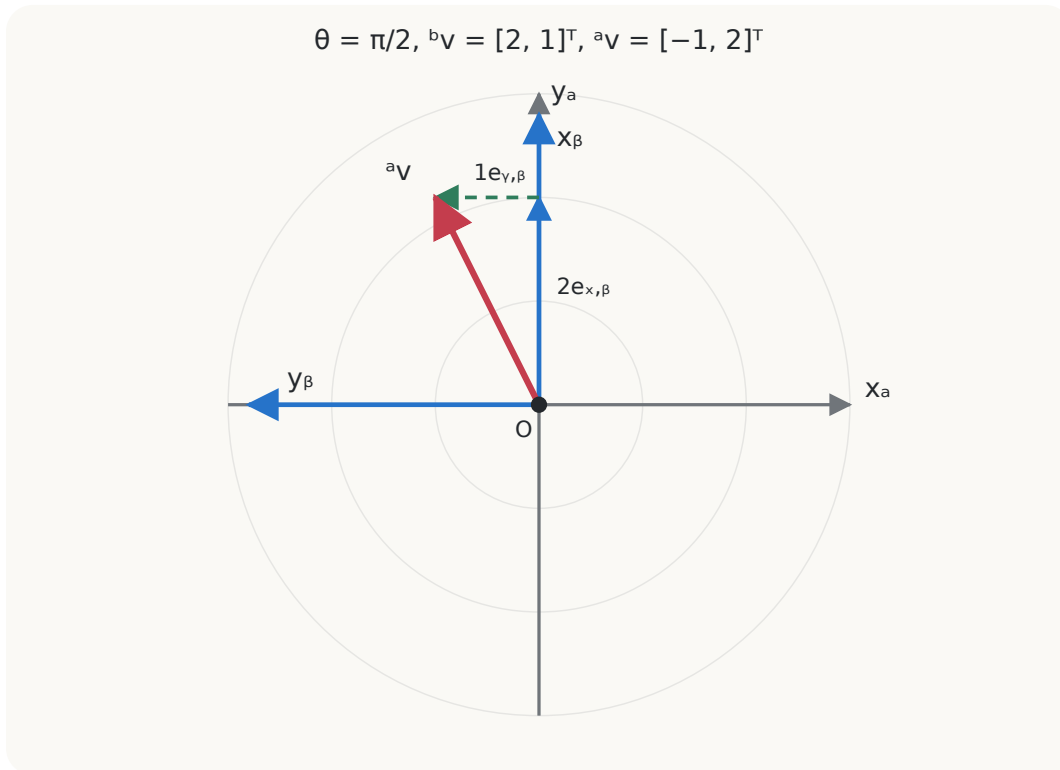


Figure 2.2: Static PDF view of the frame rotation at $\theta = \pi/2$ with ${}^b\mathbf{v} = [2 \ 1]^\top$ and ${}^a\mathbf{v} = [-1 \ 2]^\top$.

2.3.1.1 Equivalent NumPy calculation

The following NumPy snippet is the direct software translation of ${}^a\mathbf{v} = \mathbf{R}_{ab}(\theta){}^b\mathbf{v}$. The scalar `theta` stores θ in radians; `R_ab` is a NumPy array with shape `(2, 2)` representing $\mathbf{R}_{ab}(\theta) \in \mathbb{R}^{2 \times 2}$; `v_b` is a NumPy array with shape `(2,)` representing the fixed coordinate column ${}^b\mathbf{v} = [2 \ 1]^\top$; and `v_a` is a NumPy array with shape `(2,)` representing ${}^a\mathbf{v} \in \mathbb{R}^2$. The operator `@` performs matrix-vector multiplication.

```
import numpy as np

theta = np.pi / 2 # radians; replace this value with the selected angle

R_ab = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta),  np.cos(theta)],
])

v_b = np.array([2.0, 1.0])
v_a = R_ab @ v_b
```

For $\theta = \pi/2$ rad, the calculation gives `v_a` approximately equal to `[-1.0, 2.0]`. Changing `theta` recomputes the same values displayed by the interactive graph while `v_b` remains fixed.

2.4 Differential-drive interpretation

For a planar mobile robot, let frame $\{w\}$ be fixed to the world and frame $\{b\}$ be attached to the robot. Let $\theta \in \mathbb{R}$ be the robot's heading angle, measured counter-clockwise in radians from the positive x_w -axis to the positive x_b -axis. Let $v \in \mathbb{R}$ be the signed forward speed, measured in m s^{-1} ; positive v means motion along the positive x_b -axis.

The body-frame velocity ${}^b\mathbf{v} \in \mathbb{R}^2$ is

$${}^b\mathbf{v} = \begin{bmatrix} v \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Define $\mathbf{R}_{wb}(\theta) \in \mathbb{R}^{2 \times 2}$ as the rotation matrix that maps planar coordinates from the robot frame $\{b\}$ to the world frame $\{w\}$. The world-frame velocity ${}^w\mathbf{v} \in \mathbb{R}^2$ is therefore

$${}^w\mathbf{v} = \mathbf{R}_{wb}(\theta) {}^b\mathbf{v} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix} \text{ m s}^{-1}.$$

This equation is the coordinate-frame foundation of the differential-drive kinematic model. It shows why a controller must distinguish a forward command in the robot frame from motion observed in the world frame.

2.5 Failure modes and engineering checks

2.5.1 Missing frame labels

Two identical columns can describe different physical directions when they are expressed in different frames. Interfaces should name the frame explicitly.

2.5.2 Reversed transformation

Using $\mathbf{R}_{ba}$ where $\mathbf{R}_{ab}$ is required rotates coordinates in the opposite direction. Check the subscript rule before multiplying.

2.5.3 Mixed units

Degrees passed to a function expecting radians produce valid-looking but wrong numbers. Convert at the system boundary and use radians internally.

2.5.4 Point-vector confusion

A point changes under translation; a free vector does not. Applying a translation to velocity or direction data is a modelling error.

2.5.5 Invalid numerical rotation

Let $\mathbf{R} \in \mathbb{R}^{n \times n}$ be a numerically stored matrix intended to represent a rotation, and let $\mathbf{I}_n \in \mathbb{R}^{n \times n}$ be the identity matrix. Rounding, integration drift, or corrupted data can make $\mathbf{R}$ non-orthonormal.

For a matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ with entries m_{ij} , define the Frobenius norm by

$$\|\mathbf{M}\|_F := \sqrt{\sum_{i=1}^n \sum_{j=1}^n m_{ij}^2}.$$

Let $\varepsilon_{\text{orth}} > 0$ and $\varepsilon_{\text{det}} > 0$ be declared numerical tolerances. A robust interface checks

$$\|\mathbf{R}^T \mathbf{R} - \mathbf{I}_n\|_F \leq \varepsilon_{\text{orth}}$$

and

$$|\det(\mathbf{R}) - 1| \leq \varepsilon_{\text{det}}.$$

The first condition tests approximate orthonormality; the second tests approximate preservation of right-handed orientation.

Quick test B: coordinate frames and rotations

1. **Recall:** What do the columns of $\mathbf{R}_{ab}$ represent?
2. **Explanation:** Explain in words why $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ba}$.
3. **Derivation:** Starting with ${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}$, solve for ${}^b\mathbf{v}$.
4. **Application:** A robot has heading $\theta = \pi/2$ and forward speed $v = 0.5 \text{ m s}^{-1}$. Find its world-frame velocity.
5. **Limitations:** Give two ways a numerically stored matrix intended to be a rotation matrix can become invalid.
6. **Derivation:** Let $\{a\}$ and $\{b\}$ be right-handed orthonormal planar frames. Frame $\{b\}$ is rotated counter-clockwise by θ rad relative to frame $\{a\}$, and $\mathbf{R}_{ab}(\theta) \in \mathbb{R}^{2 \times 2}$ maps coordinate columns from frame $\{b\}$ to frame $\{a\}$. First express the x_b -axis and y_b -axis as coordinate columns ${}^a\mathbf{e}_{x,b}, {}^a\mathbf{e}_{y,b} \in \mathbb{R}^2$; then place those columns side by side to derive $\mathbf{R}_{ab}(\theta)$.

Answers and hints

1. The axes of frame $\{b\}$ expressed in coordinates of frame $\{a\}$.
2. Reversing the frame change undoes the original change; for an orthonormal rotation matrix, the inverse is its transpose.
3. Premultiply by $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ba}$: ${}^b\mathbf{v} = \mathbf{R}_{ba} {}^a\mathbf{v}$.
4. $[0 \ 0.5]^T \text{ m s}^{-1}$, up to floating-point rounding.
5. Examples include accumulated integration error, rounded entries, malformed input, unit mistakes in angle construction, and data corruption.

6. Since the x_b -axis makes angle θ with the x_a -axis, its frame- $\{a\}$ coordinate column is ${}^a\mathbf{e}_{x,b} = [\cos \theta \ \sin \theta]^\top$. Right-handed orthonormality places the y_b -axis at angle $\theta + \pi/2$, so ${}^a\mathbf{e}_{y,b} = [\cos(\theta + \pi/2) \ \sin(\theta + \pi/2)]^\top = [-\sin \theta \ \cos \theta]^\top$. Because these two coordinate columns are the columns of $\mathbf{R}_{ab}(\theta)$,

$$\mathbf{R}_{ab}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

Cumulative chapter test

1. **Recall and meaning:** Define a three-dimensional coordinate frame $\{a\}$. Then state what the columns of $\mathbf{R}_{ab} \in \mathbb{R}^{3 \times 3}$ represent and the direction in which this matrix maps coordinate columns.
2. **Derivation, application, and explanation:** Frame $\{b\}$ is rotated counter-clockwise by $\theta = \pi/2$ rad relative to frame $\{a\}$. Derive $\mathbf{R}_{ab}(\pi/2)$ by expressing the three axes of frame $\{b\}$ in frame $\{a\}$. Then let

$${}^b\mathbf{v} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} \text{ m.}$$

Compute ${}^a\mathbf{v}$. Which physical property of the geometric free vector is unchanged, and why?

3. **Point–vector contrast:** Continue from Question 2: use the same frames, with frame $\{b\}$ rotated counter-clockwise by $\theta = \pi/2$ rad relative to frame $\{a\}$, and use the rotation matrix $\mathbf{R}_{ab}(\pi/2)$ derived there. Now let the position of O_b relative to O_a , expressed in frame $\{a\}$, be

$${}^a\mathbf{p}_b = \begin{bmatrix} 3 \\ -1 \\ 0 \end{bmatrix} \text{ m,}$$

and let a physical point P have ${}^b\mathbf{p}_P = [2 \ 1 \ 0]^\top$ m. Compute ${}^a\mathbf{p}_P$. Then compute the frame- $\{a\}$ coordinates of a free displacement vector whose frame- $\{b\}$ coordinates are also $[2 \ 1 \ 0]^\top$ m. Explain why the two calculations differ.

4. **Derivation:** Starting from $\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \mathbf{I}_3$, derive $\mathbf{R}_{ba} = \mathbf{R}_{ab}^\top$. Then prove that changing the coordinate representation of a free vector preserves its Euclidean norm. Finally, show that applying the same point transformation to two physical points P and Q preserves the distance between them.

5. **Judgement:** Consider

$$\mathbf{M} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Determine whether $\mathbf{M}$ is a valid rotation matrix. State which numerical rotation checks it passes or fails and explain the physical meaning of the failure.

6. **Rigid-body application:** At one instant, all of the following coordinate columns are expressed in the same frame $\{a\}$:

$${}^a\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \text{ rad s}^{-1}, \quad {}^a\mathbf{r}_{OP} = \begin{bmatrix} 0.5 \\ -0.25 \\ 0 \end{bmatrix} \text{ m}, \quad {}^a\mathbf{v}_O = \begin{bmatrix} 0.2 \\ -0.1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Compute ${}^a\mathbf{v}_{P/O}$ and ${}^a\mathbf{v}_P$. Verify by a dot product which velocity contribution must be perpendicular to ${}^a\mathbf{r}_{OP}$, and state why the same conclusion need not hold for the total velocity.

7. **Transfer to differential drive:** A planar robot has heading $\theta = -\pi/2$ rad and signed forward speed $v = 0.6 \text{ m s}^{-1}$. Compute its world-frame velocity ${}^w\mathbf{v}$. Then use the reverse transformation to recover the original body-frame velocity.
8. **Failure diagnosis:** A software interface documents $\mathbf{R}_{ab}$ as mapping coordinate columns from frame $\{b\}$ to frame $\{a\}$, but its implementation computes `R_ab.transpose() * v_b` and labels the result as frame $\{a\}$. Explain the semantic error. Give one simple basis-axis test that would expose it.
9. **Contrast two situations:** Explain the difference between these statements: (a) one unchanged geometric vector is represented by ${}^b\mathbf{v}$ and ${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}$; and (b) the coordinate column ${}^b\mathbf{v}$ is held fixed while frame $\{b\}$ physically rotates relative to frame $\{a\}$. What changes physically in each situation?
10. **Limitations and engineering checks:** Name at least four declarations or validation checks required at a coordinate-transformation interface. Then name one real-world source of error that valid frame algebra alone cannot remove.

Cumulative answers and hints

1. A frame $\{a\}$ consists of an origin O_a and three right-handed orthonormal basis vectors $\mathbf{e}_{x,a}$, $\mathbf{e}_{y,a}$, and $\mathbf{e}_{z,a}$. The columns of $\mathbf{R}_{ab}$ are the axes of frame $\{b\}$ expressed in frame $\{a\}$, and $\mathbf{R}_{ab}$ maps coordinate columns from frame $\{b\}$ to frame $\{a\}$.

2. At $\theta = \pi/2$, the frame- $\{b\}$ axes expressed in frame $\{a\}$ are

$${}^a\mathbf{e}_{x,b} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad {}^a\mathbf{e}_{y,b} = \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix}, \quad {}^a\mathbf{e}_{z,b} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Placing these axes in the columns gives

$$\mathbf{R}_{ab}(\pi/2) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The coordinate change is

$${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v} = \begin{bmatrix} -1 \\ 2 \\ 0 \end{bmatrix} \text{ m.}$$

The geometric vector and its length are unchanged; only its coordinate representation changes. Its norm remains $\sqrt{5}$ m because a rotation matrix is orthogonal.

3. A point requires both rotation and translation:

$${}^a\mathbf{p}_P = \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b = \begin{bmatrix} -1 \\ 2 \\ 0 \end{bmatrix} + \begin{bmatrix} 3 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} \text{ m.}$$

A free displacement vector is unaffected by the origin separation, so

$${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v} = \begin{bmatrix} -1 \\ 2 \\ 0 \end{bmatrix} \text{ m.}$$

A point is located relative to a frame origin, whereas a free vector has direction and magnitude but no fixed origin.

4. Since $\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \mathbf{I}_3$, the transpose satisfies the defining equation for the inverse, so $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top$. The reverse coordinate map is the inverse of the forward map, hence $\mathbf{R}_{ba} = \mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top$. For ${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}$,

$$\begin{aligned}
\|{}^a\mathbf{v}\|_2^2 &= ({}^a\mathbf{v})^\top {}^a\mathbf{v} \\
&= ({}^b\mathbf{v})^\top \mathbf{R}_{ab}^\top \mathbf{R}_{ab} {}^b\mathbf{v} \\
&= ({}^b\mathbf{v})^\top {}^b\mathbf{v} \\
&= \|{}^b\mathbf{v}\|_2^2.
\end{aligned}$$

Norms are nonnegative, so equality of the squared norms implies equality of the norms. For two points transformed using the same $\mathbf{R}_{ab}$ and ${}^a\mathbf{p}_b$, subtracting their frame- $\{a\}$ coordinates cancels the translation:

$$\begin{aligned}
{}^a\mathbf{p}_P - {}^a\mathbf{p}_Q &= (\mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b) - (\mathbf{R}_{ab} {}^b\mathbf{p}_Q + {}^a\mathbf{p}_b) \\
&= \mathbf{R}_{ab} ({}^b\mathbf{p}_P - {}^b\mathbf{p}_Q).
\end{aligned}$$

This difference is a free displacement vector, so the norm-preservation result gives

$$\|{}^a\mathbf{p}_P - {}^a\mathbf{p}_Q\|_2 = \|{}^b\mathbf{p}_P - {}^b\mathbf{p}_Q\|_2.$$

5. The matrix passes the orthonormality check because $\mathbf{M}^\top \mathbf{M} = \mathbf{I}_3$, but it fails the orientation check because $\det(\mathbf{M}) = -1$. It is a reflection across the yz -plane, not a right-handed rigid-body rotation. This is why an orthonormality check alone is insufficient.
6. The rotational contribution is

$${}^a\mathbf{v}_{P/O} = {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP} = \begin{bmatrix} 0.5 \\ 1.0 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Therefore,

$${}^a\mathbf{v}_P = {}^a\mathbf{v}_O + {}^a\mathbf{v}_{P/O} = \begin{bmatrix} 0.7 \\ 0.9 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

The required orthogonality check is

$$({}^a\mathbf{r}_{OP})^\top {}^a\mathbf{v}_{P/O} = 0.5(0.5) - 0.25(1.0) = 0.$$

The total velocity also contains the independent translation ${}^a\mathbf{v}_O$, so it need not be perpendicular to ${}^a\mathbf{r}_{OP}$; here their dot product is $0.125 \text{ m}^2 \text{ s}^{-1}$.

7. Since

$$\mathbf{R}_{wb}(-\pi/2) = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix},$$

the world-frame velocity is

$${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v} = \begin{bmatrix} 0 \\ -0.6 \end{bmatrix} \text{ m s}^{-1}.$$

The reverse map is $\mathbf{R}_{bw} = \mathbf{R}_{wb}^\top$ and

$${}^b\mathbf{v} = \mathbf{R}_{bw} {}^w\mathbf{v} = \begin{bmatrix} 0.6 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

8. The transpose is $\mathbf{R}_{ab}^\top = \mathbf{R}_{ba}$, whose declared input is a frame- $\{a\}$ coordinate column and whose output is a frame- $\{b\}$ coordinate column. Applying it to ${}^b\mathbf{v}$ violates the frame contract, even though the matrix dimensions agree, and the result cannot validly be labelled ${}^a\mathbf{v}$. A basis-axis test can provide ${}^b\mathbf{e}_{x,b} = [1 \ 0 \ 0]^\top$ and require the output to equal the first column of $\mathbf{R}_{ab}$; the transposed implementation generally fails this test.
9. In situation (a), the geometric vector is unchanged and only its numerical description changes with the coordinate basis. In situation (b), the components relative to frame $\{b\}$ stay fixed while the basis vectors of $\{b\}$ turn relative to frame $\{a\}$, so the geometric vector constructed from those fixed components rotates physically with frame $\{b\}$. The same matrix entries can participate in both calculations, but the physical operation and the meanings of the input and output are different.
10. Suitable declarations and checks include source and destination frame identifiers, point-versus-free-vector semantics, SI units and angle units, matrix and vector dimensions, finite inputs, the intended multiplication direction, approximate orthonormality, determinant near $+1$, and declared numerical tolerances. Valid algebra cannot remove effects such as sensor noise, calibration error, structural compliance, timing error, or model mismatch.

References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics*, Chapter 3, especially Sections 3.1 and 3.2.1 on free vectors, frames, rotation matrices, and rigid-body motions.
- Steven M. LaValle, *Planning Algorithms*, Chapters 3 and 4 on geometric representations, transformations, and configuration spaces.

3 Differential-Drive Kinematics and Numerical Integration: Model and Derivation

Why this chapter matters

Kinematics describes motion without asking which forces or torques produce it. For a differential-drive robot, kinematics connects wheel angular speeds to the time evolution of position and heading. Numerical integration converts this continuous-time model into sampled updates used by simulation, odometry, planning, and control.

This chapter develops that chain in four steps: define planar pose and its derivative, derive the no-sideways-motion model, derive body velocity from wheel speeds, and compare exact constant-input motion with Forward Euler integration.

Prerequisites and assumptions

The reader should understand vectors and matrices from Chapter 1, coordinate frames and rotations from Chapter 2, and elementary derivatives, integrals, sine, and cosine.

The system considered here is a differential-drive ground robot: a rigid chassis propelled by left and right drive wheels that are mounted on a common axle and can be driven independently. The robot operates on a stationary, level, rigid floor, and the model describes its motion as viewed from above. Any additional support hardware does not affect the ideal kinematic model.

Frame $\{w\}$ is fixed to the floor and is called the world frame. Frame $\{b\}$ is rigidly attached to the robot and is called the body frame. Its origin O_b lies midway between the two drive-wheel ground-contact centre lines. The positive x_b -axis points forward, the positive y_b -axis points left, and the positive z_b -axis points upward, so frame $\{b\}$ is right-handed. As the robot moves, frame $\{b\}$ translates and rotates relative to the stationary frame $\{w\}$.

Both drive wheels have the same effective rolling radius $r > 0$, measured in metres. The track width $L > 0$, measured in metres, is the lateral distance between the left and right wheel ground-contact centre lines.

The kinematic model uses the following assumptions:

- The chassis remains rigid, and its pose changes only through horizontal position and rotation about the upward z_b -axis; its height, roll, and pitch remain unchanged.
- Each wheel maintains contact with the floor and rolls without longitudinal slip, so the distance travelled in the wheel's rolling direction equals the arc length swept out at its effective radius.
- The scalar body-frame lateral component ${}^b v_y$, also written v_y when the frame is clear, is measured in m s^{-1} and lies along the $+y_b$ -direction. The wheel constraints impose ${}^b v_y = 0$ at O_b . The robot can still turn because its forward direction changes continuously.
- The effective wheel radius r and track width L remain constant.
- Wheel deformation, tyre slip, backlash, encoder error, actuator dynamics, collisions, and uneven or moving terrain are ignored.
- Length is measured in metres, time in seconds, and angle in radians; corresponding velocities are measured in m s^{-1} and rad s^{-1} .

Together, these assumptions define an ideal planar kinematic model rather than a complete physical model. A real robot violates them to some degree, especially during rapid acceleration, braking, collision, wheel slip, or motion over compliant or uneven terrain.

Figure Figure 3.1 summarizes the frames, wheel dimensions, rolling condition, and lateral-velocity constraint used by the model.

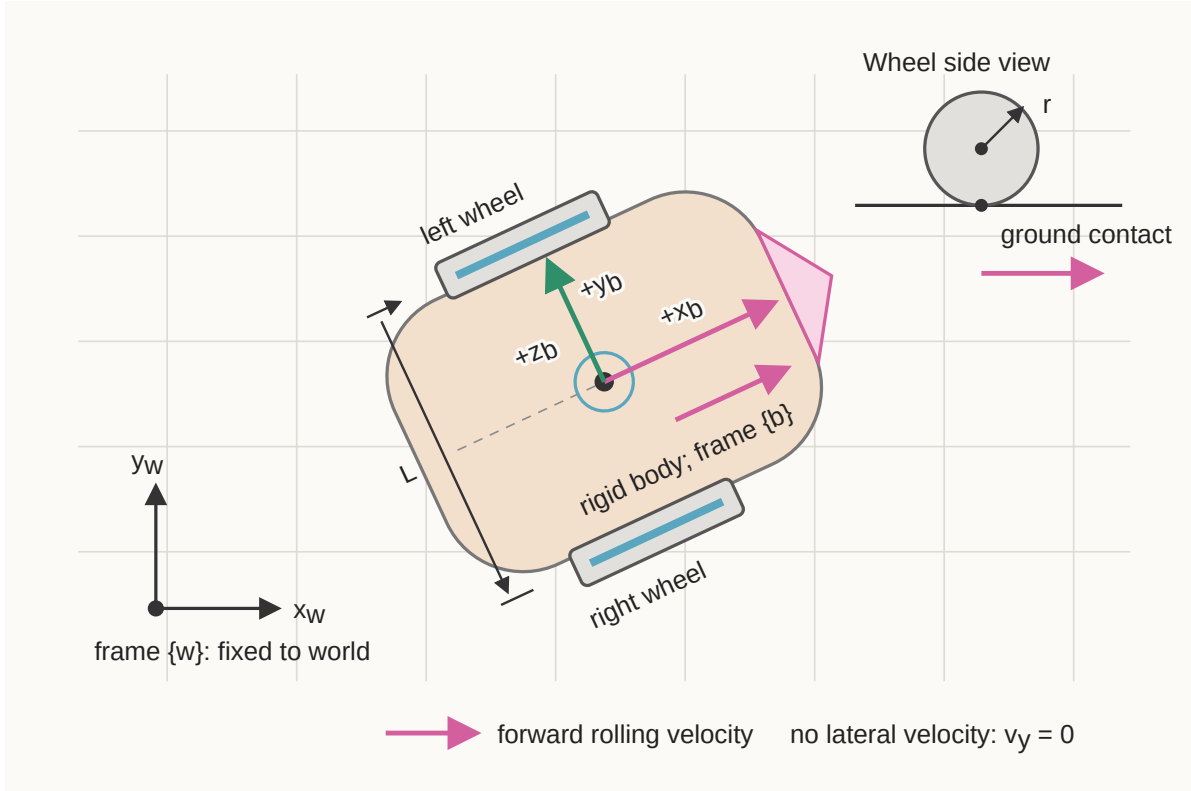


Figure 3.1: Differential-drive robot geometry and ideal rolling constraints.

As a preview of the pure-rolling relation, $v_i = r\dot{\phi}_i$ for $i \in \{L, R\}$, where v_i is the signed

longitudinal ground speed of wheel i and $\dot{\phi}_i$ is its signed angular speed. Section 3.2.1 derives how the individual wheel speeds determine the body-forward speed v and yaw rate ω .

3.1 Planar pose and its derivative

Let $t \in \mathbb{R}_{\geq 0}$ denote continuous time, measured in seconds. Define the planar robot pose by

$$\mathbf{q}(t) := \begin{bmatrix} x(t) \\ y(t) \\ \theta(t) \end{bmatrix}.$$

Here, $x(t), y(t) \in \mathbb{R}$ are the world-frame coordinates of body-frame origin O_b , measured in metres, and $\theta(t) \in \mathbb{R}$ is the counter-clockwise heading of frame $\{b\}$ relative to frame $\{w\}$, measured in radians. Angles that differ by an integer multiple of 2π represent the same physical heading.

The pose derivative is

$$\dot{\mathbf{q}}(t) := \frac{d\mathbf{q}(t)}{dt} = \begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \\ \dot{\theta}(t) \end{bmatrix}.$$

The components $\dot{x}$ and $\dot{y}$ are measured in m s^{-1} , while $\dot{\theta}$ is measured in rad s^{-1} . Although they appear in one column, their units are not identical.

Define the body-velocity input by

$$\mathbf{u}(t) := \begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix},$$

where $v(t) \in \mathbb{R}$ is signed forward speed along $+x_b$, measured in m s^{-1} , and $\omega(t) \in \mathbb{R}$ is signed yaw rate about $+z_b$, measured in rad s^{-1} . Positive ω produces counter-clockwise rotation when viewed from above.

3.1.1 Deriving the planar model

Because the ideal robot has no body-frame lateral velocity, its translational velocity expressed in frame $\{b\}$ is

$${}^b\mathbf{v} = \begin{bmatrix} v \\ 0 \end{bmatrix} \in \mathbb{R}^2.$$

The rotation matrix $\mathbf{R}_{wb}(\theta) \in \mathbb{R}^{2 \times 2}$ maps coordinates from frame $\{b\}$ to frame $\{w\}$. Therefore,

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}}_{\mathbf{R}_{wb}(\theta)} \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix}.$$

Together with $\dot{\theta} = \omega$, this gives

$$\dot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, \mathbf{u}) = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}.$$

Here, $\mathbf{f}$ is the kinematic state-derivative function: it maps current pose $\mathbf{q}$ and input $\mathbf{u}$ to instantaneous pose derivative $\dot{\mathbf{q}}$. The position rates depend on θ because the body forward direction changes relative to the world frame.

3.1.2 The no-sideways-motion constraint

Define the axle-midpoint linear velocity expressed in the world frame by

$${}^w\mathbf{v} := \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \in \mathbb{R}^2,$$

where $\dot{x}$ and $\dot{y}$ are its components along $+x_w$ and $+y_w$, respectively, and are measured in m s^{-1} .

The geometric unit vector along the body-frame $+y_b$ -axis, expressed using world-frame coordinates, is the second column of $\mathbf{R}_{wb}(\theta)$:

$${}^w\mathbf{e}_{y,b} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix} \in \mathbb{R}^2.$$

Both ${}^w\mathbf{e}_{y,b}$ and ${}^w\mathbf{v}$ are expressed in frame $\{w\}$, and ${}^w\mathbf{e}_{y,b}$ has unit length. Their dot product therefore gives the signed component of the velocity along the robot's lateral $+y_b$ -direction:

$$\begin{aligned} {}^bv_y &= ({}^w\mathbf{e}_{y,b})^\top {}^w\mathbf{v} \\ &= \begin{bmatrix} -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \\ &= -\dot{x} \sin \theta + \dot{y} \cos \theta. \end{aligned}$$

The left superscript in bv_y makes explicit that this scalar is the y -component of velocity measured along a body-frame axis, even though the projection was calculated using world-frame coordinates.

The matrix $\mathbf{R}_{wb}(\theta)$ maps coordinate columns from frame $\{b\}$ to frame $\{w\}$. Its columns are orthonormal, so its inverse equals its transpose. Define the reverse coordinate map by

$$\begin{aligned} \mathbf{R}_{bw}(\theta) &:= \mathbf{R}_{wb}(\theta)^{-1} \\ &= \mathbf{R}_{wb}(\theta)^\top \\ &= \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}. \end{aligned}$$

Therefore, $\mathbf{R}_{bw}(\theta)$ maps coordinate columns from frame $\{w\}$ to frame $\{b\}$. The notation uses the same heading θ because both matrices depend on the robot's heading, but the reverse map applies the opposite rotation: $\mathbf{R}_{bw}(\theta) = \mathbf{R}_{wb}(-\theta)$.

The complete change from world-frame velocity components to body-frame velocity components is

$${}^b\mathbf{v} = \begin{bmatrix} {}^bv_x \\ {}^bv_y \end{bmatrix} = \mathbf{R}_{bw}(\theta) {}^w\mathbf{v} = \mathbf{R}_{wb}(\theta)^\top {}^w\mathbf{v} = \begin{bmatrix} \dot{x} \cos \theta + \dot{y} \sin \theta \\ -\dot{x} \sin \theta + \dot{y} \cos \theta \end{bmatrix}.$$

Here, bv_x and bv_y are the signed forward and lateral velocity components along $+x_b$ and $+y_b$, respectively, and both are measured in m s^{-1} . Thus, $\dot{x}$ and $\dot{y}$ are world-frame components, whereas bv_x and bv_y are body-frame components.

The ideal no-sideways-motion assumption is therefore written most clearly as

$$\boxed{{}^bv_y = -\dot{x} \sin \theta + \dot{y} \cos \theta = 0}.$$

The signed forward speed v defined earlier equals bv_x , so the constrained body-frame velocity and its world-frame representation are

$${}^b\mathbf{v} = \begin{bmatrix} v \\ 0 \end{bmatrix}, \quad {}^w\mathbf{v} = \mathbf{R}_{wb}(\theta) \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix}.$$

Consequently, zero body-frame lateral velocity does not imply $\dot{y} = 0$. For example, when $\theta = \pi/4$, forward motion gives $\dot{x} = \dot{y} = v/\sqrt{2}$. The angle θ specifies orientation; it does not itself create translation. If $v = 0$ and $\omega \neq 0$, the robot rotates in place about O_b , so $\dot{x} = \dot{y} = 0$ while θ changes.

This equation is a nonholonomic velocity constraint. It restricts the directions in which the robot can move at the current pose, but it does not require the pose variables x , y , and θ to remain on a fixed lower-dimensional surface. In contrast, a holonomic condition such as $y = 0$ would confine every allowable pose to the world x_w -axis.

The distinction concerns instantaneous motion versus accumulated motion. At every instant, the axle midpoint translates only along the robot's current x_b -axis, but turning changes the direction of x_b relative to the world. For example, a robot initially aligned with $+x_w$ can rotate by $\pi/2$, drive forward a distance $d > 0$, and rotate back by $-\pi/2$. Its final heading equals its initial heading, yet it has acquired a world-frame $+y_w$ displacement of d . It never moved along its instantaneous y_b -axis; the net sideways displacement resulted from a sequence of allowed rotations and forward motions.

Quick test A: planar kinematics

1. Define every component of $\mathbf{q} = [x \ y \ \theta]^T$, including its frame and units.
2. Why do $\dot{x}$ and $\dot{y}$ depend on θ , while v does not?
3. Starting with ${}^b\mathbf{v} = [v \ 0]^T$, derive $\dot{y} = v \sin \theta$.
4. If $v = 1 \text{ m s}^{-1}$ and $\theta = \pi/2 \text{ rad}$, find $\dot{x}$ and $\dot{y}$.
5. Does the no-sideways-motion constraint prevent a net sideways displacement? Explain.

Answers and hints

1. x and y are the world-frame position coordinates of O_b in metres; θ is counter-clockwise body heading relative to the world frame in radians.
2. v is defined along the moving x_b -axis. Its world-frame components depend on the orientation of that axis.
3. The second row of $\mathbf{R}_{wb}(\theta)[v \ 0]^T$ is $v \sin \theta$.
4. $\dot{x} = 0 \text{ m s}^{-1}$ and $\dot{y} = 1 \text{ m s}^{-1}$.
5. No. The constraint forbids instantaneous sideways velocity, but combined forward and turning motions can produce a sideways displacement.

3.2 Differential-drive wheel kinematics

Let $\phi_L(t), \phi_R(t) \in \mathbb{R}$ be the rotational positions of the left and right wheels about their axles at time t , measured in radians. The subscripts L and R identify the left and right wheels. These are rolling angles, not steering angles: the wheel planes remain fixed relative to the chassis.

Choose $\phi_L = \phi_R = 0$ at an arbitrary reference position. A change of 2π radians is one complete wheel revolution. Although the marked orientation of a wheel repeats after each revolution, ϕ_L and ϕ_R are treated as unwrapped accumulated angles in $\mathbb{R}$, so they may increase beyond 2π or decrease below zero.

Their time derivatives $\dot{\phi}_L(t)$ and $\dot{\phi}_R(t)$ are the left and right wheel angular speeds, measured in rad s^{-1} . Positive angular speed is defined to roll the corresponding wheel and the robot forward.

For a wheel of effective radius r , let $\Delta\phi$ be a signed angular change and let Δs be the corresponding signed arc displacement swept out at the wheel rim. Both r and Δs are measured in metres. By the definition of radian measure,

$$\Delta\phi = \frac{\Delta s}{r}, \quad \frac{\text{m}}{\text{m}} = 1.$$

The metre units cancel, so a radian is dimensionless in the International System of Units (SI). Nevertheless, angles are labelled in rad and angular speeds in rad s^{-1} so their physical meanings remain clear. Rearranging the radian definition gives the swept arc displacement $\Delta s = r\Delta\phi$. Under rolling without slip, this swept arc displacement equals the signed longitudinal displacement along the ground.

For wheel $i \in \{L, R\}$, let $s_i(t)$ be its signed accumulated longitudinal ground displacement. Over a time interval $\Delta t > 0$, pure rolling gives $\Delta s_i = r\Delta\phi_i$. Because r is constant, dividing by Δt and taking the limit $\Delta t \rightarrow 0$ gives

$$v_i := \dot{s}_i = \lim_{\Delta t \rightarrow 0} \frac{\Delta s_i}{\Delta t} = r \lim_{\Delta t \rightarrow 0} \frac{\Delta\phi_i}{\Delta t} = r\dot{\phi}_i.$$

Applying this relation to the left and right wheels gives

$$v_L = r\dot{\phi}_L, \quad v_R = r\dot{\phi}_R,$$

where $v_L, v_R \in \mathbb{R}$ are measured in m s^{-1} . The unit calculation for either wheel is

$$\begin{aligned}
\text{metres} \times \frac{\text{radians}}{\text{second}} &= \text{m} \cdot \frac{\text{rad}}{\text{s}} \\
&= \text{m} \cdot \frac{1}{\text{s}} \\
&= \text{m s}^{-1}.
\end{aligned}$$

The symbol rad is retained to communicate that ϕ is an angle, even though it contributes no independent physical dimension in the unit calculation.

3.2.1 Relating wheel speeds to body motion

The rigid-body point-velocity relation derived in the [geometry and coordinate-frames chapter](#) is

$${}^b\mathbf{v}_P = {}^b\mathbf{v}_O + {}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{OP},$$

provided that every vector is expressed in the same orthonormal frame, here the body frame $\{b\}$. The vector ${}^b\mathbf{v}_P$ is the velocity of a rigid-body point P , ${}^b\mathbf{v}_O$ is the velocity of a reference point O , ${}^b\mathbf{r}_{OP}$ is the position of P relative to O , and ${}^b\boldsymbol{\omega}$ is the rigid body angular-velocity vector.

For the differential-drive chassis, choose the reference point as the axle midpoint $O = O_b$. Let C_L and C_R denote the left and right wheel centres on the axle. Both points are fixed relative to the chassis. Under ideal rolling without longitudinal slip, the $+x_b$ components of their translational velocities equal the longitudinal wheel speeds v_L and v_R defined above.

Because $+y_b$ points left, the wheel-centre position vectors are

$${}^b\mathbf{r}_{O_b C_L} = \begin{bmatrix} 0 \\ L/2 \\ 0 \end{bmatrix}, \quad {}^b\mathbf{r}_{O_b C_R} = \begin{bmatrix} 0 \\ -L/2 \\ 0 \end{bmatrix}, \quad {}^b\mathbf{r}_{O_b C_L}, {}^b\mathbf{r}_{O_b C_R} \in \mathbb{R}^3.$$

Their components have units of metres. The axle midpoint moves forward with scalar speed v , and the chassis yaws about $+z_b$ with scalar rate ω , so

$${}^b\mathbf{v}_{O_b} = \begin{bmatrix} v \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^3, \quad {}^b\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix} \in \mathbb{R}^3.$$

The components of ${}^b\mathbf{v}_{O_b}$ have units of m s^{-1} , and the components of ${}^b\boldsymbol{\omega}$ have units of rad s^{-1} .

Applying the rigid-body relation to each wheel-centre point gives

$${}^b\mathbf{v}_{C_L} = {}^b\mathbf{v}_{O_b} + {}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_L}, \quad {}^b\mathbf{v}_{C_R} = {}^b\mathbf{v}_{O_b} + {}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_R}.$$

For the left wheel, the rotational contribution is

$${}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_L} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix} \times \begin{bmatrix} 0 \\ L/2 \\ 0 \end{bmatrix} = \begin{bmatrix} -L\omega/2 \\ 0 \\ 0 \end{bmatrix}.$$

For the right wheel, it is

$${}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_R} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix} \times \begin{bmatrix} 0 \\ -L/2 \\ 0 \end{bmatrix} = \begin{bmatrix} L\omega/2 \\ 0 \\ 0 \end{bmatrix}.$$

Therefore, the wheel-centre velocity vectors are

$${}^b\mathbf{v}_{C_L} = \begin{bmatrix} v - L\omega/2 \\ 0 \\ 0 \end{bmatrix}, \quad {}^b\mathbf{v}_{C_R} = \begin{bmatrix} v + L\omega/2 \\ 0 \\ 0 \end{bmatrix}.$$

The longitudinal wheel speeds v_L and v_R are the $+x_b$ components of these vectors. Consequently,

$$\boxed{v_L = v - \frac{L}{2}\omega, \quad v_R = v + \frac{L}{2}\omega.}$$

The signs follow from the right-hand-rule identities $\mathbf{e}_{z,b} \times \mathbf{e}_{y,b} = -\mathbf{e}_{x,b}$ and $\mathbf{e}_{z,b} \times (-\mathbf{e}_{y,b}) = \mathbf{e}_{x,b}$. For a positive yaw rate, the robot turns left: rotation reduces the left-wheel forward speed and increases the right-wheel forward speed. The product $L\omega$ has units m s^{-1} because radians are dimensionless in dimensional calculations.

Adding the two equations isolates forward speed, while subtracting the left equation from the right equation isolates yaw rate:

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{L}.$$

Substituting the pure-rolling relations produces

$$v = \frac{r}{2} (\dot{\phi}_R + \dot{\phi}_L), \quad \omega = \frac{r}{L} (\dot{\phi}_R - \dot{\phi}_L).$$

In matrix form,

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \underbrace{\begin{bmatrix} r/2 & r/2 \\ -r/L & r/L \end{bmatrix}}_{\mathbf{G} \in \mathbb{R}^{2 \times 2}} \begin{bmatrix} \dot{\phi}_L \\ \dot{\phi}_R \end{bmatrix}.$$

The matrix $\mathbf{G}$ maps the wheel-speed column $[\dot{\phi}_L \ \dot{\phi}_R]^\top$ to the body-velocity column $[v \ \omega]^\top$. In this chapter, this input-output direction is called the **wheel-to-body velocity mapping**: its inputs are the two wheel angular speeds, and its outputs are the resulting body-forward speed and yaw rate. It is also called the **forward differential-drive velocity kinematics**. Here, *velocity kinematics* means an algebraic relationship among instantaneous velocities without integrating the pose or modelling the forces that produce the motion. The word *forward* describes the calculation from wheel motion to body motion; it does not mean that the robot must travel in the positive forward direction, because the signed wheel speeds and v may be negative.

The two rows of $\mathbf{G}$ have different physical units because the two outputs have different units.

Because $r > 0$ and $L > 0$, the mapping can be inverted. Solving for the wheel speeds gives

$$\dot{\phi}_L = \frac{v}{r} - \frac{L\omega}{2r}, \quad \dot{\phi}_R = \frac{v}{r} + \frac{L\omega}{2r}.$$

This opposite input-output direction is called the **body-to-wheel velocity mapping**, or the **inverse differential-drive velocity kinematics**. Its inputs are a desired body-forward speed v and yaw rate ω , and its outputs are the ideal left and right wheel angular speeds $\dot{\phi}_L$ and $\dot{\phi}_R$ required by the no-slip model. The word *inverse* means that this calculation reverses the wheel-to-body mapping; it does not mean that the robot is commanded to drive backwards. In a controller, the body-to-wheel mapping converts a desired chassis motion into wheel-speed commands, whereas the wheel-to-body mapping can convert measured wheel speeds into an estimate of chassis motion.

3.2.2 Characteristic motions

- If $\dot{\phi}_L = \dot{\phi}_R$, then $\omega = 0$ and the robot moves straight.
- If $\dot{\phi}_L = -\dot{\phi}_R$, then $v = 0$ and the robot rotates about O_b .
- If one wheel is stationary, the ideal robot follows a circle about that wheel contact point.
- If the wheel speeds are unequal and not opposite, the robot follows an arc about an instantaneous centre; the signed turning radius is derived next.

3.2.2.1 Instantaneous centre of curvature and signed turning radius

Assume $\omega \neq 0$ at time t . At that instant, the planar rigid-body motion can be regarded as rotation about a point C in the plane whose instantaneous linear velocity is zero. This point is called the instantaneous centre of curvature (ICC). The body origin O_b therefore moves instantaneously along a circular arc centred at C .

The no-sideways-motion constraint makes the velocity of O_b tangent to the arc and parallel to the body x_b -axis. A circle radius is perpendicular to its tangent, so the line joining O_b and C lies along the lateral y_b -axis.

Let ${}^w\mathbf{p}_{O_b}, {}^w\mathbf{p}_C \in \mathbb{R}^2$ be the world-frame position columns of O_b and C , respectively. Define the ordinary nonnegative radius ρ , measured in metres, by

$$\rho := \text{distance}(O_b, C) = \|{}^w\mathbf{p}_C - {}^w\mathbf{p}_{O_b}\|_2 \geq 0.$$

The magnitude ρ records only the distance between the two points. Define the signed turning radius $R \in \mathbb{R}$ by using its sign to record which side of the robot contains the ICC:

$$R := \begin{cases} +\rho, & C \text{ is to the left of } O_b, \\ -\rho, & C \text{ is to the right of } O_b. \end{cases}$$

Consequently, $|R| = \rho$. The relative-position vector from O_b to C , expressed in world-frame coordinates, is

$${}^w\mathbf{r}_{O_b C} := {}^w\mathbf{p}_C - {}^w\mathbf{p}_{O_b} = R {}^w\mathbf{e}_{y,b}, \quad {}^w\mathbf{e}_{y,b} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix} \in \mathbb{R}^2.$$

The unit vector ${}^w\mathbf{e}_{y,b}$ points along the robot lateral $+y_b$ -axis and is expressed in world coordinates. Multiplying it by $R > 0$ places C to the left of O_b , while multiplying it by $R < 0$ reverses the direction and places C to the right. Figure Figure 3.2 shows both

cases. The forward velocity is tangent to the circular path and therefore perpendicular to the radius direction.

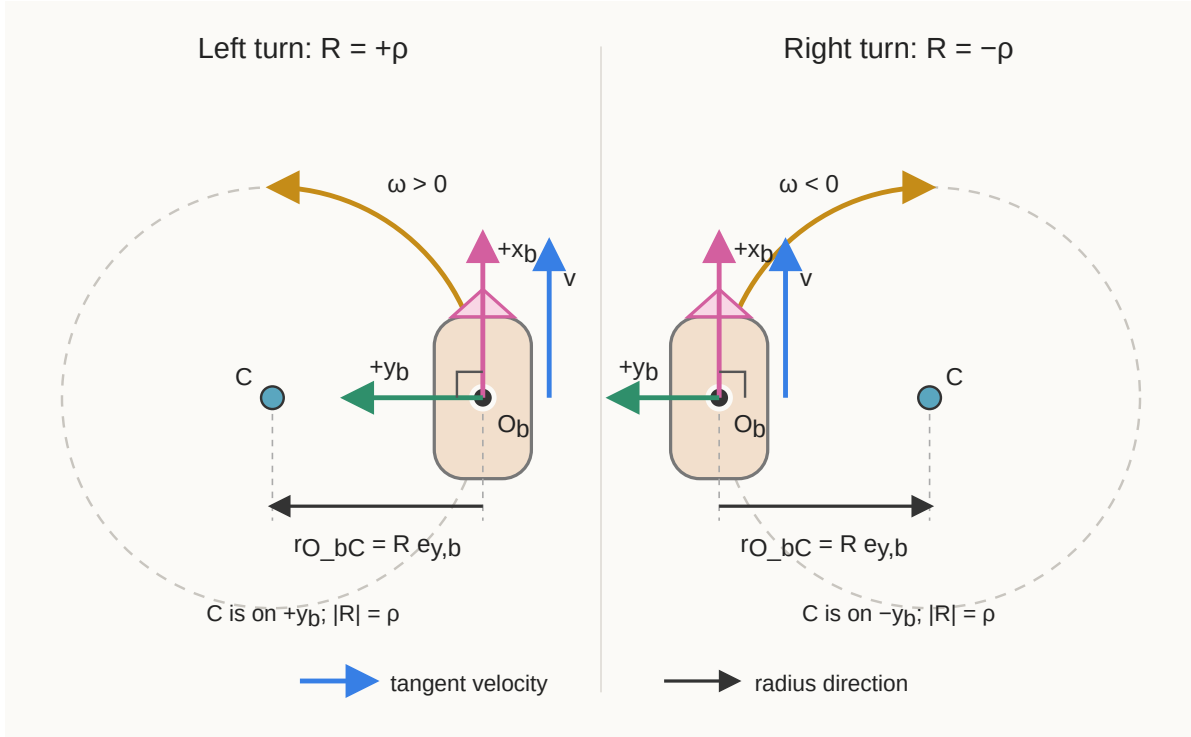


Figure 3.2: Relationship among the body origin, body axes, instantaneous centre of curvature, tangent velocity, and signed turning radius.

To derive the relationship among R , v , and ω , consider a short interval $\Delta t > 0$ during which the instantaneous values are treated as constant. Define the signed heading change by

$$\Delta\theta := \theta(t + \Delta t) - \theta(t),$$

where $\Delta\theta$ is measured in radians. Let $\Delta\ell \in \mathbb{R}$ be the signed distance travelled by O_b along the arc, measured in metres; $\Delta\ell > 0$ denotes travel in the $+x_b$ -direction and $\Delta\ell < 0$ denotes travel in the $-x_b$ -direction.

The radian definition gives arc length equal to radius times angular change. With the signed quantities just defined, it becomes

$$\boxed{\Delta\ell = R \Delta\theta}.$$

For a forward left turn, both R and $\Delta\theta$ are positive. For a forward right turn, both are negative. Their product is therefore the positive forward distance $\Delta\ell$ in either case; the same signed equation also remains valid for backward motion.

Divide the signed arc equation by Δt :

$$\frac{\Delta \ell}{\Delta t} = R \frac{\Delta \theta}{\Delta t}.$$

Taking the limit as $\Delta t \rightarrow 0$ gives

$$\frac{d\ell}{dt} = R \frac{d\theta}{dt}.$$

By definition, the signed forward speed is $v = d\ell/dt$, measured in m s^{-1} , and the signed yaw rate is $\omega = d\theta/dt$, measured in rad s^{-1} . Therefore,

$$\boxed{v = R\omega}.$$

When $\omega \neq 0$, division by ω yields

$$\boxed{R = \frac{v}{\omega}}, \quad \omega \neq 0.$$

The quotient has units of metres because radians are dimensionless in dimensional calculations. If $\omega = 0$ and $v \neq 0$, the robot moves straight and has no finite ICC; its turning radius is informally described as infinite. If $v = 0$ and $\omega \neq 0$, then $R = 0$ and the robot rotates in place about O_b .

The symbol R in this subsection is a scalar signed turning radius, not a rotation matrix.

Quick test B: wheel-to-body mapping

1. Define $\phi_L, \phi_R, \dot{\phi}_L, \dot{\phi}_R, r$, and L , including their units, and distinguish wheel angle from angular speed.
2. Why is a radian dimensionless in an SI unit calculation, and why does $r\dot{\phi}$ have units of metres per second?
3. Starting from ${}^b\mathbf{v}_P = {}^b\mathbf{v}_O + {}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{OP}$, use the two wheel-centre position vectors to derive $v_L = v - L\omega/2$ and $v_R = v + L\omega/2$.
4. Why does equal wheel speed produce $\omega = 0$?
5. Derive $\omega = r(\dot{\phi}_R - \dot{\phi}_L)/L$.
6. Let $r = 0.10 \text{ m}$, $L = 0.50 \text{ m}$, $\dot{\phi}_L = 2 \text{ rad s}^{-1}$, and $\dot{\phi}_R = 4 \text{ rad s}^{-1}$. Find v and ω .
7. Define the ICC C , ordinary radius ρ , signed radius R , signed distance $\Delta\ell$, and heading change $\Delta\theta$. Starting from $\Delta\ell = R\Delta\theta$, derive $R = v/\omega$ and explain the signs for left and right turns.
8. Name two physical effects that can make predicted and real motion disagree.

Answers and hints

1. ϕ_L and ϕ_R are unwrapped left and right wheel rotational positions in radians; $\dot{\phi}_L$ and $\dot{\phi}_R$ are their angular speeds in rad s^{-1} ; r is effective wheel radius in metres; and L is track width in metres.
2. Radian measure is an arc-length-to-radius ratio, so its metre units cancel. Therefore, $r\dot{\phi}$ has units $\text{m} \cdot \text{rad s}^{-1} = \text{m s}^{-1}$.
3. In frame $\{b\}$, use ${}^b\mathbf{r}_{O_b C_L} = [0, L/2, 0]^\top$, ${}^b\mathbf{r}_{O_b C_R} = [0, -L/2, 0]^\top$, ${}^b\mathbf{v}_{O_b} = [v, 0, 0]^\top$, and ${}^b\boldsymbol{\omega} = [0, 0, \omega]^\top$. The cross products contribute $[-L\omega/2, 0, 0]^\top$ on the left and $[L\omega/2, 0, 0]^\top$ on the right. Adding ${}^b\mathbf{v}_{O_b}$ and selecting the $+x_b$ components gives the two required equations.
4. Equal wheel speeds have zero difference, so the yaw-rate equation gives $\omega = 0$.
5. Subtract $v_L = v - L\omega/2$ from $v_R = v + L\omega/2$, then substitute $v_R = r\dot{\phi}_R$ and $v_L = r\dot{\phi}_L$.
6. $v = 0.30 \text{ m s}^{-1}$ and $\omega = 0.40 \text{ rad s}^{-1}$.
7. The ICC C is the point about which the robot is instantaneously rotating. The radius $\rho = \text{distance}(O_b, C) \geq 0$ is an ordinary distance, while $R = +\rho$ for an ICC on the left and $R = -\rho$ for one on the right. The quantity $\Delta\ell$ is signed forward distance and $\Delta\theta$ is signed heading change. Dividing $\Delta\ell = R\Delta\theta$ by Δt and taking the limit gives $v = R\omega$, hence $R = v/\omega$ for $\omega \neq 0$. A forward left turn has $R > 0$ and $\omega > 0$; a forward right turn has $R < 0$ and $\omega < 0$.
8. Examples include wheel slip, unequal wheel radii, tyre deformation, backlash, encoder error, an incorrect track width, and uneven ground.

3.3 From velocity to a finite pose change

The differential equation $\dot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, \mathbf{u})$ specifies instantaneous motion. A simulator or odometry system needs the pose after a finite interval.

Let t_k denote sample time k , where $k \in \{0, 1, 2, \dots\}$, and define

$$\Delta t := t_{k+1} - t_k > 0.$$

The sampling interval Δt is measured in seconds. Define $\mathbf{q}_k := \mathbf{q}(t_k)$ and assume that v and ω remain constant on $[t_k, t_{k+1}]$. This is a zero-order-hold input assumption.

3.3.1 Exact straight motion

If $\omega = 0$, the heading remains constant. Recall that

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix}.$$

Integrating the three rate equations gives

$$\begin{aligned} x_{k+1} &= x_k + v\Delta t \cos \theta_k, \\ y_{k+1} &= y_k + v\Delta t \sin \theta_k, \\ \theta_{k+1} &= \theta_k. \end{aligned}$$

3.3.2 Exact constant-curvature motion

If $\omega \neq 0$, let $\tau \in [0, \Delta t]$ denote elapsed time within the interval. Then

$$\theta(t_k + \tau) = \theta_k + \omega\tau.$$

Substituting this heading into the position-rate equations and integrating gives

$$\begin{aligned} x_{k+1} - x_k &= \int_0^{\Delta t} v \cos(\theta_k + \omega\tau) d\tau \\ &= \frac{v}{\omega} [\sin(\theta_k + \omega\Delta t) - \sin \theta_k], \\ y_{k+1} - y_k &= \int_0^{\Delta t} v \sin(\theta_k + \omega\tau) d\tau \\ &= \frac{v}{\omega} [\cos \theta_k - \cos(\theta_k + \omega\Delta t)]. \end{aligned}$$

Therefore,

$$\begin{aligned} x_{k+1} &= x_k + \frac{v}{\omega} [\sin(\theta_k + \omega\Delta t) - \sin \theta_k], \\ y_{k+1} &= y_k + \frac{v}{\omega} [\cos \theta_k - \cos(\theta_k + \omega\Delta t)], \\ \theta_{k+1} &= \theta_k + \omega\Delta t. \end{aligned}$$

The signed turning radius is $R = v/\omega$, measured in metres, and the signed angular change is $\Delta\theta = \omega\Delta t$, measured in radians. The robot therefore follows a circular arc.

The formulas contain v/ω and therefore cannot be evaluated directly when $\omega = 0$. To express their continuous limit, define the unnormalised sinc function $\text{sinc} : \mathbb{R} \rightarrow \mathbb{R}$ by

$$\text{sinc}(z) := \begin{cases} \frac{\sin z}{z}, & z \neq 0, \\ 1, & z = 0. \end{cases}$$

Here, $z \in \mathbb{R}$ is a dimensionless scalar, and the value at zero follows from $\lim_{z \rightarrow 0} \sin z / z = 1$. This convention must be stated because some software libraries instead define a normalised sinc function containing πz .

Using the trigonometric identities

$$\sin(\theta_k + \Delta\theta) - \sin \theta_k = 2 \cos\left(\theta_k + \frac{\Delta\theta}{2}\right) \sin\left(\frac{\Delta\theta}{2}\right)$$

and

$$\cos \theta_k - \cos(\theta_k + \Delta\theta) = 2 \sin\left(\theta_k + \frac{\Delta\theta}{2}\right) \sin\left(\frac{\Delta\theta}{2}\right),$$

the common scale factor in both position increments can be rewritten, for $\omega \neq 0$, as

$$\begin{aligned} \frac{v}{\omega} 2 \sin\left(\frac{\Delta\theta}{2}\right) &= v \Delta t \frac{\sin(\Delta\theta/2)}{\Delta\theta/2} \\ &= v \Delta t \text{sinc}\left(\frac{\Delta\theta}{2}\right), \end{aligned}$$

where $\Delta\theta = \omega \Delta t$ was used in the first equality. The exact update is consequently equivalent to

$$\boxed{\begin{aligned} x_{k+1} &= x_k + v \Delta t \text{sinc}\left(\frac{\Delta\theta}{2}\right) \cos\left(\theta_k + \frac{\Delta\theta}{2}\right), \\ y_{k+1} &= y_k + v \Delta t \text{sinc}\left(\frac{\Delta\theta}{2}\right) \sin\left(\theta_k + \frac{\Delta\theta}{2}\right), \\ \theta_{k+1} &= \theta_k + \Delta\theta. \end{aligned}}$$

As $\omega \rightarrow 0$, the angular change $\Delta\theta \rightarrow 0$, $\text{sinc}(\Delta\theta/2) \rightarrow 1$, and the midpoint heading $\theta_k + \Delta\theta/2 \rightarrow \theta_k$. The position equations therefore reduce smoothly to the straight-motion update

$$x_{k+1} = x_k + v\Delta t \cos \theta_k, \quad y_{k+1} = y_k + v\Delta t \sin \theta_k.$$

Software may use an explicit straight-motion case when $|\omega|$ is below a declared tolerance or a sinc implementation designed to remain numerically stable near zero. Directly computing $\sin z/z$ at $z = 0$ would still divide by zero despite the function's well-defined limiting value.

Quick test C: exact finite motion

1. What does the zero-order-hold assumption say about v and ω ?
2. Why is $\theta(t_k + \tau) = \theta_k + \omega\tau$ when ω is constant?
3. Where does the factor $1/\omega$ in the position integral come from?
4. Starting from $\mathbf{q}_k = [0 \ 0 \ 0]^\top$, let $v = 1 \text{ m s}^{-1}$, $\omega = 1 \text{ rad s}^{-1}$, and $\Delta t = 1 \text{ s}$. Find the exact next pose.
5. Define the unnormalised sinc function and explain how it exposes the straight-motion limit.
6. Why should software not directly evaluate either v/ω at $\omega = 0$ or $\sin z/z$ at $z = 0$?

Answers and hints

1. Both inputs remain constant throughout the sample interval.
2. Integrating $\dot{\theta} = \omega$ for elapsed time τ gives heading change $\omega\tau$.
3. With $z = \theta_k + \omega\tau$, $dz = \omega d\tau$, so $d\tau = dz/\omega$.
4. $\mathbf{q}_{k+1} = [\sin 1 \quad 1 - \cos 1 \quad 1]^\top \approx [0.8415 \quad 0.4597 \quad 1.0000]^\top$, with position in metres and heading in radians.
5. The unnormalised function is $\text{sinc}(z) = \sin z/z$ for $z \neq 0$ and $\text{sinc}(0) = 1$. Rewriting the update with $\text{sinc}(\Delta\theta/2)$ makes the limit visible: as $\omega \rightarrow 0$, $\Delta\theta \rightarrow 0$, the sinc factor tends to 1, and the midpoint heading tends to θ_k .
6. Both direct expressions would divide by zero even though their mathematical limits are well defined. Software must select the straight case or evaluate sinc with a numerically stable near-zero implementation.

3.4 Numerical integration with Forward Euler

For a general time-varying input, the exact solution may not have a convenient closed form. The integral form of the state equation over one interval is

$$\mathbf{q}(t_{k+1}) = \mathbf{q}(t_k) + \int_{t_k}^{t_{k+1}} \mathbf{f}(\mathbf{q}(t), \mathbf{u}(t)) dt.$$

Numerical integration approximates this integral using a finite number of evaluations of $\mathbf{f}$.

3.4.1 Derivation from a Taylor expansion

Assume $\mathbf{q}(t)$ is sufficiently differentiable near t_k . Its Taylor expansion is

$$\mathbf{q}(t_k + \Delta t) = \mathbf{q}(t_k) + \Delta t \dot{\mathbf{q}}(t_k) + \frac{\Delta t^2}{2} \ddot{\mathbf{q}}(\xi_k),$$

for some $\xi_k \in (t_k, t_{k+1})$. Substituting $\dot{\mathbf{q}}(t_k) = \mathbf{f}(\mathbf{q}_k, \mathbf{u}_k)$ and discarding the term proportional to Δt^2 gives the Forward Euler update

$$\boxed{\mathbf{q}_{k+1}^E = \mathbf{q}_k^E + \Delta t \mathbf{f}(\mathbf{q}_k^E, \mathbf{u}_k)}.$$

The superscript E identifies the Euler approximation, and $\mathbf{u}_k := \mathbf{u}(t_k)$ is the sampled input. Forward Euler uses the slope at the beginning of the interval and treats it as constant across the whole interval.

For the planar kinematic model,

$$\boxed{\begin{aligned} x_{k+1}^E &= x_k^E + \Delta t v_k \cos \theta_k^E, \\ y_{k+1}^E &= y_k^E + \Delta t v_k \sin \theta_k^E, \\ \theta_{k+1}^E &= \theta_k^E + \Delta t \omega_k. \end{aligned}}$$

Here, v_k and ω_k are the body-forward speed and yaw rate used for sample k .

3.4.2 Local and accumulated error

Let $E(\Delta t) \geq 0$ denote a chosen error magnitude produced using step size $\Delta t > 0$. For an exponent $p > 0$, the statement

$$E(\Delta t) = O(\Delta t^p) \quad \text{as} \quad \Delta t \rightarrow 0$$

means that there are constants $C > 0$ and $\Delta t_0 > 0$, independent of Δt , such that

$$E(\Delta t) \leq C \Delta t^p \quad \text{for every} \quad 0 < \Delta t \leq \Delta t_0.$$

The exponent p is the convergence order. The big- O statement is a bound on how the error scales for sufficiently small steps, not an equation claiming that the error equals Δt^p . When the leading error term is nonzero and Δt is sufficiently small, one commonly observes the approximate scaling $E(\Delta t) \approx K \Delta t^p$ for a constant $K > 0$ that does not depend on Δt .

The local truncation error is the error introduced in one step when that step begins from the exact state. Forward Euler has local truncation error $O(\Delta t^2)$. Over a fixed total time interval, reducing Δt creates more steps, and their errors accumulate. The resulting global error of Forward Euler is generally $O(\Delta t)$. In the sufficiently-small-step regime where the leading first-order term dominates, halving Δt therefore approximately halves the global error; it does not usually quarter it.

3.4.3 Worked comparison with the exact update

Use the initial pose and constant input

$$\mathbf{q}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad v = 1 \text{ m s}^{-1}, \quad \omega = 1 \text{ rad s}^{-1}, \quad \Delta t = 1 \text{ s}.$$

The exact update is

$$\mathbf{q}_1 \approx \begin{bmatrix} 0.8415 \text{ m} \\ 0.4597 \text{ m} \\ 1.0000 \text{ rad} \end{bmatrix}.$$

Forward Euler evaluates the direction only at initial heading $\theta_0 = 0$:

$$\mathbf{q}_1^E = \begin{bmatrix} 0 + 1 \cos 0 \\ 0 + 1 \sin 0 \\ 0 + 1 \end{bmatrix} = \begin{bmatrix} 1 \text{ m} \\ 0 \text{ m} \\ 1 \text{ rad} \end{bmatrix}.$$

Both methods produce the same heading because ω is constant and the heading equation is linear. The Euler position is wrong because the forward direction rotates substantially during the one-second interval. Define one-step position error by

$$e_p := \sqrt{(x_1^E - x_1)^2 + (y_1^E - y_1)^2}.$$

For this example, $e_p \approx 0.4863 \text{ m}$. Smaller steps allow Euler to update the direction more often. When constant-input differential-drive motion is assumed, however, the exact arc update is available and more accurate.

3.4.4 Choosing and checking an integration method

- Use the exact straight-or-arc update when v and ω are held constant and the ideal planar model is appropriate.
- Use a numerical method when the input or model varies within the interval and no convenient exact solution is available.
- Reduce Δt when the state changes too much during one step, while recognising that smaller steps increase computation.
- Compare a numerical trajectory with a known exact case before trusting it on a harder case.
- Compute Δt from the intended time source; do not silently assume perfectly periodic execution.

Quick test D: Forward Euler

1. State the Forward Euler update and define every symbol in it.
2. In plain language, what approximation does Forward Euler make during one step?
3. Suppose one Forward Euler step begins from the exact pose and v and ω remain constant during the step. Why does Forward Euler compute the exact position when $\omega = 0$, but generally not when the robot turns with $\omega \neq 0$?
4. What does the global-error statement $E(\Delta t) = O(\Delta t)$ mean, and what approximate change in global error should occur when a sufficiently small Δt is halved over the same fixed total time interval?
5. Give one exact verification case and one physical limitation for an odometry integrator.

Answers and hints

1. $\mathbf{q}_{k+1}^E = \mathbf{q}_k^E + \Delta t \mathbf{f}(\mathbf{q}_k^E, \mathbf{u}_k)$. The terms are the next approximate state, current approximate state, step duration, state-derivative function, and sampled input.
2. It uses the derivative at the start as if that derivative remained constant across the interval.
3. When $\omega = 0$, the heading θ remains constant. Constant v then gives the constant world-frame velocity $[v \cos \theta_k \ v \sin \theta_k]^\top$, so multiplying the initial velocity by Δt evaluates the position integral exactly. When $\omega \neq 0$ and $v \neq 0$, the heading and therefore the world-frame velocity direction change continuously during the step. Forward Euler holds the initial direction constant and produces a straight tangent displacement instead of the exact arc displacement. The word “generally” allows special cases such as pure rotation with $v = 0$, for which the position remains unchanged and is therefore exact.
4. The statement $E(\Delta t) = O(\Delta t)$ means that constants $C > 0$ and $\Delta t_0 > 0$ exist such that $E(\Delta t) \leq C\Delta t$ whenever $0 < \Delta t \leq \Delta t_0$. It describes first-order scaling as $\Delta t \rightarrow 0$, not exact equality. In the regime where the leading first-order

error dominates, $E(\Delta t) \approx K\Delta t$, so replacing Δt by $\Delta t/2$ approximately halves the global error over the same fixed total time interval.

5. Exact cases include constant straight motion, pure rotation, and constant-curvature motion. Physical limitations include slip, parameter error, encoder quantisation, and timing error.

3.5 Engineering implications and failure modes

1. **Frame inconsistency:** The scalar v is a body-frame forward speed, but $\dot{x}$ and $\dot{y}$ are world-frame components. Assigning $\dot{x} = v$ is valid only when $\theta = 0$ modulo 2π .
2. **Sign-convention mismatch:** Swapping left and right wheels or using a clockwise-positive yaw convention reverses the predicted turn. Under this chapter convention, $\dot{\phi}_L < \dot{\phi}_R$ must produce positive counter-clockwise yaw.
3. **Unit mismatch:** Wheel speed may arrive in revolutions per minute while the model expects radians per second. Heading may arrive in degrees while trigonometric functions expect radians. Convert units at explicit interface boundaries.
4. **Incorrect or variable sampling time:** Using an assumed Δt when measurements arrive irregularly causes distance and angle errors. An update should validate timestamp differences and define behaviour for zero, negative, missing, or unreasonably large intervals.
5. **Angle representation:** The equations allow θ to grow without bound, but an interface may require $\theta \in (-\pi, \pi]$. Wrapping changes the numerical representative, not the physical orientation. Heading differences require a wrap-aware calculation.
6. **Model error versus integration error:** Reducing Δt reduces integration error but cannot correct an inaccurate physical model. If a wheel slips, even exact integration of the ideal no-slip equations gives the wrong physical trajectory.
7. **Encoder integration and drift:** Wheel encoders measure incremental rotation rather than global position. Integrating these increments estimates pose relative to an initial condition. Small systematic and random errors accumulate, so wheel odometry alone generally drifts and requires external sensing for long-term correction.

Cumulative chapter test

1. Define $\mathbf{q}$, $\mathbf{u}$, $\mathbf{f}$, r , L , $\dot{\phi}_L$, $\dot{\phi}_R$, t_k , and Δt , including dimensions or units.
2. Starting with ${}^b\mathbf{v} = [v \ 0]^\top$, derive $\dot{x} = v \cos \theta$ and $\dot{y} = v \sin \theta$.
3. Derive the no-sideways-motion constraint from the world-frame coordinate column of the y_b -axis.

4. Derive the wheel-to-body equations for v and ω and the body-to-wheel equations for $\dot{\phi}_L$ and $\dot{\phi}_R$.
5. A robot has $r = 0.08$ m, $L = 0.40$ m, $\dot{\phi}_L = 5$ rad s⁻¹, and $\dot{\phi}_R = 7$ rad s⁻¹. Find v and ω and state the turn direction.
6. Derive the exact constant-curvature update by integrating the position rates over one interval.
7. Derive Forward Euler from the first two Taylor terms and explain the discarded term.
8. Explain why exact mathematical integration can still produce inaccurate physical odometry.
9. Design three verification cases that separately reveal a wheel-order error, a yaw-sign error, and excessive integration error.

Cumulative answers and hints

1. $\mathbf{q} = [x \ y \ \theta]^\top$ is planar pose with units m, m, rad; $\mathbf{u} = [v \ \omega]^\top$ contains forward speed and yaw rate in m s⁻¹ and rad s⁻¹; $\mathbf{f}$ maps pose and input to pose derivative; r is wheel radius in metres; L is track width in metres; $\dot{\phi}_L$ and $\dot{\phi}_R$ are wheel angular speeds in radians per second; t_k is sample time in seconds; and $\Delta t = t_{k+1} - t_k$ is the positive integration interval in seconds.
2. Premultiply ${}^b\mathbf{v}$ by $\mathbf{R}_{wb}(\theta)$ and read the two components.
3. Use ${}^w\mathbf{e}_{y,b} = [-\sin \theta \ \cos \theta]^\top$ and set its dot product with $[\dot{x} \ \dot{y}]^\top$ equal to zero.
4. Use $r\dot{\phi}_L = v - L\omega/2$ and $r\dot{\phi}_R = v + L\omega/2$. Addition isolates v and subtraction isolates ω ; solving the pair for wheel speeds gives the inverse equations.
5. $v = (0.08/2)(5+7) = 0.48$ m s⁻¹ and $\omega = (0.08/0.40)(7-5) = 0.40$ rad s⁻¹. The positive yaw rate is a counter-clockwise or left turn.
6. Write $\theta(t_k + \tau) = \theta_k + \omega\tau$, integrate over $\tau \in [0, \Delta t]$, and use antiderivatives $\sin(\theta_k + \omega\tau)/\omega$ and $-\cos(\theta_k + \omega\tau)/\omega$.
7. Taylor expansion gives $\mathbf{q}(t_k + \Delta t) = \mathbf{q}(t_k) + \Delta t\dot{\mathbf{q}}(t_k) + O(\Delta t^2)$. Replacing $\dot{\mathbf{q}}$ with $\mathbf{f}$ and omitting the second-order remainder gives Forward Euler. The discarded term represents within-step change of the derivative.
8. Exact integration eliminates approximation for the stated model and input assumption; it cannot eliminate slip, parameter error, encoder error, timing error, or other model violations.
9. For wheel order, command unequal known wheel speeds and check the turn direction. For yaw sign, command pure positive rotation and check that heading increases counter-clockwise. For integration error, compare with the exact constant-curvature solution at decreasing values of Δt and verify convergence.

References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapter 13, especially the wheeled-mobile-robot kinematic models and odometry discussion.
- Steven M. LaValle, *Planning Algorithms*, Chapter 13, especially continuous-time differential models, velocity constraints, and wheeled-vehicle models.

4 Differential-Drive Kinematics and Numerical Integration: Implementation

The code explained in this companion is in `ros_ws/src/differential_drive_motion_model`.

Purpose and scope

This companion translates the differential-drive mathematics derived in [Differential-Drive Kinematics and Numerical Integration: Model and Derivation](#) into reusable software contracts, invariants, validation rules, and tests. It records how those equations map into dependable software without repeating the derivations.

4.1 Wheel-to-body velocity mapping

Let $r > 0$ be the common effective wheel radius in metres and let $L > 0$ be the track width in metres. Let $\dot{\phi}_L$ and $\dot{\phi}_R$ be the signed left and right wheel angular speeds in rad s^{-1} , where positive angular speed rolls the corresponding wheel forward.

The software implements

$$v = \frac{r}{2} (\dot{\phi}_R + \dot{\phi}_L), \quad \omega = \frac{r}{L} (\dot{\phi}_R - \dot{\phi}_L),$$

where v is signed body-forward speed in m s^{-1} and ω is signed counter-clockwise yaw rate in rad s^{-1} .

4.1.1 Software contract

The `wheel_to_body` operation accepts four finite scalar inputs: left wheel angular speed, right wheel angular speed, wheel radius, and track width. The wheel radius and track width must be strictly positive. Invalid inputs raise `ValueError`; the operation does not clamp values or substitute default geometry.

The operation returns an immutable `BodyVelocity` value containing `forward_speed_m_s` and `yaw_rate_rad_s`. It is deterministic, does not mutate its inputs, and does not depend on ROS state, timestamps, configuration files, or hidden global state.

4.1.2 Simplified reference snippet

The following equation-focused snippet shows how the mathematical factors map to Python expressions. It assumes that all inputs are finite and that `wheel_radius_m` and `track_width_m` are strictly positive; the production operation enforces those preconditions before performing the calculation.

```
@dataclass(frozen=True, slots=True)
class BodyVelocity:
    """Reduced planar body velocity in SI units."""

    forward_speed_m_s: float
    yaw_rate_rad_s: float

def wheel_to_body(
    *,
    left_angular_speed_rad_s: float,
    right_angular_speed_rad_s: float,
    wheel_radius_m: float,
    track_width_m: float,
) -> BodyVelocity:
    """Apply the wheel-to-body equations to validated inputs."""
    forward_speed_m_s = (
        wheel_radius_m / 2
        * (right_angular_speed_rad_s + left_angular_speed_rad_s)
    )
    yaw_rate_rad_s = (
        wheel_radius_m / track_width_m
        * (right_angular_speed_rad_s - left_angular_speed_rad_s)
    )

    return BodyVelocity(
```



```

        forward_speed_m_s=forward_speed_m_s,
        yaw_rate_rad_s=yaw_rate_rad_s,
    )

```

4.1.3 Invariants

- Equal wheel speeds produce zero yaw rate.
- Equal and opposite wheel speeds produce zero forward speed.
- Swapping the left and right wheel speeds preserves forward speed and reverses yaw rate.
- Two zero wheel speeds produce zero forward speed and zero yaw rate.
- Positive right-wheel speed relative to left-wheel speed produces positive counter-clockwise yaw.

4.2 Body-to-wheel velocity mapping

The `body_to_wheel` operation implements the inverse of the wheel-to-body mapping. Its inputs are signed body-forward speed v in m s^{-1} , signed counter-clockwise yaw rate ω in rad s^{-1} , wheel radius $r > 0$ in metres, and track width $L > 0$ in metres.

The required wheel angular speeds are

$$\dot{\phi}_L = \frac{v}{r} - \frac{L\omega}{2r}, \quad \dot{\phi}_R = \frac{v}{r} + \frac{L\omega}{2r},$$

where $\dot{\phi}_L$ and $\dot{\phi}_R$ are the signed left and right wheel angular speeds in rad s^{-1} .

4.2.1 Software contract

The operation accepts four finite scalar inputs. The wheel radius and track width must be strictly positive. Invalid inputs raise `ValueError`; the operation does not clamp values or substitute geometry.

The result is an immutable `WheelAngularSpeeds` value containing `left_angular_speed_rad_s` and `right_angular_speed_rad_s`.

4.2.2 Implementation mapping

The calculation separates forward and turning contributions:

```

@dataclass(frozen=True, slots=True)
class WheelAngularSpeeds:
    """Signed left and right wheel angular speeds in radians per second."""

    left_angular_speed_rad_s: float
    right_angular_speed_rad_s: float

def body_to_wheel(
    *,
    forward_speed_m_s: float,
    yaw_rate_rad_s: float,
    wheel_radius_m: float,
    track_width_m: float,
) -> WheelAngularSpeeds:
    """Apply the body-to-wheel equations to validated inputs."""
    forward_component_rad_s = forward_speed_m_s / wheel_radius_m
    turning_component_rad_s = (
        track_width_m * yaw_rate_rad_s / (2 * wheel_radius_m)
    )
    left_angular_speed_rad_s = (
        forward_component_rad_s - turning_component_rad_s
    )
    right_angular_speed_rad_s = (
        forward_component_rad_s + turning_component_rad_s
    )

    return WheelAngularSpeeds(
        left_angular_speed_rad_s=left_angular_speed_rad_s,
        right_angular_speed_rad_s=right_angular_speed_rad_s,
    )

```

The forward component contributes equally to both wheels. The turning component is subtracted from the left wheel and added to the right wheel. Therefore, a positive yaw rate makes the right wheel turn faster than the left wheel.

4.2.3 Invariants and required tests

- Zero yaw rate produces equal wheel speeds of v/r .
- Zero forward speed produces equal-and-opposite wheel speeds.
- Reversing the yaw-rate sign swaps the left and right wheel speeds.
- Applying `body_to_wheel` and then `wheel_to_body` recovers the original v and ω within the declared floating-point tolerance.
- Non-finite inputs and non-positive geometry raise `ValueError`.

4.3 Exact constant-input pose integration

Let the initial planar pose be (x_k, y_k, θ_k) , where x_k and y_k are the world-frame coordinates of the axle midpoint in metres and θ_k is the unwrapped counter-clockwise heading in radians.

Assume that forward speed v in m s^{-1} and yaw rate ω in rad s^{-1} remain constant over a time step $\Delta t > 0$ in seconds. Define the heading change

$$\Delta\theta = \omega\Delta t.$$

The exact update is

$$\begin{aligned} x_{k+1} &= x_k + v\Delta t \operatorname{sinc}\left(\frac{\Delta\theta}{2}\right) \cos\left(\theta_k + \frac{\Delta\theta}{2}\right), \\ y_{k+1} &= y_k + v\Delta t \operatorname{sinc}\left(\frac{\Delta\theta}{2}\right) \sin\left(\theta_k + \frac{\Delta\theta}{2}\right), \\ \theta_{k+1} &= \theta_k + \Delta\theta, \end{aligned}$$

where the unnormalised sinc function is

$$\operatorname{sinc}(z) = \begin{cases} \frac{\sin z}{z}, & z \neq 0, \\ 1, & z = 0. \end{cases}$$

The update is exact for the ideal continuous kinematic model only when v and ω remain constant during the time step. It does not account for wheel slip, changing commands, actuator dynamics, or model error.

4.3.1 Software contract

The `integrate_exact` operation accepts an immutable `PlanarPose`, finite forward speed, finite yaw rate, and a finite positive time step. A non-finite input or non-positive time step raises `ValueError`.

The operation returns a new `PlanarPose`. It does not mutate the initial pose or wrap the resulting heading into a fixed angular interval.

4.3.2 Numerically stable implementation mapping

The implementation avoids evaluating v/ω , which would be undefined when $\omega = 0$. Instead, it uses the sinc form:

```
@dataclass(frozen=True, slots=True)
class PlanarPose:
    """Pose of the robot's axle midpoint in the world frame.

    Attributes:
        position_x_m: World-frame x-coordinate in metres.
        position_y_m: World-frame y-coordinate in metres.
        heading_rad: Counter-clockwise heading from the world-frame
                     positive x-axis in radians.
    """

    position_x_m: float
    position_y_m: float
    heading_rad: float

def _sinc(value: float) -> float:
    """Return the unnormalised sinc function, including its value at zero."""
    if value == 0.0:
        return 1.0

    return sin(value) / value

heading_change_rad = yaw_rate_rad_s * time_step_s
half_heading_change_rad = heading_change_rad / 2
midpoint_heading_rad = pose.heading_rad + half_heading_change_rad

signed_displacement_m = (
    forward_speed_m_s
    * time_step_s
    * _sinc(half_heading_change_rad)
)

next_pose = PlanarPose(
    position_x_m=(
        pose.position_x_m
        + signed_displacement_m * cos(midpoint_heading_rad)
    ),
```

```

position_y_m=(
    pose.position_y_m
    + signed_displacement_m * sin(midpoint_heading_rad)
),
heading_rad=pose.heading_rad + heading_change_rad,
)

```

When $\omega = 0$, we have $\Delta\theta = 0$ and $\text{sinc}(0) = 1$. The same implementation therefore reduces smoothly to straight motion without a separate division by zero.

4.3.3 Invariants and required tests

- Zero yaw rate produces the exact straight-motion update.
- Zero forward speed changes only the heading.
- Constant nonzero forward speed and yaw rate produce constant-curvature motion.
- The input pose remains unchanged.
- The output heading remains unwrapped.
- Non-finite inputs raise `ValueError`.
- A zero or negative time step raises `ValueError`.

4.4 Forward Euler pose integration

Forward Euler approximates motion over a time step $\Delta t > 0$ by evaluating the pose derivative at the beginning of the step. For initial pose (x_k, y_k, θ_k) , constant forward speed v , and constant yaw rate ω , the update is

$$\begin{aligned}
 x_{k+1}^E &= x_k + \Delta t v \cos \theta_k, \\
 y_{k+1}^E &= y_k + \Delta t v \sin \theta_k, \\
 \theta_{k+1}^E &= \theta_k + \Delta t \omega.
 \end{aligned}$$

The superscript E identifies a Forward Euler approximation. Position is calculated entirely from the beginning-of-step heading θ_k ; the changing heading within the step does not affect that step's position increment.

4.4.1 Software contract

The `integrate_forward_euler` operation accepts the same inputs and validation rules as `integrate_exact`: an immutable `PlanarPose`, finite forward speed, finite yaw rate, and a finite positive time step.

The operation returns a new `PlanarPose`, does not mutate the input pose, and leaves the heading unwrapped. Invalid inputs raise `ValueError`.

4.4.2 Implementation mapping

```
next_pose = PlanarPose(
    position_x_m=(
        pose.position_x_m
        + time_step_s * forward_speed_m_s * cos(pose.heading_rad)
    ),
    position_y_m=(
        pose.position_y_m
        + time_step_s * forward_speed_m_s * sin(pose.heading_rad)
    ),
    heading_rad=(
        pose.heading_rad + time_step_s * yaw_rate_rad_s
    ),
)
```

4.4.3 Exact and approximate cases

When $\omega = 0$, the heading remains constant. The world-frame velocity is therefore constant, so one Forward Euler step produces the exact straight-motion position.

When $v = 0$, position remains unchanged and the heading update is exact because $\dot{\theta} = \omega$ is constant.

When both $v \neq 0$ and $\omega \neq 0$, the true path curves during the time step. Forward Euler instead follows the tangent defined by θ_k , so its one-step position is generally approximate.

For repeated integration over a fixed duration, Forward Euler has global error $O(\Delta t)$. In this regime, halving Δt should approximately halve the accumulated error.

4.4.4 Invariants and required tests

- Straight constant-speed motion agrees with the exact update.
- Pure rotation changes only the heading.
- A turning step uses the beginning-of-step heading.
- Smaller time steps reduce accumulated turning-position error.
- Halving the time step approximately halves global error in the first-order regime.
- Non-finite inputs and non-positive time steps raise `ValueError`.

5 ROS 2 Package Development Workflow

Goal

This workflow creates, builds, runs, and tests a small ROS 2 Python package from scratch. It uses `my_robot_package` as an example package name and `my_node` as an example executable name; replace them in a real project.

A **workspace** holds packages developed and built together. A **ROS 2 package** combines source code with the metadata needed for building, installing, and discovery. An **executable** is a program installed by a package and started with `ros2 run`.

The generated `my_node` executable only prints a message. It demonstrates the package workflow but does not yet communicate with other ROS 2 nodes.

5.1 Source the ROS 2 underlay

For Zsh, run:

```
source /opt/ros/jazzy/setup.zsh
```

For Bash, run:

```
source /opt/ros/jazzy/setup.bash
```

Use the script matching the current shell.

What happens: `/opt/ros/jazzy` is the ROS 2 Jazzy **underlay**, meaning the existing installation on which the new workspace depends. Its setup script adds ROS installation prefixes, Python packages, and executables to the current shell's search environment.

The command uses `source` because the environment of the current shell must change. If the script ran in a separate **child process**, meaning a process started by the shell, its environment changes would disappear when that child exited.

Verify the environment:

```
printf '%s\n' "$ROS_DISTRO"
command -v ros2
command -v colcon
```

The expected observations are `jazzy` and executable paths for `ros2` and `colcon`.

5.2 Create a workspace

For a new standalone workspace, run:

```
mkdir -p ~/ros_ws/src
cd ~/ros_ws
```

The workspace develops into:

```
ros_ws/
  src/      # Editable source
  build/    # Generated build workspace
  install/  # Generated runnable workspace image
  log/      # Generated build and test records
```

Only `src/` exists initially. `colcon` creates the other directories. Edit `src/`, not the generated files in `build/`, `install/`, or `log/`.

5.3 Create a Python package

From the workspace root, run:

```
cd src
ros2 pkg create \
  --build-type ament_python \
  --license Apache-2.0 \
  --node-name my_node \
  my_robot_package
cd ..
```

Run this command only when the package does not already exist.

What happens: `ros2 pkg create` generates `src/my_robot_package.`

- `ament_python` selects Python packaging through `setuptools`.
- `Apache-2.0` is recorded in `package.xml`, and a matching `LICENSE` file is created.

- `--node-name my_node` creates a minimal Python executable and its `setup.py` entry point.

Omit `--node-name` when creating a library-only package.

For comparison, create a C++ package with:

```
ros2 pkg create \
  --build-type ament_cmake \
  --license Apache-2.0 \
  robot_controller
```

The common build types are `ament_python` for Python with `setuptools` and `ament_cmake` for C++ with CMake.

5.4 Inspect the generated package

```
src/my_robot_package/
  LICENSE
  package.xml
  setup.py
  setup.cfg
  resource/
    my_robot_package
  my_robot_package/
    __init__.py
    my_node.py
  test/
    test_copyright.py
    test_flake8.py
    test_pep257.py
```

The outer `my_robot_package/` is the ROS package root. The inner directory is the Python import package used by `import my_robot_package`; the repeated name does not indicate nested repositories.

File or directory	Purpose
<code>package.xml</code>	Declares package identity, licence, dependencies, and build type
<code>setup.py</code>	Selects Python packages, data files, and executables to install
<code>setup.cfg</code>	Selects the ROS-compatible destination for Python executables

File or directory	Purpose
<code>resource/my_robot_package</code>	Registers the installed package in the ament resource index
<code>my_robot_package/</code>	Contains importable Python source
<code>test/</code>	Contains checks run by <code>pytest</code> and <code>colcon test</code>

The resource marker is intentionally empty. Its filename identifies the package.

5.5 Complete the package metadata

Replace placeholder descriptions and maintainer details in `package.xml` and `setup.py`. Keep their package name, version, and licence consistent. The `package.xml` export tells `colcon` which build integration to use:

```
<export>
  <build_type>ament_python</build_type>
</export>
```

A concise `setup.py` for the example is:

```
from setuptools import find_packages, setup

package_name = "my_robot_package"

setup(
    name=package_name,
    version="0.0.0",
    packages=find_packages(exclude=["test"]),
    data_files=[
        (
            "share/ament_index/resource_index/packages",
            [f"resource/{package_name}"],
        ),
        (f"share/{package_name}", ["package.xml"]),
    ],
    install_requires=["setuptools"],
    zip_safe=True,
    maintainer="Your Name",
    maintainer_email="you@example.com",
    description="Example ROS 2 Python package.",
```

```

    license="Apache-2.0",
    entry_points={
        "console_scripts": [
            "my_node = my_robot_package.my_node:main",
        ],
    },
)

```

The important connections are:

- `packages` installs importable Python packages.
- `data_files` installs the resource marker and `package.xml` where ROS expects them.
- `console_scripts` creates an executable that calls a Python function.

The entry point `my_node = my_robot_package.my_node:main` maps the executable name `my_node` to the function `main()` in the Python module `my_robot_package.my_node`.

The matching `setup.cfg` is:

```

[develop]
script_dir=$base/lib/my_robot_package

[install]
install_scripts=$base/lib/my_robot_package

```

Here, `$base` is the package installation prefix. `setup.cfg` specifies that executable scripts belong under `lib/my_robot_package`, where `ros2 run` looks for them. It does not create an executable by itself; the `console_scripts` entry does that.

5.6 Resolve dependencies and run fast tests

Resolve dependencies declared by packages under `src/`:

```

rosdep install \
  --from-paths src \
  --ignore-src \
  --rosdistro jazzy \
  -r \
  -y

```

`rosdep` ignores dependencies supplied by source packages in the workspace, maps the remaining dependency keys to operating-system packages, and installs missing packages.

Run fast tests directly against the source tree:

```
python3 -m pytest \
  src/my_robot_package/test -q
```

Source-level tests provide fast feedback but do not verify ROS metadata or the installed layout. Continue only when they pass; the count and duration depend on the package.

5.7 Build and install the package

From the workspace root, run:

```
colcon build \
  --packages-select my_robot_package \
  --symlink-install
```

What happens: `colcon` discovers packages under `src/`, reads their manifests, orders them by declared dependencies, and selects the appropriate build integration. `ament_python` then invokes `setuptools` using `setup.py` and `setup.cfg`.

The logical installed layout is:

```
install/my_robot_package/
  lib/
    python3.12/
      site-packages/
        my_robot_package/
          __init__.py
          my_node.py
        my_robot_package/
          my_node
  share/
    ament_index/
      resource_index/
        packages/
          my_robot_package
    my_robot_package/
      package.xml
```

The Python version in the path depends on the system.

- `site-packages/my_robot_package` supplies importable Python code.
- `lib/my_robot_package/my_node` supplies the executable used by `ros2 run`.
- The resource marker registers the package with ROS.
- The installed `package.xml` supplies package metadata.

ROS 2 needs this standard runtime layout to locate packages, executables, metadata, launch files, configuration, Python imports, and dependency environment hooks.

5.7.1 What `--symlink-install` changes

Supported files are linked back to editable source rather than copied:

```
install/.../my_node.py
    symbolic link
src/my_robot_package/my_robot_package/my_node.py
```

Python packages may use an `.egg-link` instead of the copied directory shown in the logical tree. Consequently, edits to existing linked Python files may appear without rebuilding, but `install/` is still needed for metadata, executables, and environment scripts.

Rebuild after changing `setup.py`, `setup.cfg`, `console_scripts`, installed data, dependencies, CMake configuration, or compiled C++ code. Rebuild after structural changes and before formal verification to avoid stale state.

5.8 Source the overlay and run the package

After a successful build, source the script matching the current shell.

```
source install/setup.zsh
```

```
source install/setup.bash
```

This command does not install anything. It adds the workspace's `install/` tree as an **overlay** on the ROS underlay by updating variables such as `AMENT_PREFIX_PATH`, `PYTHONPATH`, and `PATH`.

Sourcing affects only the current shell. A new terminal must source the ROS underlay and then the workspace overlay again.

Verify package discovery, Python import, and executable discovery:

```
ros2 pkg prefix my_robot_package
python3 -c "import my_robot_package"
ros2 run my_robot_package my_node
```

The final command should print:

```
Hi from my_robot_package.
```

`ros2 pkg prefix` queries the ament resource index. `ros2 run` uses the first argument as the package name and the second as the executable name, then starts `lib/my_robot_package/my_node`.

5.9 Run tests through `colcon`

```
colcon test --packages-select my_robot_package
colcon test-result --verbose
```

`colcon test` runs the package's configured tests and stores structured results under the build tree. `colcon test-result` aggregates them. A completed command is not necessarily successful; verification passes only when the summary reports no errors or failures.

5.10 Repeat the daily development loop

From the workspace root, repeat:

```
# 1. Edit source.
vim src/my_robot_package/my_robot_package/my_node.py

# 2. Run fast tests.
python3 -m pytest src/my_robot_package/test -q

# 3. Build and install.
colcon build \
  --packages-select my_robot_package \
  --symlink-install

# 4. Reload the overlay.
source install/setup.zsh

# 5. Run the executable.
ros2 run my_robot_package my_node

# 6. Run package-level verification.
colcon test --packages-select my_robot_package
colcon test-result --verbose
```

Use `source install/setup.bash` in Bash.

Common failure checks

Symptom	Check
<code>ros2: command not found</code>	Source the ROS underlay in the current shell
Package absent from <code>colcon list</code>	Check its location under <code>src/</code> and validate <code>package.xml</code>
Python package is not installed	Check <code>__init__.py</code> and the <code>packages</code> argument in <code>setup.py</code>
<code>ros2 pkg prefix</code> cannot find the package	Build and source the overlay
<code>ros2 run</code> cannot find the executable	Check <code>console_scripts</code> and <code>setup.cfg</code> , then rebuild and source
Test status is unclear	Run <code>colcon test-result --verbose</code>

References

- Open Robotics, [“Developing a ROS 2 package,”](#) *ROS 2 Jazzy Documentation*.
- Open Robotics, [“Using Python Packages with ROS 2,”](#) *ROS 2 Jazzy Documentation*.
- colcon, [“What is a Workspace?”](#) *colcon Documentation*.

6 Planar Rigid-Body Motion: $SE(2)$ and Twists

Why this chapter matters

The differential-drive pose $\mathbf{q} = [x \ y \ \theta]^T$ is convenient for storing three coordinates, but it does not by itself expose how rigid motions compose, how the same motion is expressed in different frames, or why constant body velocity produces the exact curved-motion update derived previously. This chapter develops the planar rigid-motion structure needed to answer those questions. It begins by distinguishing a pose from its coordinates and representing a planar pose by a homogeneous transformation in the special Euclidean group $SE(2)$.

Prerequisites and assumptions

The reader should understand matrices and matrix multiplication from Chapter 1, coordinate frames and rotation matrices from Chapter 2, and planar pose notation from Chapter 3.

All frames in this learning block are right-handed planar frames embedded in three-dimensional physical space with a common vertical axis. Frame $\{w\}$ is the fixed world frame and frame $\{b\}$ is attached rigidly to the robot body. Distances are measured in metres and angles in radians.

6.1 Pose coordinates and rigid transformations

6.1.1 A pose is a relative geometric relationship

A planar rigid body's **pose** specifies both its position and its orientation relative to a reference frame. The word *relative* is essential: saying that a robot is at position $(2, 1)$ with heading $\pi/2$ is incomplete until the reference frame and sign convention are known.

Let O_w and O_b denote the origins of frames $\{w\}$ and $\{b\}$. The position of O_b relative to O_w , expressed in world-frame coordinates, is

$${}^w\mathbf{p}_b = \begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2,$$

where x and y are measured in metres. The orientation of frame $\{b\}$ relative to frame $\{w\}$ is represented by the rotation matrix $\mathbf{R}_{wb}(\theta) \in \mathbb{R}^{2 \times 2}$:

$$\mathbf{R}_{wb}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

The subscript order wb means that $\mathbf{R}_{wb}$ maps vector coordinates from frame $\{b\}$ to frame $\{w\}$. The pose of $\{b\}$ relative to $\{w\}$ can therefore be described by the pair

$$(\mathbf{R}_{wb}, {}^w\mathbf{p}_b).$$

This pair separates orientation from position. A homogeneous transformation will combine them into one matrix without confusing their different roles.

6.1.2 Planar rotations and $SO(2)$

The set of valid planar rotation matrices is called the **special orthogonal group in two dimensions**, written $SO(2)$ and defined by

$$SO(2) := \{\mathbf{R} \in \mathbb{R}^{2 \times 2} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I}_2, \quad \det(\mathbf{R}) = 1\}.$$

The word *orthogonal* refers to $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_2$, which means that the columns form an orthonormal basis and that $\mathbf{R}^{-1} = \mathbf{R}^\top$. The word *special* adds the condition $\det(\mathbf{R}) = 1$, excluding reflections whose determinant is -1 .

The word *group* means that the set is equipped with an operation satisfying four properties: closure, associativity, an identity element, and an inverse for every element. For $SO(2)$, the operation is matrix multiplication.

Closure means that multiplying any two elements of $SO(2)$ produces another element of $SO(2)$. If $\mathbf{R}(\alpha)$ and $\mathbf{R}(\beta)$ represent planar rotations through angles α and β , then

$$\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha + \beta) \in SO(2).$$

Geometrically, performing one planar rotation and then another still produces a valid planar rotation; it does not introduce scaling, shear, or reflection.

Associativity means that the placement of parentheses does not change the result of composing three rotations:

$$(\mathbf{R}_1 \mathbf{R}_2) \mathbf{R}_3 = \mathbf{R}_1 (\mathbf{R}_2 \mathbf{R}_3).$$

Associativity changes only the grouping, not the order, of the matrices. It should not be confused with commutativity, which would permit the order to be exchanged. The identity element is $\mathbf{I}_2$, and every $\mathbf{R} \in SO(2)$ has the inverse $\mathbf{R}^\top \in SO(2)$.

Every planar rotation matrix can be written as $\mathbf{R}(\theta)$ for some $\theta \in \mathbb{R}$. The angle is not unique because

$$\mathbf{R}(\theta + 2\pi n) = \mathbf{R}(\theta), \quad n \in \mathbb{Z},$$

where $\mathbb{Z}$ is the set of integers. Thus, θ is a **coordinate** used to describe an orientation, while $\mathbf{R}(\theta)$ represents the **orientation** itself. Angles that differ by an integer multiple of 2π describe the same element of $SO(2)$.

6.1.3 Transforming point coordinates by rotation and translation

Let P be a geometric point fixed to the robot. Its position relative to O_b , expressed in body-frame coordinates, is ${}^b\mathbf{p}_P \in \mathbb{R}^2$ and is measured in metres. To obtain its world-frame position, first use $\mathbf{R}_{wb}$ to re-express this body-frame displacement along the world axes and then add the world-frame position of the body origin:

$$\boxed{{}^w\mathbf{p}_P = \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b.}$$

This equation follows by tracing the geometric displacement from O_w to P . First travel from O_w to O_b by ${}^w\mathbf{p}_b$. Then travel from O_b to P by the displacement whose body-frame coordinate is ${}^b\mathbf{p}_P$. Multiplication by $\mathbf{R}_{wb}$ expresses that same displacement in world coordinates so that the two terms can be added. This passive coordinate mapping does not physically move P .

A free vector, such as a displacement or velocity vector, has direction and magnitude but no fixed origin. It therefore changes coordinates using rotation alone:

$${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}.$$

Translation appears in the point equation because a point is located relative to a frame origin. It does not appear in the free-vector equation because a free vector is not attached to an origin.

6.1.4 Homogeneous transformations and $SE(2)$

The point mapping in Section 6.1.3 contains both a matrix multiplication and a vector addition. Homogeneous coordinates package these two operations into one matrix multiplication. Define the body- and world-frame homogeneous coordinate columns of point P by appending a 1:

$${}^b\overline{\mathbf{p}}_P := \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix} \in \mathbb{R}^3, \quad {}^w\overline{\mathbf{p}}_P := \begin{bmatrix} {}^w\mathbf{p}_P \\ 1 \end{bmatrix} \in \mathbb{R}^3.$$

The overline identifies a homogeneous coordinate representation, not a different geometric point. The first two entries have units of metres, while the appended 1 is dimensionless and exists only to make translation part of a matrix multiplication. Define the **homogeneous transformation** from frame $\{b\}$ to frame $\{w\}$ by

$$\mathbf{T}_{wb} := \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 3},$$

where $\mathbf{0}_2 = [0 \ 0]^\top \in \mathbb{R}^2$. The subscript order means that $\mathbf{T}_{wb}$ maps homogeneous point coordinates from frame $\{b\}$ to frame $\{w\}$:

$$\begin{aligned} {}^w\overline{\mathbf{p}}_P &= \mathbf{T}_{wb} {}^b\overline{\mathbf{p}}_P \\ &= \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b \\ 1 \end{bmatrix}. \end{aligned}$$

The upper two entries reproduce the point-transformation equation. Appending 1 causes the translation column to contribute to the result. Thus, $\mathbf{T}_{wb}$ re-expresses the point's body-relative displacement in world coordinates and then adds the world position of the body origin.

Using ${}^w\mathbf{p}_b = [x \ y]^\top$ and $\mathbf{R}_{wb} = \mathbf{R}_{wb}(\theta)$ gives the component form

$$\mathbf{T}_{wb} = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix}.$$

The rotation block is dimensionless, while the first two entries of the translation column have units of metres. The final row is an algebraic device that permits rotation and translation to be applied by one matrix multiplication; it does not represent another physical spatial dimension.

To preserve the rotation-only mapping for a free vector, define its homogeneous representation by appending a 0:

$${}^b\bar{\mathbf{v}} := \begin{bmatrix} {}^b\mathbf{v} \\ 0 \end{bmatrix}.$$

Then

$$\mathbf{T}_{wb} \begin{bmatrix} {}^b\mathbf{v} \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{v} \\ 0 \end{bmatrix}.$$

Appending 0 prevents the translation column from contributing. This algebraic distinction encodes the geometric fact that changing a point's reference frame requires rotation and translation, whereas changing the coordinates of a free vector requires only rotation.

The set of all planar homogeneous transformations is the **special Euclidean group in two dimensions**:

$$SE(2) := \left\{ \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \mid \mathbf{R} \in SO(2), \mathbf{p} \in \mathbb{R}^2 \right\}.$$

Here, *Euclidean* indicates rigid motions that preserve Euclidean distances and angles. Matrix multiplication is the group operation, the 3×3 identity $\mathbf{I}_3$ is the identity motion, and every element has an inverse that reverses the coordinate mapping. Section 6.1.6 derives that inverse explicitly.

6.1.5 Composing transformations

Introduce a third frame $\{c\}$. Let $\mathbf{T}_{ab} \in SE(2)$ map coordinates from frame $\{b\}$ to frame $\{a\}$, and let $\mathbf{T}_{bc} \in SE(2)$ map coordinates from frame $\{c\}$ to frame $\{b\}$. Applying the two mappings to a point gives

$${}^a\bar{\mathbf{p}}_P = \mathbf{T}_{ab} ({}^b\bar{\mathbf{p}}_P) = (\mathbf{T}_{ab}\mathbf{T}_{bc}) {}^c\bar{\mathbf{p}}_P.$$

Therefore, the transform that maps directly from frame $\{c\}$ to frame $\{a\}$ is

$$\boxed{\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}}.$$

The adjacent frame label b identifies the intermediate frame. Expanding the block multiplication gives

$$\begin{aligned}\mathbf{T}_{ab}\mathbf{T}_{bc} &= \begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{bc} & {}^b\mathbf{p}_c \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_{ab}\mathbf{R}_{bc} & \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix}.\end{aligned}$$

Consequently,

$$\boxed{\mathbf{R}_{ac} = \mathbf{R}_{ab}\mathbf{R}_{bc}, \quad {}^a\mathbf{p}_c = \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b}.$$

The translation vectors cannot be added directly until they are expressed in the same frame. The product first re-expresses ${}^b\mathbf{p}_c$ in frame $\{a\}$ and only then adds ${}^a\mathbf{p}_b$.

6.1.6 Inverting a transformation

The inverse transform $\mathbf{T}_{ba} = \mathbf{T}_{ab}^{-1}$ must undo both the translation and the rotation. Starting from the point mapping

$${}^a\mathbf{p}_P = \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b,$$

subtract ${}^a\mathbf{p}_b$ and premultiply by $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top$:

$${}^b\mathbf{p}_P = \mathbf{R}_{ab}^\top ({}^a\mathbf{p}_P - {}^a\mathbf{p}_b).$$

Distributing the rotation gives

$${}^b\mathbf{p}_P = \mathbf{R}_{ab}^\top {}^a\mathbf{p}_P - \mathbf{R}_{ab}^\top {}^a\mathbf{p}_b.$$

Therefore,

$$\boxed{\mathbf{T}_{ab}^{-1} = \mathbf{T}_{ba} = \begin{bmatrix} \mathbf{R}_{ab}^\top & -\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix}}.$$

The inverse translation is not generally $-{}^a\mathbf{p}_b$ because that column is expressed in frame $\{a\}$. It must first be negated and then expressed in frame $\{b\}$ by $\mathbf{R}_{ab}^\top$.

6.1.7 Pose coordinates versus the pose itself

The familiar column

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

has the following meaning:

- $x \in \mathbb{R}$ is the x_w -coordinate of the body origin O_b , expressed in the world frame and measured in metres;
- $y \in \mathbb{R}$ is the y_w -coordinate of O_b , expressed in the world frame and measured in metres; and
- $\theta \in \mathbb{R}$ is the counter-clockwise orientation of body frame $\{b\}$ relative to world frame $\{w\}$, measured in radians.

Therefore, the position of the body origin relative to the world origin, expressed in world-frame coordinates, is

$${}^w\mathbf{p}_b = \begin{bmatrix} x \\ y \end{bmatrix}.$$

The notation ${}^w\mathbf{p}_b$ means “the position of the body origin O_b , expressed in world-frame coordinates.” Thus, $\mathbf{q}$ is a coordinate description of the pose of frame $\{b\}$ relative to frame $\{w\}$. The same pose can be represented by $\mathbf{T}_{wb} \in SE(2)$:

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \longmapsto \mathbf{T}_{wb}(\mathbf{q}) = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix}.$$

The coordinate column is useful for storage, plotting, and differential equations, but it is not an ordinary geometric vector. Its components represent different physical quantities, θ and $\theta + 2\pi n$ represent the same orientation, and componentwise addition does not in general reproduce rigid-motion composition. Transformations should be composed by matrix multiplication with explicit frame labels.

6.1.8 Worked example

Suppose the body origin is at

$${}^w\mathbf{p}_b = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \text{ m}$$

and the body heading is $\theta = \pi/2$ rad. A point P fixed one metre along the body $+x_b$ -axis has body-frame coordinates

$${}^b\mathbf{p}_P = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ m.}$$

The rotation and homogeneous transformation are

$$\mathbf{R}_{wb} \left(\frac{\pi}{2} \right) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{T}_{wb} = \begin{bmatrix} 0 & -1 & 2 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

Applying the transform gives

$$\begin{aligned} {}^w\bar{\mathbf{p}}_P &= \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P \\ &= \begin{bmatrix} 0 & -1 & 2 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}. \end{aligned}$$

Thus, ${}^w\mathbf{p}_P = [2 \ 2]^\top$ m. The one-metre body-frame displacement is rotated into the world $+y_w$ -direction and then added to the body origin at $[2 \ 1]^\top$ m.

Quick test A: pose and $SE(2)$

1. What geometric information is contained in the pair $(\mathbf{R}_{wb}, {}^w\mathbf{p}_b)$, and what does the subscript order wb mean?
2. Define $SO(2)$. What do its orthogonality and determinant conditions exclude or guarantee?
3. Why do θ and $\theta + 2\pi$ describe the same orientation?

4. Starting from the block definitions of $\mathbf{T}_{ab}$ and $\mathbf{T}_{bc}$, derive the translation column of $\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}$.
5. Why is the inverse translation $-\mathbf{R}_{ab}^T {}^a\mathbf{p}_b$ rather than simply $-{}^a\mathbf{p}_b$?
6. A body frame has world position $[3 \ 2]^T$ m and heading $-\pi/2$ rad. Find the world coordinates of a point whose body coordinates are $[0.5 \ 0]^T$ m.
7. Explain why $\mathbf{q}_1 + \mathbf{q}_2$ is not generally the composition of the two poses represented by $\mathbf{q}_1$ and $\mathbf{q}_2$.
8. Name two transformations that cannot be represented by an element of $SE(2)$ and explain why.

Answers and hints

1. $\mathbf{R}_{wb}$ describes the orientation of frame $\{b\}$ relative to frame $\{w\}$, while ${}^w\mathbf{p}_b$ locates O_b relative to O_w using world coordinates. The order wb means that the rotation maps coordinate columns from $\{b\}$ to $\{w\}$.
2. $SO(2) = \{\mathbf{R} \in \mathbb{R}^{2 \times 2} \mid \mathbf{R}^T \mathbf{R} = \mathbf{I}_2, \det \mathbf{R} = 1\}$. Orthogonality preserves lengths and angles and gives $\mathbf{R}^{-1} = \mathbf{R}^T$; determinant $+1$ preserves orientation and excludes reflections.
3. Sine and cosine are 2π -periodic, so every entry of $\mathbf{R}(\theta + 2\pi)$ equals the corresponding entry of $\mathbf{R}(\theta)$.
4. Block multiplication gives ${}^a\mathbf{p}_c = \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b$. The first term re-expresses the b -frame translation in frame $\{a\}$ before the two translations are added.
5. The reversed displacement must be expressed in frame $\{b\}$, whereas ${}^a\mathbf{p}_b$ is expressed in frame $\{a\}$. Premultiplication by $\mathbf{R}_{ab}^T$ performs the necessary coordinate change.
6. $\mathbf{R}_{wb}(-\pi/2)[0.5 \ 0]^T = [0 \ -0.5]^T$ m, so ${}^w\mathbf{p}_P = [3 \ 1.5]^T$ m.
7. Rigid-motion composition must account for the frame in which each translation is expressed and for the order of rotations and translations. Componentwise addition does neither, and angular coordinates are periodic.
8. Examples include nonuniform scaling, shear, and elastic deformation. They do not preserve every distance and angle, so they are not rigid motions and do not belong to $SE(2)$.

6.2 Passive coordinate changes and active rigid motions

The same numerical rotation or homogeneous-transformation matrix can appear in different geometric questions. Correct interpretation requires identifying what remains fixed, what changes, and the frame in which every coordinate column is expressed.

6.2.1 Passive coordinate change

A **passive coordinate change** keeps the geometric point, vector, or rigid body unchanged and changes only the coordinate frame used to describe it. The word *passive* indicates that nothing physically moves.

For example, let the geometric point P remain fixed while its coordinates are changed from frame $\{b\}$ to frame $\{w\}$. The passive point-coordinate transformation is

$${}^w\bar{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P.$$

The two coordinate columns ${}^b\bar{\mathbf{p}}_P$ and ${}^w\bar{\mathbf{p}}_P$ contain different numbers in general, but both describe the same geometric point P . The transform $\mathbf{T}_{wb}$ accounts for the position and orientation of frame $\{b\}$ relative to frame $\{w\}$.

For a free vector $\mathbf{v}$, translation is irrelevant and only the rotation block is used:

$${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}.$$

Suppose frame $\{b\}$ is rotated counter-clockwise by $\pi/2$ relative to frame $\{w\}$ and the vector lies along $+x_b$. Its world-frame coordinates are

$$\begin{aligned} {}^w\mathbf{v} &= \mathbf{R}_{wb} {}^b\mathbf{v} \\ &= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \end{aligned}$$

The vector has not rotated physically. It is the same geometric vector described once using the axes of $\{b\}$ and once using the axes of $\{w\}$.

6.2.2 Active rigid-body motion

An **active transformation** keeps the reference frame fixed and moves a geometric point, vector, or rigid body. The initial point P and final point P' are different geometric points, but both coordinate columns are expressed in the same frame.

Let $\mathbf{R}_\Delta \in SO(2)$ be an applied planar rotation and let $\mathbf{d}_\Delta \in \mathbb{R}^2$ be an applied translation expressed in world-frame coordinates and measured in metres. Define the active rigid-motion operator

$$\mathbf{T}_\Delta := \begin{bmatrix} \mathbf{R}_\Delta & \mathbf{d}_\Delta \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in SE(2).$$

Applied to point P , it produces the new point P' according to

$$\boxed{{}^w\bar{\mathbf{p}}_{P'} = \mathbf{T}_\Delta {}^w\bar{\mathbf{p}}_P},$$

or, using ordinary two-dimensional coordinates,

$${}^w\mathbf{p}_{P'} = \mathbf{R}_\Delta {}^w\mathbf{p}_P + \mathbf{d}_\Delta.$$

Both sides are expressed in frame $\{w\}$. The point changes from P to P' , unlike a passive coordinate change, where the point remains P and the leading frame superscript changes.

For example, let $\mathbf{R}_\Delta = \mathbf{R}(\pi/2)$, $\mathbf{d}_\Delta = [2 \ 1]^\top \text{ m}$, and ${}^w\mathbf{p}_P = [1 \ 0]^\top \text{ m}$. Then

$${}^w\mathbf{p}_{P'} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \text{ m}.$$

Here, the coordinates changed because the geometric point moved, not because a different coordinate frame was selected.

6.2.3 One matrix form, three interpretations

A homogeneous transformation may describe a relative pose, perform a passive coordinate change, or act as an active rigid-motion operator. The matrix arithmetic can be identical while the geometric meaning differs.

Interpretation	What changes?	Representative statement
Relative-pose description	Nothing is being operated on; the matrix records the pose of one frame relative to another.	$\mathbf{T}_{wb}$ describes frame $\{b\}$ relative to frame $\{w\}$.
Passive coordinate change	The coordinate column and its frame label change; the geometric point remains fixed.	${}^w\bar{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P$.

Interpretation	What changes?	Representative statement
Active rigid motion	The geometric point changes; the coordinate frame remains fixed.	${}^w\bar{\mathbf{p}}_{P'} = \mathbf{T}_\Delta {}^w\bar{\mathbf{p}}_P$.

The project notation reserves labelled transforms such as $\mathbf{T}_{wb}$ for relative poses and coordinate mappings. A symbol such as $\mathbf{T}_\Delta$ must be explicitly declared when the same matrix form is used as an active motion operator.

6.2.4 Composition order

Transformations act on column vectors from the left, so the rightmost matrix acts first. For a passive coordinate chain,

$${}^a\bar{\mathbf{p}}_P = \mathbf{T}_{ab}\mathbf{T}_{bc} {}^c\bar{\mathbf{p}}_P,$$

the rightmost transform $\mathbf{T}_{bc}$ first changes coordinates from frame $\{c\}$ to frame $\{b\}$, and $\mathbf{T}_{ab}$ then changes them from frame $\{b\}$ to frame $\{a\}$. Thus,

$$\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}.$$

For two active motions $\mathbf{T}_1$ and $\mathbf{T}_2$,

$${}^w\bar{\mathbf{p}}_{P''} = \mathbf{T}_2\mathbf{T}_1 {}^w\bar{\mathbf{p}}_P,$$

motion $\mathbf{T}_1$ occurs first and motion $\mathbf{T}_2$ occurs second. Associativity permits the parentheses in a longer product to be regrouped, but it does not permit the matrix order to be reversed.

6.2.5 Why composition in $SE(2)$ is not commutative

Planar rotations alone commute, but planar rigid transformations generally do not because rotation changes the direction of a translation. Consider a translation of one metre along $+x_w$ and a counter-clockwise rotation of $\pi/2$ about O_w :

$$\mathbf{T}_D = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{T}_R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Translate first and rotate second:

$$\mathbf{T}_R \mathbf{T}_D = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

Rotate first and translate second:

$$\mathbf{T}_D \mathbf{T}_R = \begin{bmatrix} 0 & -1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The products differ. Applied to the world origin ${}^w\bar{\mathbf{p}}_O = [0 \ 0 \ 1]^T$, they give

$$\mathbf{T}_R \mathbf{T}_D {}^w\bar{\mathbf{p}}_O = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{T}_D \mathbf{T}_R {}^w\bar{\mathbf{p}}_O = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Therefore,

$$\boxed{\mathbf{T}_R \mathbf{T}_D \neq \mathbf{T}_D \mathbf{T}_R}.$$

This noncommutativity is why a transformation sequence must state both the individual motions and their order.

6.2.6 Interpretation failures and engineering checks

- **Missing frame labels:** An equation such as $\mathbf{p}' = \mathbf{T}\mathbf{p}$ does not reveal whether $\mathbf{T}$ changes coordinates or moves the point. Declare the interpretation and frames.
- **Applying transformations in the wrong order:** For column-vector notation, transformations act from right to left. In $\mathbf{T}_2 \mathbf{T}_1 \mathbf{p}$, $\mathbf{T}_1$ acts first and $\mathbf{T}_2$ acts second. Verify an implementation using a point whose expected final location is easy to calculate.
- **Mixing coordinate frames:** Translation columns may be added only after they are expressed in the same frame.
- **Assuming wrong commutativity:** Reordering a rotation and translation generally changes the resulting pose.

Quick test B: interpretation and composition

1. Define a passive coordinate change. What changes, and what remains fixed?
2. Define an active rigid-body motion. What changes, and what remains fixed?
3. Explain the three interpretations of a homogeneous transformation used in this chapter.
4. In $\mathbf{T}_{ab}\mathbf{T}_{bc}^c\bar{\mathbf{p}}_P$, which matrix acts first, and what frame sequence does the product represent?
5. For active motions, what sequence is represented by $\mathbf{T}_2\mathbf{T}_1^w\bar{\mathbf{p}}_P$?
6. Using $\mathbf{T}_D$ and $\mathbf{T}_R$ from Section 6.2.5, explain geometrically why applying them to the origin in opposite orders gives different results.
7. Does the commutativity of planar rotations imply that all elements of $SE(2)$ commute? Explain.
8. A program receives an unlabelled matrix $\mathbf{T}$ and an unlabelled coordinate column $\mathbf{p}$. What information is missing before the multiplication can be interpreted safely?

Answers and hints

1. A passive coordinate change preserves the geometric object and changes only the frame used to express its coordinates. The coordinate values and frame label may change.
2. An active rigid-body motion preserves the reference frame and moves the geometric object. The initial and final coordinates describe different points or poses in the same frame.
3. A homogeneous transformation can describe the pose of one frame relative to another, map coordinates of an unchanged object between frames, or actively move an object while its coordinates remain expressed in one frame.
4. $\mathbf{T}_{bc}$ acts first, mapping from $\{c\}$ to $\{b\}$; $\mathbf{T}_{ab}$ then maps from $\{b\}$ to $\{a\}$. The complete product maps from $\{c\}$ to $\{a\}$.
5. $\mathbf{T}_1$ moves the point first, and $\mathbf{T}_2$ moves the resulting point second.
6. Translating the origin first places it at $[1 \ 0]^T$, which the subsequent rotation moves to $[0 \ 1]^T$. Rotating the origin first leaves it at the origin, after which translation places it at $[1 \ 0]^T$.
7. No. Although $SO(2)$ rotations commute with each other, an $SE(2)$ transformation also contains translation. Rotation changes the direction of a translation, so general rotation-translation products do not commute.
8. The program needs to know whether $\mathbf{p}$ represents a point or free vector, the frame in which it is expressed, the source and destination frames or active-motion interpretation of $\mathbf{T}$, the units, and the intended multiplication order.

6.3 Time-varying rotations and rigid-body point velocity

A pose describes where a rigid body is at one instant. A time-varying pose describes motion. This section differentiates the position and orientation components of $\mathbf{T}_{wb}(t)$ and derives the velocity of any point fixed to the body.

6.3.1 Translational and angular velocity

Let the planar body pose vary continuously with time:

$$\mathbf{T}_{wb}(t) = \begin{bmatrix} \mathbf{R}_{wb}(t) & {}^w\mathbf{p}_b(t) \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in SE(2),$$

where $t \in \mathbb{R}_{\geq 0}$ is time in seconds. The world-frame position of the body origin is

$${}^w\mathbf{p}_b(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} \in \mathbb{R}^2.$$

Its derivative, taken with respect to the fixed world frame and expressed in world-frame coordinates, is the translational velocity of O_b :

$$\boxed{{}^w\mathbf{v}_b(t) := \frac{d}{dt} {}^w\mathbf{p}_b(t) = \begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \end{bmatrix}} \in \mathbb{R}^2.$$

The entries of ${}^w\mathbf{v}_b$ are measured in m s^{-1} . The heading is $\theta(t) \in \mathbb{R}$, measured in radians, and its derivative is the signed yaw rate

$$\boxed{\omega(t) := \dot{\theta}(t)} \in \mathbb{R},$$

measured in rad s^{-1} . Positive ω denotes counter-clockwise rotation about the common $+z$ -axis of the planar frames.

6.3.2 Deriving the rotation-matrix derivative

The time-varying planar rotation matrix is

$$\mathbf{R}_{wb}(t) = \begin{bmatrix} \cos \theta(t) & -\sin \theta(t) \\ \sin \theta(t) & \cos \theta(t) \end{bmatrix}.$$

Differentiate every entry and apply the chain rule:

$$\begin{aligned} \dot{\mathbf{R}}_{wb}(t) &= \dot{\theta}(t) \begin{bmatrix} -\sin \theta(t) & -\cos \theta(t) \\ \cos \theta(t) & -\sin \theta(t) \end{bmatrix} \\ &= \omega(t) \begin{bmatrix} -\sin \theta(t) & -\cos \theta(t) \\ \cos \theta(t) & -\sin \theta(t) \end{bmatrix}. \end{aligned}$$

A square matrix $\mathbf{A}$ is **skew-symmetric** when its transpose equals its negative:

$$\boxed{\mathbf{A}^\top = -\mathbf{A}}.$$

Transposition reflects the entries across the main diagonal. The condition therefore means that entries mirrored across the diagonal have equal magnitudes and opposite signs; every diagonal entry must be zero. Define the dimensionless planar skew-symmetric generator

$$\boxed{\mathbf{S}_2 := \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}}.$$

The matrix $\mathbf{S}_2$ is skew-symmetric because $\mathbf{S}_2^\top = -\mathbf{S}_2$. Premultiplication by $\mathbf{S}_2$ rotates any planar coordinate column counter-clockwise by $\pi/2$:

$$\mathbf{S}_2 \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} -b \\ a \end{bmatrix}.$$

Direct multiplication gives

$$\mathbf{S}_2 \mathbf{R}_{wb}(\theta) = \mathbf{R}_{wb}(\theta) \mathbf{S}_2 = \begin{bmatrix} -\sin \theta & -\cos \theta \\ \cos \theta & -\sin \theta \end{bmatrix}.$$

Therefore,

$$\boxed{\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb} = \mathbf{R}_{wb} \omega \mathbf{S}_2}.$$

The two products are equal in the planar case because every element of $SO(2)$ commutes with S_2 . This equality is a special simplification of planar rotation; matrix order becomes important for three-dimensional angular velocity.

6.3.3 Why $\dot{\mathbf{R}}_{wb}$ is not a rotation matrix

Although $\mathbf{R}_{wb}(t) \in SO(2)$ at every time t , its derivative $\dot{\mathbf{R}}_{wb}(t)$ is generally not a rotation matrix. A rotation matrix is a dimensionless map that represents a finite change of orientation, whereas $\dot{\mathbf{R}}_{wb}$ measures how rapidly that map is changing and has entries with units of s^{-1} . The derivative therefore describes an instantaneous direction of rotational motion rather than another finite rotation.

6.3.3.1 The rotation constraint restricts its derivative

Every planar rotation matrix satisfies

$$\mathbf{R}_{wb}^\top \mathbf{R}_{wb} = \mathbf{I}_2.$$

Differentiating this constraint with respect to time gives

$$\dot{\mathbf{R}}_{wb}^\top \mathbf{R}_{wb} + \mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = \mathbf{0}_{2 \times 2}.$$

Rearranging shows that

$$(\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb})^\top = -\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb}.$$

Thus, $\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb}$ is **skew-symmetric**: its transpose equals its negative. The factor $\mathbf{R}_{wb}^\top$ expresses the rotational rate relative to the current body orientation; it does not turn $\dot{\mathbf{R}}_{wb}$ into a finite rotation.

The set of all 2×2 real skew-symmetric matrices is

$$\mathfrak{so}(2) := \{\mathbf{A} \in \mathbb{R}^{2 \times 2} \mid \mathbf{A}^\top = -\mathbf{A}\} = \{\alpha \mathbf{S}_2 \mid \alpha \in \mathbb{R}\},$$

where α is a real scalar and

$$\mathbf{S}_2 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

To justify the second equality, begin with a general matrix

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad a, b, c, d \in \mathbb{R}.$$

The condition $\mathbf{A}^\top = -\mathbf{A}$ requires corresponding entries to satisfy

$$a = -a, \quad d = -d, \quad c = -b.$$

Therefore, $a = d = 0$. Setting $\alpha = c = -b$ gives

$$\mathbf{A} = \begin{bmatrix} 0 & -\alpha \\ \alpha & 0 \end{bmatrix} = \alpha \mathbf{S}_2.$$

Thus, every 2×2 skew-symmetric matrix is a scalar multiple of $\mathbf{S}_2$. Conversely, for every $\alpha \in \mathbb{R}$,

$$(\alpha \mathbf{S}_2)^\top = \alpha \mathbf{S}_2^\top = -\alpha \mathbf{S}_2,$$

so every scalar multiple of $\mathbf{S}_2$ is skew-symmetric. The two sets are therefore equal. The notation $\mathfrak{so}(2)$ is read “little s o two.”

6.3.3.2 Why these matrices form a tangent space

The skew-symmetric matrix $\mathbf{S}_2$ is tangent to $SO(2)$ at the identity rotation $\mathbf{I}_2$. More generally, every scalar multiple $\alpha \mathbf{S}_2$, where $\alpha \in \mathbb{R}$, is a possible tangent direction there. This subsection explains the geometric meaning of that statement and proves that these are all the possible tangent directions.

A **tangent direction** to a curve at a point is the limiting direction of secant lines drawn from that point to nearby points on the curve. Equivalently, it is the derivative at that point of a differentiable path that remains on the curve. This familiar definition applies to $SO(2)$ because every rotation matrix $\mathbf{R}(\theta)$ is determined by the pair $(\cos \theta, \sin \theta)$ on the unit circle. Under this correspondence,

$$\mathbf{R}(0) = \mathbf{I}_2 \quad \longleftrightarrow \quad (\cos 0, \sin 0) = (1, 0).$$

For a nonzero angular increment h measured in radians, the secant direction of the unit circle from $(1, 0)$ to $(\cos h, \sin h)$ is

$$\frac{1}{h} \left(\begin{bmatrix} \cos h \\ \sin h \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right).$$

As $h \rightarrow 0$, this secant direction approaches $[0 \ 1]^T$, the usual tangent direction to the circle at $(1, 0)$. The corresponding difference quotient in matrix coordinates is

$$\frac{\mathbf{R}(h) - \mathbf{I}_2}{h} = \begin{bmatrix} \frac{\cos h - 1}{h} & -\frac{\sin h}{h} \\ \frac{\sin h}{h} & \frac{\cos h - 1}{h} \end{bmatrix}.$$

The scalar limits

$$\lim_{h \rightarrow 0} \frac{\sin h}{h} = 1, \quad \lim_{h \rightarrow 0} \frac{\cos h - 1}{h} = 0$$

give

$$\left. \frac{d\mathbf{R}(h)}{dh} \right|_{h=0} = \lim_{h \rightarrow 0} \frac{\mathbf{R}(h) - \mathbf{I}_2}{h} = \mathbf{S}_2.$$

The second scalar limit follows from $\cos h - 1 = -2 \sin^2(h/2)$, the first limit, and the continuity of sine at zero. Therefore, $\mathbf{S}_2$ is tangent to the rotation curve at $\mathbf{I}_2$ in exactly the same sense that $[0 \ 1]^T$ is tangent to the unit circle at $(1, 0)$.

To collect every possible tangent direction, let $\mathbf{Q}(s) \in SO(2)$ be any differentiable rotation curve defined for the dimensionless real parameter s near zero, with $\mathbf{Q}(0) = \mathbf{I}_2$. The **tangent space of $SO(2)$ at $\mathbf{I}_2$** is

$$T_{\mathbf{I}_2} SO(2) := \left\{ \left. \frac{d\mathbf{Q}(s)}{ds} \right|_{s=0} \mid \mathbf{Q}(s) \in SO(2), \mathbf{Q}(0) = \mathbf{I}_2 \right\}.$$

The symbol T denotes a tangent space, and its subscript $\mathbf{I}_2$ identifies the point where the tangent directions are attached. To prove that this space equals $\mathfrak{so}(2)$, both set inclusions must be established: every tangent direction must be shown to be skew-symmetric, and every skew-symmetric matrix must be shown to arise from a valid rotation curve.

First, take any $\mathbf{A} \in T_{\mathbf{I}_2} SO(2)$. By definition, $\mathbf{A} = \mathbf{Q}'(0)$ for some rotation curve $\mathbf{Q}(s)$ through $\mathbf{I}_2$. Differentiating $\mathbf{Q}(s)^T \mathbf{Q}(s) = \mathbf{I}_2$ gives

$$\mathbf{Q}'(s)^T \mathbf{Q}(s) + \mathbf{Q}(s)^T \mathbf{Q}'(s) = \mathbf{0}_{2 \times 2}.$$

Now evaluate at $(s=0)$. Because the curve passes through the identity,

$$\mathbf{Q}(0) = \mathbf{I}_2, \quad \mathbf{Q}'(0) = \mathbf{A}.$$

Substitution gives

$$\mathbf{A}^\top + \mathbf{A} = \mathbf{0}_{2 \times 2}.$$

Thus, $\mathbf{A}$ is skew-symmetric and belongs to $\mathfrak{so}(2)$. This proves

$$T_{\mathbf{I}_2} SO(2) \subseteq \mathfrak{so}(2).$$

Second, take any $\mathbf{A} \in \mathfrak{so}(2)$. It has the form $\mathbf{A} = \alpha \mathbf{S}_2$ for some $\alpha \in \mathbb{R}$. The curve $\mathbf{Q}(s) = \mathbf{R}(\alpha s)$ remains in $SO(2)$, passes through $\mathbf{I}_2$ at $s = 0$, and satisfies

$$\mathbf{Q}'(0) = \left. \frac{d}{ds} \mathbf{R}(\alpha s) \right|_{s=0} = \alpha \mathbf{S}_2 = \mathbf{A}.$$

Hence, every element of $\mathfrak{so}(2)$ is produced by a valid curve through the identity, which proves

$$\mathfrak{so}(2) \subseteq T_{\mathbf{I}_2} SO(2).$$

The two inclusions establish

$$T_{\mathbf{I}_2} SO(2) = \mathfrak{so}(2) = \{\alpha \mathbf{S}_2 \mid \alpha \in \mathbb{R}\}.$$

This proves that the skew-symmetric matrices in $\mathfrak{so}(2)$ are exactly the instantaneous directions in which a rotation curve can pass through $\mathbf{I}_2$. They describe rotational directions, not finite rotations. The next subsection shows how these directions provide a first-order approximation to nearby rotation matrices.

6.3.3.3 Lie-group terminology and the local linear model

Section 6.1.2 established that $SO(2)$ is a group under matrix multiplication. Its matrices also vary smoothly with the continuous angle θ , and its multiplication and inverse operations are differentiable. A group with such a compatible smooth structure is called a **Lie group**. The tangent space at the identity of a Lie group, together with an operation called the Lie bracket, is its **Lie algebra**. Thus, $\mathfrak{so}(2)$ is the Lie algebra of $SO(2)$.

The Lie algebra is useful because it is linear even though the rotation group is not. For any $\alpha, \beta, c \in \mathbb{R}$,

$$\alpha \mathbf{S}_2 + \beta \mathbf{S}_2 = (\alpha + \beta) \mathbf{S}_2, \quad c(\alpha \mathbf{S}_2) = (c\alpha) \mathbf{S}_2,$$

and both results remain in $\mathfrak{so}(2)$. Instantaneous rotational directions can therefore be added and scaled using ordinary linear algebra, whereas adding or scaling finite rotation matrices does not generally produce valid rotations.

The tangent space is also a local linear approximation of the group. For a small angle h ,

$$\boxed{\mathbf{R}(h) = \mathbf{I}_2 + h\mathbf{S}_2 + O(h^2)}.$$

This result follows more directly from the first-order Taylor expansion of the matrix-valued function $\mathbf{R}(h)$ about $h = 0$:

$$\mathbf{R}(h) = \mathbf{R}(0) + h \left. \frac{d\mathbf{R}(h)}{dh} \right|_{h=0} + O(h^2).$$

The definition of $\mathbf{R}(h)$ and the preceding limit give

$$\mathbf{R}(0) = \mathbf{I}_2, \quad \left. \frac{d\mathbf{R}(h)}{dh} \right|_{h=0} = \mathbf{S}_2.$$

Substituting these two values immediately gives $\mathbf{R}(h) = \mathbf{I}_2 + h\mathbf{S}_2 + O(h^2)$. The derivative is written $d\mathbf{R}/dh$ rather than $\dot{\mathbf{R}}$ because h is an angular argument, not time. For this matrix equation, $O(h^2)$ means that the norm of the Taylor remainder is bounded by $C|h|^2$ for some constant $C > 0$ when h is sufficiently close to zero; this bound exists because the sine and cosine entries of $\mathbf{R}(h)$ have continuous second derivatives near $h = 0$.

If the same rotation increment is instead parameterised by time as $h(t) = \omega t$ with constant angular velocity ω , then the time-domain Taylor expansion over a short interval Δt is

$$\mathbf{R}(\omega\Delta t) = \mathbf{R}(0) + \Delta t \dot{\mathbf{R}}(0) + O(\Delta t^2) = \mathbf{I}_2 + \omega\Delta t \mathbf{S}_2 + O(\Delta t^2),$$

because $\dot{\mathbf{R}}(0) = \omega\mathbf{S}_2$. This is the time-parameterised version of the same first-order approximation.

In the angle-domain approximation, $\mathbf{I}_2 + h\mathbf{S}_2$ starts at $\mathbf{I}_2$ and changes by $h\mathbf{S}_2$ along the tangent direction. It is generally not an exact rotation matrix, but its difference from the true rotation is only of order h^2 . The approximation therefore matches both the value and the first derivative of the rotation curve at $h = 0$.

6.3.3.4 Returning to the physical rotation trajectory

For the physical trajectory $\mathbf{R}_{wb}(t)$, Section 6.3.2 established

$$\dot{\mathbf{R}}_{wb} = \omega \mathbf{R}_{wb} \mathbf{S}_2.$$

Thus, $\dot{\mathbf{R}}_{wb}$ is tangent to $SO(2)$ at the current rotation $\mathbf{R}_{wb}$, rather than at the identity. Premultiplication by $\mathbf{R}_{wb}^\top$ carries this direction back to the tangent space at the identity:

$$\boxed{\boldsymbol{\Omega}_b := \mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \in \mathfrak{so}(2)}.$$

Here, $\boldsymbol{\Omega}_b$ is the body-frame angular-velocity matrix, and its entries have units of s^{-1} . The time-domain Taylor expansion above connects this instantaneous generator $\omega \mathbf{S}_2$ to a nearby finite rotation. The derivative is not itself a rotation matrix; it specifies the tangent direction from which the finite rotation develops over time.

Finally, the **Lie bracket** measures whether two infinitesimal matrix motions commute. For matrices it is the commutator

$$[\mathbf{A}, \mathbf{B}] := \mathbf{AB} - \mathbf{BA}, \quad \mathbf{A}, \mathbf{B} \in \mathfrak{so}(2).$$

For $\mathfrak{so}(2)$, write $\mathbf{A} = \alpha \mathbf{S}_2$ and $\mathbf{B} = \beta \mathbf{S}_2$. Then $\mathbf{AB} = \mathbf{BA}$, so $[\mathbf{A}, \mathbf{B}] = \mathbf{0}_{2 \times 2}$. The bracket is trivial for planar rotations because they commute, but it becomes important for rigid motions in $SE(2)$ and rotations in $SO(3)$, where motion order generally matters.

6.3.4 Velocity of a point fixed to the body

Let P be a point rigidly fixed to the body. Its body-frame coordinates ${}^b \mathbf{p}_P \in \mathbb{R}^2$ are constant, so

$$\frac{d}{dt} {}^b \mathbf{p}_P = \mathbf{0}_2.$$

Its world-frame position is

$${}^w \mathbf{p}_P(t) = \mathbf{R}_{wb}(t) {}^b \mathbf{p}_P + {}^w \mathbf{p}_b(t).$$

Differentiate both sides:

$$\begin{aligned} {}^w\mathbf{v}_P &:= \frac{d}{dt} {}^w\mathbf{p}_P = \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P + \mathbf{R}_{wb} \frac{d}{dt} {}^b\mathbf{p}_P + {}^w\mathbf{v}_b \\ &= \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{v}_b. \end{aligned}$$

Define the relative-position vector from O_b to P , expressed in world-frame coordinates, by

$${}^w\mathbf{r}_{O_bP} := {}^w\mathbf{p}_P - {}^w\mathbf{p}_b = \mathbf{R}_{wb} {}^b\mathbf{p}_P \in \mathbb{R}^2.$$

Using $\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb}$ gives

$$\boxed{{}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \omega \mathbf{S}_2 {}^w\mathbf{r}_{O_bP}}.$$

The first term is the translational velocity shared with the body origin. The second term is the additional velocity caused by rotation about the body origin. Because $\mathbf{S}_2$ rotates ${}^w\mathbf{r}_{O_bP}$ by $\pi/2$, the rotational contribution is perpendicular to the relative-position vector.

The two panels in Figure 6.1 separate the position-vector identity from the velocity-vector addition. In the left panel, the two solid vectors add head to tail to produce the dashed world-position vector of P . In the right panel, translated copies of the two velocity contributions form a parallelogram whose diagonal is ${}^w\mathbf{v}_P$; the right-angle marker applies specifically to the rotational contribution and ${}^w\mathbf{r}_{O_bP}$.

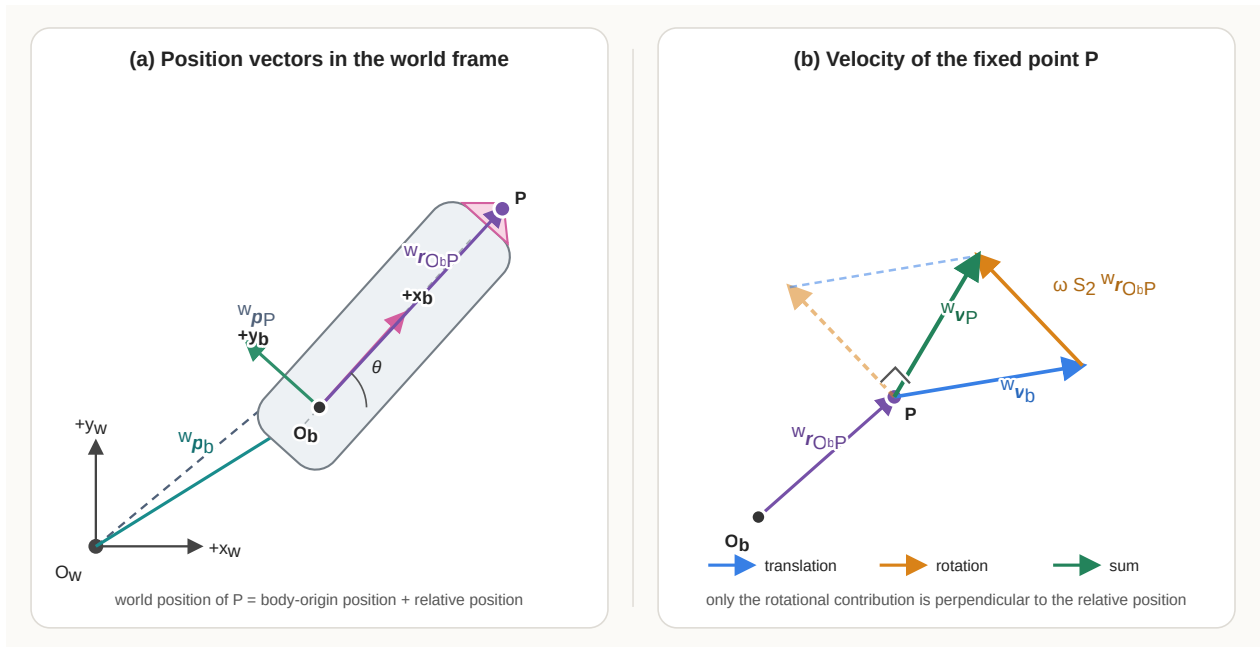


Figure 6.1: Position and velocity relations for a point fixed to a planar rigid body.

The same result can be written with the three-dimensional cross product by embedding the planar quantities in $\mathbb{R}^3$:

$${}^w\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix}, \quad {}^w\mathbf{r}_{O_b P}^{(3)} = \begin{bmatrix} r_x \\ r_y \\ 0 \end{bmatrix}.$$

Here, r_x and r_y , measured in metres, are the two entries of ${}^w\mathbf{r}_{O_b P}$, the superscript (3) marks its three-dimensional embedding, and $\times$ denotes the vector cross product.

Then

$${}^w\boldsymbol{\omega} \times {}^w\mathbf{r}_{O_b P}^{(3)} = \begin{bmatrix} -\omega r_y \\ \omega r_x \\ 0 \end{bmatrix},$$

whose first two entries equal $\omega \mathbf{S}_2 {}^w\mathbf{r}_{O_b P}$. Thus, the planar formula is the two-dimensional form of the rigid-body velocity relation

$$\mathbf{v}_P = \mathbf{v}_{O_b} + \boldsymbol{\omega} \times \mathbf{r}_{O_b P}.$$

6.3.5 Worked example

At one instant, suppose the body heading is $\theta = \pi/2$ rad, the body-origin velocity is

$${}^w\mathbf{v}_b = \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} \text{ m s}^{-1},$$

and the yaw rate is $\omega = 2 \text{ rad s}^{-1}$. Point P is fixed one metre along the body $+x_b$ -axis:

$${}^b\mathbf{p}_P = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ m}.$$

At $\theta = \pi/2$, its relative-position vector in the world frame is

$${}^w\mathbf{r}_{O_b P} = \mathbf{R}_{wb} \left(\frac{\pi}{2} \right) {}^b\mathbf{p}_P = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \text{ m}.$$

The rotational contribution is

$$\omega \mathbf{S}_2 {}^w \mathbf{r}_{O_b P} = 2 \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Therefore,

$$\begin{aligned} {}^w \mathbf{v}_P &= {}^w \mathbf{v}_b + \omega \mathbf{S}_2 {}^w \mathbf{r}_{O_b P} \\ &= \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} + \begin{bmatrix} -2 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} -1.5 \\ 0 \end{bmatrix} \text{ m s}^{-1}. \end{aligned}$$

The point lies one metre above the body origin in world coordinates. Counter-clockwise rotation therefore moves it instantaneously toward $-x_w$ at 2 m s^{-1} , while translation of the body origin contributes 0.5 m s^{-1} toward $+x_w$.

6.3.6 Assumptions, failure modes, and checks

- **Body-fixed point assumption:** The derivation sets $d({}^b \mathbf{p}_P)/dt = \mathbf{0}_2$. If P moves relative to the body, the additional term $\mathbf{R}_{wb} d({}^b \mathbf{p}_P)/dt$ must be retained.
- **Frame consistency:** ${}^w \mathbf{v}_b$ and $\omega \mathbf{S}_2 {}^w \mathbf{r}_{O_b P}$ must be expressed in the same frame before they are added.
- **Yaw sign:** With the stated convention, $\omega > 0$ produces counter-clockwise rotation. A sign error reverses the rotational velocity of every off-origin point.
- **Derivative versus rotation:** $\dot{\mathbf{R}}_{wb}$ is not a rotation matrix. Code must not validate or normalise it as though it belonged to $SO(2)$.
- **Numerical differentiation:** Estimating $\dot{\mathbf{R}}$ or velocity from sampled poses amplifies measurement noise and depends on an accurate sampling interval.

Quick test C: rotation derivatives and point velocity

1. Define ${}^w \mathbf{v}_b$ and ω , including their frames and units.
2. Differentiate $\mathbf{R}_{wb}(\theta)$ and show that $\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb}$.
3. What geometric operation does $\mathbf{S}_2$ perform on a planar vector?
4. Why is $\dot{\mathbf{R}}_{wb}$ generally not an element of $SO(2)$?
5. Differentiate $\mathbf{R}_{wb}^\top \mathbf{R}_{wb} = \mathbf{I}_2$ and use the result to show that $\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb}$ is skew-symmetric.
6. Starting from ${}^w \mathbf{p}_P = \mathbf{R}_{wb} {}^b \mathbf{p}_P + {}^w \mathbf{p}_b$, derive the velocity of a point fixed to the body.

7. Why is the rotational contribution to point velocity perpendicular to ${}^w\mathbf{r}_{O_b P}$?
8. How does the velocity equation change if P moves relative to frame $\{b\}$?

Answers and hints

1. ${}^w\mathbf{v}_b = d({}^w\mathbf{p}_b)/dt = [\dot{x} \ \dot{y}]^T$ is the velocity of the body origin relative to the fixed world frame and expressed in world coordinates, measured in m s^{-1} . The scalar $\omega = \dot{\theta}$ is the signed yaw rate, measured in rad s^{-1} .
2. Entrywise differentiation and the chain rule give the matrix displayed in Section 6.3.2. Direct multiplication shows that its coefficient of ω equals $\mathbf{S}_2\mathbf{R}_{wb}$, so $\dot{\mathbf{R}}_{wb} = \omega\mathbf{S}_2\mathbf{R}_{wb}$.
3. $\mathbf{S}_2[a \ b]^T = [-b \ a]^T$, so it rotates the coordinate column counter-clockwise by $\pi/2$ without changing its length.
4. Its entries have units of s^{-1} , and it does not generally satisfy $\dot{\mathbf{R}}_{wb}^T\dot{\mathbf{R}}_{wb} = \mathbf{I}_2$ or have determinant 1.
5. Differentiation gives $\dot{\mathbf{R}}^T\mathbf{R} + \mathbf{R}^T\dot{\mathbf{R}} = \mathbf{0}$. Therefore, $(\mathbf{R}^T\dot{\mathbf{R}})^T = \dot{\mathbf{R}}^T\mathbf{R} = -\mathbf{R}^T\dot{\mathbf{R}}$.
6. Since P is body-fixed, $d({}^b\mathbf{p}_P)/dt = \mathbf{0}_2$. Therefore, ${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \dot{\mathbf{R}}_{wb}{}^b\mathbf{p}_P = {}^w\mathbf{v}_b + \omega\mathbf{S}_2{}^w\mathbf{r}_{O_b P}$.
7. $\mathbf{S}_2$ rotates the relative-position vector by $\pi/2$, so their dot product is zero. This is the tangential direction of instantaneous circular motion about O_b .
8. Retain the relative-motion term: ${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \dot{\mathbf{R}}_{wb}{}^b\mathbf{p}_P + \mathbf{R}_{wb}d({}^b\mathbf{p}_P)/dt$.

6.4 Planar twists and $\mathfrak{se}(2)$

The previous section described translational and angular velocity separately. A **planar twist** combines them into one representation of the instantaneous motion of a rigid body. This representation will connect the pose derivative $\dot{\mathbf{T}}_{wb}$ to the differential-drive inputs v and ω .

6.4.1 Twist coordinates

In this chapter, a planar twist is represented by the coordinate column

$$\boldsymbol{\xi} := \begin{bmatrix} \nu_x \\ \nu_y \\ \omega \end{bmatrix} \in \mathbb{R}^3.$$

Here, ν_x and ν_y use the Greek letter *nu*, ν , not the Latin letter *v*. They are **generic placeholders** for the x - and y -components of the twist's linear part. Because neither the coordinate frame nor the point chosen as the velocity reference has been specified yet, these symbols do not yet identify a unique physical velocity. Together they form the column $\boldsymbol{\nu} = [\nu_x \ \nu_y]^\top \in \mathbb{R}^2$, whose entries are measured in m s^{-1} . The body and spatial cases are defined in Section 6.4.3 and Section 6.4.4.

The third entry $\omega \in \mathbb{R}$ is angular velocity, measured in rad s^{-1} . The symbol $\boldsymbol{\xi}$ is the Greek letter *xi*. The word *twist* refers to an instantaneous rigid-body velocity, not to a pose or a finite displacement.

This chapter uses the component order $[\nu_x \ \nu_y \ \omega]^\top$. Some textbooks and software libraries place angular velocity first, so the component order must always be declared. Although $\boldsymbol{\xi}$ is mathematically a three-entry real column, its entries have different physical units. A norm such as $\|\boldsymbol{\xi}\|_2$ therefore has no direct physical meaning unless a length scale is introduced to compare translation with rotation.

6.4.2 The hat operator and $\mathfrak{se}(2)$

Section 6.3.3 showed that $\mathfrak{so}(2)$ contains the tangent directions of the rotation group $SO(2)$ at $\mathbf{I}_2$. A planar rigid motion also includes translation, so its instantaneous direction must combine one rotational component with two translational components.

6.4.2.1 From twist coordinates to a matrix

The **hat operator** maps the twist coordinate column to a 3×3 matrix:

$$\hat{\boldsymbol{\xi}} := \begin{bmatrix} 0 & -\omega & \nu_x \\ \omega & 0 & \nu_y \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \omega \mathbf{S}_2 & \boldsymbol{\nu} \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

The wide hat in $\hat{\boldsymbol{\xi}}$ denotes this mapping; it does not denote an estimate. The inverse operation, which recovers $\boldsymbol{\xi}$ from its matrix representation, is commonly called the **vee operator** and is written $(\hat{\boldsymbol{\xi}})^\vee = \boldsymbol{\xi}$.

The upper-left block $\omega \mathbf{S}_2$ represents the instantaneous rotation direction. The upper-right column $\boldsymbol{\nu}$ represents the two-component linear part, and the bottom row is zero because differentiating the constant bottom row $[0 \ 0 \ 1]$ of a homogeneous transformation gives $[0 \ 0 \ 0]$.

6.4.2.2 Why these matrices are tangent to $SE(2)$

Recall that an element of $SE(2)$ is a finite planar homogeneous transformation

$$\mathbf{G} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix}, \quad \mathbf{R} \in SO(2), \quad \mathbf{p} \in \mathbb{R}^2.$$

It rotates and translates planar point coordinates. The **identity transformation** is

$$\mathbf{I}_3 = \begin{bmatrix} \mathbf{I}_2 & \mathbf{0}_2 \\ \mathbf{0}_2^\top & 1 \end{bmatrix}.$$

It represents zero rotation angle and zero translation, so it leaves every homogeneous point coordinate $\bar{\mathbf{p}}_P = [\mathbf{p}_P^\top \ 1]^\top \in \mathbb{R}^3$ unchanged:

$$\mathbf{I}_3 \bar{\mathbf{p}}_P = \bar{\mathbf{p}}_P.$$

Let $\mathbf{G}(s) \in SE(2)$ be a differentiable transformation curve with dimensionless real parameter s near zero and $\mathbf{G}(0) = \mathbf{I}_3$. Its derivative $\mathbf{G}'(0)$ is a tangent direction to $SE(2)$ at $\mathbf{I}_3$. The tangent space is the set of all such derivatives:

$$T_{\mathbf{I}_3} SE(2) := \{ \mathbf{G}'(0) \mid \mathbf{G}(s) \in SE(2), \mathbf{G}(0) = \mathbf{I}_3 \}.$$

Now define the candidate set

$$\mathcal{C} := \left\{ \begin{bmatrix} a\mathbf{S}_2 & \mathbf{u} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \mid a \in \mathbb{R}, \mathbf{u} \in \mathbb{R}^2 \right\},$$

where $\mathcal{C}$ is merely a temporary name for the proposed tangent-matrix structure. The goal is to prove

$$T_{\mathbf{I}_3} SE(2) = \mathcal{C}.$$

Equality requires two set inclusions. The first shows that no other tangent-matrix form is possible. The second shows that every matrix in the proposed set is actually attainable.

First inclusion: every tangent direction has the proposed form. Take any $\mathbf{A} \in T_{\mathbf{I}_3} SE(2)$. By the definition of the tangent space, there is a differentiable curve

$$\mathbf{G}(s) = \begin{bmatrix} \mathbf{R}(s) & \mathbf{p}(s) \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in SE(2)$$

such that $\mathbf{G}(0) = \mathbf{I}_3$ and $\mathbf{A} = \mathbf{G}'(0)$. The identity condition implies $\mathbf{R}(0) = \mathbf{I}_2$ and $\mathbf{p}(0) = \mathbf{0}_2$. Differentiating the block matrix gives

$$\mathbf{A} = \mathbf{G}'(0) = \begin{bmatrix} \mathbf{R}'(0) & \mathbf{p}'(0) \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

Because $\mathbf{R}(s) \in SO(2)$, it satisfies $\mathbf{R}(s)^\top \mathbf{R}(s) = \mathbf{I}_2$. Differentiating this constraint with respect to s and evaluating at $s = 0$ gives

$$\mathbf{R}'(0)^\top \mathbf{R}(0) + \mathbf{R}(0)^\top \mathbf{R}'(0) = \mathbf{0}_{2 \times 2}.$$

Using $\mathbf{R}(0) = \mathbf{I}_2$ reduces this equation to

$$\mathbf{R}'(0)^\top + \mathbf{R}'(0) = \mathbf{0}_{2 \times 2}.$$

Thus, $\mathbf{R}'(0)$ is skew-symmetric and belongs to $\mathfrak{so}(2)$. Section 6.3.3 established that every matrix in $\mathfrak{so}(2)$ has the form $a\mathbf{S}_2$ for some $a \in \mathbb{R}$, so $\mathbf{R}'(0) = a\mathbf{S}_2$. Define $\mathbf{u} := \mathbf{p}'(0) \in \mathbb{R}^2$. Substitution gives

$$\mathbf{A} = \begin{bmatrix} a\mathbf{S}_2 & \mathbf{u} \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

Therefore, $\mathbf{A} \in \mathcal{C}$. Because $\mathbf{A}$ was an arbitrary tangent direction, this proves

$$T_{\mathbf{I}_3} SE(2) \subseteq \mathcal{C}.$$

Second inclusion: every matrix of the proposed form is attainable. Take any $\mathbf{A} \in \mathcal{C}$. By the definition of $\mathcal{C}$, there are values $a \in \mathbb{R}$ and $\mathbf{u} \in \mathbb{R}^2$ such that

$$\mathbf{A} = \begin{bmatrix} a\mathbf{S}_2 & \mathbf{u} \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

Construct the curve

$$\mathbf{G}_{a,\mathbf{u}}(s) := \begin{bmatrix} \mathbf{R}(as) & s\mathbf{u} \\ \mathbf{0}_2^\top & 1 \end{bmatrix}.$$

For every s , $\mathbf{R}(as) \in SO(2)$ and $s\mathbf{u} \in \mathbb{R}^2$, so $\mathbf{G}_{a,\mathbf{u}}(s) \in SE(2)$. At $s = 0$,

$$\mathbf{G}_{a,\mathbf{u}}(0) = \begin{bmatrix} \mathbf{R}(0) & \mathbf{0}_2 \\ \mathbf{0}_2^\top & 1 \end{bmatrix} = \mathbf{I}_3.$$

The chain rule and $d\mathbf{R}(\theta)/d\theta|_{\theta=0} = \mathbf{S}_2$ give its derivative at zero:

$$\begin{aligned} \mathbf{G}'_{a,\mathbf{u}}(0) &= \begin{bmatrix} \left. \frac{d}{ds} \mathbf{R}(as) \right|_{s=0} & \left. \frac{d}{ds} (s\mathbf{u}) \right|_{s=0} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} a\mathbf{S}_2 & \mathbf{u} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} = \mathbf{A}. \end{aligned}$$

The curve is valid, passes through $\mathbf{I}_3$, and has the prescribed derivative $\mathbf{A}$. Hence, $\mathbf{A} \in T_{\mathbf{I}_3} SE(2)$. Because $\mathbf{A}$ was an arbitrary member of $\mathcal{C}$, this proves

$$\mathcal{C} \subseteq T_{\mathbf{I}_3} SE(2).$$

Conclusion. The two inclusions prove $T_{\mathbf{I}_3} SE(2) = \mathcal{C}$. This tangent space is denoted $\mathfrak{se}(2)$:

$$\mathfrak{se}(2) := T_{\mathbf{I}_3} SE(2) = \mathcal{C} = \left\{ \begin{bmatrix} a\mathbf{S}_2 & \mathbf{u} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \mid a \in \mathbb{R}, \mathbf{u} \in \mathbb{R}^2 \right\}.$$

The notation $\mathfrak{se}(2)$ is read “little s e two.” Because $SE(2)$ is a Lie group, its tangent space at the identity is called its **Lie algebra**.

This vector space is three-dimensional because each matrix is determined by three independent real numbers: one scalar a for rotation and the two entries of $\mathbf{u}$ for translation. Elements of $SE(2)$ represent finite poses or finite rigid transformations. Elements of $\mathfrak{se}(2)$ represent directions in which such transformations can change instantaneously from the identity.

6.4.2.3 Physical interpretation

When the curve parameter is physical time, the rotational coefficient becomes angular velocity $a = \omega$, and the translation derivative becomes a linear-velocity column $\mathbf{u} = \boldsymbol{\nu}$. The corresponding tangent matrix is exactly the hatted twist $\hat{\boldsymbol{\xi}}$.

When $\hat{\boldsymbol{\xi}}$ represents physical velocity, the entries of its 2×2 block have units of s^{-1} and the entries of its right column have units of $m s^{-1}$. The matrix is therefore a structured representation of a physical rate, not a finite homogeneous transformation.

For a sufficiently short time interval $\Delta t > 0$, a smooth transformation curve through the identity with $\mathbf{G}'(0) = \hat{\boldsymbol{\xi}}$ has the first-order expansion

$$\mathbf{G}(\Delta t) = \mathbf{I}_3 + \Delta t \hat{\boldsymbol{\xi}} + O(\Delta t^2).$$

This equation explains the term **infinitesimal rigid motion**: $\hat{\boldsymbol{\xi}}$ is not itself a finite transformation in $SE(2)$, but it gives the first-order direction from which a nearby finite transformation develops.

6.4.3 Body twist

6.4.3.1 Plain-language meaning of pose and twist

A **pose** describes where a rigid body is and which direction it is facing relative to a chosen reference frame. In planar motion, the pose contains two position coordinates and one orientation angle. The homogeneous transformation $\mathbf{T}_{wb}$ represents the pose of body frame $\{b\}$ relative to world frame $\{w\}$.

A **twist** describes how a rigid body's pose is changing at one instant. It combines linear velocity and angular velocity in one coordinate column. The word *twist* does not mean that the rigid body bends or deforms; the body remains rigid. In this planar chapter, a twist has two linear-velocity components and one angular-velocity component.

A **body twist** is the motion of the body relative to the world, expressed using the body's instantaneous axes. The phrase *relative to the world* identifies the reference used to measure the motion, whereas *expressed using the body axes* identifies the directions used to report its numerical components. The body frame moves through the world with the rigid body, but it remains fixed relative to the body.

For compactness in this derivation, write

$$\mathbf{T} := \mathbf{T}_{wb} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix}, \quad \dot{\mathbf{T}} = \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix},$$

where $\mathbf{R} = \mathbf{R}_{wb} \in SO(2)$ and $\mathbf{p} = {}^w\mathbf{p}_b \in \mathbb{R}^2$. The entries of $\mathbf{p}$ are measured in metres, and $\dot{\mathbf{p}} = {}^w\mathbf{v}_b$ is the world-frame velocity of the body origin.

Premultiply $\dot{\mathbf{T}}$ by $\mathbf{T}^{-1}$:

$$\begin{aligned}
\mathbf{T}^{-1}\dot{\mathbf{T}} &= \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{R}^\top \dot{\mathbf{R}} & \mathbf{R}^\top \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\
&= \begin{bmatrix} \omega \mathbf{S}_2 & \boldsymbol{\nu}_b \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.
\end{aligned}$$

Here,

$$\boldsymbol{\nu}_b := \mathbf{R}_{wb}^\top {}^w \mathbf{v}_b = \begin{bmatrix} \nu_{x_b} \\ \nu_{y_b} \end{bmatrix}$$

is the velocity of the body origin expressed along the body axes x_b and y_b . Its entries are measured in m s^{-1} . The **body twist** and its matrix representation are therefore

$$\boldsymbol{\xi}_b := \begin{bmatrix} \nu_{x_b} \\ \nu_{y_b} \\ \omega \end{bmatrix}, \quad \hat{\boldsymbol{\xi}}_b = \mathbf{T}_{wb}^{-1} \dot{\mathbf{T}}_{wb}.$$

Equivalently,

$$\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb} \hat{\boldsymbol{\xi}}_b.$$

The subscript b states that the twist is resolved in the instantaneous body frame. In planar motion, ω has the same scalar value in the world and body descriptions because both frames share the same rotation axis; this simplification does not extend to the three-component angular-velocity coordinates used in spatial motion.

The coordinate conversion

$$\boldsymbol{\nu}_b = \mathbf{R}_{wb}^\top {}^w \mathbf{v}_b$$

does not create a different physical velocity. The column ${}^w \mathbf{v}_b = [\dot{x} \ \dot{y}]^\top$ describes the body origin's velocity *relative to the world* using the world axes. Multiplication by $\mathbf{R}_{wb}^\top$ expresses that same velocity using the body's forward and sideways axes:

$$\boldsymbol{\nu}_b = \begin{bmatrix} \nu_{x_b} \\ \nu_{y_b} \end{bmatrix}.$$

Thus, ν_{x_b} is the forward component and ν_{y_b} is the sideways component of the world-relative velocity. For example, a robot facing north and moving north at 1 m s^{-1} has world-frame velocity components $[0 \ 1]^T \text{ m s}^{-1}$ but body-frame components $[1 \ 0]^T \text{ m s}^{-1}$. These two columns describe the same physical motion.

The body origin does not move relative to its own attached frame. Its body-frame position is always ${}^b\mathbf{p}_b = \mathbf{0}_2$, so the derivative of that relative position is also zero. *This zero relative velocity is different from $\boldsymbol{\nu}_b$, which is the body's velocity relative to the world merely expressed using body axes.*

Nor does the coordinate conversion imply that the body directly feels or measures constant linear velocity. A robot estimates its motion relative to the ground or another external reference using information such as wheel encoders, cameras, lidar, satellite positioning, or sensor fusion. The body axes then provide convenient forward and sideways directions in which to express that estimate.

6.4.3.2 Physical meaning and use

The two boxed equations describe the same relationship in opposite directions. The equation

$$\hat{\boldsymbol{\xi}}_b = \mathbf{T}_{wb}^{-1} \dot{\mathbf{T}}_{wb}$$

extracts motion from a pose trajectory: if $\mathbf{T}_{wb}(t)$ and its derivative are known, it returns the instantaneous angular and linear velocity resolved along the moving body axes. The inverse rotation inside $\mathbf{T}_{wb}^{-1}$ removes the body's current world orientation from the pose rate.

The equation

$$\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb} \hat{\boldsymbol{\xi}}_b$$

predicts how the pose changes: given the current pose and a body-frame twist, it produces the corresponding world-frame pose rate. This is the form commonly used to propagate mobile-robot odometry, simulate motion, and predict a future pose from a commanded or estimated body velocity.

For an ideal differential-drive robot, the no-sideways-motion constraint gives $\nu_{y_b} = 0$. The wheel-to-body mapping from Section 3.2.1 therefore supplies

$$\boxed{\boldsymbol{\xi}_b = \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix}}, \quad v = \frac{r}{2} (\dot{\phi}_L + \dot{\phi}_R), \quad \omega = \frac{r}{L} (\dot{\phi}_R - \dot{\phi}_L).$$

Here, $r > 0$ is the wheel radius in metres, $L > 0$ is the track width in metres, and $\dot{\phi}_L$ and $\dot{\phi}_R$ are the left and right wheel angular speeds in rad s^{-1} . The resulting engineering flow is

$$(\dot{\phi}_L, \dot{\phi}_R) \longrightarrow \boldsymbol{\xi}_b \longrightarrow \dot{\mathbf{T}}_{wb} \longrightarrow \mathbf{T}_{wb}(t + \Delta t).$$

Wheel encoders measure wheel rotation, not the true body twist. Inferring $\boldsymbol{\xi}_b$ from them relies on the wheel geometry and ideal rolling assumptions; slip, deformation, unequal effective radii, and encoder errors make the inferred twist differ from the physical motion.

6.4.4 Spatial twist expressed in the world frame

Postmultiplying $\dot{\mathbf{T}}$ by $\mathbf{T}^{-1}$ produces a different element of $\mathfrak{se}(2)$:

$$\begin{aligned} \dot{\mathbf{T}}\mathbf{T}^{-1} &= \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \dot{\mathbf{R}}\mathbf{R}^\top & \dot{\mathbf{p}} - \dot{\mathbf{R}}\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \omega \mathbf{S}_2 & \boldsymbol{\nu}_w \\ \mathbf{0}_2^\top & 0 \end{bmatrix}, \end{aligned}$$

where

$$\boldsymbol{\nu}_w := \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p}.$$

Write $\boldsymbol{\nu}_w = [\nu_{x_w} \ \nu_{y_w}]^\top \in \mathbb{R}^2$, where ν_{x_w} and ν_{y_w} are its components along the world axes and are measured in m s^{-1} .

The **spatial twist expressed in the world frame** is

$$\boldsymbol{\xi}_w := \begin{bmatrix} \nu_{x_w} \\ \nu_{y_w} \\ \omega \end{bmatrix}, \quad \hat{\boldsymbol{\xi}}_w = \dot{\mathbf{T}}_{wb} \mathbf{T}_{wb}^{-1}.$$

Equivalently,

$$\dot{\mathbf{T}}_{wb} = \hat{\boldsymbol{\xi}}_w \mathbf{T}_{wb}.$$

Here, *spatial* means that the twist is represented relative to the fixed space or world frame; it does not mean that the motion is three-dimensional.

A crucial distinction is

$$\boldsymbol{\nu}_w \neq \dot{\mathbf{p}} = {}^w\mathbf{v}_b$$

in general. The column $\boldsymbol{\nu}_w$ is the linear part of the spatial twist, not simply the body-origin velocity expressed in world coordinates. Its meaning follows from the rigid-body point-velocity equation derived in Section 6.3.4:

$${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \omega \mathbf{S}_2 {}^w\mathbf{r}_{O_b P}.$$

Here, ${}^w\mathbf{v}_P \in \mathbb{R}^2$ is the velocity of a body-fixed point P relative to the world and expressed in world coordinates, ${}^w\mathbf{v}_b = \dot{\mathbf{p}} \in \mathbb{R}^2$ is the corresponding velocity of the body origin O_b , and ${}^w\mathbf{r}_{O_b P} \in \mathbb{R}^2$ is the relative-position vector from O_b to P , also expressed in world coordinates. Because

$${}^w\mathbf{r}_{O_b P} = {}^w\mathbf{p}_P - {}^w\mathbf{p}_b = {}^w\mathbf{p}_P - \mathbf{p},$$

substitution and regrouping give

$$\begin{aligned} {}^w\mathbf{v}_P &= \dot{\mathbf{p}} + \omega \mathbf{S}_2 ({}^w\mathbf{p}_P - \mathbf{p}) \\ &= \dot{\mathbf{p}} + \omega \mathbf{S}_2 {}^w\mathbf{p}_P - \omega \mathbf{S}_2 \mathbf{p} \\ &= \underbrace{(\dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p})}_{\boldsymbol{\nu}_w} + \omega \mathbf{S}_2 {}^w\mathbf{p}_P. \end{aligned}$$

Using the definition $\boldsymbol{\nu}_w = \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p}$ therefore produces the world-frame velocity field

$$\boxed{{}^w\mathbf{v}_P = \boldsymbol{\nu}_w + \omega \mathbf{S}_2 {}^w\mathbf{p}_P}.$$

The symbol O_w denotes the fixed origin of world frame $\{w\}$. Its world-coordinate column is $\mathbf{0}_2$. Evaluating the velocity field at that location by setting ${}^w\mathbf{p}_P = \mathbf{0}_2$ gives ${}^w\mathbf{v}_P = \boldsymbol{\nu}_w$. Thus, $\boldsymbol{\nu}_w$ is the velocity, expressed in world coordinates, assigned by the instantaneous rigid-motion field to the location O_w . *The world origin need not be a physical point on the robot; the rigid-motion velocity field is being extended mathematically to that location.* This dependence on the chosen origin explains why changing the reference origin changes the linear part of a twist.

6.4.4.1 Physical meaning and use

As in the body case, the two boxed equations support two complementary calculations. The equation

$$\hat{\xi}_w = \dot{T}_{wb} T_{wb}^{-1}$$

extracts the instantaneous rigid-body motion relative to the fixed world axes and the fixed world origin. The equation

$$\dot{T}_{wb} = \hat{\xi}_w T_{wb}$$

uses that world-referenced motion to calculate the pose rate. The multiplication side records the convention: a body twist acts on T_{wb} from the right, whereas a spatial twist acts from the left.

The spatial twist describes the velocity of the entire rigid body through the world-frame velocity field derived above. Once $\hat{\xi}_w$ is known, substituting the world position ${}^w\mathbf{p}_P$ of any body-fixed point P gives that point's world-frame velocity. At the body origin, ${}^w\mathbf{p}_P = \mathbf{p}$ and $P = O_b$, so $\nu_w = \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p}$ gives

$$\boxed{{}^w\mathbf{v}_b = \dot{\mathbf{p}} = \nu_w + \omega \mathbf{S}_2 \mathbf{p}.}$$

Therefore,

$$\underbrace{{}^w\mathbf{v}_b}_{\text{velocity at } O_b} = \underbrace{\nu_w}_{\text{velocity field at } O_w} + \underbrace{\omega \mathbf{S}_2 \mathbf{p}}_{\text{change caused by the different location}}. \quad (6.1)$$

The linear part ν_w and the body-origin velocity ${}^w\mathbf{v}_b$ refer to different locations. They are equal only when the rotational contribution $\omega \mathbf{S}_2 \mathbf{p}$ is zero, such as when $\omega = 0$ or when O_b currently coincides with O_w .

The spatial representation is useful when an algorithm uses a common fixed world frame: for example, when comparing the motions of several bodies, evaluating point velocities at world-frame locations, or formulating world-referenced estimation and control. Differential-drive wheel kinematics more naturally produces a body twist, so such an application first maps that body twist to the equivalent spatial twist. The next section derives this conversion. *Both twists describe the same physical instantaneous motion; only the reference frame and reference origin used for their coordinates differ.*

6.4.5 Relating body and spatial twists

The body and spatial twists describe the same instantaneous motion, so one must be convertible into the other. Start from their matrix definitions:

$$\hat{\xi}_b = \mathbf{T}^{-1}\dot{\mathbf{T}}, \quad \hat{\xi}_w = \dot{\mathbf{T}}\mathbf{T}^{-1}.$$

The body-twist definition implies $\dot{\mathbf{T}} = \mathbf{T}\hat{\xi}_b$. Substituting this result into the spatial-twist definition gives

$$\begin{aligned} \hat{\xi}_w &= \dot{\mathbf{T}}\mathbf{T}^{-1} \\ &= (\mathbf{T}\hat{\xi}_b)\mathbf{T}^{-1} \\ &= \mathbf{T}\hat{\xi}_b\mathbf{T}^{-1}. \end{aligned}$$

The product $\mathbf{T}\hat{\xi}_b\mathbf{T}^{-1}$ is called **conjugation by $\mathbf{T}$** . To find what it does to the twist coordinates, write the three matrices in block form:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix}, \quad \hat{\xi}_b = \begin{bmatrix} \omega \mathbf{S}_2 & \boldsymbol{\nu}_b \\ \mathbf{0}_2^\top & 0 \end{bmatrix}, \quad \mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix}.$$

First multiply the two matrices on the left:

$$\mathbf{T}\hat{\xi}_b = \begin{bmatrix} \mathbf{R}\omega \mathbf{S}_2 & \mathbf{R}\boldsymbol{\nu}_b \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

Postmultiplying this result by $\mathbf{T}^{-1}$ gives

$$\mathbf{T}\hat{\xi}_b\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}\omega \mathbf{S}_2 \mathbf{R}^\top & -\mathbf{R}\omega \mathbf{S}_2 \mathbf{R}^\top \mathbf{p} + \mathbf{R}\boldsymbol{\nu}_b \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

As shown in Section 6.3.2, every planar rotation $\mathbf{R}$ commutes with $\mathbf{S}_2$. Using $\mathbf{R}\mathbf{S}_2 = \mathbf{S}_2\mathbf{R}$ and $\mathbf{R}\mathbf{R}^\top = \mathbf{I}_2$ gives $\mathbf{R}\mathbf{S}_2\mathbf{R}^\top = \mathbf{S}_2$. Therefore,

$$\hat{\xi}_w = \begin{bmatrix} \omega \mathbf{S}_2 & \mathbf{R}\boldsymbol{\nu}_b - \omega \mathbf{S}_2 \mathbf{p} \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

By definition, the same spatial-twist matrix is

$$\hat{\xi}_w = \begin{bmatrix} \omega \mathbf{S}_2 & \boldsymbol{\nu}_w \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

Comparing the upper-right columns gives

$$\boxed{\boldsymbol{\nu}_w = \mathbf{R}\boldsymbol{\nu}_b - \omega \mathbf{S}_2 \mathbf{p}}.$$

This result also follows directly from the definition of $\boldsymbol{\nu}_w$ or ${}^w\mathbf{v}_b = \boldsymbol{\nu}_w + \omega \mathbf{S}_2 \mathbf{p}$ in Equation 6.1. The body-origin velocity expressed in world coordinates is ${}^w\mathbf{v}_b = \mathbf{R}\boldsymbol{\nu}_b$, so that equation becomes

$$\mathbf{R}\boldsymbol{\nu}_b = \boldsymbol{\nu}_w + \omega \mathbf{S}_2 \mathbf{p}.$$

Rearranging gives $\boldsymbol{\nu}_w = \mathbf{R}\boldsymbol{\nu}_b - \omega \mathbf{S}_2 \mathbf{p}$, which agrees with the block-matrix derivation.

The first term rotates the body-axis components of the linear velocity into world-axis components. The second term changes the reference location from the body origin O_b to the world origin O_w . A twist therefore does not transform like a free vector, for which rotation alone would be sufficient.

Now collect the linear and angular coordinates into

$$\xi_b = \begin{bmatrix} \boldsymbol{\nu}_b \\ \omega \end{bmatrix}, \quad \xi_w = \begin{bmatrix} \boldsymbol{\nu}_w \\ \omega \end{bmatrix}.$$

The preceding component equation can be written as one linear mapping:

$$\begin{aligned} \xi_w &= \begin{bmatrix} \mathbf{R} & -\mathbf{S}_2 \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{\nu}_b \\ \omega \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}\boldsymbol{\nu}_b - \omega \mathbf{S}_2 \mathbf{p} \\ \omega \end{bmatrix}. \end{aligned}$$

This 3×3 coordinate mapping is called the **adjoint representation** of $\mathbf{T}$:

$$\boxed{\text{Ad}_{\mathbf{T}} = \begin{bmatrix} \mathbf{R} & -\mathbf{S}_2 \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 3}}.$$

Equivalently, the adjoint is the linear mapping defined by

$$\widehat{\text{Ad}_{\mathbf{T}} \xi} = \mathbf{T} \hat{\xi} \mathbf{T}^{-1}.$$

For $\mathbf{T} = \mathbf{T}_{wb}$, it converts the body-twist coordinates into the spatial-twist coordinates:

$$\boxed{\boldsymbol{\xi}_w = \text{Ad}_{\mathbf{T}_{wb}} \boldsymbol{\xi}_b}.$$

6.4.6 Differential-drive body twist

Under the ideal rolling constraints, a differential-drive robot has no lateral velocity at the body origin. Its body-frame linear velocity is

$$\boldsymbol{\nu}_b = \begin{bmatrix} v \\ 0 \end{bmatrix},$$

where v is the signed forward speed in m s^{-1} . Its body twist is therefore

$$\boxed{\boldsymbol{\xi}_b = \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix}, \quad \hat{\boldsymbol{\xi}}_b = \begin{bmatrix} 0 & -\omega & v \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}}.$$

Substitute the hatted body twist into $\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb} \hat{\boldsymbol{\xi}}_b$. In block form,

$$\mathbf{T}_{wb} = \begin{bmatrix} \mathbf{R}_{wb} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix}, \quad \hat{\boldsymbol{\xi}}_b = \begin{bmatrix} \omega \mathbf{S}_2 & \begin{bmatrix} v \\ 0 \end{bmatrix} \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

Multiplying these matrices gives

$$\begin{aligned} \mathbf{T}_{wb} \hat{\boldsymbol{\xi}}_b &= \begin{bmatrix} \mathbf{R}_{wb} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} \omega \mathbf{S}_2 & \begin{bmatrix} v \\ 0 \end{bmatrix} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_{wb} \omega \mathbf{S}_2 & \mathbf{R}_{wb} \begin{bmatrix} v \\ 0 \end{bmatrix} \\ \mathbf{0}_2^\top & 0 \end{bmatrix}. \end{aligned}$$

The terms involving the translation column $\mathbf{p}$ disappear because they multiply the zero blocks in $\hat{\boldsymbol{\xi}}_b$. On the other hand,

$$\dot{\mathbf{T}}_{wb} = \begin{bmatrix} \dot{\mathbf{R}}_{wb} & \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

Because these two block matrices are equal, their corresponding upper-left and upper-right blocks must be equal:

$$\dot{\mathbf{R}}_{wb} = \mathbf{R}_{wb}\omega\mathbf{S}_2, \quad \dot{\mathbf{p}} = \mathbf{R}_{wb} \begin{bmatrix} v \\ 0 \end{bmatrix}.$$

Since $\dot{\mathbf{p}} = {}^w\mathbf{v}_b$ and

$$\mathbf{R}_{wb} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix},$$

the translation block becomes

$${}^w\mathbf{v}_b = \mathbf{R}_{wb} \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix}.$$

The rotation derivative derived in Section 6.3.2 is $\dot{\mathbf{R}}_{wb} = \mathbf{R}_{wb}\dot{\theta}\mathbf{S}_2$. Comparing it with $\dot{\mathbf{R}}_{wb} = \mathbf{R}_{wb}\omega\mathbf{S}_2$ gives $\dot{\theta} = \omega$.

Therefore, the familiar differential-drive state equation is recovered:

$$\dot{\mathbf{q}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix}.$$

The twist formulation has not introduced a different motion model. It has expressed the same model in a form that respects rigid-motion composition and generalises naturally to three-dimensional robotics.

6.4.7 Worked example: the two twist representations

Suppose

$${}^w\mathbf{p}_b = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \text{ m}, \quad \theta = \frac{\pi}{2} \text{ rad},$$

and the differential-drive body twist has $v = 0.5 \text{ m s}^{-1}$ and $\omega = 2 \text{ rad s}^{-1}$. The body-twist column is

$$\xi_b = \begin{bmatrix} 0.5 \\ 0 \\ 2 \end{bmatrix},$$

where the first two entries have units of m s^{-1} and the last has units of rad s^{-1} . At $\theta = \pi/2$, the velocity of the body origin expressed in the world frame is

$$\dot{\mathbf{p}} = \mathbf{R}_{wb} \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \text{m s}^{-1}.$$

The spatial twist's linear part is

$$\begin{aligned} \nu_w &= \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p} \\ &= \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} - 2 \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 \\ -3.5 \end{bmatrix} \text{m s}^{-1}. \end{aligned}$$

This does not contradict ${}^w\mathbf{v}_b = [0 \ 0.5]^\top \text{m s}^{-1}$. The two columns refer to different points in the same instantaneous velocity field. Substituting the body-origin position into the spatial velocity field recovers its correct velocity:

$$\nu_w + \omega \mathbf{S}_2 {}^w\mathbf{p}_b = \begin{bmatrix} 2 \\ -3.5 \end{bmatrix} + \begin{bmatrix} -2 \\ 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \text{m s}^{-1}.$$

6.4.8 Assumptions, failure modes, and checks

- **Undeclared component order:** A column containing the same three numbers can represent different twists if one convention uses linear velocity first and another uses angular velocity first.
- **Confusing a twist with a pose:** ξ and $\hat{\xi}$ describe instantaneous motion; neither is an element of $SE(2)$ or a finite pose.
- **Confusing ν_w with ${}^w\mathbf{v}_b$:** They are equal only when $\omega = 0$, $\mathbf{p} = \mathbf{0}_2$, or their correction term happens to vanish.
- **Rotating a twist like a free vector:** Changing the reference origin introduces the translation-dependent term in Ad_Γ .
- **Assuming zero lateral velocity for every rigid body:** The condition $\nu_{y_b} = 0$ comes from the ideal differential-drive rolling constraint, not from the definition of a planar twist.

- **Mixing finite and infinitesimal quantities:** In $\mathbf{T}_{wb}$, the rotation block $\mathbf{R}_{wb}$ is dimensionless and the translation column $\mathbf{p}$ has units of metres. In $\hat{\boldsymbol{\xi}}$, the corresponding blocks have units of s^{-1} and m s^{-1} . Therefore, $\mathbf{T}_{wb}$ and $\hat{\boldsymbol{\xi}}$ cannot be added directly. A twist must first be used as a rate over a time interval through the appropriate pose-evolution equation.

Quick test D: planar twists

1. What physical quantities are stored in $\boldsymbol{\xi} = [\nu_x \ \nu_y \ \omega]^\top$, and what are their units?
2. Define the hat operator for the component order used in this chapter.
3. What is $\mathfrak{se}(2)$, and how does it differ from $SE(2)$?
4. Starting from $\mathbf{T}_{wb}$ and $\dot{\mathbf{T}}_{wb}$, derive $\hat{\boldsymbol{\xi}}_b = \mathbf{T}_{wb}^{-1} \dot{\mathbf{T}}_{wb}$.
5. Why is the linear part $\boldsymbol{\nu}_w$ of the spatial twist generally different from ${}^w\mathbf{v}_b$?
6. State the two equivalent pose-rate equations involving the body and spatial twists.
7. Derive the differential-drive state equation from $\boldsymbol{\xi}_b = [v \ 0 \ \omega]^\top$.
8. A planar twist is $\boldsymbol{\xi} = [\nu_x \ \nu_y \ \omega]^\top$, where ν_x and ν_y are measured in m s^{-1} and ω is measured in rad s^{-1} . Why is the ordinary Euclidean expression $\|\boldsymbol{\xi}\|_2 = \sqrt{\nu_x^2 + \nu_y^2 + \omega^2}$ not a physically meaningful twist magnitude? How can a chosen characteristic length $\ell > 0$, measured in metres, make the terms comparable?

Answers and hints

1. ν_x and ν_y are the two components of a linear velocity, measured in m s^{-1} ; ω is angular velocity, measured in rad s^{-1} . Their frame and reference point must also be declared.
2. The hat matrix has $\omega \mathbf{S}_2$ as its upper-left 2×2 block, $\boldsymbol{\nu}$ as its upper-right 2×1 block, and a zero bottom row, as displayed in Section 6.4.2.
3. $SE(2)$ is the group of finite planar rigid transformations. Its Lie algebra $\mathfrak{se}(2)$ is the vector space of structured matrices representing infinitesimal rigid motions and instantaneous velocities.
4. Block multiplication gives upper-left block $\mathbf{R}^\top \dot{\mathbf{R}}$, upper-right block $\mathbf{R}^\top \dot{\mathbf{p}}$, and a zero bottom row. Identify $\mathbf{R}^\top \dot{\mathbf{R}} = \omega \mathbf{S}_2$ and $\boldsymbol{\nu}_b = \mathbf{R}^\top \dot{\mathbf{p}}$.
5. A spatial twist uses the fixed world origin as its velocity reference. Therefore, its linear part is $\boldsymbol{\nu}_w = \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p}$; the second term accounts for the offset between the world and body origins.
6. $\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb} \hat{\boldsymbol{\xi}}_b$ and $\dot{\mathbf{T}}_{wb} = \hat{\boldsymbol{\xi}}_w \mathbf{T}_{wb}$.
7. Matrix multiplication gives ${}^w\mathbf{v}_b = \mathbf{R}_{wb} [v \ 0]^\top = [v \cos \theta \ v \sin \theta]^\top$ and $\dot{\theta} = \omega$.

8. The expression adds squared linear speeds to a squared angular speed, so its terms represent different physical quantities and its numerical value depends arbitrarily on the chosen units. A characteristic length converts angular speed into the linear-speed scale $\ell\omega$, measured in m s^{-1} , giving a weighted magnitude such as $\sqrt{\nu_x^2 + \nu_y^2 + (\ell\omega)^2}$. The choice of ℓ is application-dependent—for example, it might be related to the robot's size—so this is not a unique intrinsic norm for all tasks.

Cumulative chapter review

1. What does the pose $\mathbf{T}_{wb} \in SE(2)$ describe, and how does it transform the body-frame coordinates of a point into world-frame coordinates?
2. Distinguish a passive coordinate change from an active rigid motion. For column-vector notation, which transformation acts first in $\mathbf{T}_2\mathbf{T}_1\mathbf{p}$?
3. Starting from $\mathbf{R}_{wb}^\top \mathbf{R}_{wb} = \mathbf{I}_2$, explain why $\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb}$ is skew-symmetric and identify its planar form.
4. Derive the world-frame velocity of a body-fixed point P from ${}^w\mathbf{p}_P = {}^w\mathbf{p}_b + \mathbf{R}_{wb} {}^b\mathbf{p}_P$.
5. How do $SE(2)$ and $\mathfrak{se}(2)$ differ physically and mathematically? What does the hat operator do?
6. State the physical meanings of the body twist ξ_b and spatial twist ξ_w . Why is ν_w generally not the velocity of the body origin?
7. A robot has $\theta = \pi/2$, $\mathbf{p} = [2 \ 0]^\top \text{ m}$, and $\xi_b = [1 \ 0 \ 0.5]^\top$, with linear entries in m s^{-1} and angular entry in rad s^{-1} . Find ${}^w\mathbf{v}_b$ and ν_w .
8. Starting from the differential-drive body twist $\xi_b = [v \ 0 \ \omega]^\top$, recover the state equation for $\dot{\mathbf{q}}$.

Answers and hints

1. $\mathbf{T}_{wb}$ describes the position and orientation of body frame $\{b\}$ relative to world frame $\{w\}$. It maps a body-frame homogeneous point by ${}^w\bar{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P$, or equivalently ${}^w\mathbf{p}_P = \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b$.
2. A passive change describes the same geometric object in different coordinates; an active motion moves the object while retaining the reference frame. In $\mathbf{T}_2\mathbf{T}_1\mathbf{p}$, the rightmost transformation $\mathbf{T}_1$ acts first.
3. Differentiation gives $\dot{\mathbf{R}}_{wb}^\top \mathbf{R}_{wb} + \mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = \mathbf{0}_{2 \times 2}$. Therefore, $\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb}$ equals the negative of its transpose and has the planar form $\omega \mathbf{S}_2 \in \mathfrak{so}(2)$.
4. Differentiating gives ${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P$. Using $\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb}$ and ${}^w\mathbf{r}_{O_b P} = \mathbf{R}_{wb} {}^b\mathbf{p}_P$ gives ${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \omega \mathbf{S}_2 {}^w\mathbf{r}_{O_b P}$.

5. $SE(2)$ is the nonlinear group of finite planar poses and rigid transformations. $\mathfrak{se}(2)$ is its three-dimensional tangent vector space at the identity and represents infinitesimal motions or rates. The hat operator maps a twist coordinate column $[\nu_x \ \nu_y \ \omega]^\top$ to its structured 3×3 matrix in $\mathfrak{se}(2)$.
6. ξ_b describes world-relative motion using body axes and the body origin as its linear-velocity reference. ξ_w describes the same motion using world axes and the world origin. Its linear part is the velocity field at O_w , so the velocity at O_b is instead ${}^w\mathbf{v}_b = \nu_w + \omega \mathbf{S}_2 \mathbf{p}$.
7. At $\theta = \pi/2$, $\mathbf{R}_{wb}[1 \ 0]^\top = [0 \ 1]^\top$, so ${}^w\mathbf{v}_b = [0 \ 1]^\top \text{ m s}^{-1}$. Because $0.5\mathbf{S}_2[2 \ 0]^\top = [0 \ 1]^\top \text{ m s}^{-1}$, $\nu_w = {}^w\mathbf{v}_b - \omega \mathbf{S}_2 \mathbf{p} = \mathbf{0}_2 \text{ m s}^{-1}$.
8. Substitution into $\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb}\hat{\xi}_b$ gives $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, and $\dot{\theta} = \omega$, so $\dot{\mathbf{q}} = [v \cos \theta \ v \sin \theta \ \omega]^\top$.

References

- John J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Chapters 2 and 5, especially frame mappings, transformation interpretations, angular velocity, and rigid-body point velocity.
- Peter Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, 3rd ed., Chapters 2-3, especially time-varying pose, angular velocity, and derivatives of orientation and pose.
- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapters 2-3, especially configuration spaces, angular velocity, skew-symmetric representations, body and spatial twists, and rigid-body motions. The $SE(3)$ twist definitions are specialised here to planar motion in $SE(2)$.

7 Degrees of Freedom and Spatial Motion: $SO(3)$ and $SE(3)$

Why this chapter matters

The preceding chapter described a rigid body constrained to a plane by three pose coordinates and represented its configuration by an element of $SE(2)$. A robot operating freely in three-dimensional physical space can translate in three independent directions and rotate about three independent directions. This chapter develops the corresponding six-degree-of-freedom model, represents spatial orientation with $SO(3)$, combines orientation and position with $SE(3)$, and extends planar angular velocity and twists to three dimensions.

These ideas are required for robot arms, aerial vehicles, underwater vehicles, cameras, lidar sensors, and any system whose orientation cannot be represented by one planar heading angle.

Prerequisites and assumptions

The reader should understand vectors, matrices, rank, and constraints from Chapter 1; coordinate frames, cross products, and three-dimensional point transformations from Chapter 2; and planar rigid transformations, tangent spaces, and twists from Chapter 6.

Physical space is modelled as three-dimensional Euclidean space. Every body considered here is rigid, so the distance between any two body-fixed points remains constant. Frames are right-handed and orthonormal. Position is measured in metres, time in seconds, and angles in radians.

7.1 Degrees of freedom and configuration space

7.1.1 Configuration is a complete geometric description

A system's **configuration** is a complete geometric specification of the position of every labelled material point of the system at one instant. A *material point* is a particular point

fixed in the physical material of the body; its identity does not change as the body moves. Configuration describes where the system is, not how fast it is moving or what forces act on it.

This definition can be written as a map. Let $\mathcal{B}$ denote the set of labelled material points belonging to one planar rigid body. After attaching frame $\{b\}$ to the body, each point $P \in \mathcal{B}$ has a constant body-frame coordinate ${}^b\mathbf{p}_P \in \mathbb{R}^2$. A configuration is a **placement map** χ that assigns a world-frame position to every material point:

$$\chi : \mathcal{B} \rightarrow \mathbb{R}^2, \quad P \mapsto \chi(P) = {}^w\mathbf{p}_P.$$

The symbol χ is the Greek lowercase letter *chi*. Because the body is rigid, a valid placement must preserve the distance between every pair of material points:

$$\|\chi(P) - \chi(Q)\|_2 = \|{}^b\mathbf{p}_P - {}^b\mathbf{p}_Q\|_2 \quad \text{for every } P, Q \in \mathcal{B}.$$

The set of all placement maps allowed by the system's physical constraints is its **configuration space**, abbreviated **C-space** and denoted here by $\mathcal{C}$. One configuration is therefore one element $\chi \in \mathcal{C}$. Calling a configuration a *point in C-space* means one abstract element of this set; it does not mean one physical point of the rigid body or one point in the world.

7.1.1.1 Why one transformation represents the whole rigid body

Listing $\chi(P)$ separately for every $P \in \mathcal{B}$ would be highly redundant. All body-frame point coordinates are fixed by rigidity, so a planar rotation $\mathbf{R}_{wb} \in SO(2)$ and the world position ${}^w\mathbf{p}_b \in \mathbb{R}^2$ of the body-frame origin determine every value of the placement map:

$$\boxed{\chi(P) = {}^w\mathbf{p}_P = \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b \quad \text{for every } P \in \mathcal{B}.}$$

The homogeneous transformation

$$\mathbf{T}_{wb} = \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in SE(2)$$

packages the same rotation and translation. For one selected point P , the multiplication

$${}^w\bar{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P$$

evaluates the placement map only at that point and returns its world-frame homogeneous coordinate. Applying the same $\mathbf{T}_{wb}$ to every $P \in \mathcal{B}$ produces the world positions of all

the body's material points and therefore specifies the complete configuration. The transformation is not itself the world coordinate of one point; it is the single rule that locates the entire rigid body.

Every configuration uses the same fixed world frame $\{w\}$. Two configurations χ_1 and χ_2 differ when their body-origin positions ${}^w\mathbf{p}_b^{(1)}$ and ${}^w\mathbf{p}_b^{(2)}$ differ, their rotations $\mathbf{R}_{wb}^{(1)}$ and $\mathbf{R}_{wb}^{(2)}$ differ, or both differ. They are the same configuration only when both the translation and rotation agree, equivalently when $\mathbf{T}_{wb}^{(1)} = \mathbf{T}_{wb}^{(2)}$.

Once the body frame has been fixed relative to the body, each orientation-preserving planar placement determines exactly one $\mathbf{T}_{wb} \in SE(2)$, and each $\mathbf{T}_{wb} \in SE(2)$ determines exactly one such placement. The two sets are therefore in one-to-one correspondence, written

$$\boxed{\mathcal{C} \cong SE(2)},$$

where $\cong$ means that the two sets carry the same configuration information through this correspondence. Robotics texts consequently often identify the configuration space of a free planar rigid body with $SE(2)$ and write $\mathcal{C} = SE(2)$. Here, an element of $SE(2)$ is being used as a representation of a complete body placement, even though the same matrix can also act as an operator that transforms point coordinates.

7.1.1.2 Configuration coordinates are not the configuration itself

For the planar rigid body, one convenient coordinate column is

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}.$$

Although $\mathbf{q}$ has three entries, it does not describe a point in three-dimensional physical space. The body is constrained to move in a plane: x and y are the entries of ${}^w\mathbf{p}_b = [x \ y]^\top$, measured in metres, while θ is the orientation of frame $\{b\}$ relative to frame $\{w\}$, measured in radians about the axis normal to the plane. Translation normal to the plane and rotation about either axis lying in the plane are excluded. Thus the body has two independent position coordinates and one independent orientation coordinate. The column $\mathbf{q}$ is an ordered coordinate description with mixed physical units, not a three-dimensional geometric vector with one common unit, and its ordinary Euclidean norm has no direct physical meaning.

If θ is stored as an unwrapped real number, $\mathbf{q}$ can be manipulated locally as a three-component real column. Globally, however, θ and $\theta + 2\pi n$ for any integer n describe

the same orientation. The planar configuration-coordinate domain is therefore not ordinary three-dimensional Cartesian physical space. These three **generalised coordinates** construct the transformation representing the configuration:

$$\mathbf{q} \mapsto \mathbf{T}_{wb}(\mathbf{q}) = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix} \in SE(2).$$

Thus, χ denotes the complete physical placement, $\mathbf{T}_{wb}$ represents that placement as a rigid transformation, and $\mathbf{q}$ is a coordinate description used to construct the transformation. These objects contain equivalent placement information once the world and body frames, axis directions, units, positive rotation direction, and transformation direction have been defined, but they have different mathematical types. In particular, an unrestricted real-valued angle does not give a unique global coordinate for each configuration.

A configuration space is different from a physical **workspace**. A workspace is a region of physical space that a robot or end-effector can reach. A C-space contains complete robot configurations. Different joint arrangements can place an end-effector at the same workspace point while representing different points in C-space.

7.1.2 Definition of degree of freedom

A **degree of freedom**, abbreviated **DOF**, is one independent local direction in which a configuration can change while continuing to satisfy all configuration constraints. The number of degrees of freedom counts these independent allowable directions.

System	Independently chosen coordinates	DOF
Point constrained to a straight rail	Distance s along the rail	1
Point free in a plane	Position (x, y)	2
Door attached by an ideal hinge	Hinge angle θ	1
Free planar rigid body	Position and heading (x, y, θ)	3
Free spatial rigid body	Three position and three orientation coordinates	6

The door illustrates independence. Once its hinge angle is chosen, the positions of all its material points are determined. Recording the world coordinates of its corners would store many scalar values, but the hinge and rigidity constraints prevent those values from being varied independently.

7.1.2.1 Mathematical definition from allowable small changes

Suppose a candidate configuration is represented by a variable column

$$\mathbf{z} = [z_1 \quad \cdots \quad z_{n_v}]^T \in \mathbb{R}^{n_v},$$

where $n_v \in \mathbb{N}_0$ is the number of stored scalar variables. Valid configurations must satisfy $n_c \in \mathbb{N}_0$ scalar equality constraints collected in the mapping

$$\mathbf{g} : \mathbb{R}^{n_v} \rightarrow \mathbb{R}^{n_c}, \quad \mathbf{g}(\mathbf{z}) = \mathbf{0}_{n_c}.$$

The **constraint Jacobian**, or constraint-derivative matrix, is $\mathbf{J}_g(\mathbf{z}) \in \mathbb{R}^{n_c \times n_v}$ with entry

$$[\mathbf{J}_g(\mathbf{z})]_{ij} := \frac{\partial g_i}{\partial z_j}.$$

To describe an allowable local direction at a selected valid configuration $\mathbf{z}_0 \in \mathbb{R}^{n_v}$, let $\mathbf{z}(s)$ be any differentiable curve of valid configurations, where $s \in \mathbb{R}$ is a dimensionless curve parameter, $\mathbf{z}(0) = \mathbf{z}_0$, and $\mathbf{g}(\mathbf{z}(s)) = \mathbf{0}_{n_c}$ for every s near zero. Define the curve's direction at $\mathbf{z}_0$ by $\delta \mathbf{z} := \mathbf{z}'(0) \in \mathbb{R}^{n_v}$. Differentiating the constraint equation with respect to s and applying the multivariable chain rule gives

$$\left. \frac{d}{ds} \mathbf{g}(\mathbf{z}(s)) \right|_{s=0} = \mathbf{J}_g(\mathbf{z}_0) \delta \mathbf{z} = \mathbf{0}_{n_c}.$$

The set of all vectors satisfying this linear equation is the null space of the Jacobian evaluated at $\mathbf{z}_0$:

$$\mathcal{N}(\mathbf{J}_g(\mathbf{z}_0)) = \left\{ \delta \mathbf{z} \in \mathbb{R}^{n_v} \mid \mathbf{J}_g(\mathbf{z}_0) \delta \mathbf{z} = \mathbf{0}_{n_c} \right\}.$$

This is not the same set as the configuration constraint set $\{\mathbf{z} \in \mathbb{R}^{n_v} \mid \mathbf{g}(\mathbf{z}) = \mathbf{0}_{n_c}\}$. The constraint set contains valid configurations and may be curved; the null space contains allowable infinitesimal changes at the one selected configuration $\mathbf{z}_0$ and is a linear space.

At a **regular configuration**, meaning that the rank of the constraint Jacobian remains constant in a neighbourhood of $\mathbf{z}_0$, these null-space vectors are exactly the allowable infinitesimal directions. Each independent null-space direction is one independent way to change the configuration locally, so

$$n_{\text{dof}}(\mathbf{z}_0) := \dim \mathcal{N}(\mathbf{J}_g(\mathbf{z}_0)).$$

Applying the rank-nullity theorem from Section 1.2.2.1 gives

$$n_{\text{dof}}(\mathbf{z}_0) = n_v - \text{rank}(\mathbf{J}_g(\mathbf{z}_0)).$$

If the n_c constraints are independent near $\mathbf{z}_0$, then $\text{rank}(\mathbf{J}_g(\mathbf{z}_0)) = n_c$, and the familiar counting rule follows:

$$n_{\text{dof}} = n_v - n_c.$$

7.1.2.2 Example: a point constrained to a circle

Consider a point with Cartesian coordinate column $\mathbf{p} = [x \ y]^\top \in \mathbb{R}^2$ constrained to a circle of fixed radius $r_c > 0$ metres centred at the world origin. Its two stored variables satisfy

$$g(\mathbf{p}) := x^2 + y^2 - r_c^2 = 0.$$

Here, $n_v = 2$ and there is one scalar constraint. Its constraint Jacobian is

$$\mathbf{J}_g(\mathbf{p}) = [2x \ 2y] \in \mathbb{R}^{1 \times 2}.$$

An allowable change $\delta\mathbf{p} = [\delta x \ \delta y]^\top$ must satisfy

$$[2x \ 2y] \begin{bmatrix} \delta x \\ \delta y \end{bmatrix} = 2x \delta x + 2y \delta y = 0.$$

For this local calculation, $\mathbf{p} = [x \ y]^\top$ is one selected, fixed configuration on the circle. Thus, x and y are given constants, while the allowable change $\delta\mathbf{p}$ is the variable being determined. Because $r_c > 0$, the selected point is not $\mathbf{0}_2$, so the row $\mathbf{J}_g(\mathbf{p}) = [2x \ 2y]$ is nonzero and has rank one. The rank-nullity formula therefore gives

$$n_{\text{dof}} = \dim \mathcal{N}(\mathbf{J}_g(\mathbf{p})) = 2 - 1 = 1.$$

The equation $x \delta x + y \delta y = 0$ also shows the actual allowable direction: $\delta\mathbf{p}$ must be perpendicular to the fixed radial vector $\mathbf{p}$. Every vector perpendicular to $[x \ y]^\top$ in $\mathbb{R}^2$ is a scalar multiple of $[-y \ x]^\top$, so

$$\mathcal{N}(\mathbf{J}_g(\mathbf{p})) = \left\{ \alpha \begin{bmatrix} -y \\ x \end{bmatrix} \mid \alpha \in \mathbb{R} \right\},$$

Here, x and y remain fixed and only the dimensionless scalar $\alpha \in \mathbb{R}$ varies. Choosing α determines both components through $\delta x = -\alpha y$ and $\delta y = \alpha x$. Substitution confirms that every such change satisfies the linearised constraint:

$$\mathbf{J}_g(\mathbf{p}) \delta \mathbf{p} = \begin{bmatrix} 2x & 2y \end{bmatrix} \alpha \begin{bmatrix} -y \\ x \end{bmatrix} = \alpha(-2xy + 2yx) = 0.$$

The single varying coefficient α explicitly represents the one local degree of freedom. At a different point on the circle, the fixed values of x and y produce a different tangent direction, but the null space still has dimension one.

The number of stored values is consequently not necessarily the number of degrees of freedom. For example, a spatial rotation matrix stores nine scalar entries, but its constraints leave only three independent rotational directions. Likewise, the circle uses two Cartesian entries even though its allowable-change space is one-dimensional.

This counting rule applies directly to geometric, or **holonomic**, equality constraints that restrict configuration. Constraints imposed only on velocity require separate analysis and do not necessarily reduce the dimension of the configuration space.

7.1.3 Why a free planar rigid body has three degrees of freedom

Select three non-collinear points A , B , and C fixed to a planar rigid body. *Non-collinear* means that the points do not all lie on one line, so they form a non-degenerate triangle that visibly represents a two-dimensional rigid shape.

Three points are convenient but not minimal: two distinct labelled points already determine planar position and orientation, whereas one point cannot reveal rotation about itself. The triangle is used here because its fixed pairwise distances make the rigidity constraints especially clear.

If the three points could move independently, their planar coordinates would provide six scalar variables:

$$(x_A, y_A, x_B, y_B, x_C, y_C) \in \mathbb{R}^6.$$

Rigidity fixes the three pairwise distances

$$\|\mathbf{p}_A - \mathbf{p}_B\|_2 = d_{AB}, \quad \|\mathbf{p}_B - \mathbf{p}_C\|_2 = d_{BC}, \quad \|\mathbf{p}_A - \mathbf{p}_C\|_2 = d_{AC},$$

where $\mathbf{p}_A, \mathbf{p}_B, \mathbf{p}_C \in \mathbb{R}^2$ are the point-coordinate columns and the positive constants d_{AB} , d_{BC} , and d_{AC} are the fixed distances measured in metres. For a non-degenerate triangle, these are three independent scalar constraints. Therefore,

$$n_{\text{dof}} = 6 - 3 = 3.$$

These three freedoms can be represented more conveniently by two translations and one rotation:

free planar rigid body: $(x, y, \theta) \implies 3 \text{ DOF}.$
--

The fixed distances preserve the body's shape; they do not prevent the complete triangle from translating or rotating as one rigid object.

7.1.4 Why a free spatial rigid body has six degrees of freedom

Allow the same three non-collinear body-fixed points to move in three-dimensional space. Each point has three Cartesian coordinates, so the three points provide nine scalar variables. Rigidity preserves the three pairwise distances d_{AB} , d_{BC} , and d_{AC} , which are three independent constraints for a non-degenerate triangle. Therefore,

$$n_{\text{dof}} = 9 - 3 = 6.$$

The six freedoms are interpreted as three translational and three rotational freedoms:

free spatial rigid body: $\underbrace{(x, y, z)}_{3 \text{ translations}} + \underbrace{\text{spatial orientation}}_{3 \text{ rotational DOF}} \implies 6 \text{ DOF}.$

The phrase *three rotational degrees of freedom* does not yet prescribe three particular angles. Euler angles, rotation vectors, rotation matrices, and unit quaternions are different representations of the same three-dimensional orientation freedom, each with different coordinate properties and limitations. The next learning block develops the rotation-matrix representation $SO(3)$ before introducing any minimal angle convention.

7.1.5 Constraints, actuators, state variables, and degrees of freedom

Degrees of freedom are often confused with other counts in a robot model. The following quantities answer different questions:

- **Configuration DOF:** How many independent real coordinates are needed locally to specify where the system is?
- **Actuator count:** How many independently commanded physical inputs does the system have?
- **Instantaneous mobility:** How many independent velocity directions are allowed at one configuration?

- **State dimension:** How many variables are required by a chosen dynamic model to predict future behaviour?
- **Representation size:** How many numbers are stored by a particular coordinate representation?

These counts need not be equal. A free spatial rigid body has six configuration degrees of freedom. A dynamic state that stores both its pose and velocity may require twelve local scalar coordinates. A rotation matrix stores nine numbers even though spatial orientation has only three degrees of freedom, because its entries satisfy orthonormality and determinant constraints.

A differential-drive robot provides another important distinction. Its configuration is represented by $\mathbf{T}_{wb} \in SE(2)$, its configuration space is modelled by $SE(2)$, and it has three degrees of freedom described locally by (x, y, θ) . Under ideal rolling, however, its instantaneous body velocity has only two independently commanded components, v and ω , because sideways velocity is constrained to zero. The robot cannot move sideways instantaneously, but it can change all three pose coordinates through a sequence of forward and turning motions. Therefore, its two instantaneous control directions do not reduce its configuration space to two dimensions.

7.1.6 Worked examples

7.1.6.1 A hinged door

A free spatial door panel would have six degrees of freedom. An ideal revolute hinge permits only rotation about its hinge axis and prevents the other five independent motions. The constrained door therefore has

$$n_{\text{dof}} = 6 - 5 = 1,$$

which can be represented by one hinge angle $\theta \in \mathbb{R}$ measured in radians over the mechanically allowed interval.

7.1.6.2 A free aerial vehicle

Treat the vehicle as one rigid body with no geometric connection to the environment. Its configuration has six degrees of freedom: three position freedoms and three orientation freedoms. This statement does not imply that the vehicle can independently accelerate in all six directions at one instant. Its actuator arrangement and dynamics determine which accelerations can actually be produced.

7.1.7 Assumptions, limitations, and checks

- **Counting redundant constraints:** Several equations may express the same geometric restriction. Check the rank of the constraint derivatives rather than counting written equations blindly.
- **Confusing coordinates with physical freedom:** A rotation matrix has nine entries but only three rotational degrees of freedom. Identify the independent constraints imposed on a representation.
- **Confusing actuator count with DOF:** A robot may be underactuated or may have redundant actuators. State whether a count refers to configuration, velocity, inputs, or state.
- **Applying $n_v - n_c$ at a singular configuration:** Constraint rank can change at singularities. Verify that the constraints are regular in the region being analysed.
- **Treating a velocity constraint as a configuration constraint:** Under ideal rolling, a wheel moves without sliding sideways at its ground contact. This condition is **non-holonomic** when it restricts the allowable velocities but cannot be replaced by an equivalent equation involving only the configuration. For example, a differential-drive robot cannot move sideways instantaneously, but turning and driving can still change all three coordinates (x, y, θ) . The rolling constraint therefore reduces instantaneous mobility without reducing the dimension of its C-space; analyse valid configurations and valid velocities separately.

Quick test A: configuration and degrees of freedom

1. Define configuration and configuration space using the placement map χ .
2. For a planar rigid body, distinguish the placement map χ , the transformation T_{wb} , and the generalised-coordinate column $\mathbf{q}$.
3. Why does applying T_{wb} to one body-fixed point return only that point's world coordinate, while the same transformation still represents the body's complete configuration?
4. Express the number of degrees of freedom using the null space and rank of a constraint Jacobian. Apply the definition to a point constrained to a circle.
5. Use three non-collinear body-fixed points to explain why a free planar rigid body has three degrees of freedom.
6. Why does an ideal hinged door have one degree of freedom rather than six?
7. Distinguish configuration DOF, instantaneous mobility, actuator count, and state dimension.
8. Why does a differential-drive robot have three configuration degrees of freedom even though its ideal body velocity is specified by only v and ω ?
9. Under what condition is the counting rule $n_{\text{dof}} = n_v - n_c$ locally valid?

Answers and hints

1. A configuration is a placement map χ assigning a world position $\chi(P) = {}^w\mathbf{p}_P$ to every labelled material point P while satisfying the system's geometric constraints. The configuration space $\mathcal{C}$ is the set of all allowed placement maps.
2. The map χ is the complete placement of all body points. The matrix $\mathbf{T}_{wb} \in SE(2)$ represents that placement through one rotation and translation. The column $\mathbf{q} = [x \ y \ \theta]^T$ is a set of generalised coordinates used to construct $\mathbf{T}_{wb}$.
3. Multiplication by one point coordinate evaluates the placement rule at that one point. Because every material point has a fixed body-frame coordinate, applying the same transformation to all such coordinates reconstructs the complete placement map; one transformation therefore represents the whole rigid body.
4. At a selected valid configuration $\mathbf{z}_0$, allowable small changes satisfy $\mathbf{J}_g(\mathbf{z}_0)\delta\mathbf{z} = \mathbf{0}_{n_c}$, so $n_{\text{dof}}(\mathbf{z}_0) = \dim \mathcal{N}(\mathbf{J}_g(\mathbf{z}_0)) = n_v - \text{rank}(\mathbf{J}_g(\mathbf{z}_0))$. For $g(x, y) = x^2 + y^2 - r_c^2$, the Jacobian $[2x \ 2y]$ has rank one on a circle with $r_c > 0$, and its null space is spanned by $[-y \ x]^T$, so $n_{\text{dof}} = 2 - 1 = 1$.
5. Three planar points provide six scalar coordinates. Rigidity fixes their three pairwise distances, giving three independent constraints for a non-degenerate triangle. Thus, $6 - 3 = 3$ freedoms remain: two translations and one rotation.
6. The hinge imposes five independent constraints on a free spatial body and leaves only rotation about the hinge axis, so $6 - 5 = 1$.
7. Configuration DOF counts independent pose coordinates; instantaneous mobility counts allowed velocity directions at one configuration; actuator count counts independent command inputs; and state dimension counts the variables retained by a dynamic model.
8. Its configuration still requires (x, y, θ) , but the rolling constraint removes instantaneous sideways velocity. Sequences of allowed forward and turning motions can nevertheless change all three configuration coordinates.
9. The rule applies locally when the equality constraints are independent and regular, meaning that their derivative has constant rank in the neighbourhood being considered.

7.2 Spatial orientation and $SO(3)$

7.2.1 Constructing a spatial rotation matrix

The **spatial orientation** of body frame $\{b\}$ relative to world frame $\{w\}$ specifies how the three body axes point when expressed using world-frame coordinates; the locations of the frame origins are irrelevant to orientation. As established in Chapter 2, let $\mathbf{e}_{x,b}$, $\mathbf{e}_{y,b}$, and $\mathbf{e}_{z,b}$ be the geometric unit vectors along the body axes. Their world-frame coordinate columns are

$${}^w\mathbf{e}_{x,b}, \quad {}^w\mathbf{e}_{y,b}, \quad {}^w\mathbf{e}_{z,b} \in \mathbb{R}^3.$$

Each column is dimensionless because it describes a unit direction. Placing the columns side by side defines

$$\mathbf{R}_{wb} := \begin{bmatrix} {}^w\mathbf{e}_{x,b} & {}^w\mathbf{e}_{y,b} & {}^w\mathbf{e}_{z,b} \end{bmatrix} \in \mathbb{R}^{3 \times 3}.$$

The first column states where the positive x_b -axis points in world coordinates, and the other columns do the same for the positive y_b - and z_b -axes. The subscript order wb means that $\mathbf{R}_{wb}$ maps coordinate columns from frame $\{b\}$ to frame $\{w\}$.

To derive this mapping, let the geometric vector $\mathbf{v}$ have body-frame coordinate column

$${}^b\mathbf{v} = \begin{bmatrix} v_{x_b} \\ v_{y_b} \\ v_{z_b} \end{bmatrix} \in \mathbb{R}^3,$$

where the entries are its scalar components along the body axes and carry the physical units of $\mathbf{v}$. Expanding the vector in the body basis and then expressing the result in world coordinates gives

$$\begin{aligned} {}^w\mathbf{v} &= v_{x_b} {}^w\mathbf{e}_{x,b} + v_{y_b} {}^w\mathbf{e}_{y,b} + v_{z_b} {}^w\mathbf{e}_{z,b} \\ &= \begin{bmatrix} {}^w\mathbf{e}_{x,b} & {}^w\mathbf{e}_{y,b} & {}^w\mathbf{e}_{z,b} \end{bmatrix} \begin{bmatrix} v_{x_b} \\ v_{y_b} \\ v_{z_b} \end{bmatrix}. \end{aligned}$$

Therefore,

$${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}.$$

This multiplication changes the coordinate description of $\mathbf{v}$; it does not change the geometric vector itself.

7.2.2 Rotation constraints and the definition of $SO(3)$

Write the columns of $\mathbf{R}_{wb}$ as $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbb{R}^3$. Because they represent the axes of an orthonormal frame, each column has unit length and each distinct pair is perpendicular:

$$\mathbf{r}_i^\top \mathbf{r}_i = 1, \quad \mathbf{r}_i^\top \mathbf{r}_j = 0 \quad \text{for } i \neq j, \quad i, j \in \{1, 2, 3\}.$$

Matrix multiplication collects these pairwise dot products:

$$\mathbf{R}_{wb}^\top \mathbf{R}_{wb} = \begin{bmatrix} \mathbf{r}_1^\top \mathbf{r}_1 & \mathbf{r}_1^\top \mathbf{r}_2 & \mathbf{r}_1^\top \mathbf{r}_3 \\ \mathbf{r}_2^\top \mathbf{r}_1 & \mathbf{r}_2^\top \mathbf{r}_2 & \mathbf{r}_2^\top \mathbf{r}_3 \\ \mathbf{r}_3^\top \mathbf{r}_1 & \mathbf{r}_3^\top \mathbf{r}_2 & \mathbf{r}_3^\top \mathbf{r}_3 \end{bmatrix} = \mathbf{I}_3.$$

A real square matrix satisfying $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_3$ is **orthogonal**. This condition permits both right-handed orientations and reflections. For a right-handed body frame, $\mathbf{r}_1 \times \mathbf{r}_2 = \mathbf{r}_3$, so the scalar-triple-product formula gives

$$\det(\mathbf{R}_{wb}) = \mathbf{r}_3^\top (\mathbf{r}_1 \times \mathbf{r}_2) = \mathbf{r}_3^\top \mathbf{r}_3 = 1.$$

The **special orthogonal group in three dimensions**, read “S O three,” is

$$SO(3) := \{ \mathbf{R} \in \mathbb{R}^{3 \times 3} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I}_3, \quad \det(\mathbf{R}) = 1 \}.$$

The word *orthogonal* refers to the orthonormal columns, while *special* selects determinant +1 and excludes determinant−1 reflections. The group operation is matrix multiplication. For any $\mathbf{R}_1, \mathbf{R}_2 \in SO(3)$,

$$\begin{aligned} (\mathbf{R}_1 \mathbf{R}_2)^\top (\mathbf{R}_1 \mathbf{R}_2) &= \mathbf{R}_2^\top \mathbf{R}_1^\top \mathbf{R}_1 \mathbf{R}_2 = \mathbf{I}_3, \\ \det(\mathbf{R}_1 \mathbf{R}_2) &= \det(\mathbf{R}_1) \det(\mathbf{R}_2) = 1. \end{aligned}$$

Thus, $\mathbf{R}_1 \mathbf{R}_2 \in SO(3)$, which is the **closure** property. Matrix multiplication is associative, $\mathbf{I}_3$ is the identity orientation, and every element has an inverse in the set. From $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_3$,

$$\mathbf{R}^{-1} = \mathbf{R}^\top.$$

A rotation matrix stores nine scalar entries, but the symmetric equation $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_3$ supplies six independent scalar constraints: three unit-length equations and three pairwise-orthogonality equations. The determinant requirement selects the right-handed component rather than removing another continuous direction. Consequently,

$$\dim SO(3) = 9 - 6 = 3.$$

Here, $\dim SO(3) = 3$ means that three independent continuous coordinates are required near any selected orientation. This is the same local meaning of dimension used in the earlier DOF definition.

7.2.3 Example: a positive quarter-turn about the world z_w -axis

Suppose frame $\{b\}$ starts aligned with frame $\{w\}$ and is rotated through $\pi/2$ rad about the positive z_w -axis according to the right-hand rule. Its body axes, expressed in world coordinates, become

$${}^w\mathbf{e}_{x,b} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad {}^w\mathbf{e}_{y,b} = \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix}, \quad {}^w\mathbf{e}_{z,b} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Placing these axes into the columns gives

$$\mathbf{R}_{wb} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \in SO(3).$$

Direct calculation gives $\mathbf{R}_{wb}^\top \mathbf{R}_{wb} = \mathbf{I}_3$ and $\det(\mathbf{R}_{wb}) = 1$. A vector pointing along the positive body x_b -axis has ${}^b\mathbf{v} = [1 \ 0 \ 0]^\top$ in arbitrary vector units, so

$${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}.$$

The result agrees with the first column of $\mathbf{R}_{wb}$: the positive body x_b -axis points along the positive world y_w -axis.

Quick test B: spatial orientation and $SO(3)$

1. What does column j of $\mathbf{R}_{wb}$ represent?
2. Starting from orthonormal body axes, derive $\mathbf{R}_{wb}^T \mathbf{R}_{wb} = \mathbf{I}_3$.
3. Why is $\det(\mathbf{R}) = 1$ required in addition to $\mathbf{R}^T \mathbf{R} = \mathbf{I}_3$?
4. Why does a rotation matrix store nine entries but represent only three rotational degrees of freedom?
5. For the quarter-turn matrix above, calculate $\mathbf{R}_{bw}$ and use it to express ${}^w\mathbf{v} = [0 \ 1 \ 0]^T$ in body coordinates.

Answers and hints

1. Columns 1, 2, and 3 are respectively the world-frame coordinate columns of the body x_b -, y_b -, and z_b -axes.
2. Entry (i, j) of $\mathbf{R}_{wb}^T \mathbf{R}_{wb}$ is $\mathbf{r}_i^T \mathbf{r}_j$. It equals 1 when $i = j$ and 0 otherwise, producing $\mathbf{I}_3$.
3. Orthogonality alone permits determinant -1 reflections. The determinant-1 condition selects right-handed rigid-body orientations.
4. Orthonormality imposes six independent scalar constraints on the nine entries: three unit-length constraints and three pairwise-orthogonality constraints. Thus, $9 - 6 = 3$ independent continuous values remain.
5. $\mathbf{R}_{bw} = \mathbf{R}_{wb}^{-1} = \mathbf{R}_{wb}^T$. Therefore, ${}^b\mathbf{v} = \mathbf{R}_{bw} {}^w\mathbf{v} = [1 \ 0 \ 0]^T$.

7.3 Coordinate-axis rotations and composition

7.3.1 Constructing coordinate-axis rotation matrices

Begin with the coordinate-change equation derived in Section 7.2.1:

$$\boxed{{}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}}.$$

Here, $\mathbf{v}$ is one unchanged geometric vector, while the two columns are its coordinates in body frame $\{b\}$ and world frame $\{w\}$. The same rotation matrix was defined from the world-frame coordinate columns of the body axes:

$$\boxed{\mathbf{R}_{wb} = \begin{bmatrix} {}^w\mathbf{e}_{x,b} & {}^w\mathbf{e}_{y,b} & {}^w\mathbf{e}_{z,b} \end{bmatrix}}.$$

The task is therefore to calculate these three coordinate columns from the geometry and place them into the matrix. Multiplication by standard basis columns is not needed for this construction.

Suppose frames $\{b\}$ and $\{w\}$ are initially aligned, and then frame $\{b\}$ is rotated through $\theta \in \mathbb{R}$ radians about the positive world x_w -axis. A positive rotation follows the right-hand rule. Define

$$c_\theta := \cos \theta, \quad s_\theta := \sin \theta.$$

The x_b -axis remains aligned with x_w , so its world coordinates are $[1 \ 0 \ 0]^\top$. The y_b - and z_b -axes rotate in the world $y_w z_w$ -plane. Their projections onto the world axes give

$${}^w\mathbf{e}_{x,b} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad {}^w\mathbf{e}_{y,b} = \begin{bmatrix} 0 \\ c_\theta \\ s_\theta \end{bmatrix}, \quad {}^w\mathbf{e}_{z,b} = \begin{bmatrix} 0 \\ -s_\theta \\ c_\theta \end{bmatrix}.$$

By definition, the first, second, and third columns of the orientation matrix are the world-frame coordinates of the x_b -, y_b -, and z_b -axes, respectively. Substituting the three columns above gives

$$\mathbf{R}_x(\theta) = [{}^w\mathbf{e}_{x,b} \quad {}^w\mathbf{e}_{y,b} \quad {}^w\mathbf{e}_{z,b}] = \left[\begin{array}{c|c|c} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{array} \right].$$

Reading the array by columns shows the substitution directly: $[1 \ 0 \ 0]^\top$ is column one, $[0 \ c_\theta \ s_\theta]^\top$ is column two, and $[0 \ -s_\theta \ c_\theta]^\top$ is column three. Therefore,

$$\boxed{\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{bmatrix}}.$$

The column placement can be verified by multiplying $\mathbf{R}_x(\theta)$ by each standard basis column:

$$\begin{aligned} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} &= \mathbf{R}_x(\theta) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \\ \begin{bmatrix} 0 \\ c_\theta \\ s_\theta \end{bmatrix} &= \mathbf{R}_x(\theta) \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \\ \begin{bmatrix} 0 \\ -s_\theta \\ c_\theta \end{bmatrix} &= \mathbf{R}_x(\theta) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. \end{aligned}$$

The three input columns select columns one, two, and three of $\mathbf{R}_x(\theta)$, respectively. These products verify the assembled matrix; the preceding geometric projection determined the column values.

For a positive rotation about the world y_w -axis, the y_b -axis remains fixed. The right-hand rule sends the positive x_b -axis toward negative z_w and the positive z_b -axis toward positive x_w . Their world-frame coordinate columns are therefore

$${}^w\mathbf{e}_{x,b} = \begin{bmatrix} c_\theta \\ 0 \\ -s_\theta \end{bmatrix}, \quad {}^w\mathbf{e}_{y,b} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad {}^w\mathbf{e}_{z,b} = \begin{bmatrix} s_\theta \\ 0 \\ c_\theta \end{bmatrix}.$$

For a positive rotation about the world z_w -axis, the z_b -axis remains fixed, the positive x_b -axis rotates toward positive y_w , and the positive y_b -axis rotates toward negative x_w . Hence,

$${}^w\mathbf{e}_{x,b} = \begin{bmatrix} c_\theta \\ s_\theta \\ 0 \end{bmatrix}, \quad {}^w\mathbf{e}_{y,b} = \begin{bmatrix} -s_\theta \\ c_\theta \\ 0 \end{bmatrix}, \quad {}^w\mathbf{e}_{z,b} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Placing each set of three world-frame body-axis coordinates into the corresponding matrix columns gives

$$\boxed{\mathbf{R}_y(\theta) = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}, \quad \mathbf{R}_z(\theta) = \begin{bmatrix} c_\theta & -s_\theta & 0 \\ s_\theta & c_\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}}.$$

Each matrix has right-handed orthonormal columns and therefore belongs to $SO(3)$.

7.3.1.1 The active-rotation interpretation

The same numerical matrices can also be used as **active rotation operators**. An active rotation changes a geometric vector while the frame used to describe it remains fixed. Let $\mathbf{v}$ denote the vector before rotation and $\mathbf{v}'$ the vector after rotation. Their world-frame coordinate columns satisfy

$$\boxed{{}^w\mathbf{v}' = \mathbf{R}_i(\theta) {}^w\mathbf{v}, \quad i \in \{x, y, z\}}.$$

The prime labels the vector after rotation and does not denote a derivative. The coordinate-change equation describes one geometric vector in two frames; the active-rotation equation describes two geometric vectors in one fixed frame.

Why can the same numerical matrix perform both calculations? Suppose frame $\{b\}$ initially coincides with frame $\{w\}$ and is then physically rotated. The active rotation sends the original world-axis directions to the new body-axis directions:

$$\mathbf{e}_{x,w} \mapsto {}^w\mathbf{e}_{x,b}, \quad \mathbf{e}_{y,w} \mapsto {}^w\mathbf{e}_{y,b}, \quad \mathbf{e}_{z,w} \mapsto {}^w\mathbf{e}_{z,b}.$$

A linear transformation is determined by the images of its basis vectors, and those images form its matrix columns. Its active-rotation matrix is therefore

$$\begin{bmatrix} {}^w\mathbf{e}_{x,b} & {}^w\mathbf{e}_{y,b} & {}^w\mathbf{e}_{z,b} \end{bmatrix} = \mathbf{R}_{wb}.$$

The identical columns also re-express a fixed vector from body coordinates to world coordinates. The numerical operation is the same, but the labels distinguish a changed vector in one frame from an unchanged vector described in two frames.

7.3.2 Composition order: the rightmost rotation acts first

Let $\mathbf{Q}_1, \mathbf{Q}_2 \in SO(3)$ be two active rotation operators whose inputs and outputs are expressed in the same fixed frame. If $\mathbf{Q}_1$ acts first and $\mathbf{Q}_2$ acts second, then

$$\mathbf{v}' = \mathbf{Q}_1 \mathbf{v}, \quad \mathbf{v}'' = \mathbf{Q}_2 \mathbf{v}' = \mathbf{Q}_2 \mathbf{Q}_1 \mathbf{v}.$$

Thus, in the product $\mathbf{Q}_2 \mathbf{Q}_1 \mathbf{v}$, the rightmost matrix $\mathbf{Q}_1$ acts first. This is a consequence of function composition, not a special convention invented for rotations.

Spatial rotations generally do not commute. To demonstrate this, define

$$\mathbf{R}_x := \mathbf{R}_x\left(\frac{\pi}{2}\right), \quad \mathbf{R}_y := \mathbf{R}_y\left(\frac{\pi}{2}\right), \quad \mathbf{e}_z = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Rotating first about x and then about y gives

$$\mathbf{R}_y \mathbf{R}_x \mathbf{e}_z = \mathbf{R}_y \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix}.$$

Reversing the order gives

$$\mathbf{R}_x \mathbf{R}_y \mathbf{e}_z = \mathbf{R}_x \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

The outputs differ, so

$$\boxed{\mathbf{R}_y \mathbf{R}_x \neq \mathbf{R}_x \mathbf{R}_y}.$$

This calculation assumes that both rotations are active rotations about axes of the same fixed frame. Changing which frame supplies the rotation axes changes the meaning of the sequence.

7.3.3 Fixed-frame and body-frame increments

Consider a rigid body moving while the world frame $\{w\}$ remains fixed. Before the body turns, its three body-fixed axes point in particular world-frame directions. The body then turns, carrying all its body-fixed points and axes with it, and the axes point in new world-frame directions afterward.

An **orientation** describes this rotational state at one instant; it answers how the body axes currently point relative to a reference frame. A **turn** is the physical rotational motion that changes one orientation into another over a time interval. Let frame $\{b\}$ denote the body axes before the turn and frame $\{b'\}$ denote the same body axes afterward. Then $\mathbf{R}_{wb} \in SO(3)$ is the orientation before the turn, whereas $\mathbf{R}_{wb'} \in SO(3)$ is the orientation afterward.

A rotation matrix $\mathbf{Q} \in SO(3)$ that describes only the change from the old orientation to the new orientation is called an **incremental rotation**. The word *incremental* describes the matrix's role as an orientation change; $\mathbf{Q}$ is still an ordinary rotation matrix, and the turn need not be infinitesimally small. Conceptually,

$$\boxed{\text{new orientation} = \text{rotation change} \circ \text{old orientation}},$$

where $\circ$ denotes composition: apply the old orientation first and then apply the rotation change. Matrix multiplication will represent this composition, but the side on which $\mathbf{Q}$ multiplies depends on the coordinate frame used to describe the change.

First describe the change using world-frame coordinates and denote it by $\mathbf{Q}_w \in SO(3)$. The subscript w means that $\mathbf{Q}_w$ acts on world-coordinate columns. For any direction fixed in the body, if ${}^w\mathbf{v}_{\text{old}} \in \mathbb{R}^3$ and ${}^w\mathbf{v}_{\text{new}} \in \mathbb{R}^3$ are its world-coordinate columns before and after the turn, respectively, then the definition of $\mathbf{Q}_w$ is

$${}^w\mathbf{v}_{\text{new}} = \mathbf{Q}_w {}^w\mathbf{v}_{\text{old}}.$$

Write the current orientation as

$$\mathbf{R}_{wb} = [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{r}_3],$$

where $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbb{R}^3$ are the current world-frame coordinate columns of the body x_b -, y_b -, and z_b -axes. Applying $\mathbf{Q}_w$ to the physical body rotates each of these three axes, so the updated body-axis columns are

$${}^w\mathbf{e}_{x,b'} = \mathbf{Q}_w \mathbf{r}_1, \quad {}^w\mathbf{e}_{y,b'} = \mathbf{Q}_w \mathbf{r}_2, \quad {}^w\mathbf{e}_{z,b'} = \mathbf{Q}_w \mathbf{r}_3.$$

The updated orientation matrix is constructed by placing these new body-axis coordinates into its columns:

$$\begin{aligned} \mathbf{R}_{wb'} &= [{}^w\mathbf{e}_{x,b'} \quad {}^w\mathbf{e}_{y,b'} \quad {}^w\mathbf{e}_{z,b'}] \\ &= [\mathbf{Q}_w \mathbf{r}_1 \quad \mathbf{Q}_w \mathbf{r}_2 \quad \mathbf{Q}_w \mathbf{r}_3] \\ &= \mathbf{Q}_w [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{r}_3] \\ &= \boxed{\mathbf{Q}_w \mathbf{R}_{wb}}. \end{aligned}$$

Thus, an increment described using world-frame axes **premultiplies** the current orientation, where *premultiply* means multiply on the left. This product answers the practical question posed at the start: given the old orientation and an additional world-described rotation, what is the new orientation?

As an alternative, suppose the increment is described relative to the current body axes. Let $\mathbf{Q}_b := \mathbf{R}_{bb'} \in SO(3)$ describe the orientation of the new body frame $\{b'\}$ relative to the old body frame $\{b\}$. Coordinate-frame composition gives

$$\boxed{\mathbf{R}_{wb'} = \mathbf{R}_{wb} \mathbf{R}_{bb'} = \mathbf{R}_{wb} \mathbf{Q}_b}.$$

The matrices in this composition have different roles:

- $\mathbf{R}_{wb'}$ is the resulting orientation of the body relative to the world frame.

- $\mathbf{R}_{wb}$ is the body's orientation relative to the world frame before the turn.
- $\mathbf{R}_{bb'} = \mathbf{Q}_b$ is the orientation of the new body frame $\{b'\}$ relative to the old body frame $\{b\}$; it represents the incremental rotation expressed along the old body axes.

Thus, composing the old world-relative orientation with the body-relative orientation change produces the new world-relative orientation.

The body-frame increment therefore **postmultiplies** the current orientation, where *post-multiply* means multiply on the right. These rules can be remembered from their frame labels:

$$\underbrace{\mathbf{R}_{wb}}_{b \rightarrow w} \underbrace{\mathbf{R}_{bb'}}_{b' \rightarrow b} = \underbrace{\mathbf{R}_{wb'}}_{b' \rightarrow w}.$$

The two update formulas are not interchangeable. Premultiplication applies a rotation about an axis fixed in the world description; postmultiplication applies a rotation about an axis attached to the current body description.

Quick test C: coordinate-axis rotations and composition

1. Starting from ${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}$, derive the columns of $\mathbf{R}_y(\theta)$ from the body axes expressed in world coordinates.
2. In $\mathbf{Q}_3\mathbf{Q}_2\mathbf{Q}_1\mathbf{v}$, which rotation acts first?
3. Use $\mathbf{e}_z$ to show that $\mathbf{R}_y(\pi/2)\mathbf{R}_x(\pi/2) \neq \mathbf{R}_x(\pi/2)\mathbf{R}_y(\pi/2)$.
4. If an orientation increment is expressed about world-frame axes, on which side of $\mathbf{R}_{wb}$ does it multiply?
5. If $\mathbf{Q}_b = \mathbf{R}_{bb'}$ is an increment relative to the current body frame, derive the updated orientation $\mathbf{R}_{wb'}$ from the frame labels.
6. Distinguish an active rotation from a coordinate change.

Answers and hints

1. After a positive rotation about y_w , the world-frame body-axis columns are $[c_\theta \ 0 \ -s_\theta]^\top$, $[0 \ 1 \ 0]^\top$, and $[s_\theta \ 0 \ c_\theta]^\top$. Placing them into the first, second, and third columns gives $\mathbf{R}_y(\theta)$.
2. $\mathbf{Q}_1$ acts first, followed by $\mathbf{Q}_2$ and then $\mathbf{Q}_3$.
3. The first product maps $\mathbf{e}_z$ to $-\mathbf{e}_y$, while the reversed product maps it to $\mathbf{e}_x$; therefore, the operators are unequal.
4. It premultiplies: $\mathbf{R}'_{wb} = \mathbf{Q}_w\mathbf{R}_{wb}$.
5. The labels compose as $(b \rightarrow w)(b' \rightarrow b) = (b' \rightarrow w)$, so $\mathbf{R}_{wb'} = \mathbf{R}_{wb}\mathbf{R}_{bb'} = \mathbf{R}_{wb}\mathbf{Q}_b$.

6. An active rotation changes a geometric vector while retaining one coordinate frame. A coordinate change keeps the geometric vector fixed and changes the frame used to express its coordinate column.

7.4 Angular velocity as the rate of orientation change

7.4.1 From a finite turn to an instantaneous rate

The preceding section compared the body's orientation before and after a finite turn. Motion also requires a rate that describes how rapidly that orientation is changing at one instant. Consider a rigid body whose orientation relative to the fixed world frame is the differentiable function $\mathbf{R}_{wb}(t) \in SO(3)$, where t is time measured in seconds.

Over a time interval from t to $t + \Delta t$, where $\Delta t > 0$ is measured in seconds, the body undergoes a relative rotation from $\mathbf{R}_{wb}(t)$ to $\mathbf{R}_{wb}(t + \Delta t)$. **Euler's rotation theorem**, used here as a stated geometric result, says that every relative orientation change in $SO(3)$ can be represented as one rotation through an angle about an axis direction. Accordingly, let $\mathbf{a}_w(t, \Delta t) \in \mathbb{R}^3$ be a unit rotation-axis column and let $\Delta\theta(t, \Delta t) \in \mathbb{R}$ be the signed rotation angle measured in radians. The subscript w states that the axis components are expressed along the world axes, and

$$\|\mathbf{a}_w(t, \Delta t)\|_2 = 1.$$

The **rotation axis** in an orientation description specifies only a direction $\mathbf{a}_w$. It does not identify a particular line in physical space. To specify a particular line, also choose the world position $\mathbf{q}_w \in \mathbb{R}^3$ of one point on it, measured in metres. The line is then

$$\mathcal{L} := \{\mathbf{q}_w + \lambda \mathbf{a}_w \mid \lambda \in \mathbb{R}\} \subset \mathbb{R}^3,$$

where $\mathcal{L}$ denotes the axis line and λ is a scalar distance measured in metres. Because $\mathbf{R}_{wb}$ describes orientation only, it contains the axis direction associated with the rotation but not the location of a physical pivot line.

The sign of $\Delta\theta$ follows the right-hand rule about $\mathbf{a}_w$. Replacing $\mathbf{a}_w$ by $-\mathbf{a}_w$ and $\Delta\theta$ by $-\Delta\theta$ describes the same turn, so their product is unchanged.

The vector

$$\mathbf{a}_w(t, \Delta t) \frac{\Delta\theta(t, \Delta t)}{\Delta t}$$

describes the rotation-change rate over that interval: its direction gives the rotation axis and its signed magnitude gives the angle change per unit time. Taking the limit as the

interval shrinks defines the body's **angular velocity relative to the world frame, expressed in world coordinates**:

$$\boldsymbol{\omega}_w(t) := \lim_{\Delta t \rightarrow 0^+} \mathbf{a}_w(t, \Delta t) \frac{\Delta \theta(t, \Delta t)}{\Delta t} \in \mathbb{R}^3.$$

The unit axis $\mathbf{a}_w$ is dimensionless, while $\Delta \theta / \Delta t$ has units of rad s^{-1} . Therefore, $\boldsymbol{\omega}_w$ has units of rad s^{-1} . The limit is assumed to exist because $\mathbf{R}_{wb}(t)$ is differentiable.

Orientation and angular velocity describe different physical quantities. The matrix $\mathbf{R}_{wb}(t)$ is the rotational state at time t , whereas $\boldsymbol{\omega}_w(t)$ is the instantaneous rate at which that state is changing. Knowing the orientation at one instant does not determine the angular velocity: a stationary body and a turning body can momentarily have the same orientation but different angular velocities.

To connect angular velocity to the orientation matrix, write its three columns explicitly:

$$\mathbf{R}_{wb}(t) = [\mathbf{r}_1(t) \quad \mathbf{r}_2(t) \quad \mathbf{r}_3(t)].$$

For each $i \in \{1, 2, 3\}$, the dimensionless column $\mathbf{r}_i(t) \in \mathbb{R}^3$ is the world-coordinate representation of one unit body axis fixed in the rigid body. Thus, $\|\mathbf{r}_i(t)\|_2 = 1$. The geometric construction below selects one column $\mathbf{r}_i$; Figure 7.1 illustrates the choice $i = 1$ and draws the other two columns as dashed vectors.

When $\boldsymbol{\omega}_w \neq \mathbf{0}_3$, define its unit direction by $\mathbf{a}_w := \boldsymbol{\omega}_w / \|\boldsymbol{\omega}_w\|_2$ and let $\varphi_i \in [0, \pi]$ be the angle between $\mathbf{a}_w$ and $\mathbf{r}_i$. Decompose the selected column into components parallel and perpendicular to the angular-velocity direction:

$$\mathbf{r}_{i,\parallel} := (\mathbf{a}_w^\top \mathbf{r}_i) \mathbf{a}_w, \quad \mathbf{r}_{i,\perp} := \mathbf{r}_i - \mathbf{r}_{i,\parallel}.$$

For the drawing, place the tail of these direction vectors at a common coordinate origin O and call the tip of $\mathbf{r}_i$ point P_i . The point P_i is a graphical endpoint, not necessarily a material point on the body. Let C_i be the tip of $\mathbf{r}_{i,\parallel}$. The segment from C_i to P_i represents $\mathbf{r}_{i,\perp}$ and is perpendicular to the angular-velocity direction.

The figure is a two-dimensional projection of three-dimensional geometry. The orange radius vector $\mathbf{r}_{1,\perp}$, whose length is ρ_1 , is perpendicular to $\boldsymbol{\omega}_w$, and $\mathbf{r}_1$ is perpendicular to $\mathbf{r}_{1,\perp}$ in three dimensions. The right-angle marker at P_1 therefore appears skewed in the projection even though it denotes a true three-dimensional right angle.

At the selected instant, the tip P_i has the same tangent direction as a point moving on a circle centred at C_i . The circle radius is the length of the perpendicular component:

$$\rho_i = \|\mathbf{r}_{i,\perp}\|_2 = \|\mathbf{r}_i\|_2 \sin \varphi_i = \sin \varphi_i.$$

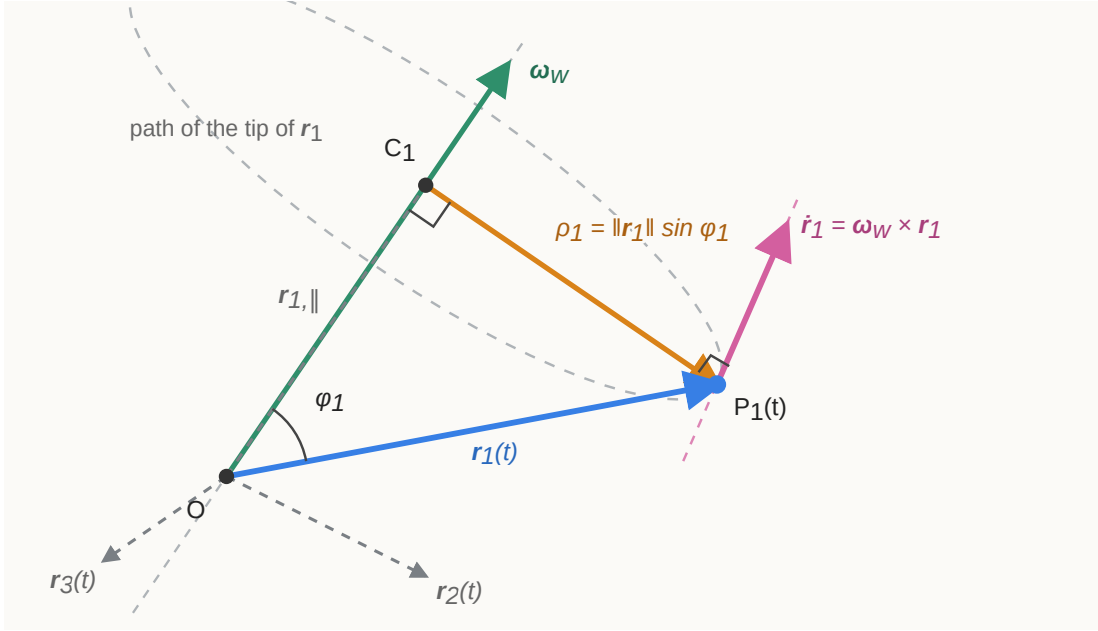


Figure 7.1: Angular velocity and the motion of one orientation-matrix column. The solid vector $\mathbf{r}_1$ is the first column of $\mathbf{R}_{wb}$; the dashed vectors $\mathbf{r}_2$ and $\mathbf{r}_3$ are its other columns.

The last equality uses $\|\mathbf{r}_i\|_2 = 1$. Multiplying the radius by the angular speed gives the speed of the tip:

$$\|\dot{\mathbf{r}}_i\|_2 = \|\boldsymbol{\omega}_w\|_2 \rho_i = \|\boldsymbol{\omega}_w\|_2 \|\mathbf{r}_i\|_2 \sin \varphi_i.$$

This is exactly the magnitude of $\boldsymbol{\omega}_w \times \mathbf{r}_i$, and the cross product's right-hand-rule direction is the tangent direction shown in the figure. When $\boldsymbol{\omega}_w = \mathbf{0}_3$, both the cross product and the column's rate are zero. Therefore, for each of the three columns,

$$\boxed{\dot{\mathbf{r}}_i(t) = \boldsymbol{\omega}_w(t) \times \mathbf{r}_i(t), \quad i \in \{1, 2, 3\}}.$$

These equations describe how the three columns of the orientation matrix change. To combine them into one equation for $\dot{\mathbf{R}}_{wb}$, the cross product can be represented as a matrix multiplication.

7.4.2 Cross-product matrices and the orientation derivative

Let

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^3.$$

The **cross-product matrix** of $\mathbf{x}$ is defined by

$$[\mathbf{x}]_{\times} := \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix} \in \mathbb{R}^{3 \times 3}.$$

The notation $[\cdot]_{\times}$ means “form the matrix that performs a cross product with the enclosed vector.” For any $\mathbf{y} = [y_1 \ y_2 \ y_3]^T \in \mathbb{R}^3$, direct multiplication gives

$$\begin{aligned} [\mathbf{x}]_{\times} \mathbf{y} &= \begin{bmatrix} -x_3 y_2 + x_2 y_3 \\ x_3 y_1 - x_1 y_3 \\ -x_2 y_1 + x_1 y_2 \end{bmatrix} \\ &= \boxed{\mathbf{x} \times \mathbf{y}}. \end{aligned}$$

Thus, $[\mathbf{x}]_{\times}$ is a linear mapping from an input vector $\mathbf{y} \in \mathbb{R}^3$ to the output vector $\mathbf{x} \times \mathbf{y} \in \mathbb{R}^3$. Its entries have the same units as the components of $\mathbf{x}$.

Transposing the matrix changes the sign of every off-diagonal entry:

$$[\mathbf{x}]_{\times}^T = -[\mathbf{x}]_{\times}.$$

A square matrix $\mathbf{A}$ satisfying $\mathbf{A}^T = -\mathbf{A}$ is called **skew-symmetric**. The set of all real 3×3 skew-symmetric matrices is

$$\mathfrak{so}(3) := \{\mathbf{A} \in \mathbb{R}^{3 \times 3} \mid \mathbf{A}^T = -\mathbf{A}\} = \{[\mathbf{x}]_{\times} \mid \mathbf{x} \in \mathbb{R}^3\}.$$

The second equality follows from the structure imposed by skew symmetry. The diagonal entries must be zero, and each entry below the diagonal must be the negative of the corresponding entry above it. Hence, every $\mathbf{A} \in \mathfrak{so}(3)$ is determined by three scalars and can be written uniquely as

$$\mathbf{A} = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix} = [\mathbf{x}]_{\times}.$$

As in the planar case developed in Section 6.3.3, $\mathfrak{so}(3)$ is the **Lie algebra** of $SO(3)$: it is the vector space of instantaneous rotation directions at the identity rotation $\mathbf{I}_3$. A finite rotation matrix belongs to $SO(3)$, whereas an angular-velocity matrix belongs to $\mathfrak{so}(3)$ and describes a rate rather than an orientation.

Apply the cross-product matrix to the three column-rate equations from Section 7.4.1:

$$\begin{aligned}\dot{\mathbf{R}}_{wb} &= [\dot{\mathbf{r}}_1 \quad \dot{\mathbf{r}}_2 \quad \dot{\mathbf{r}}_3] \\ &= [[\boldsymbol{\omega}_w]_{\times} \mathbf{r}_1 \quad [\boldsymbol{\omega}_w]_{\times} \mathbf{r}_2 \quad [\boldsymbol{\omega}_w]_{\times} \mathbf{r}_3] \\ &= [\boldsymbol{\omega}_w]_{\times} [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{r}_3].\end{aligned}$$

Because the final matrix of columns is $\mathbf{R}_{wb}$,

$$\boxed{\dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb}}.$$

Here, $\mathbf{R}_{wb} \in SO(3)$ is dimensionless, $\boldsymbol{\omega}_w \in \mathbb{R}^3$ has units of rad s^{-1} , and both $[\boldsymbol{\omega}_w]_{\times}$ and $\dot{\mathbf{R}}_{wb}$ have units of s^{-1} because radians are dimensionless in SI. This equation says that the world-coordinate rate of every body-axis column is produced by the same world-expressed angular velocity.

Right-multiplying by $\mathbf{R}_{wb}^T = \mathbf{R}_{wb}^{-1}$ isolates the angular-velocity matrix:

$$\begin{aligned}\dot{\mathbf{R}}_{wb} \mathbf{R}_{wb}^T &= [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb} \mathbf{R}_{wb}^T \\ &= [\boldsymbol{\omega}_w]_{\times} \mathbf{I}_3.\end{aligned}$$

Therefore,

$$\boxed{[\boldsymbol{\omega}_w]_{\times} = \dot{\mathbf{R}}_{wb} \mathbf{R}_{wb}^T}.$$

This formula recovers the angular velocity expressed along the world axes from the orientation and its derivative.

The same physical angular velocity can instead be expressed along the moving body axes. Define

$$\boxed{\boldsymbol{\omega}_b := \mathbf{R}_{wb}^T \boldsymbol{\omega}_w \in \mathbb{R}^3}.$$

The subscript b does not mean that the body rotates relative to itself. It means that the body's angular velocity relative to the world is resolved into components along x_b , y_b , and z_b .

Claim to prove. The world-expressed orientation-rate equation derived above is

$$\dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb}.$$

We claim that the same orientation rate can be written using the body-expressed angular velocity as

$$\boxed{\dot{\mathbf{R}}_{wb} = \mathbf{R}_{wb} [\boldsymbol{\omega}_b]_{\times}}.$$

This is the result to be proved. It states that the world-expressed angular-velocity matrix premultiplies $\mathbf{R}_{wb}$, whereas the body-expressed angular-velocity matrix postmultiplies it.

It is sufficient to prove the intermediate coordinate-conversion identity

$$\boxed{[\boldsymbol{\omega}_w]_{\times} = \mathbf{R}_{wb} [\boldsymbol{\omega}_b]_{\times} \mathbf{R}_{wb}^{\top}}.$$

Indeed, substituting this identity into $\dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb}$ will make $\mathbf{R}_{wb}^{\top} \mathbf{R}_{wb} = \mathbf{I}_3$ cancel and produce the claimed body-expressed equation. The remaining task is therefore to prove how a cross-product matrix changes between rotated coordinate frames.

Let $\mathbf{R} \in SO(3)$ map coordinate columns from a source frame to a target frame, and let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ be arbitrary vector-coordinate columns in the source frame. The letters $\mathbf{a}$ and $\mathbf{b}$ are generic placeholders; $\mathbf{a}$ will later be replaced by $\boldsymbol{\omega}_b$. Their target-frame coordinate columns are $\mathbf{Ra}$ and $\mathbf{Rb}$. The general transformation law from Section 1.3.4 is

$$(\mathbf{Aa}) \times (\mathbf{Ab}) = \det(\mathbf{A}) \mathbf{A}^{-\top} (\mathbf{a} \times \mathbf{b}).$$

Set $\mathbf{A} = \mathbf{R}$. Because a proper rotation satisfies $\det(\mathbf{R}) = 1$ and $\mathbf{R}^{-\top} = \mathbf{R}$, the general law reduces to

$$\boxed{\mathbf{R}(\mathbf{a} \times \mathbf{b}) = (\mathbf{Ra}) \times (\mathbf{Rb})}, \quad \mathbf{a}, \mathbf{b} \in \mathbb{R}^3.$$

The left-hand side first computes $\mathbf{a} \times \mathbf{b}$ in the source coordinates and then converts the result to the target coordinates. The right-hand side first converts both inputs and then computes their cross product in the target coordinates. The equality says that both procedures describe the same geometric cross-product vector.

We now use this vector identity to convert the cross-product matrix. Keep $\mathbf{a}$ as the source-frame vector that generates $[\mathbf{a}]_{\times}$, and let $\mathbf{y} \in \mathbb{R}^3$ be an arbitrary target-frame input column. In the product $\mathbf{R}[\mathbf{a}]_{\times} \mathbf{R}^{\top} \mathbf{y}$, the operations occur from right to left: $\mathbf{R}^{\top}$ converts $\mathbf{y}$ to the source coordinates, $[\mathbf{a}]_{\times}$ forms the cross product there, and $\mathbf{R}$ converts the result back to the target coordinates. Algebraically,

$$\begin{aligned}
 \mathbf{R}[\mathbf{a}]_{\times} \mathbf{R}^T \mathbf{y} &= \mathbf{R}(\mathbf{a} \times (\mathbf{R}^T \mathbf{y})) \\
 &= (\mathbf{R}\mathbf{a}) \times (\mathbf{R}\mathbf{R}^T \mathbf{y}) \\
 &= (\mathbf{R}\mathbf{a}) \times \mathbf{y} \\
 &= [\mathbf{R}\mathbf{a}]_{\times} \mathbf{y}.
 \end{aligned}$$

Because the two matrices give the same output for every $\mathbf{y}$, they are equal:

$$\boxed{\mathbf{R}[\mathbf{a}]_{\times} \mathbf{R}^T = [\mathbf{R}\mathbf{a}]_{\times}}.$$

Finally, specialize the generic identity to the angular velocity by setting $\mathbf{R} = \mathbf{R}_{wb}$ and $\mathbf{a} = \boldsymbol{\omega}_b$. The definition $\boldsymbol{\omega}_b = \mathbf{R}_{wb}^T \boldsymbol{\omega}_w$ is equivalent to $\boldsymbol{\omega}_w = \mathbf{R}_{wb} \boldsymbol{\omega}_b$, so

$$\boxed{\mathbf{R}_{wb}[\boldsymbol{\omega}_b]_{\times} \mathbf{R}_{wb}^T = [\mathbf{R}_{wb} \boldsymbol{\omega}_b]_{\times} = [\boldsymbol{\omega}_w]_{\times}}.$$

This is exactly the required intermediate identity. Substituting it into the known world-expressed orientation-rate equation completes the proof:

$$\begin{aligned}
 \dot{\mathbf{R}}_{wb} &= [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb} \\
 &= \mathbf{R}_{wb}[\boldsymbol{\omega}_b]_{\times} \mathbf{R}_{wb}^T \mathbf{R}_{wb} \\
 &= \boxed{\mathbf{R}_{wb}[\boldsymbol{\omega}_b]_{\times}}.
 \end{aligned}$$

The same intermediate identity can be rearranged to recover the body-expressed cross-product matrix. Premultiplying by $\mathbf{R}_{wb}^T$ and postmultiplying by $\mathbf{R}_{wb}$ gives

$$[\boldsymbol{\omega}_b]_{\times} = \mathbf{R}_{wb}^T [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb}.$$

Substituting $[\boldsymbol{\omega}_w]_{\times} = \dot{\mathbf{R}}_{wb} \mathbf{R}_{wb}^T$ then gives

$$\begin{aligned}
 [\boldsymbol{\omega}_b]_{\times} &= \mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb} \mathbf{R}_{wb}^T \mathbf{R}_{wb} \\
 &= \mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb}.
 \end{aligned}$$

The two coordinate descriptions and the corresponding orientation-rate equations are therefore

$$\boxed{\begin{aligned} [\boldsymbol{\omega}_w]_{\times} &= \dot{\mathbf{R}}_{wb} \mathbf{R}_{wb}^T, & \dot{\mathbf{R}}_{wb} &= [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb}, \\ [\boldsymbol{\omega}_b]_{\times} &= \mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb}, & \dot{\mathbf{R}}_{wb} &= \mathbf{R}_{wb} [\boldsymbol{\omega}_b]_{\times}. \end{aligned}}$$

7.4.2.1 Connection to finite fixed-frame and body-frame increments

The multiplication sides follow directly from the finite composition rules in Section 7.3.3. To see this, let

$$\mathbf{R} := \mathbf{R}_{wb}(t), \quad \mathbf{R}^+ := \mathbf{R}_{wb}(t + \Delta t),$$

where $\mathbf{R}$ is the orientation at time t , $\mathbf{R}^+$ is the orientation after a short interval $\Delta t > 0$, and both matrices belong to $SO(3)$.

First express the orientation change using the fixed world axes. The finite composition rule is

$$\mathbf{R}^+ = \mathbf{Q}_w \mathbf{R},$$

where $\mathbf{Q}_w \in SO(3)$ is the incremental rotation expressed using world coordinates. At $\Delta t = 0$, there is no orientation change, so $\mathbf{Q}_w(t, 0) = \mathbf{I}_3$. Its derivative with respect to the interval length at zero is the world-expressed angular-velocity matrix $[\boldsymbol{\omega}_w(t)]_\times$. The first-order Taylor expansion is therefore

$$\mathbf{Q}_w = \mathbf{I}_3 + \Delta t [\boldsymbol{\omega}_w]_\times + O(\Delta t^2).$$

Substituting this expansion into the finite composition rule gives

$$\begin{aligned} \mathbf{R}^+ &= (\mathbf{I}_3 + \Delta t [\boldsymbol{\omega}_w]_\times + O(\Delta t^2)) \mathbf{R} \\ &= \mathbf{R} + \Delta t [\boldsymbol{\omega}_w]_\times \mathbf{R} + O(\Delta t^2). \end{aligned}$$

Subtract $\mathbf{R}$, divide by Δt , and take the limit $\Delta t \rightarrow 0$:

$$\boxed{\dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_w]_\times \mathbf{R}_{wb}}.$$

Thus, a rotation change expressed using the fixed world axes premultiplies the finite orientation, and its world-expressed angular-velocity matrix likewise multiplies the orientation on the left.

Now express the same orientation change using the old body axes. The finite composition rule is

$$\mathbf{R}^+ = \mathbf{R} \mathbf{Q}_b,$$

where $\mathbf{Q}_b \in SO(3)$ is the incremental rotation expressed using body coordinates. Similarly, $\mathbf{Q}_b(t, 0) = \mathbf{I}_3$, and its derivative with respect to the interval length at zero is $[\boldsymbol{\omega}_b(t)]_\times$. Its first-order Taylor expansion is

$$\mathbf{Q}_b = \mathbf{I}_3 + \Delta t[\boldsymbol{\omega}_b]_{\times} + O(\Delta t^2).$$

Substitution gives

$$\begin{aligned}\mathbf{R}^+ &= \mathbf{R}(\mathbf{I}_3 + \Delta t[\boldsymbol{\omega}_b]_{\times} + O(\Delta t^2)) \\ &= \mathbf{R} + \Delta t\mathbf{R}[\boldsymbol{\omega}_b]_{\times} + O(\Delta t^2).\end{aligned}$$

Taking the same difference-quotient limit gives

$$\boxed{\dot{\mathbf{R}}_{wb} = \mathbf{R}_{wb}[\boldsymbol{\omega}_b]_{\times}}.$$

Thus, a rotation change expressed using the moving body axes postmultiplies the finite orientation, and its body-expressed angular-velocity matrix likewise multiplies the orientation on the right. Both equations describe the same physical turn; only the axes used to express that turn differ.

Coordinates used for the change	Finite composition	Instantaneous orientation rate
Fixed world axes	$\mathbf{R}^+ = \mathbf{Q}_w \mathbf{R}$	$\dot{\mathbf{R}} = [\boldsymbol{\omega}_w]_{\times} \mathbf{R}$
Moving body axes	$\mathbf{R}^+ = \mathbf{R} \mathbf{Q}_b$	$\dot{\mathbf{R}} = \mathbf{R}[\boldsymbol{\omega}_b]_{\times}$

Finally, these rate matrices are necessarily skew-symmetric. Differentiating the rotation constraint $\mathbf{R}_{wb}^T \mathbf{R}_{wb} = \mathbf{I}_3$ gives

$$\dot{\mathbf{R}}_{wb}^T \mathbf{R}_{wb} + \mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb} = \mathbf{0}_{3 \times 3}.$$

Therefore, $(\mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb})^T = -\mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb}$, confirming that $[\boldsymbol{\omega}_b]_{\times} \in \mathfrak{so}(3)$. Differentiating $\mathbf{R}_{wb} \mathbf{R}_{wb}^T = \mathbf{I}_3$ similarly confirms that $[\boldsymbol{\omega}_w]_{\times} = \dot{\mathbf{R}}_{wb} \mathbf{R}_{wb}^T \in \mathfrak{so}(3)$. These products remove the current finite orientation and express its instantaneous change at the identity rotation.

Quick test D: angular velocity

1. What physical information is contained in the direction and magnitude of $\boldsymbol{\omega}_w$?
2. Why does knowing $\mathbf{R}_{wb}(t)$ at one instant not determine $\boldsymbol{\omega}_w(t)$?
3. Explain geometrically why a body-axis column satisfies $\dot{\mathbf{r}}_i = \boldsymbol{\omega}_w \times \mathbf{r}_i$.
4. Suppose $\boldsymbol{\omega}_w = [0 \ 0 \ 2]^T \text{ rad s}^{-1}$ and $\mathbf{r}_i = [1 \ 0 \ 0]^T$. Calculate $\dot{\mathbf{r}}_i$ and interpret its direction.

5. Construct $[\mathbf{x}]_{\times}$ for $\mathbf{x} = [1 \ 2 \ 3]^T$ and verify that it is skew-symmetric.
6. Starting from $\dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb}$, isolate $[\boldsymbol{\omega}_w]_{\times}$.
7. Do $\boldsymbol{\omega}_w$ and $\boldsymbol{\omega}_b$ describe different physical turns? Explain their relationship.
8. Why is $\mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb}$ skew-symmetric rather than a rotation matrix?

Answers and hints

1. Its direction gives the instantaneous rotation axis according to the right-hand rule, and its magnitude gives the angular speed in rad s^{-1} .
2. Orientation is a state, whereas angular velocity is its rate of change. Different orientation trajectories can pass through the same $\mathbf{R}_{wb}(t)$ with different derivatives.
3. The tip of $\mathbf{r}_i$ has a tangent velocity about the instantaneous rotation-axis direction. The cross product has that tangent direction and magnitude $\|\boldsymbol{\omega}_w\|_2 \|\mathbf{r}_i\|_2 \sin \varphi_i$.
4. $\dot{\mathbf{r}}_i = \boldsymbol{\omega}_w \times \mathbf{r}_i = [0 \ 2 \ 0]^T \text{ s}^{-1}$. The direction initially points toward positive y_w , as expected for a positive turn about z_w .
5. $[\mathbf{x}]_{\times} = \begin{bmatrix} 0 & -3 & 2 \\ 3 & 0 & -1 \\ -2 & 1 & 0 \end{bmatrix}$, and direct transposition gives $[\mathbf{x}]_{\times}^T = -[\mathbf{x}]_{\times}$.
6. Right-multiply by $\mathbf{R}_{wb}^T$ and use $\mathbf{R}_{wb} \mathbf{R}_{wb}^T = \mathbf{I}_3$ to obtain $[\boldsymbol{\omega}_w]_{\times} = \dot{\mathbf{R}}_{wb} \mathbf{R}_{wb}^T$.
7. No. They describe the same angular velocity relative to the world but resolve it along different axes: $\boldsymbol{\omega}_b = \mathbf{R}_{wb}^T \boldsymbol{\omega}_w$.
8. Differentiating $\mathbf{R}_{wb}^T \mathbf{R}_{wb} = \mathbf{I}_3$ shows that $(\mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb})^T = -\mathbf{R}_{wb}^T \dot{\mathbf{R}}_{wb}$. It represents an instantaneous rotation rate in $\mathfrak{so}(3)$, not a finite orientation in $SO(3)$.

7.5 Finite rotations from axis–angle coordinates

7.5.1 The problem: converting a rotation rate into a finite turn

The angular-velocity equations in Section 7.4 describe an instantaneous orientation rate. A simulation, estimator, or controller also needs a finite orientation change over a nonzero interval. This section solves the forward problem: given a rotation-axis direction and a signed rotation angle, construct the corresponding matrix in $SO(3)$.

Let $\mathbf{a}_w \in \mathbb{R}^3$ be a unit vector expressed in world coordinates,

$$\|\mathbf{a}_w\|_2 = 1,$$

and let $\alpha \in \mathbb{R}$ be a signed rotation angle measured in radians. The pair $(\mathbf{a}_w, \alpha)$ is an **axis–angle representation** of the intended orientation change: $\mathbf{a}_w$ gives the positive axis direction, and the sign of α selects the direction of rotation by the right-hand rule. This

orientation-only description specifies an axis direction through the coordinate origin; locating a physical pivot line would additionally require a point, as explained in Section 7.4.1.

The three-component column

$$\boldsymbol{\varphi}_w := \alpha \mathbf{a}_w \in \mathbb{R}^3$$

is called the **rotation vector**, or the **exponential coordinates of the rotation**. Its direction gives the rotation axis and its norm gives the angle magnitude:

$$\|\boldsymbol{\varphi}_w\|_2 = |\alpha|.$$

The word *exponential* does not refer to the definition $\boldsymbol{\varphi}_w = \alpha \mathbf{a}_w$ itself. It anticipates the construction derived below: the skew-symmetric matrix $[\boldsymbol{\varphi}_w]_\times$ is supplied to the matrix exponential to recover the finite rotation $\mathbf{Q}_w = \exp([\boldsymbol{\varphi}_w]_\times)$.

If the same rotation is produced during a time interval $[t_0, t_1]$ about the fixed axis $\mathbf{a}_w$, let $\alpha(t_0) = 0$ and $\alpha(t_1) = \alpha$. The world-expressed angular velocity and accumulated angle then satisfy

$$\boldsymbol{\omega}_w(t) = \dot{\alpha}(t) \mathbf{a}_w, \quad \alpha = \int_{t_0}^{t_1} \dot{\alpha}(t) dt.$$

Because $\mathbf{a}_w$ is constant over this interval,

$$\boldsymbol{\varphi}_w = \alpha \mathbf{a}_w = \int_{t_0}^{t_1} \boldsymbol{\omega}_w(t) dt.$$

The complete fixed-axis relationship is therefore

$$\underbrace{\boldsymbol{\omega}_w(t)}_{\text{angular velocity}} \xrightarrow{\text{integrate over } [t_0, t_1]} \underbrace{\boldsymbol{\varphi}_w = \alpha \mathbf{a}_w}_{\text{accumulated rotation coordinates}} \xrightarrow{\exp([\cdot]_\times)} \underbrace{\mathbf{Q}_w}_{\text{finite rotation}}.$$

The vector $\boldsymbol{\varphi}_w$ is dimensionless in SI because radians are dimensionless, although its entries are labelled in radians to preserve their physical meaning. It represents accumulated finite rotation coordinates, not angular velocity; angular velocity has units of rad s^{-1} .

7.5.2 The matrix exponential

To construct the finite rotation, first consider a differentiable world-coordinate vector $\mathbf{p}(s) \in \mathbb{R}^3$ rotating about $\mathbf{a}_w$, where $s \in \mathbb{R}$ is the accumulated signed rotation angle, measured in radians and dimensionless in SI. The derivation in Section 7.4.1 gave $\dot{\mathbf{r}}_i = \boldsymbol{\omega}_w \times \mathbf{r}_i$ for each body-axis column; the same vector-rate equation applies to any vector rotating with the body:

$$\frac{d\mathbf{p}}{dt} = \boldsymbol{\omega}_w \times \mathbf{p}.$$

If $s = s(t)$, then $\boldsymbol{\omega}_w = \mathbf{a}_w ds/dt$. Applying the chain rule gives

$$\frac{d\mathbf{p}}{dt} = \frac{d\mathbf{p}}{ds} \frac{ds}{dt} = (\mathbf{a}_w \times \mathbf{p}) \frac{ds}{dt}.$$

Cancelling ds/dt when it is nonzero, with the stationary case obtained by continuity, gives the vector-rate equation with angle rather than time as the independent variable:

$$\frac{d\mathbf{p}}{ds} = \mathbf{a}_w \times \mathbf{p}(s) = [\mathbf{a}_w]_{\times} \mathbf{p}(s).$$

The initial condition is $\mathbf{p}(0) = \mathbf{p}_0$, where $\mathbf{p}_0 \in \mathbb{R}^3$ is the vector before the turn. The coefficient matrix $[\mathbf{a}_w]_{\times} \in \mathfrak{so}(3)$ is constant because the axis is fixed in world coordinates.

For any square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the **matrix exponential** is defined by the convergent power series

$$\exp(\mathbf{A}) := \mathbf{I}_n + \mathbf{A} + \frac{\mathbf{A}^2}{2!} + \frac{\mathbf{A}^3}{3!} + \cdots.$$

Here, $\mathbf{A}^0 := \mathbf{I}_n$, $0! := 1$, and $k! := 1 \cdot 2 \cdots k$ for each positive integer k . The dots denote an **infinite series**, defined as the limit of its finite partial sums:

$$\exp(\mathbf{A}) := \lim_{N \rightarrow \infty} \sum_{k=0}^N \frac{\mathbf{A}^k}{k!}.$$

Convergence means that every entry of these partial-sum matrices approaches a finite value as $N \rightarrow \infty$. This definition mirrors the scalar series $e^z = \sum_{k=0}^{\infty} z^k/k!$ and converges for every finite square matrix $\mathbf{A}$. For a constant matrix $\mathbf{A}$, term-by-term differentiation of the series gives

$$\frac{d}{ds} \exp(s\mathbf{A}) = \mathbf{A} \exp(s\mathbf{A}).$$

The scalar separation step dp/p cannot be used here because division by a vector and the ordinary logarithm of a vector are not defined. Its matrix counterpart is an **integrating factor**, meaning a matrix chosen so that the differential equation becomes the derivative of one product. Set $\mathbf{A} := [\mathbf{a}_w]_{\times} \in \mathbb{R}^{3 \times 3}$ and left-multiply $d\mathbf{p}/ds = \mathbf{A}\mathbf{p}$ by $\exp(-s\mathbf{A})$. The product rule gives

$$\begin{aligned} \frac{d}{ds} [\exp(-s\mathbf{A})\mathbf{p}(s)] &= -\mathbf{A} \exp(-s\mathbf{A})\mathbf{p}(s) + \exp(-s\mathbf{A}) \frac{d\mathbf{p}}{ds} \\ &= -\mathbf{A} \exp(-s\mathbf{A})\mathbf{p}(s) + \exp(-s\mathbf{A})\mathbf{A}\mathbf{p}(s) \\ &= \mathbf{0}_3. \end{aligned}$$

The last equality uses $\mathbf{A} \exp(-s\mathbf{A}) = \exp(-s\mathbf{A})\mathbf{A}$, which holds because the exponential series contains only powers of $\mathbf{A}$. The product is therefore constant. Evaluating it at $s = 0$ and using $\exp(\mathbf{0}_{3 \times 3}) = \mathbf{I}_3$ gives

$$\exp(-s\mathbf{A})\mathbf{p}(s) = \mathbf{p}_0.$$

Left-multiplying by $\exp(s\mathbf{A})$ and using $\exp(s\mathbf{A})^{-1} = \exp(-s\mathbf{A})$ yields

$$\boxed{\mathbf{p}(s) = \exp(s[\mathbf{a}_w]_{\times}) \mathbf{p}_0}.$$

This integrating-factor argument derives the solution. Differentiating the result provides a separate verification:

$$\begin{aligned} \frac{d\mathbf{p}}{ds} &= \frac{d}{ds} [\exp(s[\mathbf{a}_w]_{\times})] \mathbf{p}_0 \\ &= [\mathbf{a}_w]_{\times} \exp(s[\mathbf{a}_w]_{\times}) \mathbf{p}_0 \\ &= [\mathbf{a}_w]_{\times} \mathbf{p}(s), \end{aligned}$$

and $\mathbf{p}(0) = \exp(\mathbf{0}_{3 \times 3})\mathbf{p}_0 = \mathbf{I}_3\mathbf{p}_0 = \mathbf{p}_0$. The cross-product-matrix construction is linear, so $[\boldsymbol{\varphi}_w]_{\times} = [\alpha\mathbf{a}_w]_{\times} = \alpha[\mathbf{a}_w]_{\times}$. Setting $s = \alpha$ therefore defines the finite rotation

$$\boxed{\mathbf{Q}_w(\mathbf{a}_w, \alpha) := \exp(\alpha[\mathbf{a}_w]_{\times}) = \exp([\boldsymbol{\varphi}_w]_{\times})}.$$

The mapping

$$\exp : \mathfrak{so}(3) \rightarrow SO(3)$$

is called the **exponential map** for rotations. The arrow states that the function accepts an element of $\mathfrak{so}(3)$ and returns an element of $SO(3)$. For the rotation vector introduced above,

$$\underbrace{[\varphi_w]_{\times}}_{\in \mathfrak{so}(3)} \mapsto \underbrace{\exp([\varphi_w]_{\times})}_{\in SO(3)}.$$

Thus, the exponential map converts the dimensionless skew-symmetric matrix form $[\varphi_w]_{\times}$ of the exponential coordinates into a finite rotation matrix $\mathbf{Q}_w$. Although $[\varphi_w]_{\times}$ belongs to the tangent space $\mathfrak{so}(3)$, it need not represent an infinitesimal change: its scale is set by the possibly finite angle α . During time-dependent motion, the genuinely infinitesimal rotation coordinate accumulated over an infinitesimal time interval dt is

$$\boxed{[d\varphi_w]_{\times} = [\omega_w]_{\times} dt},$$

where $\omega_w \in \mathbb{R}^3$ is the world-expressed angular velocity in rad s^{-1} .

To verify that the result is a rotation matrix, set $\mathbf{\Omega} := [\mathbf{a}_w]_{\times}$. Because $\mathbf{\Omega}^T = -\mathbf{\Omega}$, transposing the exponential series gives

$$\mathbf{Q}_w^T = \exp(-\alpha \mathbf{\Omega}).$$

The inverse property of the matrix exponential gives $\exp(-\alpha \mathbf{\Omega}) = \exp(\alpha \mathbf{\Omega})^{-1}$. Hence,

$$\mathbf{Q}_w^T \mathbf{Q}_w = \exp(-\alpha \mathbf{\Omega}) \exp(\alpha \mathbf{\Omega}) = \exp(\alpha \mathbf{\Omega})^{-1} \exp(\alpha \mathbf{\Omega}) = \mathbf{I}_3.$$

Thus, $\mathbf{Q}_w$ is orthogonal. Taking determinants of $\mathbf{Q}_w^T \mathbf{Q}_w = \mathbf{I}_3$ gives

$$\det(\mathbf{Q}_w)^2 = 1,$$

so an orthogonal matrix can have determinant only $+1$ or -1 . At $\alpha = 0$,

$$\mathbf{Q}_w(0) = \exp(\mathbf{0}_{3 \times 3}) = \mathbf{I}_3, \quad \det(\mathbf{Q}_w(0)) = 1.$$

As α changes continuously, every entry of $\mathbf{Q}_w(\alpha) = \exp(\alpha \mathbf{\Omega})$ also changes continuously. The determinant is calculated from those entries using sums and products, so $\det(\mathbf{Q}_w(\alpha))$ is continuous. To change from $+1$ to -1 , it would have to pass through intermediate values, including zero. This is impossible while $\mathbf{Q}_w$ remains orthogonal because an orthogonal matrix has determinant only $+1$ or -1 and is never singular. Therefore, $\det(\mathbf{Q}_w) = 1$ for every α , proving $\mathbf{Q}_w \in SO(3)$.

7.5.3 Deriving Rodrigues' rotation formula

Rodrigues' formula is a finite closed form of the matrix exponential derived above. Setting $\mathbf{\Omega} := [\mathbf{a}_w]_\times$ in the exponential-series definition gives

$$\mathbf{Q}_w = \exp(\alpha \mathbf{\Omega}) = \mathbf{I}_3 + \alpha \mathbf{\Omega} + \frac{\alpha^2 \mathbf{\Omega}^2}{2!} + \frac{\alpha^3 \mathbf{\Omega}^3}{3!} + \dots$$

This expression contains infinitely many powers of $\mathbf{\Omega}$. The powers can be reduced to a repeating pattern involving only $\mathbf{I}_3$, $\mathbf{\Omega}$, and $\mathbf{\Omega}^2$. To find that pattern, apply the vector triple-product identity from Section 1.1.5,

$$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a}^\top \mathbf{c}) - \mathbf{c}(\mathbf{a}^\top \mathbf{b}),$$

with $\mathbf{a} = \mathbf{b} = \mathbf{a}_w$ and $\mathbf{c} = \mathbf{y}$, where $\mathbf{y} \in \mathbb{R}^3$. This substitution gives

$$\begin{aligned} \mathbf{\Omega}^2 \mathbf{y} &= \mathbf{a}_w \times (\mathbf{a}_w \times \mathbf{y}) \\ &= \mathbf{a}_w (\mathbf{a}_w^\top \mathbf{y}) - \mathbf{y} (\mathbf{a}_w^\top \mathbf{a}_w) \\ &= (\mathbf{a}_w \mathbf{a}_w^\top - \mathbf{I}_3) \mathbf{y}. \end{aligned}$$

The second equality is the triple-product identity after the stated substitution: $\mathbf{a}_w^\top \mathbf{y}$ is the scalar projection factor in the first term, while $\mathbf{a}_w^\top \mathbf{a}_w$ is the squared norm of the axis. The last equality uses $\mathbf{a}_w^\top \mathbf{a}_w = 1$, rewrites $\mathbf{a}_w (\mathbf{a}_w^\top \mathbf{y})$ as $(\mathbf{a}_w \mathbf{a}_w^\top) \mathbf{y}$, and rewrites $\mathbf{y}$ as $\mathbf{I}_3 \mathbf{y}$. Because the equation holds for every $\mathbf{y}$,

$$\boxed{\mathbf{\Omega}^2 = \mathbf{a}_w \mathbf{a}_w^\top - \mathbf{I}_3}.$$

Furthermore, $\mathbf{\Omega} \mathbf{a}_w = \mathbf{a}_w \times \mathbf{a}_w = \mathbf{0}_3$, so

$$\begin{aligned} \mathbf{\Omega}^3 &= \mathbf{\Omega} (\mathbf{a}_w \mathbf{a}_w^\top - \mathbf{I}_3) \\ &= (\mathbf{\Omega} \mathbf{a}_w) \mathbf{a}_w^\top - \mathbf{\Omega} \\ &= -\mathbf{\Omega}. \end{aligned}$$

Consequently, the odd powers alternate between $\mathbf{\Omega}$ and $-\mathbf{\Omega}$, while the positive even powers alternate between $\mathbf{\Omega}^2$ and $-\mathbf{\Omega}^2$. Expand the matrix exponential and group those two power sequences:

$$\begin{aligned}
 \exp(\alpha \mathbf{\Omega}) &= \mathbf{I}_3 + \alpha \mathbf{\Omega} + \frac{\alpha^2}{2!} \mathbf{\Omega}^2 + \frac{\alpha^3}{3!} \mathbf{\Omega}^3 + \dots \\
 &= \mathbf{I}_3 + \left(\alpha - \frac{\alpha^3}{3!} + \frac{\alpha^5}{5!} - \dots \right) \mathbf{\Omega} \\
 &\quad + \left(\frac{\alpha^2}{2!} - \frac{\alpha^4}{4!} + \frac{\alpha^6}{6!} - \dots \right) \mathbf{\Omega}^2.
 \end{aligned}$$

The first scalar series is $\sin \alpha$, and the second is $1 - \cos \alpha$. Therefore,

$$\boxed{\mathbf{Q}_w(\mathbf{a}_w, \alpha) = \exp(\alpha [\mathbf{a}_w]_{\times}) = \mathbf{I}_3 + \sin \alpha [\mathbf{a}_w]_{\times} + (1 - \cos \alpha) [\mathbf{a}_w]_{\times}^2.}$$

This closed form is **Rodrigues' rotation formula**. Its geometric meaning becomes clear by decomposing any vector $\mathbf{y} \in \mathbb{R}^3$ into a component parallel to the axis and a component perpendicular to it:

$$\mathbf{y}_{\parallel} := (\mathbf{a}_w^{\top} \mathbf{y}) \mathbf{a}_w, \quad \mathbf{y}_{\perp} := \mathbf{y} - \mathbf{y}_{\parallel}.$$

Because $\mathbf{a}_w$ is a unit vector, $\mathbf{y} = \mathbf{y}_{\parallel} + \mathbf{y}_{\perp}$. Moreover, $\mathbf{a}_w^{\top} \mathbf{y}_{\perp} = \mathbf{a}_w^{\top} \mathbf{y} - (\mathbf{a}_w^{\top} \mathbf{y})(\mathbf{a}_w^{\top} \mathbf{a}_w) = 0$, so $\mathbf{y}_{\perp}$ is perpendicular to the axis. The parallel component is a scalar multiple of $\mathbf{a}_w$, so its cross product with $\mathbf{a}_w$ is zero. Therefore,

$$\begin{aligned}
 \mathbf{\Omega} \mathbf{y} &= \mathbf{a}_w \times (\mathbf{y}_{\parallel} + \mathbf{y}_{\perp}) \\
 &= \underbrace{\mathbf{a}_w \times \mathbf{y}_{\parallel}}_{\mathbf{0}_3} + \mathbf{a}_w \times \mathbf{y}_{\perp} \\
 &= \mathbf{a}_w \times \mathbf{y}_{\perp}.
 \end{aligned}$$

The identity $\mathbf{\Omega}^2 = \mathbf{a}_w \mathbf{a}_w^{\top} - \mathbf{I}_3$ derived above gives the second required result:

$$\begin{aligned}
 \mathbf{\Omega}^2 \mathbf{y} &= \mathbf{a}_w (\mathbf{a}_w^{\top} \mathbf{y}) - \mathbf{y} \\
 &= \mathbf{y}_{\parallel} - (\mathbf{y}_{\parallel} + \mathbf{y}_{\perp}) \\
 &= -\mathbf{y}_{\perp}.
 \end{aligned}$$

Apply Rodrigues' matrix formula to $\mathbf{y}$ and substitute these two results:

$$\begin{aligned}
 \mathbf{Q}_w \mathbf{y} &= \left[\mathbf{I}_3 + \sin \alpha \mathbf{\Omega} + (1 - \cos \alpha) \mathbf{\Omega}^2 \right] \mathbf{y} \\
 &= \mathbf{y} + \sin \alpha (\mathbf{a}_w \times \mathbf{y}_{\perp}) - (1 - \cos \alpha) \mathbf{y}_{\perp} \\
 &= \mathbf{y}_{\parallel} + \cos \alpha \mathbf{y}_{\perp} + \sin \alpha (\mathbf{a}_w \times \mathbf{y}_{\perp}).
 \end{aligned}$$

Thus,

$$\mathbf{Q}_w \mathbf{y} = \mathbf{y}_{\parallel} + \cos \alpha \mathbf{y}_{\perp} + \sin \alpha (\mathbf{a}_w \times \mathbf{y}_{\perp}).$$

Figure 7.2 illustrates this decomposition for $\mathbf{a}_w = \mathbf{e}_z$. The parallel component remains unchanged. In the plane perpendicular to $\mathbf{a}_w$, the vectors $\mathbf{y}_{\perp}$ and $\mathbf{a}_w \times \mathbf{y}_{\perp}$ are perpendicular and have equal length, so the coefficients $\cos \alpha$ and $\sin \alpha$ rotate $\mathbf{y}_{\perp}$ through angle α according to the right-hand rule.

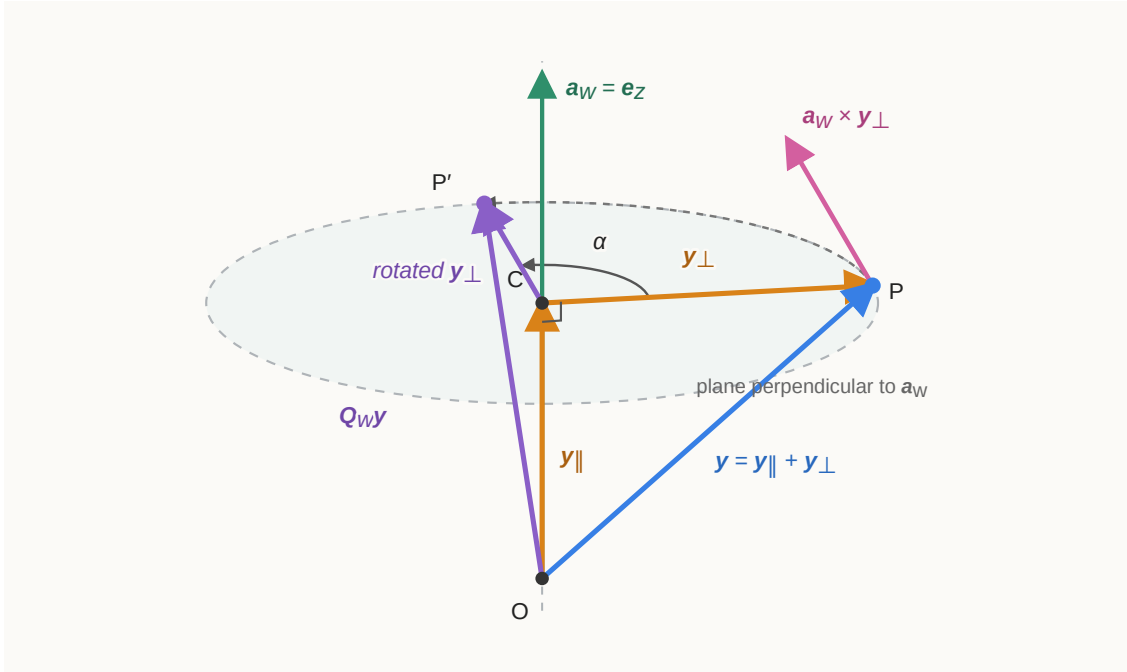


Figure 7.2: Geometric interpretation of Rodrigues' formula for the example axis $\mathbf{a}_w = \mathbf{e}_z$. The vector's parallel component remains fixed while its perpendicular component rotates through α .

7.5.4 Euler's rotation theorem: every spatial rotation has an axis

The matrix exponential and Rodrigues' formula solve the forward problem: a unit axis and an angle produce a rotation matrix. To use axis–angle coordinates for an arbitrary orientation, the reverse existence question must also be answered: does every matrix in $SO(3)$ represent one rotation about some axis? **Euler's rotation theorem** answers yes.

Let $\mathbf{Q} \in SO(3)$ be any rotation matrix, interpreted here as an active finite rotation with components expressed in one declared frame. Euler's rotation theorem states that there exist a dimensionless unit axis column $\mathbf{a} \in \mathbb{R}^3$ and a principal angle $\alpha \in [0, \pi]$ such that

$$\boxed{\|\mathbf{a}\|_2 = 1, \quad \mathbf{Q}\mathbf{a} = \mathbf{a}, \quad \mathbf{Q} = \exp([\mathbf{a}]_{\times}\alpha)}.$$

The equation $\mathbf{Q}\mathbf{a} = \mathbf{a}$ says that the rotation leaves the axis direction fixed. If $\mathbf{Q} = \mathbf{I}_3$, choose $\alpha = 0$ and any unit axis. If $\mathbf{Q} \neq \mathbf{I}_3$, choose $\alpha \in (0, \pi]$. For $0 < \alpha < \pi$, the oriented axis is fixed by the principal-angle convention. At $\alpha = \pi$, the columns $\mathbf{a}$ and $-\mathbf{a}$ describe the same unoriented axis line and the same rotation.

Proof

The proof first shows that every 3×3 proper rotation has a fixed nonzero direction. It then shows that the remaining perpendicular plane undergoes an ordinary two-dimensional rotation, which Rodrigues' formula represents as one axis-angle exponential.

Because $\mathbf{Q} \in SO(3)$,

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_3, \quad \det(\mathbf{Q}) = 1.$$

To prove that 1 is an eigenvalue, consider $\det(\mathbf{Q} - \mathbf{I}_3)$. Left-multiplying the matrix inside the determinant by $\mathbf{Q}^T$ does not change its determinant because $\det(\mathbf{Q}^T) = 1$:

$$\begin{aligned} \det(\mathbf{Q} - \mathbf{I}_3) &= \det[\mathbf{Q}^T(\mathbf{Q} - \mathbf{I}_3)] \\ &= \det(\mathbf{I}_3 - \mathbf{Q}^T) \\ &= \det[-(\mathbf{Q}^T - \mathbf{I}_3)] \\ &= -\det(\mathbf{Q}^T - \mathbf{I}_3) \\ &= -\det(\mathbf{Q} - \mathbf{I}_3). \end{aligned}$$

The last equality uses $\det(\mathbf{A}^T) = \det(\mathbf{A})$ for every square matrix $\mathbf{A}$. Let $d := \det(\mathbf{Q} - \mathbf{I}_3) \in \mathbb{R}$. The calculation shows that $d = -d$. Adding d to both sides gives $2d = 0$, and hence

$$\det(\mathbf{Q} - \mathbf{I}_3) = 0.$$

Set $\mathbf{M} := \mathbf{Q} - \mathbf{I}_3 \in \mathbb{R}^{3 \times 3}$. A square matrix has zero determinant exactly when it is not invertible; such a matrix is called **singular**. Therefore $\mathbf{M}$ is singular and $\text{rank}(\mathbf{M}) < 3$.

The **null space**, or **kernel**, of $\mathbf{M}$ is the set of input columns mapped to the zero column:

$$\ker(\mathbf{M}) := \{\mathbf{x} \in \mathbb{R}^3 \mid \mathbf{M}\mathbf{x} = \mathbf{0}_3\}.$$

The rank-nullity theorem gives

$$\dim(\ker(\mathbf{M})) = 3 - \text{rank}(\mathbf{M}) > 0.$$

Thus the null space contains more than only $\mathbf{0}_3$: there is at least one column $\mathbf{a}_0 \in \mathbb{R}^3$ satisfying

$$\mathbf{a}_0 \neq \mathbf{0}_3, \quad (\mathbf{Q} - \mathbf{I}_3)\mathbf{a}_0 = \mathbf{0}_3.$$

Normalise this nonzero column by defining $\mathbf{a} := \mathbf{a}_0 / \|\mathbf{a}_0\|_2$. Linearity preserves the null-space equation, so

$$\|\mathbf{a}\|_2 = 1, \quad (\mathbf{Q} - \mathbf{I}_3)\mathbf{a} = \mathbf{0}_3, \quad \mathbf{Q}\mathbf{a} = \mathbf{a}.$$

Thus at least one axis direction is fixed. Define its perpendicular plane by

$$\mathcal{V} := \mathbf{a}^\perp = \{\mathbf{x} \in \mathbb{R}^3 \mid \mathbf{a}^\top \mathbf{x} = 0\}.$$

The plane $\mathcal{V}$ is preserved by $\mathbf{Q}$. Indeed, $\mathbf{Q}^{-1} = \mathbf{Q}^\top$ and $\mathbf{Q}\mathbf{a} = \mathbf{a}$ imply $\mathbf{Q}^\top \mathbf{a} = \mathbf{a}$. Hence, for every $\mathbf{x} \in \mathcal{V}$,

$$\mathbf{a}^\top \mathbf{Q}\mathbf{x} = (\mathbf{Q}^\top \mathbf{a})^\top \mathbf{x} = \mathbf{a}^\top \mathbf{x} = 0,$$

so $\mathbf{Q}\mathbf{x} \in \mathcal{V}$.

The plane $\mathcal{V}$ is two-dimensional: its columns satisfy the one independent linear constraint $\mathbf{a}^\top \mathbf{x} = 0$ in $\mathbb{R}^3$. Choose any unit column $\mathbf{u} \in \mathcal{V}$ and define $\mathbf{v} := \mathbf{a} \times \mathbf{u}$. Because $\mathbf{a}$ and $\mathbf{u}$ are perpendicular unit columns, $\mathbf{v}$ is a unit column perpendicular to both of them. Thus $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, $\mathbf{u}^\top \mathbf{v} = 0$, and $\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$. The two nonzero orthogonal columns are linearly independent; because $\dim(\mathcal{V}) = 2$, they form an orthonormal basis of $\mathcal{V}$. Moreover, the vector triple-product identity gives

$$\mathbf{u} \times \mathbf{v} = \mathbf{u} \times (\mathbf{a} \times \mathbf{u}) = \mathbf{a}(\mathbf{u}^\top \mathbf{u}) - \mathbf{u}(\mathbf{u}^\top \mathbf{a}) = \mathbf{a},$$

so $(\mathbf{u}, \mathbf{v}, \mathbf{a})$ is a right-handed orthonormal basis of $\mathbb{R}^3$.

Because $\mathbf{Q}$ preserves $\mathcal{V}$, the column $\mathbf{Q}\mathbf{u}$ belongs to $\mathcal{V}$ and therefore has a unique expansion in the basis $(\mathbf{u}, \mathbf{v})$. Hence there are real scalars c and s such that

$$\mathbf{Q}\mathbf{u} = c\mathbf{u} + s\mathbf{v}.$$

The matrix $\mathbf{Q}$ preserves lengths, so $\|\mathbf{Q}\mathbf{u}\|_2 = \|\mathbf{u}\|_2 = 1$. Orthogonality of $\mathbf{u}$ and $\mathbf{v}$ then gives

$$1 = \|c\mathbf{u} + s\mathbf{v}\|_2^2 = c^2 + s^2.$$

The **restriction of $\mathbf{Q}$ to $\mathcal{V}$** is the map $\mathbf{Q}|_{\mathcal{V}} : \mathcal{V} \rightarrow \mathcal{V}$ obtained by allowing only inputs from $\mathcal{V}$. It is a two-dimensional orthogonal map because, for every $\mathbf{x}, \mathbf{y} \in \mathcal{V}$,

$$(\mathbf{Q}\mathbf{x})^\top(\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top \mathbf{Q}^\top \mathbf{Q} \mathbf{y} = \mathbf{x}^\top \mathbf{y}.$$

The fixed-axis equation $\mathbf{Q}\mathbf{a} = \mathbf{a}$ also says that 1 is an eigenvalue of $\mathbf{Q}$. By definition, a scalar λ is an eigenvalue of a matrix when there is a nonzero column $\mathbf{z}$ satisfying $\mathbf{Q}\mathbf{z} = \lambda\mathbf{z}$. Here $\mathbf{a} \neq \mathbf{0}_3$ and $\mathbf{Q}\mathbf{a} = 1\mathbf{a}$, so the eigenvalue is 1.

Let $\mathcal{B} := (\mathbf{u}, \mathbf{v}, \mathbf{a})$ denote the ordered basis of $\mathbb{R}^3$. If a column $\mathbf{y} \in \mathbb{R}^3$ is written as $\mathbf{y} = y_u\mathbf{u} + y_v\mathbf{v} + y_a\mathbf{a}$, define its coordinate column in this basis by

$$[\mathbf{y}]_{\mathcal{B}} := \begin{bmatrix} y_u \\ y_v \\ y_a \end{bmatrix}.$$

The **matrix representation of $\mathbf{Q}$ in the basis $\mathcal{B}$** is the unique matrix $[\mathbf{Q}]_{\mathcal{B}} \in \mathbb{R}^{3 \times 3}$ satisfying

$$\boxed{[\mathbf{Q}\mathbf{y}]_{\mathcal{B}} = [\mathbf{Q}]_{\mathcal{B}}[\mathbf{y}]_{\mathcal{B}} \quad \text{for every } \mathbf{y} \in \mathbb{R}^3}.$$

The basis vectors themselves have coordinate columns

$$[\mathbf{u}]_{\mathcal{B}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \mathbf{e}_1, \quad [\mathbf{v}]_{\mathcal{B}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \mathbf{e}_2, \quad [\mathbf{a}]_{\mathcal{B}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \mathbf{e}_3,$$

where $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3 \in \mathbb{R}^3$ are the standard coordinate columns. Applying the defining equation to $\mathbf{u}$ gives

$$[\mathbf{Q}\mathbf{u}]_{\mathcal{B}} = [\mathbf{Q}]_{\mathcal{B}}\mathbf{e}_1.$$

Multiplication by $\mathbf{e}_1$ selects the first matrix column, so the first column of $[\mathbf{Q}]_{\mathcal{B}}$ is $[\mathbf{Q}\mathbf{u}]_{\mathcal{B}}$. Similarly, multiplication by $\mathbf{e}_2$ and $\mathbf{e}_3$ shows that the second and third columns are $[\mathbf{Q}\mathbf{v}]_{\mathcal{B}}$ and $[\mathbf{Q}\mathbf{a}]_{\mathcal{B}}$, respectively. Thus

$$[\mathbf{Q}]_{\mathcal{B}} = \left[\begin{array}{c|c|c} | & | & | \\ [\mathbf{Q}\mathbf{u}]_{\mathcal{B}} & [\mathbf{Q}\mathbf{v}]_{\mathcal{B}} & [\mathbf{Q}\mathbf{a}]_{\mathcal{B}} \\ | & | & | \end{array} \right].$$

Let $\mathbf{Q}_{\mathcal{V}} \in \mathbb{R}^{2 \times 2}$ represent the restricted map $\mathbf{Q}|_{\mathcal{V}}$ in the ordered basis $(\mathbf{u}, \mathbf{v})$. Write

$$\mathbf{Q}\mathbf{u} = c\mathbf{u} + s\mathbf{v}, \quad \mathbf{Q}\mathbf{v} = r\mathbf{u} + t\mathbf{v},$$

where $r, t \in \mathbb{R}$ are not yet known. The first column of $\mathbf{Q}_{\mathcal{V}}$ is $\begin{bmatrix} c & s \end{bmatrix}^T$ because it contains the $(\mathbf{u}, \mathbf{v})$ -coordinates of $\mathbf{Q}\mathbf{u}$. Similarly, the second column is $\begin{bmatrix} r & t \end{bmatrix}^T$ because it contains the coordinates of $\mathbf{Q}\mathbf{v}$. Hence

$$\mathbf{Q}_{\mathcal{V}} = \begin{bmatrix} c & r \\ s & t \end{bmatrix}.$$

Because $\mathbf{Q}\mathbf{u}$ and $\mathbf{Q}\mathbf{v}$ remain in $\mathcal{V}$, their coordinate columns in $\mathcal{B}$ have zero $\mathbf{a}$ components. To determine the third column, use $\mathbf{Q}\mathbf{a} = \mathbf{a}$. The coordinate column $[\mathbf{a}]_{\mathcal{B}}$ stores the coefficients multiplying the ordered basis vectors $\mathbf{u}, \mathbf{v}$, and $\mathbf{a}$. Since

$$\mathbf{a} = 0\mathbf{u} + 0\mathbf{v} + 1\mathbf{a},$$

it follows that

$$[\mathbf{Q}\mathbf{a}]_{\mathcal{B}} = [\mathbf{a}]_{\mathcal{B}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

This coordinate column is relative to $\mathcal{B}$; it does not claim that $\mathbf{a}$ has entries $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T$ in the original coordinate frame. Therefore the full coordinate matrix has the block form

$$[\mathbf{Q}]_{\mathcal{B}} = \begin{bmatrix} \mathbf{Q}_{\mathcal{V}} & \mathbf{0}_2 \\ \mathbf{0}_2^T & 1 \end{bmatrix}.$$

The upper-left block describes the action within $\mathcal{V}$; the zero blocks state that $\mathbf{Q}$ does not mix the plane $\mathcal{V}$ with the axis direction $\mathbf{a}$; and the lower-right entry states that $\mathbf{a}$ is multiplied by the eigenvalue 1.

A change of basis does not change a matrix's determinant. Therefore

$$1 = \det(\mathbf{Q}) = \det([\mathbf{Q}]_{\mathcal{B}}) = \det(\mathbf{Q}_{\mathcal{V}}) 1,$$

so $\det(\mathbf{Q}_{\mathcal{V}}) = 1$. Thus the restricted map preserves the orientation of the ordered basis $(\mathbf{u}, \mathbf{v})$.

Because $\mathbf{Q}_{\mathcal{V}}$ is orthogonal, its second column $[r \ t]^T$ must be a unit column perpendicular to its first column $[c \ s]^T$. The two possibilities are

$$\begin{bmatrix} -s \\ c \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} s \\ -c \end{bmatrix}.$$

The determinant selects the first possibility because

$$\det \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = c^2 + s^2 = 1, \quad \det \begin{bmatrix} c & s \\ s & -c \end{bmatrix} = -(c^2 + s^2) = -1.$$

Therefore $[r \ t]^T = [-s \ c]^T$. Translating these coordinates back into the basis vectors gives

$$\mathbf{Q}\mathbf{v} = -s\mathbf{u} + c\mathbf{v}.$$

Since $c^2 + s^2 = 1$, there is a signed angle $\beta \in (-\pi, \pi]$ such that $c = \cos \beta$ and $s = \sin \beta$. Thus $\mathbf{Q}$ fixes $\mathbf{a}$ and rotates every vector in $\mathcal{V}$ through β :

$$\mathbf{Q}\mathbf{u} = \cos \beta \mathbf{u} + \sin \beta \mathbf{v}, \quad \mathbf{Q}\mathbf{v} = -\sin \beta \mathbf{u} + \cos \beta \mathbf{v}.$$

If $\beta \geq 0$, set $\mathbf{a}' = \mathbf{a}$ and $\alpha = \beta$. If $\beta < 0$, set $\mathbf{a}' = -\mathbf{a}$ and $\alpha = -\beta$. Reversing the axis reverses the positive right-hand-rule direction, so in both cases $\|\mathbf{a}'\|_2 = 1$ and $\alpha \in [0, \pi]$ describe the same rotation $\mathbf{Q}$.

The connection to Rodrigues' geometric formula is direct. Define $\mathbf{E} := \exp([\mathbf{a}']_{\times} \alpha)$. Because $\mathbf{u}$ and $\mathbf{v}$ are perpendicular to the axis, Rodrigues' formula contains only their perpendicular-plane terms. Using $\mathbf{v} = \mathbf{a} \times \mathbf{u}$, $\mathbf{a} \times \mathbf{v} = -\mathbf{u}$, and the axis-and-angle choice made above gives

$$\begin{aligned} \mathbf{E}\mathbf{u} &= \cos \alpha \mathbf{u} + \sin \alpha (\mathbf{a}' \times \mathbf{u}) \\ &= \cos \beta \mathbf{u} + \sin \beta \mathbf{v} \\ &= \mathbf{Q}\mathbf{u}, \\ \mathbf{E}\mathbf{v} &= \cos \alpha \mathbf{v} + \sin \alpha (\mathbf{a}' \times \mathbf{v}) \\ &= -\sin \beta \mathbf{u} + \cos \beta \mathbf{v} \\ &= \mathbf{Q}\mathbf{v}. \end{aligned}$$

Rodrigues' formula also fixes every vector parallel to $\mathbf{a}'$. Since $\mathbf{a}' = \pm \mathbf{a}$, it follows that $\mathbf{E}\mathbf{a} = \mathbf{a} = \mathbf{Q}\mathbf{a}$. Thus $\mathbf{E}$ and $\mathbf{Q}$ agree on the basis $(\mathbf{u}, \mathbf{v}, \mathbf{a})$ and therefore on every column in $\mathbb{R}^3$. Consequently,

$$\mathbf{Q} = \exp([\mathbf{a}']_{\times} \alpha).$$

Renaming the constructed axis $\mathbf{a}'$ as $\mathbf{a}$ gives the theorem's stated notation and completes the proof.

7.5.5 Check: rotation about the world z_w -axis

Choose

$$\mathbf{a}_w = \mathbf{e}_z = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad [\mathbf{a}_w]_{\times} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Squaring the cross-product matrix gives

$$[\mathbf{a}_w]_{\times}^2 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Substitution into Rodrigues' formula produces

$$\begin{aligned} \mathbf{Q}_w(\mathbf{e}_z, \alpha) &= \mathbf{I}_3 + \sin \alpha \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + (1 - \cos \alpha) \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

This is exactly the coordinate-axis rotation $\mathbf{R}_z(\alpha)$ derived in Section 7.3.1.

7.5.6 Applying a constant angular velocity over a time interval

Suppose the world-expressed angular velocity $\boldsymbol{\omega}_w \in \mathbb{R}^3$ remains constant during a time interval $\Delta t > 0$. If $\boldsymbol{\omega}_w \neq \mathbf{0}_3$, define

$$\mathbf{a}_w := \frac{\boldsymbol{\omega}_w}{\|\boldsymbol{\omega}_w\|_2}, \quad \alpha := \|\boldsymbol{\omega}_w\|_2 \Delta t.$$

The rotation vector over the interval is then

$$\boldsymbol{\varphi}_w = \alpha \mathbf{a}_w = \Delta t \boldsymbol{\omega}_w.$$

Because the angular velocity is expressed using the fixed world axes, the finite update premultiplies the current orientation:

$$\boxed{\mathbf{R}_{wb}(t + \Delta t) = \exp(\Delta t [\boldsymbol{\omega}_w]_{\times}) \mathbf{R}_{wb}(t)}.$$

If the constant angular velocity is instead expressed using the body axes, the update post-multiplies:

$$\boxed{\mathbf{R}_{wb}(t + \Delta t) = \mathbf{R}_{wb}(t) \exp(\Delta t [\boldsymbol{\omega}_b]_{\times})}.$$

When the angular velocity is zero, define the finite increment as $\mathbf{I}_3$; the orientation then remains unchanged. These finite updates are exact only when the corresponding world- or body-expressed angular velocity remains constant throughout the interval.

Axis-angle coordinates are not globally unique. The pairs $(\mathbf{a}_w, \alpha)$ and $(-\mathbf{a}_w, -\alpha)$ describe the same turn, adding $2\pi k$ to the angle for any $k \in \mathbb{Z}$ produces the same rotation, and when $\alpha = 0$ the axis direction is arbitrary. These facts matter when recovering axis-angle coordinates from a rotation matrix or comparing two numerical orientation representations.

Quick test E: axis-angle coordinates and the exponential map

1. What information is stored in the axis-angle pair $(\mathbf{a}_w, \alpha)$, and how does the rotation vector $\boldsymbol{\varphi}_w$ combine it?
2. State the matrix-exponential definition and explain why $\mathbf{p}(s) = \exp(s[\mathbf{a}_w]_{\times})\mathbf{p}_0$ solves $d\mathbf{p}/ds = [\mathbf{a}_w]_{\times}\mathbf{p}$.
3. Use the vector triple-product identity to derive $[\mathbf{a}_w]_{\times}^2 = \mathbf{a}_w \mathbf{a}_w^T - \mathbf{I}_3$ for a unit axis.

4. Starting from the matrix-exponential series and $[\mathbf{a}_w]_{\times}^3 = -[\mathbf{a}_w]_{\times}$, derive Rodrigues' rotation formula.
5. Why does Rodrigues' formula leave the vector component parallel to $\mathbf{a}_w$ unchanged?
6. State Euler's rotation theorem. In its proof, why does $\det(\mathbf{Q} - \mathbf{I}_3) = 0$ imply a fixed axis direction?
7. A constant angular velocity $\boldsymbol{\omega}_w = [0 \ 0 \ 2]^T \text{ rad s}^{-1}$ acts for $\Delta t = 0.25 \text{ s}$. Find the axis, angle, rotation vector, and finite orientation increment.
8. Why does a world-expressed constant angular velocity premultiply $\mathbf{R}_{wb}$, while a body-expressed constant angular velocity postmultiplies it?
9. Give two reasons why an axis–angle representation of a rotation is not unique.

Answers and hints

1. The unit vector $\mathbf{a}_w$ gives the positive axis direction in world coordinates, and α gives the signed right-hand-rule angle. Their product $\boldsymbol{\varphi}_w = \alpha \mathbf{a}_w$ stores the same information in one three-component column.
2. $\exp(\mathbf{A}) = \mathbf{I} + \mathbf{A} + \mathbf{A}^2/2! + \dots$. Differentiating $\exp(s\mathbf{A})$ term by term gives $\mathbf{A} \exp(s\mathbf{A})$; setting $\mathbf{A} = [\mathbf{a}_w]_{\times}$ produces the required differential equation and the identity exponential satisfies the initial condition.
3. For every $\mathbf{y}$, $\mathbf{a}_w \times (\mathbf{a}_w \times \mathbf{y}) = \mathbf{a}_w(\mathbf{a}_w^T \mathbf{y}) - (\mathbf{a}_w^T \mathbf{a}_w)\mathbf{y} = (\mathbf{a}_w \mathbf{a}_w^T - \mathbf{I}_3)\mathbf{y}$. Equality for every $\mathbf{y}$ proves the matrix identity.
4. Replace successive odd powers by alternating $[\mathbf{a}_w]_{\times}$ and $-[\mathbf{a}_w]_{\times}$, and successive positive even powers by alternating $[\mathbf{a}_w]_{\times}^2$ and $-[\mathbf{a}_w]_{\times}^2$. Their scalar coefficients sum to $\sin \alpha$ and $1 - \cos \alpha$, respectively.
5. The cross product of the axis with a parallel vector is zero, so both $[\mathbf{a}_w]_{\times} \mathbf{y}_{\parallel}$ and $[\mathbf{a}_w]_{\times}^2 \mathbf{y}_{\parallel}$ vanish.
6. Euler's rotation theorem states that, for every $\mathbf{Q} \in SO(3)$, there are a unit axis $\mathbf{a} \in \mathbb{R}^3$ and a principal angle $\alpha \in [0, \pi]$ such that $\mathbf{Q}\mathbf{a} = \mathbf{a}$ and $\mathbf{Q} = \exp([\mathbf{a}]_{\times} \alpha)$. The zero determinant makes $\mathbf{Q} - \mathbf{I}_3$ singular, so its null space contains a nonzero column $\mathbf{a}$; normalising it gives the fixed axis direction $\mathbf{Q}\mathbf{a} = \mathbf{a}$.
7. $\mathbf{a}_w = [0 \ 0 \ 1]^T$, $\alpha = 2(0.25) = 0.5 \text{ rad}$, and $\boldsymbol{\varphi}_w = [0 \ 0 \ 0.5]^T \text{ rad}$. The increment is $\mathbf{Q}_w = \mathbf{R}_z(0.5) = \begin{bmatrix} \cos 0.5 & -\sin 0.5 & 0 \\ \sin 0.5 & \cos 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$.
8. A finite increment expressed using world axes acts on the world-coordinate columns and therefore premultiplies. An increment expressed using the old body axes composes after the body-to-world mapping and therefore postmultiplies.
9. Examples are $(\mathbf{a}_w, \alpha) \sim (-\mathbf{a}_w, -\alpha)$, angles differing by $2\pi k$ give the same rotation, and the axis is arbitrary for zero rotation.

7.6 Spatial pose and $SE(3)$

7.6.1 From a point transformation to a homogeneous matrix

Orientation alone does not locate a rigid body in three-dimensional space. Suppose frame $\{b\}$ is fixed to a rigid body and frame $\{w\}$ is fixed in the world. A **spatial pose** is the physical placement of frame $\{b\}$ relative to frame $\{w\}$, consisting of:

- the orientation $\mathbf{R}_{wb} \in SO(3)$ of frame $\{b\}$ relative to frame $\{w\}$; and
- the position ${}^w\mathbf{p}_b \in \mathbb{R}^3$ of the body origin O_b , expressed in world coordinates and measured in metres.

Consider a material point P fixed in the body. Its body-frame coordinate ${}^b\mathbf{p}_P \in \mathbb{R}^3$ is constant and measured in metres. The product $\mathbf{R}_{wb} {}^b\mathbf{p}_P$ expresses the displacement from O_b to P along the world axes. It changes the coordinate description of that displacement; it does not physically move P . This displacement is not yet the world position of P . Adding the world position of O_b gives

$${}^w\mathbf{p}_P = \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b.$$

Every term in this equation belongs to $\mathbb{R}^3$ and has units of metres. The equation first re-expresses the body-relative displacement in world coordinates and then adds the translation from the world origin O_w to the body origin O_b .

The rotation and translation can be packaged into one matrix by appending a final coordinate equal to one. Define the **homogeneous point coordinates**

$${}^b\bar{\mathbf{p}}_P := \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix} \in \mathbb{R}^4, \quad {}^w\bar{\mathbf{p}}_P := \begin{bmatrix} {}^w\mathbf{p}_P \\ 1 \end{bmatrix} \in \mathbb{R}^4.$$

The overline distinguishes the four-entry homogeneous coordinate from the ordinary three-entry position coordinate. Its first three entries have units of metres, while the appended 1 is dimensionless and exists only to make translation part of a matrix multiplication.

Define the **homogeneous transformation**

$$\mathbf{T}_{wb} := \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}.$$

Block multiplication now reproduces the physical point-transformation equation:

$$\begin{aligned}
\mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P &= \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b \\ 1 \end{bmatrix} \\
&= {}^w\bar{\mathbf{p}}_P.
\end{aligned}$$

For a free vector $\mathbf{v}$, translation is irrelevant because a free vector has no fixed origin. Append a 0 instead of a 1 to define its homogeneous coordinate column:

$${}^b\bar{\mathbf{v}} := \begin{bmatrix} {}^b\mathbf{v} \\ 0 \end{bmatrix} \in \mathbb{R}^4.$$

Applying $\mathbf{T}_{wb}$ then gives

$$\mathbf{T}_{wb} \begin{bmatrix} {}^b\mathbf{v} \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{v} \\ 0 \end{bmatrix}.$$

The appended 0 prevents the translation column from contributing, so this equation reproduces the rotation-only coordinate mapping ${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}$ from Section 7.2.1.

As distinguished for planar transformations in Section 6.2.3, the same homogeneous-matrix form has three possible interpretations:

- As a **relative-pose description**, $\mathbf{T}_{wb}$ stores the orientation and position of frame $\{b\}$ relative to frame $\{w\}$.
- As a **passive coordinate change**, ${}^w\bar{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P$ maps the coordinates of the unchanged geometric point P from frame $\{b\}$ to frame $\{w\}$.
- As an **active rigid-motion operator**, an explicitly declared $\mathbf{T}_\Delta \in SE(3)$ moves a geometric point from P to P' while both coordinate columns remain expressed in the same frame, for example ${}^w\bar{\mathbf{p}}_{P'} = \mathbf{T}_\Delta {}^w\bar{\mathbf{p}}_P$.

The project notation therefore uses a frame-labelled symbol such as $\mathbf{T}_{wb}$ for the first two interpretations and a separately declared symbol such as $\mathbf{T}_\Delta$ for an active motion. A transformation matrix is not itself the position of one point; applying the relative-pose mapping $\mathbf{T}_{wb}$ to every body-fixed point reconstructs the body's complete spatial placement.

The set containing every matrix of this form is the **special Euclidean group in three dimensions**:

$$SE(3) := \left\{ \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \mid \mathbf{R} \in SO(3), \mathbf{p} \in \mathbb{R}^3 \right\}.$$

Here, *Euclidean* indicates that the transformation preserves Euclidean distances and angles, while *special* indicates that $\det(\mathbf{R}) = 1$, so the transformation preserves orientation and excludes reflections. In a physical pose, the entries of $\mathbf{p}$ are measured in metres even though the abstract set definition writes $\mathbf{p} \in \mathbb{R}^3$.

For a single free rigid body, its spatial pose determines its complete configuration: the body-frame coordinates of all material points are fixed, so the pose's one rotation and one translation locate every point. After the world and body frames have been fixed, each spatial pose produces exactly one matrix $\mathbf{T}_{wb} \in SE(3)$. Conversely, each matrix $\mathbf{T}_{wb} \in SE(3)$ specifies exactly one spatial pose because its rotation block gives $\mathbf{R}_{wb}$ and its translation column gives ${}^w\mathbf{p}_b$. The configuration space of a free spatial rigid body can therefore be identified with $SE(3)$:

$$\boxed{\mathcal{C} \cong SE(3)}.$$

7.6.2 Composing and inverting spatial transformations

To **compose** transformations means to perform two coordinate mappings in sequence and replace them with one direct mapping. Introduce frames $\{a\}$, $\{b\}$, and $\{c\}$. Let $\mathbf{T}_{ab} \in SE(3)$ map point coordinates from frame $\{b\}$ to frame $\{a\}$, and let $\mathbf{T}_{bc} \in SE(3)$ map them from frame $\{c\}$ to frame $\{b\}$. Applying both mappings to a geometric point P gives

$$\begin{aligned} {}^a\bar{\mathbf{p}}_P &= \mathbf{T}_{ab}({}^b\mathbf{T}_{bc} {}^c\bar{\mathbf{p}}_P) \\ &= (\mathbf{T}_{ab}\mathbf{T}_{bc}) {}^c\bar{\mathbf{p}}_P. \end{aligned}$$

The rightmost transformation $\mathbf{T}_{bc}$ acts first. The direct mapping from frame $\{c\}$ to frame $\{a\}$ is therefore

$$\boxed{\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}}.$$

The repeated inner label b identifies the intermediate frame and provides a useful frame-cancellation check. To see how the orientation and position parts combine, write

$$\mathbf{T}_{ab} = \begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \mathbf{T}_{bc} = \begin{bmatrix} \mathbf{R}_{bc} & {}^b\mathbf{p}_c \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

Here, ${}^a\mathbf{p}_b \in \mathbb{R}^3$ is the position of origin O_b relative to origin O_a , expressed in frame $\{a\}$, and ${}^b\mathbf{p}_c \in \mathbb{R}^3$ is the position of origin O_c relative to origin O_b , expressed in frame $\{b\}$; both are measured in metres. Block multiplication gives

$$\mathbf{T}_{ab}\mathbf{T}_{bc} = \begin{bmatrix} \mathbf{R}_{ab}\mathbf{R}_{bc} & \mathbf{R}_{ab}{}^b\mathbf{p}_c + {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

Consequently,

$$\boxed{\mathbf{R}_{ac} = \mathbf{R}_{ab}\mathbf{R}_{bc}, \quad {}^a\mathbf{p}_c = \mathbf{R}_{ab}{}^b\mathbf{p}_c + {}^a\mathbf{p}_b}.$$

The position columns cannot be added until they are expressed along the same axes. Multiplication by $\mathbf{R}_{ab}$ re-expresses ${}^b\mathbf{p}_c$ in frame $\{a\}$ before it is added to ${}^a\mathbf{p}_b$. Because $\mathbf{R}_{ab}\mathbf{R}_{bc} \in SO(3)$ and the new translation belongs to $\mathbb{R}^3$, the product remains in $SE(3)$. This proves that $SE(3)$ is closed under matrix multiplication.

The **inverse transformation** reverses a coordinate mapping. Starting from

$${}^a\mathbf{p}_P = \mathbf{R}_{ab}{}^b\mathbf{p}_P + {}^a\mathbf{p}_b,$$

subtract ${}^a\mathbf{p}_b$ and premultiply by $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top$:

$$\begin{aligned} {}^b\mathbf{p}_P &= \mathbf{R}_{ab}^\top ({}^a\mathbf{p}_P - {}^a\mathbf{p}_b) \\ &= \mathbf{R}_{ab}^\top {}^a\mathbf{p}_P - \mathbf{R}_{ab}^\top {}^a\mathbf{p}_b. \end{aligned}$$

Comparing this result with the standard point-transformation form shows that the reverse rotation and translation are

$$\mathbf{R}_{ba} = \mathbf{R}_{ab}^\top, \quad {}^b\mathbf{p}_a = -\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b.$$

Therefore,

$$\boxed{\mathbf{T}_{ab}^{-1} = \mathbf{T}_{ba} = \begin{bmatrix} \mathbf{R}_{ab}^\top & -\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3)}.$$

The inverse translation is not generally $-{}^a\mathbf{p}_b$, because that column is expressed in frame $\{a\}$. The factor $\mathbf{R}_{ab}^\top$ re-expresses the negated displacement in frame $\{b\}$, producing the position ${}^b\mathbf{p}_a$ of origin O_a relative to origin O_b .

The identity transformation is

$$\boxed{\mathbf{T}_{aa} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_3^\top & 1 \end{bmatrix} = \mathbf{I}_4}.$$

It leaves every homogeneous point coordinate unchanged. Matrix multiplication is associative, the product of two elements remains in $SE(3)$, $\mathbf{I}_4$ is the identity element, and the inverse derived above belongs to $SE(3)$. These four properties establish that $SE(3)$ is a group under matrix multiplication. The operation is generally not commutative: changing the order of spatial rotations and translations can change the final pose.

7.6.3 Worked example: composition and inverse round trip

Consider three coordinate frames $\{a\}$, $\{b\}$, and $\{c\}$. This example uses rotations about two different axes so that the calculation is genuinely three-dimensional. Let frame $\{b\}$ have the following orientation and origin position relative to frame $\{a\}$:

$$\mathbf{R}_{ab} = \mathbf{R}_z\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad {}^a\mathbf{p}_b = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \text{ m.}$$

The matrix $\mathbf{R}_{ab} \in SO(3)$ maps coordinate columns from frame $\{b\}$ to frame $\{a\}$, while ${}^a\mathbf{p}_b \in \mathbb{R}^3$ is the position of origin O_b relative to origin O_a , expressed along the axes of frame $\{a\}$. Let frame $\{c\}$ have the following orientation and origin position relative to frame $\{b\}$:

$$\mathbf{R}_{bc} = \mathbf{R}_x\left(\frac{\pi}{2}\right) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}, \quad {}^b\mathbf{p}_c = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \text{ m.}$$

The direct orientation from frame $\{c\}$ to frame $\{a\}$ is found by composition:

$$\begin{aligned} \mathbf{R}_{ac} &= \mathbf{R}_{ab}\mathbf{R}_{bc} \\ &= \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}. \end{aligned}$$

To combine the origin positions, ${}^b\mathbf{p}_c$ must first be re-expressed in frame $\{a\}$. Therefore,

$$\begin{aligned}
{}^a\mathbf{p}_c &= \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b \\
&= \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \\
&= \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \\
&= \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix} \text{ m.}
\end{aligned}$$

The composed transformation is consequently

$$\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc} = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The first three entries of the last column are measured in metres; the rotation entries and the homogeneous last row are dimensionless.

To check both the numerical result and the multiplication order, choose a geometric point P whose coordinates in frame $\{c\}$ are

$${}^c\mathbf{p}_P = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m.}$$

First map its coordinates from frame $\{c\}$ to frame $\{b\}$:

$$\begin{aligned}
{}^b\mathbf{p}_P &= \mathbf{R}_{bc} {}^c\mathbf{p}_P + {}^b\mathbf{p}_c \\
&= \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \\
&= \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \text{ m.}
\end{aligned}$$

Then map the result from frame $\{b\}$ to frame $\{a\}$:

$$\begin{aligned} {}^a\mathbf{p}_P &= \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b \\ &= \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} \text{ m.} \end{aligned}$$

The direct transformation gives the same coordinates:

$$\begin{aligned} {}^a\mathbf{p}_P &= \mathbf{R}_{ac} {}^c\mathbf{p}_P + {}^a\mathbf{p}_c \\ &= \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} \text{ m.} \end{aligned}$$

This agreement confirms that $\mathbf{T}_{bc}$ must act first and $\mathbf{T}_{ab}$ second in the product $\mathbf{T}_{ab}\mathbf{T}_{bc}$. These equations re-express the coordinates of the same point P ; they do not physically move the point.

The inverse should recover the original frame- $\{c\}$ coordinates. Using the inverse formula from Section 7.6.2 gives

$$\begin{aligned} \mathbf{R}_{ca} &= \mathbf{R}_{ac}^\top = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \\ {}^c\mathbf{p}_a &= -\mathbf{R}_{ac}^\top {}^a\mathbf{p}_c \\ &= -\begin{bmatrix} 3 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ -1 \\ -1 \end{bmatrix} \text{ m.} \end{aligned}$$

Therefore,

$$\mathbf{T}_{ca} = \mathbf{T}_{ac}^{-1} = \begin{bmatrix} 0 & 1 & 0 & -3 \\ 0 & 0 & 1 & -1 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Applying this reverse mapping to the calculated frame- $\{a\}$ coordinates gives

$$\begin{aligned} {}^c\mathbf{p}_P &= \mathbf{R}_{ca} {}^a\mathbf{p}_P + {}^c\mathbf{p}_a \\ &= \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} + \begin{bmatrix} -3 \\ -1 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m.} \end{aligned}$$

The round trip returns the original coordinates exactly. Point mapping through both intermediate frames and inverse round trips are therefore useful checks for a spatial-transformation calculation.

Quick test F: spatial pose and $SE(3)$

1. In the transformation

$$\mathbf{T}_{wb} = \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

what do $\mathbf{R}_{wb}$ and ${}^w\mathbf{p}_b$ describe, and which coordinate mapping does $\mathbf{T}_{wb}$ perform?

2. Why is a point represented by $[\mathbf{p}^\top \ 1]^\top$ in homogeneous coordinates, while a free vector is represented by $[\mathbf{v}^\top \ 0]^\top$?
3. Let $\mathbf{T}_{ab}$ map coordinates from frame $\{b\}$ to frame $\{a\}$ and let $\mathbf{T}_{bc}$ map coordinates from frame $\{c\}$ to frame $\{b\}$. Derive the rotation and translation blocks of $\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}$.
4. Why must ${}^b\mathbf{p}_c$ be multiplied by $\mathbf{R}_{ab}$ before it can be added to ${}^a\mathbf{p}_b$?
5. Derive $\mathbf{T}_{ab}^{-1}$. Why is its translation block generally not equal to $-{}^a\mathbf{p}_b$?

6. Suppose

$$\mathbf{R}_{wb} = \mathbf{R}_z\left(\frac{\pi}{2}\right), \quad {}^w\mathbf{p}_b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \text{ m}, \quad {}^b\mathbf{p}_P = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix} \text{ m}.$$

Calculate ${}^w\mathbf{p}_P$.

7. Explain the difference between interpreting $\mathbf{T}_{wb}$ as a passive coordinate mapping and interpreting a separately declared $\mathbf{T}_\Delta$ as an active rigid motion.

Answers and hints

1. The rotation $\mathbf{R}_{wb} \in SO(3)$ describes the orientation of frame $\{b\}$ relative to frame $\{w\}$ and maps coordinate columns from body axes to world axes. The position ${}^w\mathbf{p}_b \in \mathbb{R}^3$ locates body origin O_b relative to world origin O_w , expressed along world axes and measured in metres. The complete transformation maps body-frame point coordinates to world-frame point coordinates:

$${}^w\bar{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P.$$

2. Multiplying a homogeneous point by $\mathbf{T}_{wb}$ gives

$$\mathbf{T}_{wb} \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b \\ 1 \end{bmatrix},$$

so the appended 1 allows the translation column to contribute. A free vector has no fixed origin, so translation must not affect it:

$$\mathbf{T}_{wb} \begin{bmatrix} {}^b\mathbf{v} \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{v} \\ 0 \end{bmatrix}.$$

3. Block multiplication gives

$$\boxed{\mathbf{R}_{ac} = \mathbf{R}_{ab}\mathbf{R}_{bc}, \quad {}^a\mathbf{p}_c = \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b.}$$

4. The columns ${}^b\mathbf{p}_c$ and ${}^a\mathbf{p}_b$ initially use different coordinate axes. The product $\mathbf{R}_{ab} {}^b\mathbf{p}_c$ re-expresses the first displacement along frame- $\{a\}$ axes, after which the two frame- $\{a\}$ coordinate columns can be added.

5. Solving ${}^a\mathbf{p}_P = \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b$ for ${}^b\mathbf{p}_P$ gives

$$\mathbf{T}_{ab}^{-1} = \mathbf{T}_{ba} = \begin{bmatrix} \mathbf{R}_{ab}^\top & -\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

The reverse displacement must be expressed along frame- $\{b\}$ axes. The factor $\mathbf{R}_{ab}^\top$ performs this coordinate change, so merely negating the frame- $\{a\}$ column is insufficient.

6. A positive quarter-turn about z_w maps $[2 \ 0 \ 1]^\top$ to $[0 \ 2 \ 1]^\top$. Therefore,

$$\begin{aligned} {}^w\mathbf{p}_P &= \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b \\ &= \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 4 \\ 4 \end{bmatrix} \text{ m.} \end{aligned}$$

7. In the passive interpretation, the geometric point remains unchanged and $\mathbf{T}_{wb}$ changes only its coordinate description from frame $\{b\}$ to frame $\{w\}$. In the active interpretation, $\mathbf{T}_\Delta$ physically moves a point or rigid body while the input and output coordinates are expressed in the same declared frame. The same matrix multiplication can implement either interpretation, so the frames and intended meaning must be stated.

7.7 Embedding planar poses in $SE(3)$

A mobile robot may be modelled as moving only in a plane even though its software exchanges three-dimensional poses with other robots, sensors, and maps. A planar pose must therefore be represented inside the spatial-pose space without changing its geometric meaning.

Choose the world plane $z_w = 0$ as the plane containing the body origin. Restrict the body to translation along x_w and y_w and rotation about z_w . The body cannot translate along z_w or rotate about the horizontal x_w and y_w axes under this model. Its planar pose coordinates are

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix},$$

Here:

- x is the x_w -coordinate of body origin O_b relative to world origin O_w , measured in metres;
- y is the y_w -coordinate of O_b relative to O_w , measured in metres;
- θ is the orientation of body frame $\{b\}$ relative to world frame $\{w\}$, measured as a positive right-hand rotation about the shared z -axis in radians.

The corresponding world-frame position of the body origin is

$${}^w\mathbf{p}_b = \begin{bmatrix} x \\ y \\ 0 \end{bmatrix} \text{ m.}$$

To distinguish matrices of different sizes, let the superscripts (2) and (3) label planar and spatial representations; they are dimension labels, not matrix powers. The planar transformation from Section 6.2.3 is

$$\mathbf{T}_{wb}^{(2)} = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix} \in SE(2).$$

The same physical placement is represented spatially by

$$\mathbf{T}_{wb}^{(3)} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & x \\ \sin \theta & \cos \theta & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \in SE(3).$$

The upper-left 3×3 block is the rotation $\mathbf{R}_z(\theta) \in SO(3)$ derived in Section 7.3.1. Its third column $[0 \ 0 \ 1]^\top$ states that z_b remains aligned with z_w . The translation column $[x \ y \ 0]^\top$ states that the body origin remains in the chosen world plane.

Write a generic planar transformation using dimensionless rotation entries r_{ij} , where $i, j \in \{1, 2\}$ are row and column indices, and translation entries p_1 and p_2 , measured in metres. The conversion can then be defined by the mapping

$$\iota : SE(2) \longrightarrow SE(3),$$

$$\begin{bmatrix} r_{11} & r_{12} & p_1 \\ r_{21} & r_{22} & p_2 \\ 0 & 0 & 1 \end{bmatrix} \mapsto \begin{bmatrix} r_{11} & r_{12} & 0 & p_1 \\ r_{21} & r_{22} & 0 & p_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The Greek letter ι names this **group embedding**. A group embedding is a one-to-one mapping that places one group inside another while preserving the group operation. This mapping is one-to-one because all four planar rotation entries and both planar translation entries can be read back uniquely from the spatial matrix.

To verify that point transformations are preserved, let a point P in the body plane have body-frame coordinates

$${}^b\mathbf{p}_P^{(2)} = \begin{bmatrix} p_x \\ p_y \end{bmatrix} \in \mathbb{R}^2, \quad {}^b\mathbf{p}_P^{(3)} = \begin{bmatrix} p_x \\ p_y \\ 0 \end{bmatrix} \in \mathbb{R}^3,$$

where p_x and p_y are measured in metres. The spatial transformation gives

$$\begin{aligned} {}^w\mathbf{p}_P^{(3)} &= \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ 0 \end{bmatrix} + \begin{bmatrix} x \\ y \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} \cos \theta p_x - \sin \theta p_y + x \\ \sin \theta p_x + \cos \theta p_y + y \\ 0 \end{bmatrix}. \end{aligned}$$

The first two entries are exactly the result of the planar transformation $\mathbf{T}_{wb}^{(2)}$, while the third entry remains zero. The spatial matrix therefore reproduces the planar point mapping rather than approximating it.

The planar motion model does not flatten the robot into a two-dimensional object. If a body-fixed point has height p_z relative to the body origin, then

$${}^b\mathbf{p}_P^{(3)} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \implies {}^w\mathbf{p}_P^{(3)} = \begin{bmatrix} \cos \theta p_x - \sin \theta p_y + x \\ \sin \theta p_x + \cos \theta p_y + y \\ p_z \end{bmatrix}.$$

Here, p_z is measured in metres. The body can therefore have three-dimensional geometry; the restriction is that its origin has no vertical motion and its orientation has no rotation away from the shared z -axis.

The embedding also preserves composition. Let

$$\mathbf{T}_1^{(2)} = \begin{bmatrix} \mathbf{R}_1 & \mathbf{p}_1 \\ \mathbf{0}_2^\top & 1 \end{bmatrix}, \quad \mathbf{T}_2^{(2)} = \begin{bmatrix} \mathbf{R}_2 & \mathbf{p}_2 \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in SE(2),$$

where $\mathbf{R}_1, \mathbf{R}_2 \in SO(2)$ and $\mathbf{p}_1, \mathbf{p}_2 \in \mathbb{R}^2$, with the position entries measured in metres. Their planar product is

$$\mathbf{T}_1^{(2)} \mathbf{T}_2^{(2)} = \begin{bmatrix} \mathbf{R}_1 \mathbf{R}_2 & \mathbf{R}_1 \mathbf{p}_2 + \mathbf{p}_1 \\ \mathbf{0}_2^\top & 1 \end{bmatrix}.$$

In each embedded matrix, the planar rotation and translation occupy the same entries, while the added z -axis row and column remain unchanged. Multiplying the embedded matrices therefore produces the embedded planar product:

$$\boxed{\iota(\mathbf{T}_1^{(2)} \mathbf{T}_2^{(2)}) = \iota(\mathbf{T}_1^{(2)}) \iota(\mathbf{T}_2^{(2)})}.$$

The planar identity maps to the spatial identity:

$$\iota(\mathbf{I}_3) = \mathbf{I}_4.$$

Applying the composition identity to $\mathbf{T}^{(2)}(\mathbf{T}^{(2)})^{-1} = \mathbf{I}_3$ therefore gives

$$\boxed{\iota((\mathbf{T}^{(2)})^{-1}) = \iota(\mathbf{T}^{(2)})^{-1}}.$$

Thus, planar composition and inversion can be performed before or after embedding with the same result.

For example, take $x = 2$ m, $y = -1$ m, and $\theta = \pi/2$. Then

$$\mathbf{T}_{wb}^{(3)} = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

A body-frame point ${}^b\mathbf{p}_P^{(3)} = [1 \ 0 \ 0]^\top$ m is mapped to

$${}^w\mathbf{p}_P^{(3)} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} \text{ m}.$$

7.7.1 Recovering and checking a planar pose

The reverse conversion is defined only for spatial poses that satisfy the planar restrictions. Let

$$\mathbf{T}^{(3)} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3), \quad \mathbf{e}_z := \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Here, $\mathbf{p} = [p_1 \ p_2 \ p_3]^\top \in \mathbb{R}^3$ is the translation column, measured in metres. When $\mathbf{T}^{(3)} = \mathbf{T}_{wb}^{(3)}$, it is the world-frame position ${}^w\mathbf{p}_b$ of body origin O_b .

The spatial pose belongs to the embedded copy $\iota(SE(2)) \subset SE(3)$ exactly when

$$\boxed{\mathbf{R}\mathbf{e}_z = \mathbf{e}_z, \quad \mathbf{e}_z^\top \mathbf{p} = 0}.$$

The second condition selects the third component of the translation column:

$$\mathbf{e}_z^\top \mathbf{p} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} = p_3.$$

The first equation says that rotation leaves the common z -axis unchanged. Because $\mathbf{R} \in SO(3)$, its remaining two columns must then be a right-handed orthonormal basis of the x - y plane. The second equation says that the body origin has zero world-frame z -coordinate. The transformation must consequently have the structure

$$\mathbf{T}^{(3)} = \begin{bmatrix} r_{11} & r_{12} & 0 & p_1 \\ r_{21} & r_{22} & 0 & p_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} r_{11} & r_{12} \\ r_{21} & r_{22} \end{bmatrix} \in SO(2).$$

The planar coordinates can therefore be recovered as

$$\boxed{x = p_1, \quad y = p_2, \quad \theta = \text{atan2}(r_{21}, r_{11})}.$$

The function $\text{atan2}(u, v)$ returns the signed angle α satisfying $\sin \alpha = u/\sqrt{u^2 + v^2}$ and $\cos \alpha = v/\sqrt{u^2 + v^2}$. Here, $r_{21} = \sin \theta$ and $r_{11} = \cos \theta$, so using both entries identifies the correct quadrant. Its usual output interval is $(-\pi, \pi]$; angles differing by an integer multiple of 2π represent the same orientation.

An exact planar matrix uses zeros and ones in the restricted entries, but computed matrices contain rounding error. Let $\tilde{\mathbf{R}}$ and $\tilde{\mathbf{p}}$ denote computed rotation and translation blocks after the complete $SE(3)$ transformation has first been validated. A numerical planar check can use the residuals

$$\delta_R := \|\tilde{\mathbf{R}}\mathbf{e}_z - \mathbf{e}_z\|_2, \quad \delta_p := |\mathbf{e}_z^T \tilde{\mathbf{p}}|.$$

The orientation residual δ_R is dimensionless, while the position residual δ_p is measured in metres. With declared tolerances $\varepsilon_R > 0$ and $\varepsilon_p > 0$ having the same respective units, the pose may be treated as planar only when

$$\boxed{\delta_R \leq \varepsilon_R, \quad \delta_p \leq \varepsilon_p}.$$

The tolerances should reflect the numerical precision and the application's geometric accuracy rather than being hidden constants. A pose that fails either check should not be silently converted to (x, y, θ) . It must either be rejected as non-planar or passed to an explicitly declared projection that acknowledges the discarded information.

The embedding represents only the three-degree-of-freedom subset of $SE(3)$ satisfying the planar restrictions. It is not a projection of an arbitrary spatial pose onto a plane: such a projection would discard vertical position and rotations about horizontal axes and could not be inverted uniquely.

Quick test G: planar poses inside $SE(3)$

1. Write the $SE(3)$ matrix that embeds the planar pose coordinates (x, y, θ) .
2. Which entries of the embedded matrix enforce the planar-motion restrictions?
3. Does embedding a planar pose require every point of the robot to have zero z -coordinate? Explain with a body-fixed point $[p_x \ p_y \ p_z]^T$.
4. For a validated $\mathbf{T}^{(3)} \in SE(3)$ with rotation block $\mathbf{R}$ and translation block $\mathbf{p}$, state the two equations that determine whether it belongs to $\iota(SE(2))$, and show how to recover x , y , and θ .
5. Why should software use separate orientation and position tolerances when checking whether a computed spatial pose is planar?

Answers and hints

1. The embedding is

$$\mathbf{T}_{wb}^{(3)} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & x \\ \sin \theta & \cos \theta & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

2. The third rotation column and row keep z_b aligned with z_w , and the zero third translation entry keeps the body origin in the plane:

$$\mathbf{R}\mathbf{e}_z = \mathbf{e}_z, \quad \mathbf{e}_z^\top \mathbf{p} = 0.$$

3. No. The pose restricts the motion of the body frame, not the shape of the rigid body. An embedded pose maps the point to

$${}^w\mathbf{p}_P = \begin{bmatrix} \cos \theta p_x - \sin \theta p_y + x \\ \sin \theta p_x + \cos \theta p_y + y \\ p_z \end{bmatrix},$$

so the point retains its height p_z relative to the body origin.

4. For $\mathbf{e}_z = [0 \ 0 \ 1]^\top$, the exact conditions and recovered coordinates are

$$\boxed{\mathbf{R}\mathbf{e}_z = \mathbf{e}_z, \quad \mathbf{e}_z^\top \mathbf{p} = 0, \quad x = p_1, \quad y = p_2, \quad \theta = \text{atan2}(r_{21}, r_{11})}.$$

5. The rotation residual is dimensionless, whereas the vertical-position residual is measured in metres. One numerical threshold cannot express meaningful limits for both quantities. The tolerances must also be declared so that the accepted geometric error is visible and testable.

7.8 Unit quaternions and planar yaw

A rotation matrix is convenient for geometric calculations, but it stores nine entries even though a spatial orientation has only three degrees of freedom. Many robotics interfaces therefore store the same orientation using four quaternion components constrained to have unit norm. Unit quaternions also provide a compact rule for composing rotations. Specialised quaternion interpolation methods can construct intermediate orientations between two given orientations, but interpolation is outside this chapter's scope. The immediate problem in this section is to understand what the four quaternion components mean, derive how they represent and compose finite rotations, and then convert the planar yaw angle θ into a unit quaternion representing the same orientation as $\mathbf{R}_z(\theta)$.

7.8.1 Why introduce unit quaternions

The axis-angle representation from Section 7.5 describes a finite rotation using a unit axis direction $\mathbf{a} \in \mathbb{R}^3$, expressed in a declared coordinate frame, and a signed rotation angle $\alpha \in \mathbb{R}$ measured in radians. A unit quaternion packages this same rotation information into the ordered scalar-vector pair

$$\mathbf{q} := \left(\underbrace{\cos \frac{\alpha}{2}}_{q_w}, \underbrace{\mathbf{a} \sin \frac{\alpha}{2}}_{\mathbf{q}_v} \right), \quad \mathbf{q}_v := \begin{bmatrix} q_x \\ q_y \\ q_z \end{bmatrix} \in \mathbb{R}^3,$$

where $\mathbf{q}$ denotes the quaternion, $q_w \in \mathbb{R}$ is its scalar part, and $\mathbf{q}_v \in \mathbb{R}^3$ is its vector part. The vector part is the rotation axis $\mathbf{a}$ scaled by $\sin(\alpha/2)$, while the scalar part is $\cos(\alpha/2)$; together they encode the axis and signed angle. The half-angle does not mean that the physical rotation is only $\alpha/2$; the quaternion rotation operation derived below produces the full rotation angle α .

The quaternion is therefore a representation of a three-dimensional orientation, not a position or a physical vector in four-dimensional space. Its components can be collected into a column in $\mathbb{R}^4$ for storage and computation, but the individual values q_x , q_y , and q_z are not three independent rotation angles. They are the components of the scaled axis $\mathbf{a} \sin(\alpha/2)$. All four components are dimensionless.

Because $\mathbf{a}$ is a unit vector, $\mathbf{a}^\top \mathbf{a} = 1$, the four components satisfy

$$\|\mathbf{q}\|_2^2 := q_w^2 + \mathbf{q}_v^\top \mathbf{q}_v = q_w^2 + q_x^2 + q_y^2 + q_z^2 = \cos^2 \frac{\alpha}{2} + \sin^2 \frac{\alpha}{2} \mathbf{a}^\top \mathbf{a} = 1.$$

A quaternion satisfying this equation is called a **unit quaternion**. Although it stores four numbers, the unit-norm equation imposes one scalar constraint, leaving three independent local degrees of freedom, matching a spatial orientation in $SO(3)$. For zero rotation, $\alpha = 0$ gives $\mathbf{q} = (1, \mathbf{0}_3)$; for a rotation through π radians, it gives $\mathbf{q} = (0, \mathbf{a})$.

7.8.2 The Hamilton product and quaternion rotation

Quaternion multiplication is not componentwise multiplication. For $\mathbf{q}_1 = (q_{w1}, \mathbf{q}_{v1})$ and $\mathbf{q}_2 = (q_{w2}, \mathbf{q}_{v2})$, the **Hamilton product** $\otimes$ is defined by

$$\mathbf{q}_1 \otimes \mathbf{q}_2 := (q_{w1}q_{w2} - \mathbf{q}_{v1}^\top \mathbf{q}_{v2}, \quad q_{w1}\mathbf{q}_{v2} + q_{w2}\mathbf{q}_{v1} + \mathbf{q}_{v1} \times \mathbf{q}_{v2}).$$

The first output is the scalar part and the second output is the three-dimensional vector part. The cross-product term makes the Hamilton product generally noncommutative: changing the order can change the result.

The **Hamilton-product identity** is the quaternion

$$1_q := (1, \mathbf{0}_3),$$

because substituting it into the Hamilton-product definition gives

$$\mathbf{q} \otimes 1_q = 1_q \otimes \mathbf{q} = \mathbf{q}.$$

The inverse $\mathbf{q}^{-1}$ of a nonzero quaternion is defined by the requirement

$$\mathbf{q} \otimes \mathbf{q}^{-1} = \mathbf{q}^{-1} \otimes \mathbf{q} = 1_q.$$

Both multiplication orders are stated because the Hamilton product is generally noncommutative. To calculate this inverse, define the **quaternion conjugate** by reversing the sign of the vector part:

$$\mathbf{q}^* := (q_w, -\mathbf{q}_v).$$

Substitution into the Hamilton-product definition gives the same result in both orders:

$$\mathbf{q} \otimes \mathbf{q}^* = \mathbf{q}^* \otimes \mathbf{q} = (q_w^2 + \mathbf{q}_v^\top \mathbf{q}_v, \mathbf{0}_3).$$

The scalar component is the squared quaternion norm $\|\mathbf{q}\|_2^2$. Therefore, if $\|\mathbf{q}\|_2 > 0$, dividing the conjugate by this scalar gives

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|_2^2}.$$

Indeed, multiplying $\mathbf{q}$ by this expression from either side gives 1_q . For a unit quaternion, $\|\mathbf{q}\|_2^2 = 1$, so the inverse simplifies to

$$\mathbf{q}^{-1} = \mathbf{q}^*.$$

Thus, the identity defines what an inverse means, whereas the calculation proves that the conjugate is the inverse specifically when the quaternion has unit norm.

To rotate a coordinate column $\mathbf{y} \in \mathbb{R}^3$, first represent it as the **pure quaternion** $\mathbf{y} := (0, \mathbf{y})$, where *pure* means that the scalar part is zero. Quaternion rotation is defined by

$$\boxed{\mathbf{y}' = \mathbf{q} \otimes \mathbf{y} \otimes \mathbf{q}^*},$$

where $\mathbf{y}' = (0, \mathbf{y}')$ contains the rotated coordinate column $\mathbf{y}'$. To connect this definition to rotation matrices, first apply one Hamilton product:

$$\mathbf{q} \otimes \mathbf{y} = (-\mathbf{q}_v^T \mathbf{y}, \quad q_w \mathbf{y} + \mathbf{q}_v \times \mathbf{y}).$$

Multiplying this result by $\mathbf{q}^* = (q_w, -\mathbf{q}_v)$ makes the scalar part zero because

$$\begin{aligned} & -q_w \mathbf{q}_v^T \mathbf{y} + (q_w \mathbf{y} + \mathbf{q}_v \times \mathbf{y})^T \mathbf{q}_v \\ &= -q_w \mathbf{q}_v^T \mathbf{y} + q_w \mathbf{y}^T \mathbf{q}_v + (\mathbf{q}_v \times \mathbf{y})^T \mathbf{q}_v \\ &= -q_w \mathbf{q}_v^T \mathbf{y} + q_w \mathbf{q}_v^T \mathbf{y} + 0 \\ &= 0. \end{aligned}$$

The second equality distributes the transpose over the sum. The third equality uses the symmetry $\mathbf{y}^T \mathbf{q}_v = \mathbf{q}_v^T \mathbf{y}$ of the dot product and the fact that $\mathbf{q}_v \times \mathbf{y}$ is perpendicular to $\mathbf{q}_v$. Thus, the two dot-product terms cancel and the result remains a pure quaternion.

The vector part follows from the same Hamilton product. First distribute the scalar multiplication and cross product:

$$\begin{aligned} \mathbf{y}' &= (\mathbf{q}_v^T \mathbf{y}) \mathbf{q}_v + q_w (q_w \mathbf{y} + \mathbf{q}_v \times \mathbf{y}) - (q_w \mathbf{y} + \mathbf{q}_v \times \mathbf{y}) \times \mathbf{q}_v \\ &= (\mathbf{q}_v^T \mathbf{y}) \mathbf{q}_v + q_w^2 \mathbf{y} + q_w (\mathbf{q}_v \times \mathbf{y}) - q_w (\mathbf{y} \times \mathbf{q}_v) - (\mathbf{q}_v \times \mathbf{y}) \times \mathbf{q}_v. \end{aligned}$$

Anticommutativity gives $\mathbf{y} \times \mathbf{q}_v = -\mathbf{q}_v \times \mathbf{y}$. The vector triple-product identity from Section 1.1.5 gives

$$(\mathbf{q}_v \times \mathbf{y}) \times \mathbf{q}_v = (\mathbf{q}_v^T \mathbf{q}_v) \mathbf{y} - (\mathbf{q}_v^T \mathbf{y}) \mathbf{q}_v.$$

Substituting these two identities and collecting like terms gives

$$\begin{aligned} \mathbf{y}' &= (\mathbf{q}_v^T \mathbf{y}) \mathbf{q}_v + q_w^2 \mathbf{y} + 2q_w (\mathbf{q}_v \times \mathbf{y}) - (\mathbf{q}_v^T \mathbf{q}_v) \mathbf{y} + (\mathbf{q}_v^T \mathbf{y}) \mathbf{q}_v \\ &= (q_w^2 - \mathbf{q}_v^T \mathbf{q}_v) \mathbf{y} + 2\mathbf{q}_v (\mathbf{q}_v^T \mathbf{y}) + 2q_w (\mathbf{q}_v \times \mathbf{y}). \end{aligned}$$

The final expression separates the rotated result into a term parallel to $\mathbf{y}$, a term parallel to $\mathbf{q}_v$, and a term perpendicular to both vectors. It describes the active rotation of $\mathbf{y}$ while its coordinate frame remains fixed.

7.8.3 Why the half-angle formula produces the full rotation

It remains to verify that the unit quaternion defined above represents rotation through the full angle α . The required half-angle identities are

$$\begin{aligned} \cos^2 \frac{\alpha}{2} - \sin^2 \frac{\alpha}{2} &= \cos \alpha, \\ 2 \sin^2 \frac{\alpha}{2} &= 1 - \cos \alpha, \\ 2 \sin \frac{\alpha}{2} \cos \frac{\alpha}{2} &= \sin \alpha \end{aligned}$$

Substitute $q_w = \cos(\alpha/2)$ and $\mathbf{q}_v = \mathbf{a} \sin(\alpha/2)$ into each coefficient of the quaternion-rotation result. Because $\mathbf{a}$ is a unit vector, $\mathbf{a}^\top \mathbf{a} = 1$, the first coefficient becomes

$$\begin{aligned} q_w^2 - \mathbf{q}_v^\top \mathbf{q}_v &= \cos^2 \frac{\alpha}{2} - \sin^2 \frac{\alpha}{2} (\mathbf{a}^\top \mathbf{a}) \\ &= \cos^2 \frac{\alpha}{2} - \sin^2 \frac{\alpha}{2} \\ &= \cos \alpha. \end{aligned}$$

For the term parallel to the rotation axis,

$$\begin{aligned} 2\mathbf{q}_v(\mathbf{q}_v^\top \mathbf{y}) &= 2 \left(\mathbf{a} \sin \frac{\alpha}{2} \right) \left(\sin \frac{\alpha}{2} \mathbf{a}^\top \mathbf{y} \right) \\ &= 2 \sin^2 \frac{\alpha}{2} \mathbf{a}(\mathbf{a}^\top \mathbf{y}) \\ &= (1 - \cos \alpha) \mathbf{a}(\mathbf{a}^\top \mathbf{y}). \end{aligned}$$

For the cross-product term, bilinearity of the cross product allows the scalar $\sin(\alpha/2)$ to be taken outside:

$$\begin{aligned} 2q_w(\mathbf{q}_v \times \mathbf{y}) &= 2 \cos \frac{\alpha}{2} \left[\left(\mathbf{a} \sin \frac{\alpha}{2} \right) \times \mathbf{y} \right] \\ &= 2 \sin \frac{\alpha}{2} \cos \frac{\alpha}{2} (\mathbf{a} \times \mathbf{y}) \\ &= \sin \alpha (\mathbf{a} \times \mathbf{y}). \end{aligned}$$

Therefore,

$$\mathbf{y}' = \cos \alpha \mathbf{y} + (1 - \cos \alpha) \mathbf{a}(\mathbf{a}^\top \mathbf{y}) + \sin \alpha (\mathbf{a} \times \mathbf{y}).$$

To make the connection with the rotation matrix explicit, Section 7.5 derived Rodrigues' formula for the rotation matrix $\mathbf{Q}(\mathbf{a}, \alpha) \in SO(3)$:

$$\mathbf{Q}(\mathbf{a}, \alpha) = \mathbf{I}_3 + \sin \alpha [\mathbf{a}]_{\times} + (1 - \cos \alpha) [\mathbf{a}]_{\times}^2.$$

Applying this matrix to $\mathbf{y}$ and using $[\mathbf{a}]_{\times} \mathbf{y} = \mathbf{a} \times \mathbf{y}$ gives

$$\mathbf{Q}(\mathbf{a}, \alpha) \mathbf{y} = \mathbf{y} + \sin \alpha (\mathbf{a} \times \mathbf{y}) + (1 - \cos \alpha) \mathbf{a} \times (\mathbf{a} \times \mathbf{y}).$$

The vector triple-product identity and $\mathbf{a}^T \mathbf{a} = 1$ give

$$\mathbf{a} \times (\mathbf{a} \times \mathbf{y}) = \mathbf{a}(\mathbf{a}^T \mathbf{y}) - \mathbf{y}.$$

Substitution therefore produces

$$\begin{aligned} \mathbf{Q}(\mathbf{a}, \alpha) \mathbf{y} &= \mathbf{y} + \sin \alpha (\mathbf{a} \times \mathbf{y}) + (1 - \cos \alpha) [\mathbf{a}(\mathbf{a}^T \mathbf{y}) - \mathbf{y}] \\ &= \cos \alpha \mathbf{y} + (1 - \cos \alpha) \mathbf{a}(\mathbf{a}^T \mathbf{y}) + \sin \alpha (\mathbf{a} \times \mathbf{y}) \\ &= \mathbf{y}'. \end{aligned}$$

Thus, quaternion multiplication and Rodrigues' matrix transform every input vector $\mathbf{y}$ in the same way. They are two mathematical representations of the same finite rotation through angle α about axis $\mathbf{a}$.

The geometry is clearer after decomposing $\mathbf{y}$ into components parallel and perpendicular to the axis:

$$\mathbf{y}_{\parallel} := \mathbf{a}(\mathbf{a}^T \mathbf{y}), \quad \mathbf{y}_{\perp} := \mathbf{y} - \mathbf{y}_{\parallel}.$$

By definition, $\mathbf{a}(\mathbf{a}^T \mathbf{y}) = \mathbf{y}_{\parallel}$. Moreover, $\mathbf{a} \times \mathbf{y}_{\parallel} = \mathbf{0}_3$, so $\mathbf{a} \times \mathbf{y} = \mathbf{a} \times \mathbf{y}_{\perp}$. Substituting these relations and $\mathbf{y} = \mathbf{y}_{\parallel} + \mathbf{y}_{\perp}$ into the quaternion result gives

$$\begin{aligned} \mathbf{y}' &= \cos \alpha (\mathbf{y}_{\parallel} + \mathbf{y}_{\perp}) + (1 - \cos \alpha) \mathbf{y}_{\parallel} + \sin \alpha (\mathbf{a} \times \mathbf{y}_{\perp}) \\ &= [\cos \alpha + (1 - \cos \alpha)] \mathbf{y}_{\parallel} + \cos \alpha \mathbf{y}_{\perp} + \sin \alpha (\mathbf{a} \times \mathbf{y}_{\perp}) \\ &= \boxed{\mathbf{y}_{\parallel} + \cos \alpha \mathbf{y}_{\perp} + \sin \alpha (\mathbf{a} \times \mathbf{y}_{\perp})}. \end{aligned}$$

This is exactly the parallel-perpendicular geometric form derived in Section 7.5. The parallel component $\mathbf{y}_{\parallel}$ remains unchanged. In the plane perpendicular to $\mathbf{a}$, the two perpendicular directions $\mathbf{y}_{\perp}$ and $\mathbf{a} \times \mathbf{y}_{\perp}$ combine through $\cos \alpha$ and $\sin \alpha$, which rotates $\mathbf{y}_{\perp}$ through angle α according to the right-hand rule.

7.8.4 Composing active rotations

Suppose the unit quaternion q_1 actively rotates a geometric vector first and the unit quaternion q_2 actively rotates the result second, while all coordinate columns remain expressed in the same fixed frame. The question is which single quaternion produces the combined rotation.

Let $y = (0, \mathbf{y})$ be the pure quaternion formed from the original coordinate column $\mathbf{y} \in \mathbb{R}^3$. After the first rotation,

$$y_1 = q_1 \otimes y \otimes q_1^*.$$

Applying the second rotation gives

$$\begin{aligned} y_2 &= q_2 \otimes y_1 \otimes q_2^* \\ &= q_2 \otimes (q_1 \otimes y \otimes q_1^*) \otimes q_2^*. \end{aligned}$$

The Hamilton product is **associative**, meaning that regrouping consecutive products does not change their value:

$$(q_a \otimes q_b) \otimes q_c = q_a \otimes (q_b \otimes q_c).$$

It is generally not commutative, so regrouping is allowed but reversing the factors is not. Regrouping the two-rotation expression gives

$$y_2 = (q_2 \otimes q_1) \otimes y \otimes (q_1^* \otimes q_2^*).$$

To identify the final factor as a conjugate, write $q_i = (q_{wi}, \mathbf{q}_{vi})$ for $i \in \{1, 2\}$. The Hamilton-product definition gives

$$q_2 \otimes q_1 = (q_{w2}q_{w1} - \mathbf{q}_{v2}^\top \mathbf{q}_{v1}, \quad q_{w2}\mathbf{q}_{v1} + q_{w1}\mathbf{q}_{v2} + \mathbf{q}_{v2} \times \mathbf{q}_{v1}).$$

Conjugating this result negates its vector part. Conversely, multiplying the two conjugates in reversed order gives the same scalar part and the vector part

$$\begin{aligned} & q_{w1}(-\mathbf{q}_{v2}) + q_{w2}(-\mathbf{q}_{v1}) + (-\mathbf{q}_{v1}) \times (-\mathbf{q}_{v2}) \\ &= -q_{w1}\mathbf{q}_{v2} - q_{w2}\mathbf{q}_{v1} + \mathbf{q}_{v1} \times \mathbf{q}_{v2} \\ &= -(q_{w2}\mathbf{q}_{v1} + q_{w1}\mathbf{q}_{v2} + \mathbf{q}_{v2} \times \mathbf{q}_{v1}), \end{aligned}$$

where the second equality uses $\mathbf{q}_{v1} \times \mathbf{q}_{v2} = -\mathbf{q}_{v2} \times \mathbf{q}_{v1}$. Thus, conjugating a product reverses the factor order:

$$(q_2 \otimes q_1)^* = q_1^* \otimes q_2^*.$$

The expression for y_2 therefore has the standard quaternion-rotation form

$$y_2 = \underbrace{(q_2 \otimes q_1)}_{q_{\text{total}}} \otimes y \otimes \underbrace{(q_2 \otimes q_1)^*}_{q_{\text{total}}^*}.$$

Hence, the single quaternion representing the two active rotations is

$$q_{\text{total}} = q_2 \otimes q_1, \quad q_1 \text{ acts first and } q_2 \text{ acts second.}$$

The rightmost rotation therefore acts first, just as in the matrix product $\mathbf{R}_{\text{total}} = \mathbf{R}_2 \mathbf{R}_1$. Reversing the quaternion order generally produces a different orientation because spatial rotations are not generally commutative.

7.8.5 Planar yaw and ROS 2 component order

For planar yaw, the rotation axis is $\mathbf{a} = \mathbf{e}_z = [0 \ 0 \ 1]^T$ and the rotation angle is $\alpha = \theta$. The scalar–vector quaternion is consequently

$$q_{wb}^{\text{yaw}} = \left(\cos \frac{\theta}{2}, \begin{bmatrix} 0 \\ 0 \\ \sin(\theta/2) \end{bmatrix} \right).$$

The ROS 2 [geometry_msgs/msg/Quaternion](#) message stores the vector components first and the scalar component last. Its field-order column is therefore

$$\begin{bmatrix} q_x \\ q_y \\ q_z \\ q_w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \sin(\theta/2) \\ \cos(\theta/2) \end{bmatrix}.$$

The quaternion is already normalized because

$$q_x^2 + q_y^2 + q_z^2 + q_w^2 = \sin^2 \frac{\theta}{2} + \cos^2 \frac{\theta}{2} = 1.$$

For example, a positive yaw of $\theta = \pi/2$ gives

$$\begin{bmatrix} q_x \\ q_y \\ q_z \\ q_w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \sqrt{2}/2 \\ \sqrt{2}/2 \end{bmatrix}.$$

The quaternion components q_x , q_y , and q_z must not be confused with the position coordinates x , y , and z in a pose message. A spatial pose stores position and orientation as separate fields.

7.8.6 Equivalent signs and numerical normalization

The unit quaternions q and $-q$ represent the same orientation. Their rotation actions are identical because

$$(-q) \otimes y \otimes (-q)^* = q \otimes y \otimes q^*.$$

The two minus signs cancel because the Hamilton product is linear in each input. For the yaw formula, increasing θ by 2π changes both half-angle terms' signs and therefore changes q to $-q$, while leaving the physical orientation unchanged. Componentwise quaternion equality is consequently too strict for testing orientation equality unless a sign convention has first been imposed.

Floating-point computations may produce a finite quaternion component column $\tilde{\mathbf{c}}_{xyzw} := [\tilde{q}_x \ \tilde{q}_y \ \tilde{q}_z \ \tilde{q}_w]^\top \in \mathbb{R}^4$ whose norm is close to, but not exactly, one. Define its dimensionless unit-norm residual by

$$e_q := \left| \|\tilde{\mathbf{c}}_{xyzw}\|_2 - 1 \right|.$$

Let $\varepsilon_{\text{norm}} > 0$ be a declared dimensionless normalization tolerance. If all components are finite, the norm is safely nonzero, and $e_q \leq \varepsilon_{\text{norm}}$, the deviation is classified as small numerical drift and normalization gives

$$\mathbf{c}_{xyzw} = \frac{\tilde{\mathbf{c}}_{xyzw}}{\|\tilde{\mathbf{c}}_{xyzw}\|_2}.$$

The tolerance does not allow the norm error to remain: normalization reduces the accepted residual to floating-point precision. Instead, the tolerance decides whether the input is close enough to a unit quaternion to repair. If $e_q > \varepsilon_{\text{norm}}$, silently normalizing could hide corrupted data or an incorrect calculation, so the input should be rejected unless the interface explicitly specifies a different projection policy. Normalization also cannot repair

non-finite components, a zero or near-zero norm, an incorrect frame convention, or an incorrect rotation.

Quick test H: unit quaternions and planar yaw

1. For a unit rotation axis $\mathbf{a} \in \mathbb{R}^3$ and signed angle $\alpha \in \mathbb{R}$, write the corresponding unit quaternion as a scalar–vector pair. What do its two parts mean?
2. Why does a unit quaternion store four real components but describe only three local degrees of freedom?
3. Starting from the Hamilton-product definition, state the identity quaternion and derive the inverse of a nonzero quaternion. How does the result simplify for a unit quaternion?
4. Let q_1 act first and q_2 act second under the active-rotation rule $y' = q \otimes y \otimes q^*$. Derive the quaternion representing the combined rotation and explain why its order matters.
5. Why does the half-angle in $q = (\cos(\alpha/2), \mathbf{a} \sin(\alpha/2))$ still produce a physical rotation through the full angle α ?
6. Convert the planar yaw angle $\theta = \pi/2$ into the ROS 2 component order $[q_x \ q_y \ q_z \ q_w]^T$.
7. Why do q and $-q$ represent the same orientation, and what problem does this cause for direct componentwise comparisons?
8. A computed component column is finite and has norm $1 + 10^{-8}$. What declared check determines whether normalization is appropriate? Why would normalization be unsafe for a zero-norm or non-finite input?

Answers and hints

1. The axis–angle quaternion is

$$q = \left(\cos \frac{\alpha}{2}, \mathbf{a} \sin \frac{\alpha}{2} \right).$$

The scalar part stores $\cos(\alpha/2)$, while the vector part points along the rotation axis and has magnitude $|\sin(\alpha/2)|$. All components are dimensionless.

2. The four components satisfy the independent unit-norm constraint $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$. Therefore, only three components can vary independently in a local coordinate description: $4 - 1 = 3$ degrees of freedom.

3. The Hamilton-product identity is $1_q = (1, \mathbf{0}_3)$. For $q \neq (0, \mathbf{0}_3)$,

$$q \otimes q^* = (\|q\|_2^2, \mathbf{0}_3),$$

so

$$q^{-1} = \frac{q^*}{\|q\|_2^2}.$$

If $\|q\|_2 = 1$, then $q^{-1} = q^*$.

4. Two successive rotations give

$$\begin{aligned} y_2 &= q_2 \otimes (q_1 \otimes y \otimes q_1^*) \otimes q_2^* \\ &= (q_2 \otimes q_1) \otimes y \otimes (q_2 \otimes q_1)^*. \end{aligned}$$

Therefore, $q_{\text{total}} = q_2 \otimes q_1$. The Hamilton product is generally not commutative, so reversing the order generally changes the final orientation.

5. Substitution into the quaternion rotation action produces

$$\mathbf{y}' = \cos \alpha \mathbf{y} + (1 - \cos \alpha) \mathbf{a}(\mathbf{a}^\top \mathbf{y}) + \sin \alpha (\mathbf{a} \times \mathbf{y}),$$

Because $\mathbf{a}(\mathbf{a}^\top \mathbf{y}) = \mathbf{y}_\parallel$, $\mathbf{y} = \mathbf{y}_\parallel + \mathbf{y}_\perp$, and $\mathbf{a} \times \mathbf{y} = \mathbf{a} \times \mathbf{y}_\perp$, the same equation becomes

$$\mathbf{y}' = \mathbf{y}_\parallel + \cos \alpha \mathbf{y}_\perp + \sin \alpha (\mathbf{a} \times \mathbf{y}_\perp),$$

which is the geometric Rodrigues formula from Section 7.5. The full-angle terms arise from the half-angle identities used when the quaternion appears twice in $q \otimes y \otimes q^*$.

6. Substituting $\theta = \pi/2$ gives

$$\begin{bmatrix} q_x \\ q_y \\ q_z \\ q_w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \sqrt{2}/2 \\ \sqrt{2}/2 \end{bmatrix}.$$

7. Both signs produce the same sandwich product because the two negative factors cancel. Direct componentwise comparison can therefore report two equivalent orientations as unequal; a comparison must account for the sign ambiguity or compare their rotation actions.

8. Compute the residual $e_q = \|\tilde{\mathbf{c}}_{xyzw}\|_2 - 1 = 10^{-8}$ and compare it with the declared normalization tolerance $\varepsilon_{\text{norm}}$. If $e_q \leq \varepsilon_{\text{norm}}$ and the norm is safely nonzero, divide the column by its norm; the tolerance classifies the error as repairable rather than allowing it to remain. A zero or near-zero norm makes the division undefined or unstable, while non-finite components cannot be repaired by scaling; such inputs must be rejected.

7.9 Spatial rigid-body velocity and twists

A pose $\mathbf{T}_{wb}(t) \in SE(3)$ locates a rigid body at time t , but control and estimation also require its instantaneous motion. The body has one angular velocity and one translational velocity at a chosen reference point, yet different body-fixed points generally have different linear velocities when the body rotates. Before defining a spatial twist, this section derives the velocity of any body-fixed point from those two quantities.

7.9.1 Velocity of a body-fixed point

Let $t \in \mathbb{R}$ denote time in seconds and let

$$\mathbf{T}_{wb}(t) = \begin{bmatrix} \mathbf{R}_{wb}(t) & {}^w\mathbf{p}_b(t) \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3)$$

represent the pose of body frame $\{b\}$ relative to world frame $\{w\}$. Here, $\mathbf{R}_{wb}(t) \in SO(3)$ describes the body's orientation relative to the world, and ${}^w\mathbf{p}_b(t) \in \mathbb{R}^3$ is the position of the body origin O_b , expressed in world coordinates and measured in metres.

Choose a material point P fixed in the rigid body. Its body-frame coordinate ${}^b\mathbf{p}_P \in \mathbb{R}^3$, measured in metres, is constant because both P and frame $\{b\}$ move with the same rigid body. The point transformation from Section 7.6.1 gives its world position:

$${}^w\mathbf{p}_P(t) = \mathbf{R}_{wb}(t) {}^b\mathbf{p}_P + {}^w\mathbf{p}_b(t).$$

Differentiate this equation with respect to time. Because ${}^b\mathbf{p}_P$ is constant, its derivative is zero, so

$$\begin{aligned} {}^w\mathbf{v}_P &:= \frac{d}{dt} {}^w\mathbf{p}_P = \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P + \frac{d}{dt} {}^w\mathbf{p}_b \\ &= \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{v}_b, \end{aligned}$$

where ${}^w\mathbf{v}_P \in \mathbb{R}^3$ is the velocity of point P expressed in world coordinates and ${}^w\mathbf{v}_b := d({}^w\mathbf{p}_b)/dt \in \mathbb{R}^3$ is the velocity of the body origin expressed in world coordinates. Both velocities are measured in m s^{-1} .

The angular-velocity derivation in Section 7.4 gave the world-expressed orientation-rate equation

$$\dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb},$$

where $\boldsymbol{\omega}_w \in \mathbb{R}^3$ is the body's angular velocity relative to the world, expressed along the world axes and measured in rad s^{-1} . Define the relative-position vector from O_b to P , expressed in world coordinates, by

$${}^w\mathbf{r}_{O_bP} := {}^w\mathbf{p}_P - {}^w\mathbf{p}_b = \mathbf{R}_{wb} {}^b\mathbf{p}_P \in \mathbb{R}^3.$$

The vector ${}^w\mathbf{r}_{O_bP}$ is measured in metres. It describes the displacement from the body origin to P at the selected instant; it is not the world position of either endpoint by itself.

Substituting these two relations into the point-velocity equation gives

$$\begin{aligned} {}^w\mathbf{v}_P &= {}^w\mathbf{v}_b + [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb} {}^b\mathbf{p}_P \\ &= {}^w\mathbf{v}_b + [\boldsymbol{\omega}_w]_{\times} {}^w\mathbf{r}_{O_bP} \\ &= \boxed{{}^w\mathbf{v}_b + \boldsymbol{\omega}_w \times {}^w\mathbf{r}_{O_bP}}. \end{aligned}$$

The first term is the translational velocity shared with the body origin. The second term is the additional velocity caused by rotation about the body origin. It is perpendicular to both $\boldsymbol{\omega}_w$ and ${}^w\mathbf{r}_{O_bP}$, and its magnitude is

$$\|\boldsymbol{\omega}_w \times {}^w\mathbf{r}_{O_bP}\|_2 = \|\boldsymbol{\omega}_w\|_2 \|{}^w\mathbf{r}_{O_bP}\|_2 \sin \varphi,$$

where $\varphi \in [0, \pi]$ is the angle between the angular-velocity vector and the relative-position vector. A point farther from the rotation axis therefore has a larger rotational speed. If $P = O_b$, if $\boldsymbol{\omega}_w = \mathbf{0}_3$, or if ${}^w\mathbf{r}_{O_bP}$ is parallel to $\boldsymbol{\omega}_w$, the rotational contribution is zero.

For example, suppose that, at one selected instant, a sensor is located 0.2 m from O_b along the positive world direction x_w , while

$${}^w\mathbf{r}_{O_bP} = \begin{bmatrix} 0.2 \\ 0 \\ 0 \end{bmatrix} \text{ m}, \quad {}^w\mathbf{v}_b = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m s}^{-1}, \quad \boldsymbol{\omega}_w = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \text{ rad s}^{-1}.$$

The rotational contribution is

$$\boldsymbol{\omega}_w \times {}^w\mathbf{r}_{O_bP} = \begin{bmatrix} 0 \\ 0.2 \\ 0 \end{bmatrix} \text{ m s}^{-1},$$

so the sensor velocity is

$${}^w\mathbf{v}_P = \begin{bmatrix} 1 \\ 0.2 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

The body origin moves only along x_w at this instant, but the offset sensor also moves along y_w because the body is rotating about z_w .

The planar point-velocity equation from Section 6.3.4 is the special case

$$\boldsymbol{\omega}_w = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix}, \quad {}^w\mathbf{r}_{O_bP} = \begin{bmatrix} r_x \\ r_y \\ 0 \end{bmatrix}.$$

Indeed,

$$\boldsymbol{\omega}_w \times {}^w\mathbf{r}_{O_bP} = \begin{bmatrix} -\omega r_y \\ \omega r_x \\ 0 \end{bmatrix},$$

whose first two components equal $\omega \mathbf{S}_2 [r_x \ r_y]^\top$. The three-dimensional equation is therefore a direct extension of the planar rigid-body velocity relation.

7.9.2 Body twist and $\mathfrak{se}(3)$

The point-velocity equation requires two quantities: the velocity of the body origin and the angular velocity of the body. A **spatial rigid-body twist** packages these two parts into one six-component representation of the body's instantaneous motion. This subsection first expresses both parts along the moving body axes; the resulting representation is called a **body twist**.

The word *body* describes the coordinate axes used for the components. It does not mean that the body origin moves relative to its own attached frame. The physical motion is still measured relative to the world; only its numerical components are expressed along the instantaneous body axes.

Expressing the two velocity parts along the body axes

For compactness, write the time-varying pose and its derivative as

$$\mathbf{T} := \mathbf{T}_{wb} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \dot{\mathbf{T}} = \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_3^\top & 0 \end{bmatrix},$$

where $\mathbf{R} = \mathbf{R}_{wb} \in SO(3)$ is the orientation of frame $\{b\}$ relative to frame $\{w\}$, $\mathbf{p} = {}^w\mathbf{p}_b \in \mathbb{R}^3$ is the world position of the body origin measured in metres, and $\dot{\mathbf{p}} = {}^w\mathbf{v}_b \in \mathbb{R}^3$ is that origin's velocity relative to the world, expressed in world coordinates and measured in m s^{-1} .

The inverse pose from Section 7.6.2 is

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

Premultiplying the pose derivative by this inverse gives

$$\begin{aligned} \mathbf{T}^{-1} \dot{\mathbf{T}} &= \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}^\top \dot{\mathbf{R}} & \mathbf{R}^\top \dot{\mathbf{p}} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}. \end{aligned}$$

The translation column $-\mathbf{R}^\top \mathbf{p}$ contributes zero because it multiplies the zero bottom row of $\dot{\mathbf{T}}$. The angular-velocity relations from Section 7.4 are

$$\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_b]_\times, \quad \boldsymbol{\omega}_b = \mathbf{R}_{wb}^\top \boldsymbol{\omega}_w \in \mathbb{R}^3,$$

where $\boldsymbol{\omega}_b$ is the body's angular velocity relative to the world, expressed along the body axes and measured in rad s^{-1} . Define the body-expressed linear velocity of the body origin by

$$\boldsymbol{\nu}_b := \mathbf{R}_{wb}^\top {}^w\mathbf{v}_b = \begin{bmatrix} \nu_{x_b} \\ \nu_{y_b} \\ \nu_{z_b} \end{bmatrix} \in \mathbb{R}^3.$$

The entries ν_{x_b} , ν_{y_b} , and ν_{z_b} are the forward, lateral, and vertical components of the world-relative body-origin velocity along the body axes x_b , y_b , and z_b , respectively. They are measured in m s^{-1} . Multiplication by $\mathbf{R}_{wb}^\top$ changes the coordinate description; it does not create a different physical velocity.

Because $\mathbf{R} = \mathbf{R}_{wb}$ and $\dot{\mathbf{p}} = {}^w\mathbf{v}_b$, the two nonzero blocks in the product are therefore

$$\mathbf{R}^\top \dot{\mathbf{R}} = [\boldsymbol{\omega}_b]_\times, \quad \mathbf{R}^\top \dot{\mathbf{p}} = \mathbf{R}_{wb}^\top {}^w\mathbf{v}_b = \boldsymbol{\nu}_b.$$

Substituting both results back into the block product completes the calculation:

$$\mathbf{T}^{-1} \dot{\mathbf{T}} = \begin{bmatrix} [\boldsymbol{\omega}_b]_\times & \boldsymbol{\nu}_b \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

Thus, premultiplication by $\mathbf{T}^{-1}$ uses the current pose to express the pose rate in body coordinates. The terms containing the current position $\mathbf{p}$ vanish because they multiply zeros, while $\mathbf{R}^\top$ converts the remaining orientation and position rates into angular and linear velocity components expressed along the body axes. The body's motion is re-expressed, not removed.

Twist coordinates and the hat operator

Using the component order established for planar twists in Section 6.4.3, define the six-component **body-twist column** by

$$\boldsymbol{\xi}_b := \begin{bmatrix} \boldsymbol{\nu}_b \\ \boldsymbol{\omega}_b \end{bmatrix} = \begin{bmatrix} \nu_{x_b} \\ \nu_{y_b} \\ \nu_{z_b} \\ \omega_{x_b} \\ \omega_{y_b} \\ \omega_{z_b} \end{bmatrix} \in \mathbb{R}^6.$$

The first three entries are linear-velocity components in m s^{-1} , and the final three entries are angular-velocity components in rad s^{-1} . Thus, $\boldsymbol{\xi}_b$ is a six-entry real coordinate column, but its entries do not all have the same physical units. Some references place angular velocity before linear velocity; a twist's component order must therefore always be declared.

The three-dimensional **hat operator** maps this coordinate column to a structured 4×4 matrix:

$$\hat{\boldsymbol{\xi}}_b := \begin{bmatrix} [\boldsymbol{\omega}_b]_\times & \boldsymbol{\nu}_b \\ \mathbf{0}_3^\top & 0 \end{bmatrix} = \begin{bmatrix} 0 & -\omega_{z_b} & \omega_{y_b} & \nu_{x_b} \\ \omega_{z_b} & 0 & -\omega_{x_b} & \nu_{y_b} \\ -\omega_{y_b} & \omega_{x_b} & 0 & \nu_{z_b} \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The upper-left block represents the angular-velocity operation $\boldsymbol{\omega}_b \times (\cdot)$, the upper-right column stores the body-origin linear velocity, and the bottom row is zero because the bottom row of every homogeneous transformation is the constant row $[0 \ 0 \ 0 \ 1]$. The inverse mapping from the structured matrix to the six-component column is called the **vee operator** and is written $(\hat{\boldsymbol{\xi}}_b)^\vee = \boldsymbol{\xi}_b$.

Comparing the hat-operator definition with the completed block product above gives

$$\hat{\boldsymbol{\xi}}_b = \mathbf{T}_{wb}^{-1} \dot{\mathbf{T}}_{wb}.$$

Multiplying both sides on the left by $\mathbf{T}_{wb}$ gives the equivalent pose-rate equation

$$\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb} \hat{\boldsymbol{\xi}}_b.$$

The first equation extracts the body twist from a differentiable pose trajectory. The second equation computes the world-frame pose rate from the current pose and a known body twist, making it suitable for kinematic prediction and numerical integration.

Pose rate and body twist: same motion, different forms

The pose rate and body twist do not describe two different physical motions. Given the current pose $\mathbf{T}_{wb}$, they contain equivalent information about the same instantaneous rigid-body motion, and either one can be calculated from the other:

$$\dot{\mathbf{T}}_{wb} \longleftrightarrow \hat{\boldsymbol{\xi}}_b, \quad \hat{\boldsymbol{\xi}}_b = \mathbf{T}_{wb}^{-1} \dot{\mathbf{T}}_{wb}, \quad \dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb} \hat{\boldsymbol{\xi}}_b.$$

They differ in mathematical form and coordinate meaning. The pose rate $\dot{\mathbf{T}}_{wb} \in \mathbb{R}^{4 \times 4}$ is the derivative of the homogeneous pose representation at its current value. Its blocks $\dot{\mathbf{R}}_{wb}$ and $\dot{\mathbf{p}} = {}^w\mathbf{v}_b$ describe how the body axes and body-origin position change relative to the world. It is not an element of $SE(3)$ because $\dot{\mathbf{R}}_{wb}$ is not a finite rotation matrix and its bottom-right entry is zero.

The body twist $\boldsymbol{\xi}_b \in \mathbb{R}^6$ instead packages the motion into three linear-velocity and three angular-velocity components expressed along the instantaneous body axes. Its linear part $\boldsymbol{\nu}_b = \mathbf{R}_{wb}^T \dot{\mathbf{p}}$ is not the derivative of the body origin's position relative to its own frame: that relative position is always ${}^b\mathbf{p}_b = \mathbf{0}_3$. Rather, $\boldsymbol{\nu}_b$ is the body origin's velocity relative to the world, expressed using body-axis coordinates.

For example, consider the body twist

$$\boldsymbol{\xi}_b = [1 \ 0 \ 0 \ 0 \ 0 \ 0]^\top,$$

whose first three entries are measured in m s^{-1} and final three entries are measured in rad s^{-1} . It describes motion at 1 m s^{-1} along positive x_b with no rotation. If $\mathbf{R}_{wb} = \mathbf{I}_3$, then

$$\dot{\mathbf{p}} = \mathbf{R}_{wb} \boldsymbol{\nu}_b = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

If the same body is instead oriented by $\mathbf{R}_{wb} = \mathbf{R}_z(\pi/2)$, the same body twist gives

$$\dot{\mathbf{p}} = \mathbf{R}_{wb} \boldsymbol{\nu}_b = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

The body-frame instruction “move along positive x_b ” is unchanged, but its world-coordinate direction changes with the current orientation. Consequently, the body twist is the same while the world-frame pose rate is different. The current pose supplies the conversion between them.

The useful mental model is therefore

pose rate and body twist encode the same instantaneous motion

but

they express it in different mathematical forms and coordinates.

Why the hat matrix belongs to $\mathfrak{se}(3)$

The set $SE(3)$ contains finite spatial rigid transformations. Its instantaneous directions at the identity transformation $\mathbf{I}_4$ form the tangent space $T_{\mathbf{I}_4} SE(3)$, denoted $\mathfrak{se}(3)$ and read “little s e three.” This statement can be established directly from the block structure of a curve in $SE(3)$.

Let

$$\mathbf{G}(s) = \begin{bmatrix} \mathbf{Q}(s) & \mathbf{d}(s) \\ \mathbf{0}_3^T & 1 \end{bmatrix} \in SE(3), \quad \mathbf{G}(0) = \mathbf{I}_4,$$

be any differentiable curve with dimensionless parameter $s \in \mathbb{R}$. The identity condition gives $\mathbf{Q}(0) = \mathbf{I}_3$ and $\mathbf{d}(0) = \mathbf{0}_3$. Differentiation at $s = 0$ gives

$$\mathbf{G}'(0) = \begin{bmatrix} \mathbf{Q}'(0) & \mathbf{d}'(0) \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

Because $\mathbf{Q}(s) \in SO(3)$, differentiating $\mathbf{Q}(s)^\top \mathbf{Q}(s) = \mathbf{I}_3$ at $s = 0$ gives

$$\mathbf{Q}'(0)^\top + \mathbf{Q}'(0) = \mathbf{0}_{3 \times 3}.$$

Therefore, $\mathbf{Q}'(0)$ is skew-symmetric and has the unique form $[\mathbf{a}]_\times$ for some $\mathbf{a} \in \mathbb{R}^3$. The translation derivative $\mathbf{d}'(0)$ may be any column $\mathbf{u} \in \mathbb{R}^3$. Hence, every tangent direction must have the form

$$\begin{bmatrix} [\mathbf{a}]_\times & \mathbf{u} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

The converse is also true. Given any $\mathbf{a}, \mathbf{u} \in \mathbb{R}^3$, the curve

$$\mathbf{G}_{\mathbf{a}, \mathbf{u}}(s) := \begin{bmatrix} \exp(s[\mathbf{a}]_\times) & s\mathbf{u} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}$$

belongs to $SE(3)$, passes through $\mathbf{I}_4$ at $s = 0$, and has derivative

$$\mathbf{G}'_{\mathbf{a}, \mathbf{u}}(0) = \begin{bmatrix} [\mathbf{a}]_\times & \mathbf{u} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

The first direction shows that no other tangent-matrix structure is possible; the converse shows that every matrix with the stated structure is attainable by a valid curve through the identity. Therefore,

$$\mathfrak{se}(3) := T_{\mathbf{I}_4} SE(3) = \left\{ \begin{bmatrix} [\mathbf{a}]_\times & \mathbf{u} \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \mid \mathbf{a}, \mathbf{u} \in \mathbb{R}^3 \right\}.$$

This is a six-dimensional vector space because three independent numbers determine $\mathbf{a}$ and three determine $\mathbf{u}$. When the curve parameter is physical time, $\mathbf{a}$ becomes angular velocity, $\mathbf{u}$ becomes linear velocity, and the resulting matrix is the hatted twist. An element of $SE(3)$ describes a finite pose or coordinate transformation; an element of $\mathfrak{se}(3)$ describes an instantaneous direction or rate and is not itself a finite pose.

Worked coordinate check

Suppose that, at one instant, the body orientation and world-expressed velocities are

$$\mathbf{R}_{wb} = \mathbf{R}_z\left(\frac{\pi}{2}\right), \quad {}^w\mathbf{v}_b = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m s}^{-1}, \quad \boldsymbol{\omega}_w = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \text{ rad s}^{-1}.$$

The body is turned by $\pi/2$ about z_w , so its positive y_b -axis points along negative x_w . Expressing the same velocities along the body axes gives

$$\begin{aligned} \boldsymbol{\nu}_b &= \mathbf{R}_{wb}^\top {}^w\mathbf{v}_b = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} \text{ m s}^{-1}, \\ \boldsymbol{\omega}_b &= \mathbf{R}_{wb}^\top \boldsymbol{\omega}_w = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \text{ rad s}^{-1}. \end{aligned}$$

Thus,

$$\boldsymbol{\xi}_b = [0 \quad -1 \quad 0 \quad 0 \quad 0 \quad 2]^\top,$$

with the first three entries measured in m s^{-1} and the final three measured in rad s^{-1} . The negative second component does not indicate a different motion: it states that motion along positive x_w points along negative y_b at this orientation.

7.9.3 Spatial twist expressed in the world frame

The body twist expresses instantaneous motion using the moving body axes and the body origin as its reference. Some calculations instead need one fixed reference shared by every body, such as comparing several robots in a world map or evaluating velocities at world-coordinate locations. The corresponding representation is called the **spatial twist expressed in the world frame**. Here, *spatial* means that the fixed space, or world, frame supplies both the coordinate axes and the reference origin.

Retain the compact pose notation from Section 7.9.2:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \dot{\mathbf{T}} = \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_3^\top & 0 \end{bmatrix},$$

where $\mathbf{R} = \mathbf{R}_{wb} \in SO(3)$, $\mathbf{p} = {}^w\mathbf{p}_b \in \mathbb{R}^3$, and $\dot{\mathbf{p}} = {}^w\mathbf{v}_b \in \mathbb{R}^3$. To express the pose rate relative to the fixed world frame, postmultiply $\dot{\mathbf{T}}$ by the inverse pose:

$$\begin{aligned}\dot{\mathbf{T}}\mathbf{T}^{-1} &= \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top\mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \dot{\mathbf{R}}\mathbf{R}^\top & \dot{\mathbf{p}} - \dot{\mathbf{R}}\mathbf{R}^\top\mathbf{p} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.\end{aligned}$$

The upper-left block is the world-expressed angular-velocity matrix derived in Section 7.4:

$$\dot{\mathbf{R}}\mathbf{R}^\top = [\boldsymbol{\omega}_w]_\times,$$

where $\boldsymbol{\omega}_w \in \mathbb{R}^3$ is the body's angular velocity relative to the world, expressed along the world axes and measured in rad s^{-1} . The upper-right block is therefore

$$\dot{\mathbf{p}} - [\boldsymbol{\omega}_w]_\times\mathbf{p} = \dot{\mathbf{p}} - \boldsymbol{\omega}_w \times \mathbf{p}.$$

Define this column as

$$\boldsymbol{\nu}_w := \dot{\mathbf{p}} - \boldsymbol{\omega}_w \times \mathbf{p} = \begin{bmatrix} \nu_{x_w} \\ \nu_{y_w} \\ \nu_{z_w} \end{bmatrix} \in \mathbb{R}^3.$$

Its entries are components along x_w , y_w , and z_w and are measured in m s^{-1} . Radians are dimensionless, so $(\text{rad s}^{-1})(\text{m}) = \text{m s}^{-1}$ for the cross-product term.

The physical meaning of $\boldsymbol{\nu}_w$ can be stated immediately. The relative-position vector from the body origin O_b , currently at $\mathbf{p}$, to the world origin O_w , currently at $\mathbf{0}_3$, is

$${}^w\mathbf{r}_{O_b O_w} = \mathbf{0}_3 - \mathbf{p} = -\mathbf{p}.$$

The rigid-body point-velocity relation therefore gives

$$\underbrace{\boldsymbol{\nu}_w}_{\text{velocity field at } O_w} = \underbrace{\dot{\mathbf{p}}}_{\text{velocity at } O_b} + \underbrace{\boldsymbol{\omega}_w \times (-\mathbf{p})}_{\text{velocity change caused by the different location}}.$$

Thus, $\boldsymbol{\nu}_w$ is the velocity that the body's instantaneous rigid-motion field assigns to the world-origin location, expressed along the world axes. It is generally not the actual velocity $\dot{\mathbf{p}}$ of the body origin. The detailed point-velocity derivation below verifies this interpretation. Substituting both named blocks back into the matrix product now completes the calculation:

$$\dot{\mathbf{T}}\mathbf{T}^{-1} = \begin{bmatrix} [\boldsymbol{\omega}_w]_{\times} & \boldsymbol{\nu}_w \\ \mathbf{0}_3^T & 0 \end{bmatrix}.$$

Using the same linear-first component order as the body twist, define the **spatial-twist column** and its hat matrix by

$$\boldsymbol{\xi}_w := \begin{bmatrix} \boldsymbol{\nu}_w \\ \boldsymbol{\omega}_w \end{bmatrix} \in \mathbb{R}^6, \quad \hat{\boldsymbol{\xi}}_w := \begin{bmatrix} [\boldsymbol{\omega}_w]_{\times} & \boldsymbol{\nu}_w \\ \mathbf{0}_3^T & 0 \end{bmatrix} \in \mathfrak{se}(3).$$

Comparison with the completed block product gives

$$\hat{\boldsymbol{\xi}}_w = \dot{\mathbf{T}}_{wb} \mathbf{T}_{wb}^{-1}, \quad \dot{\mathbf{T}}_{wb} = \hat{\boldsymbol{\xi}}_w \mathbf{T}_{wb}.$$

The first equation extracts the world-referenced spatial twist from a pose trajectory. The second reconstructs the pose rate from the current pose and that twist. The multiplication side distinguishes the two conventions: a body-twist matrix multiplies the pose on the right, whereas a spatial-twist matrix multiplies it on the left.

Physical meaning of $\boldsymbol{\nu}_w$

The column $\boldsymbol{\nu}_w$ is generally not the body-origin velocity $\dot{\mathbf{p}}$. Its physical meaning follows from the body-fixed point-velocity equation in Section 7.9.1:

$${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \boldsymbol{\omega}_w \times {}^w\mathbf{r}_{O_bP},$$

where ${}^w\mathbf{v}_P \in \mathbb{R}^3$ is the world-relative velocity of a body-fixed point P , expressed in world coordinates, and ${}^w\mathbf{r}_{O_bP} \in \mathbb{R}^3$ is the relative-position vector from the body origin O_b to P , also expressed in world coordinates. Because

$${}^w\mathbf{v}_b = \dot{\mathbf{p}}, \quad {}^w\mathbf{r}_{O_bP} = {}^w\mathbf{p}_P - \mathbf{p},$$

substitution and regrouping give

$$\begin{aligned}
{}^w\mathbf{v}_P &= \dot{\mathbf{p}} + \boldsymbol{\omega}_w \times ({}^w\mathbf{p}_P - \mathbf{p}) \\
&= \dot{\mathbf{p}} - \boldsymbol{\omega}_w \times \mathbf{p} + \boldsymbol{\omega}_w \times {}^w\mathbf{p}_P \\
&= \boldsymbol{\nu}_w + \boldsymbol{\omega}_w \times {}^w\mathbf{p}_P.
\end{aligned}$$

Thus, the spatial twist defines the world-frame **rigid-motion velocity field**—a rule that calculates the instantaneous velocity of any body-fixed point from that point's world-coordinate location:

$$\boxed{{}^w\mathbf{v}_P = \boldsymbol{\nu}_w + \boldsymbol{\omega}_w \times {}^w\mathbf{p}_P}.$$

Let O_w denote the fixed world origin, whose world-coordinate column is $\mathbf{0}_3$. Evaluating the field at that location gives

$${}^w\mathbf{v}_P|_{{}^w\mathbf{p}_P=\mathbf{0}_3} = \boldsymbol{\nu}_w.$$

Therefore, $\boldsymbol{\nu}_w$ is the velocity that the instantaneous rigid-motion field assigns to the location O_w , expressed along the world axes. The world origin does not need to be a physical point on the robot; the same rigid-motion rule can be evaluated mathematically at that location.

At the body origin, ${}^w\mathbf{p}_P = \mathbf{p}$ and $P = O_b$, so the same field gives

$$\boxed{
\begin{array}{ccccc}
\underbrace{{}^w\mathbf{v}_b}_{\text{velocity at } O_b} & = & \underbrace{\boldsymbol{\nu}_w}_{\text{velocity field at } O_w} & + & \underbrace{\boldsymbol{\omega}_w \times \mathbf{p}}_{\text{change caused by the different location}}
\end{array}
}. \quad (7.1)$$

This equation also follows immediately by rearranging $\boldsymbol{\nu}_w = \dot{\mathbf{p}} - \boldsymbol{\omega}_w \times \mathbf{p}$. It shows why $\boldsymbol{\nu}_w$ alone must not be called the body-origin velocity: the two quantities refer to different locations. They are equal when $\boldsymbol{\omega}_w \times \mathbf{p} = \mathbf{0}_3$, including pure translation or a body origin lying on the instantaneous rotation-axis line through O_w .

Worked reference-origin check

Suppose that, at one instant,

$$\mathbf{p} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} \text{ m}, \quad \dot{\mathbf{p}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m s}^{-1}, \quad \boldsymbol{\omega}_w = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \text{ rad s}^{-1}.$$

The velocity difference caused by rotation between the world-origin location and the body origin is

$$\boldsymbol{\omega}_w \times \mathbf{p} = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \times \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 4 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Therefore, the linear part of the spatial twist is

$$\boldsymbol{\nu}_w = \dot{\mathbf{p}} - \boldsymbol{\omega}_w \times \mathbf{p} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Because $\boldsymbol{\omega}_w \times \mathbf{p} \neq \mathbf{0}_3$, the spatial-twist component $\boldsymbol{\nu}_w = [3 \ -4 \ 0]^\top \text{ m s}^{-1}$ differs from the given body-origin velocity $\dot{\mathbf{p}} = [1 \ 0 \ 0]^\top \text{ m s}^{-1}$. The first is the velocity field evaluated at O_w ; the second is its value at O_b . Their difference is exactly the rotational term $\boldsymbol{\omega}_w \times \mathbf{p}$. The next subsection will derive how the current pose converts between the complete body and spatial twists.

7.9.4 Relating body and spatial twists through the adjoint

The body twist $\boldsymbol{\xi}_b$ and spatial twist $\boldsymbol{\xi}_w$ describe the same instantaneous rigid-body motion, but their components use different axes and different reference origins. A conversion between them must therefore account for both the orientation $\mathbf{R}_{wb}$ and position $\mathbf{p} = {}^w\mathbf{p}_b$ in the current pose $\mathbf{T}_{wb}$.

Convert the angular and linear parts

Write $\mathbf{R} := \mathbf{R}_{wb}$. The angular-velocity coordinate relation was already established in Section 7.4.2:

$$\boxed{\boldsymbol{\omega}_w = \mathbf{R}\boldsymbol{\omega}_b}.$$

This equation rotates the coordinate column of the same physical angular velocity from the body axes into the world axes. The present derivation uses this previously established relation.

For the linear part, $\boldsymbol{\nu}_b$ is the velocity of the body origin O_b expressed along the body axes. Its world-coordinate column is therefore

$${}^w\mathbf{v}_b = \mathbf{R}\boldsymbol{\nu}_b.$$

The spatial-twist velocity field in Equation 7.1 gives the same body-origin velocity as

$${}^w\mathbf{v}_b = \boldsymbol{\nu}_w + \boldsymbol{\omega}_w \times \mathbf{p}.$$

Equating these two descriptions of ${}^w\mathbf{v}_b$ and solving for $\boldsymbol{\nu}_w$ gives

$$\begin{aligned}\boldsymbol{\nu}_w &= \mathbf{R}\boldsymbol{\nu}_b - \boldsymbol{\omega}_w \times \mathbf{p} \\ &= \mathbf{R}\boldsymbol{\nu}_b + \mathbf{p} \times \boldsymbol{\omega}_w \\ &= \boxed{\mathbf{R}\boldsymbol{\nu}_b + [\mathbf{p}]_{\times} \mathbf{R}\boldsymbol{\omega}_b}.\end{aligned}$$

The second equality uses cross-product anticommutativity, $-\boldsymbol{\omega}_w \times \mathbf{p} = \mathbf{p} \times \boldsymbol{\omega}_w$. The final equality uses $\mathbf{p} \times \boldsymbol{\omega}_w = [\mathbf{p}]_{\times} \boldsymbol{\omega}_w$ and the already-established relation $\boldsymbol{\omega}_w = \mathbf{R}\boldsymbol{\omega}_b$.

The two terms have different purposes. The term $\mathbf{R}\boldsymbol{\nu}_b$ changes the linear-velocity components from body axes to world axes. The term $[\mathbf{p}]_{\times} \mathbf{R}\boldsymbol{\omega}_b$ changes the reference location for the linear part from O_b to O_w . Angular velocity needs no reference-location correction because every point of a rigid body shares the same angular velocity.

Confirm the conversion using the hat matrices

The same result follows from the two hat-matrix definitions:

$$\hat{\boldsymbol{\xi}}_b = \mathbf{T}^{-1}\dot{\mathbf{T}}, \quad \hat{\boldsymbol{\xi}}_w = \dot{\mathbf{T}}\mathbf{T}^{-1}.$$

The body-twist definition gives $\dot{\mathbf{T}} = \mathbf{T}\hat{\boldsymbol{\xi}}_b$. Substitution into the spatial-twist definition gives

$$\begin{aligned}\hat{\boldsymbol{\xi}}_w &= \dot{\mathbf{T}}\mathbf{T}^{-1} \\ &= (\mathbf{T}\hat{\boldsymbol{\xi}}_b)\mathbf{T}^{-1} \\ &= \boxed{\mathbf{T}\hat{\boldsymbol{\xi}}_b\mathbf{T}^{-1}}.\end{aligned}$$

This three-matrix product is called **conjugation by $\mathbf{T}$** . Here, conjugation uses the current pose and its inverse to change the reference used by the hatted twist while preserving the represented physical motion. The following block multiplication shows exactly how its six coordinates change.

Write the matrices as

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^T & 1 \end{bmatrix}, \quad \hat{\boldsymbol{\xi}}_b = \begin{bmatrix} [\boldsymbol{\omega}_b]_{\times} & \boldsymbol{\nu}_b \\ \mathbf{0}_3^T & 0 \end{bmatrix}, \quad \mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T\mathbf{p} \\ \mathbf{0}_3^T & 1 \end{bmatrix},$$

where $\mathbf{R} = \mathbf{R}_{wb} \in SO(3)$ and $\mathbf{p} = {}^w\mathbf{p}_b \in \mathbb{R}^3$. First multiply the two matrices on the left:

$$\mathbf{T}\hat{\boldsymbol{\xi}}_b = \begin{bmatrix} \mathbf{R}[\boldsymbol{\omega}_b]_{\times} & \mathbf{R}\boldsymbol{\nu}_b \\ \mathbf{0}_3^T & 0 \end{bmatrix}.$$

Multiplying this result by $\mathbf{T}^{-1}$ gives

$$\begin{aligned} \mathbf{T}\hat{\boldsymbol{\xi}}_b\mathbf{T}^{-1} &= \begin{bmatrix} \mathbf{R}[\boldsymbol{\omega}_b]_{\times} & \mathbf{R}\boldsymbol{\nu}_b \\ \mathbf{0}_3^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T\mathbf{p} \\ \mathbf{0}_3^T & 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}[\boldsymbol{\omega}_b]_{\times}\mathbf{R}^T & -\mathbf{R}[\boldsymbol{\omega}_b]_{\times}\mathbf{R}^T\mathbf{p} + \mathbf{R}\boldsymbol{\nu}_b \\ \mathbf{0}_3^T & 0 \end{bmatrix}. \end{aligned}$$

The proper-rotation cross-product identity established in Section 7.4.2 gives

$$\mathbf{R}[\boldsymbol{\omega}_b]_{\times}\mathbf{R}^T = [\mathbf{R}\boldsymbol{\omega}_b]_{\times}.$$

The previously established coordinate relation $\boldsymbol{\omega}_w = \mathbf{R}\boldsymbol{\omega}_b$ then gives

$$\boxed{\mathbf{R}[\boldsymbol{\omega}_b]_{\times}\mathbf{R}^T = [\boldsymbol{\omega}_w]_{\times}}.$$

No new angular-velocity relation is being derived in this step. The proper-rotation identity only confirms that conjugating the body-coordinate skew block produces the angular block required by $\hat{\boldsymbol{\xi}}_w$.

The conjugated upper-right block becomes

$$\begin{aligned} & -\mathbf{R}[\boldsymbol{\omega}_b]_{\times}\mathbf{R}^T\mathbf{p} + \mathbf{R}\boldsymbol{\nu}_b \\ &= -[\boldsymbol{\omega}_w]_{\times}\mathbf{p} + \mathbf{R}\boldsymbol{\nu}_b \\ &= \mathbf{R}\boldsymbol{\nu}_b - \boldsymbol{\omega}_w \times \mathbf{p} \\ &= \boldsymbol{\nu}_w, \end{aligned}$$

where the final equality is the direct linear relation derived at the start of this subsection. Therefore, the complete conjugated matrix is

$$\mathbf{T}\hat{\boldsymbol{\xi}}_b\mathbf{T}^{-1} = \begin{bmatrix} [\boldsymbol{\omega}_w]_{\times} & \boldsymbol{\nu}_w \\ \mathbf{0}_3^T & 0 \end{bmatrix} = \hat{\boldsymbol{\xi}}_w.$$

The component calculation and the conjugation calculation therefore agree: both convert the body-referenced description of one instantaneous motion into its world-referenced description.

The adjoint matrix

Collect the linear and angular relations using the declared linear-first twist order:

$$\begin{aligned}\xi_w &= \begin{bmatrix} \nu_w \\ \omega_w \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R} & [\mathbf{p}]_{\times} \mathbf{R} \\ \mathbf{0}_{3 \times 3} & \mathbf{R} \end{bmatrix} \begin{bmatrix} \nu_b \\ \omega_b \end{bmatrix}.\end{aligned}$$

The 6×6 matrix in this equation is called the **adjoint representation** of $\mathbf{T}$. The word *representation* means that the finite pose $\mathbf{T} \in SE(3)$ is represented by a linear operator acting on six-component twist coordinates. For the component order $[\nu^\top \omega^\top]^\top$ used in this book, define

$$\text{Ad}_{\mathbf{T}} := \begin{bmatrix} \mathbf{R} & [\mathbf{p}]_{\times} \mathbf{R} \\ \mathbf{0}_{3 \times 3} & \mathbf{R} \end{bmatrix} \in \mathbb{R}^{6 \times 6}.$$

Each block is 3×3 . The rotation blocks are dimensionless, whereas $[\mathbf{p}]_{\times} \mathbf{R}$ has units of metres. Multiplying that block by ω_b in rad s^{-1} produces the required m s^{-1} contribution to ν_w . The zero block shows that linear velocity does not contribute to angular velocity, while the upper-right block shows that angular velocity contributes to the linear part whenever the two reference origins differ.

For $\mathbf{T} = \mathbf{T}_{wb}$, the complete conversion is

$$\xi_w = \text{Ad}_{\mathbf{T}_{wb}} \xi_b.$$

Equivalently, the adjoint can be characterised directly by its action on any twist column $\xi \in \mathbb{R}^6$:

$$\widehat{\text{Ad}_{\mathbf{T}} \xi} = \mathbf{T} \hat{\xi} \mathbf{T}^{-1}.$$

Inverse adjoint operator

The forward operator $\text{Ad}_{\mathbf{T}_{wb}} : \mathbb{R}^6 \rightarrow \mathbb{R}^6$ converts body-twist coordinates into spatial-twist coordinates. Recovering the body twist requires the inverse of this conversion. Two similar-looking operators appear:

- $\text{Ad}_{\mathbf{T}_{wb}}^{-1}$ means first form the inverse pose $\mathbf{T}_{wb}^{-1} \in SE(3)$ and then construct the adjoint associated with that pose.

- $\text{Ad}_{T_{wb}}^{-1}$ means first construct the 6×6 adjoint matrix $\text{Ad}_{T_{wb}}$ and then take its ordinary matrix inverse.

These procedures produce the same 6×6 linear operator. To prove this, let $\xi \in \mathbb{R}^6$ be arbitrary and first apply Ad_T , followed by $\text{Ad}_{T^{-1}}$. Applying the hat definition twice gives

$$\begin{aligned} \widehat{\text{Ad}_{T^{-1}}(\text{Ad}_T \xi)} &= T^{-1} \widehat{\text{Ad}_T \xi} T \\ &= T^{-1} (T \hat{\xi} T^{-1}) T \\ &= \hat{\xi}. \end{aligned}$$

The hat map is one-to-one: each twist column has exactly one hatted matrix in $\mathfrak{se}(3)$, and each matrix in $\mathfrak{se}(3)$ determines exactly one twist column. The equality of the hatted matrices therefore implies

$$\text{Ad}_{T^{-1}}(\text{Ad}_T \xi) = \xi$$

for every $\xi \in \mathbb{R}^6$. Hence, the product of the two adjoint matrices is the identity operator on $\mathbb{R}^6$:

$$\text{Ad}_{T^{-1}} \text{Ad}_T = \mathbf{I}_6.$$

Repeating the same calculation in the opposite order, equivalently replacing T by T^{-1} , gives

$$\text{Ad}_T \text{Ad}_{T^{-1}} = \mathbf{I}_6.$$

Thus, $\text{Ad}_{T^{-1}}$ is both a left and a right inverse of Ad_T . By the definition of a matrix inverse,

$$\boxed{\text{Ad}_{T^{-1}} = \text{Ad}_T^{-1}}.$$

Setting $T = T_{wb}$ and applying this inverse operator to ξ_w gives the inverse twist-coordinate conversion:

$$\boxed{\xi_b = \text{Ad}_{T_{wb}^{-1}} \xi_w = \text{Ad}_{T_{wb}}^{-1} \xi_w}.$$

Physically, T_{wb}^{-1} reverses the pose-based change of reference from the body frame to the world frame. Its adjoint therefore reverses the corresponding change from body-twist coordinates to spatial-twist coordinates.

If the two frame origins coincide, then $\mathbf{p} = \mathbf{0}_3$ and the cross-product block vanishes. In that special case, both the linear and angular parts change only by rotation. In general, a twist is not a pair of free vectors: changing its reference frame requires both rotation and the position-dependent correction encoded by the adjoint.

Quick test I: spatial rigid-body velocity and twists

1. **Recall:** State the world-frame velocity of a body-fixed point P in terms of the body-origin velocity, angular velocity, and relative-position vector.
2. **Contrast:** Define the body- and spatial-twist hat matrices from $\mathbf{T}_{wb}$ and $\dot{\mathbf{T}}_{wb}$. Which side of the pose does each twist matrix multiply when reconstructing $\dot{\mathbf{T}}_{wb}$?
3. **Explanation:** What physical quantity does $\boldsymbol{\nu}_b$ describe? What does $\boldsymbol{\nu}_w$ describe, and why is $\boldsymbol{\nu}_w$ generally not the velocity of the body origin?
4. **Application:** At one instant, let $\mathbf{p} = [1 \ 0 \ 0]^T \text{ m}$, $\dot{\mathbf{p}} = [0 \ 1 \ 0]^T \text{ m s}^{-1}$, and $\boldsymbol{\omega}_w = [0 \ 0 \ 2]^T \text{ rad s}^{-1}$. Calculate $\boldsymbol{\nu}_w$. Then use the spatial-twist velocity field to calculate the velocity of a body-fixed point currently at ${}^w\mathbf{p}_P = [1 \ 1 \ 0]^T \text{ m}$.
5. **Derivation:** Starting from $\boldsymbol{\omega}_w = \mathbf{R}\boldsymbol{\omega}_b$ and $\boldsymbol{\nu}_w = \mathbf{R}\boldsymbol{\nu}_b + [\mathbf{p}]_{\times}\mathbf{R}\boldsymbol{\omega}_b$, stack the equations to derive $\hat{\boldsymbol{\xi}}_w = \text{Ad}_{\mathbf{T}}\hat{\boldsymbol{\xi}}_b$ for the linear-first component order.
6. **Reference origin:** Under what condition does the adjoint reduce to rotating the linear and angular parts separately? Why does the position-dependent block vanish in that case?
7. **Inverse conversion:** Explain the difference between $\text{Ad}_{\mathbf{T}^{-1}}$ and $\text{Ad}_{\mathbf{T}}^{-1}$. Why are they equal?
8. **Common error:** Why is the rule $\boldsymbol{\nu}_w = \mathbf{R}\boldsymbol{\nu}_b$ generally insufficient for converting the linear part of a twist, even though $\boldsymbol{\omega}_w = \mathbf{R}\boldsymbol{\omega}_b$ is sufficient for the angular part?

Answers and hints

1. With all quantities expressed in world coordinates,

$${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \boldsymbol{\omega}_w \times {}^w\mathbf{r}_{O_bP}.$$

The first term is the translational velocity at the body origin. The second is the additional velocity caused by the point's displacement from that origin while the body rotates.

2. The definitions and reconstruction equations are

$$\hat{\boldsymbol{\xi}}_b = \mathbf{T}_{wb}^{-1}\dot{\mathbf{T}}_{wb}, \quad \dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb}\hat{\boldsymbol{\xi}}_b,$$

and

$$\hat{\xi}_w = \dot{T}_{wb} T_{wb}^{-1}, \quad \dot{T}_{wb} = \hat{\xi}_w T_{wb}.$$

The body-twist matrix multiplies the pose on the right, whereas the spatial-twist matrix multiplies it on the left.

3. The column $\nu_b = R_{wb}^\top {}^w \mathbf{v}_b$ is the velocity of the body origin relative to the world, expressed along the body axes. The column ν_w is the value of the instantaneous rigid-motion velocity field at the world-origin location, expressed along the world axes. They satisfy

$${}^w \mathbf{v}_b = \nu_w + \omega_w \times \mathbf{p},$$

so they differ whenever the rotational contribution between O_w and O_b is nonzero.

4. First,

$$\omega_w \times \mathbf{p} = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Therefore,

$$\nu_w = \dot{\mathbf{p}} - \omega_w \times \mathbf{p} = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

At the declared point,

$$\begin{aligned} {}^w \mathbf{v}_P &= \nu_w + \omega_w \times {}^w \mathbf{p}_P \\ &= \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} + \begin{bmatrix} -2 \\ 2 \\ 0 \end{bmatrix} \\ &= \boxed{\begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} \text{ m s}^{-1}}. \end{aligned}$$

5. Using $\xi = [\nu^\top \omega^\top]^\top$,

$$\begin{aligned}\xi_w &= \begin{bmatrix} \nu_w \\ \omega_w \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}\nu_b + [\mathbf{p}]_\times \mathbf{R}\omega_b \\ \mathbf{R}\omega_b \end{bmatrix} \\ &= \underbrace{\begin{bmatrix} \mathbf{R} & [\mathbf{p}]_\times \mathbf{R} \\ \mathbf{0}_{3 \times 3} & \mathbf{R} \end{bmatrix}}_{\text{Ad}_T} \begin{bmatrix} \nu_b \\ \omega_b \end{bmatrix}.\end{aligned}$$

6. If the source and target frame origins coincide, then $\mathbf{p} = \mathbf{0}_3$. Consequently, $[\mathbf{p}]_\times = \mathbf{0}_{3 \times 3}$ and

$$\text{Ad}_T = \begin{bmatrix} \mathbf{R} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{R} \end{bmatrix}.$$

No reference-origin shift remains, so both parts require only a change of coordinate axes.

7. The expression $\text{Ad}_{T^{-1}}$ means invert the pose first and then construct its adjoint. The expression Ad_T^{-1} means construct the 6×6 adjoint matrix first and then invert that matrix. Applying the two maps in succession gives

$$\text{Ad}_{T^{-1}} \text{Ad}_T = \text{Ad}_T \text{Ad}_{T^{-1}} = \mathbf{I}_6,$$

because conjugation by $\mathbf{T}$ followed by conjugation by $\mathbf{T}^{-1}$ restores the original hatted twist. Therefore, $\text{Ad}_{T^{-1}} = \text{Ad}_T^{-1}$.

8. Angular velocity is the same at every point of a rigid body, so changing its coordinate frame requires only $\omega_w = \mathbf{R}\omega_b$. The linear part of a twist is tied to a reference location as well as a coordinate basis. Changing from the body origin to the world origin contributes

$$[\mathbf{p}]_\times \mathbf{R}\omega_b = \mathbf{p} \times \omega_w,$$

so the complete relation is $\nu_w = \mathbf{R}\nu_b + [\mathbf{p}]_\times \mathbf{R}\omega_b$.

7.10 Representation limitations and numerical checks

The preceding sections describe exact mathematical objects. Software stores their components as finite-precision numbers, so round-off, an incorrect convention, or an invalid input can produce an array that no longer represents the intended geometry. Validation must therefore check the defining constraints of each representation rather than merely checking its array shape.

7.10.1 Equivalent information does not make representations interchangeable

A physical orientation can be represented by a rotation matrix, an axis–angle pair, or a unit quaternion. A physical pose can likewise be represented by a homogeneous transformation or by a chosen coordinate column. These representations may encode equivalent geometric information, but they have different mathematical types, constraints, and composition rules.

For example, $\mathbf{R}_{wb} \in SO(3)$ contains nine entries constrained by orthogonality and determinant conditions, whereas a unit quaternion contains four components constrained by one unit-norm equation. The two unit quaternions $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation, so comparing their component columns directly can report a large difference even when their represented orientations are identical.

Frame and storage conventions are also part of the data’s meaning. An implementation must state whether a rotation maps body coordinates to world coordinates or the reverse, whether an operation is active or passive, whether a quaternion is stored in scalar-first or scalar-last order, and whether a twist is body or spatial. This chapter uses the twist component order

$$\xi = \begin{bmatrix} \nu \\ \omega \end{bmatrix} \in \mathbb{R}^6,$$

with the three linear components first and the three angular components second. Reordering these six numbers without changing the associated formulas changes the represented motion.

7.10.2 Checking rotation matrices, transformations, and quaternions

Before checking a geometric constraint, first verify that every stored component is finite. A component equal to NaN or $\pm\infty$ does not represent a valid physical quantity and makes later residual tests unreliable.

For a real matrix $\mathbf{M} = [m_{ij}] \in \mathbb{R}^{m \times n}$, define the **Frobenius norm**

$$\|\mathbf{M}\|_{\text{F}} := \sqrt{\sum_{i=1}^m \sum_{j=1}^n m_{ij}^2}.$$

It measures the combined size of all matrix entries. For a candidate rotation matrix $\tilde{\mathbf{R}} \in \mathbb{R}^{3 \times 3}$, useful dimensionless residuals are

$$e_{\text{orth}} := \|\tilde{\mathbf{R}}^{\text{T}} \tilde{\mathbf{R}} - \mathbf{I}_3\|_{\text{F}}, \quad e_{\text{det}} := |\det(\tilde{\mathbf{R}}) - 1|.$$

The first residual checks whether the columns are mutually perpendicular unit vectors. The second distinguishes a proper rotation from a reflection. For example, $\text{diag}(1, 1, -1)$ has orthonormal columns but determinant -1 , so its orthogonality residual is zero while its determinant residual is two. An implementation accepts the matrix only when both residuals are below declared dimensionless tolerances.

A candidate homogeneous transformation $\tilde{\mathbf{T}} \in \mathbb{R}^{4 \times 4}$ must have the structure

$$\tilde{\mathbf{T}} = \begin{bmatrix} \tilde{\mathbf{R}} & \tilde{\mathbf{p}} \\ \mathbf{0}_3^{\text{T}} & 1 \end{bmatrix},$$

where $\tilde{\mathbf{R}}$ passes the rotation checks, $\tilde{\mathbf{p}} \in \mathbb{R}^3$ is a finite translation column measured in metres, and the bottom row equals $[0 \ 0 \ 0 \ 1]$. Rotation residuals are dimensionless, whereas translation errors have units of metres; they should not be combined into one unscaled residual.

For a quaternion component column $\tilde{\mathbf{c}} = [q_w \ q_x \ q_y \ q_z]^{\text{T}} \in \mathbb{R}^4$, the unit-constraint residual is

$$e_q := \left| \|\tilde{\mathbf{c}}\|_2 - 1 \right|.$$

The quaternion section in Section 7.8 explains when a small norm error may be corrected by normalisation. Validation and repair are different operations: validation reports whether the input satisfies a requirement, while repair changes the input according to an explicit policy. A zero, near-zero, or non-finite quaternion cannot be repaired safely by division, and an arbitrary invalid matrix should not be silently treated as a rotation.

7.10.3 Checking twists and their units

A twist must have six finite components, the declared component order, a declared expression frame, and a declared reference origin. Its linear part $\boldsymbol{\nu} \in \mathbb{R}^3$ is measured in m s^{-1} , while its angular part $\boldsymbol{\omega} \in \mathbb{R}^3$ is measured in rad s^{-1} . The associated matrix must also have the structure

$$\hat{\boldsymbol{\xi}} = \begin{bmatrix} [\boldsymbol{\omega}]_{\times} & \boldsymbol{\nu} \\ \mathbf{0}_3^T & 0 \end{bmatrix},$$

so its upper-left block is skew-symmetric and its bottom row is zero.

An ordinary Euclidean expression such as

$$\|\boldsymbol{\xi}\|_2^2 = \|\boldsymbol{\nu}\|_2^2 + \|\boldsymbol{\omega}\|_2^2$$

adds squared quantities with different physical units and therefore is **not**, by itself, a meaningful physical magnitude. If an application needs one scalar for comparison, it must choose a characteristic length $\ell > 0$ measured in metres and may define

$$\|\boldsymbol{\xi}\|_{\ell} := \sqrt{\|\boldsymbol{\nu}\|_2^2 + \ell^2 \|\boldsymbol{\omega}\|_2^2}.$$

This quantity has units of m s^{-1} . The product $\ell \|\boldsymbol{\omega}\|_2$ is the tangential-speed scale associated with rotation at distance ℓ from the reference axis. Different choices of ℓ express different engineering priorities, so this weighted magnitude is not a universal property of the rigid-body motion.

7.10.4 Round-trip checks and implementation consequences

For numerical tests, write $\mathbf{A} \approx \mathbf{B}$ when a declared residual between the two arrays is below a chosen tolerance. The tolerance must account for the quantity's units, expected numerical scale, and floating-point precision. Useful round-trip and consistency checks include

$$\begin{aligned} \mathbf{R}^T \mathbf{R} &\approx \mathbf{I}_3, & \mathbf{T} \mathbf{T}^{-1} &\approx \mathbf{I}_4, \\ \text{Ad}_{\mathbf{T}^{-1}} \text{Ad}_{\mathbf{T}} &\approx \mathbf{I}_6, & \widehat{\text{Ad}_{\mathbf{T}} \boldsymbol{\xi}} &\approx \mathbf{T} \hat{\boldsymbol{\xi}} \mathbf{T}^{-1}. \end{aligned}$$

The first check tests a rotation constraint. The second tests the transformation inverse. The third tests the forward and inverse twist-coordinate mappings. The fourth checks

that the coordinate adjoint and matrix-conjugation definitions describe the same operation. A point transformation can also be checked by mapping body coordinates to world coordinates and then applying $\mathbf{T}^{-1}$ to recover the original body coordinates.

The body-expressed orientation-rate equation derived in Section 7.4 is

$$\dot{\mathbf{R}}_{wb} = \mathbf{R}_{wb} [\boldsymbol{\omega}_b]_{\times}.$$

For a numerical step beginning at time t_k , define

$$\mathbf{R}_k := \mathbf{R}_{wb}(t_k), \quad \boldsymbol{\omega}_{b,k} := \boldsymbol{\omega}_b(t_k), \quad \boldsymbol{\Omega}_k := [\boldsymbol{\omega}_{b,k}]_{\times}.$$

Evaluating the orientation-rate equation at t_k gives

$$\dot{\mathbf{R}}_{wb}(t_k) = \mathbf{R}_k \boldsymbol{\Omega}_k.$$

A direct Forward Euler update illustrates why preserving the representation constraints matters. Forward Euler replaces the next orientation by the current orientation plus the time step multiplied by the current orientation rate:

$$\tilde{\mathbf{R}}_{k+1} := \mathbf{R}_k + \Delta t \dot{\mathbf{R}}_{wb}(t_k) = \mathbf{R}_k + \Delta t \mathbf{R}_k \boldsymbol{\Omega}_k = \mathbf{R}_k (\mathbf{I}_3 + \Delta t \boldsymbol{\Omega}_k).$$

Because $\boldsymbol{\Omega}_k^T = -\boldsymbol{\Omega}_k$ and $\mathbf{R}_k^T \mathbf{R}_k = \mathbf{I}_3$,

$$\begin{aligned} \tilde{\mathbf{R}}_{k+1}^T \tilde{\mathbf{R}}_{k+1} &= (\mathbf{I}_3 - \Delta t \boldsymbol{\Omega}_k) (\mathbf{I}_3 + \Delta t \boldsymbol{\Omega}_k) \\ &= \mathbf{I}_3 - \Delta t^2 \boldsymbol{\Omega}_k^2, \end{aligned}$$

which is generally not $\mathbf{I}_3$. The one-step orthogonality error is of order $O(\Delta t^2)$, so repeated raw matrix updates can drift away from $SO(3)$.

The exact finite-rotation factor is available when the body-expressed angular velocity remains constant throughout the interval. The body-expressed update from Section 7.5.6 becomes

$$\mathbf{R}_{k+1} = \mathbf{R}_k \underbrace{\exp(\Delta t \boldsymbol{\Omega}_k)}_{\text{exact finite-rotation factor}}.$$

The matrix exponential is a valid element of $SO(3)$. Its series begins

$$\exp(\Delta t \boldsymbol{\Omega}_k) = \mathbf{I}_3 + \Delta t \boldsymbol{\Omega}_k + O(\Delta t^2).$$

Forward Euler keeps only the first two terms and therefore replaces the exact factor $\exp(\Delta t \mathbf{\Omega}_k)$ by $\mathbf{I}_3 + \Delta t \mathbf{\Omega}_k$. A production implementation must use a declared structure-preserving update or a documented correction policy. The following chapter develops finite rigid-body motion and integration methods; the present result establishes the requirement they must satisfy.

The principal implementation rules from this chapter are therefore:

- store frames, units, component order, and active or passive meaning with the interface contract;
- use the transpose as a validated rotation's inverse and the block formula from Section 7.6.2 as a validated homogeneous transformation's inverse;
- compose transformations in the physical order of the motions rather than adding pose components;
- reject non-finite values and geometric-constraint violations deliberately;
- normalise quaternions only under a stated tolerance and failure policy; and
- test representation invariants, inverse round trips, and equivalent conversion paths.

Cumulative chapter review

1. What is the difference among a rigid body's physical configuration, a homogeneous transformation $\mathbf{T}_{wb}$, and a coordinate column such as $[x \ y \ \theta]^T$?
2. For constraints $\mathbf{g}(\mathbf{z}) = \mathbf{0}$ at a regular configuration $\mathbf{z}_0 \in \mathbb{R}^{n_v}$, state the Jacobian null-space description of allowable infinitesimal changes and the corresponding DOF formula. Why does a free spatial rigid body have six DOF?
3. State the defining equations of $SO(3)$ and explain the geometric meaning of the columns of $\mathbf{R}_{wb}$.
4. If $\mathbf{Q}_w$ is an orientation change expressed along fixed world axes and $\mathbf{Q}_b$ is an orientation change expressed along the old body axes, how does each update $\mathbf{R}_{wb}$?
5. Give the world- and body-expressed angular-velocity equations for $\dot{\mathbf{R}}_{wb}$. Why do the two angular-velocity columns describe the same physical turning?
6. Let $\mathbf{a} \in \mathbb{R}^3$ be a unit rotation axis and $\alpha \in \mathbb{R}$ an angle. State the exponential-coordinate vector, its matrix exponential, and Rodrigues' formula.
7. State how $\mathbf{T}_{wb} \in SE(3)$ transforms a body-frame point coordinate into world coordinates, and derive the inverse transformation.
8. What constraints identify the planar-motion subgroup embedded in $SE(3)$, and how is planar yaw recovered from its rotation matrix?
9. Write the unit quaternion for an axis-angle rotation. Why do $\mathbf{q}$ and $-\mathbf{q}$ represent the same orientation, and what is the Hamilton-product order when $\mathbf{q}_1$ acts first and $\mathbf{q}_2$ acts second?
10. Distinguish a pose rate $\dot{\mathbf{T}}_{wb}$, a body twist ξ_b , and a spatial twist ξ_w . What physical quantities do their linear parts represent?
11. Let $\mathbf{R} = \mathbf{R}_z(\pi/2)$, $\mathbf{p} = [1 \ 2 \ 3]^T \text{ m}$, $\nu_b = [1 \ 0 \ 0]^T \text{ m s}^{-1}$, and $\omega_b = [0 \ 0 \ 2]^T \text{ rad s}^{-1}$. Use the adjoint relation to calculate ω_w and ν_w .

12. Why must software check both orthogonality and determinant for a candidate rotation matrix? Why is an ordinary Euclidean norm of a twist not a physically meaningful magnitude?
13. Show why the Forward Euler orientation update $\tilde{\mathbf{R}}_{k+1} = \mathbf{R}_k(\mathbf{I}_3 + \Delta t[\boldsymbol{\omega}_b]_{\times})$ generally leaves $SO(3)$.

Answers and hints

1. The configuration is the body's physical placement relative to the fixed world. The matrix $\mathbf{T}_{wb}$ represents that placement as an element of $SE(3)$ and maps body-frame point coordinates to world-frame coordinates. A coordinate column is a chosen list of numbers used to construct a representation; it is not itself the physical body or an ordinary free vector.
2. Linearising the constraints gives

$$\mathbf{J}_g(\mathbf{z}_0)\delta\mathbf{z} = \mathbf{0},$$

so the allowable infinitesimal changes form

$$\mathcal{N}(\mathbf{J}_g(\mathbf{z}_0)) = \{\delta\mathbf{z} \in \mathbb{R}^{n_v} \mid \mathbf{J}_g(\mathbf{z}_0)\delta\mathbf{z} = \mathbf{0}\}.$$

Therefore,

$$n_{\text{dof}} = \dim \mathcal{N}(\mathbf{J}_g(\mathbf{z}_0)) = n_v - \text{rank}(\mathbf{J}_g(\mathbf{z}_0)).$$

A free spatial rigid body has three independent translation coordinates and three independent local orientation coordinates, giving six DOF.

3. A proper rotation satisfies

$$\mathbf{R}^T \mathbf{R} = \mathbf{I}_3, \quad \det(\mathbf{R}) = 1.$$

The columns of $\mathbf{R}_{wb}$ are the world-coordinate columns of the body axes x_b , y_b , and z_b . They form a right-handed orthonormal basis.

4. A world-expressed change premultiplies, whereas a body-expressed change post-multiplies:

$$\boxed{\mathbf{R}_{wb'} = \mathbf{Q}_w \mathbf{R}_{wb}}, \quad \boxed{\mathbf{R}_{wb'} = \mathbf{R}_{wb} \mathbf{Q}_b}.$$

5. The two rate equations are

$$\boxed{\dot{\mathbf{R}}_{wb} = [\boldsymbol{\omega}_w]_{\times} \mathbf{R}_{wb} = \mathbf{R}_{wb} [\boldsymbol{\omega}_b]_{\times}}, \quad \boldsymbol{\omega}_b = \mathbf{R}_{wb}^T \boldsymbol{\omega}_w.$$

The physical turning is the same; multiplication by $\mathbf{R}_{wb}^T$ only resolves its components along the body axes instead of the world axes.

6. Define

$$\boldsymbol{\varphi} := \alpha \mathbf{a}, \quad [\boldsymbol{\varphi}]_{\times} = \alpha [\mathbf{a}]_{\times}.$$

The finite rotation is

$$\mathbf{Q} = \exp([\boldsymbol{\varphi}]_{\times}) = \mathbf{I}_3 + \sin \alpha [\mathbf{a}]_{\times} + (1 - \cos \alpha) [\mathbf{a}]_{\times}^2.$$

Applied to $\mathbf{y} \in \mathbb{R}^3$, Rodrigues' formula is

$$\mathbf{Q}\mathbf{y} = \cos \alpha \mathbf{y} + (1 - \cos \alpha) \mathbf{a}(\mathbf{a}^T \mathbf{y}) + \sin \alpha (\mathbf{a} \times \mathbf{y}).$$

7. With

$$\mathbf{T}_{wb} = \begin{bmatrix} \mathbf{R}_{wb} & {}^w \mathbf{p}_b \\ \mathbf{0}_3^T & 1 \end{bmatrix},$$

point coordinates satisfy

$${}^w \mathbf{p}_P = \mathbf{R}_{wb} {}^b \mathbf{p}_P + {}^w \mathbf{p}_b.$$

Rearranging and using $\mathbf{R}_{wb}^{-1} = \mathbf{R}_{wb}^T$ gives

$${}^b \mathbf{p}_P = \mathbf{R}_{wb}^T ({}^w \mathbf{p}_P - {}^w \mathbf{p}_b),$$

and therefore

$$\mathbf{T}_{wb}^{-1} = \begin{bmatrix} \mathbf{R}_{wb}^T & -\mathbf{R}_{wb}^T {}^w \mathbf{p}_b \\ \mathbf{0}_3^T & 1 \end{bmatrix}.$$

8. Using $\mathbf{e}_z = [0 \ 0 \ 1]^T$, planar motion satisfies

$$\mathbf{R} \mathbf{e}_z = \mathbf{e}_z, \quad \mathbf{e}_z^T \mathbf{p} = 0.$$

For the embedded yaw rotation, $\theta = \text{atan2}(R_{21}, R_{11})$.

9. For a unit axis $\mathbf{a}$,

$$\mathbf{q} = \left(\cos \frac{\alpha}{2}, \mathbf{a} \sin \frac{\alpha}{2} \right).$$

Negating both parts does not change the quaternion sandwich product $\mathbf{q} \otimes \mathbf{y} \otimes \mathbf{q}^*$, so $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation. If $\mathbf{q}_1$ acts first and $\mathbf{q}_2$ acts second, then $\mathbf{q}_{\text{total}} = \mathbf{q}_2 \otimes \mathbf{q}_1$.

10. The pose rate $\dot{\mathbf{T}}_{wb}$ is the entrywise derivative of a pose matrix and depends on the current pose. The body and spatial twists are obtained by

$$\hat{\xi}_b = \mathbf{T}_{wb}^{-1} \dot{\mathbf{T}}_{wb}, \quad \hat{\xi}_w = \dot{\mathbf{T}}_{wb} \mathbf{T}_{wb}^{-1}.$$

The body linear part is the world-relative velocity of the body origin expressed along body axes:

$$\boldsymbol{\nu}_b = \mathbf{R}_{wb}^\top {}^w \mathbf{v}_b.$$

The spatial linear part is the value of the rigid-motion velocity field at the world-origin location:

$$\boldsymbol{\nu}_w = {}^w \mathbf{v}_b - \boldsymbol{\omega}_w \times \mathbf{p}.$$

11. Rotation about z leaves the angular-velocity column unchanged and maps the body x -axis to the world y -axis:

$$\boldsymbol{\omega}_w = \mathbf{R} \boldsymbol{\omega}_b = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \text{ rad s}^{-1}, \quad \mathbf{R} \boldsymbol{\nu}_b = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

The origin-shift term is

$$[\mathbf{p}]_{\times} \boldsymbol{\omega}_w = \mathbf{p} \times \boldsymbol{\omega}_w = \begin{bmatrix} 4 \\ -2 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Hence,

$$\boxed{\boldsymbol{\nu}_w = \begin{bmatrix} 4 \\ -1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

12. Orthogonality alone admits both proper rotations and reflections. The determinant check rejects the reflection case by requiring $\det(\mathbf{R}) = 1$. A twist's linear and angular components have different units, so their squared Euclidean norms cannot be added as a physical magnitude without first choosing a length scale that converts angular rate to a linear-speed scale.
13. Let $\mathbf{\Omega} = [\boldsymbol{\omega}_b]_{\times}$. Then

$$\begin{aligned}\tilde{\mathbf{R}}_{k+1}^{\top} \tilde{\mathbf{R}}_{k+1} &= (\mathbf{I}_3 - \Delta t \mathbf{\Omega}) \mathbf{R}_k^{\top} \mathbf{R}_k (\mathbf{I}_3 + \Delta t \mathbf{\Omega}) \\ &= \mathbf{I}_3 - \Delta t^2 \mathbf{\Omega}^2.\end{aligned}$$

This is generally not $\mathbf{I}_3$, so the updated matrix is generally not in $SO(3)$ even though its one-step constraint error is $O(\Delta t^2)$.

References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapter 2 for configuration space and degrees of freedom, and Chapter 3, especially Sections 3.2.1-3.2.3 on spatial rotation matrices, angular velocity, exponential coordinates, and Rodrigues' formula, Section 3.3.1 on homogeneous transformation matrices, and Section 3.3.2 on spatial rigid-body velocities and twists.
- John J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Chapters 1-2, especially degrees of freedom, body-attached frames, proper orthonormal rotation matrices, spatial orientation, homogeneous transformations, equivalent angle-axis representations, and Euler parameters.
- Peter Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, 3rd ed., Chapters 2-3, especially rotation matrices, the matrix exponential, Rodrigues' formula, unit quaternions, rotational velocity, and body- and world-frame expressions of spatial motion.
- ROS 2 Jazzy `geometry_msgs`, [Quaternion message definition](#), for the `x`, `y`, `z`, and `w` field order.

8 Screw Motion, the $SE(3)$ Exponential, and Spatial Pose Integration

Prerequisites and assumptions

The reader should understand vectors, matrices, rank, null spaces, and constraints from Chapter 1; coordinate frames and cross products from Chapter 2; and spatial poses, angular velocity, twists, the adjoint, and the project's linear-first twist order from Chapter 7.

Physical space is modelled as three-dimensional Euclidean space. Frames are right-handed and orthonormal. Position and length are measured in metres, time in seconds, and angles in radians. Unless stated otherwise, every screw-axis coordinate column is referred to one declared fixed frame: that frame supplies both the coordinate axes and the reference origin of the associated spatial-twist velocity field.

8.1 Robot configurations, poses, and joint coordinates

This chapter develops the spatial geometry needed to represent the motion permitted by one robot joint. It begins by reviewing configuration and pose for rigid links connected by joints, then introduces joint coordinates, screw-axis representations, finite rigid displacements through the $SE(3)$ exponential and logarithm, and constant-twist pose integration.

The chapter treats ideal rigid links and one-degree-of-freedom revolute or prismatic joints. Finite-pitch helical motion is introduced to explain the general screw model, but it is not included as a supported robot-joint type. Composing several joints into a serial-chain forward map and differentiating that map to obtain robot Jacobians belong to Chapter 10. This chapter does not model joint compliance, backlash, closed kinematic loops, collision, forces, actuator dynamics, or feedback control.

Configuration and pose for links and joints

An open serial robot consists of rigid links connected in sequence by joints. Each one-degree-of-freedom joint restricts the relative motion of two neighbouring links to one independent rotation or translation. Predicting the robot's geometry therefore requires more

than locating one free rigid body: it requires determining the simultaneous placement of every link while satisfying every joint constraint.

A **material point** is an ideal labelled point fixed in one physical part of the robot. Its label identifies the same point as the robot moves; its coordinate column may change.

A **link** is a collection of material points modelled as moving together as one rigid object. The distance between every pair of its material points remains fixed. A link need not look like a straight bar: a base, chassis, bracket, shell, or several components bolted together may be one link when their relative motion and deformation are neglected.

For example, a simple arm has the sequence

base link — shoulder joint — upper-arm link — elbow joint — forearm link.

The **upper-arm link** is the rigid assembly that moves as one object. The **forearm link** is another rigid assembly. The **elbow joint** allows the forearm link to rotate relative to the upper-arm link.

Let I be the finite set of link indices. For every $i \in I$, let L_i denote link i and let $\mathcal{B}_i$ be the set of labelled material points modelled as belonging to L_i . The robot's complete material-point set is

$$\mathcal{B} = \bigsqcup_{i \in I} \mathcal{B}_i,$$

where $\bigsqcup$ denotes a **disjoint union**: every point retains its link identity. A joint is not another material-point set; it constrains the allowable relative placement of two links. Physical joint components are assigned to those links, represented as additional links when their relative motion matters, or omitted from the ideal model.

A **configuration** is one complete geometric arrangement of the modelled robot at an instant. Reusing the placement-map definition from Section 7.1, a configuration χ assigns a world position to every material point in $\mathcal{B}$:

$$\chi : \mathcal{B} \rightarrow \mathbb{R}^3, \quad P \mapsto \chi(P).$$

Each value $\chi(P) \in \mathbb{R}^3$ is the position of material point P relative to the fixed world, expressed in world coordinates and measured in metres. The configuration is the complete assignment χ , not one material point and not only the end effector. Rigid-link preservation means that, for every link index $i \in I$ and every pair $P, Q \in \mathcal{B}_i$,

$$\|\chi(P) - \chi(Q)\|_2 = d_i(P, Q),$$

where $d_i(P, Q) \in \mathbb{R}_{\geq 0}$ is the fixed reference distance between the two labelled material points, measured in metres. Joint constraints impose additional relationships between points or attached frames belonging to different links. The **configuration space** $\mathcal{C}$ is therefore

$$\mathcal{C} := \left\{ \chi : \mathcal{B} \rightarrow \mathbb{R}^3 \mid \begin{array}{l} \|\chi(P) - \chi(Q)\|_2 = d_i(P, Q) \text{ for every } i \in I \text{ and every } P, Q \in \mathcal{B}_i, \\ \chi \text{ satisfies every declared joint constraint} \end{array} \right\}.$$

The vertical bar means “such that.” The set begins with every complete placement map $\chi : \mathcal{B} \rightarrow \mathbb{R}^3$ and retains only the maps that preserve every rigid link and obey every declared joint constraint.

The relationship among a configuration map, a pose, and a transformation matrix is clearest for one rigid link. Choose one link with material-point set $\mathcal{B}_b \subseteq \mathcal{B}$, and attach frame $\{b\}$ rigidly to it. Before placing the link in the world, assign every material point its constant body-frame coordinate:

$$c_b : \mathcal{B}_b \rightarrow \mathbb{R}^3, \quad P \mapsto c_b(P) := {}^b\mathbf{p}_P.$$

The **body-coordinate assignment** c_b is fixed model data: it describes the link’s reference geometry in frame $\{b\}$. It is not a configuration because its output is a body-frame coordinate column rather than the point’s current world position, and it does not change when the link moves.

The **pose of frame $\{b\}$ relative to frame $\{w\}$** is the physical position and orientation of the attached body frame relative to the fixed world frame. One representation of this pose is the ordered pair

$$(\mathbf{R}_{wb}, {}^w\mathbf{p}_b) \in SO(3) \times \mathbb{R}^3,$$

where the dimensionless rotation matrix $\mathbf{R}_{wb} \in SO(3)$ represents the orientation of $\{b\}$ relative to $\{w\}$ and maps body-frame vector coordinates to world-frame vector coordinates, and ${}^w\mathbf{p}_b \in \mathbb{R}^3$ is the position of the body-frame origin relative to the world-frame origin, expressed in world coordinates and measured in metres.

The same pose is represented by the homogeneous transformation matrix

$$\mathbf{T}_{wb} := \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3).$$

Here, $\mathbf{0}_3 := [0 \ 0 \ 0]^\top \in \mathbb{R}^3$ is the three-component zero column. Thus it is acceptable to say that the pose is represented by $(\mathbf{R}_{wb}, {}^w\mathbf{p}_b)$ or, equivalently, by $\mathbf{T}_{wb}$. The physical

pose and either coordinate representation should still be distinguished. To distinguish the matrix $\mathbf{T}_{wb}$ from the coordinate mapping that it induces, define

$$A_{\mathbf{T}_{wb}} : \mathbb{R}^3 \rightarrow \mathbb{R}^3, \quad {}^b\mathbf{p} \mapsto A_{\mathbf{T}_{wb}}({}^b\mathbf{p}) := \mathbf{R}_{wb} {}^b\mathbf{p} + {}^w\mathbf{p}_b.$$

Both the input ${}^b\mathbf{p}$ and output $A_{\mathbf{T}_{wb}}({}^b\mathbf{p})$ are point-coordinate columns in metres. For a point, define its homogeneous-coordinate column by ${}^b\bar{\mathbf{p}} := [({}^b\mathbf{p})^\top, 1]^\top \in \mathbb{R}^4$, with the final entry dimensionless. The same action is then written as ${}^w\bar{\mathbf{p}} = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}$. Thus the 4×4 matrix $\mathbf{T}_{wb}$ acts directly on homogeneous point-coordinate columns, whereas $A_{\mathbf{T}_{wb}}$ names its induced affine action on ordinary three-component point coordinates.

The current configuration of this rigid link is the composition of the fixed body-coordinate assignment with the pose-induced coordinate map:

$$\chi_{\mathbf{T}_{wb}} := A_{\mathbf{T}_{wb}} \circ c_b : \mathcal{B}_b \rightarrow \mathbb{R}^3.$$

This relationship is often written in the compact robotics shorthand requested here:

$$P \xrightarrow{c_b} {}^b\mathbf{p}_P \xrightarrow{\mathbf{T}_{wb}} {}^w\mathbf{p}_P.$$

The label $\mathbf{T}_{wb}$ over the second arrow is shorthand for applying the transformation represented by that matrix; it does not declare that a 4×4 matrix is literally a function from $\mathbb{R}^3$ to $\mathbb{R}^3$. The type-explicit ordinary-coordinate chain is

$$P \xrightarrow{c_b} {}^b\mathbf{p}_P \xrightarrow{A_{\mathbf{T}_{wb}}} {}^w\mathbf{p}_P = \chi_{\mathbf{T}_{wb}}(P).$$

The configuration map $\chi_{\mathbf{T}_{wb}}$ and the pose-induced map $A_{\mathbf{T}_{wb}}$ therefore have different domains. The first accepts a labelled material point $P \in \mathcal{B}_b$; the second accepts a point-coordinate column in $\mathbb{R}^3$. The map $A_{\mathbf{T}_{wb}}$ is affine because it includes translation; it is not the transformation law for a free vector, whose components would be transformed by $\mathbf{R}_{wb}$ alone. The two maps encode the same rigid placement only after the fixed assignment c_b connects a material point to its body-frame coordinates.

Let $\mathcal{C}_b$ be the configuration space of this one free, orientation-preserving rigid link. Assume that $\mathcal{B}_b$ contains at least three non-collinear labelled material points, so their placement determines the link's rigid pose.¹

¹In three dimensions, **one point** determines only a location; the link can still rotate freely about that point. **Two distinct points** determine a line; the link can still rotate about that line. **Three non-collinear points** form a triangle; the triangle fixes two independent directions and therefore a third direction through their cross product, so no rotational freedom remains.

For this rigid-link model, fixed frames $\{w\}$ and $\{b\}$, and fixed body-coordinate assignment c_b , each $\mathbf{T}_{wb} \in SE(3)$ produces exactly one configuration through $A_{\mathbf{T}_{wb}} \circ c_b$, and each configuration in $\mathcal{C}_b$ determines exactly one pose of $\{b\}$ relative to $\{w\}$. Define the correspondence map Φ_b by

$$\begin{aligned}\Phi_b : SE(3) &\longrightarrow \mathcal{C}_b, \\ \mathbf{T}_{wb} &\longmapsto \Phi_b(\mathbf{T}_{wb}) := \chi_{\mathbf{T}_{wb}} = A_{\mathbf{T}_{wb}} \circ c_b.\end{aligned}$$

The map Φ_b is one-to-one and onto: different pose matrices give different configuration maps, and every admissible configuration map is obtained from one pose matrix. Thus Φ_b is the one-to-one correspondence; it is neither a physical configuration nor a pose matrix. The correspondence is summarised by

rigid-body configuration χ $\longleftrightarrow$ body pose represented by $\mathbf{T}_{wb}$.
--

This correspondence is not an equality of mathematical types: χ is a placement map on material points, the pose is a physical relation between frames, $\mathbf{T}_{wb}$ is a matrix representation of that pose, and $A_{\mathbf{T}_{wb}}$ is the coordinate map induced by the matrix.

From free-body coordinates to joint coordinates

A **fixed-base robot** has one base link whose pose relative to the world frame is constant, so no coordinates are needed to describe motion of the base. An **open serial chain** connects links one after another without forming a closed kinematic loop.

A **closed kinematic loop** occurs when a sequence of links and joints eventually connects back to an earlier link. Consequently, there are two or more joint paths between some links. For example, a four-bar mechanism forms a loop:

$$\text{base} \rightarrow L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow \text{base}.$$

This differs from an open serial chain:

$$\text{base} \rightarrow L_1 \rightarrow L_2 \rightarrow \text{end effector}.$$

In the ideal model used here, each joint has one degree of freedom: it permits one independent relative motion between its two neighbouring links while preventing the other relative motions.

Let $N \in \mathbb{N}$ be the number of physical joints and let $j \in \{1, \dots, N\}$ be a joint index.

For joint j , define the **zero arrangement** and oriented **joint axis** by

zero arrangement _{j} := relative placement of the two links at coordinate zero,
 oriented joint axis _{j} := $(\mathcal{L}_j, \mathbf{s}_j)$, $\|\mathbf{s}_j\|_2 = 1$, $\mathbf{s}_j \parallel \mathcal{L}_j$,

where $\mathcal{L}_j$ is the geometric axis line and the dimensionless unit vector $\mathbf{s}_j$ chooses its positive direction. In a declared frame $\{a\}$, the axis line has the coordinate representation

$${}^a\mathcal{L} = \{{}^a\mathbf{p}_\ell + \lambda {}^a\mathbf{s} \mid \lambda \in \mathbb{R}\}.$$

Here ${}^a\mathbf{p}_\ell \in \mathbb{R}^3$ is the coordinate column of any point on the line, expressed in frame $\{a\}$ and measured in metres; ${}^a\mathbf{s} \in \mathbb{R}^3$ is its dimensionless unit positive direction; and λ is signed distance along the line, measured in metres. The supported one-degree-of-freedom joint models are

revolute joint: $\theta_j \in \mathbb{R}$ radians, relative rotation by θ_j about $\mathcal{L}_j$,
 prismatic joint: $d_j \in \mathbb{R}$ metres, relative translation $d_j\mathbf{s}_j$.

Positive revolute motion follows the right-hand rule about $\mathbf{s}_j$; positive prismatic motion is along $\mathbf{s}_j$. For a prismatic joint, the location of $\mathcal{L}_j$ does not affect the translation.

Let $\mathcal{Q}$ be the **generalised-coordinate domain**: the set of all valid generalised-coordinate columns for the model, including any declared joint limits. Let $n \in \mathbb{N}$ be the number of scalar generalised coordinates. A **generalised-coordinate column** is

$$\mathbf{q} = [q_1 \ \cdots \ q_n]^\top \in \mathcal{Q}.$$

Here $i \in \{1, \dots, n\}$ is a coordinate index, each q_i is a **generalised coordinate**, and the entries of $\mathbf{q}$ are independent scalar parameters chosen so that $\mathbf{q}$ locally specifies one complete system configuration. The coordinate-to-configuration relationship is

$$\mathbf{q} \in \mathcal{Q} \mapsto \chi(\mathbf{q}) \in \mathcal{C}.$$

Here $\mathcal{C}$ is the configuration space defined above. The meaning, units, and valid values of every q_i must be declared for the model.

For the fixed-base open chain considered here, N , the number of physical joints, equals n , the number of generalised coordinates, because every physical joint has one independent coordinate. Thus $n = N$. The coordinate associated with joint j is

$$q_j := \begin{cases} \theta_j, & \text{if joint } j \text{ is revolute,} \\ d_j, & \text{if joint } j \text{ is prismatic.} \end{cases}$$

A robot containing both joint types has a coordinate column whose entries have different physical units, so $\mathbf{q}$ is not a geometric vector with one common unit and an unscaled Euclidean norm $\|\mathbf{q}\|_2$ has no direct physical meaning.

Given the fixed base, link geometry, joint types, axis directions, and zero conventions, $\mathbf{q}$ determines a complete robot configuration $\chi(\mathbf{q}) \in \mathcal{C}$. The coordinate description need not be globally unique: for example, revolute coordinates that differ by a full turn can represent the same physical arrangement when no joint limit distinguishes them. A floating-base robot would additionally require coordinates for the base pose; that model is outside the present fixed-base scope.²

Next choose a **space frame** $\{s\}$ whose pose relative to the world frame $\{w\}$ is constant. In general, the space frame may be:

- the world frame itself, so $\{s\} = \{w\}$;
- a frame rigidly attached to the fixed base link; or
- any other frame whose pose relative to $\{w\}$ is constant.

Also choose a frame $\{e\}$ rigidly attached to the selected end-effector link. The **pose of $\{e\}$ relative to $\{s\}$** is the position and orientation of that one frame, represented by $\mathbf{T}_{se} \in SE(3)$ as defined in Section 7.6. The transformation direction is from end-effector point coordinates to space-frame point coordinates.

The chapter therefore uses the following modelling chain:

$$\boxed{\begin{array}{ccc} \underbrace{\mathbf{q} \in \mathcal{Q}}_{\text{joint-coordinate description}} & \longmapsto & \underbrace{\chi(\mathbf{q}) \in \mathcal{C}}_{\text{complete robot configuration}} \longmapsto \underbrace{\mathbf{T}_{se}(\mathbf{q}) \in SE(3)}_{\text{pose of the selected frame}} \end{array}}$$

The composition of these two arrows is the **forward-kinematics map** for frame $\{e\}$:

$$\boxed{\mathbf{F}_e : \mathcal{Q} \rightarrow SE(3), \quad \mathbf{q} \longmapsto \mathbf{F}_e(\mathbf{q}) := \mathbf{T}_{se}(\mathbf{q})}$$

The map $\mathbf{F}_e$ and its value $\mathbf{T}_{se}(\mathbf{q})$ are different mathematical objects. The map is the rule defined for allowable joint coordinates; the matrix is the pose produced when that rule is evaluated at one selected $\mathbf{q}$. Robotics texts often use the shorthand $\mathbf{T}(\mathbf{q})$ for both the forward-kinematics formula and its evaluated value, so the intended meaning must be read from context.

The one-to-one rigid-body correspondence applies separately to each link. For an articulated robot, the complete configuration contains the compatible placement of every link, whereas $\mathbf{T}_{se}(\mathbf{q})$ represents the pose of only the selected end-effector frame. The remaining links may have different placements, and distinct joint-coordinate columns may produce the same end-effector pose.

²A **floating-base robot** has no base link fixed relative to the world. Its base pose can change with three translational and three rotational degrees of freedom, represented by an element of $SE(3)$, in addition to any internal joint coordinates. *Floating* is a modelling term; the robot need not literally float.

This chapter expresses each joint's geometry in declared coordinates, composes those joint motions to construct $\mathbf{F}_e$, and later differentiates the forward kinematics to obtain a Jacobian. A rigid-body Jacobian maps joint rates to a declared body or spatial representation of the selected frame's twist; a task Jacobian instead maps joint rates to the rate of a declared task output, such as end-effector position alone.

8.2 Screw axes and one-degree-of-freedom joint motion

8.2.1 The physical problem: one joint, one allowed motion

A door hinge and a straight linear slide each have one degree of freedom, so one scalar joint coordinate is sufficient to describe the allowed relative configuration. Their motions are nevertheless different. The hinge permits rotation about a fixed line, whereas the slide permits translation along a fixed direction. A useful joint model must describe both cases, locate the permitted motion in physical space, and produce the instantaneous twist caused by a joint rate.

Imagine turning an ideal threaded screw. As the screw rotates about its centreline, it also advances along that same line. If θ is the signed rotation in radians and $\Delta\ell$ is the signed advance in metres, the thread geometry imposes

$$\Delta\ell = h\theta,$$

where h is the signed **pitch**, measured in metres per radian. This coupled rotation and translation is a **screw motion**. Its **screw axis** specifies the centreline, its positive direction, and the pitch h . A door hinge is the pure-rotation case $h = 0$: it rotates without advancing. A straight linear slide is the pure-translation case: it moves along the axis direction with zero rotation. These are the revolute and prismatic joint models used by most robot manipulators.

The physical motion must be distinguished from its representation. A joint axis is a geometric feature of the mechanism. Its coordinate column changes when the reference frame changes, even though the physical joint does not. The current joint angle or displacement is also separate from the axis: the axis states which motion is allowed, while the joint coordinate states how far the joint has moved.

8.2.2 An oriented line in a declared frame

Let $\{a\}$ be a fixed reference frame with origin O_a . The origin is a reference point chosen by the modeller; it need not be a material point on the moving link. The modeller may place O_a :

- at the world origin;

- on the fixed base link;
- on the joint axis for convenience; or
- anywhere else fixed relative to the world.

Let ${}^a\mathbf{p}_\ell \in \mathbb{R}^3$ be the coordinate column of any point on a joint-axis line, expressed along the axes of $\{a\}$ and measured in metres. Let ${}^a\mathbf{s} \in \mathbb{R}^3$ be a dimensionless unit column pointing in the chosen positive direction of that axis, so

$$\|{}^a\mathbf{s}\|_2 = 1.$$

The coordinate representation of the oriented line is the set

$${}^a\mathcal{L} = \{ {}^a\mathbf{p}_\ell + \lambda {}^a\mathbf{s} \mid \lambda \in \mathbb{R} \},$$

where the scalar λ is measured in metres. The point ${}^a\mathbf{p}_\ell$ is not unique: adding any distance along the line gives another valid point. The unit direction fixes an orientation for the line, so replacing ${}^a\mathbf{s}$ by $-{}^a\mathbf{s}$ reverses the positive joint direction.

This line description contains only geometry. It does not contain a joint position, a joint rate, or a finite rigid displacement.

8.2.3 Revolute motion about the line

For the revolute joint introduced above, the permitted motion is pure rotation about its joint-axis line. Let $t \in \mathbb{R}$ denote time in seconds, let j denote the joint index, let $\theta_j(t) \in \mathbb{R}$ be joint j 's angle in radians, and let $\dot{\theta}_j(t)$ be its angular rate in rad s^{-1} . The positive direction ${}^a\mathbf{s}$ and the right-hand rule determine the sign of θ_j .

The joint's angular velocity relative to the fixed frame, expressed along the axes of $\{a\}$, is

$${}^a\boldsymbol{\omega} = {}^a\mathbf{s} \dot{\theta}_j.$$

The spatial-twist definition in Section 7.9.3 showed that a twist referred to a fixed frame defines the rigid-motion velocity field

$${}^a\mathbf{v}_P = {}^a\boldsymbol{\nu} + {}^a\boldsymbol{\omega} \times {}^a\mathbf{p}_P,$$

where ${}^a\mathbf{p}_P \in \mathbb{R}^3$ is the current position of a body-fixed point P in metres, ${}^a\mathbf{v}_P \in \mathbb{R}^3$ is its velocity in m s^{-1} , and ${}^a\boldsymbol{\nu} \in \mathbb{R}^3$ is the linear part of the spatial twist referred to O_a , also in m s^{-1} . The quantity ${}^a\boldsymbol{\nu}$ is the velocity field evaluated at O_a ; it is not generally the velocity of a link-frame origin.

$${}^a\boldsymbol{\xi} = {}^a\mathbf{S}_R \dot{\theta}_j.$$

Here *normalised* means that the axis-direction column satisfies $\|{}^a\mathbf{s}\|_2 = 1$; it does not mean that the complete six-component column has unit Euclidean norm. Using the project's linear-first twist convention and combining the two derived components gives

$${}^a\boldsymbol{\xi} = \begin{bmatrix} {}^a\boldsymbol{\nu} \\ {}^a\boldsymbol{\omega} \end{bmatrix} = \underbrace{\begin{bmatrix} -{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell \\ {}^a\mathbf{s} \end{bmatrix}}_{{}^a\mathbf{S}_R} \dot{\theta}_j.$$

This separates the geometry of the allowed motion from the amount of motion: ${}^a\mathbf{S}_R$ encodes the oriented axis line and the pure-rotation rule in frame $\{a\}$, θ_j records the signed accumulated rotation about that axis, and $\dot{\theta}_j$ records how quickly that amount changes. Thus the same ${}^a\mathbf{S}_R$ can describe different joint angles and rates, while the resulting twist changes with $\dot{\theta}_j$.

Therefore the explicit coordinate column referred to frame $\{a\}$ is

$${}^a\mathbf{S}_R := \begin{bmatrix} -{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell \\ {}^a\mathbf{s} \end{bmatrix} \in \mathbb{R}^6.$$

The column ${}^a\mathbf{S}_R$ describes the complete instantaneous rigid motion produced by one unit of revolute joint rate. Its blocks mean:

- The bottom block ${}^a\mathbf{s}$ is the positive rotation-axis direction.
- The top block $-{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell$ is the velocity-field value at O_a per unit joint rate. It accounts for the axis not necessarily passing through O_a .

The subscript R labels the revolute case.

The first three entries of ${}^a\mathbf{S}_R$ have units of metres per radian, conventionally written as metres because radians are dimensionless, and the final three entries are dimensionless. Multiplication by $\dot{\theta}_j$ produces linear velocity in m s^{-1} and angular velocity in rad s^{-1} .

8.2.4 Helical motion and screw pitch

A **helical joint** permits rotation about an axis and translation along that same axis in a fixed proportion. Its **pitch** $h \in \mathbb{R}$ is the signed axial distance travelled per positive radian of rotation, measured in m rad^{-1} . The axial speed is therefore $h\dot{\theta}_j$, and its direction is ${}^a\mathbf{s}$.

For a helical motion, a point on the axis is no longer stationary. Its velocity is $h^a \dot{\theta}_j$. Evaluate the spatial velocity field at ${}^a \mathbf{p}_\ell$:

$$h^a \dot{\theta}_j = {}^a \boldsymbol{\nu} + ({}^a \mathbf{s} \dot{\theta}_j) \times {}^a \mathbf{p}_\ell.$$

Rearranging separates the geometry from the angular rate:

$${}^a \boldsymbol{\nu} = (-{}^a \mathbf{s} \times {}^a \mathbf{p}_\ell + h^a \mathbf{s}) \dot{\theta}_j.$$

To obtain the velocity of an arbitrary body-fixed point P , substitute this linear part and ${}^a \boldsymbol{\omega} = {}^a \mathbf{s} \dot{\theta}_j$ into the spatial velocity field:

$$\begin{aligned} {}^a \mathbf{v}_P &= {}^a \boldsymbol{\nu} + {}^a \boldsymbol{\omega} \times {}^a \mathbf{p}_P \\ &= (-{}^a \mathbf{s} \times {}^a \mathbf{p}_\ell + h^a \mathbf{s}) \dot{\theta}_j + ({}^a \mathbf{s} \dot{\theta}_j) \times {}^a \mathbf{p}_P \\ &= {}^a \boldsymbol{\omega} \times ({}^a \mathbf{p}_P - {}^a \mathbf{p}_\ell) + h^a \mathbf{s} \dot{\theta}_j. \end{aligned}$$

The first term is the rotational velocity about the axis line and is tangent to the circular cross-section through P . The second term is the axial velocity parallel to ${}^a \mathbf{s}$. When P is on the axis, ${}^a \mathbf{p}_P - {}^a \mathbf{p}_\ell = \mathbf{0}_3$, so the rotational term vanishes and the equation reduces to ${}^a \mathbf{v}_\ell = h^a \mathbf{s} \dot{\theta}_j$.

Figure 8.2 fixes one instant with $\theta_j = \pi/2$, $\dot{\theta}_j > 0$, and positive pitch $h > 0$. The body-fixed point P has rotated through one quarter-turn and advanced by $h\theta_j$ along the oriented axis. Its rotational velocity about the line and its axial velocity are placed head to tail to produce the tangent velocity ${}^a \mathbf{v}_P$. A body-fixed point currently on the axis has no circular velocity contribution, but it retains the axial velocity $h^a \mathbf{s} \dot{\theta}_j$ and is therefore not stationary.

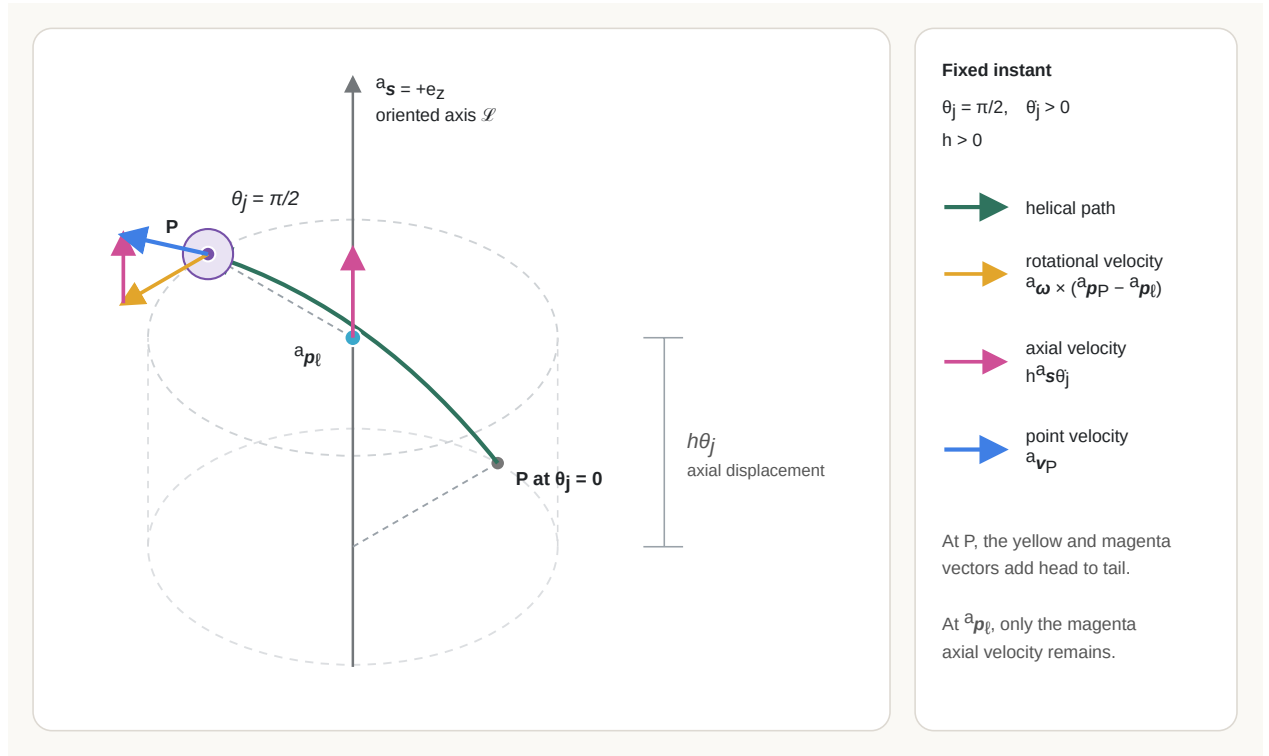


Figure 8.2: Helical motion at a fixed joint angle, showing axial displacement and the instantaneous velocity components.

Hence the normalised helical screw-axis coordinates are

$${}^a\mathbf{S}_H := \begin{bmatrix} -{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell + h {}^a\mathbf{s} \\ {}^a\mathbf{s} \end{bmatrix} \in \mathbb{R}^6, \quad {}^a\boldsymbol{\xi} = {}^a\mathbf{S}_H \dot{\theta}_j.$$

The subscript H labels the helical case. Setting $h = 0$ recovers the revolute screw axis. A positive pitch translates in the ${}^a\mathbf{s}$ direction during positive rotation; a negative pitch translates in the opposite direction.

The choice of point on the line does not change the screw-axis coordinates. If ${}^a\mathbf{p}'_\ell = {}^a\mathbf{p}_\ell + \lambda {}^a\mathbf{s}$ for any $\lambda \in \mathbb{R}$ measured in metres, then

$$\begin{aligned} -{}^a\mathbf{s} \times {}^a\mathbf{p}'_\ell + h {}^a\mathbf{s} &= -{}^a\mathbf{s} \times ({}^a\mathbf{p}_\ell + \lambda {}^a\mathbf{s}) + h {}^a\mathbf{s} \\ &= -{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell - \lambda \underbrace{{}^a\mathbf{s} \times {}^a\mathbf{s}}_{\mathbf{0}_3} + h {}^a\mathbf{s} \\ &= -{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell + h {}^a\mathbf{s}. \end{aligned}$$

The coordinate column therefore depends on the oriented line, pitch, and reference frame, but not on which point is selected along that line.

8.2.5 Prismatic motion along a direction

For the prismatic joint introduced above, the permitted motion is pure translation along a fixed direction. Let $d_j(t) \in \mathbb{R}$ be the joint displacement in metres and let $\dot{d}_j(t)$ be its rate in m s^{-1} . There is no angular velocity, and every point of the moving link has the same instantaneous velocity:

$${}^a\boldsymbol{\omega} = \mathbf{0}_3, \quad {}^a\mathbf{v}_P = {}^a\mathbf{s}\dot{d}_j.$$

The normalised prismatic screw-axis coordinates and resulting twist are therefore

$$\boxed{{}^a\mathbf{S}_P := \begin{bmatrix} {}^a\mathbf{s} \\ \mathbf{0}_3 \end{bmatrix} \in \mathbb{R}^6, \quad {}^a\boldsymbol{\xi} = {}^a\mathbf{S}_P\dot{d}_j.}$$

The subscript P labels the prismatic case. The first three entries of ${}^a\mathbf{S}_P$ are dimensionless direction components, and multiplication by $\dot{d}_j$ produces linear velocity in m s^{-1} . The angular entries remain zero. Unlike a revolute or helical motion, a pure translation has no unique finite axis location; its direction is sufficient.

Pure translation is sometimes described as screw motion with infinite pitch. That phrase is a geometric limiting interpretation, not a numerical instruction. A software model should use the finite prismatic form ${}^a\mathbf{S}_P$ and should not store $h = \infty$.

8.2.6 The normalisation rule and the declared component order

Write any normalised screw-axis coordinate column in the project's linear-first order as

$${}^a\mathbf{S} = \begin{bmatrix} {}^a\mathbf{s}_v \\ {}^a\mathbf{s}_\omega \end{bmatrix} \in \mathbb{R}^6.$$

The subscripts v and ω identify the coefficients that generate the linear and angular parts of the twist after multiplication by the appropriate scalar joint rate.

The linear coefficient ${}^a\mathbf{s}_v$ must not be read as the velocity of a moving link-frame origin. Multiplication by the scalar screw rate gives ${}^a\boldsymbol{\nu} = {}^a\mathbf{s}_v\dot{\sigma}$, which is the rigid-motion velocity field evaluated at the reference origin O_a . If a link-frame origin O_b is instead at ${}^a\mathbf{p}_b$, its velocity is obtained by evaluating the complete field at that point:

$${}^a\mathbf{v}_{O_b} = ({}^a\mathbf{s}_v + {}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_b) \dot{\sigma}.$$

Only when O_b is currently located at O_a does ${}^a\mathbf{v}_{O_b} = {}^a\mathbf{s}_v\dot{\sigma}$. For rotational or helical motion, ${}^a\mathbf{s}_v$ is therefore the velocity-field coefficient at O_a per radian; for prismatic motion, it is the unit translation direction.

The two valid normalisation cases are

$$\boxed{\begin{array}{ll} \|{}^a\mathbf{s}_\omega\|_2 = 1, & {}^a\mathbf{s}_v = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega \quad \text{for revolute or helical motion,} \\ {}^a\mathbf{s}_\omega = \mathbf{0}_3, \quad \|{}^a\mathbf{s}_v\|_2 = 1 & \text{for prismatic motion.} \end{array}}$$

These are separate normalisation rules because the coordinate column has mixed physical units. Taking one ordinary Euclidean norm of all six entries would add squared lengths to squared dimensionless quantities and would not define a physical magnitude.

Modern Robotics writes twists and screw axes in angular-first order, $[\boldsymbol{\omega}^\top \boldsymbol{\nu}^\top]^\top$. This project uses the linear-first order established in Section 7.9.2 and Section 7.9.3, $[\boldsymbol{\nu}^\top \boldsymbol{\omega}^\top]^\top$. The formulas above have already been reordered; copying an angular-first six-vector directly into this convention would describe the wrong motion.

If the same physical screw is represented in another fixed frame $\{b\}$, its coordinate column changes through the adjoint already defined in Section 7.9.4. Let $\mathbf{T}_{ab} \in SE(3)$ transform point coordinates from $\{b\}$ to $\{a\}$, and write

$$\mathbf{T}_{ab} = \begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

Here $\mathbf{R}_{ab}$ maps vector-coordinate columns from the axes of $\{b\}$ to the axes of $\{a\}$, and ${}^a\mathbf{p}_b$ is the position of O_b relative to O_a , expressed in $\{a\}$ and measured in metres. For the project's linear-first ordering, the adjoint matrix is

$$\text{Ad}_{\mathbf{T}_{ab}} = \begin{bmatrix} \mathbf{R}_{ab} & [{}^a\mathbf{p}_b]_\times \mathbf{R}_{ab} \\ \mathbf{0}_{3 \times 3} & \mathbf{R}_{ab} \end{bmatrix}.$$

The skew-symmetric matrix $[{}^a\mathbf{p}_b]_\times$ represents the cross product: $[{}^a\mathbf{p}_b]_\times \mathbf{x} = {}^a\mathbf{p}_b \times \mathbf{x}$ for any $\mathbf{x} \in \mathbb{R}^3$.

To see why screw-axis coordinate columns follow this transformation rule, let $\dot{q}_j$ be the scalar rate of the same one-degree-of-freedom joint. In frame $\{b\}$, its twist is

$${}^b\boldsymbol{\xi} = {}^b\mathbf{S} \dot{q}_j.$$

Twists transform through the adjoint:

$${}^a\boldsymbol{\xi} = \text{Ad}_{\mathbf{T}_{ab}} {}^b\boldsymbol{\xi}.$$

Substitution gives

$${}^a\boldsymbol{\xi} = \text{Ad}_{\mathbf{T}_{ab}} {}^b\mathbf{S} \dot{q}_j.$$

The screw-axis coordinate column in frame $\{a\}$ must also satisfy

$${}^a\boldsymbol{\xi} = {}^a\mathbf{S} \dot{q}_j.$$

Because these equations describe the same twist for every joint rate, the columns multiplying $\dot{q}_j$ must be equal:

$$\boxed{{}^a\mathbf{S} = \text{Ad}_{\mathbf{T}_{ab}} {}^b\mathbf{S}}.$$

The physical joint axis has not moved in this equation. Only the reference axes and origin used to represent it have changed.

8.2.7 Worked checks

Consider a pure revolute joint whose axis points along positive z_a and passes through

$${}^a\mathbf{p}_\ell = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \text{ m}, \quad {}^a\mathbf{s} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

The linear coefficient is

$$-{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell = - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ 0 \end{bmatrix} \text{ m},$$

so the linear-first screw-axis coordinates are

$${}^a\mathbf{S}_R = [2 \quad -1 \quad 0 \quad 0 \quad 0 \quad 1]^\top.$$

At $\dot{\theta}_j = 3 \text{ rad s}^{-1}$, the twist parts are

$${}^a\boldsymbol{\nu} = \begin{bmatrix} 6 \\ -3 \\ 0 \end{bmatrix} \text{ m s}^{-1}, \quad {}^a\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix} \text{ rad s}^{-1}.$$

Let ${}^a\mathbf{v}_\ell \in \mathbb{R}^3$ denote the velocity field evaluated at the selected axis point, expressed in frame $\{a\}$ and measured in m s^{-1} . Its value verifies the required zero velocity:

$$\begin{aligned} {}^a\mathbf{v}_\ell &= {}^a\boldsymbol{\nu} + {}^a\boldsymbol{\omega} \times {}^a\mathbf{p}_\ell \\ &= \begin{bmatrix} 6 \\ -3 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix} \times \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 6 \\ -3 \\ 0 \end{bmatrix} + \begin{bmatrix} -6 \\ 3 \\ 0 \end{bmatrix} = \mathbf{0}_3 \text{ m s}^{-1}. \end{aligned}$$

As a contrasting prismatic example, take the unit direction

$${}^a\mathbf{s} = \frac{1}{3} \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

Its norm is one because $\sqrt{1^2 + 2^2 + 2^2}/3 = 1$. At $\dot{d}_j = 0.3 \text{ m s}^{-1}$, every moving-link point has velocity

$${}^a\mathbf{v}_P = {}^a\mathbf{s}\dot{d}_j = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.2 \end{bmatrix} \text{ m s}^{-1},$$

and the angular velocity is zero.

8.2.8 Contrasting the geometric and motion quantities

The terms introduced above describe related but different objects:

Quantity	What it describes	What it depends on
Oriented axis line $\mathcal{L}$	A physical line and its positive direction	Joint geometry; its coordinate representation depends on the frame
Screw pitch h	Signed axial travel per radian of rotation	Joint geometry, not the current joint rate
Normalised screw-axis coordinates ${}^a\mathbf{S}$	A six-entry coordinate representation of the permitted instantaneous motion per unit joint rate	Axis, pitch, component order, and reference frame
Twist ${}^a\boldsymbol{\xi}$	The actual instantaneous rigid-motion velocity field at one instant	Screw-axis coordinates and the current joint rate
Joint coordinate θ_j or d_j	The current amount of rotation or translation from a declared zero configuration	The current joint configuration and zero convention

A finite relative pose is a further object. It will be obtained from a screw axis and a finite joint-coordinate change through the $SE(3)$ exponential in Section 8.3. A screw-axis column alone is neither the current pose nor a finite transformation.

Quick test A: screw axes and joint motion

1. **Recall:** What three geometric ingredients describe a finite-pitch screw axis before it is converted to a six-entry coordinate column?
2. **Contrast:** Distinguish an oriented axis line, normalised screw-axis coordinates, a twist, and a joint coordinate.
3. **Derivation:** A pure revolute axis has unit direction ${}^a\mathbf{s}$ and passes through ${}^a\mathbf{p}_\ell$. Starting from the spatial velocity field, derive the linear coefficient $-{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell$.
4. **Application:** Find the linear-first screw-axis coordinates of a pure revolute joint about positive z_a through $[1 \ 2 \ 0]^T$ m.
5. **Application:** A prismatic joint translates along negative y_a with $\dot{d}_j = 0.4 \text{ m s}^{-1}$. Write its normalised screw-axis coordinates and the resulting twist.
6. **Pitch:** A helical joint has $h = 0.01 \text{ m rad}^{-1}$ and $\dot{\theta}_j = 5 \text{ rad s}^{-1}$. What is its signed axial speed?
7. **Proof:** Show that replacing ${}^a\mathbf{p}_\ell$ by another point on the same line leaves the revolute or helical screw-axis coordinates unchanged.
8. **Limitation:** Why should software use the prismatic screw form instead of storing an infinite pitch? Why should it not normalise a general six-entry screw column with one ordinary Euclidean norm?

Answers and hints

1. Any point on the axis, a unit direction along the axis, and the signed pitch.
2. The line is physical joint geometry; ${}^a\mathbf{S}$ is its frame-dependent six-entry motion coordinate; ${}^a\boldsymbol{\xi}$ is the instantaneous motion after multiplication by the current joint rate; and θ_j or d_j records the joint's current displacement from a declared zero.
3. For pure rotation, a point on the axis is stationary. Substitute ${}^a\boldsymbol{\omega} = {}^a\mathbf{s}\dot{\theta}_j$ and ${}^a\mathbf{p}_P = {}^a\mathbf{p}_\ell$ into ${}^a\mathbf{v}_P = {}^a\boldsymbol{\nu} + {}^a\boldsymbol{\omega} \times {}^a\mathbf{p}_P$, set the result to zero, and collect the coefficient of $\dot{\theta}_j$.
4. The result is $[2 \ -1 \ 0 \ 0 \ 0 \ 1]^T$ in the project's linear-first order.
5. The unit direction is $[0 \ -1 \ 0]^T$, so ${}^a\mathbf{S}_P = [0 \ -1 \ 0 \ 0 \ 0 \ 0]^T$ and ${}^a\boldsymbol{\xi} = [0 \ -0.4 \ 0 \ 0 \ 0 \ 0]^T$, with linear entries in m s^{-1} and angular entries in rad s^{-1} .
6. The signed axial speed is $h\dot{\theta}_j = 0.05 \text{ m s}^{-1}$ in the positive axis direction.
7. Write ${}^a\mathbf{p}'_\ell = {}^a\mathbf{p}_\ell + \lambda {}^a\mathbf{s}$ and use ${}^a\mathbf{s} \times {}^a\mathbf{s} = \mathbf{0}_3$.
8. Infinity is not needed to represent pure translation and can create non-finite arithmetic. The six entries have mixed physical units, so their squared values cannot be added into one physically meaningful norm. Normalise the angular part for revolute or helical motion and the linear part for prismatic motion.

8.3 Rigid motion from the $SE(3)$ exponential

8.3.1 From a constant twist to a finite pose

The screw-axis coordinate column from Section 8.2 describes an instantaneous motion per unit joint rate. It does not yet answer the finite question: after the joint moves through a signed angle or displacement, what rigid transformation has occurred?

Let frame $\{a\}$ be the fixed frame in which the screw is represented, and write a normalised screw-axis coordinate column in the project's linear-first order as

$${}^a\mathbf{S} = \begin{bmatrix} {}^a\mathbf{s}_v \\ {}^a\mathbf{s}_\omega \end{bmatrix} \in \mathbb{R}^6.$$

The columns ${}^a\mathbf{s}_v, {}^a\mathbf{s}_\omega \in \mathbb{R}^3$ are the linear and angular coefficients defined in the previous section. Apply the same hat operator used for twists to define the **matrix representation of the screw axis**:

$$\widehat{{}^a\mathbf{S}} := \begin{bmatrix} [{}^a\mathbf{s}_\omega]_\times & {}^a\mathbf{s}_v \\ \mathbf{0}_3^T & 0 \end{bmatrix} \in \mathfrak{se}(3). \quad (8.1)$$

Here $[\hat{\mathbf{s}}_\omega]_\times \in \mathbb{R}^{3 \times 3}$ is the skew-symmetric matrix satisfying $[\hat{\mathbf{s}}_\omega]_\times \mathbf{x} = {}^a\mathbf{s}_\omega \times \mathbf{x}$ for every $\mathbf{x} \in \mathbb{R}^3$. The hat operator changes the representation from a six-entry column to a structured 4×4 matrix; it does not change the represented screw motion.

Let $\sigma \in \mathbb{R}$ denote the signed amount of motion along the normalised screw. For a revolute or finite-pitch helical screw, $\sigma = \theta$ is an angle in radians. For a prismatic screw, $\sigma = d$ is a displacement in metres. The product ${}^a\mathbf{S}\sigma \in \mathbb{R}^6$ is called the **exponential-coordinate column** of the finite rigid displacement. The screw axis ${}^a\mathbf{S}$ specifies *which* one-parameter motion is followed, whereas σ specifies *how far* it is followed.

For any square matrix $\mathbf{A} \in \mathbb{R}^{m \times m}$, its **matrix exponential** is defined by the convergent power series

$$\exp(\mathbf{A}) := \mathbf{I}_m + \mathbf{A} + \frac{\mathbf{A}^2}{2!} + \frac{\mathbf{A}^3}{3!} + \dots$$

This is a matrix operation, not an entry-by-entry scalar exponential. Applying it to the hatted exponential coordinates produces the finite rigid transformation

$$\boxed{\mathbf{T}_\Delta(\sigma) := \exp(\widehat{{}^a\mathbf{S}}\sigma) \in SE(3)}. \quad (8.2)$$

The notation $\mathbf{T}_\Delta(\sigma)$ follows the active-displacement convention established in Section 7.6: input and output point coordinates are expressed in frame $\{a\}$, and the matrix moves the represented point or rigid body.

Let P denote a material point before the displacement and let P' denote the same material point after the displacement. Their homogeneous position-coordinate columns, both expressed in frame $\{a\}$, satisfy

$${}^a\bar{\mathbf{p}}_{P'} = \mathbf{T}_\Delta(\sigma) {}^a\bar{\mathbf{p}}_P.$$

Writing the active-displacement matrix in block form gives

$$\mathbf{T}_\Delta(\sigma) = \begin{bmatrix} \mathbf{R}_\Delta(\sigma) & {}^a\mathbf{p}_\Delta(\sigma) \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

where $\mathbf{R}_\Delta(\sigma) \in SO(3)$ is the dimensionless rotation block and ${}^a\mathbf{p}_\Delta(\sigma) \in \mathbb{R}^3$ is the translation column expressed in frame $\{a\}$ and measured in metres. The important distinctions are:

- P and P' are different physical positions of the same material point.
- Both coordinate columns are expressed in the same frame $\{a\}$.
- $\mathbf{T}_\Delta(\sigma)$ rotates and translates the point through the screw displacement.

- Applying the same transformation to every material point moves the complete rigid body.

Because every term in Equation 8.2 contains the same generator $\widehat{^a\mathbf{S}}$, two successive displacements along this same screw combine by adding their signed amounts:

$$\mathbf{T}_\Delta(\sigma_1)\mathbf{T}_\Delta(\sigma_2) = \mathbf{T}_\Delta(\sigma_1 + \sigma_2), \quad \sigma_1, \sigma_2 \in \mathbb{R}.$$

In particular, $\mathbf{T}_\Delta(0) = \mathbf{I}_4$ and $\mathbf{T}_\Delta(-\sigma) = \mathbf{T}_\Delta(\sigma)^{-1}$. The exponential therefore converts one fixed instantaneous motion direction into a one-parameter family of finite rigid displacements.

8.3.2 Closed forms for rotational and translational screws

The exponential above solves the forward problem: a chosen screw axis and signed amount produce a finite transformation. The **Chasles–Mozzi theorem** supplies the converse:

for every $\mathbf{T}_\Delta \in SE(3)$, there exist $^a\mathbf{S}$ and σ such that $\mathbf{T}_\Delta = \exp(\widehat{^a\mathbf{S}} \sigma)$.

This is an existence statement about a given initial-to-final transformation. Write that transformation as

$$\mathbf{T}_\Delta = \begin{bmatrix} \mathbf{R}_\Delta & ^a\mathbf{p}_\Delta \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3).$$

The theorem separates its possible forms into three cases:

- If $\mathbf{R}_\Delta \neq \mathbf{I}_3$, there exist a dimensionless unit direction $^a\mathbf{s}_\omega \in \mathbb{R}^3$, a principal angle $\theta \in (0, \pi]$ in radians, an axis point $^a\mathbf{p}_\ell \in \mathbb{R}^3$ in metres, and a pitch $h \in \mathbb{R}$ in m rad^{-1} such that

$$\mathbf{R}_\Delta = \exp([\mathbf{s}_\omega]_\times \theta), \quad ^a\mathbf{p}_\Delta = (\mathbf{I}_3 - \mathbf{R}_\Delta)^a\mathbf{p}_\ell + h\theta^a\mathbf{s}_\omega.$$

Thus the same endpoint transformation is produced by rotating through θ about the oriented line $\{^a\mathbf{p}_\ell + \lambda^a\mathbf{s}_\omega \mid \lambda \in \mathbb{R}\}$, where λ is measured in metres, and translating through $h\theta$ along that line. The case $h = 0$ is pure rotation.

- If $\mathbf{R}_\Delta = \mathbf{I}_3$ and ${}^a\mathbf{p}_\Delta \neq \mathbf{0}_3$, define

$$d := \|{}^a\mathbf{p}_\Delta\|_2, \quad {}^a\mathbf{s}_v := \frac{{}^a\mathbf{p}_\Delta}{d}, \quad {}^a\mathbf{s}_\omega := \mathbf{0}_3.$$

The transformation is pure translation through the distance $d > 0$ along the oriented direction ${}^a\mathbf{s}_v$ and satisfies $\mathbf{T}_\Delta = \exp(\widehat{{}^a\mathbf{S}} d)$ for ${}^a\mathbf{S} = [({}^a\mathbf{s}_v)^\top \mathbf{0}_3^\top]^\top$.

- If $\mathbf{T}_\Delta = \mathbf{I}_4$, choose $\sigma = 0$. Then $\exp(\widehat{{}^a\mathbf{S}} 0) = \mathbf{I}_4$ for every admissible normalised screw axis, so the axis is not unique.

These cases cover every element of $SE(3)$. They construct a screw path with the specified initial and final poses; they do not assert that the body's actual past trajectory followed that path. The complete theorem and proof are given in [the Chasles–Mozzi appendix](#).

For the development below, the theorem is the completeness bridge: it guarantees that the rotational or helical closed form for a non-identity rotation and the prismatic closed form for a pure translation are together sufficient to represent every finite spatial rigid displacement, rather than being only selected examples.

The power-series definition is general, but the structure of a normalised screw gives simpler formulas with a direct geometric interpretation. Consider first a revolute or finite-pitch helical screw. Define

$$\boldsymbol{\Omega} := [{}^a\mathbf{s}_\omega]_\times \in \mathbb{R}^{3 \times 3}.$$

where $\|{}^a\mathbf{s}_\omega\|_2 = 1$. The already-defined column ${}^a\mathbf{s}_v \in \mathbb{R}^3$ is the linear screw coefficient, measured in metres per radian; it is not an instantaneous velocity until it is multiplied by an angular rate. The finite displacement parameter is the signed angle $\theta \in \mathbb{R}$ in radians.

The hatted matrix representing this screw is

$$\widehat{{}^a\mathbf{S}} = \begin{bmatrix} \boldsymbol{\Omega} & {}^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

The powers of the hatted screw have a repeated block structure. For every integer $k \geq 1$, direct block multiplication gives

$$(\widehat{{}^a\mathbf{S}})^k = \begin{bmatrix} \boldsymbol{\Omega}^k & \boldsymbol{\Omega}^{k-1} {}^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

Substituting these powers into the exponential series separates its rotation and translation blocks:

$$\exp(\widehat{a\mathbf{S}}\theta) = \begin{bmatrix} \mathbf{I}_3 + \sum_{k=1}^{\infty} \frac{\boldsymbol{\Omega}^k \theta^k}{k!} & \sum_{k=1}^{\infty} \frac{\boldsymbol{\Omega}^{k-1} \theta^k}{k!} {}^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

The upper-left series is $\exp(\boldsymbol{\Omega}\theta)$. Because ${}^a\mathbf{s}_\omega$ is a unit column, $\boldsymbol{\Omega}^3 = -\boldsymbol{\Omega}$. Grouping the odd and even powers therefore gives Rodrigues' formula

$$\mathbf{R}(\theta) := \exp(\boldsymbol{\Omega}\theta) = \mathbf{I}_3 + \sin\theta \boldsymbol{\Omega} + (1 - \cos\theta)\boldsymbol{\Omega}^2 \in SO(3). \quad (8.3)$$

The upper-right series contains one lower power of $\boldsymbol{\Omega}$. Define

$$\begin{aligned} \mathbf{G}(\theta) &:= \mathbf{I}_3\theta + \frac{\boldsymbol{\Omega}\theta^2}{2!} + \frac{\boldsymbol{\Omega}^2\theta^3}{3!} + \dots \\ &= \mathbf{I}_3\theta + (1 - \cos\theta)\boldsymbol{\Omega} + (\theta - \sin\theta)\boldsymbol{\Omega}^2. \end{aligned}$$

The second equality follows because the coefficient of $\boldsymbol{\Omega}$ is $\theta^2/2! - \theta^4/4! + \dots = 1 - \cos\theta$, while the coefficient of $\boldsymbol{\Omega}^2$ is $\theta^3/3! - \theta^5/5! + \dots = \theta - \sin\theta$. Combining the two blocks yields the closed form

$$\exp(\widehat{a\mathbf{S}}\theta) = \begin{bmatrix} \mathbf{R}(\theta) & \mathbf{G}(\theta){}^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 1 \end{bmatrix}. \quad (8.4)$$

Radians are dimensionless in the algebra, but retaining the unit label prevents physical mistakes: ${}^a\mathbf{s}_v$ has units of m rad^{-1} , so $\mathbf{G}(\theta){}^a\mathbf{s}_v$ is a position column measured in metres.

The translation block becomes more transparent when the screw is written using its physical axis and pitch. Let ${}^a\mathbf{p}_\ell \in \mathbb{R}^3$ be the position column of any point P_ℓ on the oriented axis, expressed in frame $\{a\}$ and measured in metres, and let $h \in \mathbb{R}$ be the pitch in m rad^{-1} . The helical screw-axis derivation in Section 8.2.4 already established that the linear block of the normalised screw is

$${}^a\mathbf{s}_v = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega.$$

Because $\boldsymbol{\Omega} = [{}^a\mathbf{s}_\omega]_\times$, the same result can be written in the matrix form needed by the exponential calculation:

$${}^a\mathbf{s}_v = -\boldsymbol{\Omega}{}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega.$$

The term $-\mathbf{\Omega}^a \mathbf{p}_\ell = -{}^a \mathbf{s}_\omega \times {}^a \mathbf{p}_\ell$ accounts for the axis being offset from O_a ; it is the rotational velocity-field value at O_a per unit angle. The term $h {}^a \mathbf{s}_\omega$ adds the translation along the axis per unit angle. Choosing another point ${}^a \mathbf{p}_\ell + \lambda {}^a \mathbf{s}_\omega$ on the same axis does not change ${}^a \mathbf{s}_v$ because $\mathbf{\Omega}^a \mathbf{s}_\omega = \mathbf{0}_3$.

Rodrigues' formula implies

$$\mathbf{G}(\theta) \mathbf{\Omega} = \mathbf{R}(\theta) - \mathbf{I}_3.$$

Also, $\mathbf{\Omega}^a \mathbf{s}_\omega = \mathbf{0}_3$, so $\mathbf{G}(\theta) {}^a \mathbf{s}_\omega = \theta {}^a \mathbf{s}_\omega$. Substitution into the translation block of Equation 8.4 gives

$$\begin{aligned} \mathbf{G}(\theta) {}^a \mathbf{s}_v &= -\mathbf{G}(\theta) \mathbf{\Omega}^a \mathbf{p}_\ell + h \mathbf{G}(\theta) {}^a \mathbf{s}_\omega \\ &= (\mathbf{I}_3 - \mathbf{R}(\theta)) {}^a \mathbf{p}_\ell + h \theta {}^a \mathbf{s}_\omega. \end{aligned}$$

Therefore a finite-pitch screw has the geometric closed form

$$\mathbf{T}_{\Delta, H}(\theta) = \begin{bmatrix} \mathbf{R}(\theta) & (\mathbf{I}_3 - \mathbf{R}(\theta)) {}^a \mathbf{p}_\ell + h \theta {}^a \mathbf{s}_\omega \\ \mathbf{0}_3^\top & 1 \end{bmatrix}. \quad (8.5)$$

To interpret the transformation geometrically, let a point initially have position coordinates ${}^a \mathbf{x} \in \mathbb{R}^3$, expressed in frame $\{a\}$ and measured in metres. Applying $\mathbf{T}_{\Delta, H}(\theta)$ gives its final position

$${}^a \mathbf{x}' = \mathbf{R}(\theta) {}^a \mathbf{x} + (\mathbf{I}_3 - \mathbf{R}(\theta)) {}^a \mathbf{p}_\ell + h \theta {}^a \mathbf{s}_\omega.$$

Collecting the terms containing ${}^a \mathbf{p}_\ell$ rewrites the same point transformation as

$${}^a \mathbf{x}' = \mathbf{R}(\theta) ({}^a \mathbf{x} - {}^a \mathbf{p}_\ell) + {}^a \mathbf{p}_\ell + h \theta {}^a \mathbf{s}_\omega.$$

This form exposes the four geometric operations:

1. Subtracting ${}^a \mathbf{p}_\ell$ forms the displacement ${}^a \mathbf{x} - {}^a \mathbf{p}_\ell$ from the selected axis point P_ℓ to the moving point.
2. Multiplication by $\mathbf{R}(\theta)$ rotates that relative displacement through θ about the direction ${}^a \mathbf{s}_\omega$.
3. Adding ${}^a \mathbf{p}_\ell$ places the rotated point back around the line through P_ℓ , rather than around the frame origin O_a .
4. Adding $h \theta {}^a \mathbf{s}_\omega$ translates the result along the axis. Because h is signed axial distance per radian, $h \theta$ is the signed axial distance for the finite rotation θ .

The translation column therefore contains two geometrically different contributions:

$${}^a\mathbf{p}_\Delta(\theta) = \underbrace{(\mathbf{I}_3 - \mathbf{R}(\theta)){}^a\mathbf{p}_\ell}_{\text{correction for an axis offset from } O_a} + \underbrace{h\theta{}^a\mathbf{s}_\omega}_{\text{translation along the axis}}.$$

To check the axis itself, consider a point on the line with position ${}^a\mathbf{x} = {}^a\mathbf{p}_\ell + \lambda{}^a\mathbf{s}_\omega$, where λ is signed distance along the axis in metres. Because $\mathbf{R}(\theta){}^a\mathbf{s}_\omega = {}^a\mathbf{s}_\omega$, substitution into the rearranged point transformation gives

$${}^a\mathbf{x}' = {}^a\mathbf{p}_\ell + (\lambda + h\theta){}^a\mathbf{s}_\omega.$$

Thus an axis point remains on the axis and advances by $h\theta$, while an off-axis point rotates around the axis and advances along it. Setting $h = 0$ removes the axial advance and gives pure revolute motion about the offset line.

For example, consider a positive z_a revolute axis through ${}^a\mathbf{p}_\ell = [1 \ 0 \ 0]^\top$ m and a rotation $\theta = \pi/2$ rad. Its unit direction and linear screw coefficient are

$${}^a\mathbf{s}_\omega = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad {}^a\mathbf{s}_v = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} \text{ m rad}^{-1}.$$

The rotation and translation blocks are

$$\mathbf{R}\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \left(\mathbf{I}_3 - \mathbf{R}\left(\frac{\pi}{2}\right)\right){}^a\mathbf{p}_\ell = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \text{ m}.$$

Hence the finite displacement is

$$\mathbf{T}_{\Delta,R}\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Applying this transformation to the axis point returns the same point because $\mathbf{R}{}^a\mathbf{p}_\ell + (\mathbf{I}_3 - \mathbf{R}){}^a\mathbf{p}_\ell = {}^a\mathbf{p}_\ell$. This confirms that the finite motion rotates about the declared offset line rather than about O_a .

Now consider a normalised prismatic screw ${}^a\mathbf{S}_P = [({}^a\mathbf{s})^\top \ 0_3^\top]^\top$ with $\|{}^a\mathbf{s}\|_2 = 1$ and signed displacement $d \in \mathbb{R}$ in metres. Its hatted matrix satisfies

$$\widehat{{}^a\mathbf{S}_P} = \begin{bmatrix} \mathbf{0}_{3 \times 3} & {}^a\mathbf{s} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}, \quad (\widehat{{}^a\mathbf{S}_P})^2 = \mathbf{0}_{4 \times 4}.$$

All exponential-series terms of order two and higher therefore vanish, giving

$$\mathbf{T}_{\Delta,P}(d) = \exp(\widehat{{}^a\mathbf{S}_P} d) = \mathbf{I}_4 + \widehat{{}^a\mathbf{S}_P} d = \begin{bmatrix} \mathbf{I}_3 & {}^a\mathbf{s}d \\ \mathbf{0}_3^\top & 1 \end{bmatrix}. \quad (8.6)$$

This transformation has no rotation and translates every point by the signed vector ${}^a\mathbf{s}d$. The rotational and prismatic formulas are different branches of the same exponential map, but software should evaluate each with formulas appropriate to its normalisation and numerical domain.

8.3.3 Recovering screw coordinates with the $SE(3)$ logarithm

The exponential solves the forward problem: given a screw axis and a signed displacement, construct a rigid transformation. The inverse problem starts from a relative transformation and asks for a constant screw displacement that would produce it. This inverse is the **matrix logarithm** on $SE(3)$.

Before deriving the logarithm, recall from Section 8.2 the physical meaning of the two three-entry blocks that it must recover. For a rotational or finite-pitch helical motion, the normalised screw-axis coordinate column referred to frame $\{a\}$ is

$${}^a\mathbf{S} = \begin{bmatrix} {}^a\mathbf{s}_v \\ {}^a\mathbf{s}_\omega \end{bmatrix}.$$

Begin with the physical screw geometry. Let ${}^a\mathbf{s}_\omega \in \mathbb{R}^3$ be the dimensionless unit column pointing in the positive direction of the rotation axis, let ${}^a\mathbf{p}_\ell \in \mathbb{R}^3$ be the position of any point P_ℓ on that axis in metres, and let $h \in \mathbb{R}$ be the signed axial travel per radian in metres per radian. At one instant, consider a moving material point P at position ${}^a\mathbf{x} \in \mathbb{R}^3$. Its displacement from the selected axis point is ${}^a\mathbf{x} - {}^a\mathbf{p}_\ell$.

If the screw rotates at rate $\dot{\theta}$, its angular velocity is ${}^a\boldsymbol{\omega} = {}^a\mathbf{s}_\omega \dot{\theta}$. Rotation gives P the tangential velocity ${}^a\mathbf{s}_\omega \times ({}^a\mathbf{x} - {}^a\mathbf{p}_\ell) \dot{\theta}$, while the pitch gives it the axial velocity $h {}^a\mathbf{s}_\omega \dot{\theta}$. Adding these two physical contributions gives

$${}^a\mathbf{v}_P = [{}^a\mathbf{s}_\omega \times ({}^a\mathbf{x} - {}^a\mathbf{p}_\ell) + h {}^a\mathbf{s}_\omega] \dot{\theta}.$$

Expanding the cross product separates the part that depends on the point position ${}^a\mathbf{x}$ from the part that does not:

$${}^a\mathbf{v}_P = \left[\underbrace{-{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega}_{{}^a\mathbf{s}_v} + {}^a\mathbf{s}_\omega \times {}^a\mathbf{x} \right] \dot{\theta}.$$

This equation defines the linear screw coefficient

$$\boxed{{}^a\mathbf{s}_v := -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega}.$$

The coefficient ${}^a\mathbf{s}_v$ is therefore not introduced as an independent translation direction. It packages two effects that do not depend on which moving point P is being evaluated: rotation about an axis that is not passing the frame origin, represented by $-{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell$, and translation along the axis, represented by $h{}^a\mathbf{s}_\omega$. With this definition, the velocity of any moving material point is

$$\boxed{{}^a\mathbf{v}_P = ({}^a\mathbf{s}_v + {}^a\mathbf{s}_\omega \times {}^a\mathbf{x}) \dot{\theta}}.$$

The normalised screw-axis column describes motion per unit angular rate. Multiplying it by the current rate produces the actual twist

$$\boxed{{}^a\boldsymbol{\xi} = {}^a\mathbf{S}\dot{\theta} = \begin{bmatrix} {}^a\mathbf{s}_v\dot{\theta} \\ {}^a\mathbf{s}_\omega\dot{\theta} \end{bmatrix} = \begin{bmatrix} {}^a\boldsymbol{\nu} \\ {}^a\boldsymbol{\omega} \end{bmatrix}},$$

where ${}^a\boldsymbol{\nu} = {}^a\mathbf{s}_v\dot{\theta}$ is the linear velocity-field value at O_a and ${}^a\boldsymbol{\omega} = {}^a\mathbf{s}_\omega\dot{\theta}$ is the angular velocity. Neither ${}^a\mathbf{S}$ nor ${}^a\boldsymbol{\xi}$ contains the position of one selected material point; each represents the complete rigid-motion field. The point position enters only when that field is evaluated. In matrix form,

$$\boxed{\widehat{{}^a\boldsymbol{\xi}} \begin{bmatrix} {}^a\mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} {}^a\boldsymbol{\omega} \times {}^a\mathbf{x} + {}^a\boldsymbol{\nu} \\ 0 \end{bmatrix} = \begin{bmatrix} {}^a\mathbf{v}_P \\ 0 \end{bmatrix}}.$$

Thus the twist represents the common rigid-motion field, its hatted matrix acts as the corresponding operator, and ${}^a\mathbf{x}$ selects the material point at which the field is evaluated. Different points have different linear velocities because the term ${}^a\boldsymbol{\omega} \times {}^a\mathbf{x}$ depends on position, but they share the same twist because they belong to the same rigid motion.

The origin O_a of frame $\{a\}$ has position column ${}^a\mathbf{x} = \mathbf{0}_3$. Mathematically evaluating the velocity field there gives ${}^a\mathbf{s}_v\dot{\theta}$, which is why ${}^a\mathbf{s}_v$ is called the velocity-field coefficient at O_a per unit angular rate. This does not imply that a moving material point or a link-frame origin is located at O_a . If a moving link-frame origin instead has position ${}^a\mathbf{p}_b$, its velocity is $({}^a\mathbf{s}_v + {}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_b)\dot{\theta}$.

Pure prismatic motion is a separate normalisation. It has no angular component, so ${}^a\mathbf{s}_\omega = \mathbf{0}_3$; its normalised linear block is the dimensionless unit translation direction, and multiplying that block by the prismatic rate $\dot{d}$ gives the velocity of every moving point.

Assume that the given transformation has already been validated as an element of $SE(3)$ and write

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3),$$

where $\mathbf{R} \in SO(3)$ and $\mathbf{p} \in \mathbb{R}^3$ is measured in metres. A matrix logarithm seeks a screw axis ${}^a\mathbf{S}$ and signed displacement σ satisfying

$$\boxed{\mathbf{T} = \exp(\widehat{{}^a\mathbf{S}} \sigma), \quad \log(\mathbf{T}) = \widehat{{}^a\mathbf{S}} \sigma \in \mathfrak{se}(3)}. \quad (8.7)$$

The matrix $\log(\mathbf{T})$ and the column ${}^a\mathbf{S}\sigma$ contain the same exponential coordinates in hatted and six-entry representations. The operation $\log(\mathbf{T})$ is a matrix logarithm; taking a scalar logarithm of each entry of $\mathbf{T}$ would not recover a rigid motion.

The recovery has three geometric cases. First, if $\mathbf{R} = \mathbf{I}_3$ and $\mathbf{p} = \mathbf{0}_3$, then $\mathbf{T} = \mathbf{I}_4$. The principal logarithm is the zero matrix:

$$\log(\mathbf{I}_4) = \mathbf{0}_{4 \times 4}.$$

There is no unique normalised screw axis for zero displacement because following any screw through zero distance leaves the identity unchanged.

Second, suppose that $\mathbf{R} = \mathbf{I}_3$ but $\mathbf{p} \neq \mathbf{0}_3$. The principal motion is pure translation. Define

$$d := \|\mathbf{p}\|_2 > 0, \quad {}^a\mathbf{s} := \frac{\mathbf{p}}{d}, \quad {}^a\mathbf{S}_P = \begin{bmatrix} {}^a\mathbf{s} \\ \mathbf{0}_3 \end{bmatrix}.$$

Then $\|{}^a\mathbf{s}\|_2 = 1$, $\mathbf{p} = {}^a\mathbf{s}d$, and the prismatic result Equation 8.6 gives

$$\log(\mathbf{T}) = \widehat{{}^a\mathbf{S}_P} d = \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{p} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

Third, suppose that $\mathbf{R} \neq \mathbf{I}_3$. Euler's rotation theorem from Section 7.5.4 guarantees that there exist a principal angle $\theta \in (0, \pi]$ and a dimensionless unit axis ${}^a\mathbf{s}_\omega \in \mathbb{R}^3$ such that

$$\mathbf{R} = \exp([{}^a\mathbf{s}_\omega]_\times \theta).$$

Physically, the rotation block $\mathbf{R}$ therefore recovers the angular part of the screw description: ${}^a\mathbf{s}_\omega$ is the oriented axis direction and θ is the selected finite rotation about it. The translation block $\mathbf{p}$ must then be used to recover ${}^a\mathbf{s}_v$, the linear velocity-field coefficient at O_a per radian. It should not be interpreted directly as ${}^a\mathbf{s}_v\theta$, because rotation about an axis offset from O_a also contributes to the finite translation column.

After choosing one such principal axis-angle pair, define the corresponding branch-selected $SO(3)$ **logarithm** by

$$\log_{SO(3)}(\mathbf{R}) := [{}^a\mathbf{s}_\omega]_\times \theta \in \mathfrak{so}(3), \quad \exp(\log_{SO(3)}(\mathbf{R})) = \mathbf{R}.$$

For $0 < \theta < \pi$, the principal-angle convention determines the oriented axis. At $\theta = \pi$, the two axis choices ${}^a\mathbf{s}_\omega$ and $-{}^a\mathbf{s}_\omega$ produce different logarithm matrices whose exponentials are the same rotation, so an additional branch convention is required. The identity case was handled separately above. Thus the logarithm used here is defined by a selected axis-angle branch.

Let $\mathbf{\Omega} = [{}^a\mathbf{s}_\omega]_\times$. The translation equation from Equation 8.4 is $\mathbf{p} = \mathbf{G}(\theta){}^a\mathbf{s}_v$. The matrix $\mathbf{G}(\theta)$ accounts for how the linear velocity-field coefficient accumulates while the body rotates through θ . Recovering the coefficient ${}^a\mathbf{s}_v$ from the observed finite translation $\mathbf{p}$ therefore requires the inverse of $\mathbf{G}(\theta)$.

To derive that inverse, define the dimensionless scalars $c_1 := 1 - \cos \theta$ and $c_2 := \theta - \sin \theta$, so

$$\mathbf{G}(\theta) = \theta \mathbf{I}_3 + c_1 \mathbf{\Omega} + c_2 \mathbf{\Omega}^2.$$

Every power of $\mathbf{\Omega}$ reduces to a linear combination of $\mathbf{I}_3$, $\mathbf{\Omega}$, and $\mathbf{\Omega}^2$ because $\mathbf{\Omega}^3 = -\mathbf{\Omega}$. Therefore seek the inverse in the form

$$\mathbf{H} = a\mathbf{I}_3 + b\mathbf{\Omega} + c\mathbf{\Omega}^2, \quad \mathbf{H} \in \mathbb{R}^{3 \times 3}, \quad a, b, c \in \mathbb{R}.$$

Multiplying $\mathbf{H}$ by $\mathbf{G}(\theta)$ and reducing $\mathbf{\Omega}^3$ and $\mathbf{\Omega}^4 = -\mathbf{\Omega}^2$ gives

$$\mathbf{H}\mathbf{G}(\theta) = a\theta\mathbf{I}_3 + [c_1(a - c) + b \sin \theta] \mathbf{\Omega} + [ac_2 + bc_1 + c \sin \theta] \mathbf{\Omega}^2.$$

For $\mathbf{H}$ to be the inverse, the coefficient of $\mathbf{I}_3$ must be one and the other two coefficients must be zero. Hence

$$a\theta = 1, \quad c_1(a - c) + b \sin \theta = 0, \quad ac_2 + bc_1 + c \sin \theta = 0.$$

For the principal rotational branch $\theta \in (0, \pi]$, solving these three scalar equations and using the half-angle identities gives³

$$a = \frac{1}{\theta}, \quad b = -\frac{1}{2}, \quad c = \frac{1}{\theta} - \frac{1}{2} \cot \frac{\theta}{2}.$$

Thus ${}^a\mathbf{s}_v = \mathbf{G}(\theta)^{-1}\mathbf{p}$, with

$${}^a\mathbf{s}_v = \mathbf{G}(\theta)^{-1}\mathbf{p}, \quad \mathbf{G}(\theta)^{-1} = \frac{1}{\theta}\mathbf{I}_3 - \frac{1}{2}\boldsymbol{\Omega} + \left(\frac{1}{\theta} - \frac{1}{2} \cot \frac{\theta}{2}\right) \boldsymbol{\Omega}^2. \quad (8.8)$$

The recovered normalised screw-axis coordinate column is therefore

$${}^a\mathbf{S} = \begin{bmatrix} {}^a\mathbf{s}_v \\ {}^a\mathbf{s}_\omega \end{bmatrix}.$$

Using the linear-first hat definition, its matrix representation is

$$\widehat{{}^a\mathbf{S}} = \begin{bmatrix} \boldsymbol{\Omega} & {}^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \in \mathfrak{se}(3).$$

The hat map is linear, so $\widehat{{}^a\mathbf{S}\theta} = \widehat{{}^a\mathbf{S}}\theta$. Consequently, the branch-selected matrix logarithm is

$$\log(\mathbf{T}) = \widehat{{}^a\mathbf{S}}\theta = \begin{bmatrix} \boldsymbol{\Omega}\theta & {}^a\mathbf{s}_v\theta \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

The physical displacement has already been described completely by $\mathbf{T}$. Its translation column $\mathbf{p}$ is the **net result after the whole finite screw displacement**. To derive how this translation accumulates, let $\varphi \in [0, \theta]$ denote an intermediate signed rotation angle and define the partial screw displacement

$$\mathbf{T}(\varphi) = \exp(\widehat{{}^a\mathbf{S}}\varphi) = \begin{bmatrix} \mathbf{R}(\varphi) & \mathbf{p}(\varphi) \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \mathbf{T}(0) = \mathbf{I}_4, \quad \mathbf{p}(0) = \mathbf{0}_3.$$

³The first equation gives $a = 1/\theta$. Because $c_1 = 1 - \cos \theta > 0$, the second gives $c = a + b \sin \theta / c_1$. Substitution into the third, together with $a\theta = 1$, gives $1 + b(c_1 + \sin^2 \theta / c_1) = 0$. The identity $c_1^2 + \sin^2 \theta = 2c_1$ then gives $b = -1/2$. Finally, $\sin \theta / (1 - \cos \theta) = \cot(\theta/2)$ gives $c = 1/\theta - \frac{1}{2} \cot(\theta/2)$.

The matrix $\mathbf{R}(\varphi) \in SO(3)$ and column $\mathbf{p}(\varphi) \in \mathbb{R}^3$ are the rotation and translation blocks accumulated up to angle φ . Because the generator $\widehat{^a\mathbf{S}}$ is constant, differentiating the matrix exponential gives

$$\frac{d\mathbf{T}}{d\varphi} = \mathbf{T}(\varphi) \widehat{^a\mathbf{S}}.$$

Substituting the block forms and multiplying them gives

$$\begin{aligned} \frac{d\mathbf{T}}{d\varphi} &= \begin{bmatrix} \mathbf{R}(\varphi) & \mathbf{p}(\varphi) \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{\Omega} & ^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}(\varphi)\boldsymbol{\Omega} & \mathbf{R}(\varphi)^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 0 \end{bmatrix}. \end{aligned}$$

The upper-right block on the left is $d\mathbf{p}/d\varphi$. Equating the upper-right blocks and using $\mathbf{R}(\varphi) = \exp(\boldsymbol{\Omega}\varphi)$ therefore gives

$$\boxed{\frac{d\mathbf{p}}{d\varphi} = \mathbf{R}(\varphi)^a\mathbf{s}_v = \exp(\boldsymbol{\Omega}\varphi)^a\mathbf{s}_v.}$$

Thus the rotational part continually changes the direction in which the linear screw contribution accumulates. Integrating this differential equation from the initial condition $\mathbf{p}(0) = \mathbf{0}_3$ to the final angle θ gives

$$\mathbf{p} = \int_0^\theta \exp(\boldsymbol{\Omega}\varphi)^a\mathbf{s}_v d\varphi.$$

The matrix $\boldsymbol{\Omega}$ and the column $^a\mathbf{s}_v$ are constant with respect to φ . Rodrigues' formula for the unit axis $^a\mathbf{s}_\omega$ gives

$$\exp(\boldsymbol{\Omega}\varphi) = \mathbf{I}_3 + \sin \varphi \boldsymbol{\Omega} + (1 - \cos \varphi) \boldsymbol{\Omega}^2.$$

The constant matrices can therefore be taken outside their scalar integrals:

$$\begin{aligned} \int_0^\theta \exp(\boldsymbol{\Omega}\varphi) d\varphi &= \left(\int_0^\theta 1 d\varphi \right) \mathbf{I}_3 + \left(\int_0^\theta \sin \varphi d\varphi \right) \boldsymbol{\Omega} + \left(\int_0^\theta (1 - \cos \varphi) d\varphi \right) \boldsymbol{\Omega}^2 \\ &= \theta \mathbf{I}_3 + [-\cos \varphi]_0^\theta \boldsymbol{\Omega} + [\varphi - \sin \varphi]_0^\theta \boldsymbol{\Omega}^2 \\ &= \theta \mathbf{I}_3 + (1 - \cos \theta) \boldsymbol{\Omega} + (\theta - \sin \theta) \boldsymbol{\Omega}^2 \\ &= \mathbf{G}(\theta). \end{aligned}$$

Because ${}^a\mathbf{s}_v$ is also constant with respect to φ , it can be placed outside the integral. Hence

$$\mathbf{p} = \left(\int_0^\theta \exp(\mathbf{\Omega}\varphi) d\varphi \right) {}^a\mathbf{s}_v = \mathbf{G}(\theta) {}^a\mathbf{s}_v.$$

Consequently, $\mathbf{G}(\theta)^{-1}$ does not move the body and does not describe another transformation. It only solves this accumulated-displacement equation backwards: after $\mathbf{R}$, $\mathbf{p}$, θ , and ${}^a\mathbf{s}_\omega$ have been obtained from the final pose, $\mathbf{G}(\theta)^{-1}\mathbf{p}$ recovers the constant linear screw coefficient ${}^a\mathbf{s}_v$ that generated that finite translation column. For example, a pure rotation about an axis that does not pass through O_a has zero pitch but generally has $\mathbf{p} \neq \mathbf{0}_3$, because the represented body or frame revolves around the offset axis. The inverse separates this offset-axis effect from the recovered screw description; it does not undo the physical displacement.

When the recovered motion has finite pitch, its axis geometry can also be reconstructed. Define

$$h := ({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_v \in \mathbb{R}$$

in metres per radian. To see why this scalar is the pitch, recall the geometric decomposition

$${}^a\mathbf{s}_v = \underbrace{-{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell}_{\text{offset-axis contribution}} + \underbrace{h {}^a\mathbf{s}_\omega}_{\text{motion along the axis per radian}}.$$

The construction in Figure 8.3 uses ${}^a\mathbf{s}_\omega = \mathbf{e}_z$, ${}^a\mathbf{p}_\ell = 3\mathbf{e}_x$ m, $h = 1$ m rad⁻¹, and a 58° oblique view to show the vector sum and its cancellation at the axis point.

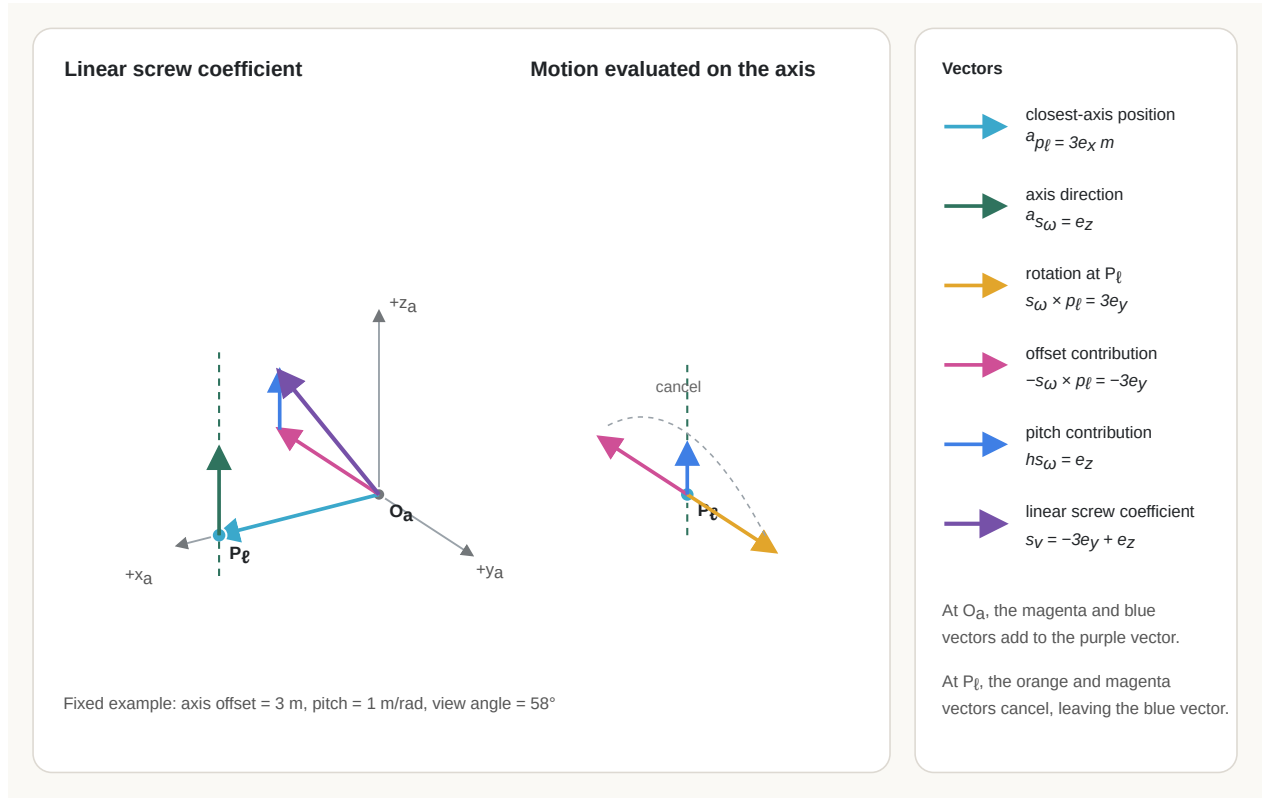


Figure 8.3: Decomposition of the linear screw coefficient into the offset-axis and pitch contributions, with their cancellation at a point on the screw axis.

The first term is perpendicular to the axis: it accounts for rotation about a line that is offset from O_a and contributes no motion along that line. The second term is parallel to the axis and has signed magnitude h because ${}^a s_\omega$ is a unit column. Therefore projecting ${}^a s_v$ onto the axis removes the perpendicular offset contribution and retains only the signed axial displacement per radian:

$$\begin{aligned} ({}^a s_\omega)^\top {}^a s_v &= -({}^a s_\omega)^\top ({}^a s_\omega \times {}^a p_\ell) + h({}^a s_\omega)^\top {}^a s_\omega \\ &= 0 + h(1) \\ &= h. \end{aligned}$$

Equivalently, a point on the screw axis has no tangential velocity caused by rotation about that same axis; per positive radian it advances only by $h {}^a s_\omega$. Thus $h > 0$ means translation in the chosen positive axis direction, $h < 0$ means translation in the opposite direction, and $h = 0$ gives pure revolute motion.

A convenient unique point on the axis is the point P_ℓ closest to O_a . Its position column is perpendicular to the unit axis direction, so

$$({}^a s_\omega)^\top {}^a p_\ell = 0, \quad ({}^a s_\omega)^\top {}^a s_\omega = 1.$$

To recover this point from the screw coordinates, cross the screw relation with ${}^a\mathbf{s}_\omega$ on the left:

$$\begin{aligned} {}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_v &= -{}^a\mathbf{s}_\omega \times ({}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell) + h{}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_\omega \\ &= -{}^a\mathbf{s}_\omega \times ({}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell), \end{aligned}$$

because a column crossed with itself is zero. The vector triple-product identity

$$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a}^\top \mathbf{c}) - \mathbf{c}(\mathbf{a}^\top \mathbf{b})$$

gives

$${}^a\mathbf{s}_\omega \times ({}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell) = {}^a\mathbf{s}_\omega (({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{p}_\ell) - {}^a\mathbf{p}_\ell (({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_\omega) = -{}^a\mathbf{p}_\ell.$$

Substituting this result into the crossed screw relation yields

$${}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_v = -(-{}^a\mathbf{p}_\ell) = {}^a\mathbf{p}_\ell.$$

Hence the closest axis point is

$$\boxed{{}^a\mathbf{p}_\ell = {}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_v},$$

measured in metres. Other points on the axis are obtained by adding $\lambda {}^a\mathbf{s}_\omega$ for any $\lambda \in \mathbb{R}$ measured in metres.

The logarithm requires a branch convention because finite rotations repeat after integer multiples of 2π . The principal convention above chooses $\theta \in [0, \pi]$, uses zero for the identity, and uses pure translation when $\mathbf{R} = \mathbf{I}_3$ but $\mathbf{p} \neq \mathbf{0}_3$. Even with that convention, the rotation axis needs special handling at $\theta = \pi$, and direct evaluation of $\mathbf{G}(\theta)^{-1}$ suffers cancellation for very small nonzero θ . Section 8.5 owns the stable small-angle, near- π , validation, and deterministic branch policies; the equations here establish the mathematical result those algorithms must preserve.

Quick test B: the $SE(3)$ exponential and logarithm

1. **Contrast:** Distinguish a normalised screw-axis coordinate column ${}^a\mathbf{S}$, an exponential-coordinate column ${}^a\mathbf{S}\sigma$, its hatted matrix, and the finite displacement $\mathbf{T}_\Delta(\sigma)$.
2. **Derivation:** For a rotational screw, show by block multiplication that the upper-right block of $(\widehat{{}^a\mathbf{S}})^k$ is $\boldsymbol{\Omega}^{k-1} {}^a\mathbf{s}_v$ for every integer $k \geq 1$.
3. **Geometry:** Starting from ${}^a\mathbf{s}_v = -\boldsymbol{\Omega}^a \mathbf{p}_\ell + h^a \mathbf{s}$, derive the finite translation $(\mathbf{I}_3 - \mathbf{R})^a \mathbf{p}_\ell + h\theta^a \mathbf{s}$.

4. **Application:** A revolute axis points along positive z_a and passes through $[1 \ 0 \ 0]^T$ m. Calculate the transformation for a positive rotation of $\pi/2$ rad and verify that the selected axis point is fixed.
5. **Prismatic case:** Why does the exponential series for a prismatic screw stop after its first-order term? What finite translation results from ${}^a\mathbf{s} = [0 \ -1 \ 0]^T$ and $d = 0.4$ m?
6. **Inverse problem:** State the three principal logarithm cases for a transformation $\mathbf{T} = [\mathbf{R} \ \mathbf{p}; \mathbf{0}_3^T \ 1]$.
7. **Limitation:** Why are special numerical branches needed near zero rotation and at a rotation of π ?

Answers and hints

1. ${}^a\mathbf{S}$ is the frame-dependent instantaneous motion per unit scalar rate; ${}^a\mathbf{S}\sigma$ adds the finite amount of motion; the hat operator writes those six coordinates as a structured element of $\mathfrak{se}(3)$; and the matrix exponential maps that element to a finite transformation in $SE(3)$.
2. The statement is true for $k = 1$. Multiplying $[\Omega^k \ \Omega^{k-1} {}^a\mathbf{s}_v; \mathbf{0}_3^T \ 0]$ by $[\Omega \ {}^a\mathbf{s}_v; \mathbf{0}_3^T \ 0]$ produces $[\Omega^{k+1} \ \Omega^k {}^a\mathbf{s}_v; \mathbf{0}_3^T \ 0]$, which proves the pattern by induction.
3. Use $\mathbf{G}\Omega = \mathbf{R} - \mathbf{I}_3$ and $\mathbf{G}^a\mathbf{s} = \theta^a\mathbf{s}$, then distribute $\mathbf{G}$ over the two terms in ${}^a\mathbf{s}_v$.
4. This is a pure revolute motion, so $h = 0$. The unit axis direction and selected axis point are

$${}^a\mathbf{s}_\omega = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad {}^a\mathbf{p}_\ell = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m.}$$

The linear screw coefficient is

$${}^a\mathbf{s}_v = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell = - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} \text{ m rad}^{-1}.$$

A positive rotation of $\theta = \pi/2$ about z_a has rotation matrix

$$\mathbf{R}(\pi/2) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Because $h = 0$, the translation block for an axis through ${}^a\mathbf{p}_\ell$ is

$$\begin{aligned}
{}^a\mathbf{p}_\Delta &= (\mathbf{I}_3 - \mathbf{R}) {}^a\mathbf{p}_\ell \\
&= \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \\
&= \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \text{ m.}
\end{aligned}$$

Therefore the finite transformation is

$$\mathbf{T}_{\Delta,R}(\pi/2) = \begin{bmatrix} 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

To verify the selected axis point, apply the transformation to its homogeneous position column:

$$\mathbf{T}_{\Delta,R}(\pi/2) \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

The point therefore remains at $[1 \ 0 \ 0]^\top$ m, as every point on the axis must for a pure revolute motion.

5. The normalised prismatic screw has unit translation direction ${}^a\mathbf{s} = [0 \ -1 \ 0]^\top$ and zero angular block:

$${}^a\mathbf{s}_P = \begin{bmatrix} 0 \\ -1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \widehat{{}^a\mathbf{s}_P} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Block multiplication gives

$$\left(\widehat{{}^a\mathbf{s}_P}\right)^2 = \mathbf{0}_{4 \times 4}.$$

Every higher power also vanishes. Consequently, the matrix-exponential series

$$\exp(\widehat{{}^a\mathbf{S}_P}d) = \mathbf{I}_4 + \widehat{{}^a\mathbf{S}_P}d + \frac{(\widehat{{}^a\mathbf{S}_P}d)^2}{2!} + \dots$$

stops after its first-order term. For $d = 0.4$ m,

$$\mathbf{T}_{\Delta,P}(0.4) = \mathbf{I}_4 + \widehat{{}^a\mathbf{S}_P}(0.4) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Thus there is no rotation, and every material point is translated by

$${}^a\mathbf{s}d = \begin{bmatrix} 0 \\ -0.4 \\ 0 \end{bmatrix} \text{ m},$$

which is 0.4 m in the negative y_a direction.

6. Examine $\mathbf{R}$ and $\mathbf{p}$ and select one of three cases.

Identity: If $\mathbf{R} = \mathbf{I}_3$ and $\mathbf{p} = \mathbf{0}_3$, then $\mathbf{T} = \mathbf{I}_4$ and $\log(\mathbf{T}) = \mathbf{0}_{4 \times 4}$. The displacement is zero, so no unique normalised screw axis exists: every screw axis produces the identity when multiplied by zero motion.

Pure translation: If $\mathbf{R} = \mathbf{I}_3$ but $\mathbf{p} \neq \mathbf{0}_3$, define

$$d = \|\mathbf{p}\|_2, \quad {}^a\mathbf{s} = \frac{\mathbf{p}}{d}.$$

Here $d > 0$ is the translation distance and ${}^a\mathbf{s}$ is its unit direction. The normalised prismatic screw is ${}^a\mathbf{S}_P = [{}^a\mathbf{s}^\top \mathbf{0}_3^\top]^\top$, and

$$\log(\mathbf{T}) = \widehat{{}^a\mathbf{S}_P}d = \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{p} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

Rotation present: If $\mathbf{R} \neq \mathbf{I}_3$, use the declared principal branch to recover $\theta \in (0, \pi]$ and a unit oriented axis ${}^a\mathbf{s}_\omega$ such that $\mathbf{R} = \exp([{}^a\mathbf{s}_\omega]_\times \theta)$. Then recover the linear screw coefficient from the translation column:

$${}^a\mathbf{s}_v = \mathbf{G}(\theta)^{-1}\mathbf{p}.$$

With $\mathbf{\Omega} = [{}^a\mathbf{s}_\omega]_\times$ and ${}^a\mathbf{S} = [{}^a\mathbf{s}_v^\top \ {}^a\mathbf{s}_\omega^\top]^\top$, the logarithm is

$$\log(\mathbf{T}) = \widehat{{}^a\mathbf{S}} \theta = \begin{bmatrix} \mathbf{\Omega} \theta & {}^a\mathbf{s}_v \theta \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

This case represents pure rotation when the recovered pitch is zero and helical motion when it is nonzero.

7. Near zero, subtracting nearly equal trigonometric quantities causes loss of floating-point precision. At $\theta = \pi$, formulas that divide by $\sin \theta$ cannot recover the rotation axis. Both cases need algebraically equivalent branch-specific formulas.

8.3.4 From constrained motion to finite exponential coordinates

A rigid link's physical configuration specifies the world position of every labelled material point on that link. A pose matrix represents the same configuration through the position and orientation of a frame rigidly attached to the link. A one-degree-of-freedom joint restricts the relative configurations of two links to a one-parameter family: the scalar parameter is a rotation angle θ for a revolute or helical joint and a translation distance d for a prismatic joint.

Conceptually, the physical motion comes first. Differentiating the motion of every material point produces a rigid velocity field. That field has one common angular coefficient and one common linear coefficient, so it can be represented compactly by a twist. Conversely, once the twist is known, the velocity of any selected point can be recovered by evaluating the field at that point.

The joint's physical geometry determines how that scalar motion moves the complete rigid link. For a rotational or finite-pitch helical motion, an oriented axis direction ${}^a\mathbf{s}_\omega$, an axis point ${}^a\mathbf{p}_\ell$, and a pitch h produce the normalised screw-axis coordinate column

$${}^a\mathbf{S}_H = \begin{bmatrix} -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h {}^a\mathbf{s}_\omega \\ {}^a\mathbf{s}_\omega \end{bmatrix}.$$

Setting $h = 0$ gives a pure revolute screw. A prismatic joint with unit translation direction ${}^a\mathbf{s}$ instead has

$${}^a\mathbf{S}_P = \begin{bmatrix} {}^a\mathbf{s} \\ \mathbf{0}_3 \end{bmatrix}.$$

The screw-axis column describes the permitted rigid motion per unit scalar rate; it does not describe the velocity of only one point. Let σ denote the appropriate joint-motion

parameter, so $\sigma = \theta$ for rotational or helical motion and $\sigma = d$ for prismatic motion. Multiplication by the current rate gives the actual twist

$${}^a\xi = {}^a\mathbf{S}\dot{\sigma} = \begin{bmatrix} {}^a\nu \\ {}^a\omega \end{bmatrix}.$$

The twist represents one instantaneous rigid-motion velocity field. For a material point P currently at ${}^a\mathbf{x}$, its velocity is obtained by evaluating that field at the point:

$${}^a\mathbf{v}_P = {}^a\nu + {}^a\omega \times {}^a\mathbf{x}, \quad \begin{bmatrix} {}^a\mathbf{v}_P \\ 0 \end{bmatrix} = \widehat{{}^a\xi} \begin{bmatrix} {}^a\mathbf{x} \\ 1 \end{bmatrix}.$$

Different material points have different linear velocities because their position columns ${}^a\mathbf{x}$ differ, but all points on the same rigid link share the same twist. This is why the point position belongs in the evaluation equation rather than in ${}^a\mathbf{S}$ or ${}^a\xi$.

A twist is instantaneous and is not itself a pose. To obtain a finite displacement, its motion must be accumulated. When the normalised screw generator ${}^a\mathbf{S}$ remains constant over a signed motion amount σ , the $SE(3)$ exponential performs that accumulation:

$$\mathbf{T}_\Delta(\sigma) = \exp(\widehat{{}^a\mathbf{S}}\sigma) \in SE(3).$$

If the actual twist ${}^a\xi$ is constant for a time interval Δt and $\sigma = \dot{\sigma}\Delta t$, then $\widehat{{}^a\mathbf{S}}\sigma = \widehat{{}^a\xi}\Delta t$, so the same update can be written as

$$\mathbf{T}_\Delta(\dot{\sigma}\Delta t) = \exp(\widehat{{}^a\xi}\Delta t).$$

This is the precise sense in which a constant twist produces a finite transformation: the exponential integrates the instantaneous motion over the interval. A time-varying twist cannot generally be replaced by one ordinary exponential without an integration rule.

The resulting matrix is a finite rigid-displacement operator. The same transformation acts on every material point, although different points generally have different displacement vectors when rotation is present:

$${}^a\bar{\mathbf{x}}' = \mathbf{T}_\Delta(\sigma){}^a\bar{\mathbf{x}}, \quad {}^a\bar{\mathbf{x}} = \begin{bmatrix} {}^a\mathbf{x} \\ 1 \end{bmatrix}.$$

If a moving frame initially coincides with $\{a\}$, $\mathbf{T}_\Delta(\sigma)$ is also its pose relative to $\{a\}$ after the displacement. If the moving frame already has a non-identity pose, the displacement

must be composed with that pose on the correct side; the space-versus-body distinction in Section 8.4 determines that multiplication order.

The logarithm follows the reverse representational direction. A logarithm branch is a declared rule for choosing one logarithm when the same transformation has several valid logarithms. Given a validated finite transformation and a selected branch, the logarithm recovers exponential coordinates for one constant screw path:

$$\log(\mathbf{T}_\Delta) = \widehat{{}^a\boldsymbol{\eta}} = \widehat{{}^a\mathbf{S}}\sigma.$$

Here the finite exponential-coordinate column ${}^a\boldsymbol{\eta} \in \mathbb{R}^6$ is

$${}^a\boldsymbol{\eta} := {}^a\mathbf{S}\sigma = \begin{bmatrix} {}^a\boldsymbol{\rho} \\ {}^a\phi \end{bmatrix}$$

The upper block ${}^a\boldsymbol{\rho} \in \mathbb{R}^3$ is the finite linear exponential-coordinate block, measured in metres. The lower block ${}^a\phi \in \mathbb{R}^3$ is the finite rotation-vector block, measured in radians. Both blocks are expressed in frame $\{a\}$. The logarithm does not prove that the body followed the recovered screw path in the past; it identifies one constant screw displacement with the same initial and final poses.

8.4 Constant-twist pose integration

The preceding section constructed a finite relative displacement

$$\mathbf{T}_\Delta(\sigma) = \exp(\hat{\mathbf{S}}\sigma),$$

whose exponential path begins at $\mathbf{I}_4$ when $\sigma = 0$. A moving link, however, usually has an existing pose $\mathbf{T}_{sb}(t_0) \neq \mathbf{I}_4$. We must therefore determine how the displacement increment composes with that existing pose. The multiplication side is determined by the frame in which the twist is represented.

Let $t_0, t_1 \in \mathbb{R}$ be times in seconds with $t_1 \geq t_0$, and define $\Delta t := t_1 - t_0$. Let $\{s\}$ be a fixed space frame and let $\{b\}$ be a frame rigidly attached to the moving link. Its differentiable pose trajectory is

$$\mathbf{T}_{sb}(t) = \begin{bmatrix} \mathbf{R}_{sb}(t) & {}^s\mathbf{p}_b(t) \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3),$$

where $\mathbf{R}_{sb}(t) \in SO(3)$ is the orientation of $\{b\}$ relative to $\{s\}$ and ${}^s\mathbf{p}_b(t) \in \mathbb{R}^3$ is the position of O_b relative to O_s , expressed in $\{s\}$ and measured in metres. The matrix $\mathbf{T}_{sb}$

transforms point coordinates from $\{b\}$ to $\{s\}$. The increment labels ${}^s\mathbf{T}_\Delta$ and ${}^b\mathbf{T}_\Delta$ introduced below state whether an active displacement is represented using space- or body-referred screw coordinates; unlike the two-frame subscript sb , a single leading label is not a source-to-destination frame mapping.

Recall the project's linear-first body- and space-twist definitions from Section 7.9.2 and Section 7.9.3:

$$\hat{\xi}_b := \mathbf{T}_{sb}^{-1} \dot{\mathbf{T}}_{sb}, \quad \hat{\xi}_s := \dot{\mathbf{T}}_{sb} \mathbf{T}_{sb}^{-1}.$$

Here $\xi_b = [\nu_b^\top \omega_b^\top]^\top \in \mathbb{R}^6$ is the body twist and $\xi_s = [\nu_s^\top \omega_s^\top]^\top \in \mathbb{R}^6$ is the space twist. Their linear entries are measured in m s^{-1} and their angular entries in rad s^{-1} . The subscript states the representation: the body-twist components use the moving body axes and body origin, whereas the space-twist components use the fixed space axes and space origin. It does not state which observer measured the motion.

Rearranging the two definitions gives two equivalent pose-rate equations:

$$\dot{\mathbf{T}}_{sb} = \mathbf{T}_{sb} \hat{\xi}_b = \hat{\xi}_s \mathbf{T}_{sb}. \quad (8.9)$$

These equations describe the same physical motion. They differ only in the coordinates and reference origin used for its instantaneous velocity field.

8.4.1 Space-twist pose updates

Suppose first that the space twist is constant throughout $[t_0, t_1]$:

$$\xi_s(t) = \xi_s, \quad t \in [t_0, t_1].$$

Here *constant* means that the six numerical components referred to the fixed frame $\{s\}$ do not change with time. It does not mean that the pose is constant or that every material point has constant linear velocity. For example, a constant rotational twist produces circular point paths whose velocity directions change.

The space-twist form of Equation 8.9 is the matrix differential equation

$$\dot{\mathbf{T}}_{sb}(t) = \hat{\xi}_s \mathbf{T}_{sb}(t). \quad (8.10)$$

Its block form exposes the physical meaning of the space-twist components. Write its angular and linear blocks as $[\omega_s]_\times$ and ν_s , respectively. Then

$$\begin{aligned}\dot{\mathbf{T}}_{sb} &= \begin{bmatrix} [\boldsymbol{\omega}_s]_{\times} & \boldsymbol{\nu}_s \\ \mathbf{0}_3^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{sb} & {}^s\mathbf{p}_b \\ \mathbf{0}_3^T & 1 \end{bmatrix} \\ &= \begin{bmatrix} [\boldsymbol{\omega}_s]_{\times} \mathbf{R}_{sb} & \boldsymbol{\nu}_s + \boldsymbol{\omega}_s \times {}^s\mathbf{p}_b \\ \mathbf{0}_3^T & 0 \end{bmatrix}.\end{aligned}$$

Equating blocks gives

$$\boxed{\dot{\mathbf{R}}_{sb} = [\boldsymbol{\omega}_s]_{\times} \mathbf{R}_{sb}, \quad {}^s\dot{\mathbf{p}}_b = \boldsymbol{\nu}_s + \boldsymbol{\omega}_s \times {}^s\mathbf{p}_b.} \quad (8.11)$$

Thus $\boldsymbol{\omega}_s$ is the angular velocity expressed along the fixed space axes. The column $\boldsymbol{\nu}_s$ is the velocity field evaluated at O_s , not generally the velocity of O_b ; evaluating the same field at the current body-origin position ${}^s\mathbf{p}_b$ adds the rotational term $\boldsymbol{\omega}_s \times {}^s\mathbf{p}_b$.

To solve Equation 8.10, consider the candidate trajectory

$$\mathbf{T}_{sb}(t) = \exp(\hat{\boldsymbol{\xi}}_s(t - t_0)) \mathbf{T}_{sb}(t_0).$$

Because $\hat{\boldsymbol{\xi}}_s$ is constant, differentiation gives

$$\begin{aligned}\frac{d}{dt} [\exp(\hat{\boldsymbol{\xi}}_s(t - t_0)) \mathbf{T}_{sb}(t_0)] &= \hat{\boldsymbol{\xi}}_s \exp(\hat{\boldsymbol{\xi}}_s(t - t_0)) \mathbf{T}_{sb}(t_0) \\ &= \hat{\boldsymbol{\xi}}_s \mathbf{T}_{sb}(t).\end{aligned}$$

At $t = t_0$, the exponential is $\mathbf{I}_4$, so the candidate also satisfies the initial pose. The solution is unique. To see this directly, let $\mathbf{D}(t)$ be the difference between any two solutions with the same initial pose. Then $\dot{\mathbf{D}} = \hat{\boldsymbol{\xi}}_s \mathbf{D}$ and $\mathbf{D}(t_0) = \mathbf{0}_{4 \times 4}$. Because $\hat{\boldsymbol{\xi}}_s$ is constant, the product rule gives

$$\begin{aligned}\frac{d}{dt} [\exp(-\hat{\boldsymbol{\xi}}_s(t - t_0)) \mathbf{D}(t)] &= -\hat{\boldsymbol{\xi}}_s \exp(-\hat{\boldsymbol{\xi}}_s(t - t_0)) \mathbf{D}(t) + \exp(-\hat{\boldsymbol{\xi}}_s(t - t_0)) \dot{\mathbf{D}}(t) \\ &= \exp(-\hat{\boldsymbol{\xi}}_s(t - t_0)) (-\hat{\boldsymbol{\xi}}_s \mathbf{D}(t) + \dot{\mathbf{D}}(t)) \\ &= \mathbf{0}_{4 \times 4}.\end{aligned}$$

The second equality uses the fact that a matrix commutes with its own exponential, and the last equality uses $\dot{\mathbf{D}} = \hat{\boldsymbol{\xi}}_s \mathbf{D}$. Therefore the product remains equal to its initial value $\mathbf{0}_{4 \times 4}$; because the exponential is invertible, $\mathbf{D}(t) = \mathbf{0}_{4 \times 4}$ for the complete interval. The candidate is therefore the required solution. Evaluating it at t_1 gives the exact constant-space-twist update

$$\mathbf{T}_{sb}(t_1) = \underbrace{\exp(\hat{\xi}_s \Delta t)}_{{}^s\mathbf{T}_\Delta} \mathbf{T}_{sb}(t_0). \quad (8.12)$$

The increment ${}^s\mathbf{T}_\Delta \in SE(3)$ is expressed relative to the fixed space frame and therefore multiplies the current pose on the left. If $\hat{\xi}_s = {}^s\mathbf{S}\dot{\sigma}$ for a constant normalised screw ${}^s\mathbf{S}$ and constant scalar rate $\dot{\sigma}$, then $\Delta\sigma = \dot{\sigma}\Delta t$ and

$$\exp(\hat{\xi}_s \Delta t) = \exp(\widehat{{}^s\mathbf{S}} \Delta\sigma).$$

Writing the actual twist and the normalised screw in linear-first components makes this equality explicit:

$$\xi_s = \begin{bmatrix} \nu_s \\ \omega_s \end{bmatrix} = \begin{bmatrix} {}^s\mathbf{s}_v \dot{\sigma} \\ {}^s\mathbf{s}_\omega \dot{\sigma} \end{bmatrix}, \quad {}^s\mathbf{S} = \begin{bmatrix} {}^s\mathbf{s}_v \\ {}^s\mathbf{s}_\omega \end{bmatrix}.$$

Therefore, using $\Delta\sigma = \dot{\sigma}\Delta t$ and the linearity of the hat and cross-product maps,

$$\begin{aligned} \hat{\xi}_s \Delta t &= \begin{bmatrix} [\omega_s]_\times \Delta t & \nu_s \Delta t \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} [{}^s\mathbf{s}_\omega]_\times \Delta\sigma & {}^s\mathbf{s}_v \Delta\sigma \\ \mathbf{0}_3^\top & 0 \end{bmatrix} = \widehat{{}^s\mathbf{S}} \Delta\sigma. \end{aligned}$$

For a rotational or helical screw, $\Delta\sigma$ is an angular displacement in radians, ${}^s\mathbf{s}_\omega$ is dimensionless, and ${}^s\mathbf{s}_v$ is measured in metres per radian. For a prismatic screw, $\Delta\sigma$ is a displacement in metres, ${}^s\mathbf{s}_v$ is a dimensionless unit direction, and ${}^s\mathbf{s}_\omega = \mathbf{0}_3$.

This equality connects time integration of an actual twist to the finite screw displacement developed in Section 8.3.

8.4.2 Body-twist pose updates

Now suppose that the body twist is constant throughout the same interval:

$$\xi_b(t) = \xi_b, \quad t \in [t_0, t_1].$$

Here *constant* means that the six numerical components expressed along the moving axes of $\{b\}$ do not change. The body-twist form of Equation 8.9 is

$$\dot{\mathbf{T}}_{sb}(t) = \mathbf{T}_{sb}(t) \hat{\xi}_b. \quad (8.13)$$

Write the angular and linear blocks of $\hat{\xi}_b$ as $[\omega_b]_\times$ and ν_b , respectively. Block multiplication gives

$$\begin{aligned}\dot{\mathbf{T}}_{sb} &= \begin{bmatrix} \mathbf{R}_{sb} & {}^s\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} [\omega_b]_\times & \nu_b \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_{sb}[\omega_b]_\times & \mathbf{R}_{sb}\nu_b \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.\end{aligned}$$

Therefore

$$\boxed{\dot{\mathbf{R}}_{sb} = \mathbf{R}_{sb}[\omega_b]_\times, \quad {}^s\dot{\mathbf{p}}_b = \mathbf{R}_{sb}\nu_b.} \quad (8.14)$$

The column ω_b is the angular velocity expressed along the current body axes. The column ν_b is the world-relative velocity of the body origin O_b , also expressed along the current body axes; multiplication by $\mathbf{R}_{sb}$ converts it to the space-coordinate velocity ${}^s\dot{\mathbf{p}}_b$.

The candidate solution of Equation 8.13 is

$$\mathbf{T}_{sb}(t) = \mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b(t - t_0)).$$

Differentiating verifies the multiplication side:

$$\begin{aligned}\frac{d}{dt} [\mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b(t - t_0))] &= \mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b(t - t_0)) \hat{\xi}_b \\ &= \mathbf{T}_{sb}(t) \hat{\xi}_b.\end{aligned}$$

The candidate equals $\mathbf{T}_{sb}(t_0)$ at $t = t_0$. Uniqueness follows by applying the same difference argument to $\mathbf{D}(t) \exp(-\hat{\xi}_b(t - t_0))$, whose derivative is zero. The exact constant-body-twist update is therefore

$$\boxed{\mathbf{T}_{sb}(t_1) = \mathbf{T}_{sb}(t_0) \underbrace{\exp(\hat{\xi}_b \Delta t)}_{{}^b\mathbf{T}_\Delta}.} \quad (8.15)$$

The increment ${}^b\mathbf{T}_\Delta \in SE(3)$ is expressed relative to the moving body frame and therefore multiplies the current pose on the right. If $\hat{\xi}_b = {}^b\mathbf{S}\dot{\sigma}$ with constant rate, then the same update can be written with $\exp(\widehat{{}^b\mathbf{S}} \Delta\sigma)$.

8.4.3 Multiplication side and physical interpretation

The side of multiplication is a consequence of transformation composition. Let P be a material point rigidly attached to the moving link, and let ${}^b\bar{\mathbf{x}}_P$ be its constant homogeneous position column in the body frame. At t_0 its space-frame position is

$${}^s\bar{\mathbf{x}}_P(t_0) = \mathbf{T}_{sb}(t_0){}^b\bar{\mathbf{x}}_P.$$

A space-referred increment acts on the current space-frame position, so

$${}^s\bar{\mathbf{x}}_P(t_1) = {}^s\mathbf{T}_\Delta {}^s\bar{\mathbf{x}}_P(t_0) = {}^s\mathbf{T}_\Delta \mathbf{T}_{sb}(t_0){}^b\bar{\mathbf{x}}_P.$$

Because this equation must hold for every body-fixed point, the updated pose is ${}^s\mathbf{T}_\Delta \mathbf{T}_{sb}(t_0)$: the space increment appears on the left. A body-referred increment is first interpreted along the body axes and then mapped into space by the current pose:

$${}^s\bar{\mathbf{x}}_P(t_1) = \mathbf{T}_{sb}(t_0){}^b\mathbf{T}_\Delta {}^b\bar{\mathbf{x}}_P.$$

It therefore appears on the right. The two rules are

space-referred increment: $\mathbf{T}_{sb}(t_1) = {}^s\mathbf{T}_\Delta \mathbf{T}_{sb}(t_0),$ body-referred increment: $\mathbf{T}_{sb}(t_1) = \mathbf{T}_{sb}(t_0){}^b\mathbf{T}_\Delta.$	(8.16)
--	--------

These are not two competing updates for the same numerical six-vector. The space and body twists representing the same physical motion are related by the adjoint from Section 7.9.4:

$\xi_s(t) = \text{Ad}_{\mathbf{T}_{sb}(t)} \xi_b(t), \quad \hat{\xi}_s(t) = \mathbf{T}_{sb}(t)\hat{\xi}_b(t)\mathbf{T}_{sb}(t)^{-1}.$

For a constant-twist trajectory, it is sufficient to convert the generators at the initial pose:

$$\hat{\xi}_s = \mathbf{T}_{sb}(t_0)\hat{\xi}_b\mathbf{T}_{sb}(t_0)^{-1}.$$

The matrix exponential preserves this similarity transformation. Indeed, for every integer $k \geq 0$,

$$\left(\mathbf{T}_{sb}(t_0)\hat{\xi}_b\mathbf{T}_{sb}(t_0)^{-1}\right)^k = \mathbf{T}_{sb}(t_0)\hat{\xi}_b^k\mathbf{T}_{sb}(t_0)^{-1},$$

because every adjacent pair $\mathbf{T}_{sb}(t_0)^{-1}\mathbf{T}_{sb}(t_0)$ cancels. Substitution into the exponential series gives

$$\exp(\hat{\xi}_s \Delta t) = \mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b \Delta t) \mathbf{T}_{sb}(t_0)^{-1}.$$

Therefore

$$\begin{aligned} \exp(\hat{\xi}_s \Delta t) \mathbf{T}_{sb}(t_0) &= \mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b \Delta t) \mathbf{T}_{sb}(t_0)^{-1} \mathbf{T}_{sb}(t_0) \\ &= \mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b \Delta t). \end{aligned}$$

Thus the left and right updates produce exactly the same final pose when their twist coordinates have been converted consistently.

The conversion also remains valid throughout the trajectory. Substituting $\mathbf{T}_{sb}(t) = \mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b(t - t_0))$ gives

$$\begin{aligned} \mathbf{T}_{sb}(t) \hat{\xi}_b \mathbf{T}_{sb}(t)^{-1} &= \mathbf{T}_{sb}(t_0) \exp(\hat{\xi}_b(t - t_0)) \hat{\xi}_b \exp(-\hat{\xi}_b(t - t_0)) \mathbf{T}_{sb}(t_0)^{-1} \\ &= \mathbf{T}_{sb}(t_0) \hat{\xi}_b \mathbf{T}_{sb}(t_0)^{-1} \\ &= \hat{\xi}_s. \end{aligned}$$

The middle equality holds because a matrix commutes with every power of itself and therefore with its own exponential. Hence a body-constant generator and its consistently converted space generator are both constant along this one-parameter motion.

For a concrete contrast, consider two separate translation experiments with the same initial pose. The body origin initially coincides with the space origin, and the body is rotated by $\pi/2$ about positive z_s :

$$\mathbf{R}_{sb}(t_0) = \mathbf{R}_z\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad {}^s\mathbf{p}_b(t_0) = \mathbf{0}_3.$$

The complete initial pose is therefore

$$\mathbf{T}_{sb}(t_0) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The first three entries of the fourth column in this and the following homogeneous transformations are positions measured in metres. Let the interval be $\Delta t = 1$ s and the translational speed be 1 m s^{-1} , so the travelled distance is $d = 1$ m.

First choose a **space-referred input**: translate by d along positive x_s . Its increment is

$${}^s\mathbf{T}_\Delta := \begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The space increment multiplies the initial pose on the left. The resulting pose is

$$\begin{aligned} \mathbf{T}_{sb}(t_1) &= {}^s\mathbf{T}_\Delta \mathbf{T}_{sb}(t_0) \\ &= \begin{bmatrix} 0 & -1 & 0 & d \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

Thus the derived final position of the body origin is ${}^s\mathbf{p}_b(t_1) = [d \ 0 \ 0]^\top = [1 \ 0 \ 0]^\top \text{ m}$.

Now restart from the same initial pose and choose a **body-referred input**: translate by d along positive x_b . Its body-coordinate increment contains the same numerical entries,

$${}^b\mathbf{T}_\Delta := \begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

but the column $[d \ 0 \ 0]^\top$ is now referred to the body axes. The body increment multiplies on the right, giving

$$\begin{aligned} \mathbf{T}_{sb}(t_1) &= \mathbf{T}_{sb}(t_0) {}^b\mathbf{T}_\Delta \\ &= \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & d \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

Indeed, $\mathbf{R}_z(\pi/2)[d \ 0 \ 0]^\top = [0 \ d \ 0]^\top$, so positive x_b initially points along positive y_s . The derived final position is therefore ${}^s\mathbf{p}_b(t_1) = [0 \ d \ 0]^\top = [0 \ 1 \ 0]^\top \text{ m}$.

The two chosen increment matrices have identical numerical entries, but those entries are referred to different axes; they are not coordinate-converted descriptions of the same physical translation. In general, space- and body-referred increments describe the same physical motion only when

$${}^s\mathbf{T}_\Delta = \mathbf{T}_{sb}(t_0)^b \mathbf{T}_\Delta \mathbf{T}_{sb}(t_0)^{-1}.$$

The closed forms above are exact only while the selected twist is constant. For a time-varying twist sampled over intervals $[t_k, t_{k+1}]$ of duration Δt_k , a piecewise-constant exponential update is

$$\begin{aligned} \mathbf{T}_{k+1} &= \exp(\hat{\xi}_{s,k} \Delta t_k) \mathbf{T}_k, & \text{space form,} \\ \mathbf{T}_{k+1} &= \mathbf{T}_k \exp(\hat{\xi}_{b,k} \Delta t_k), & \text{body form.} \end{aligned} \tag{8.17}$$

The factors must remain in chronological order. If twist matrices at different times do not commute, the exact solution is not generally $\exp(\int \hat{\xi}(t) dt)$ with an ordinary matrix integral. A time-ordered integration method is required. Likewise, the additive Forward Euler update $\mathbf{T}_{k+1} = \mathbf{T}_k + \dot{\mathbf{T}}_k \Delta t_k$ does not generally remain in $SE(3)$ because matrix addition is not the group operation. Exponential updates remain in $SE(3)$ in exact arithmetic; Section 8.5 develops the small-angle formulas, floating-point checks, and branch policies needed in software.

Quick test C: constant-twist pose integration

1. **Recall:** Write the body- and space-twist definitions in terms of $\mathbf{T}_{sb}$ and $\dot{\mathbf{T}}_{sb}$. Which side does each hatted twist multiply when reconstructing the pose rate?
2. **Derivation:** Differentiate the constant-space-twist solution and show that it satisfies both the differential equation and initial pose.
3. **Interpretation:** Why is ν_s generally not the velocity of the body origin? What does ν_b represent?
4. **Application:** A body initially has orientation $\mathbf{R}_z(\pi/2)$. Contrast a one-metre translation along x_s with a one-metre translation along x_b .
5. **Equivalence:** State the adjoint relation required for a left space update and a right body update to describe the same physical motion.
6. **Limitation:** Why can a time-varying twist not generally be integrated by exponentiating the ordinary integral of its twist matrices?

Answers and hints

1. $\hat{\xi}_b = \mathbf{T}_{sb}^{-1} \dot{\mathbf{T}}_{sb}$ and $\dot{\mathbf{T}}_{sb} = \mathbf{T}_{sb} \hat{\xi}_b$, so the body twist multiplies on the right. Also, $\hat{\xi}_s = \dot{\mathbf{T}}_{sb} \mathbf{T}_{sb}^{-1}$ and $\dot{\mathbf{T}}_{sb} = \hat{\xi}_s \mathbf{T}_{sb}$, so the space twist multiplies on the left.
2. For $\mathbf{T}(t) = \exp(\hat{\xi}_s(t - t_0)) \mathbf{T}(t_0)$, differentiation gives $\dot{\mathbf{T}} = \hat{\xi}_s \mathbf{T}$. At $t = t_0$, the exponential is $\mathbf{I}_4$, so $\mathbf{T}(t_0)$ is recovered.

3. The space-twist linear component $\boldsymbol{\nu}_s$ is the velocity field evaluated at O_s . The body origin is at ${}^s\mathbf{p}_b$, so its velocity is $\boldsymbol{\nu}_s + \boldsymbol{\omega}_s \times {}^s\mathbf{p}_b$. The body-twist linear component $\boldsymbol{\nu}_b$ is that body-origin velocity expressed along the body axes.
4. Translation along x_s ends at $[1 \ 0 \ 0]^\top$ m. Because x_b initially points along y_s , translation along x_b ends at $[0 \ 1 \ 0]^\top$ m.
5. Let $\mathbf{T}_0 := \mathbf{T}_{sb}(t_0)$. The two constant twist columns must represent the same physical velocity field in different coordinates, so

$$\boldsymbol{\xi}_s = \text{Ad}_{\mathbf{T}_0} \boldsymbol{\xi}_b, \quad \hat{\boldsymbol{\xi}}_s = \mathbf{T}_0 \hat{\boldsymbol{\xi}}_b \mathbf{T}_0^{-1}.$$

The matrix relation implies

$$\begin{aligned} \exp(\hat{\boldsymbol{\xi}}_s \Delta t) &= \mathbf{T}_0 \exp(\hat{\boldsymbol{\xi}}_b \Delta t) \mathbf{T}_0^{-1}, \\ \exp(\hat{\boldsymbol{\xi}}_s \Delta t) \mathbf{T}_0 &= \mathbf{T}_0 \exp(\hat{\boldsymbol{\xi}}_b \Delta t). \end{aligned}$$

The second line is exactly the equality between the left space update and the right body update, so both produce the same final pose. The columns $\boldsymbol{\xi}_s$ and $\boldsymbol{\xi}_b$ generally do not contain the same numerical components.

6. Twist matrices at different times may not commute, so changing their chronological multiplication order changes the motion. The ordinary integral discards that ordering; an ordered product or time-ordered integration method is required.

8.5 Numerical branches and validation

The exponential and logarithm formulas above describe exact real-number mathematics, but they do not by themselves prescribe one complete numerical algorithm. A numerical implementation must additionally account for non-unique logarithms, floating-point cancellation, rotations near their branch boundary, and arrays that do not represent elements of $SE(3)$. The implementation must preserve the exact geometric map while declaring which logarithm it returns, which stable formula it evaluates in each numerical region, and which invalid inputs it rejects.

This section uses the same active-displacement convention and linear-first component order as the preceding sections. Every exponential coordinate, transformation, translation, and screw axis must carry its expression frame even when a generic symbol is used to keep an equation readable. Numerical thresholds select formulas or declare interface resolution; they are not new physical laws.

8.5.1 What logarithm branches mean

The exponential maps exponential coordinates to a rigid transformation. The inverse problem is not one-to-one because finite rotations repeat. For example, let $\mathbf{e}_z = [0 \ 0 \ 1]^\top$ be the positive z -axis unit column and let $k \in \mathbb{Z}$ be any integer. Rotating through an additional whole number of turns produces the same orientation:

$$\exp([\mathbf{e}_z]_{\times}(\theta + 2k\pi)) = \exp([\mathbf{e}_z]_{\times}\theta).$$

Consequently, one rotation matrix can have many matrix logarithms. The same non-uniqueness enters an $SE(3)$ logarithm through its rotation block. For a transformation $\mathbf{T} \in SE(3)$, define the set of all its real rigid-motion logarithms by

$$\boxed{\text{Log}(\mathbf{T}) := \{\mathbf{X} \in \mathfrak{se}(3) \mid \exp(\mathbf{X}) = \mathbf{T}\}}. \quad (8.18)$$

Here $\mathfrak{se}(3)$ is the set of hatted rigid-motion generators defined earlier, and $\text{Log}(\mathbf{T})$ is a set rather than one matrix. Every $\mathbf{X} \in \text{Log}(\mathbf{T})$ is a **logarithm** of $\mathbf{T}$. Different members may contain different rotation amounts and corresponding linear blocks while exponentiating to the same final transformation. They describe different exponential-coordinate representations $\mathbf{S}\sigma$ of a screw path with the same endpoint; they do not reveal which path the body followed historically.

A **logarithm branch** is a declared rule that chooses one member of this set. Let $\mathcal{D}_{\text{sel}} \subseteq SE(3)$ be the domain of one declared selection rule. Write that single-valued rule as

$$\boxed{\log_{\text{sel}} : \mathcal{D}_{\text{sel}} \longrightarrow \mathfrak{se}(3), \quad \log_{\text{sel}}(\mathbf{T}) \in \text{Log}(\mathbf{T})}. \quad (8.19)$$

Single-valued means that the rule returns exactly one selected logarithm for each accepted input. The textual subscript sel labels the selection convention; it is not another mathematical variable or physical motion.

The **principal-branch convention** used in this chapter selects a rotation vector ${}^a\phi \in \mathbb{R}^3$ whose magnitude

$$\theta := \|{}^a\phi\|_2$$

belongs to the interval $[0, \pi]$. The word *principal* means “the standard representative selected by the declared convention”; it does not mean that this representative is the body’s unique or historically correct motion. For $0 < \theta < \pi$, the rotation matrix determines the oriented rotation vector uniquely within this interval. At the identity, the selected finite rotation vector is $\mathbf{0}_3$, although a normalised axis is undefined. At $\theta = \pi$, the two vectors $\pi^a\mathbf{s}_\omega$ and $-\pi^a\mathbf{s}_\omega$ represent the same rotation, so an additional canonical axis-sign rule is required.

Once this principal rotation vector has been selected for

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & {}^a\mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

write the **principal-branch rigid-motion logarithm** as

$$\log_p(\mathbf{T}) := \begin{bmatrix} [{}^a\boldsymbol{\phi}]_\times & {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}, \quad \|{}^a\boldsymbol{\phi}\|_2 \in [0, \pi], \quad (8.20)$$

Here ${}^a\boldsymbol{\rho} \in \mathbb{R}^3$ is the **finite linear exponential-coordinate block** paired with the selected rotation-vector block ${}^a\boldsymbol{\phi}$. It is expressed in frame $\{a\}$ and measured in metres. Its defining role is that the complete logarithm must exponentiate back to the given transformation:

$$\boxed{\exp(\log_p(\mathbf{T})) = \mathbf{T}}.$$

The block ${}^a\boldsymbol{\rho}$ must not be confused with the translation column ${}^a\mathbf{p}$. The column ${}^a\mathbf{p}$ is the net translation contained in the final transformation, whereas ${}^a\boldsymbol{\rho}$ is the constant linear block inside the exponential generator. They are equal for pure translation but are generally different when rotation is present. The next subsection introduces the required rotation-dependent matrix and then derives the equation that recovers ${}^a\boldsymbol{\rho}$ from ${}^a\mathbf{p}$.

The identity and π cases are resolved by the conventions stated above. The shorter phrase *principal logarithm* below refers to this declared robotics convention.

Imagine increasing a rotation angle smoothly through π . Just before π , the principal rotation vector can point along one axis direction with magnitude slightly less than π . Just after π , keeping its magnitude inside $[0, \pi]$ requires the selected vector to point in the opposite direction. The physical orientation continues smoothly, but its selected coordinate vector jumps. The rotations at which this switch becomes necessary form the **branch boundary**. For the principal rotation-vector convention, the branch boundary consists of rotations through exactly π . To see this change algebraically, let $\varepsilon > 0$ be small and consider

$$\mathbf{R}_- = \mathbf{R}_z(\pi - \varepsilon), \quad \mathbf{R}_+ = \mathbf{R}_z(\pi + \varepsilon) = \mathbf{R}_z(-\pi + \varepsilon).$$

The two matrices become arbitrarily close as $\varepsilon \rightarrow 0$, but their selected principal rotation vectors are

$$\boldsymbol{\phi}_- = (\pi - \varepsilon)\mathbf{e}_z, \quad \boldsymbol{\phi}_+ = (-\pi + \varepsilon)\mathbf{e}_z.$$

The selected coordinates therefore jump by almost $2\pi\mathbf{e}_z$ even though the represented orientation changes smoothly. A canonical sign rule chooses one answer at exactly π , but it

cannot make the limits from both sides agree. This discontinuity belongs to the coordinate selection, not to the physical orientation or transformation, and no globally continuous single-valued logarithm can remove it.

The word *branch* is also used in numerical algorithms for a different kind of selection. The distinction is:

- A **mathematical logarithm branch** chooses one exact logarithm from several exact logarithms of the same transformation. Changing this branch can change the returned exponential coordinates while preserving the transformation obtained after exponentiation.
- A **numerical formula branch** chooses among algebraically equivalent ways to evaluate the already selected result, such as a Taylor series near zero or a trigonometric formula away from zero. Changing a formula branch should affect only numerical accuracy, not the represented motion.

A branch policy is **deterministic** when the same validated input, declared configuration, implementation, and floating-point environment always produce the same selected coordinates, status, and failure behaviour. Determinism therefore requires an explicit order of cases, inclusive or exclusive threshold comparisons, tie-breaking rules, and a canonical axis sign at $\theta = \pi$. It does not imply that the selected logarithm is physically unique or continuous across a branch boundary, nor does it by itself promise bit-for-bit agreement between different mathematical libraries or hardware platforms. The remainder of this section defines both the principal logarithm selection and the stable numerical formula branches needed to compute it.

8.5.2 Small angular displacements

The normalised screw representation separates motion geometry from motion amount, but dividing by the amount is undesirable when that amount is very small. Recall from Section 8.3.4 that the finite **exponential-coordinate column** referred to frame $\{a\}$ combines them without division:

$${}^a\eta = \begin{bmatrix} {}^a\rho \\ {}^a\phi \end{bmatrix} = {}^a\mathbf{S}\sigma \in \mathbb{R}^6. \quad (8.21)$$

The column ${}^a\rho \in \mathbb{R}^3$ is the finite linear exponential-coordinate block measured in metres, and ${}^a\phi \in \mathbb{R}^3$ is the finite rotation-vector block measured in radians. Radians are dimensionless in the algebra, but the label distinguishes angular displacement from translation. For a rotational or helical screw,

$${}^a\rho = {}^a\mathbf{s}_v\theta, \quad {}^a\phi = {}^a\mathbf{s}_\omega\theta.$$

For a prismatic screw, ${}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v d$ and ${}^a\boldsymbol{\phi} = \mathbf{0}_3$. For a constant actual twist integrated over Δt seconds, the same blocks are ${}^a\boldsymbol{\rho} = {}^a\boldsymbol{\nu}\Delta t$ and ${}^a\boldsymbol{\phi} = {}^a\boldsymbol{\omega}\Delta t$. Thus one numerical representation covers rotational, helical, prismatic, and constant-twist increments without first asking for a normalised axis.

Define

$$\theta := \|{}^a\boldsymbol{\phi}\|_2 \geq 0, \quad \boldsymbol{\Phi} := [{}^a\boldsymbol{\phi}]_{\times} \in \mathbb{R}^{3 \times 3}.$$

This θ is the nonnegative magnitude of the rotation-vector block. If an original signed motion amount was negative, its sign is carried by the direction of ${}^a\boldsymbol{\phi}$. When $\theta > 0$, the unit axis is ${}^a\mathbf{s}_\omega = {}^a\boldsymbol{\phi}/\theta$ and $\boldsymbol{\Phi} = \boldsymbol{\Omega}\theta$, where $\boldsymbol{\Omega} = [{}^a\mathbf{s}_\omega]_{\times}$. The hatted finite coordinate column is

$$\widehat{{}^a\boldsymbol{\eta}} = \begin{bmatrix} \boldsymbol{\Phi} & {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

Start from the normalised rotational or helical screw result derived in the preceding section. For $\theta > 0$, the finite-to-normalised relations above and the linearity of the cross-product map give

$${}^a\mathbf{s}_v = \frac{{}^a\boldsymbol{\rho}}{\theta}, \quad {}^a\mathbf{s}_\omega = \frac{{}^a\boldsymbol{\phi}}{\theta}, \quad \boldsymbol{\Omega} = \frac{\boldsymbol{\Phi}}{\theta}.$$

The previous closed form was

$$\exp(\widehat{{}^a\mathbf{S}}\theta) = \begin{bmatrix} \mathbf{R}(\theta) & \mathbf{G}(\theta){}^a\mathbf{s}_v \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

with the Rodrigues rotation block and translation matrix

$$\begin{aligned} \mathbf{R}(\theta) &= \mathbf{I}_3 + \sin\theta \boldsymbol{\Omega} + (1 - \cos\theta)\boldsymbol{\Omega}^2, \\ \mathbf{G}(\theta) &= \theta\mathbf{I}_3 + (1 - \cos\theta)\boldsymbol{\Omega} + (\theta - \sin\theta)\boldsymbol{\Omega}^2. \end{aligned}$$

The hat map is linear, so $\widehat{{}^a\boldsymbol{\eta}} = \widehat{{}^a\mathbf{S}}\theta$. Hence the previous exponential is exactly the exponential of the finite column used here. Substitute $\boldsymbol{\Omega} = \boldsymbol{\Phi}/\theta$ into its rotation block:

$$\begin{aligned} \mathbf{R}({}^a\boldsymbol{\phi}) &= \mathbf{I}_3 + \sin\theta \frac{\boldsymbol{\Phi}}{\theta} + (1 - \cos\theta) \frac{\boldsymbol{\Phi}^2}{\theta^2} \\ &= \mathbf{I}_3 + \frac{\sin\theta}{\theta} \boldsymbol{\Phi} + \frac{1 - \cos\theta}{\theta^2} \boldsymbol{\Phi}^2. \end{aligned}$$

This is Rodrigues' formula written with the finite rotation-vector matrix Φ . Now substitute both $\Omega = \Phi/\theta$ and ${}^a\mathbf{s}_v = {}^a\boldsymbol{\rho}/\theta$ into the previous translation block:

$$\begin{aligned} {}^a\mathbf{p} &= \mathbf{G}(\theta) {}^a\mathbf{s}_v \\ &= \left[\theta \mathbf{I}_3 + (1 - \cos \theta) \frac{\Phi}{\theta} + (\theta - \sin \theta) \frac{\Phi^2}{\theta^2} \right] \frac{{}^a\boldsymbol{\rho}}{\theta} \\ &= \left[\mathbf{I}_3 + \frac{1 - \cos \theta}{\theta^2} \Phi + \frac{\theta - \sin \theta}{\theta^3} \Phi^2 \right] {}^a\boldsymbol{\rho}. \end{aligned}$$

The derived rotation and translation formulas contain three dimensionless scalar ratios. Give these ratios names and extend them continuously to $\theta = 0$:

$$\begin{aligned} A(\theta) &:= \begin{cases} \frac{\sin \theta}{\theta}, & \theta > 0, \\ 1, & \theta = 0, \end{cases} \\ B(\theta) &:= \begin{cases} \frac{1 - \cos \theta}{\theta^2}, & \theta > 0, \\ \frac{1}{2}, & \theta = 0, \end{cases} \\ C(\theta) &:= \begin{cases} \frac{\theta - \sin \theta}{\theta^3}, & \theta > 0, \\ \frac{1}{6}, & \theta = 0. \end{cases} \end{aligned} \tag{8.22}$$

The dimensionless matrix multiplying ${}^a\boldsymbol{\rho}$ is called the **left Jacobian of $SO(3)$** and is denoted by $\mathbf{J}({}^a\boldsymbol{\phi})$. The results derived from the previous screw exponential can now be written compactly as

$$\boxed{\begin{aligned} \mathbf{R}({}^a\boldsymbol{\phi}) &= \mathbf{I}_3 + A(\theta)\Phi + B(\theta)\Phi^2, \\ \mathbf{J}({}^a\boldsymbol{\phi}) &:= \mathbf{I}_3 + B(\theta)\Phi + C(\theta)\Phi^2. \end{aligned}} \tag{8.23}$$

Therefore the complete exponential is

$$\boxed{\exp(\widehat{{}^a\boldsymbol{\eta}}) = \begin{bmatrix} \mathbf{R}({}^a\boldsymbol{\phi}) & \mathbf{J}({}^a\boldsymbol{\phi}) {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}}. \tag{8.24}$$

Here $\mathbf{J}({}^a\boldsymbol{\phi}) \in \mathbb{R}^{3 \times 3}$ is dimensionless. The matrix $\mathbf{J}({}^a\boldsymbol{\phi})$ maps the finite linear exponential-coordinate block ${}^a\boldsymbol{\rho}$, measured in metres, to the final translation column ${}^a\mathbf{p}$,

also measured in metres. It is not a robot manipulator Jacobian and does not map joint rates to an end-effector velocity.

The same matrix also has a direct path interpretation. Introduce a dimensionless path parameter $\tau \in [0, 1]$. It records the fraction of the finite exponential motion that has been completed; it is not elapsed time. Define the intermediate transformation

$$\mathbf{T}(\tau) := \exp(\tau \widehat{a\boldsymbol{\eta}}) = \begin{bmatrix} \mathbf{R}(\tau^a \boldsymbol{\phi}) & {}^a\mathbf{p}(\tau) \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

At $\tau = 0$, $\mathbf{T}(0) = \mathbf{I}_4$ and ${}^a\mathbf{p}(0) = \mathbf{0}_3$. At $\tau = 1$, the path reaches the complete transformation. Because the generator $\widehat{a\boldsymbol{\eta}}$ is constant along this path, differentiating the matrix exponential gives

$$\frac{d\mathbf{T}}{d\tau} = \mathbf{T}(\tau) \widehat{a\boldsymbol{\eta}}.$$

Using

$$\widehat{a\boldsymbol{\eta}} = \begin{bmatrix} \boldsymbol{\Phi} & {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix},$$

block multiplication gives

$$\frac{d\mathbf{T}}{d\tau} = \begin{bmatrix} \mathbf{R}(\tau^a \boldsymbol{\phi}) \boldsymbol{\Phi} & \mathbf{R}(\tau^a \boldsymbol{\phi}) {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

The upper-right block therefore satisfies

$$\boxed{\frac{d^a\mathbf{p}}{d\tau} = \mathbf{R}(\tau^a \boldsymbol{\phi}) {}^a\boldsymbol{\rho}}.$$

This equation explains the accumulation. During each small increase $d\tau$, the current rotation changes the direction in which the constant linear exponential-coordinate block ${}^a\boldsymbol{\rho}$ contributes to the translation. Integrating from the initial transformation to the final transformation gives

$$\begin{aligned} {}^a\mathbf{p}(1) &= \int_0^1 \mathbf{R}(\tau^a \boldsymbol{\phi}) {}^a\boldsymbol{\rho} d\tau \\ &= \left[\int_0^1 \exp(\tau \boldsymbol{\Phi}) d\tau \right] {}^a\boldsymbol{\rho}. \end{aligned}$$

The matrix multiplying ${}^a\boldsymbol{\rho}$ is therefore

$$\mathbf{J}({}^a\boldsymbol{\phi}) = \int_0^1 \exp(\tau\boldsymbol{\Phi}) \, d\tau. \quad (8.25)$$

The relationship in Equation 8.25 gives the geometric meaning of $\mathbf{J}({}^a\boldsymbol{\phi})$. For each path value $\tau \in [0, 1]$, the matrix $\exp(\tau\boldsymbol{\Phi}) = \mathbf{R}(\tau{}^a\boldsymbol{\phi})$ rotates the linear block ${}^a\boldsymbol{\rho}$ into the direction in which ${}^a\boldsymbol{\rho}$ contributes at that stage of the exponential path. The matrix $\mathbf{J}({}^a\boldsymbol{\phi})$ accumulates these intermediate rotation matrices over the complete path, so the final translation is

$${}^a\mathbf{p}(1) = \mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho}.$$

Thus $\mathbf{J}({}^a\boldsymbol{\phi})$ depends only on the finite rotation-vector block ${}^a\boldsymbol{\phi}$ and describes how that rotation changes the accumulated contribution from ${}^a\boldsymbol{\rho}$. Because the integration interval has length one, each entry of $\mathbf{J}({}^a\boldsymbol{\phi})$ is the average of the corresponding entry of the intermediate rotation matrix:

$$J_{ij}({}^a\boldsymbol{\phi}) = \int_0^1 R_{ij}(\tau{}^a\boldsymbol{\phi}) \, d\tau, \quad i, j \in \{1, 2, 3\}.$$

The matrix $\mathbf{J}({}^a\boldsymbol{\phi})$ is generally not itself a rotation matrix because averaging matrix entries does not generally preserve the rotation conditions $\mathbf{J}^T\mathbf{J} = \mathbf{I}_3$ and $\det(\mathbf{J}) = 1$. The matrix $\mathbf{J}({}^a\boldsymbol{\phi})$ maps the metre-valued column ${}^a\boldsymbol{\rho}$ to the metre-valued column ${}^a\mathbf{p}(1)$, but $\mathbf{J}({}^a\boldsymbol{\phi})$ does not represent the body's orientation.

To recover its closed form, apply Rodrigues' formula to the intermediate rotation:

$$\exp(\tau\boldsymbol{\Phi}) = \mathbf{I}_3 + \frac{\sin(\tau\theta)}{\theta}\boldsymbol{\Phi} + \frac{1 - \cos(\tau\theta)}{\theta^2}\boldsymbol{\Phi}^2.$$

The two scalar integrals are

$$\begin{aligned} \int_0^1 \frac{\sin(\tau\theta)}{\theta} \, d\tau &= \frac{1 - \cos\theta}{\theta^2} = B(\theta), \\ \int_0^1 \frac{1 - \cos(\tau\theta)}{\theta^2} \, d\tau &= \frac{\theta - \sin\theta}{\theta^3} = C(\theta). \end{aligned}$$

Therefore

$$\int_0^1 \exp(\tau \Phi) d\tau = \mathbf{I}_3 + B(\theta) \Phi + C(\theta) \Phi^2,$$

which recovers the second line of Equation 8.23 from the exponential path. The integral is dimensionless because both τ and every rotation matrix are dimensionless. It acts on ${}^a\rho$, measured in metres, to produce the final translation ${}^a\mathbf{p}(1)$, also measured in metres.

The trigonometric calculation above assumes $\theta > 0$. At $\theta = 0$, $\Phi = \mathbf{0}_{3 \times 3}$ and $\exp(\tau \Phi) = \mathbf{I}_3$ for every τ , so the integral gives $\mathbf{J}(\mathbf{0}_3) = \mathbf{I}_3$. This agrees with the continuous coefficient values $B(0) = 1/2$ and $C(0) = 1/6$ because both terms containing Φ vanish. The finite $\mathbf{J}\rho$ form is therefore well defined at zero without forming ${}^a\mathbf{s}_v = {}^a\rho/\theta$.

Directly evaluating $1 - \cos \theta$ or $\theta - \sin \theta$ for a very small θ subtracts numbers that are nearly equal. The exact result is small, so the subtraction can retain few or no significant digits. Taylor expansion supplies stable alternatives:

$$\begin{aligned} A(\theta) &= 1 - \frac{\theta^2}{6} + \frac{\theta^4}{120} - \frac{\theta^6}{5040} + O(\theta^8), \\ B(\theta) &= \frac{1}{2} - \frac{\theta^2}{24} + \frac{\theta^4}{720} - \frac{\theta^6}{40320} + O(\theta^8), \\ C(\theta) &= \frac{1}{6} - \frac{\theta^2}{120} + \frac{\theta^4}{5040} - \frac{\theta^6}{362880} + O(\theta^8). \end{aligned} \tag{8.26}$$

An implementation evaluates these polynomials below a declared dimensionless series threshold $\theta_{\text{series}} > 0$ and the trigonometric ratios above it. The threshold changes only the evaluation method, not the represented motion. Both paths should agree within a declared floating-point tolerance around the switching boundary.

At $\theta = 0$, $\Phi = \mathbf{0}_{3 \times 3}$, $\mathbf{R} = \mathbf{I}_3$, and $\mathbf{J} = \mathbf{I}_3$, so Equation 8.24 reduces continuously to

$$\exp\left(\begin{bmatrix} \mathbf{0}_{3 \times 3} & {}^a\rho \\ \mathbf{0}_3^\top & 0 \end{bmatrix}\right) = \begin{bmatrix} \mathbf{I}_3 & {}^a\rho \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

which is the pure-translation exponential. Thus the small-angle formulation does not need to invent an arbitrary rotation axis at zero.

The logarithm needs the inverse mapping from a translation column ${}^a\mathbf{p}$ to ${}^a\rho$. Hence, we need to check the inverse $\mathbf{J}^{-1}$ exists.

At $\theta = 0$, the preceding result $\mathbf{J}(\mathbf{0}_3) = \mathbf{I}_3$ already proves that this inverse exists. Now consider $0 < \theta \leq \pi$, for which the unit rotation-axis column ${}^a\mathbf{s}_\omega = {}^a\phi/\theta$ is defined. Because $\Phi = \theta[{}^a\mathbf{s}_\omega]_\times$, the axis direction satisfies

$$\Phi^a \mathbf{s}_\omega = \theta ({}^a \mathbf{s}_\omega \times {}^a \mathbf{s}_\omega) = \mathbf{0}_3.$$

Substitution into $\mathbf{J} = \mathbf{I}_3 + B(\theta)\Phi + C(\theta)\Phi^2$ gives $\mathbf{J}^a \mathbf{s}_\omega = {}^a \mathbf{s}_\omega$. Thus $\mathbf{J}$ acts as multiplication by 1 along the rotation axis.

It remains to examine columns perpendicular to the axis. Define the two-dimensional plane

$$\mathcal{V}_\perp := \{\mathbf{x} \in \mathbb{R}^3 \mid ({}^a \mathbf{s}_\omega)^\top \mathbf{x} = 0\}.$$

Choose any unit column $\mathbf{u} \in \mathcal{V}_\perp$ and define $\mathbf{v} := {}^a \mathbf{s}_\omega \times \mathbf{u}$. The ordered columns $(\mathbf{u}, \mathbf{v})$ form an orthonormal basis of $\mathcal{V}_\perp$. Recall that $\Omega = [{}^a \mathbf{s}_\omega]_\times$. The definition of $\mathbf{v}$ gives $\Omega \mathbf{u} = \mathbf{v}$. The vector triple-product identity gives

$$\Omega \mathbf{v} = {}^a \mathbf{s}_\omega \times ({}^a \mathbf{s}_\omega \times \mathbf{u}) = {}^a \mathbf{s}_\omega (({}^a \mathbf{s}_\omega)^\top \mathbf{u}) - \mathbf{u} (({}^a \mathbf{s}_\omega)^\top {}^a \mathbf{s}_\omega) = -\mathbf{u}.$$

The first equality $\Omega \mathbf{u} = \mathbf{v}$ and the second equality $\Omega \mathbf{v} = -\mathbf{u}$ determine the matrix of Ω in the ordered basis $(\mathbf{u}, \mathbf{v})$:

$$[\Omega|_{\mathcal{V}_\perp}]_{(\mathbf{u}, \mathbf{v})} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Here $\Omega|_{\mathcal{V}_\perp}$ means that the input and output of Ω are restricted to the plane $\mathcal{V}_\perp$. The displayed matrix is a 90-degree rotation within that plane, and its square is $-\mathbf{I}_2$. Because $\Phi = \theta \Omega$, the corresponding restricted matrices satisfy

$$[\Phi|_{\mathcal{V}_\perp}]_{(\mathbf{u}, \mathbf{v})} = \theta \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad [\Phi^2|_{\mathcal{V}_\perp}]_{(\mathbf{u}, \mathbf{v})} = -\theta^2 \mathbf{I}_2.$$

Restricting $\mathbf{J} = \mathbf{I}_3 + B(\theta)\Phi + C(\theta)\Phi^2$ to $\mathcal{V}_\perp$ therefore gives

$$\begin{aligned} [\mathbf{J}|_{\mathcal{V}_\perp}]_{(\mathbf{u}, \mathbf{v})} &= (1 - C(\theta)\theta^2) \mathbf{I}_2 + B(\theta)\theta \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} \frac{\sin \theta}{\theta} & -\frac{1 - \cos \theta}{\theta} \\ \frac{1 - \cos \theta}{\theta} & \frac{\sin \theta}{\theta} \end{bmatrix}, \end{aligned}$$

where $1 - C(\theta)\theta^2 = \sin \theta / \theta$ and $B(\theta)\theta = (1 - \cos \theta) / \theta$. Taking the determinant of this 2×2 matrix gives

$$\begin{aligned}
\det(\mathbf{J}|_{\mathcal{V}_\perp}) &= \left(\frac{\sin \theta}{\theta}\right)^2 + \left(\frac{1 - \cos \theta}{\theta}\right)^2 \\
&= \frac{\sin^2 \theta + (1 - \cos \theta)^2}{\theta^2} \\
&= \frac{2(1 - \cos \theta)}{\theta^2} \\
&= \frac{4 \sin^2(\theta/2)}{\theta^2}.
\end{aligned}$$

The last equality uses $1 - \cos \theta = 2 \sin^2(\theta/2)$. The determinant has continuous value 1 at $\theta = 0$ and is positive for $0 < \theta \leq \pi$. In the orthonormal basis $(\mathbf{u}, \mathbf{v}, {}^a\mathbf{s}_\omega)$, the complete matrix of $\mathbf{J}$ contains the displayed 2×2 block and the scalar axis block $[1]$. Its determinant is therefore the product of the two block determinants and is nonzero. Hence $\mathbf{J}$ is invertible throughout the complete principal interval $\theta \in [0, \pi]$. Its inverse is

$$\boxed{\mathbf{J}({}^a\phi)^{-1} = \mathbf{I}_3 - \frac{1}{2}\mathbf{\Phi} + D(\theta)\mathbf{\Phi}^2}, \quad (8.27)$$

where

$$D(\theta) := \begin{cases} \frac{1}{\theta^2} - \frac{1}{2\theta} \cot\left(\frac{\theta}{2}\right), & \theta > 0, \\ \frac{1}{12}, & \theta = 0. \end{cases} \quad (8.28)$$

This expression follows from the previously derived inverse of $\mathbf{G}(\theta)$. Because $\mathbf{G}(\theta) = \theta\mathbf{J}({}^a\phi)$, $\mathbf{\Phi} = \mathbf{\Omega}\theta$, and $\mathbf{J}^{-1} = \theta\mathbf{G}^{-1}$,

$$\begin{aligned}
\mathbf{J}^{-1} &= \mathbf{I}_3 - \frac{\theta}{2}\mathbf{\Omega} + \left[1 - \frac{\theta}{2} \cot\left(\frac{\theta}{2}\right)\right] \mathbf{\Omega}^2 \\
&= \mathbf{I}_3 - \frac{1}{2}\mathbf{\Phi} + \left[\frac{1}{\theta^2} - \frac{1}{2\theta} \cot\left(\frac{\theta}{2}\right)\right] \mathbf{\Phi}^2.
\end{aligned}$$

The two large terms in $D(\theta)$ nearly cancel at a small angle, so D also requires a series-evaluation formula branch:

$$D(\theta) = \frac{1}{12} + \frac{\theta^2}{720} + \frac{\theta^4}{30240} + \frac{\theta^6}{1209600} + O(\theta^8). \quad (8.29)$$

Once the rotation-vector block has been recovered from a validated transformation, the finite linear block is calculated as

$$\boxed{{}^a\boldsymbol{\rho} = \mathbf{J}({}^a\boldsymbol{\phi})^{-1} {}^a\mathbf{p}}. \quad (8.30)$$

Computing ${}^a\boldsymbol{\rho}$ first is numerically and conceptually important. A normalised rotational screw would require ${}^a\mathbf{s}_v = {}^a\boldsymbol{\rho}/\theta$, which is ill-conditioned as $\theta \rightarrow 0$ and unnecessary when the desired output is the matrix logarithm or finite exponential-coordinate column.

8.5.3 Recovering the principal logarithm, including rotations near π

Suppose a validated rotation matrix $\mathbf{R} \in SO(3)$ is already known. If the task is only to rotate a point, compose orientations, or store the orientation change, no logarithm is needed: use $\mathbf{R}$ directly. The inverse calculation is needed only when the finite rotation must be expressed as a rotation-vector column. Examples include forming the rotational block of $\log(\mathbf{T})$, expressing a relative orientation error as three components, constructing intermediate rotations along a selected exponential path, or recovering the screw motion represented by a transformation.

The forward calculation developed above starts from a finite rotation-vector column and constructs a rotation matrix:

$$\boxed{\begin{aligned} {}^a\boldsymbol{\phi} &= \theta {}^a\mathbf{s}_\omega, \\ {}^a\boldsymbol{\phi} &\xrightarrow{[\cdot]_\times} \boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_\times, \\ \boldsymbol{\Phi} &\xrightarrow{\exp} \mathbf{R} = \exp(\boldsymbol{\Phi}). \end{aligned}}$$

The rotation logarithm solves the inverse problem

$$\boxed{\mathbf{R} \mapsto {}^a\boldsymbol{\phi} = \theta {}^a\mathbf{s}_\omega}, \quad \theta \in [0, \pi],$$

where θ is the selected principal angle and ${}^a\mathbf{s}_\omega$ is a unit axis direction expressed in frame $\{a\}$. Recovering ${}^a\boldsymbol{\phi}$ therefore requires both the angle θ and the axis direction ${}^a\mathbf{s}_\omega$.

Start from Rodrigues' formula for a rotation through θ about ${}^a\mathbf{s}_\omega$:

$$\mathbf{R} = \cos \theta \mathbf{I}_3 + (1 - \cos \theta) {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top + \sin \theta [{}^a\mathbf{s}_\omega]_\times.$$

This form separates two symmetric matrices, $\mathbf{I}_3$ and ${}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top$, from the skew-symmetric matrix $[{}^a\mathbf{s}_\omega]_\times$. These properties let the trace and transpose expose different information about the unknown angle and axis.

The trace of $[\mathbf{s}_\omega]_\times$ is zero. The trace of the outer product is

$$\text{tr}({}^a\mathbf{s}_\omega({}^a\mathbf{s}_\omega)^\top) = ({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_\omega = 1,$$

because ${}^a\mathbf{s}_\omega$ is a unit column. Taking the trace of Rodrigues' formula therefore gives

$$\text{tr}(\mathbf{R}) = 3 \cos \theta + (1 - \cos \theta) = 1 + 2 \cos \theta.$$

Solving for the cosine gives

$$\boxed{\frac{\text{tr}(\mathbf{R}) - 1}{2} = \cos \theta}.$$

Now transpose Rodrigues' formula. Transposition leaves the two symmetric terms unchanged and changes the sign of the cross-product matrix because $[\mathbf{s}_\omega]_\times^\top = -[\mathbf{s}_\omega]_\times$:

$$\mathbf{R}^\top = \cos \theta \mathbf{I}_3 + (1 - \cos \theta) {}^a\mathbf{s}_\omega({}^a\mathbf{s}_\omega)^\top - \sin \theta [\mathbf{s}_\omega]_\times.$$

Subtracting this equation from Rodrigues' original equation cancels both symmetric terms and doubles the skew-symmetric term:

$$\boxed{\mathbf{R} - \mathbf{R}^\top = 2 \sin \theta [\mathbf{s}_\omega]_\times}.$$

This subtraction matters because the inverse problem needs both θ and ${}^a\mathbf{s}_\omega$. The trace supplies $\cos \theta$. The skew difference supplies $\sin \theta$ multiplied by the axis direction. Its magnitude will provide the nonnegative principal sine, and its direction will provide the axis whenever $\sin \theta$ is not zero.

The limitation appears at the boundary angles. At $\theta = 0$, the skew difference is zero, but this causes no physical ambiguity because the rotation vector is $\mathbf{0}_3$ and no axis is needed. At $\theta = \pi$, the skew difference is also zero even though the rotation is not the identity. For example,

$$\mathbf{R}_z(\pi) = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

is symmetric and therefore satisfies $\mathbf{R}_z(\pi) - \mathbf{R}_z(\pi)^\top = \mathbf{0}_{3 \times 3}$. The identity rotation has the same zero skew difference, although the complete matrices are different. The trace distinguishes the two angles: it gives $\cos \theta = 1$ for the identity and $\cos \theta = -1$ for the rotation through π . Near π , a different, symmetric part of $\mathbf{R}$ will be needed to recover the axis line.

Recovering the principal angle

The two boxed identities motivate two quantities calculated from $\mathbf{R}$. Define the dimensionless scalar $c_\theta \in \mathbb{R}$ and the dimensionless column ${}^a\mathbf{u} \in \mathbb{R}^3$ by

$$c_\theta := \frac{\text{tr}(\mathbf{R}) - 1}{2}, \quad {}^a\mathbf{u} := \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix} \in \mathbb{R}^3. \quad (8.31)$$

For an exact rotation matrix, the trace derivation above gives $c_\theta = \cos \theta$.

To see why these particular entries are selected, write the skew difference explicitly:

$$\mathbf{R} - \mathbf{R}^\top = \begin{bmatrix} 0 & R_{12} - R_{21} & R_{13} - R_{31} \\ R_{21} - R_{12} & 0 & R_{23} - R_{32} \\ R_{31} - R_{13} & R_{32} - R_{23} & 0 \end{bmatrix} = [{}^a\mathbf{u}]_\times.$$

The selected entries are exactly the first, second, and third components stored by a cross-product matrix: $u_1 = R_{32} - R_{23}$, $u_2 = R_{13} - R_{31}$, and $u_3 = R_{21} - R_{12}$. Comparing $[{}^a\mathbf{u}]_\times = \mathbf{R} - \mathbf{R}^\top = 2 \sin \theta [{}^a\mathbf{s}_\omega]_\times$ with the boxed skew identity gives

$${}^a\mathbf{u} = 2 \sin \theta {}^a\mathbf{s}_\omega.$$

Thus ${}^a\mathbf{u}$ is not an additional physical quantity. It is a convenient coordinate column extracted from the skew part of $\mathbf{R}$. Its direction is the axis direction, scaled by $2 \sin \theta$.

The unit-length condition $\|{}^a\mathbf{s}_\omega\|_2 = 1$ gives $\|{}^a\mathbf{u}\|_2 = 2|\sin \theta|$. The selected principal interval $\theta \in [0, \pi]$ makes $\sin \theta$ nonnegative, so define the dimensionless scalar

$$s_\theta := \frac{1}{2} \|{}^a\mathbf{u}\|_2 \geq 0.$$

For an exact rotation matrix, $s_\theta = \sin \theta$. The pair (c_θ, s_θ) therefore contains the cosine and the nonnegative sine of the principal angle. The function $\text{atan2}(y, x)$ uses the signs of both inputs to select the correct quadrant, so

$$\boxed{\theta = \text{atan2}(s_\theta, c_\theta) \in [0, \pi]}. \quad (8.32)$$

This returns 0 from $(s_\theta, c_\theta) = (0, 1)$ and π from $(0, -1)$. Near the identity, $\text{atan2}(s_\theta, c_\theta)$ is numerically preferable to $\arccos(c_\theta)$ alone because s_θ retains a change proportional to the small angle, while c_θ differs from 1 by a much smaller quantity

proportional to the angle squared.⁴ The validation section below explains how a floating-point value of c_θ may be corrected for a tiny round-off overshoot without concealing an invalid rotation matrix.

Recovering the axis away from π

When $0 < \theta < \pi$ and s_θ is not too small, divide ${}^a\mathbf{u} = 2s_\theta {}^a\mathbf{s}_\omega$ by $2s_\theta$ to obtain the unit axis. Multiplying that axis by θ gives

$$\boxed{{}^a\phi = \frac{\theta}{2s_\theta} {}^a\mathbf{u}}. \quad (8.33)$$

For a small nonzero angle, both θ and $\sin \theta$ approach zero. The following series supplies the continuous limiting value of their ratio:

$$\frac{\theta}{2 \sin \theta} = \frac{1}{2} + \frac{\theta^2}{12} + \frac{7\theta^4}{720} + O(\theta^6). \quad (8.34)$$

Using this series for a resolvable small angle preserves that nonzero rotation. It does not classify every small rotation as the identity.

Why rotations near π need the symmetric part

Near $\theta = \pi$, the relation ${}^a\mathbf{u} = 2 \sin \theta {}^a\mathbf{s}_\omega$ becomes unusable because $\sin \theta$ approaches zero. The column ${}^a\mathbf{u}$ then approaches zero for every possible axis, so dividing it by $2s_\theta$ amplifies numerical error. The symmetric part of $\mathbf{R}$ does not have this problem. Adding Rodrigues' formula to its transpose cancels the sine term and gives

$$\frac{\mathbf{R} + \mathbf{R}^\top}{2} = c_\theta \mathbf{I}_3 + (1 - c_\theta) {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top.$$

Move the known term $c_\theta \mathbf{I}_3$ to the left and divide by $1 - c_\theta$. This division is safe near π because $1 - c_\theta$ approaches 2. The result is

⁴The Taylor series $\sin \theta = \theta - \theta^3/6 + O(\theta^5)$ and $\cos \theta = 1 - \theta^2/2 + O(\theta^4)$ give $s_\theta \approx \theta$ but $1 - c_\theta \approx \theta^2/2$. For example, at $\theta = 10^{-8}$, these changes are about 10^{-8} and 5×10^{-17} , respectively. Floating-point rounding may therefore make c_θ equal to 1 and $\arccos(c_\theta)$ equal to zero, while s_θ still preserves the nonzero angle for atan2 . In exact arithmetic, both methods recover the same principal angle.

$$\mathbf{Q} := \frac{\frac{1}{2}(\mathbf{R} + \mathbf{R}^\top) - c_\theta \mathbf{I}_3}{1 - c_\theta} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top. \quad (8.35)$$

The matrix $\mathbf{Q}$ stores the axis through its outer product, not through the signed axis column itself. Writing ${}^a\mathbf{s}_\omega = [{}^a s_{\omega,1} \ {}^a s_{\omega,2} \ {}^a s_{\omega,3}]^\top$ makes its entries explicit:

$$\mathbf{Q} = \begin{bmatrix} ({}^a s_{\omega,1})^2 & {}^a s_{\omega,1} {}^a s_{\omega,2} & {}^a s_{\omega,1} {}^a s_{\omega,3} \\ {}^a s_{\omega,2} {}^a s_{\omega,1} & ({}^a s_{\omega,2})^2 & {}^a s_{\omega,2} {}^a s_{\omega,3} \\ {}^a s_{\omega,3} {}^a s_{\omega,1} & {}^a s_{\omega,3} {}^a s_{\omega,2} & ({}^a s_{\omega,3})^2 \end{bmatrix}.$$

Each diagonal entry is the square of one axis component. To recover the axis, select an index k for which ${}^a s_{\omega,k} \neq 0$ and first recover the magnitude of that component from $Q_{kk} = ({}^a s_{\omega,k})^2$. Then use each off-diagonal entry in column k . Because $Q_{jk} = {}^a s_{\omega,j} {}^a s_{\omega,k}$, dividing Q_{jk} by the recovered component ${}^a s_{\omega,k}$ gives ${}^a s_{\omega,j} = Q_{jk}/{}^a s_{\omega,k}$ for every $j \neq k$.

The denominator ${}^a s_{\omega,k}$ should be the largest-magnitude axis component. Choosing a component close to zero would make $Q_{jk}/{}^a s_{\omega,k}$ sensitive to round-off or undefined when that component is zero. Because $Q_{kk} = ({}^a s_{\omega,k})^2$, the largest diagonal entry of $\mathbf{Q}$ identifies the largest-magnitude component. Choose its index by

$$k := \arg \max_{i \in \{1,2,3\}} Q_{ii},$$

where $\arg \max$ returns the index i at which Q_{ii} is largest. Use the lowest index to break an exact numerical tie. Because the three squared components add to one, their largest value is at least $1/3$. Hence the selected component has magnitude at least $1/\sqrt{3}$ and supplies a safe denominator for a valid rotation.

The positive square root selects one of the two possible signs for the k th component. The entries in column k of $\mathbf{Q}$ then recover the remaining components:

$${}^a \tilde{s}_{\omega,k} := \sqrt{\max(0, Q_{kk})}, \quad {}^a \tilde{s}_{\omega,j} := \frac{Q_{jk}}{{}^a \tilde{s}_{\omega,k}}, \quad j \neq k, \quad (8.36)$$

The tilde marks this as a candidate obtained from floating-point data. Normalise the assembled column to unit length to remove small accumulated round-off error. The $\max(0, \cdot)$ operation may correct only a tiny negative value attributable to round-off after validation; a materially negative diagonal is a failure.

The outer product cannot distinguish an axis from its negative because

$${}^a\mathbf{s}_\omega({}^a\mathbf{s}_\omega)^\top = (-{}^a\mathbf{s}_\omega)(-{}^a\mathbf{s}_\omega)^\top.$$

For $\theta < \pi$, the small but nonzero skew column ${}^a\mathbf{u}$ supplies the missing sign. Choose the candidate's sign so that

$$({}^a\tilde{\mathbf{s}}_\omega)^\top {}^a\mathbf{u} > 0,$$

because the exact relation ${}^a\mathbf{u} = 2 \sin \theta {}^a\mathbf{s}_\omega$ makes this dot product positive. At exactly $\theta = \pi$, ${}^a\mathbf{u} = \mathbf{0}_3$ and no calculation from $\mathbf{R}$ can distinguish ${}^a\mathbf{s}_\omega$ from $-{}^a\mathbf{s}_\omega$: both describe the same rotation. A deterministic implementation must choose one. One suitable rule is to make the largest-magnitude axis component positive, again resolving a tie by the lowest index. After this sign choice, the principal rotation vector is

$${}^a\phi = \theta {}^a\mathbf{s}_\omega.$$

The canonical sign makes repeated calls return the same column. It cannot make the selected rotation vector continuous across the entire $\theta = \pi$ boundary because the two columns $\pi {}^a\mathbf{s}_\omega$ and $-\pi {}^a\mathbf{s}_\omega$ represent the same boundary rotation.

Completing the spatial logarithm

Recovering ${}^a\phi$ solves only the rotation part of the logarithm. A spatial transformation also contains the final translation ${}^a\mathbf{p}$, so return to the validated transformation

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & {}^a\mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3).$$

The finite linear logarithm block is not generally equal to ${}^a\mathbf{p}$ because translation accumulates while the body rotates. The previously derived relation ${}^a\mathbf{p} = \mathbf{J}({}^a\phi) {}^a\boldsymbol{\rho}$ must be inverted. The complete calculation is therefore:

1. Recover the principal rotation-vector block ${}^a\phi$ using the formula appropriate to the identity, nominal, or near- π case.
2. Form its cross-product matrix $\Phi = [{}^a\phi]_\times$.
3. Recover the finite linear block from ${}^a\boldsymbol{\rho} = \mathbf{J}({}^a\phi)^{-1} {}^a\mathbf{p}$.
4. Place these two blocks into the structured logarithm matrix:

$$\boxed{\log(\mathbf{T}) = \widehat{{}^a\boldsymbol{\eta}} = \begin{bmatrix} \Phi & {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}}. \quad (8.37)$$

If the rotation is the identity, then ${}^a\phi = \mathbf{0}_3$, $J^{-1} = \mathbf{I}_3$, and ${}^a\rho = {}^a\mathbf{p}$. This covers both remaining cases: ${}^a\mathbf{p} = \mathbf{0}_3$ gives the zero logarithm, while ${}^a\mathbf{p} \neq \mathbf{0}_3$ gives pure translation. Only after the finite blocks are known should an interface optionally factor out a normalised screw axis:

$$\begin{aligned} \theta > 0 : \quad {}^a\mathbf{S} &= \begin{bmatrix} {}^a\rho/\theta \\ {}^a\phi/\theta \end{bmatrix}, \quad \sigma = \theta, \\ \theta = 0, \|\rho\|_2 > 0 : \quad {}^a\mathbf{S} &= \begin{bmatrix} {}^a\rho/\|{}^a\rho\|_2 \\ \mathbf{0}_3 \end{bmatrix}, \quad \sigma = \|{}^a\rho\|_2. \end{aligned}$$

For the zero logarithm, no unique normalised screw axis exists.⁵ Near a pure translation, the finite logarithm ${}^a\eta$ remains well behaved while the finite-pitch normalisation ${}^a\rho/\theta$ can become very large. Software should therefore expose the finite logarithm as the primary result and treat normalised screw factorisation as a separate operation with its own domain.

Why the implementation needs different thresholds

A **threshold** is a declared cutoff used by an implementation to choose a case. The following thresholds answer different questions and therefore must not be merged.

The series threshold θ_{series} asks: *Is the angle small enough that a Taylor series is more accurate than direct trigonometric subtraction?* The near- π threshold ε_π asks: *Is the angle close enough to π that the skew column is unreliable and the symmetric axis formula should be used?* Both select between formulas for the same mathematical result.

The zero threshold $\varepsilon_{\text{zero}}$ asks a different question: *Does this interface intentionally treat a sufficiently small measured or modelled rotation as exactly zero?* This policy may change a nonzero input into the identity rotation, so it changes the returned model rather than merely changing the evaluation formula.

The translation-zero tolerance $\varepsilon_{p,0}$ asks whether the translation column ${}^a\mathbf{p}$ is small enough to receive a zero-translation status. It is used when an identity rotation must be classified as either a complete identity or a pure translation. Define

⁵The zero logarithm has ${}^a\eta = \mathbf{0}_6$ and may be factored as ${}^a\eta = {}^a\mathbf{S}\sigma$ with $\sigma = 0$. Therefore

$${}^a\mathbf{S} \mathbf{0} = \mathbf{0}_6$$

for every admissible normalised screw axis ${}^a\mathbf{S}$. For example, let ${}^a\mathbf{S}_{R,z}$ and ${}^a\mathbf{S}_{R,x}$ denote normalised revolute screws about the z_a - and x_a -axes through O_a , and let ${}^a\mathbf{S}_{P,y}$ denote a normalised prismatic screw along positive y_a . Then

$$\exp(\widehat{{}^a\mathbf{S}_{R,z}} \mathbf{0}) = \exp(\widehat{{}^a\mathbf{S}_{R,x}} \mathbf{0}) = \exp(\widehat{{}^a\mathbf{S}_{P,y}} \mathbf{0}) = \mathbf{I}_4.$$

The identity transformation therefore contains no information that selects one of these screw axes.

$$\varepsilon_{p,0} \in [0, \infty) \text{ m}, \quad \|{}^a\mathbf{p}\|_2 \leq \varepsilon_{p,0} \implies {}^a\mathbf{p} \text{ is classified as zero translation}.$$

This comparison is measured in metres and cannot use an angular tolerance. By itself, the classification does not replace ${}^a\mathbf{p}$ or ${}^a\boldsymbol{\rho}$ with $\mathbf{0}_3$; an interface that also canonicalises the stored or returned translation to zero must declare that separate behaviour. An interface that preserves every representable nonzero translation should set $\varepsilon_{p,0} = 0$ m.

Threshold	Unit	Numerical role	Does it change the represented motion?
θ_{series}	rad	Selects series or trigonometric evaluation near zero	No
ε_{π}	rad	Selects the symmetric near- π axis extraction	No
$\varepsilon_{\text{zero}}$	rad	Declares an angle indistinguishable from zero at an interface	Yes, if a nonzero angle is canonicalised to zero
$\varepsilon_{p,0}$	m	Classifies a translation as zero or nonzero	No, unless the interface also canonicalises the translation column

The first two thresholds choose stable formulas whose outputs should agree where their domains overlap. The angular zero threshold and translation-zero tolerance are interface-resolution policies and must be explicit. An interface that promises to preserve every representable nonzero rotation should use the small-angle series and reserve the zero case for an exact identity. If an interface deliberately canonicalises small rotations or translations to zero, it must document that behaviour.

These cutoffs are part of the public numerical behaviour. Every configured value must be finite and nonnegative, with $0 < \theta_{\text{series}} < \pi$ and $0 \leq \varepsilon_{\pi} < \pi$. Formula-selection regions should not overlap; for example, require $\theta_{\text{series}} < \pi - \varepsilon_{\pi}$. If overlap is allowed, the implementation must declare which case takes precedence. Reject an invalid threshold configuration before processing a transformation.

8.5.4 Validation, deterministic evaluation, and evidence

Why validation comes first

The logarithm formulas above assume that their input belongs to $SE(3)$. An arbitrary 4×4 array does not satisfy that assumption, even if some of its entries resemble a transformation. For example, the matrix $\text{diag}(1, 1, -1)$ has orthonormal columns but repres-

ents a reflection because its determinant is -1 . Feeding it directly to a rotation-logarithm formula could produce plausible-looking numbers that do not describe any rotation.

Validation therefore answers the question *Does this array represent the mathematical object accepted by the logarithm?* Formula selection begins only after the answer is yes. In particular, a logarithm routine must not use the trace, clamp an inverse-trigonometric input, or extract an axis before applying the representation checks from Section 7.10.

Write a candidate transformation as

$$\tilde{\mathbf{T}} = \begin{bmatrix} \tilde{\mathbf{R}} & \tilde{\mathbf{p}} \\ \tilde{\mathbf{r}}^\top & \tilde{t}_{44} \end{bmatrix} \in \mathbb{R}^{4 \times 4}.$$

Here $\tilde{\mathbf{R}} \in \mathbb{R}^{3 \times 3}$ is the candidate rotation block, $\tilde{\mathbf{p}} \in \mathbb{R}^3$ is the candidate translation column, and $[\tilde{\mathbf{r}}^\top \ \tilde{t}_{44}]$ is the candidate bottom row. The tilde marks quantities that have not yet passed validation.

First check that the array has shape 4×4 and that every component is finite. A value containing NaN or infinity cannot represent a finite rigid displacement and must be rejected before norms, determinants, or trigonometric functions are evaluated.

A valid rotation has orthonormal columns, which is equivalent to $\tilde{\mathbf{R}}^\top \tilde{\mathbf{R}} = \mathbf{I}_3$. The Frobenius-norm residual

$$e_{\text{orth}} := \|\tilde{\mathbf{R}}^\top \tilde{\mathbf{R}} - \mathbf{I}_3\|_{\text{F}}$$

measures the combined failure of those column lengths and mutual dot products. It is dimensionless and must lie within a declared dimensionless tolerance.

Orthogonality alone also accepts reflections. A proper rotation must additionally satisfy $\det(\tilde{\mathbf{R}}) = 1$, so define the separate dimensionless determinant residual

$$e_{\text{det}} := |\det(\tilde{\mathbf{R}}) - 1|.$$

This residual needs its own tolerance. The reflection $\text{diag}(1, 1, -1)$ has $e_{\text{orth}} = 0$ but $e_{\text{det}} = 2$, which shows why the determinant check cannot be omitted.

A homogeneous rigid transformation must have the exact bottom row $[0 \ 0 \ 0 \ 1]$. The residual

$$e_{\text{row}} := \left\| \begin{bmatrix} \tilde{\mathbf{r}} \\ \tilde{t}_{44} - 1 \end{bmatrix} \right\|_{\infty}$$

measures the largest absolute departure from that required row and must lie within a declared dimensionless tolerance. A valid-looking rotation block cannot compensate for a malformed bottom row.

Finally, the translation column $\tilde{\mathbf{p}}$ must be finite, expressed in the declared frame, measured in metres, and contained in the magnitude range supported by the interface. Do not combine this metre-valued check with the dimensionless rotation and bottom-row checks in one unscaled 4×4 norm.

Validation and repair remain separate operations. An interface may define a distinct, explicit projection that maps a nearby matrix to $SO(3)$, but the logarithm must not silently project an invalid matrix and return a valid-looking scientific result. A reflection, a materially non-orthogonal rotation, a malformed bottom row, or any non-finite component is rejected.

The exponential input also has a contract. The preferred finite input ${}^a\boldsymbol{\eta} = [{}^a\boldsymbol{\rho}^\top \ {}^a\phi^\top]^\top$ must contain six finite components, with the first block expressed in metres and the second in radians. If an interface instead accepts a normalised screw and a separate amount, it must distinguish the two valid normalisations:

- for a rotational or helical screw, $\|{}^a\mathbf{s}_\omega\|_2 = 1$, ${}^a\mathbf{s}_v$ is finite in m rad^{-1} , and $\sigma = \theta$ is in radians;
- for a prismatic screw, ${}^a\mathbf{s}_\omega = \mathbf{0}_3$, $\|{}^a\mathbf{s}_v\|_2 = 1$, and $\sigma = d$ is in metres.

The six-entry screw must not be normalised with one ordinary Euclidean norm because its blocks have different units. For a constant-twist update, the linear and angular twist blocks are measured in m s^{-1} and rad s^{-1} , respectively, and the time increment must be finite. The pose-integration model developed above assumes $\Delta t \geq 0$; a generic signed screw displacement is a separate interface.

Finiteness of the inputs alone does not guarantee safe intermediates. A naive sum of squares can overflow while computing a norm even when every component is finite, so an implementation should use a scaled norm or a library routine with equivalent protection. Extremely large angular arguments can also lose accuracy during trigonometric range reduction. The supported magnitude range must be bounded or the implementation must detect non-finite or unreliable intermediates. Reducing a helical displacement modulo 2π is not generally valid: the rotation repeats, but the axial displacement $h\theta^a\mathbf{s}_\omega$ does not.

Why the order of cases matters

A deterministic procedure returns the same selected coordinates, case classification, and failure status for the same validated input and the same declared configuration. This requires more than deterministic arithmetic: the procedure must test exceptional cases before it evaluates formulas that divide by quantities which vanish in those cases.

For a validated logarithm input, use the following order:

1. Compute c_θ , ${}^a\mathbf{u}$, and s_θ . A tiny post-validation round-off overshoot in c_θ may be clamped to $[-1, 1]$; a material overshoot is a validation failure. Then calculate $\theta = \text{atan2}(s_\theta, c_\theta)$.

2. Test the identity case before attempting to normalise an axis. If the rotation is an exact or explicitly canonicalised identity, set ${}^a\phi = \mathbf{0}_3$. Because $\mathbf{J}(\mathbf{0}_3) = \mathbf{I}_3$, preserve ${}^a\rho = {}^a\mathbf{p}$. Classify the complete transformation as the identity only if its translation also satisfies the separately declared metre-valued zero criterion.
3. Otherwise, test whether $\pi - \theta \leq \varepsilon_\pi$. If so, the skew column is unreliable; recover the axis from $\mathbf{Q}$ and apply the declared sign rule.
4. In the remaining range, recover ${}^a\phi$ from ${}^a\mathbf{u}$. Use the series for $\theta/(2 \sin \theta)$ when $\theta \leq \theta_{\text{series}}$ and the direct trigonometric expression above that threshold.
5. Recover translation after rotation. Form $\mathbf{J}^{-1}$ using the series for $D(\theta)$ in the small-angle region and the half-angle expression elsewhere, then calculate ${}^a\rho = \mathbf{J}^{-1}{}^a\mathbf{p}$.
6. Return the finite logarithm coordinates and a status that identifies the selected case. Factor a normalised screw only when the caller requests it and the required division is defined.

The near- π test precedes division by s_θ , and the identity test precedes every axis normalisation. This order prevents division by a quantity known to be unreliable. Threshold comparisons, tie-breaking indices, and axis-sign rules are part of deterministic behaviour and must be tested at values just below, exactly on, and just above their boundaries.

The exponential follows a shorter deterministic order:

1. Validate and combine the input into finite blocks ${}^a\rho$ and ${}^a\phi$.
2. Compute $\theta = \|{}^a\phi\|_2$ safely and $\Phi = [{}^a\phi]_\times$.
3. Evaluate A , B , and C by their series or trigonometric formula branches.
4. Construct $\mathbf{R} = \mathbf{I}_3 + A\Phi + B\Phi^2$ and ${}^a\mathbf{p} = \mathbf{J}^a\rho$, set the bottom row exactly to $[0 \ 0 \ 0 \ 1]$, and reject any non-finite result.

How to test the result independently

A test **oracle** is an independently obtained expected result or invariant. Tests must check such geometric expectations, not only compare the function with another copy of the same formulas. For example, after calculating a logarithm and exponentiating it again, compare the reconstructed transformation with the original.

For an input transformation $\mathbf{T} = [\mathbf{R} \ {}^a\mathbf{p}; \mathbf{0}_3^\top \ 1]$ and a reconstructed result $\mathbf{T}' = [\mathbf{R}' \ {}^a\mathbf{p}'; \mathbf{0}_3^\top \ 1]$, define separate round-trip residuals

$$e_{R,\text{rt}} := \|\mathbf{R}' - \mathbf{R}\|_F, \quad e_{p,\text{rt}} := \|{}^a\mathbf{p}' - {}^a\mathbf{p}\|_2.$$

The rotation residual $e_{R,\text{rt}}$ is dimensionless and measures the matrix difference between the reconstructed and original orientations. The translation residual $e_{p,\text{rt}}$ is measured in metres and measures the Euclidean distance between their translation columns. They require separate tolerances because they have different meanings and units. Useful deterministic cases and independent expectations include:

Case	Independent expectation
Identity	The logarithm is the zero matrix; no normalised axis is claimed
Pure translation	${}^a\phi = \mathbf{0}_3$ and ${}^a\rho = {}^a\mathbf{p}$
Small nonzero rotation	Series and trigonometric formula branches agree near θ_{series} and the rotation is not silently erased
Revolute axis through an offset point	The chosen axis point is fixed by the finite displacement
Finite-pitch screw	Every axis point advances by $h\theta$ and the logarithm recovers the same finite exponential coordinates
Exact π rotations	Coordinate-axis and non-coordinate-axis cases follow the declared index and sign rules
Principal round trip	$\exp(\log(\mathbf{T})) \approx \mathbf{T}$ with separate rotation and translation residuals
Left-Jacobian inverse	$\mathbf{J}\mathbf{J}^{-1} \approx \mathbf{I}_3$ for small, nominal, and near- π angles
Invalid representations	Reflections, malformed bottom rows, non-orthogonal rotations, and non-finite values return no valid logarithm

The reverse expression $\log(\exp(\widehat{{}^a\eta})) \approx \widehat{{}^a\eta}$ needs a branch qualification. It is expected when $\|{}^a\phi\|_2 < \pi$ and the input already follows the selected principal convention. For angles beyond the principal interval, the reconstructed transformation may be exact while the returned logarithm legitimately uses different exponential coordinates. At $\theta = \pi$, raw axis columns should be compared only after applying the canonical sign rule, or the reconstructed rotations should be compared instead.

Tests around numerical thresholds should use fixed, explicitly listed values rather than random samples alone. If random stress cases are added, their seed and input domain must be fixed, and they supplement rather than replace analytic cases. Comparison with a trusted library is useful as an additional oracle only when the library's twist order, active or passive convention, frame meaning, and logarithm branch have first been mapped to the project convention.

8.5.5 Worked spatial cases

The following cases are not additional definitions. Each one shows why one of the numerical branches above is needed and checks it against a geometric or algebraic expectation.

A small rotation with finite translation

This case asks whether a very small but nonzero rotation can be preserved without losing its effect on translation. The series formulas should produce a transformation close to pure translation while retaining the first-order rotational correction.

The finite exponential-coordinate formulas used in this case are

$$\begin{aligned}\theta &= \|{}^a\phi\|_2, & \Phi &= [{}^a\phi]_{\times}, \\ \mathbf{R} &= \mathbf{I}_3 + A(\theta)\Phi + B(\theta)\Phi^2, & \mathbf{J} &= \mathbf{I}_3 + B(\theta)\Phi + C(\theta)\Phi^2, \\ {}^a\mathbf{p} &= \mathbf{J}^a\boldsymbol{\rho}.\end{aligned}$$

For a sufficiently small θ , the series values $A(\theta) = 1 + O(\theta^2)$, $B(\theta) = 1/2 + O(\theta^2)$, and $C(\theta) = 1/6 + O(\theta^2)$ replace direct evaluation of the trigonometric ratios.

Let the finite exponential-coordinate blocks referred to frame $\{a\}$ be

$${}^a\boldsymbol{\rho} = \begin{bmatrix} 0.3 \\ -0.2 \\ 0.1 \end{bmatrix} \text{ m}, \quad {}^a\phi = \begin{bmatrix} 0 \\ 0 \\ 10^{-8} \end{bmatrix} \text{ rad}.$$

Then $\theta = 10^{-8}$ rad and

$$\Phi = \begin{bmatrix} 0 & -10^{-8} & 0 \\ 10^{-8} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

The small-angle series give

$$\mathbf{R} = \mathbf{I}_3 + \Phi + \frac{1}{2}\Phi^2 + O(\theta^3), \quad \mathbf{J} = \mathbf{I}_3 + \frac{1}{2}\Phi + \frac{1}{6}\Phi^2 + O(\theta^3).$$

For the translation calculation,

$$\Phi^a\boldsymbol{\rho} = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix} 10^{-9} \text{ m}, \quad \Phi^2 {}^a\boldsymbol{\rho} = \begin{bmatrix} -3 \\ 2 \\ 0 \end{bmatrix} 10^{-17} \text{ m}.$$

Hence

$$\begin{aligned}
{}^a\mathbf{p} &= \mathbf{J}^a \boldsymbol{\rho} \\
&= {}^a\boldsymbol{\rho} + \frac{1}{2}\boldsymbol{\Phi}^a \boldsymbol{\rho} + \frac{1}{6}\boldsymbol{\Phi}^{2a} \boldsymbol{\rho} + O(10^{-25} \text{ m}) \\
&\approx \begin{bmatrix} 0.300000001 \\ -0.1999999985 \\ 0.1 \end{bmatrix} \text{ m}.
\end{aligned}$$

The change in translation caused by the small rotation is physically tiny but still represented. Directly forming $1 - \cos(10^{-8})$ can lose that information in finite precision, whereas the series evaluates the limiting coefficients directly.

An exact π rotation about a non-coordinate axis

This case shows why the skew-column axis formula cannot be used at $\theta = \pi$ and how the symmetric matrix $\mathbf{Q}$ recovers the unsigned axis.

The principal-angle formulas used first are

$$\begin{aligned}
c_\theta &= \frac{\text{tr}(\mathbf{R}) - 1}{2}, \quad {}^a\mathbf{u} = \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}, \\
s_\theta &= \frac{1}{2} \|{}^a\mathbf{u}\|_2, \quad \theta = \text{atan2}(s_\theta, c_\theta).
\end{aligned}$$

When s_θ vanishes at $\theta = \pi$, the axis is recovered from

$$\mathbf{Q} = \frac{\frac{1}{2}(\mathbf{R} + \mathbf{R}^\top) - c_\theta \mathbf{I}_3}{1 - c_\theta} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top.$$

The component calculation selects $k = \arg \max_i Q_{ii}$, recovers ${}^a s_{\omega,k} = \sqrt{Q_{kk}}$, and then uses ${}^a s_{\omega,j} = Q_{jk} / {}^a s_{\omega,k}$ for $j \neq k$, followed by the declared sign rule.

Consider the unit axis and rotation

$${}^a\mathbf{s}_\omega = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad \theta = \pi.$$

For an exact π rotation, Rodrigues' formula reduces to $\mathbf{R} = 2{}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top - \mathbf{I}_3$, giving

$$\mathbf{R} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

Here $c_\theta = -1$, ${}^a\mathbf{u} = \mathbf{0}_3$, and $\theta = \text{atan2}(0, -1) = \pi$. Away from π , the ordinary skew-based formulas recover the axis and rotation vector through

$${}^a\mathbf{s}_\omega = \frac{{}^a\mathbf{u}}{2s_\theta}, \quad {}^a\phi = \frac{\theta}{2s_\theta} {}^a\mathbf{u}.$$

At $\theta = \pi$, both formulas would divide ${}^a\mathbf{u} = \mathbf{0}_3$ by $s_\theta = 0$ and therefore contain the undefined ratio $\mathbf{0}_3/0$. The symmetric near- π formula avoids this division and gives

$$\mathbf{Q} = \frac{\mathbf{R} + \mathbf{I}_3}{2} = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 1/2 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

The first two diagonal entries tie, so the declared lowest-index rule chooses $k = 1$. The component rule Equation 8.36 and the positive-largest-component sign convention recover

$${}^a\mathbf{s}_\omega = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad {}^a\phi = \frac{\pi}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

The opposite axis would describe the same physical π rotation, but the canonical rule ensures that repeated logarithms return the same component column.

Recovering ${}^a\phi$ from $\mathbf{R}$ completes only the rotational part of the logarithm. The finite linear block ${}^a\boldsymbol{\rho}$ also depends on the transformation's translation column ${}^a\mathbf{p}$, because

$${}^a\mathbf{p} = \mathbf{J}({}^a\phi) {}^a\boldsymbol{\rho}, \quad {}^a\boldsymbol{\rho} = \mathbf{J}({}^a\phi)^{-1} {}^a\mathbf{p}.$$

In this example, take the complete transformation to be a pure rotation about an axis through the frame origin O_a , so

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{0}_3 \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad {}^a\mathbf{p} = \mathbf{0}_3.$$

The principal-angle matrix $\mathbf{J}({}^a\phi)$ is invertible at $\theta = \pi$, and therefore

$${}^a\boldsymbol{\rho} = \mathbf{J}({}^a\boldsymbol{\phi})^{-1}\mathbf{0}_3 = \mathbf{0}_3.$$

If the same rotation block were paired with a nonzero translation column, the rotation vector would remain the same, but ${}^a\boldsymbol{\rho}$ would instead be obtained by applying $\mathbf{J}({}^a\boldsymbol{\phi})^{-1}$ to that declared translation column.

A finite-pitch helical round trip

This case starts from independently declared screw geometry, constructs its transformation, and then applies the logarithm. Recovering the original finite blocks checks that the exponential and logarithm agree on a motion that combines an offset rotation axis with axial translation.

The screw-coordinate formulas used to construct the finite motion are

$$\begin{aligned} {}^a\mathbf{s}_v &= -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega, \\ {}^a\boldsymbol{\rho} &= {}^a\mathbf{s}_v\theta, & {}^a\boldsymbol{\phi} &= {}^a\mathbf{s}_\omega\theta. \end{aligned}$$

The geometric helical exponential then gives

$$\begin{aligned} \mathbf{R} &= \exp([\mathbf{s}_\omega]_\times \theta), \\ {}^a\mathbf{p} &= (\mathbf{I}_3 - \mathbf{R}){}^a\mathbf{p}_\ell + h\theta{}^a\mathbf{s}_\omega, \\ \mathbf{T} &= \begin{bmatrix} \mathbf{R} & {}^a\mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}. \end{aligned}$$

For the reverse calculation, the logarithm recovers ${}^a\boldsymbol{\phi}$ from $\mathbf{R}$ and then uses ${}^a\boldsymbol{\rho} = \mathbf{J}({}^a\boldsymbol{\phi})^{-1}{}^a\mathbf{p}$.

Let the oriented axis point along positive z_a , pass through

$${}^a\mathbf{p}_\ell = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m},$$

and have pitch $h = 0.2 \text{ m rad}^{-1}$. For $\theta = \pi/2 \text{ rad}$,

$${}^a\mathbf{s}_\omega = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad {}^a\mathbf{s}_v = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega = \begin{bmatrix} 0 \\ -1 \\ 0.2 \end{bmatrix} \text{ m rad}^{-1}.$$

The finite logarithm blocks are

$${}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v\theta = \begin{bmatrix} 0 \\ -\pi/2 \\ \pi/10 \end{bmatrix} \text{ m}, \quad {}^a\boldsymbol{\phi} = {}^a\mathbf{s}_\omega\theta = \begin{bmatrix} 0 \\ 0 \\ \pi/2 \end{bmatrix} \text{ rad.}$$

To calculate the rotation block, first form the cross-product matrix of the unit axis and its square:

$$\boldsymbol{\Omega} := [{}^a\mathbf{s}_\omega]_\times = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \boldsymbol{\Omega}^2 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Rodrigues' formula evaluates the rotational exponential:

$$\begin{aligned} \mathbf{R} &= \exp(\boldsymbol{\Omega}\theta) \\ &= \mathbf{I}_3 + \sin\theta \boldsymbol{\Omega} + (1 - \cos\theta)\boldsymbol{\Omega}^2 \\ &= \mathbf{I}_3 + \boldsymbol{\Omega} + \boldsymbol{\Omega}^2 \\ &= \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

where the third equality uses $\sin(\pi/2) = 1$ and $\cos(\pi/2) = 0$. Geometrically, this matrix rotates the positive x_a direction onto positive y_a , rotates positive y_a onto negative x_a , and leaves the axis direction z_a fixed.

The translation formula from the geometric helical exponential Equation 8.5 then gives

$${}^a\mathbf{p} = (\mathbf{I}_3 - \mathbf{R}){}^a\mathbf{p}_\ell + h\theta{}^a\mathbf{s}_\omega = \begin{bmatrix} 1 \\ -1 \\ \pi/10 \end{bmatrix} \text{ m.}$$

Therefore

$$\mathbf{T} = \begin{bmatrix} 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & \pi/10 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The first three entries of the fourth column are measured in metres; the rotation block and bottom row are dimensionless.

Now apply the logarithm to the constructed transformation. The trace of $\mathbf{R}$ is 1, and its skew-difference data column is

$${}^a\mathbf{u} = \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix}.$$

Therefore

$$\begin{aligned} c_\theta &= \frac{\text{tr}(\mathbf{R}) - 1}{2} = 0, \\ s_\theta &= \frac{1}{2} \|{}^a\mathbf{u}\|_2 = 1, \\ \theta &= \text{atan2}(s_\theta, c_\theta) = \frac{\pi}{2}, \\ {}^a\mathbf{s}_\omega &= \frac{{}^a\mathbf{u}}{2s_\theta} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \\ {}^a\boldsymbol{\phi} &= \theta {}^a\mathbf{s}_\omega = \begin{bmatrix} 0 \\ 0 \\ \pi/2 \end{bmatrix} \text{ rad.} \end{aligned}$$

The rotation block has now recovered ${}^a\boldsymbol{\phi}$, but the translation column must still be converted into the finite linear exponential-coordinate block. For this rotation vector,

$$\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_\times = \begin{bmatrix} 0 & -\pi/2 & 0 \\ \pi/2 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Substituting $\theta = \pi/2$ into the left-Jacobian formula

$$\mathbf{J}({}^a\boldsymbol{\phi}) = \mathbf{I}_3 + \frac{1 - \cos \theta}{\theta^2} \boldsymbol{\Phi} + \frac{\theta - \sin \theta}{\theta^3} \boldsymbol{\Phi}^2$$

gives the following matrix. Its inverse is obtained by inverting the upper-left 2×2 block; its third coordinate remains unchanged because rotation about z_a does not mix that coordinate with the perpendicular plane:

$$\mathbf{J}({}^a\boldsymbol{\phi}) = \begin{bmatrix} 2/\pi & -2/\pi & 0 \\ 2/\pi & 2/\pi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{J}({}^a\boldsymbol{\phi})^{-1} = \begin{bmatrix} \pi/4 & \pi/4 & 0 \\ -\pi/4 & \pi/4 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Using Equation 8.30 gives

$$\begin{aligned}
 {}^a\boldsymbol{\rho} &= \mathbf{J}({}^a\boldsymbol{\phi})^{-1} {}^a\mathbf{p} \\
 &= \begin{bmatrix} \pi/4 & \pi/4 & 0 \\ -\pi/4 & \pi/4 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ \pi/10 \end{bmatrix} \text{ m} \\
 &= \begin{bmatrix} 0 \\ -\pi/2 \\ \pi/10 \end{bmatrix} \text{ m}.
 \end{aligned}$$

Finally, the finite blocks factor as ${}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v\theta$ and ${}^a\boldsymbol{\phi} = {}^a\mathbf{s}_\omega\theta$. Dividing both blocks by the recovered angle gives

$${}^a\mathbf{s}_v = \frac{{}^a\boldsymbol{\rho}}{\theta} = \begin{bmatrix} 0 \\ -1 \\ 0.2 \end{bmatrix} \text{ m rad}^{-1}, \quad {}^a\mathbf{s}_\omega = \frac{{}^a\boldsymbol{\phi}}{\theta} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

The pitch recovered from these screw coordinates is

$$h = ({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_v = 0.2 \text{ m rad}^{-1},$$

which matches the declared screw geometry. The forward calculation began with the physical axis point, axis direction, and pitch, whereas the inverse calculation used only the resulting matrix blocks $\mathbf{R}$ and ${}^a\mathbf{p}$. Recovering the original screw data therefore checks the logarithm against an independently expressed geometric construction rather than merely reversing the same matrix formula.

Quick test D: numerical branches and validation

1. **Terminology:** Distinguish a logarithm, a logarithm branch, the principal-branch logarithm, a branch boundary, a numerical formula branch, and a deterministic branch policy.
2. **Representation:** What are the blocks ${}^a\boldsymbol{\rho}$ and ${}^a\boldsymbol{\phi}$ in the finite exponential-coordinate column, and what are their units?
3. **Derivation:** Starting from ${}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v\theta$ and $\boldsymbol{\Phi} = \boldsymbol{\Omega}\theta$, show that $\mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho} = \mathbf{G}(\theta){}^a\mathbf{s}_v$.
4. **Numerical explanation:** Why are the direct expressions $1 - \cos \theta$ and $\theta - \sin \theta$ inaccurate for a sufficiently small θ ? What replaces them?
5. **Branch reasoning:** Why does the skew-difference axis formula fail near $\theta = \pi$, and why does the symmetric matrix $\mathbf{Q}$ remain useful there?

6. **Inverse problem:** Once ${}^a\phi$ has been recovered from a validated $\mathbf{T}$, how is ${}^a\rho$ obtained? Why should a normalised screw not be formed first near a pure translation?
7. **Validation:** Why must c_θ be clamped only after $SO(3)$ validation? Give two invalid inputs that must be rejected rather than repaired by clamping.
8. **Thresholds:** Distinguish θ_{series} , ε_π , and $\varepsilon_{\text{zero}}$.
9. **Round trips:** Why is $\exp(\log(\mathbf{T})) \approx \mathbf{T}$ a general principal-branch check, while $\log(\exp(\widehat{{}^a\eta})) \approx \widehat{{}^a\eta}$ requires a branch qualification?

Answers and hints

1. A logarithm is any $\mathbf{X} \in \mathfrak{se}(3)$ satisfying $\exp(\mathbf{X}) = \mathbf{T}$. A logarithm branch is a single-valued rule that selects one such $\mathbf{X}$. The principal-branch rule chooses a rotation-vector magnitude in $[0, \pi]$ and applies the declared identity and π conventions. A branch boundary is where that representative must switch, producing a coordinate discontinuity even when the transformation remains continuous. A numerical formula branch chooses an algebraically equivalent evaluation method for stability rather than a different exact logarithm. A deterministic policy fixes case order, comparisons, ties, signs, outputs, and failures so repeated calls agree within the same declared implementation and floating-point environment.
2. ${}^a\rho \in \mathbb{R}^3$ is the finite linear exponential-coordinate block in metres, and ${}^a\phi \in \mathbb{R}^3$ is the rotation-vector block in radians. Both are expressed in frame $\{a\}$, and the complete column uses the project's linear-first order.
3. Substitute $\Phi = \Omega\theta$ and ${}^a\rho = {}^a\mathbf{s}_v\theta$ into $\mathbf{J} = \mathbf{I}_3 + B\Phi + C\Phi^2$. The three resulting coefficients are θ , $1 - \cos \theta$, and $\theta - \sin \theta$, which are exactly the coefficients of $\mathbf{G}(\theta)$.
4. Each expression subtracts nearly equal floating-point numbers, so its small exact result can lose significant digits. Evaluate the Taylor series for A , B , C , and D below the declared series threshold.
5. The skew difference is ${}^a\mathbf{u} = 2 \sin \theta {}^a\mathbf{s}_\omega$, so it approaches zero as $\theta \rightarrow \pi$. The symmetric relation divides by $1 - \cos \theta$, which approaches 2, and recovers the outer product ${}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top$ without that vanishing denominator.
6. Calculate ${}^a\rho = \mathbf{J}({}^a\phi)^{-1} {}^a\mathbf{p}$. Forming ${}^a\mathbf{s}_v = {}^a\rho/\theta$ first divides by a small angle and becomes ill-conditioned as a rotational screw approaches pure translation; the finite block itself remains well behaved.
7. Clamping is only a round-off correction for a matrix already known to be a valid rotation within tolerance. A reflection such as $\text{diag}(1, 1, -1)$, a materially non-orthogonal matrix, a malformed homogeneous bottom row, or a matrix containing NaN or infinity must be rejected.
8. θ_{series} selects a stable evaluation formula near zero, and ε_π selects a stable axis-extraction formula near π ; neither should change the represented motion. $\varepsilon_{\text{zero}}$ declares interface resolution and can intentionally canonicalise a nonzero angle to zero, so it changes the returned model and must be explicit.
9. The principal logarithm is selected to reconstruct its input transformation, so exponentiating it should recover $\mathbf{T}$. The reverse direction returns the selected principal

representative: angles outside $[0, \pi]$, the sign ambiguity at π , and the undefined axis at zero can produce different exponential-coordinate columns for the same transformation.

Motion workflow and symbol summary

This section introduces no new representation. It collects the symbols already defined above and shows which ones are needed for three different questions: finite displacement, instantaneous velocity, and recovery of motion coordinates from a known transformation.

The three workflows

Use the **finite-displacement workflow** when the screw geometry ${}^a\mathbf{S}$ and signed motion amount σ are known and the required result is a transformation:

$$\begin{aligned} ({}^a\mathbf{S}, \sigma) &\mapsto {}^a\boldsymbol{\eta} = {}^a\mathbf{S}\sigma, \\ {}^a\boldsymbol{\eta} &\mapsto \widehat{{}^a\boldsymbol{\eta}}, \\ \widehat{{}^a\boldsymbol{\eta}} &\stackrel{\text{exp}}{\mapsto} \mathbf{T}_\Delta. \end{aligned}$$

The result $\mathbf{T}_\Delta$ can be applied to material points or composed with an existing pose. This is the direction used when joint geometry and joint displacement are known.

Use the **instantaneous-velocity workflow** when the screw geometry and current scalar rate $\dot{\sigma}$ are known:

$$\begin{aligned} ({}^a\mathbf{S}, \dot{\sigma}) &\mapsto {}^a\boldsymbol{\xi} = {}^a\mathbf{S}\dot{\sigma} = \begin{bmatrix} {}^a\boldsymbol{\nu} \\ {}^a\boldsymbol{\omega} \end{bmatrix}, \\ ({}^a\boldsymbol{\xi}, {}^a\mathbf{x}) &\mapsto {}^a\mathbf{v}_P = {}^a\boldsymbol{\nu} + {}^a\boldsymbol{\omega} \times {}^a\mathbf{x}. \end{aligned}$$

The twist ${}^a\boldsymbol{\xi}$ represents the common rigid velocity field. The point position ${}^a\mathbf{x}$ is needed only when that field is evaluated to obtain the velocity of a particular material point P .

Use the **inverse workflow** when $\mathbf{T}_\Delta$ is known but finite motion coordinates are required:

$$\begin{aligned} \mathbf{T}_\Delta &\stackrel{\text{selected log}}{\mapsto} \widehat{{}^a\boldsymbol{\eta}}, \\ \widehat{{}^a\boldsymbol{\eta}} &\stackrel{\text{vee}}{\mapsto} {}^a\boldsymbol{\eta} = \begin{bmatrix} {}^a\boldsymbol{\rho} \\ {}^a\phi \end{bmatrix}. \end{aligned}$$

The vee operator reverses the hat operator by extracting the six-component column from its structured 4×4 matrix. The inverse workflow is useful when recovering a screw displacement, constructing intermediate poses along the selected screw path, or converting a relative pose error into a six-component column. If the task is only to transform points or compose known poses, $\mathbf{T}_\Delta$ is already sufficient and its logarithm is unnecessary.

From ${}^a\boldsymbol{\eta}$ to $\mathbf{T}_\Delta$ and back

The finite-displacement and inverse workflows form a forward-and-back pair. The forward direction starts with the finite exponential-coordinate column

$${}^a\boldsymbol{\eta} = \begin{bmatrix} {}^a\boldsymbol{\rho} \\ {}^a\boldsymbol{\phi} \end{bmatrix},$$

where ${}^a\boldsymbol{\rho} \in \mathbb{R}^3$ is measured in metres and ${}^a\boldsymbol{\phi} \in \mathbb{R}^3$ is measured in radians. Form its hat matrix using $\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_\times$:

$$\widehat{{}^a\boldsymbol{\eta}} = \begin{bmatrix} \boldsymbol{\Phi} & {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

The exponential converts this finite generator into a rigid-displacement transformation:

$$\begin{aligned} \mathbf{T}_\Delta &= \exp(\widehat{{}^a\boldsymbol{\eta}}) \\ &= \begin{bmatrix} \mathbf{R}({}^a\boldsymbol{\phi}) & \mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}. \end{aligned}$$

Thus ${}^a\boldsymbol{\phi}$ determines the rotation block $\mathbf{R}$, while ${}^a\boldsymbol{\rho}$ and the same rotation vector together determine the final translation column

$${}^a\mathbf{p} = \mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho}.$$

The inverse direction starts with a validated transformation

$$\mathbf{T}_\Delta = \begin{bmatrix} \mathbf{R} & {}^a\mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

First apply the selected $SO(3)$ logarithm rule to $\mathbf{R}$ to recover the finite rotation vector ${}^a\boldsymbol{\phi}$. Then recover the finite linear block from the translation column:

$${}^a\boldsymbol{\rho} = \mathbf{J}({}^a\boldsymbol{\phi})^{-1} {}^a\mathbf{p}.$$

Finally assemble ${}^a\boldsymbol{\eta} = [({}^a\boldsymbol{\rho})^\top ({}^a\boldsymbol{\phi})^\top]^\top$. Equivalently, the selected matrix logarithm and vee operator perform these steps together:

$$\begin{array}{c} {}^a\boldsymbol{\eta} \xrightarrow{\text{hat}} \widehat{{}^a\boldsymbol{\eta}} \xrightarrow{\text{exp}} \mathbf{T}_\Delta, \\ \mathbf{T}_\Delta \xrightarrow{\text{selected log}} \widehat{{}^a\boldsymbol{\eta}} \xrightarrow{\text{vee}} {}^a\boldsymbol{\eta}. \end{array}$$

The column ${}^a\boldsymbol{\eta}$ contains finite displacement amounts, not rates per second, so it is not an instantaneous twist. The transformation $\mathbf{T}_\Delta$ represents the resulting finite displacement; applying it to points or composing it with an existing pose produces the corresponding finite motion.

For every validated transformation in the selected logarithm branch,

$$\exp(\log_{\text{selected}}(\mathbf{T}_\Delta)) = \mathbf{T}_\Delta.$$

Here $\log_{\text{selected}}$ denotes the declared single-valued logarithm branch, including its identity and π conventions. The reverse coordinate round trip requires a branch qualification. Starting with an arbitrary ${}^a\boldsymbol{\eta}$, exponentiating and then applying the selected logarithm returns the selected representative of that displacement, which need not be the original column when its rotation angle lies outside the principal interval or at an ambiguous branch boundary.

Separating geometry from motion amount

For a rotational or helical displacement with $\theta = \|{}^a\boldsymbol{\phi}\|_2 > 0$, the finite blocks can optionally be normalised:

$$\theta = \|{}^a\boldsymbol{\phi}\|_2, \quad {}^a\mathbf{s}_\omega = \frac{{}^a\boldsymbol{\phi}}{\theta}, \quad {}^a\mathbf{s}_v = \frac{{}^a\boldsymbol{\rho}}{\theta}.$$

This produces ${}^a\boldsymbol{\phi} = {}^a\mathbf{s}_\omega\theta$ and ${}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v\theta$. Perform this optional division only when the normalised screw geometry is needed. At $\theta = 0$, keep the finite blocks: for pure translation, ${}^a\boldsymbol{\phi} = \mathbf{0}_3$ and ${}^a\boldsymbol{\rho} = {}^a\mathbf{p}$.

Symbol and purpose reference

Symbol	Plain meaning	Main purpose
σ	Signed motion amount	States how far the screw motion proceeds
$\dot{\sigma}$	Motion amount per second	Converts screw geometry into an instantaneous twist
${}^a\mathbf{S}$	Normalised screw geometry expressed in $\{a\}$	Describes the permitted motion per unit σ
${}^a\boldsymbol{\xi}$	Twist expressed in $\{a\}$	Describes the instantaneous rigid velocity field
${}^a\boldsymbol{\eta}$	Finite exponential-coordinate column expressed in $\{a\}$	Supplies the finite generator used by exp or returned by log
${}^a\boldsymbol{\rho}$	Linear block of ${}^a\boldsymbol{\eta}$	Produces the final translation after multiplication by $\mathbf{J}$
${}^a\boldsymbol{\phi}$	Rotation-vector block of ${}^a\boldsymbol{\eta}$	Stores an axis direction multiplied by an angle; a principal logarithm selects the angle in $[0, \pi]$
$\mathbf{T}_\Delta$	Finite rigid-displacement transformation	Moves points or composes with poses
$\boldsymbol{\Omega}$	$[{}^a\mathbf{s}_\omega]_\times$	Represents cross products with the unit rotation axis
$\boldsymbol{\Phi}$	$[{}^a\boldsymbol{\phi}]_\times = \boldsymbol{\Omega}\theta$	Represents cross products with the finite rotation vector
$\mathbf{G}(\theta)$	Normalised-screw translation map	Gives ${}^a\mathbf{p} = \mathbf{G}(\theta){}^a\mathbf{s}_v$
$\mathbf{J}({}^a\boldsymbol{\phi})$	Finite-coordinate translation map	Gives ${}^a\mathbf{p} = \mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho}$

For rotational motion, $\mathbf{G}$ and $\mathbf{J}$ describe the same accumulated translation in two parametrisations:

$$\boxed{\mathbf{G}(\theta) = \theta\mathbf{J}({}^a\boldsymbol{\phi}), \quad {}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v\theta.}$$

The shortest memory rule is: ${}^a\mathbf{S}$ says **how**, σ says **how much**, and $\dot{\sigma}$ says **how fast**. Multiplying ${}^a\mathbf{S}$ by σ produces the finite coordinates ${}^a\boldsymbol{\eta}$; multiplying it by $\dot{\sigma}$ produces the instantaneous twist ${}^a\boldsymbol{\xi}$. The exponential constructs $\mathbf{T}_\Delta$. A selected logarithm recovers ${}^a\boldsymbol{\eta}$ only when the inverse description is needed.

Cumulative chapter review

Questions

1. **Physical system and representations:** Distinguish a link, a material point, a rigid-link configuration χ , an attached-frame pose $\mathbf{T}_{wb}$, and a generalised-coordinate column $\mathbf{q}$. State the correspondence between χ and $\mathbf{T}_{wb}$ for one rigid link and explain why $\mathbf{q}$ is not itself a pose.
2. **Joint model:** Define a zero arrangement and an oriented joint axis. Contrast revolute, prismatic, and finite-pitch helical motion, including the scalar coordinate and physical unit used by each model.
3. **Screw-axis derivation:** Let an oriented line expressed in frame $\{a\}$ have unit direction ${}^a\mathbf{s}_\omega$ and pass through a point with position column ${}^a\mathbf{p}_\ell$. Derive the normalised linear-first screw-axis columns for pure revolute motion, finite-pitch helical motion with pitch h , and pure prismatic motion.
4. **Recovering physical screw geometry:** A normalised rotational screw expressed in frame $\{a\}$ has

$${}^a\mathbf{s}_v = \begin{bmatrix} 0 \\ -2 \\ 0.05 \end{bmatrix} \text{ m rad}^{-1}, \quad {}^a\mathbf{s}_\omega = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Recover its pitch and the position column of the axis point closest to O_a . State the oriented axis line.

5. **Velocity field:** Starting from ${}^a\boldsymbol{\xi} = {}^a\mathbf{S}\dot{\theta} = [({}^a\boldsymbol{\nu})^\top ({}^a\boldsymbol{\omega})^\top]^\top$, write the velocity of a material point currently at ${}^a\mathbf{x}$. Explain why ${}^a\mathbf{x}$ appears when the twist is evaluated but does not appear in ${}^a\mathbf{S}$ or ${}^a\boldsymbol{\xi}$.
6. **Change of reference frame:** If $\mathbf{T}_{ab}$ transforms point coordinates from frame $\{b\}$ to frame $\{a\}$, state the linear-first adjoint matrix and derive the rule that transforms screw-axis coordinates from $\{b\}$ to $\{a\}$.
7. **Finite-displacement workflow:** Beginning with a normalised screw axis ${}^a\mathbf{S}$ and signed motion amount σ , write the complete sequence that produces $\mathbf{T}_\Delta$. Then write the equivalent finite-coordinate form using ${}^a\boldsymbol{\eta} = [({}^a\boldsymbol{\rho})^\top ({}^a\boldsymbol{\phi})^\top]^\top$ and $\mathbf{J}({}^a\boldsymbol{\phi})$.
8. **Cumulative screw application:** Use the screw in Question 4 and the signed angle $\theta = \pi/2$ rad to calculate $\mathbf{T}_\Delta$. Apply the result to an arbitrary axis point ${}^a\mathbf{x} = [2 \ 0 \ \lambda]^\top$ m and interpret its final position.
9. **Principal inverse workflow:** Given a validated transformation $\mathbf{T} = [\mathbf{R} \ {}^a\mathbf{p}; \mathbf{0}_3^\top \ 1]$, state the identity, pure-translation, and rotation-present cases of the selected principal logarithm. Explain why the identity has no unique normalised screw axis.

10. **Finite versus normalised coordinates:** Suppose the selected logarithm returns ${}^a\eta = [({}^a\rho)^\top ({}^a\phi)^\top]^\top$ with $\|{}^a\phi\|_2 > 0$. Recover θ , ${}^a\mathbf{s}_\omega$, ${}^a\mathbf{s}_v$, h , and the closest axis point ${}^a\mathbf{p}_\ell$. Why should this normalisation not be performed first near a pure translation?
11. **Space and body integration:** Let $\mathbf{T}_0 = \mathbf{T}_{sb}(t_0)$. Write the constant-space-twist and constant-body-twist updates over Δt . State the adjoint relation required for them to represent the same physical motion and show the resulting equality of the two final poses.
12. **Time-varying limitation:** Explain why a time-varying twist generally requires chronologically ordered exponential increments. Why can neither $\exp(\int \hat{\xi}(t) dt)$ nor an additive matrix Euler update be used as an unrestricted exact pose integrator?
13. **Small angular displacement:** Define ${}^a\phi$, θ , and Φ , then write the closed forms for $\mathbf{R}({}^a\phi)$ and $\mathbf{J}({}^a\phi)$. Give the first two nonzero terms of the small-angle series for $A(\theta)$, $B(\theta)$, and $C(\theta)$ and explain why these series are needed in floating-point arithmetic.
14. **Principal rotation near π :** Starting from Rodrigues' formula, state the trace and skew-difference equations used to recover the principal angle and nominal axis. Explain why the skew-based axis calculation fails at $\theta = \pi$, what symmetric matrix replaces it, and why a deterministic sign convention remains necessary.
15. **Validation and numerical policy:** List the structural checks required before treating a numerical array as an element of $SE(3)$. Contrast θ_{series} , ε_π , $\varepsilon_{\text{zero}}$, and $\varepsilon_{p,0}$, including which thresholds merely select a formula and which may change the returned model.
16. **Chasles–Mozzi theorem:** State the theorem's rotation-present, pure-translation, and identity cases. What does the theorem prove about the endpoints of a displacement, and what does it not prove about the body's actual past trajectory?
17. **Round trips and branches:** Explain why $\exp(\log_{\text{selected}}(\mathbf{T})) = \mathbf{T}$ is expected for every validated input in the selected branch, whereas $\log_{\text{selected}}(\exp(\widehat{{}^a\eta})) = \widehat{{}^a\eta}$ requires a qualification.

Answers and derivation hints

1. A material point is one labelled point fixed in the robot's matter, and a link is a set of material points modelled as one rigid object. A rigid-link configuration χ maps every labelled point of that link to its world position. An attached-frame pose $\mathbf{T}_{wb}$ maps the fixed body-frame coordinates of those material points into world coordinates. If the link contains enough non-collinear labelled points to determine its placement, the complete rigid configuration and the attached-frame pose determine one another:

$$\chi \longleftrightarrow \mathbf{T}_{wb}.$$

The generalised-coordinate column $\mathbf{q}$ is an ordered numerical description used by the robot model to select an admissible complete configuration through $\mathbf{q} \mapsto \chi(\mathbf{q})$. Its entries may be joint angles and displacements; they are not generally the position and orientation coordinates of one body.

2. The zero arrangement is the declared relative placement of the connected links at zero joint coordinate. An oriented joint axis is a geometric line with a selected positive direction. A revolute joint rotates through a signed angle θ in radians about that line. A prismatic joint translates through a signed distance d in metres parallel to its direction. A finite-pitch helical joint rotates through θ while translating $h\theta$ metres along the same line, where h is measured in m rad^{-1} .
3. A point on a pure revolute axis has zero velocity, so substituting ${}^a\boldsymbol{\omega} = {}^a\mathbf{s}_\omega \dot{\theta}$ and ${}^a\mathbf{x} = {}^a\mathbf{p}_\ell$ into the velocity field gives ${}^a\boldsymbol{\nu} = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell \dot{\theta}$. The three normalised columns are therefore

$${}^a\mathbf{S}_R = \begin{bmatrix} -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell \\ {}^a\mathbf{s}_\omega \end{bmatrix}, \quad {}^a\mathbf{S}_H = \begin{bmatrix} -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega \\ {}^a\mathbf{s}_\omega \end{bmatrix}, \quad {}^a\mathbf{S}_P = \begin{bmatrix} {}^a\mathbf{s}_v \\ \mathbf{0}_3 \end{bmatrix},$$

where $\|{}^a\mathbf{s}_\omega\|_2 = 1$ for the rotational cases and $\|{}^a\mathbf{s}_v\|_2 = 1$ for the prismatic case.

4. As derived in Section 8.3, start from ${}^a\mathbf{s}_v = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega$, with $\|{}^a\mathbf{s}_\omega\|_2 = 1$ and $({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{p}_\ell = 0$ for the closest axis point. Taking the dot product with ${}^a\mathbf{s}_\omega$ removes the perpendicular cross-product term, while crossing on the left with ${}^a\mathbf{s}_\omega$ and using the vector triple-product identity gives

$$h = ({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_v, \quad {}^a\mathbf{p}_\ell = {}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_v.$$

Therefore the pitch is

$$h = ({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_v = 0.05 \text{ m rad}^{-1}.$$

The closest axis point is

$${}^a\mathbf{p}_\ell = {}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_v = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} \text{ m}.$$

Hence the oriented axis line is

$${}^a\mathcal{L} = \left\{ \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} + \lambda \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \mid \lambda \in \mathbb{R} \right\},$$

with the point coordinates and λ measured in metres.

5. The point velocity is

$$\boxed{{}^a\mathbf{v}_P = {}^a\boldsymbol{\nu} + {}^a\boldsymbol{\omega} \times {}^a\mathbf{x}}.$$

The screw axis and twist store the position-independent coefficients of one complete rigid velocity field. The position ${}^a\mathbf{x}$ is supplied only when that field is evaluated at a particular material point, so different points receive different rotational contributions without requiring different twists.

6. Under the project's linear-first convention,

$$\text{Ad}_{\mathbf{T}_{ab}} = \begin{bmatrix} \mathbf{R}_{ab} & [{}^a\mathbf{p}_b]_{\times} \mathbf{R}_{ab} \\ \mathbf{0}_{3 \times 3} & \mathbf{R}_{ab} \end{bmatrix}.$$

In frame $\{b\}$, ${}^b\boldsymbol{\xi} = {}^b\mathbf{S}\dot{q}$. Transforming the twist gives ${}^a\boldsymbol{\xi} = \text{Ad}_{\mathbf{T}_{ab}} {}^b\boldsymbol{\xi}$. Because the same scalar rate must also satisfy ${}^a\boldsymbol{\xi} = {}^a\mathbf{S}\dot{q}$, comparison of the coefficients of $\dot{q}$ gives

$$\boxed{{}^a\mathbf{S} = \text{Ad}_{\mathbf{T}_{ab}} {}^b\mathbf{S}}.$$

7. The normalised-coordinate workflow is

$$\boxed{({}^a\mathbf{S}, \sigma) \mapsto {}^a\boldsymbol{\eta} = {}^a\mathbf{S}\sigma \mapsto \widehat{{}^a\boldsymbol{\eta}} \mapsto \mathbf{T}_{\Delta} = \exp(\widehat{{}^a\boldsymbol{\eta}})}.$$

With ${}^a\boldsymbol{\eta} = [({}^a\boldsymbol{\rho})^{\top} ({}^a\boldsymbol{\phi})^{\top}]^{\top}$ and $\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_{\times}$,

$$\boxed{\mathbf{T}_{\Delta} = \begin{bmatrix} \mathbf{R}({}^a\boldsymbol{\phi}) & \mathbf{J}({}^a\boldsymbol{\phi}) {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^{\top} & 1 \end{bmatrix}}.$$

8. For ${}^a\mathbf{s}_{\omega} = [0 \ 0 \ 1]^{\top}$, the cross-product matrix and its square are

$$\boldsymbol{\Omega} = [{}^a\mathbf{s}_{\omega}]_{\times} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \boldsymbol{\Omega}^2 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Substituting $\theta = \pi/2$ into Rodrigues' formula gives

$$\begin{aligned}\mathbf{R} &= \mathbf{I}_3 + \sin \theta \boldsymbol{\Omega} + (1 - \cos \theta) \boldsymbol{\Omega}^2 \\ &= \mathbf{I}_3 + \boldsymbol{\Omega} + \boldsymbol{\Omega}^2 \\ &= \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.\end{aligned}$$

Using the axis point and pitch from Question 4,

$${}^a\mathbf{p}_\Delta = (\mathbf{I}_3 - \mathbf{R}){}^a\mathbf{p}_\ell + h\theta {}^a\mathbf{s}_\omega = \begin{bmatrix} 2 \\ -2 \\ \pi/40 \end{bmatrix} \text{ m.}$$

Therefore

$$\mathbf{T}_\Delta = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & -2 \\ 0 & 0 & 1 & \pi/40 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

An axis point satisfies

$$\mathbf{R} \begin{bmatrix} 2 \\ 0 \\ \lambda \end{bmatrix} + {}^a\mathbf{p}_\Delta = \begin{bmatrix} 2 \\ 0 \\ \lambda + \pi/40 \end{bmatrix} \text{ m.}$$

Rotation leaves the point on the axis line, while the pitch advances it by $h\theta = \pi/40$ m along positive z_a .

9. If $\mathbf{R} = \mathbf{I}_3$ and ${}^a\mathbf{p} = \mathbf{0}_3$, the selected logarithm is $\mathbf{0}_{4 \times 4}$. If $\mathbf{R} = \mathbf{I}_3$ and ${}^a\mathbf{p} \neq \mathbf{0}_3$, the result is a pure translation with $d = \|{}^a\mathbf{p}\|_2$ and unit direction ${}^a\mathbf{s} = {}^a\mathbf{p}/d$. If $\mathbf{R} \neq \mathbf{I}_3$, recover the selected principal rotation vector ${}^a\boldsymbol{\phi}$, then calculate ${}^a\boldsymbol{\rho} = \mathbf{J}({}^a\boldsymbol{\phi})^{-1}{}^a\mathbf{p}$. At the identity, every admissible screw axis multiplied by zero motion produces the same transformation, so the displacement cannot select one unique normalised axis.
10. Recover the rotational motion amount and geometry through

$$\theta = \|{}^a\phi\|_2, \quad {}^a\mathbf{s}_\omega = \frac{{}^a\phi}{\theta}, \quad {}^a\mathbf{s}_v = \frac{{}^a\rho}{\theta}.$$

Then

$$h = ({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_v, \quad {}^a\mathbf{p}_\ell = {}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_v.$$

Dividing by a small θ magnifies numerical error as the rotation approaches zero. The finite blocks ${}^a\rho$ and ${}^a\phi$ remain the appropriate primary output; the identity and pure-translation cases use separate rules.

11. The two constant-twist updates are

$$\mathbf{T}(t_0 + \Delta t) = \exp(\hat{\xi}_s \Delta t) \mathbf{T}_0 = \mathbf{T}_0 \exp(\hat{\xi}_b \Delta t).$$

They describe the same physical motion when

$$\xi_s = \text{Ad}_{\mathbf{T}_0} \xi_b, \quad \hat{\xi}_s = \mathbf{T}_0 \hat{\xi}_b \mathbf{T}_0^{-1}.$$

Similarity of the exponentials gives $\exp(\hat{\xi}_s \Delta t) = \mathbf{T}_0 \exp(\hat{\xi}_b \Delta t) \mathbf{T}_0^{-1}$; right-multiplying by $\mathbf{T}_0$ proves the equality of the final poses.

12. Twist matrices at different times need not commute, so exchanging their chronological multiplication order changes the resulting pose. An ordinary integral followed by one exponential generally discards this ordering. An additive Euler update uses matrix addition rather than the group operation and can leave $SE(3)$. Piecewise exponential updates preserve chronological order and remain in $SE(3)$ in exact arithmetic.

13. Define

$${}^a\phi = \theta {}^a\mathbf{s}_\omega, \quad \theta = \|{}^a\phi\|_2, \quad \Phi = [{}^a\phi]_\times.$$

Then

$$\mathbf{R} = \mathbf{I}_3 + A(\theta)\Phi + B(\theta)\Phi^2, \quad \mathbf{J} = \mathbf{I}_3 + B(\theta)\Phi + C(\theta)\Phi^2,$$

where

$$A(\theta) = \frac{\sin \theta}{\theta}, \quad B(\theta) = \frac{1 - \cos \theta}{\theta^2}, \quad C(\theta) = \frac{\theta - \sin \theta}{\theta^3},$$

and their small-angle series begin with

$$\begin{aligned} A(\theta) &= 1 - \frac{\theta^2}{6} + O(\theta^4), \\ B(\theta) &= \frac{1}{2} - \frac{\theta^2}{24} + O(\theta^4), \\ C(\theta) &= \frac{1}{6} - \frac{\theta^2}{120} + O(\theta^4). \end{aligned}$$

Direct trigonometric ratios form small results by subtracting nearly equal floating-point numbers near zero. The series evaluate the same mathematical functions without that cancellation.

14. Rodrigues' formula gives

$$\frac{\text{tr}(\mathbf{R}) - 1}{2} = \cos \theta, \quad \mathbf{R} - \mathbf{R}^\top = 2 \sin \theta [{}^a\mathbf{s}_\omega]_\times.$$

The skew entries form ${}^a\mathbf{u} = 2 \sin \theta {}^a\mathbf{s}_\omega$, and the principal angle follows from $\theta = \text{atan2}(\|{}^a\mathbf{u}\|_2/2, (\text{tr}(\mathbf{R}) - 1)/2)$. At $\theta = \pi$, $\sin \theta = 0$, so the nonzero rotation has ${}^a\mathbf{u} = \mathbf{0}_3$ and the nominal axis division fails. The symmetric relation recovers the unsigned outer product

$$\mathbf{Q} = \frac{\frac{1}{2}(\mathbf{R} + \mathbf{R}^\top) - \cos \theta \mathbf{I}_3}{1 - \cos \theta} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top.$$

Because $\mathbf{Q}$ is unchanged when the axis sign is reversed, a declared component and sign rule is still needed for deterministic output.

15. Validate finiteness, the 4×4 shape, the homogeneous bottom row, rotation orthogonality, and $\det(\mathbf{R}) = +1$ before applying a logarithm. The threshold θ_{series} selects series evaluation near zero, and ε_π selects the stable near- π axis formula; these should not change the represented motion. The angular resolution $\varepsilon_{\text{zero}}$ may classify and canonicalise a small rotation as zero, while $\varepsilon_{p,0}$ classifies a small translation column as zero and changes the returned model only if the interface also canonicalises that translation. All thresholds and tolerances must be finite, nonnegative, dimensionally appropriate, and ordered so formula regions have declared precedence.
16. If the rotation block is non-identity, every rigid displacement has a representation as rotation about one oriented line plus translation along that line. If the rotation is the identity and translation is nonzero, the displacement is pure translation. The identity is zero displacement and has no unique normalised screw axis. The theorem constructs a screw path connecting the same initial and final poses; endpoint data alone do not show that the body actually followed that path in the past.

17. The selected logarithm is defined to choose a matrix whose exponential reconstructs its validated input, so $\exp(\log_{\text{selected}}(\mathbf{T})) = \mathbf{T}$. In the reverse direction, many exponential-coordinate columns can represent the same transformation because rotations repeat, the axis sign is ambiguous at π , and the axis is undefined at zero motion. Therefore the selected logarithm returns its canonical representative, which equals the original ${}^a\boldsymbol{\eta}$ only when that column already satisfies the selected branch conventions.

Appendix: Chasles–Mozzi theorem and proof

Theorem

The **Chasles–Mozzi theorem** states that every spatial rigid displacement can be represented by one rotation about an oriented line combined with one translation along that same line. Let

$$\mathbf{T}_\Delta = \begin{bmatrix} \mathbf{R}_\Delta & {}^a\mathbf{p}_\Delta \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3)$$

be any active rigid displacement represented in frame $\{a\}$. If $\mathbf{R}_\Delta \neq \mathbf{I}_3$, then there exist a dimensionless unit direction ${}^a\mathbf{s} \in \mathbb{R}^3$, a point ${}^a\mathbf{p}_\ell \in \mathbb{R}^3$ on the screw axis measured in metres, an angle $\theta \in \mathbb{R}$ in radians, and a pitch $h \in \mathbb{R}$ in m rad^{-1} such that

$$\|{}^a\mathbf{s}\|_2 = 1, \quad \mathbf{R}_\Delta = \exp([{}^a\mathbf{s}]_\times \theta), \quad {}^a\mathbf{p}_\Delta = (\mathbf{I}_3 - \mathbf{R}_\Delta) {}^a\mathbf{p}_\ell + h\theta {}^a\mathbf{s}.$$

Equivalently, define the normalised helical screw-axis coordinate column

$${}^a\mathbf{S}_H := \begin{bmatrix} -{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s} \\ {}^a\mathbf{s} \end{bmatrix}.$$

The complete displacement then satisfies

$$\boxed{\mathbf{T}_\Delta = \exp(\widehat{{}^a\mathbf{S}_H} \theta)}.$$

If $\mathbf{R}_\Delta = \mathbf{I}_3$ and ${}^a\mathbf{p}_\Delta \neq \mathbf{0}_3$, the displacement is pure translation. Define

$$d := \|{}^a\mathbf{p}_\Delta\|_2, \quad {}^a\mathbf{s} := \frac{{}^a\mathbf{p}_\Delta}{d}, \quad {}^a\mathbf{S}_P := \begin{bmatrix} {}^a\mathbf{s} \\ \mathbf{0}_3 \end{bmatrix}.$$

Then

$$\mathbf{T}_\Delta = \exp(\widehat{({}^a\mathbf{S}_P d)}).$$

The identity displacement $\mathbf{T}_\Delta = \mathbf{I}_4$ is obtained with zero displacement, but its normalised screw axis is not unique. More generally, the theorem guarantees that a screw representation exists; a logarithm branch convention selects one representation from the possible angle and axis choices. The theorem describes a single screw motion that produces the same initial and final poses; it does not claim that the body's actual past trajectory followed that screw.

Proof

The proof has two cases. When the displacement contains a nonzero rotation, first decompose its translation into components parallel and perpendicular to the rotation axis, then show that the perpendicular component is produced by choosing an appropriate offset axis line. When the displacement has no rotation, show directly that it is the exponential of a prismatic screw.

Assume first that $\mathbf{R}_\Delta \neq \mathbf{I}_3$. Euler's rotation theorem from Section 7.5.4 guarantees a dimensionless unit column ${}^a\mathbf{s} \in \mathbb{R}^3$ and a principal angle $\theta \in (0, \pi]$ such that

$$\mathbf{R}_\Delta = \exp([{}^a\mathbf{s}]_\times \theta), \quad \mathbf{R}_\Delta {}^a\mathbf{s} = {}^a\mathbf{s}.$$

Define the plane perpendicular to the rotation axis by

$$\mathcal{V} := ({}^a\mathbf{s})^\perp = \{\mathbf{x} \in \mathbb{R}^3 \mid ({}^a\mathbf{s})^\top \mathbf{x} = 0\}.$$

The next step requires solving an equation of the form $(\mathbf{I}_3 - \mathbf{R}_\Delta)\mathbf{x} = \mathbf{y}$ for columns $\mathbf{x}, \mathbf{y} \in \mathcal{V}$. We therefore need to prove that, for every $\mathbf{y} \in \mathcal{V}$, there is exactly one $\mathbf{x} \in \mathcal{V}$ satisfying this equation.

First, $\mathbf{I}_3 - \mathbf{R}_\Delta$ maps $\mathcal{V}$ into itself. Because $\mathbf{R}_\Delta {}^a\mathbf{s} = {}^a\mathbf{s}$ and $\mathbf{R}_\Delta^{-1} = \mathbf{R}_\Delta^\top$, the axis column also satisfies $\mathbf{R}_\Delta^\top {}^a\mathbf{s} = {}^a\mathbf{s}$. Hence, for every $\mathbf{x} \in \mathcal{V}$,

$$\begin{aligned} ({}^a\mathbf{s})^\top (\mathbf{I}_3 - \mathbf{R}_\Delta)\mathbf{x} &= ({}^a\mathbf{s})^\top \mathbf{x} - (\mathbf{R}_\Delta^\top {}^a\mathbf{s})^\top \mathbf{x} \\ &= 0 - 0 = 0. \end{aligned}$$

Therefore $(\mathbf{I}_3 - \mathbf{R}_\Delta)\mathbf{x} \in \mathcal{V}$. The restriction is consequently a well-defined map from $\mathcal{V}$ to $\mathcal{V}$. Denote this map by

$$\begin{aligned} \mathcal{A} : \mathcal{V} &\longrightarrow \mathcal{V}, \\ \mathbf{x} &\longmapsto (\mathbf{I}_3 - \mathbf{R}_\Delta)\mathbf{x}. \end{aligned}$$

The map $\mathcal{A}$ is linear because it is multiplication by the fixed matrix $\mathbf{I}_3 - \mathbf{R}_\Delta$. Thus, for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{V}$ and all scalars $c_1, c_2 \in \mathbb{R}$,

$$\mathcal{A}(c_1\mathbf{x}_1 + c_2\mathbf{x}_2) = c_1\mathcal{A}(\mathbf{x}_1) + c_2\mathcal{A}(\mathbf{x}_2).$$

Second, $\mathcal{A}$ sends no nonzero column to zero. If $\mathbf{x} \in \mathcal{V}$ and

$$\mathcal{A}(\mathbf{x}) = \mathbf{0}_3,$$

then $\mathbf{R}_\Delta\mathbf{x} = \mathbf{x}$. However, $\mathbf{R}_\Delta$ rotates the plane $\mathcal{V}$ through the nonzero angle $\theta \in (0, \pi]$, so it fixes no nonzero column in that plane. Hence $\mathbf{x} = \mathbf{0}_3$, and

$$\ker(\mathcal{A}) = \{\mathbf{0}_3\}.$$

This zero kernel makes $\mathcal{A}$ one-to-one. To see why, suppose that $\mathcal{A}(\mathbf{x}_1) = \mathcal{A}(\mathbf{x}_2)$. Linearity gives

$$\mathcal{A}(\mathbf{x}_1 - \mathbf{x}_2) = \mathcal{A}(\mathbf{x}_1) - \mathcal{A}(\mathbf{x}_2) = \mathbf{0}_3.$$

Therefore $\mathbf{x}_1 - \mathbf{x}_2 \in \ker(\mathcal{A})$. Because the kernel contains only $\mathbf{0}_3$, it follows that $\mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0}_3$ and hence $\mathbf{x}_1 = \mathbf{x}_2$. Equal outputs can therefore arise only from equal inputs, which is precisely the definition of a one-to-one map.

Finally, the rank–nullity theorem applies to any linear map with a finite-dimensional domain. It applies here because $\mathcal{A} : \mathcal{V} \rightarrow \mathcal{V}$ is linear and its domain is the two-dimensional plane $\mathcal{V}$. The theorem gives

$$\dim(\mathcal{V}) = \dim(\ker(\mathcal{A})) + \dim(\text{image}(\mathcal{A})).$$

Substituting $\dim(\mathcal{V}) = 2$ and $\dim(\ker(\mathcal{A})) = 0$ yields

$$\dim(\text{image}(\mathcal{A})) = 2 - 0 = 2.$$

The image is therefore a two-dimensional subspace of the two-dimensional plane $\mathcal{V}$, so the image must be all of $\mathcal{V}$. This is why one-to-one also implies onto here. Consequently, for every $\mathbf{y} \in \mathcal{V}$, there is exactly one $\mathbf{x} \in \mathcal{V}$ satisfying $(\mathbf{I}_3 - \mathbf{R}_\Delta)\mathbf{x} = \mathbf{y}$.

Decompose the translation column into parts parallel and perpendicular to the axis:

$${}^a\mathbf{p}_\Delta = {}^a\mathbf{p}_\parallel + {}^a\mathbf{p}_\perp, \quad {}^a\mathbf{p}_\parallel := {}^a\mathbf{s}({}^a\mathbf{s})^\top {}^a\mathbf{p}_\Delta, \quad {}^a\mathbf{p}_\perp := \left(\mathbf{I}_3 - {}^a\mathbf{s}({}^a\mathbf{s})^\top\right) {}^a\mathbf{p}_\Delta.$$

The defining property of a screw axis supplies the equation for its points. A point on the screw axis may move along the axis, but it has no displacement perpendicular to the axis. To apply this property, first consider any spatial point with initial position ${}^a\mathbf{x} \in \mathbb{R}^3$. Its displacement under $\mathbf{T}_\Delta$ is

$$\mathbf{R}_\Delta {}^a\mathbf{x} + {}^a\mathbf{p}_\Delta - {}^a\mathbf{x}.$$

The component of this displacement along ${}^a\mathbf{s}$ is independent of ${}^a\mathbf{x}$. Indeed,

$$\begin{aligned} ({}^a\mathbf{s})^\top (\mathbf{R}_\Delta {}^a\mathbf{x} + {}^a\mathbf{p}_\Delta - {}^a\mathbf{x}) &= (\mathbf{R}_\Delta^\top {}^a\mathbf{s} - {}^a\mathbf{s})^\top {}^a\mathbf{x} + ({}^a\mathbf{s})^\top {}^a\mathbf{p}_\Delta \\ &= ({}^a\mathbf{s})^\top {}^a\mathbf{p}_\Delta, \end{aligned}$$

because $\mathbf{R}_\Delta^\top {}^a\mathbf{s} = {}^a\mathbf{s}$. Multiplying this common scalar component by the unit direction gives the parallel vector component

$${}^a\mathbf{s} ({}^a\mathbf{s})^\top (\mathbf{R}_\Delta {}^a\mathbf{x} + {}^a\mathbf{p}_\Delta - {}^a\mathbf{x}) = {}^a\mathbf{s} ({}^a\mathbf{s})^\top {}^a\mathbf{p}_\Delta = {}^a\mathbf{p}_\parallel.$$

Therefore every point has the same displacement component parallel to the axis, namely ${}^a\mathbf{p}_\parallel$.

To find a screw-axis point, let P_ℓ be a candidate point and let ${}^a\mathbf{p}_\ell \in \mathbb{R}^3$ be its position column in frame $\{a\}$. For P_ℓ to lie on the axis, it must have no displacement perpendicular to ${}^a\mathbf{s}$, so its complete displacement must equal ${}^a\mathbf{p}_\parallel$:

$$\mathbf{R}_\Delta {}^a\mathbf{p}_\ell + {}^a\mathbf{p}_\Delta - {}^a\mathbf{p}_\ell = {}^a\mathbf{p}_\parallel.$$

Rearranging this defining condition and using ${}^a\mathbf{p}_\Delta - {}^a\mathbf{p}_\parallel = {}^a\mathbf{p}_\perp$ gives

$$\boxed{(\mathbf{I}_3 - \mathbf{R}_\Delta) {}^a\mathbf{p}_\ell = {}^a\mathbf{p}_\perp}.$$

The column ${}^a\mathbf{p}_\perp$ belongs to $\mathcal{V}$. The invertibility result above therefore guarantees a unique solution ${}^a\mathbf{p}_\ell \in \mathcal{V}$. The corresponding point P_ℓ satisfies the required axis-point condition: the rigid displacement moves it only by ${}^a\mathbf{p}_\parallel$, which is parallel to ${}^a\mathbf{s}$. It remains to verify that the complete line through this point is preserved by the displacement.

The oriented line through P_ℓ in direction ${}^a\mathbf{s}$ is

$${}^a\mathcal{L} := \{ {}^a\mathbf{p}_\ell + \lambda {}^a\mathbf{s} \mid \lambda \in \mathbb{R} \},$$

where λ is signed distance along the line, measured in metres. Every point on this line remains on it after the displacement because

$$\begin{aligned}
 \mathbf{R}_\Delta (^a\mathbf{p}_\ell + \lambda^a\mathbf{s}) + ^a\mathbf{p}_\Delta &= \mathbf{R}_\Delta ^a\mathbf{p}_\ell + ^a\mathbf{p}_\Delta + \lambda\mathbf{R}_\Delta ^a\mathbf{s} \\
 &= ^a\mathbf{p}_\ell + ^a\mathbf{p}_\parallel + \lambda^a\mathbf{s} \\
 &= (^a\mathbf{p}_\ell + \lambda^a\mathbf{s}) + ^a\mathbf{p}_\parallel.
 \end{aligned}$$

Hence the whole line is mapped to itself and every point on it moves only along the line. This proves that $^a\mathcal{L}$ is the screw axis, rather than merely declaring a line through $^a\mathbf{p}_\ell$ to be the axis.

The restriction $^a\mathbf{p}_\ell \in \mathcal{V}$ selects one convenient point from this line. It means

$$(^a\mathbf{s})^\top ^a\mathbf{p}_\ell = 0,$$

so the displacement vector from O_a to P_ℓ is perpendicular to the axis direction. Consequently, P_ℓ is the point on the axis closest to O_a ; the other axis points $^a\mathbf{p}_\ell + \lambda^a\mathbf{s}$ generally do not belong to $\mathcal{V}$. Define the pitch by

$$h := \frac{(^a\mathbf{s})^\top ^a\mathbf{p}_\Delta}{\theta}.$$

The parallel translation then satisfies

$$^a\mathbf{p}_\parallel = h\theta^a\mathbf{s}.$$

Adding the perpendicular and parallel components proves the required translation formula:

$$\boxed{^a\mathbf{p}_\Delta = (\mathbf{I}_3 - \mathbf{R}_\Delta) ^a\mathbf{p}_\ell + h\theta^a\mathbf{s}.}$$

To verify its geometric meaning, let $^a\mathbf{x} \in \mathbb{R}^3$ be the initial position of any material point, expressed in frame $\{a\}$ and measured in metres. The active displacement maps it to

$$\begin{aligned}
 ^a\mathbf{x}' &= \mathbf{R}_\Delta ^a\mathbf{x} + ^a\mathbf{p}_\Delta \\
 &= \mathbf{R}_\Delta (^a\mathbf{x} - ^a\mathbf{p}_\ell) + ^a\mathbf{p}_\ell + h\theta^a\mathbf{s}.
 \end{aligned}$$

This equation first rotates the point about the oriented line through $^a\mathbf{p}_\ell$ in direction $^a\mathbf{s}$, then translates it by $h\theta$ along that line. It is therefore exactly a screw displacement.

It remains to prove the exponential representation. Let $\varphi \in [0, \theta]$ be an intermediate rotation angle and define

$$\mathbf{R}(\varphi) := \exp([^a\mathbf{s}]_\times \varphi),$$

and

$${}^a\mathbf{x}(\varphi) := \mathbf{R}(\varphi) ({}^a\mathbf{x}(0) - {}^a\mathbf{p}_\ell) + {}^a\mathbf{p}_\ell + h\varphi {}^a\mathbf{s}.$$

To differentiate this path, first differentiate its rotation matrix. Define the constant generator $\mathbf{A} := [{}^a\mathbf{s}]_\times \in \mathbb{R}^{3 \times 3}$. Because ${}^a\mathbf{s}$ is fixed in frame $\{a\}$, $\mathbf{A}$ does not depend on φ . Differentiating the matrix-exponential series term by term gives

$$\begin{aligned} \frac{d\mathbf{R}}{d\varphi} &= \frac{d}{d\varphi} \sum_{k=0}^{\infty} \frac{\mathbf{A}^k \varphi^k}{k!} \\ &= \sum_{k=1}^{\infty} \frac{\mathbf{A}^k \varphi^{k-1}}{(k-1)!} \\ &= \mathbf{A} \sum_{j=0}^{\infty} \frac{\mathbf{A}^j \varphi^j}{j!} \\ &= \mathbf{A} \mathbf{R}(\varphi) \\ &= [{}^a\mathbf{s}]_\times \mathbf{R}(\varphi). \end{aligned}$$

Here k and j are nonnegative integer summation indices, with $j = k - 1$ in the third line. Geometrically, the identity states that increasing φ rotates every already-rotated column infinitesimally about the fixed direction ${}^a\mathbf{s}$.

The columns ${}^a\mathbf{x}(0)$ and ${}^a\mathbf{p}_\ell$, the pitch h , and the axis direction ${}^a\mathbf{s}$ are constant with respect to φ . Applying the derivative above to the point path therefore gives

$$\frac{d}{d\varphi} {}^a\mathbf{x}(\varphi) = [{}^a\mathbf{s}]_\times \mathbf{R}(\varphi) ({}^a\mathbf{x}(0) - {}^a\mathbf{p}_\ell) + h {}^a\mathbf{s}.$$

The path definition can be rearranged as

$$\mathbf{R}(\varphi) ({}^a\mathbf{x}(0) - {}^a\mathbf{p}_\ell) = {}^a\mathbf{x}(\varphi) - {}^a\mathbf{p}_\ell - h\varphi {}^a\mathbf{s}.$$

Substituting this expression and using $[{}^a\mathbf{s}]_\times {}^a\mathbf{s} = {}^a\mathbf{s} \times {}^a\mathbf{s} = \mathbf{0}_3$ yields

$$\begin{aligned} \frac{d}{d\varphi} {}^a\mathbf{x}(\varphi) &= [{}^a\mathbf{s}]_\times ({}^a\mathbf{x}(\varphi) - {}^a\mathbf{p}_\ell - h\varphi {}^a\mathbf{s}) + h {}^a\mathbf{s} \\ &= {}^a\mathbf{s} \times ({}^a\mathbf{x}(\varphi) - {}^a\mathbf{p}_\ell) + h {}^a\mathbf{s} \\ &= [{}^a\mathbf{s}]_\times {}^a\mathbf{x}(\varphi) - {}^a\mathbf{s} \times {}^a\mathbf{p}_\ell + h {}^a\mathbf{s}. \end{aligned}$$

The first term in the second line is the tangential displacement per radian produced by rotation about the line through P_ℓ ; the second is the displacement per radian along that

line. Thus this is a derivative with respect to the path angle, measured in metres per radian, rather than a velocity with respect to time.

Using homogeneous point coordinates, this differential equation becomes

$$\frac{d}{d\varphi} {}^a\bar{\mathbf{x}}(\varphi) = \underbrace{\begin{bmatrix} [{}^a\mathbf{s}]_{\times} & -{}^a\mathbf{s} \times {}^a\mathbf{p}_\ell + h^a\mathbf{s} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}}_{\widehat{{}^a\mathbf{S}_H}} {}^a\bar{\mathbf{x}}(\varphi).$$

The generator $\widehat{{}^a\mathbf{S}_H}$ is constant with respect to φ . Differentiating the matrix-exponential power series verifies that the solution from $\varphi = 0$ to $\varphi = \theta$ is

$${}^a\bar{\mathbf{x}}(\theta) = \exp(\widehat{{}^a\mathbf{S}_H} \theta) {}^a\bar{\mathbf{x}}(0).$$

This exponential and $\mathbf{T}_\Delta$ act identically on every homogeneous point column, so

$$\mathbf{T}_\Delta = \exp(\widehat{{}^a\mathbf{S}_H} \theta).$$

Now suppose that $\mathbf{R}_\Delta = \mathbf{I}_3$ and ${}^a\mathbf{p}_\Delta \neq \mathbf{0}_3$. With $d = \|{}^a\mathbf{p}_\Delta\|_2$ and ${}^a\mathbf{s} = {}^a\mathbf{p}_\Delta/d$, the prismatic screw satisfies

$$\widehat{{}^a\mathbf{S}_P} = \begin{bmatrix} \mathbf{0}_{3 \times 3} & {}^a\mathbf{s} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}, \quad (\widehat{{}^a\mathbf{S}_P})^2 = \mathbf{0}_{4 \times 4}.$$

The exponential series therefore terminates after its first-order term:

$$\begin{aligned} \exp(\widehat{{}^a\mathbf{S}_P} d) &= \mathbf{I}_4 + \widehat{{}^a\mathbf{S}_P} d \\ &= \begin{bmatrix} \mathbf{I}_3 & {}^a\mathbf{s}d \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I}_3 & {}^a\mathbf{p}_\Delta \\ \mathbf{0}_3^\top & 1 \end{bmatrix} = \mathbf{T}_\Delta. \end{aligned}$$

Finally, if $\mathbf{R}_\Delta = \mathbf{I}_3$ and ${}^a\mathbf{p}_\Delta = \mathbf{0}_3$, then $\mathbf{T}_\Delta = \mathbf{I}_4 = \exp(\mathbf{0}_{4 \times 4})$. These cases exhaust every element of $SE(3)$, completing the proof.

References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapter 2 on rigid links, joints, and robot configuration; and Chapter 3, especially Section 3.3.2 on body and space twists, Section 3.3.2.2 on the screw interpretation of a twist and normalised screw axes, and Section 3.3.3 on the $SE(3)$ exponential and logarithm.
- John J. Craig, *Introduction to Robotics: Mechanics and Control*, Chapter 2 on spatial descriptions, homogeneous transformation mappings, composition order, and rotations about general axes; and Chapter 3 on one-degree-of-freedom revolute and prismatic joints.
- Peter Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, Chapter 2, especially Sections 2.3.2.2–2.3.2.3 and 2.4.8 on the matrix exponential for pose, three-dimensional twists, and their revolute, prismatic, and finite-pitch screw cases; and Chapter 3, especially Sections 3.1.1–3.1.5 on time-varying pose, frame-dependent velocity coordinates, exponential rotation updates, and incremental rigid-body motion.

9 Spatial Geometry Kernel Implementation

The code explained in this companion is in `ros_ws/src/rigid_body_kinematics`. Public headers are under `include/rigid_body_kinematics/`, compiled implementations are under `src/`, and deterministic GTests are under `test/`.

9.1 From C++ source to a reusable ROS 2 library

The `Rotation3` code becomes useful to another program only after the compiler converts it to machine code, the build system packages that code as a library, and the installed package tells downstream software where to find the library and its headers. This section follows that complete path through the actual `rigid_body_kinematics` package.

9.1.1 The roles of C++, CMake, ament, and colcon

These tools act at different layers:

Layer	Concrete role in this package
C++ compiler	Reads <code>rotation3.cpp</code> and the headers it includes, checks the language rules and types, and produces an object file containing machine code.
Archiver and linker	The archiver collects object files into the static library; the linker combines the test object file, the library, and GoogleTest into the <code>test_rotation3</code> executable.
CMake	Reads <code>CMakeLists.txt</code> , creates named targets, records how each target is compiled and linked, and generates the lower-level build and installation rules.
<code>ament_cmake</code>	Adds ROS 2 package conventions to CMake, including GoogleTest registration, package metadata generation, dependency export, and downstream package discovery.

Layer	Concrete role in this package
colcon	Finds packages in the workspace, invokes each package's configured build type in dependency order, runs its installation rules, and stores workspace-level build, install, and log results.

Using `ament_cmake` does not make the mathematical library a ROS 2 node. The core target does not link `rcpp`, create publishers or subscribers, or depend on ROS middleware at run time. It is a ROS 2 package because `package.xml` declares the `ament_cmake` build type and because colcon and ament manage its build and discovery.

`package.xml` and `CMakeLists.txt` therefore answer different questions. `package.xml` declares the package name, version, licence, build type, and named dependencies to ROS 2 workspace tools. `CMakeLists.txt` gives CMake the concrete rules for creating, testing, installing, and exporting C++ targets.

9.1.2 What compile and link mean

The public header and source file have different jobs:

- `rotation3.hpp` contains declarations that tell another source file which C++ types and operations exist.
- `rotation3.cpp` contains the out-of-class function definitions that supply the compiled implementation.

The header is included as source text wherever its declarations are needed. The compiler separately converts each `.cpp` file into an object file, conventionally ending in `.o`. An object file contains machine code and symbol information, but it is not yet the reusable package interface or a runnable program.

For the present package, the concrete paths are

```
include/rigid_body_kinematics/rotation3.hpp
      | included while compiling
      v
src/rotation3.cpp -- compile --> rotation3.cpp.o
      |
      | archive
      v
      librigid_body_kinematics.a

test/test_rotation3.cpp -- compile --> test_rotation3.cpp.o
      |
      | link with the static library
      | and GoogleTest
```

```

v
test_rotation3

```

The extension `.a` means a static library archive. When a later executable links this library, the linker incorporates the required compiled library code into that executable. A shared Linux library would normally end in `.so` and would remain a separate run-time file. The configured package produces `librigid_body_kinematics.a`; it does not presently produce a `.so`.

This split also explains two common error families. A compiler error means that one source file could not be translated, for example because a type is unknown or a semicolon is missing. An `undefined reference` from the linker means the declaration was sufficient to compile a call, but no linked object or library supplied the required compiled definition.

9.1.3 The CMake library target

`CMakeLists.txt` is not C++ and is not compiled into the library. CMake reads it during the configure step and generates a build graph. The package begins by naming the project and locating its build-time dependencies:

```

project(rigid_body_kinematics LANGUAGES CXX)

set(CMAKE_CXX_EXTENSIONS OFF)

find_package(ament_cmake REQUIRED)
find_package(eigen3_cmake_module REQUIRED)
find_package(Eigen3 REQUIRED)

```

`find_package(...)` asks CMake to locate the named package and load the CMake information it provides. `REQUIRED` makes configuration fail immediately if that dependency cannot be found.

The library itself is a CMake **target**, meaning a named build product together with the rules and requirements attached to it:

```

add_library(${PROJECT_NAME}
  src/rotation3.cpp
)

add_library(${PROJECT_NAME}::${PROJECT_NAME}
  ALIAS ${PROJECT_NAME}
)

```

`${PROJECT_NAME}` expands to `rigid_body_kinematics`. The first command creates the library target from `rotation3.cpp`. The second creates the namespaced build-tree name `rigid_body_kinematics::rigid_body_kinematics` for the same target. The `name::name` form makes a library target visually distinct from a filename or a loose linker flag.

The target's compilation and usage requirements are then attached to that same name:

```
target_compile_features(${PROJECT_NAME}
    PUBLIC cxx_std_20
)

target_compile_options(${PROJECT_NAME}
    PRIVATE
    "$<$<COMPILE_LANG_AND_ID:CXX,GNU,Clang>:-Wall;-Wextra;-Wpedantic>"
)

target_include_directories(${PROJECT_NAME}
    PUBLIC
    $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
    $<INSTALL_INTERFACE:include>
)

target_link_libraries(${PROJECT_NAME}
    PUBLIC Eigen3::Eigen
)
```

`PUBLIC` means that a requirement is needed both to build this library and to compile a target that links against it. The `cxx_std_20` feature therefore propagates the minimum C++20 language requirement to downstream targets. The public headers expose Eigen types, so the Eigen imported target and the header search location must also reach consumers. `CMAKE_CXX_EXTENSIONS OFF` makes targets in this project use the ISO C++ language mode instead of compiler-specific GNU extensions; it does not set that option in a separate downstream project. `PRIVATE` means that a setting applies only while building this library, so the warning flags do not have to be imposed on every downstream package.

The two include interfaces solve the same header-search problem in two locations. While the library is being built, the headers live under the source package's `include/` directory, so `BUILD_INTERFACE` supplies that path. After installation, a consumer must not depend on the original source checkout, so `INSTALL_INTERFACE` supplies the installed `include/` path instead.

Although Eigen is mainly a header-defined library, `target_link_libraries(... PUBLIC Eigen3::Eigen)` is still the target-based CMake spelling. `Eigen3::Eigen` carries the include paths and other usage requirements that CMake must propagate; this command does not imply that Eigen contributes a separate `.so` to this target.

9.1.4 The test is another compiled target

When `BUILD_TESTING` is enabled, `ament` creates a separate test executable and CMake links it with the library:

```
if(BUILD_TESTING)
  find_package(ament_cmake_gtest REQUIRED)

  ament_add_gtest(test_rotation3
    test/test_rotation3.cpp
  )

  target_link_libraries(test_rotation3
    ${PROJECT_NAME}
  )
endif()
```

`test_rotation3.cpp` is compiled independently from `rotation3.cpp`. Its declarations come from the installed-style public headers, while the function definitions come from linking `test_rotation3` with the `rigid_body_kinematics` library target. This is why changing a C++ test requires an incremental rebuild before the new test can run.

9.1.5 What `colcon build` does

From the workspace root, the focused command is

```
source /opt/ros/jazzy/setup.zsh

colcon build \
  --packages-select rigid_body_kinematics \
  --symlink-install
```

Sourcing the ROS 2 installation makes the Jazzy build tools and installed dependencies discoverable. Colcon then identifies `rigid_body_kinematics` as an `ament_cmake` package, invokes CMake configuration, builds the requested targets, and runs the CMake installation rules. The default workspace layout separates three kinds of result:

```
ros_ws/
  src/      source packages edited by the developer
  build/    CMake state, object files, test executables, and other intermediates
  install/  installed libraries, headers, metadata, and environment hooks
  log/      command output and event records from colcon invocations
```

`--symlink-install` asks colcon to use symbolic links instead of copies where the package and platform permit it. It does not remove the C++ compile or link steps: changed `.cpp` files and tests still need to be rebuilt.

9.1.6 Install: create the consumer-facing file layout

Building creates artifacts in CMake's working area. Installation selects the finished artifacts and places them in a predictable package layout:

```
install(
  TARGETS ${PROJECT_NAME}
  EXPORT export_${PROJECT_NAME}
  ARCHIVE DESTINATION lib
  LIBRARY DESTINATION lib
  RUNTIME DESTINATION bin
)

install(
  DIRECTORY include/
  DESTINATION include/
)
```

`TARGETS` installs the compiled target. `ARCHIVE DESTINATION lib` places the current static `.a` library under `lib/`; `LIBRARY` provides the corresponding location if the target becomes a shared library; and `RUNTIME` covers executable or platform-specific run-time artifacts. The second command installs the complete public header tree.

After the focused build, the important installed files have this shape:

```
install/rigid_body_kinematics/
  include/rigid_body_kinematics/
    geometry_error.hpp
    numerical_policy.hpp
    rotation3.hpp
  lib/
    librigid_body_kinematics.a
  share/rigid_body_kinematics/
    cmake/          generated package and target metadata
    package.xml
    environment hooks
```

The `build/` tree is CMake's private workspace and may contain temporary or generator-specific paths. The `install/` tree is the stable boundary intended for consumers. In short, **build creates the binary; install places the binary, headers, and metadata in their reusable locations.**

9.1.7 Export: teach another package how to use the installation

Physical files under `include/` and `lib/` are not enough. A downstream CMake package also needs to know the target name, header path, library location, required C++ features, and public dependencies. The export rules generate and install that information:

```
ament_export_targets(
  export_${PROJECT_NAME}
  HAS_LIBRARY_TARGET
)

ament_export_dependencies(
  eigen3_cmake_module
  Eigen3
)

ament_package()
```

The earlier `EXPORT export_${PROJECT_NAME}` inside `install(TARGETS ...)` places the installed library in an export set. `ament_export_targets(...)` exposes that set through the package's generated CMake configuration. `ament_export_dependencies(...)` records the public Eigen dependency so it can be found for downstream consumers. Finally, `ament_package()` generates and installs the ament package metadata and must appear after the other ament export declarations.

After the workspace installation has been sourced, another CMake package can consume the library through its exported target:

```
find_package(rigid_body_kinematics REQUIRED)

add_executable(example_consumer
  src/example_consumer.cpp
)

target_link_libraries(example_consumer
  PRIVATE
    rigid_body_kinematics::rigid_body_kinematics
)
```

`find_package` loads the installed configuration. Linking the namespaced target gives `example_consumer` the installed header path, the static library location, the C++20 requirement, and the public Eigen dependency recorded by the export. The consumer does not hard-code a source checkout path or search manually for `librigid_body_kinematics.a`.

Exporting does not upload or publish the library, and it does not copy the library into the downstream source tree. It creates installed CMake metadata that describes how an-

other configured build should compile and link against the package. The complete mental model is therefore: **build creates compiled artifacts; install creates the reusable file layout; export publishes the target's usage information inside that installation.**

The target, installation, and export behaviour described here follows the official [CMake add_library documentation](#), [CMake install documentation](#), [ROS 2 ament CMake guidance](#), and [colcon workspace description](#).

9.2 Validated rotations and free-vector action

This implementation block answers two connected questions: how can the library prevent an arbitrary 3×3 array from being mistaken for a rotation, and how can a validated rotation re-express a free-vector column without adding translation? The mathematical requirements come from the definition of $SO(3)$ in Section 7.2.2. The implementation uses fixed-size Eigen columns and matrices with IEEE 754 binary64 entries, represented in C++ by `Eigen::Vector3d` and `Eigen::Matrix3d`.

9.2.1 Mathematical contract

A dimensionless matrix $\mathbf{R}_{ab} \in \mathbb{R}^{3 \times 3}$ represents the orientation of frame $\{b\}$ relative to frame $\{a\}$ only if

$$\mathbf{R}_{ab}^T \mathbf{R}_{ab} = \mathbf{I}_3, \quad \det(\mathbf{R}_{ab}) = 1.$$

The first condition requires unit, mutually perpendicular columns. It is also satisfied by a reflection, so the determinant condition is required to select a proper right-handed rotation. The implementation tests the floating-point residuals

$$e_{\text{orth}} = \|\mathbf{R}_{ab}^T \mathbf{R}_{ab} - \mathbf{I}_3\|_F, \quad e_{\text{det}} = |\det(\mathbf{R}_{ab}) - 1|,$$

against the dimensionless policy values `orthogonality_tolerance` and `determinant_tolerance`. Equality with a tolerance is accepted because rejection uses a strict `>` comparison. Acceptance does not project, orthogonalise, or otherwise change the supplied entries.

For a free vector, the validated matrix performs only the coordinate change recalled in Section 7.6.1:

$${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}.$$

The input column ${}^b\mathbf{v}$ contains components expressed along frame $\{b\}$, and the output column ${}^a\mathbf{v}$ contains components of the same free vector expressed along frame $\{a\}$. Rotation preserves the vector's physical units. Translation is absent because a free vector has no point of application.

9.2.2 The complete class design

The complete `Rotation3` declaration is short enough to read as one unit:

```
class Rotation3
{
public:
    // Create identity rotation
    static Rotation3 identity();

    // Create a rotation after validating the supplied matrix
    static Rotation3 from_matrix(
        const Eigen::Matrix3d & matrix,
        const NumericalPolicy & policy = NumericalPolicy{});

    // Apply this rotation to a vector
    [[nodiscard]] Eigen::Vector3d rotate_vector(
        const Eigen::Vector3d & vector_b) const;

private:
    // Construct from underlying matrix
    explicit Rotation3(const Eigen::Matrix3d & matrix);

    // Internal representation
    Eigen::Matrix3d matrix_;
};
```

The `public` part lists what a caller may do. `identity()` constructs the identity rotation, `from_matrix()` validates an external matrix before constructing a rotation, and `rotate_vector()` applies the stored matrix to a free-vector column. Both construction operations are `static`, so they are called on the class, for example `Rotation3::from_matrix(matrix_ab)`, rather than on an existing rotation object.

The `private` constructor is the key safety choice. Only the class's own functions may call `Rotation3(matrix)`, so ordinary callers cannot place an unchecked matrix directly into `matrix_`. The word `explicit` prevents C++ from silently converting an `Eigen::Matrix3d` into a `Rotation3`. The trailing underscore in `matrix_` is a naming convention indicating a data member stored inside each object.

The parameter type `const Eigen::Matrix3d &` passes the existing matrix without copying it and prevents the function from changing it. The default argument `NumericalPolicy{}` constructs a policy containing the default tolerances when the caller supplies no policy. The same `const &` form is used for the input vector. The final `const` in `rotate_vector(...)` prevents the operation from changing `matrix_`, while `[[nodiscard]]` asks the compiler to warn if the caller ignores the returned vector.

9.2.3 Numerical policy and error types

`NumericalPolicy` and `GeometryError` are shared types used by several kernel operations. The production headers define each type once, but this retrospective introduces their members in dependency order. The snippets in this subsection therefore show the members needed by the first `Rotation3` block, not the final chapter-wide declarations. When a later operation needs a genuinely new policy value or failure category, its section explicitly adds that member to the same shared type before using it.

The first two tolerances are stored together:

```
struct NumericalPolicy
{
    double orthogonality_tolerance{1.0e-12};
    double determinant_tolerance{1.0e-12};
};
```

A `struct` is used because these two values are simple public settings. The braces after each member give the default value used by `NumericalPolicy{}`. Both values are dimensionless because they are compared with dimensionless rotation residuals.

The first four error categories support rotation construction and free-vector action. Failures contain both a category that code can test and a message that a person can read:

```
enum class GeometryError
{
    non_finite,
    invalid_rotation,
    invalid_policy,           // a tolerance is invalid.
    unsupported_magnitude    // exceeds the finite numerical domain
};

class GeometryException : public std::runtime_error
{
public:
    GeometryException(GeometryError code, const char * message)
        : std::runtime_error(message), code_(code)
    {
```

```

    }

    [[nodiscard]] GeometryError code() const noexcept { return code_; }

private:
    GeometryError code_;
};

```

enum class keeps the names inside the `GeometryError` scope, producing names such as `GeometryError::invalid_rotation`, and prevents their accidental use as unrelated integers. `GeometryException : public std::runtime_error` means that `GeometryException` is a specialised standard run-time exception. The constructor initialiser list passes `message` to the `std::runtime_error` part and stores `code` in `code_`. A caller reads the message with `what()` and reads the category with `code()`. The final `noexcept` promises that reading the category will not throw another exception.

The four categories have distinct meanings:

- `invalid_policy`: a rotation tolerance is negative or non-finite; policy validation occurs before matrix validation.
- `non_finite`: a supplied matrix or free-vector component is NaN or infinity.
- `invalid_rotation`: finite residuals show that the matrix violates orthogonality or the proper-determinant condition.
- `unsupported_magnitude`: finite inputs cause a required intermediate or output to leave the finite binary64 range.

When `GeometryException` is thrown, construction or vector rotation stops and returns no geometric value.

9.2.4 Complete `from_matrix` implementation

The complete function shows the validation order directly:

```

Rotation3 Rotation3::from_matrix(
    const Eigen::Matrix3d & matrix, const NumericalPolicy & policy)
{
    const bool policy_is_valid = std::isfinite(policy.orthogonality_tolerance) &&
                                std::isfinite(policy.determinant_tolerance) &&
                                policy.orthogonality_tolerance >= 0.0 &&
                                policy.determinant_tolerance >= 0.0;

    if (!policy_is_valid) {
        throw GeometryException(
            GeometryError::invalid_policy,
            "Rotation tolerance must be finite and non-negative.");
    }
}

```

```

    }

    if (!matrix.allFinite()) {
        throw GeometryException(
            GeometryError::non_finite, "Rotation matrix entries must be finite.");
    }

    const Eigen::Matrix3d orthogonality_error =
        matrix.transpose() * matrix - Eigen::Matrix3d::Identity();

    const double orthogonality_residual =
        orthogonality_error.reshapeed().stableNorm(); // Frobenius norm

    const double determinant = matrix.determinant();

    if (!std::isfinite(orthogonality_residual) || !std::isfinite(determinant)) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "Rotation validation produced a non-finite result.");
    }

    const double determinant_error = std::abs(determinant - 1.0);

    if (
        orthogonality_residual > policy.orthogonality_tolerance ||
        determinant_error > policy.determinant_tolerance) {
        throw GeometryException(
            GeometryError::invalid_rotation,
            "Matrix does not satisfy the SO(3) conditions.");
    }

    return Rotation3(matrix);
}

```

The name before `::` identifies the class that owns the function: the first `Rotation3` is the return type, while `Rotation3::from_matrix` is the function's full name. The default policy appears only in the class declaration; callers use that default, but the function definition receives an ordinary `policy` reference.

The function proceeds in four concrete stages. First, `std::isfinite` and the comparisons with zero validate both tolerances. Second, `matrix.allFinite()` checks all nine input entries before matrix arithmetic begins. Third, `transpose()`, matrix multiplication, `Identity()`, `stableNorm()`, and `determinant()` calculate e_{orth} and the determinant needed for e_{det} . Fourth, the two residual comparisons either throw `invalid_rotation` or allow the private constructor to store the matrix.

The intermediate finiteness check is separate from input finiteness. A matrix may contain only finite entries while products used by $\mathbf{R}^T \mathbf{R}$ or $\det(\mathbf{R})$ overflow. Such a calculation fails as `unsupported_magnitude`, not as `non_finite`, because the caller supplied finite entries.

Eigen 3.4.0's direct matrix form `orthogonality_error.stableNorm()` triggered an internal block-index assertion in the project environment. Reshaping the nine entries into a vector selects Eigen's vector implementation. This substitution preserves the required quantity because the Euclidean norm of the vectorised entries equals the matrix Frobenius norm:

$$\|\text{vec}(\mathbf{E})\|_2 = \|\mathbf{E}\|_F.$$

9.2.5 Complete `rotate_vector` implementation

Once construction has established that `matrix_` is an accepted rotation, the second operation checks its vector input, performs the reviewed matrix-column product, and checks the output:

```
Eigen::Vector3d Rotation3::rotate_vector(const Eigen::Vector3d & vector_b) const
{
    if (!vector_b.allFinite()) {
        throw GeometryException(
            GeometryError::non_finite, "Free-vector components must be finite.");
    }

    const Eigen::Vector3d vector_a = matrix_ * vector_b;

    if (!vector_a.allFinite()) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "Rotated free vector is outside the finite numerical range.");
    }
    return vector_a;
}
```

The expression `matrix_ * vector_b` is the direct code form of ${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}$. If an input component is already NaN or infinity, the function throws `non_finite` before multiplication. If every input component is finite but the multiplication produces a non-finite component, the function throws `unsupported_magnitude`. Otherwise, `return vector_a;` gives the caller the re-expressed vector column.

9.2.6 Analytic example and focused tests

The normal non-identity fixture uses the dimensionless quarter-turn matrix

$$\mathbf{R}_{ab} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

and the free-vector input ${}^b\mathbf{v} = [1 \ 0 \ 0]^\top \text{ m}$. The expected output is ${}^a\mathbf{v} = [0 \ 1 \ 0]^\top \text{ m}$, calculated independently by matrix–column multiplication.

The focused GTest executable protects the governing behaviour with nine deterministic cases: identity leaves a vector unchanged; the quarter-turn produces the analytic result; a reflection isolates the determinant requirement; a shear isolates the orthogonality requirement; NaN matrix input returns `non_finite`; a negative tolerance is rejected before an invalid matrix is inspected; finite matrix entries whose validation overflows return `unsupported_magnitude`; an infinite free-vector component returns `non_finite`; and finite vector components whose rotated output overflows return `unsupported_magnitude`.

9.2.7 Deliberate limits

`Eigen::Matrix3d` fixes the matrix shape at compile time, so this entry point cannot receive a wrong-shaped dynamic array; a future external-array adapter must own shape validation. `Rotation3` does not store frame names or physical units, so call sites must preserve the $\mathbf{R}_{ab}$, ${}^b\mathbf{v}$, and ${}^a\mathbf{v}$ relationship through their variable and interface contracts. The tolerance test admits a bounded residual but never repairs the matrix. Point transformation remains a separate operation because adding translation to a free vector would change the represented physical object.

9.3 Validated rigid transformations and point action

This implementation block answers two related questions: how does the library accept only a valid homogeneous rigid transformation, and how does that transformation change the coordinates of a point? For $\mathbf{T}_{ab}$, the pose of frame $\{b\}$ relative to frame $\{a\}$, the reviewed homogeneous equation is

$$\begin{bmatrix} {}^a\mathbf{p}_P \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix}}_{\mathbf{T}_{ab}} \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix}.$$

Expanding the upper three rows gives the operation used by the C++ implementation:

$${}^a\mathbf{p}_P = \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b.$$

Here $\mathbf{R}_{ab}$ is dimensionless, while ${}^a\mathbf{p}_b$, ${}^b\mathbf{p}_P$, and ${}^a\mathbf{p}_P$ are measured in metres. The homogeneous matrix form remains convenient for derivation and later group composition. The implementation stores the equivalent rotation–translation pair because the bottom row is fixed structural information rather than independent geometric state.

9.3.1 The `Transform3` class

The complete declaration is

```
class Transform3
{
public:
    // Create the identity transform
    static Transform3 identity();

    // Validate a raw homogeneous matrix and create a rigid transform
    static Transform3 from_matrix(
        const Eigen::Matrix4d & matrix,
        const NumericalPolicy & policy = NumericalPolicy{});

    // Transform point coordinates from frame {b} into frame {a}
    [[nodiscard]] Eigen::Vector3d transform_point(
        const Eigen::Vector3d & point_b) const;

private:
    Transform3(
        const Rotation3 & rotation,
        const Eigen::Vector3d & translation);

    Rotation3 rotation_;
    Eigen::Vector3d translation_;
};
```

`rotation_` can contain only a `Rotation3` that has passed the $SO(3)$ checks from Section 9.2. `translation_` stores ${}^a\mathbf{p}_b$ in metres. The private constructor prevents a caller from combining an unchecked rotation with a translation. `identity()` constructs the pair $(\mathbf{I}_3, \mathbf{0}_3)$.

The class does not store the raw 4×4 input. After validation, its bottom row contains no additional information, so storing only $(\mathbf{R}_{ab}, {}^a\mathbf{p}_b)$ prevents an internally malformed homogeneous row. A later matrix accessor can reconstruct the convenient homogeneous representation with the exact bottom row $[0 \ 0 \ 0 \ 1]$.

The `Transform3` block extends the shared `NumericalPolicy` with `double homogeneous_row_tolerance{1.0e-12};`, a dimensionless absolute tolerance for the fixed bottom row. It also extends the

shared `GeometryError` enumeration with `invalid_transform`, which reports a finite homogeneous matrix whose required structure is malformed. These are additions to the types introduced in Section 9.2, not separate transform-specific policy or error types.

9.3.2 Complete `from_matrix` implementation

The raw matrix entry point validates the complete representation before constructing the pair:

```
Transform3 Transform3::from_matrix(
    const Eigen::Matrix4d & matrix, const NumericalPolicy & policy)
{
    const bool policy_is_valid =
        std::isfinite(policy.orthogonality_tolerance) &&
        std::isfinite(policy.determinant_tolerance) &&
        std::isfinite(policy.homogeneous_row_tolerance) &&
        policy.orthogonality_tolerance >= 0.0 &&
        policy.determinant_tolerance >= 0.0 &&
        policy.homogeneous_row_tolerance >= 0.0;

    if (!policy_is_valid) {
        throw GeometryException(
            GeometryError::invalid_policy,
            "Transform tolerances must be finite and non-negative.");
    }

    if (!matrix.allFinite()) {
        throw GeometryException(
            GeometryError::non_finite,
            "Transform matrix entries must be finite.");
    }

    const Eigen::Vector4d expected_bottom_row(0.0, 0.0, 0.0, 1.0);

    const Eigen::Vector4d bottom_row_error =
        matrix.row(3).transpose() - expected_bottom_row;

    const double bottom_row_residual =
        bottom_row_error.cwiseAbs().maxCoeff();

    if (bottom_row_residual > policy.homogeneous_row_tolerance) {
        throw GeometryException(
            GeometryError::invalid_transform,
```

```

    "Transform must have homogeneous bottom row [0, 0, 0, 1].");
}

const Eigen::Matrix3d rotation_matrix =
    matrix.block<3, 3>(0, 0);

const Eigen::Vector3d translation =
    matrix.block<3, 1>(0, 3);

const Rotation3 rotation =
    Rotation3::from_matrix(rotation_matrix, policy);

return Transform3(rotation, translation);
}

```

The validation order is policy, all sixteen entries, homogeneous bottom row, and then rotation block. `matrix.row(3).transpose()` converts the bottom row to a four-component column. `cwiseAbs()` takes the absolute value of each component, and `maxCoeff()` returns their infinity norm,

$$e_{\text{row}} = \max(|T_{41}|, |T_{42}|, |T_{43}|, |T_{44} - 1|).$$

The row is accepted when $e_{\text{row}} \leq \varepsilon_{\text{row}}$. A zero tolerance requires an exact structural row. The fixed-size calls `block<3, 3>(0, 0)` and `block<3, 1>(0, 3)` then extract $\mathbf{R}_{ab}$ and ${}^a\mathbf{p}_b$. Delegating the rotation check to `Rotation3::from_matrix` avoids maintaining a second implementation of the $SO(3)$ invariant.

9.3.3 Complete `transform_point` implementation

The point operation expands the upper three rows of the homogeneous multiplication:

```

Eigen::Vector3d Transform3::transform_point(
    const Eigen::Vector3d & point_b) const
{
    if (!point_b.allFinite()) {
        throw GeometryException(
            GeometryError::non_finite,
            "Point coordinates must be finite.");
    }

    const Eigen::Vector3d rotated_point =
        rotation_.rotate_vector(point_b);
}

```

```

const Eigen::Vector3d point_a =
    rotated_point + translation_;

if (!point_a.allFinite()) {
    throw GeometryException(
        GeometryError::unsupported_magnitude,
        "Transformed point is outside the finite numerical range.");
}

return point_a;
}

```

`rotation_.rotate_vector(point_b)` calculates $\mathbf{R}_{ab} {}^b\mathbf{p}_P$. Although the input describes point coordinates, this first subcalculation is the same linear action used for a free vector. Adding `translation_` completes the point transformation and is precisely what distinguishes `transform_point()` from `Rotation3::rotate_vector()`. The final finiteness check detects a finite rotated point and finite translation whose sum overflows binary64.

9.3.4 Analytic example and focused tests

The normal non-identity fixture uses

$$\mathbf{T}_{ab} = \begin{bmatrix} 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & 2 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad {}^b\mathbf{p}_P = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m.}$$

The expected result is

$${}^a\mathbf{p}_P = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \\ 3 \end{bmatrix} \text{ m.}$$

Seven focused tests protect this cycle: identity leaves point coordinates unchanged; the analytic rotation-plus-translation fixture returns $[1 \ 3 \ 3]^\top \text{ m}$; a malformed bottom row returns `invalid_transform`; a reflection in the rotation block returns `invalid_rotation`; a NaN transform entry and an infinite point return `non_finite`; and finite point and translation components whose sum overflows return `unsupported_magnitude`.

9.4 Rotation and transformation composition and inversion

This implementation block answers how the library combines consecutive frame changes, reverses a frame change, and reconstructs a matrix for inspection or interchange. The inputs must already be valid `Rotation3` or `Transform3` values. The output is another group value, and compatible frame labels remain the caller's responsibility.

9.4.1 Governing group operations

Let $\mathbf{T}_{ab}$ describe frame $\{b\}$ relative to frame $\{a\}$ and let $\mathbf{T}_{bc}$ describe frame $\{c\}$ relative to frame $\{b\}$. Products act rightmost first, so changing coordinates from $\{c\}$ through $\{b\}$ to $\{a\}$ requires

$$\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}.$$

Writing both transformations as rotation–translation pairs and multiplying their blocks gives

$$\begin{aligned}\mathbf{T}_{ab}\mathbf{T}_{bc} &= \begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{bc} & {}^b\mathbf{p}_c \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_{ab}\mathbf{R}_{bc} & \mathbf{R}_{ab}{}^b\mathbf{p}_c + {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.\end{aligned}$$

Therefore the implementation must calculate

$$\mathbf{R}_{ac} = \mathbf{R}_{ab}\mathbf{R}_{bc}, \quad {}^a\mathbf{p}_c = \mathbf{R}_{ab}{}^b\mathbf{p}_c + {}^a\mathbf{p}_b.$$

The rotations are dimensionless. The three translation columns are measured in metres, but their components initially use different frames. Multiplication by $\mathbf{R}_{ab}$ first re-expresses ${}^b\mathbf{p}_c$ in frame $\{a\}$; only then can the two metre-valued columns be added.

To reverse $\mathbf{T}_{ab}$, require its product with $\mathbf{T}_{ba}$ to equal the identity:

$$\begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{ba} & {}^b\mathbf{p}_a \\ \mathbf{0}_3^\top & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

Equating the upper-left blocks gives $\mathbf{R}_{ab}\mathbf{R}_{ba} = \mathbf{I}_3$. Because a rotation matrix is orthogonal, $\mathbf{R}_{ba} = \mathbf{R}_{ab}^\top$. Equating the upper-right blocks then gives

$$\mathbf{R}_{ab}{}^b\mathbf{p}_a + {}^a\mathbf{p}_b = \mathbf{0}_3,$$

and multiplying by $\mathbf{R}_{ab}^\top$ yields

$${}^b\mathbf{p}_a = -\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b.$$

Hence

$$\mathbf{T}_{ab}^{-1} = \mathbf{T}_{ba} = \begin{bmatrix} \mathbf{R}_{ab}^\top & -\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

9.4.2 Public operations and matrix reconstruction

The cycle adds the same three kinds of operation to the two value types:

```
// Rotation3
[[nodiscard]] Eigen::Matrix3d matrix() const;
[[nodiscard]] Rotation3 compose(const Rotation3 & rotation_bc) const;
[[nodiscard]] Rotation3 inverse() const;

// Transform3
[[nodiscard]] Eigen::Matrix4d matrix() const;
[[nodiscard]] Transform3 compose(const Transform3 & transform_bc) const;
[[nodiscard]] Transform3 inverse() const;
```

Each member function is `const` because observing, composing, or inverting a value does not modify that value. `[[nodiscard]]` asks the compiler to warn when a caller discards the returned matrix or group value. The argument names state the required right operand: calling `rotation_ab.compose(rotation_bc)` or `transform_ab.compose(transform_bc)` produces the corresponding `ac` value.

`Rotation3::matrix()` returns a copy of its stored 3×3 matrix. `Transform3::matrix()` constructs a new identity matrix and replaces only its rotation and translation blocks:

```
Eigen::Matrix4d Transform3::matrix() const
{
    Eigen::Matrix4d result = Eigen::Matrix4d::Identity();

    result.block<3, 3>(0, 0) = rotation_.matrix();
    result.block<3, 1>(0, 3) = translation_;

    return result;
}
```

Starting from `Identity()` makes the reconstructed bottom row exactly $[0\ 0\ 0\ 1]$. A small bottom-row error admitted at the raw input boundary is therefore not preserved inside the core type.

9.4.3 Complete `Rotation3` operations

The rotation implementation directly follows $\mathbf{R}_{ac} = \mathbf{R}_{ab}\mathbf{R}_{bc}$ and $\mathbf{R}_{ba} = \mathbf{R}_{ab}^T$:

```
Eigen::Matrix3d Rotation3::matrix() const { return matrix_; }

Rotation3 Rotation3::compose(const Rotation3 & rotation_bc) const
{
    const Eigen::Matrix3d matrix_ac = matrix_ * rotation_bc.matrix_;

    if (!matrix_ac.allFinite()) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "Rotation composition produced a non-finite result.");
    }

    return Rotation3(matrix_ac);
}

Rotation3 Rotation3::inverse() const
{
    const Eigen::Matrix3d matrix_ba = matrix_.transpose();
    return Rotation3(matrix_ba);
}
```

The class can read `rotation_bc.matrix_` even though that member is private because both objects have the same class type. Composition checks the calculated matrix before constructing the result. Inversion uses the transpose instead of a general matrix-inverse algorithm because membership in $SO(3)$ has already established orthogonality.

9.4.4 Complete `Transform3` operations

The transformation implementation calculates rotation composition and translation composition as separate, unit-consistent steps:

```
Transform3 Transform3::compose(const Transform3 & transform_bc) const
{
    const Rotation3 rotation_ac = rotation_.compose(transform_bc.rotation_);
```

```

    const Eigen::Vector3d translated_position =
        rotation_.rotate_vector(transform_bc.translation_);
    const Eigen::Vector3d translation_ac = translated_position + translation_;

    if (!translation_ac.allFinite()) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "Transform composition produced a non-finite translation.");
    }

    return Transform3(rotation_ac, translation_ac);
}

Transform3 Transform3::inverse() const
{
    const Rotation3 rotation_ba = rotation_.inverse();
    const Eigen::Vector3d translation_ba =
        rotation_ba.rotate_vector(-translation_);

    return Transform3(rotation_ba, translation_ba);
}

```

In `compose()`, `rotation_.rotate_vector(transform_bc.translation_)` implements $\mathbf{R}_{ab}^b \mathbf{p}_c$. The variable `translated_position` is therefore expressed in frame $\{a\}$ despite the general name; adding `translation_` produces ${}^a\mathbf{p}_c$. The final finiteness check rejects overflow in that addition.

In `inverse()`, `rotation_ba` stores $\mathbf{R}_{ab}^T$. Passing `-translation_` to its vector action calculates $-\mathbf{R}_{ab}^T {}^a\mathbf{p}_b$, which is ${}^b\mathbf{p}_a$ in metres. Reusing the validated rotation operations keeps the code aligned with the two derived component equations.

9.4.5 Analytic fixtures and focused tests

The rotation test composes quarter turns about the positive z - and x -axes in both orders. The independently calculated products satisfy $\mathbf{R}_z\mathbf{R}_x \neq \mathbf{R}_x\mathbf{R}_z$, so the test detects an accidentally reversed multiplication order. A second test uses a 45-degree rotation and verifies both $\mathbf{R}\mathbf{R}^{-1} = \mathbf{I}_3$ and $\mathbf{R}^{-1}\mathbf{R} = \mathbf{I}_3$.

The transformation composition fixture uses

$${}^a\mathbf{p}_b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \text{ m}, \quad {}^b\mathbf{p}_c = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m},$$

with $\mathbf{R}_{ab}$ a positive quarter turn about the shared z -axis and $\mathbf{R}_{bc} = \mathbf{I}_3$. The rotated second translation is $[0 \ 1 \ 0]^\top$ m, so the expected composed translation is $[1 \ 3 \ 3]^\top$ m. This fixture would fail if the implementation merely added columns expressed in different frames.

For the same $\mathbf{T}_{ab}$, the independently calculated inverse translation is

$${}^b\mathbf{p}_a = -\mathbf{R}_{ab}^\top \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \\ -3 \end{bmatrix} \text{ m.}$$

The inverse test checks that analytic matrix and both identity products. It compares rotation blocks with a dimensionless tolerance and translation columns with a metre-valued tolerance. The boundary test composes two identity rotations whose finite x -translations are both `std::numeric_limits<double>::max()`. Their sum overflows to infinity, so `compose()` throws `unsupported_magnitude` instead of returning a geometric value.

9.4.6 Deliberate limits

The core types do not store runtime frame labels, so `compose()` cannot detect a call whose conceptual middle frames are incompatible. Composition uses the private constructor because valid group elements are closed under exact mathematical composition; it checks finiteness but does not repeat the raw-input residual tests. Repeated floating-point products can therefore accumulate a small orthogonality or determinant drift. The library does not yet define a drift-monitoring or re-orthogonalisation policy, and it does not silently project a result back onto $SO(3)$. `Transform3` exposes the complete matrix but does not yet expose its rotation and translation as separate public values.

9.5 Linear-first $SO(3)$ and $SE(3)$ hat and vee maps

The next operation converts a three- or six-component coordinate column into the structured matrix used by the Lie-group equations. The inverse operation must recover the column only when the supplied matrix has the required structure. The mathematical definitions come from the cross-product matrix in Section 7.4.2 and the spatial-motion hat matrices in Section 7.9.2 and Section 7.9.3.

9.5.1 Governing equations and component order

The $SO(3)$ hat operation does not map one rotation matrix to another rotation matrix. It maps a three-component coordinate column to a matrix in the Lie algebra $\mathfrak{so}(3)$. This target space is the set of all real 3×3 skew-symmetric matrices:

$$\mathfrak{so}(3) := \{\mathbf{A} \in \mathbb{R}^{3 \times 3} \mid \mathbf{A}^\top = -\mathbf{A}\}.$$

The hat map and its inverse, the vee map, therefore have the domains and codomains

$$\begin{aligned} \text{hat}_{SO(3)} : \mathbb{R}^3 &\longrightarrow \mathfrak{so}(3), \\ \text{vee}_{SO(3)} : \mathfrak{so}(3) &\longrightarrow \mathbb{R}^3. \end{aligned}$$

For a three-component input column $\mathbf{x} = [x_1 \ x_2 \ x_3]^\top \in \mathbb{R}^3$, the forward map is

$$\text{hat}_{SO(3)}(\mathbf{x}) = \hat{\mathbf{x}} = [\mathbf{x}]_\times = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix}, \quad [\mathbf{x}]_\times \mathbf{y} = \mathbf{x} \times \mathbf{y}.$$

The output $[\mathbf{x}]_\times$ belongs to $\mathfrak{so}(3)$ because $[\mathbf{x}]_\times^\top = -[\mathbf{x}]_\times$. For an input $\mathbf{A} \in \mathfrak{so}(3)$, the inverse map selects the three independent entries:

$$\text{vee}_{SO(3)}(\mathbf{A}) = \begin{bmatrix} A_{32} \\ A_{13} \\ A_{21} \end{bmatrix}.$$

On these exact mathematical domains, the maps undo each other:

$$\text{vee}_{SO(3)}(\text{hat}_{SO(3)}(\mathbf{x})) = \mathbf{x}, \quad \text{hat}_{SO(3)}(\text{vee}_{SO(3)}(\mathbf{A})) = \mathbf{A}.$$

The corresponding $SE(3)$ operations use the matrix space

$$\mathfrak{se}(3) := \left\{ \begin{bmatrix} \Phi & \rho \\ 0_3^\top & 0 \end{bmatrix} \mid \Phi \in \mathfrak{so}(3), \rho \in \mathbb{R}^3 \right\}.$$

The project writes each six-component input in linear-first order. The label on $\mathbb{R}_{\text{linear-first}}^6$ records this representation convention; it does not create a different six-dimensional vector space. The two maps are

$$\begin{aligned} \text{hat}_{SE(3)} : \mathbb{R}_{\text{linear-first}}^6 &\longrightarrow \mathfrak{se}(3), \\ \text{vee}_{SE(3)} : \mathfrak{se}(3) &\longrightarrow \mathbb{R}_{\text{linear-first}}^6. \end{aligned}$$

For a finite exponential-coordinate column expressed in frame $\{a\}$, the input is

$${}^a\boldsymbol{\eta} = \begin{bmatrix} {}^a\boldsymbol{\rho} \\ {}^a\boldsymbol{\phi} \end{bmatrix}, \quad {}^a\boldsymbol{\rho} \in \mathbb{R}^3 \text{ m}, \quad {}^a\boldsymbol{\phi} \in \mathbb{R}^3 \text{ rad}.$$

Recall from Section 8.5 that ${}^a\boldsymbol{\eta}$ describes one finite screw displacement. Its rotation-vector block ${}^a\boldsymbol{\phi}$ has the oriented screw-axis direction as its direction and the rotation angle as its magnitude. Its finite linear block ${}^a\boldsymbol{\rho}$ records the linear part of the same screw displacement, including axis-offset and pitch effects when applicable. Both columns are expressed in frame $\{a\}$.

The forward map places the angular block in the upper-left matrix block and the linear block in the upper-right matrix block:

$$\text{hat}_{SE(3)}({}^a\boldsymbol{\eta}) = \widehat{{}^a\boldsymbol{\eta}} = \begin{bmatrix} [{}^a\boldsymbol{\phi}]_{\times} & {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^{\top} & 0 \end{bmatrix}.$$

Conversely, if $\boldsymbol{\Phi} \in \mathfrak{so}(3)$ and $\boldsymbol{\rho} \in \mathbb{R}^3$, then

$$\text{vee}_{SE(3)}\left(\begin{bmatrix} \boldsymbol{\Phi} & \boldsymbol{\rho} \\ \mathbf{0}_3^{\top} & 0 \end{bmatrix}\right) = \begin{bmatrix} \boldsymbol{\rho} \\ \text{vee}_{SO(3)}(\boldsymbol{\Phi}) \end{bmatrix}.$$

Thus, on the exact declared domains,

$$\text{vee}_{SE(3)}\left(\text{hat}_{SE(3)}({}^a\boldsymbol{\eta})\right) = {}^a\boldsymbol{\eta}.$$

The same $SE(3)$ map accepts a linear-first twist $[({}^a\boldsymbol{\nu})^{\top} ({}^a\boldsymbol{\omega})^{\top}]^{\top}$ when the blocks instead have units m s^{-1} and rad s^{-1} . The hat map only rearranges the supplied components into a structured matrix; it does not evaluate an exponential. Its output belongs to $\mathfrak{se}(3)$ and is not a homogeneous transformation in $SE(3)$: its bottom-right entry is zero rather than one. The mathematical vee maps above require exact membership in $\mathfrak{so}(3)$ or $\mathfrak{se}(3)$; the software implementation below accepts only explicitly bounded structural residuals beyond those exact domains.

The hat-and-vee block extends `NumericalPolicy` with `double tangent_tolerance{1.0e-12};`, the absolute tolerance used to accept only small structural residuals in a numerical tangent matrix. It also extends `GeometryError` with `invalid_tangent`, which reports a finite matrix that violates the required skew symmetry or $\mathfrak{se}(3)$ bottom-row structure. The hat operations do not need this tolerance because they construct the required structure directly; the vee operations use it when validating an external matrix.

9.5.2 Public functions and the fixed component-order alias

The header gives the six-component Eigen type a project-specific name and declares four free functions:

```
[[nodiscard]] Eigen::Matrix3d hat_so3(
    const Eigen::Vector3d & vector);

[[nodiscard]] Eigen::Vector3d vee_so3(
    const Eigen::Matrix3d & matrix,
    const NumericalPolicy & policy = NumericalPolicy{});

using Vector6LinearFirst = Eigen::Matrix<double, 6, 1>;

[[nodiscard]] Eigen::Matrix4d hat_se3(
    const Vector6LinearFirst & coordinates);

[[nodiscard]] Vector6LinearFirst vee_se3(
    const Eigen::Matrix4d & matrix,
    const NumericalPolicy & policy = NumericalPolicy{});
```

These are **free functions** because they perform conversions and do not own persistent state. The `using` declaration creates the readable name `Vector6LinearFirst` for Eigen's fixed-size 6×1 matrix type. The name records the convention for human readers, although C++ still cannot distinguish two six-columns that use different frames or units. The `[[nodiscard]]` attribute asks the compiler to warn when a caller discards a calculated matrix or coordinate column.

9.5.3 Implementing the $SO(3)$ maps

After rejecting a non-finite input column, `hat_so3()` fills the matrix in exactly the sign pattern of $[\mathbf{x}]_{\times}$:

```
Eigen::Matrix3d skew_matrix;

// clang-format off
skew_matrix <<
    0.0,      -vector.z(), vector.y(),
    vector.z(), 0.0,      -vector.x(),
    -vector.y(), vector.x(), 0.0;
// clang-format on

return skew_matrix;
```

Eigen's comma initialiser `<<` supplies entries one row at a time. The local `clang-format` comments preserve the visually meaningful matrix layout; they affect formatting only and do not affect compilation.

An inverse map must not treat every 3×3 array as a valid tangent matrix. For a candidate $\mathbf{A}$, the implementation calculates

$$\text{sym}(\mathbf{A}) = \frac{1}{2}(\mathbf{A} + \mathbf{A}^T), \quad e_{\text{skew}} = \max_{i,j} |\text{sym}(\mathbf{A})_{ij}|.$$

The matrix is admitted only when $e_{\text{skew}} \leq \varepsilon_{\text{tangent}}$. The implementation then takes the skew part and selects the three independent entries required by the vee definition:

```
const Eigen::Matrix3d half_matrix = 0.5 * matrix;
const Eigen::Matrix3d symmetric_part =
    half_matrix + half_matrix.transpose();

const double skew_residual =
    symmetric_part.cwiseAbs().maxCoeff();

if (skew_residual > policy.tangent_tolerance) {
    throw GeometryException(
        GeometryError::invalid_tangent,
        "Matrix is not skew-symmetric within tolerance.");
}

const Eigen::Matrix3d skew_part =
    half_matrix - half_matrix.transpose();

const Eigen::Vector3d vector(
    skew_part(2, 1), skew_part(0, 2), skew_part(1, 0));
```

Multiplying by $1/2$ before adding or subtracting reduces the chance that two large, finite entries overflow during the structural check. `cwiseAbs()` takes the absolute value of each entry, and `maxCoeff()` selects the largest one. A candidate inside the declared tolerance is converted to its skew part only after that residual has been checked; a materially symmetric component is therefore reported rather than silently discarded.

9.5.4 Implementing the linear-first $SE(3)$ maps

The six-column is separated and inserted into the matrix according to the governing block equation:

```

const Eigen::Vector3d linear_part = coordinates.head<3>();
const Eigen::Vector3d angular_part = coordinates.tail<3>();

Eigen::Matrix4d tangent_matrix = Eigen::Matrix4d::Zero();

tangent_matrix.block<3, 3>(0, 0) = hat_so3(angular_part);
tangent_matrix.block<3, 1>(0, 3) = linear_part;

return tangent_matrix;

```

`head<3>()` selects the first three entries and `tail<3>()` selects the last three. `block<3, 3>(0, 0)` means “the 3×3 block whose first entry is at row zero, column zero.” `block<3, 1>(0, 3)` selects the top-right column. Initialising the complete matrix with `Zero()` establishes the structural bottom row before the two meaningful blocks are assigned.

The inverse first requires the complete bottom row to be zero within the tangent tolerance. It then delegates the upper-left block to the already validated $SO(3)$ vee map and restores the project order explicitly:

```

const double bottom_row_residual =
    matrix.row(3).cwiseAbs().maxCoeff();

if (bottom_row_residual > policy.tangent_tolerance) {
    throw GeometryException(
        GeometryError::invalid_tangent,
        "SE(3) tangent matrix must have a zero bottom row.");
}

const Eigen::Matrix3d angular_matrix =
    matrix.block<3, 3>(0, 0);
const Eigen::Vector3d angular_part =
    vee_so3(angular_matrix, policy);
const Eigen::Vector3d linear_part =
    matrix.block<3, 1>(0, 3);

Vector6LinearFirst coordinates;
coordinates.head<3>() = linear_part;
coordinates.tail<3>() = angular_part;

return coordinates;

```

The explicit `head` and `tail` assignments protect the linear-first convention. Reversing them would compile successfully but would give the six entries the wrong physical meaning.

9.5.5 Concrete round trip

For the finite coordinate column

$${}^a\boldsymbol{\eta} = [1 \quad -2 \quad 3 \quad 0.1 \quad -0.2 \quad 0.3]^\top,$$

the first three entries are in metres and the last three are in radians. The implementation produces

$$\widehat{{}^a\boldsymbol{\eta}} = \begin{bmatrix} 0 & -0.3 & -0.2 & 1 \\ 0.3 & 0 & -0.1 & -2 \\ 0.2 & 0.1 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

A caller can perform the round trip directly:

```
Vector6LinearFirst coordinates;
coordinates << 1.0, -2.0, 3.0, 0.1, -0.2, 0.3;

const Eigen::Matrix4d tangent_matrix = hat_se3(coordinates);
const Vector6LinearFirst recovered = vee_se3(tangent_matrix);
```

The recovered column equals `coordinates`. This equality tests both the sign pattern and the linear-first block order; checking only that the output has six entries would test neither mathematical property.

9.5.6 Validation and focused evidence

Every hat input and every vee matrix is checked for finite entries. Each vee operation also requires a finite, non-negative `tangent_tolerance`; a structurally invalid input throws `invalid_tangent`, while a non-finite input throws `non_finite`. Intermediate symmetric- and skew-part calculations are checked so that arithmetic outside the finite numerical range throws `unsupported_magnitude` rather than returning a plausible column.

The focused tests cover the cross-product identity, exact skew symmetry, the zero map, non-finite rejection, $SO(3)$ hat-vee recovery, tolerated and rejected symmetric contamination, invalid tangent tolerance, the analytic linear-first $SE(3)$ block layout, the zero map, $SE(3)$ hat-vee recovery, malformed bottom rows, invalid angular blocks, and non-finite translation entries.

9.5.7 Deliberate limits

These functions validate matrix structure but do not evaluate an exponential, logarithm, or adjoint. `Vector6LinearFirst` documents component order but does not enforce a frame or a unit at compile time, so callers must keep coordinate columns in metres and radians separate from twists in metres per second and radians per second. The tangent tolerance is an absolute entrywise threshold, not a relative error model. A matrix admitted within tolerance is returned as the corresponding skew coordinates, but the input matrix itself is never modified or presented as an exact tangent matrix.

9.6 Stable $SO(3)$ exponential from a rotation vector

The $SO(3)$ exponential answers a finite-motion question: given an oriented rotation axis multiplied by a signed rotation angle, what proper rotation matrix represents that motion? The derivation and numerical branches belong to Section 8.5. This section records how the reviewed equation is represented by `Rotation3::from_rotation_vector()`.

9.6.1 Input, output, frame, and units

Let ${}^a\phi \in \mathbb{R}^3$ be a finite rotation-vector column expressed in frame $\{a\}$ and measured in radians. Its direction gives the oriented rotation axis and its Euclidean norm gives the unsigned rotation angle

$$\theta := \|{}^a\phi\|_2 \in \mathbb{R}_{\geq 0} \text{ rad.}$$

The implemented mapping is

$$\begin{aligned} \text{from_rotation_vector} : \mathbb{R}^3 &\longrightarrow SO(3), \\ {}^a\phi &\longmapsto \mathbf{R}({}^a\phi) = \exp([{}^a\phi]_{\times}). \end{aligned}$$

The C++ parameter `rotation_vector` stores the three components of ${}^a\phi$. The local variable `theta` stores $\theta = \|{}^a\phi\|_2$, and the local matrix `rotation_matrix` stores the resulting $\mathbf{R}({}^a\phi)$. The mathematical domain is $\mathbb{R}^3$, while the software domain is the bounded subset whose entries are finite and whose norm does not exceed `maximum_exponential_angle`. The returned `Rotation3` stores the dimensionless matrix $\mathbf{R}({}^a\phi) \in SO(3)$. The class does not store the label $\{a\}$, so the caller remains responsible for applying the result with a consistent frame interpretation.

9.6.2 Rodrigues' formula and the two coefficients

Define the skew-symmetric rotation generator

$$\Phi := [{}^a\phi]_{\times} \in \mathfrak{so}(3).$$

For $\theta > 0$, the implementation evaluates Rodrigues' formula in finite rotation-vector form:

$$\begin{aligned} \mathbf{R}({}^a\phi) &= \exp(\Phi) = \exp([{}^a\phi]_{\times}) = \mathbf{I}_3 + A(\theta)\Phi + B(\theta)\Phi^2 \\ A(\theta) &= \frac{\sin \theta}{\theta}, \quad B(\theta) = \frac{1 - \cos \theta}{\theta^2}. \end{aligned}$$

Here $\exp(\Phi)$ is the matrix exponential of the hatted rotation vector, and Rodrigues' formula is its closed-form evaluation for a 3×3 skew-symmetric matrix. The coefficient A scales the skew-symmetric first-order term. The coefficient B scales the symmetric second-order term. At $\theta = 0$, both displayed ratios have removable divisions by zero even though their limits are finite, so the implementation must select a separate evaluation instead of calculating the ratios directly.

9.6.3 Public factory and numerical policy

The public factory returns the existing validated rotation value type:

```
static Rotation3 from_rotation_vector(
    const Eigen::Vector3d & rotation_vector,
    const NumericalPolicy & policy = NumericalPolicy{});
```

This operation extends the existing `NumericalPolicy` with two more members:

```
// Inclusive upper bound for the small-angle series
double series_angle_threshold{1.0e-4};

// Maximum supported rotation-vector norm in radians.
double maximum_exponential_angle{1.0e6};
```

These members are added to the same policy that already contains the rotation, homogeneous-row, and tangent tolerances. The first value selects a numerically stable formula; it does not classify a small nonzero motion as zero. The second value declares the largest angular input for which the interface promises to evaluate trigonometric functions and the subsequent matrix arithmetic.

9.6.4 Validation and overflow-safe angle calculation

The factory validates the policy before inspecting the geometric input. The series threshold must be finite, strictly positive, and less than π ; the maximum supported angle must be finite and at least π . The source includes the C++20 `<numbers>` header, whose `std::numbers::pi` constant supplies the standard double-precision value of π without a handwritten project constant:

```
const bool policy_is_valid =
    std::isfinite(policy.series_angle_threshold) &&
    std::isfinite(policy.maximum_exponential_angle) &&
    policy.series_angle_threshold > 0.0 &&
    policy.series_angle_threshold < std::numbers::pi &&
    policy.maximum_exponential_angle >= std::numbers::pi;
```

After checking that every component is finite, the implementation calculates the angle with Eigen's scaled vector norm:

```
const double theta = rotation_vector.stableNorm();

if (!std::isfinite(theta)) {
    throw GeometryException(
        GeometryError::unsupported_magnitude,
        "Rotation-vector norm is outside the finite numerical range.");
}

if (theta > policy.maximum_exponential_angle) {
    throw GeometryException(
        GeometryError::unsupported_magnitude,
        "Rotation-vector angle exceeds the supported maximum.");
}
```

In this code, `rotation_vector` is ${}^a\phi$ and `theta` is $\theta = \|{}^a\phi\|_2$. `stableNorm()` rescales the 3×1 vector internally before forming its Euclidean norm, reducing avoidable overflow and underflow. The maximum-angle check occurs before calculating θ^2 and later powers, so the declared domain bounds those intermediate values.

9.6.5 Exact-zero, small-angle, and nominal branches

The exact-zero branch returns the identity without evaluating a trigonometric ratio:

```
if (theta == 0.0) {
    return identity();
}
```

```
}
```

The comparison is exact by design. A representable nonzero angle, however small, proceeds to Rodrigues' formula and therefore is not erased by a tolerance.

For $0 < \theta \leq \theta_{\text{series}}$, direct subtraction in $1 - \cos \theta$ loses relative precision because both terms approach one. The implementation instead evaluates the reviewed even-power series through the sixth-order terms:

```
const double theta_squared = theta * theta;

double coefficient_a;
double coefficient_b;

if (theta <= policy.series_angle_threshold) {
    const double theta_fourth = theta_squared * theta_squared;
    const double theta_sixth = theta_fourth * theta_squared;

    coefficient_a =
        1.0 - theta_squared / 6.0 + theta_fourth / 120.0 -
        theta_sixth / 5040.0;

    coefficient_b =
        0.5 - theta_squared / 24.0 + theta_fourth / 720.0 -
        theta_sixth / 40320.0;
} else {
    coefficient_a = std::sin(theta) / theta;
    coefficient_b =
        (1.0 - std::cos(theta)) / theta_squared;
}
```

The code names follow Rodrigues' equation directly: `theta_squared`, `theta_fourth`, and `theta_sixth` are θ^2 , θ^4 , and θ^6 ; `coefficient_a` is $A(\theta)$; and `coefficient_b` is $B(\theta)$. The coefficient variables are declared before the `if` statement because either branch must assign them and the matrix calculation after the statement must use them. The comparison `<=` makes the series threshold inclusive. The `else` branch uses the direct formulas only after the angle is large enough for their subtraction and division to be resolved reliably at the declared accuracy.

9.6.6 Constructing the proper rotation

The final lines map each part of Rodrigues' equation directly to an Eigen expression:

```

const Eigen::Matrix3d rotation_generator =
    hat_so3(rotation_vector);

const Eigen::Matrix3d rotation_matrix =
    Eigen::Matrix3d::Identity() +
    coefficient_a * rotation_generator +
    coefficient_b * rotation_generator * rotation_generator;

if (!rotation_matrix.allFinite()) {
    throw GeometryException(
        GeometryError::unsupported_magnitude,
        "Rotation exponential produced a non-finite matrix.");
}

return Rotation3(rotation_matrix);

```

Here rotation_vector is ${}^a\phi$, $\text{rotation_generator}$ is $\Phi = [{}^a\phi]_{\times}$, and $\text{rotation_generator} * \text{rotation_generator}$ is Φ^2 . Therefore $\text{coefficient_a} * \text{rotation_generator}$ is $A(\theta)\Phi$, $\text{coefficient_b} * \text{rotation_generator} * \text{rotation_generator}$ is $B(\theta)\Phi^2$, and rotation_matrix is the completed $\mathbf{R}({}^a\phi)$. The private constructor is available here because the factory is a member of `Rotation3`. The exponential equation establishes the proper-rotation structure mathematically, and the implementation separately rejects a non-finite numerical result.

9.6.7 Concrete usage

A positive quarter turn about the x_a -axis is represented by

$${}^a\phi_x = \begin{bmatrix} \pi/2 \\ 0 \\ 0 \end{bmatrix} \text{ rad.}$$

The corresponding call is

```

const double half_pi = 0.5 * std::numbers::pi;
const Eigen::Vector3d rotation_vector(half_pi, 0.0, 0.0);

const Rotation3 rotation =
    Rotation3::from_rotation_vector(rotation_vector);

```

It returns

$$\mathbf{R}_x\left(\frac{\pi}{2}\right) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}.$$

9.6.8 Verification properties

The tests are organised around the mathematical and numerical contracts of the exponential rather than around implementation line coverage. Exact identity and an analytic quarter turn provide known outputs. A 10^{-8} rad case checks that the small-angle branch preserves a nonzero rotation. Values on both sides of the series threshold check that changing formulas does not change the represented motion. The maximum-angle boundary, a non-finite input, and an invalid series threshold check the declared software domain. An arbitrary-axis case checks the group properties

$$\mathbf{R}^\top \mathbf{R} = \mathbf{I}_3, \quad \det(\mathbf{R}) = 1, \quad \mathbf{R}(-\phi) = \mathbf{R}(\phi)^\top.$$

These cases separate independent analytic examples, numerical branch boundaries, invalid inputs, and algebraic invariants. Mutable pass counts and package-linter results are operational evidence, so they are omitted from this lesson.

9.6.9 Deliberate limits

The exponential map is many-to-one: rotation vectors that differ by a complete turn about the same axis can produce the same matrix. `from_rotation_vector()` evaluates the supplied vector without reducing its angle modulo 2π or selecting a principal representative. Section 9.7 records the inverse operation and its principal selection convention.

This is a binary64 $SO(3)$ implementation, not a general matrix-exponential routine. It evaluates Rodrigues' coefficients with the small-angle series or the direct trigonometric formulas according to `NumericalPolicy`. `Rotation3` does not encode frames or angle units, so callers must supply radians and preserve the frame meaning.

9.7 Principal $SO(3)$ logarithm to a rotation vector

The principal $SO(3)$ logarithm answers the inverse finite-motion question: given a validated rotation matrix, which rotation-vector column with angle in $[0, \pi]$ represents the same orientation change? The mathematical derivation belongs to Section 8.5, especially Equation 8.31, Equation 8.32, and Equation 8.36. This section records how `Rotation3::to_principal_rotation_vector()` implements that reviewed result.

9.7.1 Input, output, frame, and units

Let $\mathbf{R} \in SO(3)$ be the dimensionless matrix already stored by a `Rotation3`. The C++ member `matrix_` stores this $\mathbf{R}$. If $\mathbf{R}$ is interpreted in frame $\{a\}$, `Rotation3::to_principal_rotation_vector()` returns a `RotationLogResult` whose member `rotation_vector` stores a finite rotation-vector column ${}^a\phi \in \mathbb{R}^3$ expressed in the same frame and measured in radians. The selected angle is

$$\theta := \|{}^a\phi\|_2 \in [0, \pi] \text{ rad.}$$

The implemented mapping is

$$\begin{aligned} \text{to_principal_rotation_vector} : SO(3) &\longrightarrow \{\phi \in \mathbb{R}^3 : \|\phi\|_2 \leq \pi\}, \\ \mathbf{R} &\longmapsto {}^a\phi, \quad \exp([{}^a\phi]_{\times}) = \mathbf{R}. \end{aligned}$$

The closed ball on the right still contains two opposite vectors at its π boundary that represent the same rotation. The implementation makes the mapping single-valued there by a deterministic axis-sign convention described below.

The operation is a non-static `const` member function:

```
[[nodiscard]] RotationLogResult to_principal_rotation_vector(
    const NumericalPolicy & policy = NumericalPolicy{}) const;
```

There is no matrix parameter because the method reads the validated `matrix_` already held by the `Rotation3` object. The trailing `const` promises that the operation does not modify that object. `[[nodiscard]]` asks the compiler to warn when a caller discards the recovered coordinates and branch information.

9.7.2 Returning the coordinates and the formula branch

The result contains both the rotation vector and the numerical formula used to recover it:

```
enum class RotationLogBranch
{
    identity,
    small_angle,
    nominal,
    near_pi
};

struct RotationLogResult
```

```
{
    Eigen::Vector3d rotation_vector;
    RotationLogBranch branch;
};
```

Near π , recovering the rotation axis requires a separate stable formula region. This operation therefore extends `NumericalPolicy` with

```
// Inclusive distance from pi for near-pi axis recovery.
double near_pi_tolerance{1.0e-6};
```

The new member is measured in radians and sets the inclusive distance below π at which the near- π formula begins. The logarithm reuses the previously added `series_angle_threshold` for its small-angle region.

The branch value makes a numerically important decision observable in tests and diagnostics. Here `nominal` means the ordinary middle-angle case in which the direct logarithm formula is suitable; it does not mean a desired or commanded angle. The four regions are

<code>theta = 0</code>	<code>identity</code>
<code>0 < theta <= series_angle_threshold</code>	<code>small_angle</code>
<code>series_angle_threshold < theta <</code> <code>pi - near_pi_tolerance</code>	<code>nominal</code>
<code>pi - near_pi_tolerance <= theta <= pi</code>	<code>near_pi</code>

The policy must leave these regions ordered and non-overlapping. In particular,

$$0 < \theta_{\text{series}} < \pi - \theta_{\text{near-}\pi}.$$

9.7.3 Recovering the principal angle

The implementation first extracts the cosine and skew data derived in Equation 8.31:

$$\cos(\theta) = \frac{\text{tr}(\mathbf{R}) - 1}{2}, \quad {}^a\mathbf{u} = \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix} = 2 \sin \theta {}^a\mathbf{s}_\omega.$$

The corresponding C++ is

```
double cosine_theta = 0.5 * (matrix_.trace() - 1.0);

// clang-format off
```

```

const Eigen::Vector3d skew_difference(
    matrix_(2, 1) - matrix_(1, 2),
    matrix_(0, 2) - matrix_(2, 0),
    matrix_(1, 0) - matrix_(0, 1));
// clang-format on

const double sine_theta =
    0.5 * skew_difference.stableNorm();

const double theta =
    std::atan2(sine_theta, cosine_theta);

```

The identifier-to-equation mapping in this block is direct: `matrix_` is $\mathbf{R}$, `cosine_theta` is $\cos \theta$, `skew_difference` is ${}^a\mathbf{u}$, `sine_theta` is $\sin \theta = \|{}^a\mathbf{u}\|_2/2$, and `theta` is the principal angle θ . In particular, the name `skew_difference` refers to the three-component column extracted from the skew matrix difference $\mathbf{R} - \mathbf{R}^\top$; it is not itself a matrix. `stableNorm()` calculates $\|{}^a\mathbf{u}\|_2$ with internal scaling. Because the principal interval makes $\sin \theta \geq 0$, `sine_theta` is the nonnegative principal sine. `atan2(sine_theta, cosine_theta)` then recovers the angle in $[0, \pi]$ without relying on `acos()` alone near the identity.

For an exact rotation, $\cos(\theta) \in [-1, 1]$. Floating-point trace arithmetic can place it just outside that interval. The implementation clamps only an overshoot no larger than the dimensionless `orthogonality_tolerance`; a larger overshoot reports `invalid_rotation`. The tolerance is reused here because both checks measure dimensionless departure from a valid rotation, not because the two residuals are algebraically identical.

9.7.4 Identity, small-angle, and nominal formulas

The identity has no unique axis, but its principal rotation vector is uniquely zero. The function therefore returns before attempting to normalise an axis:

```

if (theta == 0.0) {
    return RotationLogResult{
        Eigen::Vector3d::Zero(),
        RotationLogBranch::identity};
}

```

Away from the near- π region, both nonzero branches use

$${}^a\phi = \frac{\theta}{2 \sin \theta} {}^a\mathbf{u}.$$

For a resolvable small angle, the direct ratio contains two small quantities. The implementation evaluates the reviewed series

$$\frac{\theta}{2 \sin \theta} = \frac{1}{2} + \frac{\theta^2}{12} + \frac{7\theta^4}{720} + O(\theta^6),$$

while the nominal branch evaluates the direct ratio:

```
if (theta <= policy.series_angle_threshold) {
    const double theta_squared = theta * theta;
    const double theta_fourth = theta_squared * theta_squared;

    recovery_factor =
        0.5 + theta_squared / 12.0 + 7.0 * theta_fourth / 720.0;

    branch = RotationLogBranch::small_angle;
} else {
    recovery_factor = theta / (2.0 * sine_theta);
    branch = RotationLogBranch::nominal;
}

const Eigen::Vector3d rotation_vector =
    recovery_factor * skew_difference;
```

In this block, `theta_squared` and `theta_fourth` are θ^2 and θ^4 . The scalar `recovery_factor` is the factor $\theta/(2 \sin \theta)$, evaluated either by its series or by the direct formula. Multiplying it by `skew_difference`, which is ${}^a\mathbf{u}$, produces `rotation_vector`, which is ${}^a\boldsymbol{\phi}$. The small-angle branch preserves a representable nonzero rotation; only the exact identity returns zero. The nominal branch is used only when the direct denominator is separated from both zero-angle and near- π numerical difficulties.

9.7.5 Recovering the axis near π

As $\theta \rightarrow \pi$, `skew_difference` approaches zero and no longer supplies a reliable axis magnitude. The implementation instead constructs the symmetric outer product derived in Equation 8.35:

$$\mathbf{Q} = \frac{\frac{1}{2}(\mathbf{R} + \mathbf{R}^T) - \cos \theta \mathbf{I}_3}{1 - \cos \theta} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^T.$$

The code first constructs that matrix:

```
const double denominator = 1.0 - cosine_theta;

const Eigen::Matrix3d symmetric_part =
```

```

0.5 * (matrix_ + matrix_.transpose());

const Eigen::Matrix3d axis_outer_product =
    (symmetric_part - cosine_theta * Eigen::Matrix3d::Identity()) /
    denominator;

```

Here `denominator` is $1 - \cos \theta$, `symmetric_part` is $(\mathbf{R} + \mathbf{R}^T)/2$, and `axis_outer_product` is $\mathbf{Q} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^T$. The largest diagonal entry $Q_{kk} = ({}^a s_{\omega,k})^2$ identifies the safest component from which to recover the axis. A strict comparison retains the lowest index when two diagonal entries are equal:

```

Eigen::Index dominant_index = 0;

for (Eigen::Index index = 1; index < 3; ++index) {
    if (axis_outer_product(index, index) >
        axis_outer_product(dominant_index, dominant_index)) {
        dominant_index = index;
    }
}

const double dominant_component = std::sqrt(
    axis_outer_product(dominant_index, dominant_index));

Eigen::Vector3d axis =
    axis_outer_product.col(dominant_index) / dominant_component;

axis(dominant_index) = dominant_component;
axis /= axis.stableNorm();

```

In this block, `dominant_index` is the mathematical index k , `dominant_component` is the selected positive value $\sqrt{Q_{kk}}$, and `axis` is the recovered unit-axis candidate ${}^a\tilde{\mathbf{s}}_\omega$. The positive square root makes the selected dominant component positive, but that convention does not necessarily recover the orientation represented by the original rotation. The matrix $\mathbf{Q}$ determines only an unoriented axis line because

$$\mathbf{Q} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^T = (-{}^a\mathbf{s}_\omega)(-{}^a\mathbf{s}_\omega)^T.$$

Consequently, the candidate ${}^a\tilde{\mathbf{s}}_\omega$ can equal either the desired oriented axis ${}^a\mathbf{s}_\omega$ or its negative. For $0 < \theta < \pi$, the skew column retains the missing orientation because ${}^a\mathbf{u} = 2 \sin \theta {}^a\mathbf{s}_\omega$ and $\sin \theta > 0$. Because ${}^a\mathbf{s}_\omega$ is a unit column, the correctly oriented axis gives

$$({}^a\mathbf{s}_\omega)^T {}^a\mathbf{u} = ({}^a\mathbf{s}_\omega)^T (2 \sin \theta {}^a\mathbf{s}_\omega) = 2 \sin \theta > 0.$$

The candidate has only two possible orientations. Therefore, if its computed dot product with ${}^a\mathbf{u}$ is negative, the candidate must be the oppositely oriented axis, because

$$(-{}^a\mathbf{s}_\omega)^\top (2 \sin \theta {}^a\mathbf{s}_\omega) = -2 \sin \theta < 0.$$

The code detects this case and corrects the orientation directly:

```
const double skew_alignment = axis.dot(skew_difference);

if (skew_alignment < 0.0) {
    axis = -axis;
}

const Eigen::Vector3d rotation_vector = theta * axis;
```

Here `axis` is the candidate ${}^a\tilde{\mathbf{s}}_\omega$, `skew_difference` is ${}^a\mathbf{u}$, and `skew_alignment` is their dot product. If `skew_alignment` is negative, `axis = -axis` changes the candidate to the correct orientation; otherwise the candidate is retained. At exactly $\theta = \pi$, ${}^a\mathbf{u} = \mathbf{0}_3$, so the rotation matrix contains no information that distinguishes the two axis orientations; the strict test `< 0.0` leaves the deterministic positive-dominant-component convention unchanged. After this decision, `rotation_vector` implements ${}^a\phi = \theta {}^a\mathbf{s}_\omega$.

9.7.6 Concrete half-turn and principal-branch examples

Consider the half-turn vector

$${}^a\phi = \frac{\pi}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

Because ${}^a\phi = \theta {}^a\mathbf{s}_\omega$ and $\theta = \pi$, its unit axis is ${}^a\mathbf{s}_\omega = [1 \ 1 \ 0]^\top / \sqrt{2}$. Consequently,

$$\mathbf{Q} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 1/2 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

At $\theta = \pi$, Rodrigues' formula reduces to $\mathbf{R} = 2\mathbf{Q} - \mathbf{I}_3$, so

$$\mathbf{R} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

The first two diagonal entries of $\mathbf{Q}$ are tied. The lowest-index rule selects the x_a component, and the positive-root rule returns the displayed rotation vector deterministically.

The principal convention also changes a non-principal input when the same rotation is reconstructed through the exponential. A rotation of $3\pi/2$ about $+\mathbf{e}_z$ has the same matrix as a principal rotation whose nonnegative angle is $\pi/2$ and whose oriented axis is $-\mathbf{e}_z$:

$$\exp\left(\left[\frac{3\pi}{2}\mathbf{e}_z\right]_{\times}\right) = \exp\left(\left[\frac{\pi}{2}(-\mathbf{e}_z)\right]_{\times}\right).$$

The logarithm therefore returns ${}^a\phi = (\pi/2)(-\mathbf{e}_z)$, with $\theta = \pi/2$ and ${}^a\mathbf{s}_\omega = -\mathbf{e}_z$, rather than the original $(3\pi/2)\mathbf{e}_z$ input.

9.7.7 Validation and failure behaviour

The function validates `orthogonality_tolerance`, `series_angle_threshold`, and `near_pi_tolerance` before using the matrix. Each must be finite, their lower bounds must hold, `near_pi_tolerance` must be less than π , and the small-angle threshold must lie below the start of the near- π region. An invalid combination reports `GeometryError::invalid_policy`.

Every derived scalar, matrix, axis, norm, and output vector is checked for finiteness. A non-finite intermediate reports `unsupported_magnitude`. A materially invalid cosine, negative recovered axis square, zero dominant component, or zero recovered axis norm reports `invalid_rotation`. The stored matrix has already passed `Rotation3::from_matrix()` or was constructed by a trusted `Rotation3` operation, so the logarithm does not repeat the complete $SO(3)$ validation.

9.7.8 Verification properties

The logarithm tests are organised by the geometric or numerical behaviour that could fail. The identity checks the exact-zero convention. An analytic quarter turn checks the ordinary formula independently of the exponential implementation. A small nonzero rotation checks the series branch. Coordinate-axis, tied-axis, and just-below- π cases check symmetric axis recovery, deterministic tie-breaking, and sign recovery. A $3\pi/2$ input about $+\mathbf{e}_z$ checks selection of the principal angle $\pi/2$ about $-\mathbf{e}_z$. An arbitrary-axis case checks the exponential-logarithm inverse relationship, while an overlapping-threshold case checks rejection of an invalid numerical policy.

For every validated rotation, re-exponentiating the selected principal logarithm must recover the original matrix:

$$\text{Exp}_{SO(3)}\left(\text{Log}_{SO(3)}^{\text{principal}}(\mathbf{R})\right) = \mathbf{R}$$

For a rotation vector already inside the selected principal domain, including the canonical sign convention at $\theta = \pi$, the reverse composition must recover that vector:

$$\text{Log}_{SO(3)}^{\text{principal}}\left(\text{Exp}_{SO(3)}({}^a\phi)\right) = {}^a\phi.$$

The first identity is the general endpoint contract. The second is deliberately narrower because exponential coordinates outside the principal domain, and the unselected sign at exactly π , cannot be recovered unchanged by a principal logarithm. Mutable pass counts and package-linter results are omitted from this lesson.

9.7.9 Deliberate limits

This operation recovers only the rotational block of a logarithm. A complete $SE(3)$ logarithm must also recover the finite linear exponential-coordinate block from the transformation's translation. The selected principal rotation vector is discontinuous across the exact- π branch boundary even though the represented orientation changes continuously. [Rotation3](#) does not encode frame labels or angle units, so callers must preserve the frame interpretation and treat the returned components as radians.

9.8 Algorithm summaries for the $SO(3)$ exponential and principal logarithm

This section condenses the two preceding implementations into mathematical pseudocode. Each line performs one algorithmic assignment or decision using the mathematical quantity itself. A monospaced name in square brackets gives the corresponding C++ identifier; for example, θ [theta] means that the mathematical angle θ is stored in the C++ variable [theta](#).

9.8.1 Exponential: rotation vector to rotation matrix

The exponential receives ${}^a\phi \in \mathbb{R}^3$ in radians and returns a [Rotation3](#) storing $\mathbf{R} \in SO(3)$. The complete algorithm is

```

input:     ${}^a\phi \in \mathbb{R}^3$  rad [rotation_vector],  policy,
output:   $\mathbf{R} \in SO(3)$  [rotation_matrix],
 $\theta$  [theta]  $\leftarrow \|{}^a\phi\|_2$ ,
if  $\theta = 0$ , return  $\mathbf{I}_3$ .
```

For $\theta > 0$, the algorithm continues with the coefficient selection and Rodrigues' formula:

$$\begin{aligned}
 A_{\text{eval}}(\theta) [\text{coefficient_a}] &\leftarrow \begin{cases} 1 - \frac{\theta^2}{6} + \frac{\theta^4}{120} - \frac{\theta^6}{5040}, & 0 < \theta \leq \theta_{\text{series}}, \\ \frac{\sin \theta}{\theta}, & \theta > \theta_{\text{series}}, \end{cases} \\
 B_{\text{eval}}(\theta) [\text{coefficient_b}] &\leftarrow \begin{cases} \frac{1}{2} - \frac{\theta^2}{24} + \frac{\theta^4}{720} - \frac{\theta^6}{40320}, & 0 < \theta \leq \theta_{\text{series}}, \\ \frac{1 - \cos \theta}{\theta^2}, & \theta > \theta_{\text{series}}, \end{cases} \\
 \Phi [\text{rotation_generator}] &\leftarrow [{}^a\phi]_{\times}, \\
 \mathbf{R} [\text{rotation_matrix}] &\leftarrow \mathbf{I}_3 + A_{\text{eval}}(\theta)\Phi + B_{\text{eval}}(\theta)\Phi^2, \\
 \text{return} &\text{Rotation3}(\mathbf{R}).
 \end{aligned}$$

The two polynomial branches are the evaluated sixth-order approximations used by the code. The direct branches evaluate the exact trigonometric coefficients.

9.8.2 Principal logarithm: rotation matrix to rotation vector

The logarithm receives the validated $\mathbf{R} \in SO(3)$ stored in `matrix_` and returns the selected ${}^a\phi$ in radians together with the numerical branch. Its complete algorithm is

$$\begin{aligned}
 \text{input:} & \quad \mathbf{R} \in SO(3) [\text{matrix_}], \quad \text{policy}, \\
 \text{output:} & \quad {}^a\phi \in \mathbb{R}^3 \text{ rad} [\text{rotation_vector}], \quad b [\text{branch}], \\
 c_\theta [\text{cosine_theta}] & \leftarrow \frac{\text{tr}(\mathbf{R}) - 1}{2}, \\
 {}^a\mathbf{u} [\text{skew_difference}] & \leftarrow \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}, \\
 s_\theta [\text{sine_theta}] & \leftarrow \frac{1}{2} \|{}^a\mathbf{u}\|_2, \\
 \theta [\text{theta}] & \leftarrow \text{atan2}(s_\theta, c_\theta) \in [0, \pi], \\
 \text{if } \theta = 0, & \quad \text{return } (\mathbf{0}_3, \text{identity}).
 \end{aligned}$$

For a non-identity rotation, calculate the representable near- π boundary once and compare the recovered angle directly with that boundary. If the condition holds, begin the symmetric axis recovery:

$\theta_{\pi,b}$ [near_pi_boundary]	$\leftarrow \pi - \theta_{\text{near-}\pi},$
if $\theta \geq \theta_{\pi,b},$	enter the near-π branch:
$\mathbf{R}_{\text{sym}}$ [symmetric_part]	$\leftarrow \frac{1}{2}(\mathbf{R} + \mathbf{R}^\top),$
d_π [denominator]	$\leftarrow 1 - c_\theta,$
$\mathbf{Q}$ [axis_outer_product]	$\leftarrow \frac{\mathbf{R}_{\text{sym}} - c_\theta \mathbf{I}_3}{d_\pi} = {}^a\mathbf{s}_\omega ({}^a\mathbf{s}_\omega)^\top,$
k	$\leftarrow \min \left(\arg \max_{i \in \{1,2,3\}} Q_{ii} \right),$
k_{Eigen} [dominant_index]	$\leftarrow k - 1$ for Eigen's zero-based index,
q_k [dominant_component_squared]	$\leftarrow Q_{kk},$
d_k [dominant_component]	$\leftarrow \sqrt{q_k}.$

With the dominant component available, complete the near- π axis, sign, and output calculation:

${}^a\tilde{\mathbf{s}}_\omega$ [axis]	$\leftarrow \frac{\mathbf{Q}_{:,k}}{d_k}, \quad {}^a\tilde{s}_{\omega,k} \leftarrow d_k,$
n_s [axis_norm]	$\leftarrow \ {}^a\tilde{\mathbf{s}}_\omega\ _2,$
${}^a\tilde{\mathbf{s}}_\omega$	$\leftarrow {}^a\tilde{\mathbf{s}}_\omega / n_s,$
α [skew_alignment]	$\leftarrow ({}^a\tilde{\mathbf{s}}_\omega)^\top {}^a\mathbf{u},$
if $\alpha < 0,$	${}^a\tilde{\mathbf{s}}_\omega \leftarrow -{}^a\tilde{\mathbf{s}}_\omega,$
${}^a\phi$ [rotation_vector]	$\leftarrow \theta {}^a\tilde{\mathbf{s}}_\omega,$
return	$({}^a\phi, \text{near_pi}).$

If the near- π condition is false, select the small-angle or nominal recovery factor and finish through ${}^a\phi = f(\theta) {}^a\mathbf{u}$:

```

if  $0 < \theta \leq \theta_{\text{series}}$  :
     $f(\theta)$  [recovery_factor]  $\leftarrow \frac{1}{2} + \frac{\theta^2}{12} + \frac{7\theta^4}{720}$ ,
     $b$  [branch]  $\leftarrow$  small_angle,
else :
     $\theta_{\text{series}} < \theta < \pi - \theta_{\text{near-}\pi}$ ,
     $f(\theta)$  [recovery_factor]  $\leftarrow \frac{\theta}{2s_\theta}$ ,
     $b$  [branch]  $\leftarrow$  nominal,
     ${}^a\phi$  [rotation_vector]  $\leftarrow f(\theta){}^a\mathbf{u}$ ,
return  $({}^a\phi, b)$ .

```

9.8.3 Production checks omitted from the mathematical pseudocode

The displayed algorithms assume finite inputs, valid policy values, and a validated $\mathbf{R} \in SO(3)$. The C++ boundary adds the required guards without changing the mathematical branch equations: it validates the policy and input components, enforces the supported angle, rejects non-finite intermediate or output values, clamps only tolerated round-off in c_θ and Q_{kk} , and rejects an unrecoverable zero axis or nominal sine factor. The detailed exponential and logarithm sections above record the corresponding [GeometryError](#) results.

The two algorithms form inverse operations only after the logarithm's principal-domain convention is applied. For every validated $\mathbf{R}$,

$$\text{Exp}_{SO(3)}\left(\text{Log}_{SO(3)}^{\text{principal}}(\mathbf{R})\right) = \mathbf{R},$$

whereas $\text{Log}_{SO(3)}^{\text{principal}}(\text{Exp}_{SO(3)}({}^a\phi)) = {}^a\phi$ requires ${}^a\phi$ to lie inside the selected principal domain, including the canonical sign at $\theta = \pi$.

9.9 Why the kernel uses modelled motion coordinates

Why does the kernel use screw, twist, and exponential coordinates when a robot ultimately reports point coordinates ${}^a\mathbf{x}$, translation ${}^a\mathbf{p}$, and rotation $\mathbf{R}$? These quantities answer different questions. Point and pose coordinates describe a configuration: where a point or rigid body is. A screw describes the geometry of an allowed one-parameter motion, a twist describes the instantaneous rate of rigid motion, and exponential coordinates describe one finite motion increment. Sensor observations may be used to estimate ${}^a\mathbf{x}$, ${}^a\mathbf{p}$, or $\mathbf{R}$, while

the screw, twist, and exponential coordinates remain internal to a kinematic model, estimator, controller, or optimiser.

The full theory is derived in Section 8.2, Section 8.3, and Section 8.4. This section recalls the exact definitions needed to understand what the implementation receives and why those inputs are not normally independent sensor measurements.

9.9.1 A normalised screw describes one motion direction

Consider a rotational or helical motion whose geometry is known from design data or calibration. Let ${}^a\mathbf{s}_\omega \in \mathbb{R}^3$ be the unit direction of its oriented screw axis, let ${}^a\mathbf{p}_\ell \in \mathbb{R}^3$ be the position of the closest point on that axis relative to the reference origin O_a , and let $h \in \mathbb{R}$ be the signed distance translated along the axis per radian of rotation. Both vectors are expressed in frame $\{a\}$, ${}^a\mathbf{p}_\ell$ is measured in metres, and h is measured in metres per radian.

These geometric quantities define the linear component of the normalised screw generator:

$${}^a\mathbf{s}_v = -{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell + h{}^a\mathbf{s}_\omega.$$

The component ${}^a\mathbf{s}_v \in \mathbb{R}^3$ is measured in metres per radian. It is not a translation axis or a finite translation. Its perpendicular term $-{}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell$ records the axis offset from O_a , while its parallel term $h{}^a\mathbf{s}_\omega$ records the pitch. Placing the linear and angular components in the project's linear-first order gives the normalised screw

$${}^a\mathbf{S} = \begin{bmatrix} {}^a\mathbf{s}_v \\ {}^a\mathbf{s}_\omega \end{bmatrix}.$$

For example, a revolute or helical joint stores ${}^a\mathbf{S}$ as part of its kinematic model, while an encoder supplies the amount of motion along that fixed direction. A robot with no such one-degree-of-freedom constraint does not need to invent a normalised screw for every operation.

9.9.2 A twist describes the instantaneous motion

Let the signed rotation about the screw axis vary with time as $\theta(t)$, measured in radians. Its derivative $\dot{\theta}$ is the angular rate in radians per second. Multiplying the normalised screw by this rate gives the linear-first spatial twist

$$\begin{bmatrix} {}^a\boldsymbol{\nu} \\ {}^a\boldsymbol{\omega} \end{bmatrix} = {}^a\mathbf{S}\dot{\theta} = \begin{bmatrix} {}^a\mathbf{s}_v\dot{\theta} \\ {}^a\mathbf{s}_\omega\dot{\theta} \end{bmatrix}.$$

The angular component ${}^a\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity in radians per second. The linear component ${}^a\boldsymbol{\nu} \in \mathbb{R}^3$ is measured in metres per second. It is the value of the screw's instantaneous velocity field at O_a , not generally the velocity of a moving link-frame origin. Together, the two components determine the velocity assigned to a point at ${}^a\mathbf{x}$:

$${}^a\dot{\mathbf{x}} = {}^a\boldsymbol{\nu} + {}^a\boldsymbol{\omega} \times {}^a\mathbf{x}.$$

The linear component ${}^a\boldsymbol{\nu}$ therefore need not be measured at the possibly non-material reference origin O_a . The twist can instead be calculated from a known screw and joint rate or inferred from inertial, visual, and material-point observations. The velocity-field equation then predicts the velocity at every other point on the rigid body. The current pose $({}^a\mathbf{p}, \mathbf{R})$ alone does not contain this rate information.

9.9.3 Exponential coordinates describe one finite motion increment

Following the same rotational or helical screw through the signed finite angle θ gives the linear-first exponential-coordinate column

$${}^a\boldsymbol{\eta} = {}^a\mathbf{s}\theta = \begin{bmatrix} {}^a\boldsymbol{\rho} \\ {}^a\boldsymbol{\phi} \end{bmatrix} = \begin{bmatrix} {}^a\mathbf{s}_v\theta \\ {}^a\mathbf{s}_\omega\theta \end{bmatrix}.$$

The angular coordinate ${}^a\boldsymbol{\phi} = {}^a\mathbf{s}_\omega\theta$ is measured in radians. Its direction gives the oriented screw-axis direction, and its magnitude gives the rotation amount when the equivalent orientation with $\theta = \|{}^a\boldsymbol{\phi}\|_2 > 0$ is selected. The linear coordinate ${}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v\theta$ is measured in metres. Substitution of the screw geometry shows the information it carries:

$${}^a\boldsymbol{\rho} = -({}^a\mathbf{s}_\omega \times {}^a\mathbf{p}_\ell)\theta + h\theta {}^a\mathbf{s}_\omega.$$

The first term records the scaled axis offset, and the second records the axial advance. Thus ${}^a\boldsymbol{\rho}$ is a finite linear generator coordinate, not the final translation of the rigid transformation.

When ${}^a\boldsymbol{\phi} \neq \mathbf{0}_3$, the selected finite coordinates retain the screw geometry. Choose the equivalent screw orientation for which $\theta = \|{}^a\boldsymbol{\phi}\|_2 > 0$, then recover

$$\theta = \|{}^a\boldsymbol{\phi}\|_2, \quad {}^a\mathbf{s}_\omega = \frac{{}^a\boldsymbol{\phi}}{\theta}, \quad {}^a\mathbf{s}_v = \frac{{}^a\boldsymbol{\rho}}{\theta},$$

followed by

$$h = ({}^a\mathbf{s}_\omega)^\top {}^a\mathbf{s}_v, \quad {}^a\mathbf{p}_\ell = {}^a\mathbf{s}_\omega \times {}^a\mathbf{s}_v.$$

The dot product extracts the component of ${}^a\mathbf{s}_v$ parallel to the axis, which is the pitch h . The cross product removes that parallel component and recovers ${}^a\mathbf{p}_\ell$, the closest point on the axis. This recovery confirms that ${}^a\phi$ carries the axis direction and rotation amount, while ${}^a\rho$ carries the information needed to locate the axis and determine its pitch.

Pure translation uses a separate normalisation because there is no rotation angle. Let ${}^a\mathbf{s}_v$ now be a unit translation direction, let d be the signed translation distance in metres, and set ${}^a\mathbf{s}_\omega = \mathbf{0}_3$. The finite coordinates are then

$${}^a\boldsymbol{\eta} = \begin{bmatrix} {}^a\mathbf{s}_v d \\ \mathbf{0}_3 \end{bmatrix}, \quad {}^a\rho = {}^a\mathbf{s}_v d, \quad {}^a\phi = \mathbf{0}_3.$$

For a rotational or helical motion, the three model representations can now be compared without confusing their units or purposes:

Representation	Linear block	Angular block	Question answered
Normalised screw	${}^a\mathbf{s}_v$ in metres per radian	${}^a\mathbf{s}_\omega$ as a unit direction	What fixed screw geometry defines the motion?
Spatial twist	${}^a\boldsymbol{\nu} = {}^a\mathbf{s}_v \dot{\theta}$ in metres per second	${}^a\boldsymbol{\omega} = {}^a\mathbf{s}_\omega \dot{\theta}$ in radians per second	How is the rigid body moving now?
Finite exponential coordinates	${}^a\rho = {}^a\mathbf{s}_v \theta$ in metres	${}^a\phi = {}^a\mathbf{s}_\omega \theta$ in radians	What finite motion increment will be exponentiated?

These finite coordinates are normally modelled or derived rather than measured independently. A kinematic model supplies ${}^a\mathbf{S}$ and a joint measurement supplies θ or d . If a spatial twist expressed in frame $\{a\}$ remains constant over a time step Δt , its blocks give ${}^a\rho = {}^a\boldsymbol{\nu}\Delta t$ and ${}^a\phi = {}^a\boldsymbol{\omega}\Delta t$. An optimiser may instead choose the six coordinates directly as a local pose increment.

Finite exponential coordinates are useful because directly adding entries to $\mathbf{R}$ does not preserve $\mathbf{R}^T\mathbf{R} = \mathbf{I}_3$ and $\det(\mathbf{R}) = 1$. Exponentiation maps the six-component increment to a valid rigid transformation that can be composed with the current pose. The external state may still be stored and reported as $({}^a\mathbf{p}, \mathbf{R})$; the exponential coordinates are needed only while representing the increment. The inverse problem—recovering a selected increment from a finite pose—belongs to the principal $SE(3)$ logarithm cycle.

9.10 Linear-first $SE(3)$ exponential from finite coordinates

Using the finite coordinates defined in Section 9.9, the $SE(3)$ exponential answers the forward finite-motion question: what rigid displacement does the supplied six-component

column generate? The implementation uses the linear-first input

$${}^a\boldsymbol{\eta} = \begin{bmatrix} {}^a\boldsymbol{\rho} \\ {}^a\boldsymbol{\phi} \end{bmatrix}.$$

Both blocks are expressed in frame $\{a\}$. The linear block ${}^a\boldsymbol{\rho} \in \mathbb{R}^3$ is measured in metres, and the angular block ${}^a\boldsymbol{\phi} \in \mathbb{R}^3$ is measured in radians. They are generator coordinates: they describe the motion before exponentiation. The output is the rigid displacement

$$\mathbf{T}_\Delta = \exp(\widehat{{}^a\boldsymbol{\eta}}) = \begin{bmatrix} \mathbf{R}({}^a\boldsymbol{\phi}) & {}^a\mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \mathbf{R}({}^a\boldsymbol{\phi}) = \exp([{}^a\boldsymbol{\phi}]_\times), \quad \boxed{{}^a\mathbf{p} = \mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho}}.$$

This mapping separates two pairs of quantities that are easy to confuse. The input block ${}^a\boldsymbol{\phi}$ generates the final rotation $\mathbf{R}$, while the input block ${}^a\boldsymbol{\rho}$ contributes to the final translation ${}^a\mathbf{p}$. In general, ${}^a\boldsymbol{\rho} \neq {}^a\mathbf{p}$: rotation couples the linear and angular generators through the $SO(3)$ left Jacobian $\mathbf{J}({}^a\boldsymbol{\phi})$. The remaining question is why exponentiation applies this Jacobian instead of copying ${}^a\boldsymbol{\rho}$ directly into the transform.

9.10.1 Why the left Jacobian produces the translation

Let $\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_\times \in \mathfrak{so}(3)$. The linear-first hat map places $\boldsymbol{\Phi}$ and ${}^a\boldsymbol{\rho}$ in one $\mathfrak{se}(3)$ matrix:

$$\widehat{{}^a\boldsymbol{\eta}} = \begin{bmatrix} \boldsymbol{\Phi} & {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

For every integer $k \geq 1$, block multiplication gives

$$(\widehat{{}^a\boldsymbol{\eta}})^k = \begin{bmatrix} \boldsymbol{\Phi}^k & \boldsymbol{\Phi}^{k-1}{}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

Substitution into the matrix-exponential series separates the rotation block from the translation column:

$$\begin{aligned} \exp(\widehat{{}^a\boldsymbol{\eta}}) &= \begin{bmatrix} \sum_{k=0}^{\infty} \frac{\boldsymbol{\Phi}^k}{k!} & \sum_{k=1}^{\infty} \frac{\boldsymbol{\Phi}^{k-1}}{k!} {}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \exp(\boldsymbol{\Phi}) & \mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \end{aligned}$$

where the $SO(3)$ left Jacobian is the dimensionless mapping

$$\mathbf{J} : \mathbb{R}^3 \rightarrow \mathbb{R}^{3 \times 3}, \quad \mathbf{J}({}^a\phi) = \sum_{j=0}^{\infty} \frac{\Phi^j}{(j+1)!}.$$

With $\theta = \|{}^a\phi\|_2$ and $\Phi^3 = -\theta^2\Phi$, the powers in this series reduce to $\mathbf{I}_3$, Φ , and Φ^2 . Grouping their coefficients gives the closed form derived in Section 8.5:

$$\boxed{\mathbf{J}({}^a\phi) = \mathbf{I}_3 + B(\theta)\Phi + C(\theta)\Phi^2}, \quad B(\theta) = \frac{1 - \cos \theta}{\theta^2}, \quad C(\theta) = \frac{\theta - \sin \theta}{\theta^3}.$$

The direct quotients are unsuitable near $\theta = 0$ because their numerators subtract nearly equal floating-point values. Their sixth-order series are

$$\begin{aligned} B(\theta) &= \frac{1}{2} - \frac{\theta^2}{24} + \frac{\theta^4}{720} - \frac{\theta^6}{40320} + O(\theta^8), \\ C(\theta) &= \frac{1}{6} - \frac{\theta^2}{120} + \frac{\theta^4}{5040} - \frac{\theta^6}{362880} + O(\theta^8). \end{aligned}$$

At ${}^a\phi = \mathbf{0}_3$, the angle is $\theta = 0$ and the cross-product matrix is $\Phi = [\mathbf{0}_3]_{\times} = \mathbf{0}_{3 \times 3}$. The coefficient limits $B(0) = 1/2$ and $C(0) = 1/6$ are finite, so substitution into the closed form gives

$$\begin{aligned} \mathbf{J}(\mathbf{0}_3) &= \mathbf{I}_3 + B(0)\mathbf{0}_{3 \times 3} + C(0)\mathbf{0}_{3 \times 3}^2 \\ &= \mathbf{I}_3. \end{aligned}$$

The implementation returns this exact zero-angle result directly, so pure translation satisfies ${}^a\mathbf{p} = {}^a\boldsymbol{\rho}$ without a trigonometric calculation.

9.10.2 Public factory and complete implementation

The public factory accepts one linear-first coordinate column and returns a validated `Transform3` value:

```
// Exponentiate finite linear-first coordinates [rho; phi].
// rho is measured in metres and phi in radians.
static Transform3 from_exponential_coordinates(
    const Vector6LinearFirst & coordinates,
    const NumericalPolicy & policy = NumericalPolicy{});
```

The mathematical path through the function is:

1. Split the linear-first input ${}^a\boldsymbol{\eta} = [({}^a\boldsymbol{\rho})^\top \quad ({}^a\boldsymbol{\phi})^\top]^\top$. In the code, `coordinates.head<3>()` becomes `linear_coordinates` ${}^a\boldsymbol{\rho}$ in metres, and `coordinates.tail<3>()` becomes `rotation_vector` ${}^a\boldsymbol{\phi}$ in radians.
2. Reuse the $SO(3)$ exponential to construct `rotation` $\mathbf{R} = \exp([{}^a\boldsymbol{\phi}]_\times)$, then calculate `theta` $\theta = \|{}^a\boldsymbol{\phi}\|_2$ for the translation calculation.
3. If $\theta = 0$, use $\mathbf{J}(\mathbf{0}_3) = \mathbf{I}_3$ and return $(\mathbf{R}, {}^a\boldsymbol{\rho}) = (\mathbf{I}_3, {}^a\boldsymbol{\rho})$ exactly. This is the pure-translation branch.
4. If $\theta > 0$, calculate `coefficient_b` $B(\theta)$ and `coefficient_c` $C(\theta)$. The small-angle branch uses their series, while the remaining supported angles use their direct trigonometric formulas.
5. Construct `rotation_generator` $\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_\times$ and then construct `left_jacobian` from $\mathbf{J} = \mathbf{I}_3 + B(\theta)\boldsymbol{\Phi} + C(\theta)\boldsymbol{\Phi}^2$.
6. Calculate `translation` from ${}^a\mathbf{p} = \mathbf{J}({}^a\boldsymbol{\phi}){}^a\boldsymbol{\rho}$. This is the step that converts the finite linear generator coordinate ${}^a\boldsymbol{\rho}$ into the final translation column ${}^a\mathbf{p}$.
7. Return `Transform3(rotation, translation)`, which stores the pair $(\mathbf{R}, {}^a\mathbf{p})$ representing $\mathbf{T}_\Delta$.

The complete implementation keeps these stages in the same order:

```
Transform3 Transform3::from_exponential_coordinates(
    const Vector6LinearFirst & coordinates, const NumericalPolicy & policy)
{
    if (!coordinates.allFinite()) {
        throw GeometryException(
            GeometryError::non_finite,
            "SE(3) exponential coordinates must be finite.");
    }

    // eta = [rho; phi], with rho in metres and phi in radians.
    const Eigen::Vector3d linear_coordinates = coordinates.head<3>();
    const Eigen::Vector3d rotation_vector = coordinates.tail<3>();

    // R = exp([phi]_x). This also validates the angular policy and magnitude.
    const Rotation3 rotation =
        Rotation3::from_rotation_vector(rotation_vector, policy);

    const double theta = rotation_vector.stableNorm();

    if (!std::isfinite(theta)) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "Rotation-vector norm is outside the finite numerical range.");
    }
}
```

```

}

// When phi = 0, J(phi) = I and p = rho exactly.
if (theta == 0.0) {
    return Transform3(rotation, linear_coordinates);
}

const double theta_squared = theta * theta;

double coefficient_b;
double coefficient_c;

if (theta <= policy.series_angle_threshold) {
    const double theta_fourth = theta_squared * theta_squared;
    const double theta_sixth = theta_fourth * theta_squared;

    coefficient_b =
        0.5 - theta_squared / 24.0 + theta_fourth / 720.0 -
        theta_sixth / 40320.0;

    coefficient_c =
        1.0 / 6.0 - theta_squared / 120.0 +
        theta_fourth / 5040.0 - theta_sixth / 362880.0;
} else {
    coefficient_b = (1.0 - std::cos(theta)) / theta_squared;
    coefficient_c =
        (theta - std::sin(theta)) / (theta_squared * theta);
}

if (!std::isfinite(coefficient_b) ||
    !std::isfinite(coefficient_c)) {
    throw GeometryException(
        GeometryError::unsupported_magnitude,
        "SE(3) left-Jacobian coefficients are not finite.");
}

// Phi = [phi]_x.
const Eigen::Matrix3d rotation_generator = hat_so3(rotation_vector);

// J(phi) = I + B(theta) Phi + C(theta) Phi^2.
const Eigen::Matrix3d left_jacobian =
    Eigen::Matrix3d::Identity() + coefficient_b * rotation_generator +
    coefficient_c * rotation_generator * rotation_generator;

```

```

    if (!left_jacobian.allFinite()) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "SE(3) left Jacobian is outside the finite numerical range.");
    }

    // p = J(phi) rho.
    const Eigen::Vector3d translation =
        left_jacobian * linear_coordinates;

    if (!translation.allFinite()) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "SE(3) exponential translation is outside the finite numerical range.");
    }

    return Transform3(rotation, translation);
}

```

The line breaks in this presentation keep the code readable in both HTML and PDF. They do not change the operations performed by the production function.

9.10.3 Equation-to-code map

Mathematics	Code	Meaning
${}^a\boldsymbol{\eta} \in \mathbb{R}^6$	<code>coordinates</code>	Linear-first exponential-coordinate column
${}^a\boldsymbol{\rho} \in \mathbb{R}^3$	<code>linear_coordinates</code>	First three entries of <code>coordinates</code> , in metres
${}^a\boldsymbol{\phi} \in \mathbb{R}^3$	<code>rotation_vector</code>	Last three entries of <code>coordinates</code> , in radians
$\mathbf{R} = \exp([{}^a\boldsymbol{\phi}]_{\times})$	<code>rotation</code>	Resulting proper rotation
$\theta = \ {}^a\boldsymbol{\phi}\ _2$	<code>theta</code>	Rotation angle, in radians
$\theta^2, \theta^4, \theta^6$	<code>theta_squared,</code> <code>theta_fourth,</code> <code>theta_sixth</code>	Reused even powers of the angle
$B(\theta)$	<code>coefficient_b</code>	Coefficient multiplying $\boldsymbol{\Phi}$
$C(\theta)$	<code>coefficient_c</code>	Coefficient multiplying $\boldsymbol{\Phi}^2$
$\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_{\times}$	<code>rotation_generator</code>	Skew-symmetric rotation generator

Mathematics	Code	Meaning
$\mathbf{J}({}^a\phi)$	<code>left_jacobian</code>	Mapping from linear exponential coordinates to the translation column
${}^a\mathbf{p} = \mathbf{J}^a\boldsymbol{\rho}$	<code>translation</code>	Translation column of the resulting transform, in metres

Eigen's `head<3>()` selects the first three entries of the fixed-size six-vector, and `tail<3>()` selects the last three. The number `3` is a compile-time template argument between angle brackets, so these expressions preserve the project's declared $[\boldsymbol{\rho}; \phi]$ component order without allocating dynamic vectors.

The call to `Rotation3::from_rotation_vector` reuses the completed $SO(3)$ exponential for $\mathbf{R}$. It also owns validation of the angular numerical policy and the supported rotation-vector magnitude. The $SE(3)$ factory then uses the same `series_angle_threshold` for the two left-Jacobian coefficients, so the rotation and translation blocks switch numerical regimes at the same angle.

9.10.4 Finite-pitch screw example

Consider the normalised screw

$${}^a\mathbf{s}_v = [0 \quad -2 \quad 0.05]^\top \text{ m rad}^{-1}, \quad {}^a\mathbf{s}_\omega = [0 \quad 0 \quad 1]^\top,$$

followed through $\sigma = \theta = \pi/2$ rad. Its finite exponential coordinates are

$${}^a\boldsymbol{\rho} = {}^a\mathbf{s}_v\theta = \begin{bmatrix} 0 \\ -\pi \\ \pi/40 \end{bmatrix} \text{ m}, \quad {}^a\phi = {}^a\mathbf{s}_\omega\theta = \begin{bmatrix} 0 \\ 0 \\ \pi/2 \end{bmatrix} \text{ rad}.$$

For a rotation vector $\phi = \theta\mathbf{e}_z$,

$$\boldsymbol{\Phi} = \begin{bmatrix} 0 & -\theta & 0 \\ \theta & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \boldsymbol{\Phi}^2 = \begin{bmatrix} -\theta^2 & 0 & 0 \\ 0 & -\theta^2 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Substitution into $\mathbf{J} = \mathbf{I}_3 + B\boldsymbol{\Phi} + C\boldsymbol{\Phi}^2$ gives

$$\mathbf{J}_z(\theta) = \begin{bmatrix} \frac{\sin \theta}{\theta} & -\frac{1 - \cos \theta}{\theta} & 0 \\ \frac{1 - \cos \theta}{\theta} & \frac{\sin \theta}{\theta} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

At $\theta = \pi/2$, this becomes

$$\mathbf{J}_z\left(\frac{\pi}{2}\right) = \begin{bmatrix} 2/\pi & -2/\pi & 0 \\ 2/\pi & 2/\pi & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

and therefore

$${}^a\mathbf{p} = \mathbf{J}_z\left(\frac{\pi}{2}\right) {}^a\boldsymbol{\rho} = \begin{bmatrix} 2 \\ -2 \\ \pi/40 \end{bmatrix} \text{ m.}$$

The resulting finite displacement is

$$\mathbf{T}_\Delta = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & -2 \\ 0 & 0 & 1 & \pi/40 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The corresponding calling code preserves the same linear-first order:

```
const double pi = std::numbers::pi;

Vector6LinearFirst coordinates;
coordinates <<
    0.0, -pi, pi / 40.0, // rho in metres
    0.0, 0.0, pi / 2.0; // phi in radians

const Transform3 transform =
    Transform3::from_exponential_coordinates(coordinates);
```

This example makes the distinction between ${}^a\boldsymbol{\rho}$ and ${}^a\mathbf{p}$ concrete: the input xy coordinates $(0, -\pi)$ m become the finite translation $(2, -2)$ m because the reference velocity field is integrated while the body rotates.

9.10.5 Validation and focused evidence

The factory accepts only finite coordinate entries. It rejects a non-finite angle norm, coefficient, Jacobian, or translation as `GeometryError::unsupported_magnitude`; the last check catches the case in which finite values of $\mathbf{J}$ and ${}^a\boldsymbol{\rho}$ produce a non-finite matrix–vector product. Invalid angular policy values and unsupported angular magnitudes are rejected by the reused `Rotation3` factory before the translation is constructed.

The focused tests establish the following mathematical and numerical properties:

Case	Property established
$\phi = \mathbf{0}_3$	Pure translation returns $\mathbf{R} = \mathbf{I}_3$ and ${}^a\mathbf{p} = {}^a\boldsymbol{\rho}$ exactly
Finite-pitch quarter turn	The implementation agrees with the independently calculated $\mathbf{J}_z(\pi/2)$ and analytic homogeneous transform
$\theta = 10^{-8}$ rad	A small nonzero screw uses the stable coefficient series and is not collapsed to pure translation
Non-finite entry in ${}^a\boldsymbol{\rho}$	The public boundary reports <code>non_finite</code>
Finite inputs whose product overflows	The public boundary reports <code>unsupported_magnitude</code> rather than returning an invalid transform

The small-angle expected translation is calculated independently from the analytic z -axis Jacobian. In particular, the test evaluates $(1 - \cos \theta)/\theta$ as $2 \sin^2(\theta/2)/\theta$, which avoids using the same cancellation-prone subtraction or the same series as the production code.

9.10.6 Deliberate limits

`Transform3::from_exponential_coordinates()` implements the forward $SE(3)$ map for the project’s fixed linear-first coordinates. The factory does not itself select a principal logarithm or factor the input into a normalised screw axis and displacement. Frames and units are documented by the API and variable names but remain the caller’s responsibility because Eigen vectors do not encode them in their C++ types. The angular branch and maximum supported angle are inherited from `Rotation3::from_rotation_vector`, and the input rotation vector is not silently wrapped or canonicalised.

9.11 Principal linear-first $SE(3)$ logarithm to finite coordinates

The $SE(3)$ exponential in Section 9.10 answers the forward question: given finite linear-first coordinates, which rigid transformation do they produce? The principal $SE(3)$ logarithm answers the inverse question needed when a finite relative transformation must be represented as one selected six-entry coordinate column. Given a validated rotation and translation, it returns the finite coordinates that reproduce the transformation through the exponential, together with the numerical branch used to recover them.

The mathematical derivation belongs to Section 8.5, especially Equation 8.27, Equation 8.29, and Equation 8.37. This section records how the public `Transform3` logarithm member implements that reviewed result. Routine point transformation, pose composition, and pose storage do not require a logarithm; callers should continue to use the validated transformation directly unless a finite coordinate representation is actually needed.

9.11.1 Input, output, frame, and units

Let the stored finite displacement, expressed relative to frame $\{a\}$, be

$$\mathbf{T}_\Delta = \begin{bmatrix} \mathbf{R} & {}^a\mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3),$$

where $\mathbf{R} \in SO(3)$ is dimensionless and ${}^a\mathbf{p} \in \mathbb{R}^3$ is the translation column expressed in frame $\{a\}$ and measured in metres. A `Transform3` already owns a validated `Rotation3` and a finite translation, so the member function does not accept an arbitrary unvalidated 4×4 array.

The output coordinate column uses the project's linear-first convention:

$${}^a\boldsymbol{\eta}_{\text{principal}} = \begin{bmatrix} {}^a\boldsymbol{\rho} \\ {}^a\boldsymbol{\phi} \end{bmatrix} \in \mathbb{R}^6,$$

where ${}^a\boldsymbol{\rho} \in \mathbb{R}^3$ is measured in metres and ${}^a\boldsymbol{\phi} \in \mathbb{R}^3$ is measured in radians. The rotation-vector norm $\theta = \|{}^a\boldsymbol{\phi}\|_2$ lies in the selected principal interval $[0, \pi]$. Both blocks are expressed in frame $\{a\}$.

The public result keeps the coordinate column and the observable numerical classification together:

```

enum class TransformLogBranch
{
    identity,
    pure_translation,
    small_angle,
    nominal,
    near_pi
};

struct TransformLogResult
{
    Vector6LinearFirst coordinates;
    TransformLogBranch branch;
};

[[nodiscard]] TransformLogResult to_principal_exponential_coordinates(
    const NumericalPolicy & policy = NumericalPolicy{ }) const;

```

An `enum class` defines a scoped set of named branch values, preventing accidental conversion of a branch into an integer. The `struct` groups the two parts of one successful result without hiding either behind accessor functions. The `const` after the member declaration promises not to modify the stored transformation, and `[[nodiscard]]` asks the compiler to warn when a caller discards the recovered result unintentionally.

9.11.2 Why the linear block is recovered rather than copied

The translation column ${}^a\mathbf{p}$ answers where the transformed origin finishes. The finite linear coordinate ${}^a\boldsymbol{\rho}$ instead parameterises the constant rigid-motion field whose exponential produces that final translation while the body rotates. They coincide only when the selected rotation is the identity.

The forward relationship derived in Chapter 8 is

$${}^a\mathbf{p} = \mathbf{J}({}^a\boldsymbol{\phi}) {}^a\boldsymbol{\rho},$$

so a non-identity logarithm must invert the left-Jacobian action:

$$\boxed{{}^a\boldsymbol{\rho} = \mathbf{J}({}^a\boldsymbol{\phi})^{-1} {}^a\mathbf{p}.$$

For $\boldsymbol{\Phi} = [{}^a\boldsymbol{\phi}]_{\times}$ and $\theta = \|{}^a\boldsymbol{\phi}\|_2 > 0$, the inverse is

$$\boxed{\mathbf{J}({}^a\boldsymbol{\phi})^{-1} = \mathbf{I}_3 - \frac{1}{2}\boldsymbol{\Phi} + D(\theta)\boldsymbol{\Phi}^2},$$

with

$$\begin{aligned} D(\theta) &= \frac{1}{\theta^2} - \frac{1}{2\theta} \cot\left(\frac{\theta}{2}\right) \\ &= \frac{1}{\theta^2} - \frac{\cos(\theta/2)}{2\theta \sin(\theta/2)}. \end{aligned}$$

The second line follows from $\cot x = \cos x / \sin x$ and matches the direct expression evaluated by the C++ implementation. On the nonzero principal interval, $0 < \theta \leq \pi$, the denominator $\sin(\theta/2)$ is nonzero. Near $\theta = \pi$, it approaches one, so this half-angle form remains suitable.

Near zero, however, the two terms in the direct expression for $D(\theta)$ are individually large and nearly cancel. The implementation therefore evaluates the reviewed series

$$D(\theta) = \frac{1}{12} + \frac{\theta^2}{720} + \frac{\theta^4}{30240} + \frac{\theta^6}{1209600} + O(\theta^8)$$

whenever $0 < \theta \leq \theta_{\text{series}}$. The series and direct expressions describe the same inverse Jacobian; the threshold selects the more stable floating-point evaluation and does not change the represented motion.

9.11.3 Identity rotation and exact translation preservation

The implementation first delegates rotation recovery to the completed principal $SO(3)$ logarithm:

```
const RotationLogResult rotation_log_result =
    rotation_.to_principal_rotation_vector(policy);

Vector6LinearFirst coordinates = Vector6LinearFirst::Zero();
coordinates.tail<3>() = rotation_log_result.rotation_vector;
```

This call owns the angular policy validation and the identity, small-angle, nominal, and deterministic near- π rotation-vector calculations described in Section 9.7. Initialising the complete six-vector to zero gives every component a defined value before either block is assigned. `tail<3>()` selects the last three entries at compile time and stores the recovered ${}^a\phi$ there.

If the selected rotation is the exact identity, then $\mathbf{J}(\mathbf{0}_3) = \mathbf{I}_3$ and ${}^a\boldsymbol{\rho} = {}^a\mathbf{p}$. The function preserves that translation directly and distinguishes a complete identity from a pure translation:

```

if (rotation_log_result.branch == RotationLogBranch::identity) {
    coordinates.head<3>() = translation_;

    const bool translation_is_zero =
        (translation_.array() == 0.0).all();

    const TransformLogBranch branch =
        translation_is_zero
        ? TransformLogBranch::identity
        : TransformLogBranch::pure_translation;

    return TransformLogResult{coordinates, branch};
}

```

The expression `translation_.array() == 0.0` performs a component-wise comparison and `.all()` requires all three comparisons to be true. This is a deliberate exact-zero contract, not an approximate geometric comparison: the identity classification requires all three stored translation components to equal floating-point zero. Every representable nonzero component, including a subnormal value, produces `pure_translation` and is copied without Jacobian arithmetic. The conditional operator `condition ? first : second` selects the branch value without changing the coordinates.

9.11.4 Nonzero rotation and the inverse left Jacobian

For a nonzero rotation, the spatial branch inherits the numerically important $SO(3)$ decision. The translation does not create a separate angular branch:

TransformLogBranch	Condition	Translation calculation
<code>small_angle</code>	$0 < \theta \leq \theta_{\text{series}}$	Evaluate $D(\theta)$ by its series, then apply $\mathbf{J}^{-1}$
<code>nominal</code>	$\theta_{\text{series}} < \theta < \pi - \theta_{\text{near-}\pi}$	Evaluate $D(\theta)$ by the direct half-angle expression
<code>near_pi</code>	$\pi - \theta_{\text{near-}\pi} \leq \theta \leq \pi$	Reuse the symmetric $SO(3)$ axis recovery, then evaluate the same direct $D(\theta)$ expression

After mapping the branch, the implementation forms the inverse Jacobian and recovers the linear block:

```

const Eigen::Vector3d phi = rotation_log_result.rotation_vector;
const double theta = phi.stableNorm();
const double theta_squared = theta * theta;

double coefficient_d;
if (theta <= policy.series_angle_threshold) {
    const double theta_fourth = theta_squared * theta_squared;
    const double theta_sixth = theta_fourth * theta_squared;

    coefficient_d = 1.0 / 12.0 + theta_squared / 720.0 +
        theta_fourth / 30240.0 +
        theta_sixth / 1209600.0;
} else {
    const double half_theta = 0.5 * theta;
    coefficient_d =
        1.0 / theta_squared -
        std::cos(half_theta) /
        (2.0 * theta * std::sin(half_theta));
}

const Eigen::Matrix3d rotation_vector_hat = hat_so3(phi);

const Eigen::Matrix3d inverse_left_jacobian =
    Eigen::Matrix3d::Identity() - 0.5 * rotation_vector_hat +
    coefficient_d * rotation_vector_hat * rotation_vector_hat;

const Eigen::Vector3d linear_coordinates =
    inverse_left_jacobian * translation_;

coordinates.head<3>() = linear_coordinates;
return TransformLogResult{coordinates, branch};

```

9.11.5 Equation-to-code map

Mathematics	Code	Meaning
$\mathbf{T}_{\Delta} = (\mathbf{R}, {}^a\mathbf{p})$	<code>rotation_, translation_</code>	Validated stored finite displacement
${}^a\phi$	<code>phi</code>	Selected principal rotation-vector block, in radians

Mathematics	Code	Meaning
$\theta = \ \mathbf{a}\phi\ _2$	<code>theta</code>	Principal rotation angle, in radians
$\theta^2, \theta^4, \theta^6$	<code>theta_squared,</code> <code>theta_fourth, theta_sixth</code>	Reused even powers for the series
$D(\theta)$	<code>coefficient_d</code>	Coefficient multiplying Φ^2 in $\mathbf{J}^{-1}$
$\Phi = [\mathbf{a}\phi]_{\times}$	<code>rotation_vector_hat</code>	Skew-symmetric matrix of the selected rotation vector
$\mathbf{J}(\mathbf{a}\phi)^{-1}$	<code>inverse_left_jacobian</code>	Dimensionless map from final translation to finite linear coordinates
$\mathbf{a}\rho = \mathbf{J}^{-1}\mathbf{a}\mathbf{p}$	<code>linear_coordinates</code>	Recovered finite linear block, in metres
$[\mathbf{a}\rho; \mathbf{a}\phi]$	<code>coordinates</code>	Returned linear-first six-column
Numerical classification	<code>branch</code>	Observable identity, translation, or angular formula region

Eigen's `stableNorm()` evaluates the Euclidean norm with internal scaling, reducing avoidable overflow or underflow in the norm calculation. The fixed-size matrices and vectors preserve the declared dimensions at compile time. Matrix multiplication remains ordered: `inverse_left_jacobian * translation_` implements $\mathbf{J}^{-1}\mathbf{a}\mathbf{p}$, not the reversed product.

9.11.6 A representable near- π boundary

The principal $SO(3)$ logarithm supplies the angular branch used by the spatial result. Its near- π interval begins at the representable boundary

$$\theta_{\pi,b} = \text{fl}(\pi - \theta_{\text{near-}\pi}),$$

where `fl` denotes rounding to the binary64 result of the subtraction. The implementation calculates this value once:

```
const double near_pi_boundary =
    std::numbers::pi - policy.near_pi_tolerance;

if (theta >= near_pi_boundary) {
    // Enter the near-pi branch.
```

```
}
```

In exact real arithmetic, testing $\theta \geq \pi - \theta_{\text{near-}\pi}$ is equivalent to testing $\pi - \theta \leq \theta_{\text{near-}\pi}$. In binary64 arithmetic, recomputing `std::numbers::pi - theta` at the boundary introduces another rounded subtraction and can produce a value slightly greater than the stored tolerance. Comparing `theta` directly with the single named boundary implements the declared inclusive interval deterministically and keeps policy validation and formula selection aligned.

9.11.7 Finite-pitch quarter-turn example

Reuse the analytic displacement constructed in Section 9.10:

$$\mathbf{T}_{\Delta} = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & -2 \\ 0 & 0 & 1 & \pi/40 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The final translation is ${}^a\mathbf{p} = [2 \ -2 \ \pi/40]^{\top}$ m, but the principal logarithm must recover

$${}^a\boldsymbol{\phi} = \begin{bmatrix} 0 \\ 0 \\ \pi/2 \end{bmatrix} \text{ rad}, \quad {}^a\boldsymbol{\rho} = \begin{bmatrix} 0 \\ -\pi \\ \pi/40 \end{bmatrix} \text{ m}.$$

The corresponding use of the public API is

```
Eigen::Matrix4d matrix;
matrix <<
    0.0, -1.0, 0.0, 2.0,
    1.0, 0.0, 0.0, -2.0,
    0.0, 0.0, 1.0, pi / 40.0,
    0.0, 0.0, 0.0, 1.0;

const Transform3 transform = Transform3::from_matrix(matrix);

const TransformLogResult logarithm =
    transform.to_principal_exponential_coordinates();

const Eigen::Vector3d rho = logarithm.coordinates.head<3>();
const Eigen::Vector3d phi = logarithm.coordinates.tail<3>();
```

This is a `nominal` case because $\theta = \pi/2$ lies between the two formula-selection regions. Re-exponentiating the recovered coordinate column reconstructs the same finite transformation. The example also shows why copying the transform's translation into the first three output entries would be wrong: $[2 \ -2 \ \pi/40]^T$ and $[0 \ -\pi \ \pi/40]^T$ answer different questions.

9.11.8 Validation and focused evidence

The stored transformation has already passed `Transform3` construction, including finite-component, homogeneous-row, and proper-rotation checks. The logarithm reuses `Rotation3::to_principal_rotation_vector()` to validate the angular branch policy before any identity shortcut. An invalid or overlapping threshold configuration therefore reports `GeometryError::invalid_policy` even when the stored transform is the identity.

For a nonzero rotation, the function checks the recovered angle, $D(\theta)$, the inverse Jacobian, and the recovered linear coordinates for finiteness. A non-finite required intermediate or output reports `GeometryError::unsupported_magnitude` and returns no `TransformLogResult`. This includes a valid transform with a finite but extremely large translation for which $J^{-1}{}^a\mathbf{p}$ overflows.

The eleven focused tests establish the following properties:

Cases	Property established
Complete identity and ordinary pure translation	Exact coordinates and distinct <code>identity</code> and <code>pure_translation</code> classifications
Smallest positive subnormal translation	Every representable nonzero translation is preserved rather than classified as identity
Analytic finite-pitch quarter turn	Independent nominal recovery of both ${}^a\boldsymbol{\rho}$ and ${}^a\boldsymbol{\phi}$
Analytic 10^{-8} rad screw	Stable $D(\theta)$ series recovery without erasing the rotation
Analytic half-turn about $+z_a$	Deterministic <code>near_pi</code> recovery and correct inverse-Jacobian action
Arbitrary principal-axis round trip	Coupled exponential-logarithm compatibility and separate rotation and translation residuals
Extreme finite translation	Overflow in the recovered linear block reports <code>unsupported_magnitude</code>
Overlapping angular thresholds	Invalid numerical policy is rejected before geometric processing

Cases	Property established
Values immediately below, on, and above θ_{series}	The small-angle boundary is inclusive and the adjacent formulas recover the same motion
Values immediately below, on, and above $\theta_{\pi,b}$	The near- π boundary is inclusive and deterministic in binary64

The analytic tests construct the input matrix directly from closed-form geometry instead of calling the production exponential. This prevents matching forward and inverse errors from hiding a defect. The separate arbitrary-axis round trip is then appropriate integration evidence because the individual branches already have independent analytic checks.

9.11.9 Deliberate limits

This member returns one principal finite coordinate column; it does not return every matrix logarithm of the transformation. At an exact half-turn, the rotation axis has an unavoidable sign ambiguity, so the reused $SO(3)$ logarithm applies its documented deterministic convention. The returned coordinates reconstruct the same transformation, but another valid logarithm may use the opposite boundary axis with a corresponding linear block.

9.12 Exact constant-twist pose integration

The completed $SE(3)$ exponential in Section 9.10 constructs a finite rigid displacement whose path starts at the identity. Constant-twist integration answers the next implementation question: given an existing pose and a constant rigid-body velocity over a non-negative time interval, how should that finite displacement be composed with the existing pose? The answer must preserve the distinction between a twist expressed using fixed space coordinates and one expressed using moving body coordinates because that distinction determines the multiplication side.

9.12.1 Input, output, frame, and units

Let $t_0 \in \mathbb{R}$ be the initial time in seconds, let $\Delta t \geq 0$ be the integration duration in seconds, and let $\{a\}$ be a fixed frame and $\{b\}$ a frame rigidly attached to the moving body. The initial pose is

$$\mathbf{T}_0 := \mathbf{T}_{ab}(t_0) = \begin{bmatrix} \mathbf{R}_0 & {}^a\mathbf{p}_0 \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3),$$

where $\mathbf{R}_0 \in SO(3)$ is the orientation of $\{b\}$ relative to $\{a\}$ and ${}^a\mathbf{p}_0 \in \mathbb{R}^3$ is the position of the body origin O_b relative to O_a , expressed in $\{a\}$ and measured in metres. The required output is the updated pose $\mathbf{T}_1 := \mathbf{T}_{ab}(t_0 + \Delta t) \in SE(3)$.

The two supported constant velocity inputs use the project's linear-first order:

$${}^a\boldsymbol{\xi}_s = \begin{bmatrix} {}^a\boldsymbol{\nu}_s \\ {}^a\boldsymbol{\omega}_s \end{bmatrix}, \quad {}^b\boldsymbol{\xi}_b = \begin{bmatrix} {}^b\boldsymbol{\nu}_b \\ {}^b\boldsymbol{\omega}_b \end{bmatrix} \in \mathbb{R}^6.$$

The linear blocks ${}^a\boldsymbol{\nu}_s$ and ${}^b\boldsymbol{\nu}_b$ are measured in m s^{-1} , and the angular blocks ${}^a\boldsymbol{\omega}_s$ and ${}^b\boldsymbol{\omega}_b$ are measured in rad s^{-1} . The body linear block is the velocity of O_b expressed along the moving body axes. The space linear block instead gives the same rigid velocity field at the fixed origin O_a , expressed along the fixed axes; it is generally not the velocity of O_b .

9.12.2 Exact updates from the pose-rate equations

The definitions reviewed in Section 8.4 are

$$\widehat{{}^b\boldsymbol{\xi}_b} = \mathbf{T}_{ab}^{-1} \dot{\mathbf{T}}_{ab}, \quad \widehat{{}^a\boldsymbol{\xi}_s} = \dot{\mathbf{T}}_{ab} \mathbf{T}_{ab}^{-1}.$$

Multiplying the first equation on the left by $\mathbf{T}_{ab}$ and the second equation on the right by $\mathbf{T}_{ab}$ gives the two pose-rate equations

$$\dot{\mathbf{T}}_{ab} = \mathbf{T}_{ab} \widehat{{}^b\boldsymbol{\xi}_b}, \quad \dot{\mathbf{T}}_{ab} = \widehat{{}^a\boldsymbol{\xi}_s} \mathbf{T}_{ab}.$$

If the six numerical space-twist components remain constant throughout $[t_0, t_0 + \Delta t]$, the constant matrix generator multiplies the pose from the left. The exact solution of that linear matrix differential equation is

$$\boxed{\mathbf{T}_1 = \exp(\widehat{{}^a\boldsymbol{\xi}_s} \Delta t) \mathbf{T}_0}.$$

If the six numerical body-twist components remain constant throughout the same interval, the generator multiplies from the right and the exact solution is

$$\boxed{\mathbf{T}_1 = \mathbf{T}_0 \exp(\widehat{{}^b\boldsymbol{\xi}_b} \Delta t)}.$$

Differentiating either candidate solution recovers its pose-rate equation, and setting $\Delta t = 0$ makes the exponential $\mathbf{I}_4$, so both solutions recover the initial pose. These formulas are exact for a constant twist; they are not Forward Euler approximations and do not add entries directly to a transformation matrix.

9.12.3 From velocity to a finite $SE(3)$ increment

The implementation first multiplies each velocity block by the common duration:

$$\eta = \xi \Delta t = \begin{bmatrix} \rho \\ \phi \end{bmatrix} = \begin{bmatrix} \nu \Delta t \\ \omega \Delta t \end{bmatrix}.$$

The finite linear coordinate ρ is measured in metres and the finite rotation vector ϕ is measured in radians. The completed factory then calculates

$$\mathbf{T}_\Delta = \exp(\hat{\eta}) = \begin{bmatrix} \exp([\phi]_\times) & \mathbf{J}(\phi)\rho \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

When $\phi \neq \mathbf{0}_3$, the finite coordinate $\rho = \nu \Delta t$ is not generally the translation column of $\mathbf{T}_\Delta$. The translation is $\mathbf{J}(\phi)\rho$ because the directions used by a constant body twist rotate during the interval. The left Jacobian performs the required integration after expressing those changing velocity directions in one common coordinate frame.

The integration methods introduce duration as a new kind of public input, so the integration block extends `GeometryError` with `invalid_time`. This category reports a duration that is negative or non-finite. A non-finite twist remains `non_finite`, while overflow produced by multiplying two finite values remains `unsupported_magnitude`; `invalid_time` therefore identifies a distinct failure at the time-input boundary.

The `Transform3` public interface declares separate methods so the twist representation is explicit at every call site:

```
[[nodiscard]] Transform3 integrate_constant_space_twist(
    const Vector6LinearFirst & space_twist, double delta_time,
    const NumericalPolicy & policy = NumericalPolicy{}) const;

[[nodiscard]] Transform3 integrate_constant_body_twist(
    const Vector6LinearFirst & body_twist, double delta_time,
    const NumericalPolicy & policy = NumericalPolicy{}) const;
```

Both are `const` member functions, so the `Transform3` object before the dot is the validated initial pose $\mathbf{T}_0$ and is not mutated. A caller uses either representation as follows; omitting the final argument selects the public default numerical policy:

```
const Transform3 updated_from_space =
    initial_pose.integrate_constant_space_twist(space_twist, delta_time);

const Transform3 updated_from_body =
    initial_pose.integrate_constant_body_twist(body_twist, delta_time);
```

These calls are alternative representations of one update only when `space_twist` and `body_twist` describe the same physical motion. At the initial pose, that requires $\xi_s = \text{Ad}_{T_0} \xi_b$; supplying the same numerical six-vector to both methods does not generally satisfy that relation.

After validating the duration and twist, the two implementations reuse the same twist-time product and $SE(3)$ exponential but preserve different multiplication sides:

```
const Vector6LinearFirst exponential_coordinates = twist * delta_time;

const Transform3 increment =
    Transform3::from_exponential_coordinates(exponential_coordinates, policy);

// Space form: T_delta T_0.
return increment.compose(*this);

// Body form: T_0 T_delta.
return this->compose(increment);
```

Inside either non-static member function, `this` points to the initial `Transform3` object T_0 . The expression `*this` supplies that object as the right operand of `increment.compose(...)`, whereas `this->compose(increment)` invokes composition on T_0 and supplies the increment as its right operand.

9.12.4 Equation-to-code map

Mathematics	Code	Meaning
T_0	<code>*this</code>	Validated initial pose
${}^a\xi_s$	<code>space_twist</code>	Constant linear-first space twist expressed in $\{a\}$
${}^b\xi_b$	<code>body_twist</code>	Constant linear-first body twist expressed in $\{b\}$
Δt	<code>delta_time</code>	Non-negative duration in seconds
$\eta = \xi \Delta t$	<code>exponential_coordinates</code>	Finite linear-first coordinates in metres and radians
$T_\Delta = \exp(\hat{\eta})$	<code>increment</code>	Validated finite rigid displacement
$T_\Delta T_0$	<code>increment.compose(*this)</code>	Space-referred update
$T_0 T_\Delta$	<code>this->compose(increment)</code>	Body-referred update

Mathematics	Code	Meaning
T_1	returned <code>Transform3</code>	Updated pose

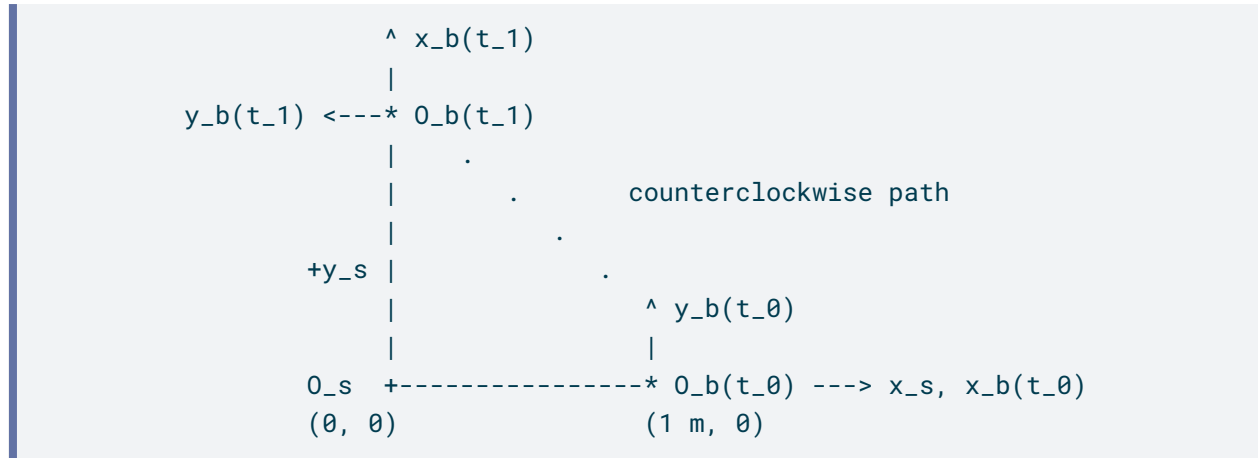
9.12.5 Equivalent body and space quarter-circle example

This example starts from one physical motion. It then describes that motion with a body twist and with a space twist. The two six-component columns will look different because they use different coordinate axes and different reference origins, but both must predict the same velocity at every physical point and the same final pose.

9.12.5.1 Begin with the physical motion

Let $\{s\}$ be a fixed space frame with origin O_s . Let $\{b\}$ be a frame fixed to a moving body, with origin O_b . At the initial time $t_0 = 0$, the body origin is at $[1 \ 0 \ 0]^T$ m in space coordinates, and the body axes align with the space axes. Observe the motion for $\Delta t = \pi/2$ s, and call the end of this interval $t_1 = t_0 + \Delta t$.

The body rotates counterclockwise about the fixed point O_s at 1 rad s^{-1} . The distance from O_s to O_b remains 1 m, so O_b follows a quarter-circle in the x_s - y_s plane during this interval. Because the frame $\{b\}$ is attached to the body, its axes rotate with the body:



At t_0 , x_b points along $+x_s$ and y_b points along $+y_s$. After a counterclockwise quarter-turn, x_b points along $+y_s$ and y_b points along $-x_s$. Notice two geometric facts in the diagram: the radius from O_s to O_b always points along $+x_b$, while the velocity tangent to the circle always points along $+y_b$.

Let $\theta(t)$ be the angle turned by elapsed time t . The constant angular speed gives

$$\theta(t) = (1 \text{ rad s}^{-1})t.$$

The geometry therefore determines the complete pose trajectory. The orientation of the body frame relative to the space frame is

$$\mathbf{R}_{sb}(t) = \mathbf{R}_z(\theta(t)),$$

and the position of O_b relative to O_s , expressed in space coordinates, is

$$\begin{aligned} {}^s\mathbf{p}_b(t) &= \mathbf{R}_z(\theta(t)) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m} \\ &= \begin{bmatrix} \cos \theta(t) \\ \sin \theta(t) \\ 0 \end{bmatrix} \text{ m}. \end{aligned}$$

This explicit column is another way to read the circle in the diagram: its first two entries are the x_s and y_s coordinates of a point at unit radius.

The elapsed interval is

$$\Delta t = \frac{\pi}{2} \text{ s}.$$

Then $\theta(\Delta t) = \pi/2$ rad, so the geometry fixes the answer before either twist calculation begins:

$$\mathbf{R}_{sb}(\Delta t) = \mathbf{R}_z(\pi/2), \quad {}^s\mathbf{p}_b(\Delta t) = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m}.$$

Thus the expected final pose is

$$\mathbf{T}_1 = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The entries in the upper-right pose column are position coordinates in metres. This independently derived pose will be the oracle for both integration forms.

9.12.5.2 The initial pose does not equal the identity pose

At t_0 , the pose is

$$\mathbf{T}_0 = \begin{bmatrix} \mathbf{R}_0 & \mathbf{p}_0 \\ \mathbf{0}_3^\top & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_3 & [1 \ 0 \ 0]^\top \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

where the translation column has units of metres.

Why is $\mathbf{R}_0 = \mathbf{I}_3$? The columns of $\mathbf{R}_0$ are the directions of x_b , y_b , and z_b expressed in space coordinates. Initially those directions are

$$\mathbf{R}_0 = [[1 \ 0 \ 0]^\top \ [0 \ 1 \ 0]^\top \ [0 \ 0 \ 1]^\top] = \mathbf{I}_3.$$

The identity rotation means that the two frames initially have the same orientation. It does not mean that their origins coincide. Their origins are separated by

$$\mathbf{p}_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ m},$$

so $\mathbf{T}_0 \neq \mathbf{I}_4$. This separation of origins is exactly why the adjoint conversion below contains a cross-product term even though $\mathbf{R}_0 = \mathbf{I}_3$.

9.12.5.3 Describe the tangent motion with a body twist

Differentiate the space-coordinate position to see the velocity of the body origin. Using $\dot{\theta} = 1 \text{ s}^{-1}$, with radians treated as dimensionless in the derivative, gives

$${}^s\dot{\mathbf{p}}_b(t) = \begin{bmatrix} -\sin \theta(t) \\ \cos \theta(t) \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

This velocity changes direction in the fixed space axes as the body moves around the circle. In the rotating body axes, however, the velocity always points along $+y_b$. Its body-coordinate column therefore remains constant:

$$\boldsymbol{\nu}_b = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

The counterclockwise angular velocity always points along the common $+z$ direction, so its body-coordinate column is also constant:

$$\boldsymbol{\omega}_b = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \text{ rad s}^{-1}.$$

Using the project's linear-first order, the constant body twist is consequently

$$\boldsymbol{\xi}_b = \begin{bmatrix} \boldsymbol{\nu}_b \\ \boldsymbol{\omega}_b \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

The first three entries carry units of m s^{-1} , and the last three carry units of rad s^{-1} . The name body twist means that these components are expressed along the moving body axes. It does not mean that the motion is different from the physical motion drawn above.

9.12.5.4 Read the same motion as a space twist

The body-twist construction expressed the velocity of the body origin along the rotating body axes. The space-twist construction asks a different question: what constant velocity field, written along the fixed space axes and referenced to the fixed space origin O_s , generates the same motion?

Start with the angular block. The rotation axis is the fixed $+z_s$ axis and the angular speed is 1 rad s^{-1} , so the space-coordinate angular velocity is

$$\boldsymbol{\omega}_s = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \text{ rad s}^{-1}.$$

Now determine the linear block. A point at space-coordinate location $\mathbf{x}$ undergoing rotation with angular velocity $\boldsymbol{\omega}_s$ about a fixed centre $\mathbf{q}$ has velocity

$${}^s\mathbf{v}(\mathbf{x}) = \boldsymbol{\omega}_s \times (\mathbf{x} - \mathbf{q}).$$

Expanding the cross product separates the part that is independent of $\mathbf{x}$ from the part that changes with the point location:

$${}^s\mathbf{v}(\mathbf{x}) = -\boldsymbol{\omega}_s \times \mathbf{q} + \boldsymbol{\omega}_s \times \mathbf{x}.$$

The standard velocity field of a space twist is

$${}^s\mathbf{v}(\mathbf{x}) = \boldsymbol{\nu}_s + \boldsymbol{\omega}_s \times \mathbf{x}.$$

Comparing the two equations identifies the spatial linear block:

$$\boxed{\boldsymbol{\nu}_s = -\boldsymbol{\omega}_s \times \mathbf{q}}.$$

This equation explains what $\boldsymbol{\nu}_s$ represents. It is the velocity assigned by the rigid-body velocity field to the space origin, not generally the velocity of the body origin. Evaluating the field at $\mathbf{x} = \mathbf{0}_3$ leaves only $\boldsymbol{\nu}_s$.

In this example, the centre of rotation is the space origin itself, so $\mathbf{q} = \mathbf{0}_3$. Therefore,

$$\boldsymbol{\nu}_s = -\boldsymbol{\omega}_s \times \mathbf{0}_3 = \mathbf{0}_3.$$

The constant space twist is consequently

$$\boldsymbol{\xi}_s = \begin{bmatrix} \boldsymbol{\nu}_s \\ \boldsymbol{\omega}_s \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

The first three zeros do not say that the body origin O_b is stationary. They say that the velocity field is zero at its reference point O_s , which is the centre of rotation.

To verify that this space twist describes the complete circular motion, evaluate its velocity field at the moving body origin. Its location is

$${}^s\mathbf{p}_b(t) = \begin{bmatrix} \cos \theta(t) \\ \sin \theta(t) \\ 0 \end{bmatrix} \text{ m.}$$

Because $\boldsymbol{\nu}_s = \mathbf{0}_3$, and because radians are dimensionless in the cross-product units, the field predicts

$$\begin{aligned}
{}^s\mathbf{v}({}^s\mathbf{p}_b(t)) &= \boldsymbol{\omega}_s \times {}^s\mathbf{p}_b(t) \\
&= \left(\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \text{ rad s}^{-1} \right) \times \left(\begin{bmatrix} \cos \theta(t) \\ \sin \theta(t) \\ 0 \end{bmatrix} \text{ m} \right) \\
&= \begin{bmatrix} -\sin \theta(t) \\ \cos \theta(t) \\ 0 \end{bmatrix} \text{ m s}^{-1}.
\end{aligned}$$

This is exactly the derivative of the circular position found in the body-twist construction. The body description says that the tangent velocity has the constant components $[0 \ 1 \ 0]^T \text{ m s}^{-1}$ along the rotating body axes. The space description says that the same tangent velocity has the changing components $[-\sin \theta(t) \ \cos \theta(t) \ 0]^T \text{ m s}^{-1}$ along the fixed space axes. These are two coordinate descriptions of the same physical velocity.

9.12.5.5 The adjoint formula expresses the same geometric comparison

The geometry also explains why the body-to-space adjoint formula has its particular cross-product term. The body-twist linear block is the velocity of O_b expressed along body axes. Rotating that column into space coordinates gives

$${}^s\mathbf{v}_{O_b} = \mathbf{R}_0 \boldsymbol{\nu}_b.$$

The space velocity field, evaluated at the same physical point O_b , gives

$${}^s\mathbf{v}_{O_b} = \boldsymbol{\nu}_s + \boldsymbol{\omega}_s \times \mathbf{p}_0.$$

These two expressions must be equal because they describe the velocity of the same point:

$$\mathbf{R}_0 \boldsymbol{\nu}_b = \boldsymbol{\nu}_s + \boldsymbol{\omega}_s \times \mathbf{p}_0.$$

Solve for $\boldsymbol{\nu}_s$ and use the antisymmetry $\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$:

$$\begin{aligned}
\boldsymbol{\nu}_s &= \mathbf{R}_0 \boldsymbol{\nu}_b - \boldsymbol{\omega}_s \times \mathbf{p}_0 \\
&= \mathbf{R}_0 \boldsymbol{\nu}_b + \mathbf{p}_0 \times \boldsymbol{\omega}_s.
\end{aligned}$$

The angular columns use different coordinate axes but refer to the same angular velocity, so

$$\boldsymbol{\omega}_s = \mathbf{R}_0 \boldsymbol{\omega}_b.$$

Substitution produces the linear-first adjoint relation

$$\boldsymbol{\nu}_s = \mathbf{R}_0 \boldsymbol{\nu}_b + \mathbf{p}_0 \times (\mathbf{R}_0 \boldsymbol{\omega}_b).$$

Stacking the linear and angular component equations gives the equivalent matrix relation

$$\boldsymbol{\xi}_s = \text{Ad}_{\mathbf{T}_0} \boldsymbol{\xi}_b, \quad \text{Ad}_{\mathbf{T}_0} = \begin{bmatrix} \mathbf{R}_0 & [\mathbf{p}_0]_{\times} \mathbf{R}_0 \\ \mathbf{0}_{3 \times 3} & \mathbf{R}_0 \end{bmatrix}.$$

The two terms now have a direct interpretation. The term $\mathbf{R}_0 \boldsymbol{\nu}_b$ expresses the velocity of O_b in space coordinates. The cross-product term $[\mathbf{p}_0]_{\times} \mathbf{R}_0$ shifts the reference point from O_b to O_s .

For the numerical values in this example,

$$\begin{aligned} \boldsymbol{\nu}_s &= \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} \\ &= \mathbf{0}_3. \end{aligned}$$

The first two lines have units of m s^{-1} ; radians are dimensionless in this product. The cancellation is not an algebraic accident. It states geometrically that the velocity field is zero at its centre of rotation, O_s .

9.12.5.6 Integrate the space twist on the left

The space twist is a pure rotation about the fixed space origin. Over $\Delta t = \pi/2$ s, its exponential is therefore

$${}^s\mathbf{T}_{\Delta} = \exp(\hat{\boldsymbol{\xi}}_s \Delta t) = \begin{bmatrix} \mathbf{R}_z(\pi/2) & \mathbf{0}_3 \\ \mathbf{0}_3^{\top} & 1 \end{bmatrix}.$$

Because this increment is expressed relative to the fixed space frame, it acts on the current pose from the left:

$$\mathbf{T}_1 = {}^s\mathbf{T}_\Delta \mathbf{T}_0.$$

Block multiplication shows what the left action does:

$$\begin{aligned}\mathbf{R}_1 &= \mathbf{R}_z(\pi/2)\mathbf{I}_3 = \mathbf{R}_z(\pi/2), \\ \mathbf{p}_1 &= \mathbf{R}_z(\pi/2) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m}.\end{aligned}$$

This is exactly the fixed-centre rotation drawn in the diagram.

9.12.5.7 Understand why the body exponential coordinate is not the translation column

Multiplying the body twist by the duration produces the finite exponential coordinates

$$\boldsymbol{\rho}_b = \boldsymbol{\nu}_b \Delta t = \begin{bmatrix} 0 \\ \pi/2 \\ 0 \end{bmatrix} \text{ m}, \quad \phi_b = \boldsymbol{\omega}_b \Delta t = \begin{bmatrix} 0 \\ 0 \\ \pi/2 \end{bmatrix} \text{ rad}.$$

The column $\boldsymbol{\rho}_b$ is not generally the translation column of the finite transform. It would be the translation column if no rotation occurred, because the velocity direction would then remain fixed. Here the velocity always points along $+y_b$, but $+y_b$ rotates during the interval. Vectors expressed along changing axes cannot be added as though they all point in one fixed direction.

The actual relative translation, expressed along the initial body axes $\{b_0\}$, is the time integral of that rotating velocity direction. With τ denoting elapsed time inside the interval,

$$\mathbf{p}_\Delta = \int_0^{\Delta t} \mathbf{R}_z((1 \text{ rad s}^{-1})\tau) \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m s}^{-1} d\tau.$$

Let α denote the dimensionless numerical value of the angle in radians. Because the angular speed has numerical value one, $\alpha = \tau/(1 \text{ s})$ and $d\tau = (1 \text{ s})d\alpha$. The change of variable gives

$$\begin{aligned}\mathbf{p}_\Delta &= \begin{bmatrix} \int_0^{\pi/2} -\sin \alpha \, d\alpha \\ \int_0^{\pi/2} \cos \alpha \, d\alpha \\ 0 \end{bmatrix} \text{ m} \\ &= \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \text{ m}.\end{aligned}$$

This distinction has a simple geometric meaning. The norm $\|\boldsymbol{\rho}_b\| = \pi/2 \text{ m}$ is the length travelled along the quarter-circle arc. The norm $\|\mathbf{p}_\Delta\| = \sqrt{2} \text{ m}$ is the straight-line displacement between the initial and final body origins. Arc length and net displacement are different.

The $SE(3)$ exponential performs the same integration through the left Jacobian:

$$\mathbf{p}_\Delta = \mathbf{J}(\boldsymbol{\phi}_b) \boldsymbol{\rho}_b.$$

For $\theta = \pi/2$,

$$\mathbf{J} \left(\begin{bmatrix} 0 \\ 0 \\ \theta \end{bmatrix} \right) = \begin{bmatrix} 2/\pi & -2/\pi & 0 \\ 2/\pi & 2/\pi & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

so

$$\mathbf{J}(\boldsymbol{\phi}_b) \left(\begin{bmatrix} 0 \\ \pi/2 \\ 0 \end{bmatrix} \text{ m} \right) = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \text{ m}.$$

Thus the Jacobian is not an arbitrary correction factor. It accumulates the translation direction while that direction rotates.

The resulting body-referred finite increment is

$${}^b\mathbf{T}_\Delta = \exp(\hat{\boldsymbol{\xi}}_b \Delta t) = \begin{bmatrix} \mathbf{R}_z(\pi/2) & [-1 \ 1 \ 0]^\top \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

where the upper-right column has units of metres.

9.12.5.8 Integrate the body twist on the right

Give the body frame at the two times distinct names: $\{b_0\}$ is the initial body frame and $\{b_1\}$ is the final body frame. Then

$$\mathbf{T}_0 = \mathbf{T}_{sb_0}, \quad {}^b\mathbf{T}_\Delta = \mathbf{T}_{b_0b_1}.$$

To map coordinates from $\{b_1\}$ into $\{s\}$, first map from $\{b_1\}$ into $\{b_0\}$ and then from $\{b_0\}$ into $\{s\}$. Transform composition therefore gives

$$\boxed{\mathbf{T}_1 = \mathbf{T}_{sb_1} = \mathbf{T}_{sb_0} \mathbf{T}_{b_0b_1} = \mathbf{T}_0 {}^b\mathbf{T}_\Delta}.$$

This frame chain is the reason for right multiplication; it is not a mnemonic chosen independently of the transform meanings.

For two transforms, block multiplication gives the translation rule

$$\mathbf{p}_1 = \mathbf{p}_0 + \mathbf{R}_0 \mathbf{p}_\Delta.$$

In this example, $\mathbf{R}_0 = \mathbf{I}_3$, so the relative displacement already has the same numerical components in the initial body and space axes:

$$\begin{aligned} \mathbf{p}_1 &= \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \mathbf{I}_3 \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m.} \end{aligned}$$

The identity $\mathbf{R}_0$ is used only because the frames align at the initial instant. The body axes still rotate during the interval; that rotation has already been accounted for inside $\mathbf{p}_\Delta$ by the exponential. If $\mathbf{R}_0$ were not identity, it would rotate the body-referred relative displacement into space coordinates before that displacement was added to $\mathbf{p}_0$.

9.12.5.9 Both representations recover the geometric answer

The space calculation and the body calculation both give

$$\exp(\hat{\xi}_s \Delta t) \mathbf{T}_0 = \mathbf{T}_0 \exp(\hat{\xi}_b \Delta t) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The equality does not mean that the two twist columns or the two increments are numerically identical. It means that the correctly framed left and right actions describe the same physical motion.

9.12.6 Validation and deliberate limits

Both public methods reject a non-finite or negative duration as `GeometryError::invalid_time` and reject a non-finite twist as `GeometryError::non_finite`. After calculating `exponential_coordinates = twist * delta_time`, they reject a non-finite product as `GeometryError::unsupported_magnitude`. The reused $SE(3)$ exponential then validates the numerical policy, maximum angular displacement, finite Jacobian coefficients, and finite increment. Composition separately rejects a non-finite translated result. A zero duration deliberately passes through the exponential, producing $\mathbf{I}_4$ while still validating the supplied twist and numerical policy; the zero-duration test requires the non-identity initial pose to be preserved exactly in both forms.

The ten focused GTest cases provide independent evidence for these contracts. The governing quarter-circle case constructs the expected pose directly from geometry and checks both integration forms with separate dimensionless rotation and metre-valued translation tolerances of 10^{-12} ; reversing either required multiplication side changes the translation and fails this oracle. The zero-duration case requires exact preservation of a non-identity pose. Paired space and body cases require `invalid_time` for a negative duration and for both `NaN` and infinite durations, `non_finite` for a non-finite twist component, and `unsupported_magnitude` when a finite twist multiplied by a finite duration overflows.

The methods integrate one constant twist over one non-negative interval. They do not integrate a time-varying twist, select a time-stepping method, accept a signed displacement in place of time, estimate a twist from sensor measurements, attach runtime frame labels, or propagate uncertainty. Piecewise-constant integration requires the caller to preserve chronological order across successive updates.

9.13 Linear-first $SE(3)$ adjoint

The constant-twist example in Section 9.12 used two coordinate columns for one physical rigid-body velocity. The adjoint answers the implementation question exposed by that example: given a stored transformation between two frames, how should the coordinates of one twist be converted from the second frame into the first? This conversion is useful whenever a velocity, screw axis, or finite motion direction must be compared or combined in one common frame.

9.13.1 Input, output, frame, and units

Let the stored transformation be

$$\mathbf{T}_{ab} = \begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3),$$

where $\mathbf{R}_{ab} \in SO(3)$ expresses frame $\{b\}$ axes in frame $\{a\}$ axes, and ${}^a\mathbf{p}_b \in \mathbb{R}^3$ is the position of the origin O_b relative to O_a , expressed in $\{a\}$ and measured in metres. Let one physical twist have the two linear-first coordinate columns

$${}^b\xi = \begin{bmatrix} {}^b\nu \\ {}^b\omega \end{bmatrix}, \quad {}^a\xi = \begin{bmatrix} {}^a\nu \\ {}^a\omega \end{bmatrix} \in \mathbb{R}^6.$$

The columns ${}^b\nu$ and ${}^a\nu$ have units of m s^{-1} , while ${}^b\omega$ and ${}^a\omega$ have units of rad s^{-1} . The superscript identifies the axes used for the numerical components; it does not identify a different physical motion.

The geometry derived in Section 9.12 established the component relations

$${}^a\omega = \mathbf{R}_{ab} {}^b\omega,$$

and

$${}^a\nu = \mathbf{R}_{ab} {}^b\nu + {}^a\mathbf{p}_b \times (\mathbf{R}_{ab} {}^b\omega).$$

The angular equation expresses the same angular velocity along the new axes. In the linear equation, $\mathbf{R}_{ab} {}^b\nu$ first expresses the body-origin velocity along frame $\{a\}$ axes, and the cross product then shifts the velocity field's reference point from O_b to O_a .

For every $\mathbf{y} \in \mathbb{R}^3$, the cross-product matrix is defined by $[{}^a\mathbf{p}_b]_\times \mathbf{y} = {}^a\mathbf{p}_b \times \mathbf{y}$. Using that definition and stacking the linear block above the angular block produces the project's linear-first adjoint:

$$\boxed{{}^a\xi = \text{Ad}_{T_{ab}} {}^b\xi}, \quad \boxed{\text{Ad}_{T_{ab}} = \begin{bmatrix} \mathbf{R}_{ab} & [{}^a\mathbf{p}_b]_{\times} \mathbf{R}_{ab} \\ \mathbf{0}_{3 \times 3} & \mathbf{R}_{ab} \end{bmatrix} \in \mathbb{R}^{6 \times 6}}.$$

The two diagonal rotation blocks are dimensionless. The upper-right block contains the translation in metres; multiplying it by angular velocity produces linear velocity because radians are dimensionless. Modern Robotics writes a twist in angular-first order, so its equivalent translation-coupling block appears in the lower-left. The project deliberately uses linear-first order, matching the declared `Vector6LinearFirst` layout, so copying an angular-first block arrangement would implement the wrong public convention.

9.13.2 The conjugation identity checks the convention

The 6×6 formula must agree with the corresponding change of coordinates for the 4×4 tangent matrix. For the linear-first twist ${}^b\xi$, the hat map is

$$\widehat{{}^b\xi} = \begin{bmatrix} [{}^b\omega]_{\times} & {}^b\nu \\ \mathbf{0}_3^{\top} & 0 \end{bmatrix}.$$

Substitute this matrix and the rigid-transform inverse into the conjugation product:

$$\begin{aligned} T_{ab} \widehat{{}^b\xi} T_{ab}^{-1} &= \begin{bmatrix} \mathbf{R}_{ab} [{}^b\omega]_{\times} \mathbf{R}_{ab}^{\top} & -\mathbf{R}_{ab} [{}^b\omega]_{\times} \mathbf{R}_{ab}^{\top} {}^a\mathbf{p}_b + \mathbf{R}_{ab} {}^b\nu \\ \mathbf{0}_3^{\top} & 0 \end{bmatrix} \\ &= \begin{bmatrix} [\mathbf{R}_{ab} {}^b\omega]_{\times} & \mathbf{R}_{ab} {}^b\nu + {}^a\mathbf{p}_b \times (\mathbf{R}_{ab} {}^b\omega) \\ \mathbf{0}_3^{\top} & 0 \end{bmatrix}. \end{aligned}$$

The second equality uses $\mathbf{R}[\omega]_{\times} \mathbf{R}^{\top} = [\mathbf{R}\omega]_{\times}$ and $-[\mathbf{y}]_{\times} \mathbf{p} = \mathbf{p} \times \mathbf{y}$.¹ The resulting rotational block and linear column are exactly the hat matrix of the adjoint-transformed twist. Therefore,

$$\boxed{\widehat{\text{Ad}_{T_{ab}} {}^b\xi} = T_{ab} \widehat{{}^b\xi} T_{ab}^{-1}}.$$

This identity is more useful as independent verification than as the production algorithm. Applying it to one twist requires a transform inverse, two 4×4 products, a hat operation, and a vee operation. Constructing the complete 6×6 adjoint through the identity

¹To verify the first identity, apply both sides to an arbitrary $\mathbf{x} \in \mathbb{R}^3$. The left side gives $\mathbf{R}(\omega \times (\mathbf{R}^{\top} \mathbf{x})) = (\mathbf{R}\omega) \times (\mathbf{R}\mathbf{R}^{\top} \mathbf{x}) = (\mathbf{R}\omega) \times \mathbf{x}$, because a proper rotation preserves cross products and $\mathbf{R}\mathbf{R}^{\top} = \mathbf{I}_3$. This is $[\mathbf{R}\omega]_{\times} \mathbf{x}$ for every $\mathbf{x}$, so the two matrices are equal. The second identity follows from cross-product antisymmetry: $-[\mathbf{y}]_{\times} \mathbf{p} = -(\mathbf{y} \times \mathbf{p}) = \mathbf{p} \times \mathbf{y}$.

would require repeating that work for all six basis columns. The direct block formula constructs the requested matrix with one 3×3 product while keeping the project's frame and component-order convention visible.

9.13.3 Public method and implementation

The public member declaration is

```
// Return the linear-first adjoint Ad_T.
// It maps twist coordinates referenced to {b} into coordinates
// referenced to {a} when this transform is T_ab.
[[nodiscard]] Eigen::Matrix<double, 6, 6> adjoint() const;
```

The method takes no argument because the stored `Transform3` is the only input needed to construct $\text{Ad}_{T_{ab}}$. The trailing `const` promises not to change that stored transformation, and `[[nodiscard]]` asks the compiler to warn when a caller discards the returned coordinate-conversion matrix.

The implementation is

```
Eigen::Matrix<double, 6, 6> Transform3::adjoint() const
{
    const Eigen::Matrix3d rotation = rotation_.matrix();
    const Eigen::Matrix3d translation_hat = hat_so3(translation_);
    const Eigen::Matrix3d translation_rotation = translation_hat * rotation;

    if (!translation_rotation.allFinite()) {
        throw GeometryException(
            GeometryError::unsupported_magnitude,
            "Adjoint is outside the supported numerical range.");
    }

    Eigen::Matrix<double, 6, 6> adjoint_matrix =
        Eigen::Matrix<double, 6, 6>::Zero();

    adjoint_matrix.block<3, 3>(0, 0) = rotation;
    adjoint_matrix.block<3, 3>(0, 3) = translation_rotation;
    adjoint_matrix.block<3, 3>(3, 3) = rotation;

    return adjoint_matrix;
}
```

`rotation` is $\mathbf{R}_{ab}$, `translation_hat` is ${}^a\mathbf{p}_b]_{\times}$, and `translation_rotation` is their required left-to-right product ${}^a\mathbf{p}_b]_{\times} \mathbf{R}_{ab}$. Matrix multiplication acts on a twist from right to left, so

this order first rotates ${}^b\boldsymbol{\omega}$ into frame $\{a\}$ and then calculates the cross product with ${}^a\mathbf{p}_b$. Replacing the product with `translation_hat` alone would be correct only when $\mathbf{R}_{ab} = \mathbf{I}_3$.

`Eigen::Matrix<double, 6, 6>::Zero()` establishes the complete zero matrix, including the required lower-left block. Each call `block<3, 3>(start_row, start_column)` then selects a fixed 3×3 region by its starting indices: `(0, 0)` is the upper-left block, `(0, 3)` is the upper-right block, and `(3, 3)` is the lower-right block. The arguments are locations for `block`, unlike the runtime-size arguments of an Eigen corner accessor.

The equation-to-code correspondence is

Mathematical object	Code expression	Meaning
$\mathbf{R}_{ab}$	<code>rotation_.matrix()</code>	Express frame $\{b\}$ axes along frame $\{a\}$ axes
${}^a\mathbf{p}_b$	<code>translation_</code>	Position of O_b relative to O_a , expressed in $\{a\}$ and measured in metres
$[{}^a\mathbf{p}_b]_{\times}$	<code>hat_so3(translation_)</code>	Matrix that calculates ${}^a\mathbf{p}_b \times \mathbf{y}$
$[{}^a\mathbf{p}_b]_{\times} \mathbf{R}_{ab}$	<code>translation_hat * rotation</code>	Rotate angular components into $\{a\}$, then shift the velocity-field reference point
$\text{Ad}_{\mathbf{T}_{ab}}$	<code>adjoint_matrix</code>	Linear-first 6×6 coordinate-conversion matrix

9.13.4 Usage and analytic result

This example uses zero relative rotation, $\mathbf{R}_{ab} = \mathbf{I}_3$, so the frame axes align. The nonzero ${}^a\mathbf{p}_b$ and ${}^b\boldsymbol{\omega}$ keep the upper-right coupling $[{}^a\mathbf{p}_b]_{\times} \mathbf{R}_{ab}$ observable. The calculation therefore isolates the shift between the frame origins and supplies the expected result for the first focused adjoint test.

Use the specification acceptance inputs ${}^a\mathbf{p}_b = [1 \ 2 \ 3]^T$ m and

$${}^b\boldsymbol{\xi} = [4 \ 5 \ 6 \ 1 \ 0 \ -1]^T,$$

where the first block has units of m s^{-1} and the second has units of rad s^{-1} . The translation contribution is

$${}^a\mathbf{p}_b \times (\mathbf{R}_{ab} {}^b\boldsymbol{\omega}) = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} -2 \\ 4 \\ -2 \end{bmatrix} \text{ m s}^{-1}.$$

Because the rotation is identity, the converted linear and angular blocks are

$${}^a\boldsymbol{\nu} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} + \begin{bmatrix} -2 \\ 4 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ 9 \\ 4 \end{bmatrix} \text{ m s}^{-1}, \quad {}^a\boldsymbol{\omega} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \text{ rad s}^{-1}.$$

Stacking these two blocks in the declared order gives ${}^a\boldsymbol{\xi} = [2 \ 9 \ 4 \ 1 \ 0 \ -1]^\top$, with the first three components in m s^{-1} and the final three components in rad s^{-1} .

The public API is used as follows:

```
Eigen::Matrix4d matrix_ab = Eigen::Matrix4d::Identity();
matrix_ab(0, 3) = 1.0;
matrix_ab(1, 3) = 2.0;
matrix_ab(2, 3) = 3.0;

const rigid_body_kinematics::Transform3 transform_ab =
    rigid_body_kinematics::Transform3::from_matrix(matrix_ab);

// xi_b = [nu_b; omega_b].
const rigid_body_kinematics::Vector6LinearFirst twist_b(
    4.0, 5.0, 6.0, 1.0, 0.0, -1.0);

const rigid_body_kinematics::Vector6LinearFirst twist_a =
    transform_ab.adjoint() * twist_b;
```

The multiplication `transform_ab.adjoint() * twist_b` follows the project's column-vector and rightmost-first convention: construct the adjoint from $\mathbf{T}_{ab}$, then apply that matrix to ${}^b\boldsymbol{\xi}$ to obtain ${}^a\boldsymbol{\xi}$.

The first focused GTest case uses this same transformation and twist, then checks all six components against the independently derived result $[2 \ 9 \ 4 \ 1 \ 0 \ -1]^\top$. The worked calculation above is therefore the test oracle, not a separate example introduced only to exercise the API.

9.13.5 Validation and focused evidence

A valid `Transform3` already guarantees finite stored translation components and a finite proper rotation. Those guarantees do not prove that the product ${}^a\mathbf{p}_b \times \mathbf{R}_{ab}$ is

representable in binary64. Two finite products may be added inside one matrix entry and overflow. The implementation therefore checks `translation_rotation.allFinite()` before placing the product in a valid-looking adjoint matrix; failure reports `GeometryError::unsupported_magnitude`.

The five focused GTest cases establish complementary properties:

Case	Property established
Specification case worked in Section 9.13.4	Independent linear-first result $[2 \ 9 \ 4 \ 1 \ 0 \ -1]^T$
Quarter-turn rotation with nonzero translation	$\mathbf{R}_{ab}$ acts before the cross product, so omitting the rotation factor is detected
Homogeneous conjugation identity	The complete 6×6 convention agrees with $\mathbf{T}_{ab} \widehat{b} \boldsymbol{\xi} \mathbf{T}_{ab}^{-1}$ for a nontrivial transform and twist
Identity transform	$\text{Ad}_{\mathbf{I}_4} = \mathbf{I}_6$ exactly
Extreme finite translation with a $\pi/4$ rotation about x	A non-finite required product reports <code>unsupported_magnitude</code> rather than returning infinities

The analytic specification case alone cannot detect a missing factor $\mathbf{R}_{ab}$ because its rotation is identity. The separate non-identity case isolates that multiplication-order defect. The conjugation test then checks the same operation through independently tested homogeneous-transform, inverse, and hat-map paths instead of copying the production block construction into the expected result.

9.13.6 Deliberate limits

The member constructs the adjoint matrix but does not attach runtime frame labels or units to that matrix, so the caller must preserve the declared $\mathbf{T}_{ab}$ direction and apply it to coordinates expressed in $\{b\}$. It does not mutate the transformation, infer a transform from twist data, or calculate a transformation exponential. Applying the returned matrix to a twist remains an explicit matrix-column product at the call site.

9.14 Planar pose embedding and extraction

9.14.1 Purpose, inputs, outputs, frames, and units

Chapter 7 derived how a planar pose is embedded in $SE(3)$ in Section 7.7. The mathematical question here is when a planar pose and a spatial transformation contain the same

pose information. The forward direction must place the planar coordinates inside $SE(3)$. The reverse direction must reject a spatial transformation containing vertical translation, roll, or pitch instead of quietly discarding those three-dimensional components.

Let $\mathbf{T}_{ab}$ describe the pose of frame $\{b\}$ relative to frame $\{a\}$ and map point coordinates from $\{b\}$ into $\{a\}$. Let x and y be the first two components of the position ${}^a\mathbf{p}_b$ of origin O_b relative to origin O_a , expressed along the axes of $\{a\}$ and measured in metres. Let θ be the right-handed rotation of the $\{b\}$ axes about the positive z_a -axis, measured in radians.

Embedding maps the three planar coordinates (x, y, θ) to

$$\mathbf{T}_{ab} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & x \\ \sin \theta & \cos \theta & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The embedded matrix has zero vertical translation, and its rotation leaves the positive z -axis unchanged. These two properties determine whether the reverse operation is applicable. Write a candidate spatial transformation as $\mathbf{T}_{ab} = [\mathbf{R}_{ab}, {}^a\mathbf{p}_b; \mathbf{0}_3^\top, 1]$ and define $\mathbf{e}_z = [0 \ 0 \ 1]^\top$. The dimensionless orientation residual and metre-valued vertical-position residual are

$$\delta_R = \|\mathbf{R}_{ab}\mathbf{e}_z - \mathbf{e}_z\|_2, \quad \delta_p = |\mathbf{e}_z^\top {}^a\mathbf{p}_b|.$$

The first residual detects roll or pitch because either motion tilts the z_b -axis away from the positive z_a -axis. The second residual detects vertical translation. For declared tolerances $\varepsilon_{R,\text{planar}}$ and $\varepsilon_{p,\text{planar}}$, extraction is applicable only when

$$\delta_R \leq \varepsilon_{R,\text{planar}}, \quad \delta_p \leq \varepsilon_{p,\text{planar}}.$$

If either inequality fails, the spatial transformation does not belong to the accepted planar subset. The operation must fail rather than project the transformation onto that subset, because projection would discard vertical translation, roll, or pitch. When both inequalities hold, the planar coordinates are

$$x = T_{14}, \quad y = T_{24}, \quad \theta = \text{atan2}(R_{21}, R_{11}) \in (-\pi, \pi].$$

The C++ interface represents these mathematical inputs and outputs with

```
struct PlanarPose
{
    double x_metres;
    double y_metres;
```

```

    double yaw_radians;
};

[[nodiscard]] Transform3 embed_planar_pose(
    const PlanarPose & pose,
    const NumericalPolicy & policy = NumericalPolicy{});

[[nodiscard]] PlanarPose extract_planar_pose(
    const Transform3 & transform,
    const NumericalPolicy & policy = NumericalPolicy{});

```

The fields `x_metres`, `y_metres`, and `yaw_radians` represent x , y , and θ . The type does not carry runtime frame labels, so the caller remains responsible for preserving the declared relationship between frames $\{a\}$ and $\{b\}$.

After the policy and pose components pass validation, `embed_planar_pose` implements the embedding matrix directly:

```

const double cos_theta = std::cos(pose.yaw_radians);
const double sin_theta = std::sin(pose.yaw_radians);

Eigen::Matrix4d matrix;
// clang-format off
matrix <<
    cos_theta, -sin_theta, 0.0, pose.x_metres,
    sin_theta,  cos_theta, 0.0, pose.y_metres,
    0.0,       0.0,      1.0, 0.0,
    0.0,       0.0,      0.0, 1.0;
// clang-format on

return Transform3::from_matrix(matrix, policy);

```

`embed_planar_pose` validates the angular policy before inspecting `pose`, rejects every non-finite pose component, and rejects a finite yaw satisfying $|\theta| > \theta_{\max}$. The function evaluates the displayed sine and cosine entries and passes the complete matrix to `Transform3::from_matrix`; this final construction preserves the existing $SE(3)$ validation boundary instead of creating a second unchecked transform representation. The yaw is validated and evaluated without first wrapping it into $(-\pi, \pi]$. Therefore, supported inputs that differ by complete turns produce the same embedded rotation, and the resulting `Transform3` does not preserve the number of complete turns.

After validating the two planar tolerances, `extract_planar_pose` computes the residuals, rejects a genuinely spatial transformation, and reads the three planar coordinates:

```

const Eigen::Matrix4d transform_matrix = transform.matrix();

const Eigen::Vector3d e_z(0.0, 0.0, 1.0);
const Eigen::Matrix3d rotation = transform_matrix.block<3, 3>(0, 0);
const double planar_orientation_error = (rotation * e_z - e_z).stableNorm();

if (planar_orientation_error > policy.planar_orientation_tolerance) {
    throw GeometryException(
        GeometryError::non_planar,
        "Planar orientation does not preserve the positive z-axis.");
}

if (std::abs(transform_matrix(2, 3)) > policy.planar_position_tolerance) {
    throw GeometryException(
        GeometryError::non_planar,
        "Vertical translation exceeds the planar-position tolerance.");
}

double yaw_radians =
    std::atan2(transform_matrix(1, 0), transform_matrix(0, 0));

if (yaw_radians <= -std::numbers::pi) {
    yaw_radians = std::numbers::pi;
}

return PlanarPose{
    transform_matrix(0, 3), transform_matrix(1, 3), yaw_radians};

```

`extract_planar_pose` begins from an already validated `Transform3`. The code stores δ_R as `planar_orientation_error` and evaluates δ_p as `std::abs(transform_matrix(2, 3))`. Their policy limits, `planar_orientation_tolerance` and `planar_position_tolerance`, represent $\varepsilon_{R,\text{planar}}$ and $\varepsilon_{p,\text{planar}}$; they are respectively dimensionless and measured in metres, and both must be finite and nonnegative. If either residual exceeds its own limit, the function reports `GeometryError::non_planar` and returns no `PlanarPose`.

When both constraints pass, the function reads x and y from matrix entries `(0, 3)` and `(1, 3)` and evaluates `std::atan2(transform_matrix(1, 0), transform_matrix(0, 0))` for θ . The C++ `atan2` range includes both signed half-turn endpoints. The public contract instead uses $(-\pi, \pi]$, so an exact result of $-\pi$ is replaced by the equivalent value $+\pi$ before `PlanarPose{x, y, theta}` is returned. No tolerance band is used for this replacement because a representable angle just greater than $-\pi$ already belongs to the promised interval and must remain unchanged.

9.15 Normalized planar-yaw quaternion conversion

9.15.1 Purpose, convention, and mathematical result

The planar pose uses one yaw angle, while a ROS-compatible spatial orientation interface uses four quaternion fields. The required conversion must represent the same right-handed rotation through yaw $\theta \in \mathbb{R}$ about the positive z -axis, return the fields in x, y, z, w order, and ensure that the returned dimensionless quaternion has unit norm. It converts an orientation representation only; it does not convert the planar position.

Chapter 7 derived the axis-angle unit quaternion in Section 7.8. For the unit rotation axis $\mathbf{e}_z = [0 \ 0 \ 1]^T$ and rotation angle θ radians, its scalar-vector form is

$$\mathbf{q}(\theta) = \left(\cos \frac{\theta}{2}, \mathbf{e}_z \sin \frac{\theta}{2} \right).$$

Placing the three vector components first and the scalar component last gives the required field-order column

$$\mathbf{q}_{xyzw}(\theta) = \begin{bmatrix} q_x \\ q_y \\ q_z \\ q_w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \sin(\theta/2) \\ \cos(\theta/2) \end{bmatrix}.$$

The x and y quaternion components are zero because the rotation axis has no x - or y -component; they are not planar position coordinates. The z component stores the z -axis contribution scaled by the half-angle sine, and the w component stores the scalar half-angle cosine. The quaternion represents a physical rotation through θ , not through $\theta/2$.

The exact column is already normalized because

$$\|\mathbf{q}_{xyzw}(\theta)\|_2 = \sqrt{\sin^2 \frac{\theta}{2} + \cos^2 \frac{\theta}{2}} = 1.$$

Yaw angles separated by one complete turn produce opposite quaternion columns:

$$\begin{aligned} \sin \left(\frac{\theta + 2\pi}{2} \right) &= -\sin \frac{\theta}{2}, \\ \cos \left(\frac{\theta + 2\pi}{2} \right) &= -\cos \frac{\theta}{2}, \end{aligned} \quad \mathbf{q}_{xyzw}(\theta + 2\pi) = -\mathbf{q}_{xyzw}(\theta).$$

The two signs represent the same orientation, so the conversion does not force one canonical quaternion sign. For two unit quaternions that differ only by sign, the absolute inner product satisfies $|\mathbf{q}_1^T \mathbf{q}_2| = 1$; this sign-invariant comparison is therefore suitable for the focused equivalence test.

9.15.2 C++ representation and implementation

The public type names the required storage order directly:

```
struct QuaternionXYZW
{
    double x;
    double y;
    double z;
    double w;
};

[[nodiscard]] QuaternionXYZW planar_yaw_to_quaternion(
    double yaw_radians,
    const NumericalPolicy & policy = NumericalPolicy{});
```

The input `yaw_radians` represents θ in radians. After validating the policy and the input angle, the implementation evaluates the two nonzero mathematical components and explicitly normalizes them:

```
double z = std::sin(0.5 * yaw_radians);
double w = std::cos(0.5 * yaw_radians);

const double norm = std::hypot(z, w);

if (!std::isfinite(norm) || norm == 0.0) {
    throw GeometryException(
        GeometryError::unsupported_magnitude,
        "Planar-yaw quaternion norm is outside the supported range.");
}

z /= norm;
w /= norm;

return QuaternionXYZW{0.0, 0.0, z, w};
```

The code variable `norm` represents the complete quaternion norm because `x` and `y` are exactly zero. `std::hypot(z, w)` calculates $\sqrt{z^2 + w^2}$ using a scaled numerical method that avoids unnecessary intermediate overflow or underflow. Dividing `z` and `w` by this norm

removes finite-precision drift from the unit-norm identity before the value crosses the public interface. A non-finite or zero norm cannot be normalized into a valid orientation, so the function reports `GeometryError::unsupported_magnitude` instead of returning a valid-looking quaternion.

The function first requires `maximum_exponential_angle` to be finite and at least π . Although this conversion does not calculate a matrix exponential, the shared policy member supplies the kernel's upper bound $\theta_{\max}$ for any angle passed to a trigonometric function. The function then rejects a non-finite `yaw_radians` as `non_finite` and rejects a finite angle satisfying $|\theta| > \theta_{\max}$ as `unsupported_magnitude`. It neither wraps the input yaw nor reduces it modulo 2π before evaluation.

9.15.3 Focused verification and deliberate limit

The quarter-turn case checks the independent result $\mathbf{q}_{xyzw}(\pi/2) = [0 \ 0 \ \sqrt{2}/2 \ \sqrt{2}/2]^T$ and verifies unit norm. The complete-turn-offset case checks that $\pi/2 + 2\pi$ returns the negative quaternion, again with unit norm, and that the absolute inner product of the two outputs is one. The invalid-policy case supplies both an invalid angular bound and a NaN yaw and verifies that `invalid_policy` is reported first, preserving the public validation order.

This operation intentionally returns a small ROS-order data carrier without depending on ROS runtime types. It does not implement general quaternion construction, multiplication, interpolation, arbitrary Euler-angle conversion, invalid-quaternion repair, or a replacement for `tf2`.

References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapter 3 on axis–angle rotations, body and spatial twists, the adjoint, and the $SE(3)$ exponential, and Appendix B on unit quaternions.
- John J. Craig, *Introduction to Robotics: Mechanics and Control*, fourth edition, Chapter 2 on equivalent axis–angle rotations and Euler-parameter unit quaternions.
- Peter Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, third edition, Chapter 2 on unit quaternions and normalized orientation representations, Section 2.4.8 on twist exponentials, and Chapter 3, Sections 3.1.3–3.1.5 on adjoint velocity conversion and incremental rigid-body motion.

10 General Robot Kinematics and Jacobians

Prerequisites and assumptions

The reader should understand vectors, matrices, rank, null spaces, and constraints from Chapter 1; coordinate frames and cross products from Chapter 2; spatial poses, angular velocity, twists, and the adjoint from Chapter 7; and screw axes, the $SE(3)$ exponential and logarithm, and space-versus-body pose integration from Chapter 8.

This chapter treats fixed-base open serial chains with ideal rigid links and independent one-degree-of-freedom revolute or prismatic joints. Frames are right-handed and orthonormal. Position and length are measured in metres, time in seconds, and angles in radians. The chapter develops forward and velocity kinematics; it does not yet solve inverse kinematics or model joint compliance, backlash, closed kinematic loops, collision, forces, actuator dynamics, or feedback control.

From one joint to a complete robot

Chapter 8 answered a local geometric question: what finite displacement or instantaneous velocity field is produced by one declared screw motion? A serial robot creates a larger problem because several joints move several connected links. The **end effector** is the physical tool or final working link selected to perform the robot's task, and an **end-effector frame** is a coordinate frame rigidly attached to it. Predicting the pose of that frame requires the effects of all upstream joints to be combined in their physical chain order.

Consider a simple arm consisting of a fixed base, a shoulder joint, an upper-arm link, an elbow joint, a forearm link, and a tool frame. Rotating the shoulder moves the upper arm and carries the elbow, forearm, and tool with it. Rotating the elbow changes the forearm and tool relative to their new placement after the shoulder motion, but it does not move the upper arm. The two joint effects therefore cannot be calculated as unrelated pose changes and added: each downstream motion starts from the placement produced by the joints before it.

At one instant, the kinematic model receives the robot's current joint values as an ordered column $\mathbf{q}$. This column is a compact input; it does not directly contain the poses of the

links. The next section defines its entries, units, signs, and allowed values. Given this input, the complete kinematic problem begins with three practical questions.

1. **Pose:** Where is the selected end-effector frame for the current value of $\mathbf{q}$? **Forward kinematics** answers this question by composing the link and joint geometry into an end-effector pose.
2. **Velocity:** If the joints move with rate column $\dot{\mathbf{q}}$, what is the instantaneous velocity of the end effector or another selected task output? A **Jacobian** is the local linear map that converts joint rates into that selected velocity.
3. **Available motion:** Can the joints produce every desired instantaneous task direction at the current configuration? The rank of the relevant Jacobian answers this question. A **kinematic singularity for a declared task** is a configuration at which that Jacobian loses rank and at least one normally available instantaneous task direction is lost.

These questions form a dependency chain. Forward kinematics must first define how joint coordinates determine a pose or another task output. Differentiating that forward relationship produces velocity kinematics and its Jacobian. Analysing the Jacobian then reveals available motion directions, joint motions that produce no first-order task change, and singular configurations. Inverse kinematics, trajectory generation, and control use these results later; they cannot be stated reliably before the forward map and its derivative are understood.

Chapter 8 supplies the transformation, screw-axis, exponential, logarithm, adjoint, and twist tools for individual rigid motions. This chapter assembles those tools at robot scale. It first distinguishes complete configurations, joint-coordinate descriptions, and selected task outputs. It then constructs serial-chain poses using transform chains and products of exponentials, derives task and rigid-body Jacobians, and finishes by examining rank, singularities, numerical conditioning, and verification checks.

10.1 Configuration coordinates and forward kinematics

The motivation above identified the robot-level forward problem: given one value for every joint coordinate, where is the selected end-effector frame or task point? This section first clarifies the coordinate input, then defines the forward output, and finally differentiates that input–output relationship. It does not yet choose a transform-chain or product-of-exponentials formula for evaluating the forward map; those constructions begin in Section 10.2.

10.1.1 From joint values to a robot configuration

Forward kinematics needs a compact description of the robot’s current arrangement. The complete configuration $\chi \in \mathcal{C}$ from Section 8.1 answers where every modelled material

point is, but asking a user or program to supply all those point positions would be impractical. A fixed-base open chain instead needs one current value for each independent joint; the fixed link geometry then determines the rest of the arrangement.

Let $n \in \mathbb{N}$ be the number of independent one-degree-of-freedom joints, and let $j \in \{1, \dots, n\}$ identify one joint. The scalar q_j is the current motion of joint j measured from its declared zero value. For a revolute joint, q_j is an angle in radians and positive rotation follows the right-hand rule about the declared positive joint axis. For a prismatic joint, q_j is a displacement in metres and positive translation follows the declared positive joint axis.

Collect the values in physical joint order:

$$\mathbf{q} := [q_1 \ \cdots \ q_n]^\top \in \mathbb{R}^n.$$

The column $\mathbf{q}$ is called the **joint-coordinate column**, or the **generalised-coordinate column**. The word *generalised* distinguishes these coordinates from the three spatial coordinates of one point: the entries describe allowed joint motions and may have different units. In a chain containing both joint types, $\mathbf{q}$ therefore has no single physical unit.

For a concrete two-joint example, let joint 1 be revolute and joint 2 be prismatic. The input

$$\mathbf{q} = \begin{bmatrix} \pi/4 \text{ rad} \\ 0.20 \text{ m} \end{bmatrix}$$

means that joint 1 is rotated by $\pi/4$ rad in its positive direction and joint 2 is extended by 0.20 m in its positive direction, both measured from their declared zero values. These two numbers do not themselves contain any link pose.

Each coordinate has an allowed set $\mathcal{Q}_j$. A revolute joint limit gives an angular interval; a prismatic joint limit gives a displacement interval. Because this chapter considers independent joints and excludes collision constraints, the set of all allowed input columns is the Cartesian product

$$\mathcal{Q} := \mathcal{Q}_1 \times \cdots \times \mathcal{Q}_n = \left\{ \mathbf{q} \mid q_j \in \mathcal{Q}_j \text{ for every } j \in \{1, \dots, n\} \right\}.$$

For the two-joint example, suppose

$$\mathcal{Q}_1 = [-\pi/2, \pi/2] \text{ rad}, \quad \mathcal{Q}_2 = [0, 0.40] \text{ m}.$$

Then the example column belongs to $\mathcal{Q} = \mathcal{Q}_1 \times \mathcal{Q}_2$. Thus $\mathcal{Q}$ means the set of every allowed numerical input, whereas $\mathbf{q}$ means one selected input from that set.

An unrestricted revolute joint needs one additional representation rule. The angles q_j and $q_j + 2\pi$ produce the same relative joint orientation. Software must therefore declare whether it keeps an unwrapped real angle or wraps the angle into a chosen interval. This choice changes the numerical representatives in $\mathcal{Q}_j$ but does not create another physical degree of freedom.

Once $\mathbf{q}$ is supplied, the fixed link geometry, joint axes, positive directions, and zero arrangement determine the complete configuration:

$$\boxed{\mathbf{q} \in \mathcal{Q} \xrightarrow{\text{robot model}} \chi(\mathbf{q}) \in \mathcal{C}.}$$

The three objects answer different questions: $\mathbf{q}$ gives the current joint values, $\mathcal{Q}$ lists which joint-value columns are allowed, and $\chi(\mathbf{q})$ gives the resulting placement of every modelled robot point. Different angle representatives can describe the same physical placement, so the coordinate-to-configuration map need not be one-to-one over the complete numerical domain.

When the robot moves, its joint-coordinate column varies with time. Let $I \subseteq \mathbb{R}$ be a time interval measured in seconds and write the motion as

$$\mathbf{q} : I \rightarrow \mathcal{Q}, \quad t \mapsto \mathbf{q}(t).$$

At a time t where every coordinate is differentiable, the **generalised-rate column** is

$$\boxed{\dot{\mathbf{q}}(t) := \frac{d\mathbf{q}}{dt}(t) = [\dot{q}_1(t) \ \cdots \ \dot{q}_n(t)]^\top.}$$

A revolute entry of $\dot{\mathbf{q}}$ is measured in rad s^{-1} , while a prismatic entry is measured in m s^{-1} . The column $\dot{\mathbf{q}}$ describes how the joint coordinates change; it is not yet the velocity of any material point or frame. When differentiating an unrestricted revolute coordinate, use a locally continuous angle representative: an artificial jump caused only by wrapping $+\pi$ to $-\pi$ is not a physical joint velocity.

10.1.2 Task variables and the forward map

The complete configuration $\chi(\mathbf{q})$ locates every modelled material point, but a calculation usually asks for a smaller output. Examples include the pose of an end-effector frame, the position of a tool point, or the distance between two selected points. A **task output** is the particular configuration-dependent quantity chosen for the calculation.

For the selected end-effector frame $\{e\}$ and fixed space frame $\{s\}$, Section 8.1 defined the forward-pose map

$$\boxed{\mathbf{F}_e : \mathcal{Q} \rightarrow SE(3), \quad \mathbf{q} \mapsto \mathbf{F}_e(\mathbf{q}) := \mathbf{T}_{se}(\mathbf{q})}.$$

The map $\mathbf{F}_e$ is the rule for all admissible coordinate inputs. Its value $\mathbf{T}_{se}(\mathbf{q})$ is one homogeneous transformation matrix: it represents the pose of frame $\{e\}$ relative to frame $\{s\}$ at the selected coordinate column $\mathbf{q}$. Forward kinematics calculates the end effector's position and orientation from known joint values. It does not calculate the joint values needed to reach a requested position and orientation.

A task may use only part of this pose. Define the position-extraction map

$$\pi_p : SE(3) \rightarrow \mathbb{R}^3, \quad \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \mapsto \pi_p \left(\begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \right) := \mathbf{p}.$$

The end-effector-origin position task is then the composition

$$\boxed{\mathbf{f}_p := \pi_p \circ \mathbf{F}_e : \mathcal{Q} \rightarrow \mathbb{R}^3, \quad \mathbf{q} \mapsto {}^s\mathbf{p}_e(\mathbf{q})}.$$

Every output column ${}^s\mathbf{p}_e(\mathbf{q})$ is expressed in frame $\{s\}$ and measured in metres. The pose map and the position-task map answer different questions: $\mathbf{F}_e$ returns both orientation and position, whereas $\mathbf{f}_p$ discards orientation deliberately.

The position map $\mathbf{f}_p$ selects one particular output from the robot configuration. More generally, a task may select a different collection of numerical outputs. To describe any such choice, let $m \in \mathbb{N}$ be the number of numerical components used to describe the selected task output, and let $\mathcal{Y} \subseteq \mathbb{R}^m$ be the set of their allowed columns. A **task-coordinate map** is a function

$$\boxed{\mathbf{f} : \mathcal{Q} \rightarrow \mathcal{Y}, \quad \mathbf{q} \mapsto \mathbf{y} := \mathbf{f}(\mathbf{q})}.$$

The position map is the special case obtained by choosing $m = 3$, $\mathcal{Y} = \mathbb{R}^3$, $\mathbf{f} = \mathbf{f}_p$, and

$$\mathbf{y} = {}^s\mathbf{p}_e = [x_e \ y_e \ z_e]^\top.$$

The bold symbol $\mathbf{y}$ names the complete task-output column; it is not the scalar Cartesian coordinate y_e . In this special case the task output happens to be a position, so the same column is also denoted by the more specific position symbol ${}^s\mathbf{p}_e$.

Here $\mathbf{y} = [y_1 \ \cdots \ y_m]^\top$ is one task-coordinate column. Each entry y_i is one number in that output column and is called a **task variable**, or **task coordinate**. For example, if the task output is a point's planar position, its two task variables can be its x - and y -coordinates. Each task variable depends on $\mathbf{q}$ through $y_i = f_i(\mathbf{q})$. The task variables are outputs of the forward map, not additional independent robot coordinates. Their units, reference

frames, and physical meanings must be declared. The dimension m is chosen by the task and need not equal the number n of generalised coordinates.

Example: Orientation as a task output

The generic column $\mathbf{y}$ can also describe orientation. Consider one revolute joint whose angle θ rotates a tool in the x_s - y_s plane. If the tool is aligned with the positive x_s -axis when $\theta = 0$, its planar orientation angle relative to that axis is $\phi_e = \theta$. An orientation-only task can therefore use

$$\mathbf{f}_\phi : \mathcal{Q} \rightarrow \mathcal{Y}, \quad \mathbf{q} = [\theta] \mapsto \mathbf{y} = [\phi_e] = [\theta].$$

Here $m = 1$, and both θ and ϕ_e are measured in radians with positive rotation about the positive z_s -axis. For example, $\mathbf{q} = [\pi/6 \text{ rad}]$ produces the orientation output $\mathbf{y} = [\pi/6 \text{ rad}]$. In this task, $\mathbf{y}$ contains orientation and no position.

Example: Two-coordinate position task

Consider an x - y positioning stage with two prismatic joints, so the selected task has $m = 2$ output components. Joint 1 moves the tool along the positive x_s -axis by d_x , and joint 2 moves it along the positive y_s -axis by d_y . At zero joint displacement, the tool position is $(x_0, y_0) = (0.50, 0.30) \text{ m}$. The joint-coordinate input and planar-position task output are

$$\mathbf{f}_{xy} : \mathcal{Q} \rightarrow \mathcal{Y}, \quad \mathbf{q} = \begin{bmatrix} d_x \\ d_y \end{bmatrix} \mapsto \mathbf{y} = \begin{bmatrix} x_e \\ y_e \end{bmatrix} = \begin{bmatrix} x_0 + d_x \\ y_0 + d_y \end{bmatrix}.$$

Suppose $d_x \in [0, 0.40] \text{ m}$ and $d_y \in [0, 0.20] \text{ m}$. Then $\mathcal{Q} = [0, 0.40] \times [0, 0.20]$ is the numerical domain of the joint-displacement column, with both entries measured in metres, and $\mathcal{Y} = [0.50, 0.90] \times [0.30, 0.50]$ is the numerical domain of the tool-position column, again in metres. For example,

$$\mathbf{q} = \begin{bmatrix} 0.20 \\ 0.10 \end{bmatrix} \text{ m} \mapsto \mathbf{y} = \begin{bmatrix} 0.70 \\ 0.40 \end{bmatrix} \text{ m}.$$

The output has two task variables, $y_1 = x_e$ and $y_2 = y_e$, so $m = 2$. Both values are calculated from the joint variables; they are not additional independently chosen robot coordinates.

The full modelling chain is therefore

$$\mathbf{q} \mapsto \chi(\mathbf{q}) \mapsto \mathbf{T}_{se}(\mathbf{q}) \mapsto \mathbf{y}(\mathbf{q}).$$

The last arrow is present only when the selected task is obtained from the end-effector pose. Some tasks depend on another link, several points, or the complete configuration

instead. In every case, the final coordinate rule can still be written as $\mathbf{y} = \mathbf{f}(\mathbf{q})$ after the physical quantity and its coordinate representation have been declared.

Example: Planar 2R forward map
Consider a planar arm with two revolute joints, link lengths $L_1, L_2 \in \mathbb{R}_{>0}$ measured in metres, and joint coordinates

$$\mathbf{q} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}.$$

The first joint axis passes through O_s , both joint axes point along positive z_s , and the end-effector origin is at the distal end of link 2. In the zero arrangement, both links point along positive x_s . The coordinate θ_1 rotates link 1 relative to the fixed base, while θ_2 rotates link 2 relative to link 1. Consequently, link 1 makes angle θ_1 with positive x_s and link 2 makes the accumulated angle $\theta_1 + \theta_2$ with positive x_s . The position of the end-effector origin in the x_s - y_s plane is therefore the sum of the two link-displacement columns:

$$\begin{aligned} {}^s\ell_1(\theta_1) &= L_1 \begin{bmatrix} \cos \theta_1 \\ \sin \theta_1 \end{bmatrix}, \\ {}^s\ell_2(\theta_1, \theta_2) &= L_2 \begin{bmatrix} \cos(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) \end{bmatrix}. \end{aligned}$$

Here ${}^s\ell_1$ and ${}^s\ell_2$ are the first- and second-link displacement columns, expressed in frame $\{s\}$ and measured in metres. Adding them gives ${}^s\mathbf{p}_e = {}^s\ell_1 + {}^s\ell_2$ and therefore the two-coordinate position task

$$\mathbf{y} = \mathbf{f}_{xy}(\mathbf{q}) = \begin{bmatrix} x_e(\mathbf{q}) \\ y_e(\mathbf{q}) \end{bmatrix} = \begin{bmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix} \in \mathbb{R}^2. \quad (10.1)$$

Both entries of $\mathbf{y}$ are expressed in frame $\{s\}$ and measured in metres. If orientation also matters, the planar end-effector angle is

$$\phi_e(\mathbf{q}) = \theta_1 + \theta_2,$$

and a separate planar pose-coordinate task could use $[x_e \ y_e \ \phi_e]^\top$. That column has two entries in metres and one in radians; it is a coordinate description of a planar pose, not a spatial position vector.

Different task choices preserve different information. Two joint configurations can produce the same end-effector position while giving different link placements or end-effector orientations. Therefore a task-coordinate map need not be one-to-one, and knowing $\mathbf{y}$ need not recover $\mathbf{q}$. This chapter first solves and differentiates the forward direction; inverse kinematics is developed later in the learning programme.

10.1.3 The total derivative and multivariable chain rule

The forward map answers where the task output is at one configuration. Velocity kinematics asks the local follow-up question: if every joint coordinate changes at a known rate, how does the selected task output change at that instant? The required mathematical object is the total derivative of the task-coordinate map.

10.1.3.1 Partial and total derivatives

The earlier task-coordinate map $\mathbf{f} : \mathcal{Q} \rightarrow \mathcal{Y}$ accepts every declared joint-coordinate column in its domain. In the declared numerical representation, $\mathcal{Q} \subseteq \mathbb{R}^n$. Differentiation is a local calculation, so choose a smaller set $\mathcal{V} \subseteq \mathcal{Q}$ that contains nearby allowed columns without crossing a joint-limit or angle-wrap boundary. Thus $\mathcal{V} \subseteq \mathcal{Q} \subseteq \mathbb{R}^n$. Because $\mathcal{Y} \subseteq \mathbb{R}^m$, the restriction of the task map to this local set can be written as $\mathbf{f} : \mathcal{V} \rightarrow \mathbb{R}^m$. Write its component functions as

$$\mathbf{f}(\mathbf{q}) = [f_1(\mathbf{q}) \quad \cdots \quad f_m(\mathbf{q})]^\top.$$

Let $\mathbf{q}_0 \in \mathcal{V}$ be an interior coordinate value, meaning that sufficiently small positive and negative changes of every coordinate remain inside $\mathcal{V}$. The **partial derivative** of output component f_i with respect to input coordinate q_j measures the first-order change in f_i when only q_j changes:

$$\frac{\partial f_i}{\partial q_j}(\mathbf{q}_0) := \lim_{h \rightarrow 0} \frac{f_i(\mathbf{q}_0 + h\mathbf{e}_j) - f_i(\mathbf{q}_0)}{h}.$$

Here $i \in \{1, \dots, m\}$ is an output index, $j \in \{1, \dots, n\}$ is an input index, $\mathbf{e}_j \in \mathbb{R}^n$ is the dimensionless standard coordinate column with entry 1 at index j and zeros elsewhere, and the scalar increment h has the physical unit of q_j . The derivative has the unit of f_i divided by the unit of q_j .

Partial derivatives examine the input directions separately. Their existence alone does not guarantee that all small input changes are described by one consistent linear map. The task map $\mathbf{f}$ is **differentiable at $\mathbf{q}_0$** if there is a linear map, represented in the chosen coordinates by a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, that separates every sufficiently small output increment into a first-order part and a smaller remainder. This means that

$$\mathbf{f}(\mathbf{q}_0 + \Delta\mathbf{q}) - \mathbf{f}(\mathbf{q}_0) = \mathbf{A}\Delta\mathbf{q} + \mathbf{r}(\Delta\mathbf{q}). \quad (10.2)$$

The remainder $\mathbf{r}(\Delta\mathbf{q})$ must decrease faster than the size of $\Delta\mathbf{q}$ as $\Delta\mathbf{q} \rightarrow \mathbf{0}_n$.¹ When such a linear map exists, it is unique and is called the **total derivative** of $\mathbf{f}$ at $\mathbf{q}_0$, denoted $D\mathbf{f}(\mathbf{q}_0) := \mathbf{A}$. Thus $D\mathbf{f}(\mathbf{q}_0)$ is the local linear model of the generally nonlinear forward map; it does not claim that $\mathbf{f}$ is globally linear.

If $\mathbf{f}$ is differentiable at $\mathbf{q}_0$, the matrix of its total derivative in the chosen input and output coordinates contains the partial derivatives:

$$D\mathbf{f}(\mathbf{q}_0) = \left[\frac{\partial f_i}{\partial q_j}(\mathbf{q}_0) \right]_{\substack{i=1,\dots,m \\ j=1,\dots,n}} \in \mathbb{R}^{m \times n}. \quad (10.3)$$

Row i collects the first-order effect on task component y_i from every input coordinate. Column j collects the first-order effects on all task components from changing q_j while holding the other input coordinates fixed. Because different inputs and outputs may have different units, different matrix entries may also have different units.

Example: Partial and total derivatives of an x–y stage

Return to the two-prismatic-joint positioning stage, with every coordinate measured in metres. Suppose its complete numerical domain is $\mathcal{Q} = [0, 0.40] \times [0, 0.20]$ and the derivative is required at

$$\mathbf{q}_0 = \begin{bmatrix} 0.20 \\ 0.10 \end{bmatrix}.$$

One suitable local set is $\mathcal{V} = (0.10, 0.30) \times (0.05, 0.15)$, so $\mathbf{q}_0 \in \mathcal{V} \subseteq \mathcal{Q}$. This set contains nearby values around $\mathbf{q}_0$ but does not reach either joint limit. The joint input and task output are

$$\mathbf{q} = \begin{bmatrix} d_x \\ d_y \end{bmatrix}, \quad \mathbf{f}_{xy}(\mathbf{q}) = \begin{bmatrix} x_0 + d_x \\ y_0 + d_y \end{bmatrix}.$$

Changing d_x moves only x_e , while changing d_y moves only y_e . Therefore

$$\frac{\partial x_e}{\partial d_x} = 1, \quad \frac{\partial x_e}{\partial d_y} = 0, \quad \frac{\partial y_e}{\partial d_x} = 0, \quad \frac{\partial y_e}{\partial d_y} = 1,$$

and the total-derivative matrix is

¹The formal limit must compare dimensionless numerical columns. Choose fixed positive input and output scales with the corresponding physical units, and divide each input increment and output remainder component by its scale. If $\widetilde{\Delta\mathbf{q}}$ and $\widetilde{\mathbf{r}}$ denote these dimensionless columns, the required condition is $\|\widetilde{\mathbf{r}}\|_2 / \|\widetilde{\Delta\mathbf{q}}\|_2 \rightarrow 0$ as $\widetilde{\Delta\mathbf{q}} \rightarrow \mathbf{0}_n$. This nondimensionalisation does not assign one physical unit or one unscaled physical norm to a mixed-unit column.

$$\mathbf{Df}_{xy}(\mathbf{q}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

For the joint increment

$$\Delta \mathbf{q} = \begin{bmatrix} 0.01 \\ -0.02 \end{bmatrix} \text{ m},$$

the task increment is

$$\Delta \mathbf{y} = \mathbf{Df}_{xy}(\mathbf{q}) \Delta \mathbf{q} = \begin{bmatrix} 0.01 \\ -0.02 \end{bmatrix} \text{ m}.$$

The first column says that the x_s task coordinate changes by one metre per metre of positive x -joint motion and that the y_s task coordinate does not change. The second column gives the corresponding effect of the y joint. This map is affine, so its remainder is exactly zero for every increment that remains in the declared domain.

10.1.3.2 From joint rates to task rates

Assume that $\mathbf{q}(t)$ is a differentiable coordinate curve and that $\mathbf{f}$ is differentiable at every value $\mathbf{q}(t)$ considered. Define the time-dependent task output by

$$\mathbf{y}(t) := \mathbf{f}(\mathbf{q}(t)).$$

Over a small time increment Δt , differentiability of the coordinate curve gives

$$\mathbf{q}(t + \Delta t) = \mathbf{q}(t) + \dot{\mathbf{q}}(t) \Delta t + \mathbf{r}_q(\Delta t),$$

where the coordinate remainder satisfies $\mathbf{r}_q(\Delta t)/\Delta t \rightarrow \mathbf{0}_n$ component by component as $\Delta t \rightarrow 0$. Each component of $\mathbf{r}_q$ has the same unit as its corresponding coordinate. Substitute this coordinate change into the first-order model Equation 10.2. The corresponding task change has the form

$$\mathbf{y}(t + \Delta t) - \mathbf{y}(t) = \mathbf{Df}(\mathbf{q}(t)) \dot{\mathbf{q}}(t) \Delta t + \mathbf{r}_y(\Delta t),$$

where $\mathbf{r}_y(\Delta t)/\Delta t \rightarrow \mathbf{0}_m$ component by component and each component of $\mathbf{r}_y$ has the same unit as its corresponding task output. Dividing by Δt and taking the limit $\Delta t \rightarrow 0$ makes the remainder quotient vanish and gives the **multivariable chain rule along the robot motion**:

$$\dot{\mathbf{y}}(t) = \mathbf{D}\mathbf{f}(\mathbf{q}(t))\dot{\mathbf{q}}(t). \quad (10.4)$$

In scalar component form, the same rule is

$$\dot{y}_i(t) = \sum_{j=1}^n \frac{\partial f_i}{\partial q_j}(\mathbf{q}(t))\dot{q}_j(t), \quad i \in \{1, \dots, m\}.$$

This equation explains the word *total*: the rate of one task component is the sum of the contributions from all changing joint coordinates. At a fixed configuration $\mathbf{q}$, the relationship is linear in $\dot{\mathbf{q}}$; as the robot changes configuration, the derivative matrix generally changes as well.

Example: Task rate of the x–y stage

For the two-prismatic-joint stage, $\mathbf{D}\mathbf{f}_{xy}$ is the identity matrix. If

$$\dot{\mathbf{q}} = \begin{bmatrix} 0.05 \\ -0.02 \end{bmatrix} \text{ m s}^{-1},$$

then the chain rule gives

$$\dot{\mathbf{y}} = \mathbf{D}\mathbf{f}_{xy}(\mathbf{q})\dot{\mathbf{q}} = \begin{bmatrix} 0.05 \\ -0.02 \end{bmatrix} \text{ m s}^{-1}.$$

The tool therefore moves at 0.05 m s^{-1} along positive x_s and 0.02 m s^{-1} along negative y_s .

10.1.3.3 Applying the task-rate rule to a nonlinear robot

Example: Planar 2R task rate

For the planar 2R position map Equation 10.1, differentiating each output with respect to each joint angle gives

$$\begin{aligned} \frac{\partial x_e}{\partial \theta_1} &= -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2), & \frac{\partial x_e}{\partial \theta_2} &= -L_2 \sin(\theta_1 + \theta_2), \\ \frac{\partial y_e}{\partial \theta_1} &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2), & \frac{\partial y_e}{\partial \theta_2} &= L_2 \cos(\theta_1 + \theta_2). \end{aligned}$$

The total-derivative matrix is therefore

$$\mathbf{Df}_{xy}(\mathbf{q}) = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}. \quad (10.5)$$

Each column is the end-effector position rate produced per unit rate of one joint while the other joint rate is zero. This particular total derivative will later be called the **position-task Jacobian**. Applying Equation 10.4 gives

$$\begin{bmatrix} \dot{x}_e \\ \dot{y}_e \end{bmatrix} = \mathbf{Df}_{xy}(\mathbf{q}) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}.$$

For example, let $L_1 = L_2 = 1 \text{ m}$, $\theta_1 = 0$, $\theta_2 = \pi/2$, and

$$\dot{\mathbf{q}} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} \text{ rad s}^{-1}.$$

At this configuration,

$$\mathbf{Df}_{xy}(\mathbf{q}) = \begin{bmatrix} -1 & -1 \\ 1 & 0 \end{bmatrix} \text{ m rad}^{-1},$$

so the end-effector position rate is

$$\begin{bmatrix} \dot{x}_e \\ \dot{y}_e \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} = \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} \text{ m s}^{-1}.$$

The units show the role of the derivative: multiplying metres per radian by radians per second produces metres per second. The result is the velocity of the selected end-effector origin expressed along the x_s and y_s axes; it is not the velocity of every point on the end-effector link.

10.1.3.4 Composition of coordinate maps

The same chain rule applies when one coordinate mapping feeds another. Let $p \in \mathbb{N}$ be the number of upstream coordinates, let $\mathbf{u} \in \mathbb{R}^p$ be an upstream coordinate column, let $\mathbf{q} = \mathbf{g}(\mathbf{u})$, and let $\mathbf{y} = \mathbf{f}(\mathbf{q})$. The map $\mathbf{g}$ could, for example, describe a transmission that converts actuator coordinates $\mathbf{u}$ into joint coordinates $\mathbf{q}$. Assume that $\mathbf{g}$ is differentiable at $\mathbf{u}$ and that $\mathbf{f}$ is differentiable at $\mathbf{g}(\mathbf{u})$. Write g_j for component j of $\mathbf{g}$, and let $k \in \{1, \dots, p\}$ be an upstream-coordinate index. The composed map is $\mathbf{y} = (\mathbf{f} \circ \mathbf{g})(\mathbf{u})$. For output index i and upstream index k ,

$$\frac{\partial y_i}{\partial u_k}(\mathbf{u}) = \sum_{j=1}^n \frac{\partial f_i}{\partial q_j}(\mathbf{g}(\mathbf{u})) \frac{\partial g_j}{\partial u_k}(\mathbf{u}).$$

Collecting these scalar equations gives the matrix form

$$\boxed{D(\mathbf{f} \circ \mathbf{g})(\mathbf{u}) = D\mathbf{f}(\mathbf{g}(\mathbf{u}))D\mathbf{g}(\mathbf{u})}. \quad (10.6)$$

Here $D\mathbf{g}(\mathbf{u}) \in \mathbb{R}^{n \times p}$ first maps an upstream increment into a joint-coordinate increment, and $D\mathbf{f}(\mathbf{g}(\mathbf{u})) \in \mathbb{R}^{m \times n}$ then maps that joint-coordinate increment into a task increment. Their product belongs to $\mathbb{R}^{m \times p}$. The multiplication order therefore follows the information flow.

Example: A geared orientation task

Consider one motor with angle u connected to one revolute joint by a reduction ratio $N \in \mathbb{R}_{>0}$. Both angles use the same positive direction, and the transmission gives

$$q = g(u) = \frac{u}{N}.$$

Suppose the tool's planar orientation is the selected task and its zero orientation agrees with the joint zero, so

$$\phi_e = f(q) = q.$$

This example has $p = n = m = 1$. The two derivatives are

$$\frac{dg}{du} = \frac{1}{N}, \quad \frac{df}{dq} = 1.$$

The composition rule therefore gives

$$\boxed{\frac{d\phi_e}{du} = \frac{df}{dq} \frac{dg}{du} = 1 \left(\frac{1}{N} \right) = \frac{1}{N}}.$$

For $N = 4$ and motor rate $\dot{u} = 2 \text{ rad s}^{-1}$, the tool orientation rate is $\dot{\phi}_e = (1/4)\dot{u} = 0.5 \text{ rad s}^{-1}$. In the derivative product, the rightmost factor dg/du acts first and maps motor changes to joint changes; the factor df/dq then maps joint changes to task-orientation changes.

10.1.3.5 Pose-valued forward maps

For a pose-valued forward map $\mathbf{F}_e : \mathcal{Q} \rightarrow SE(3)$, the output is constrained to $SE(3)$ rather than being an unconstrained Euclidean column. Differentiating its matrix entries produces $\dot{\mathbf{T}}_{se}$, but $\dot{\mathbf{T}}_{se}$ is not itself a six-component rigid velocity. The established relations

$$\widehat{\boldsymbol{\xi}}_s = \dot{\mathbf{T}}_{se} \mathbf{T}_{se}^{-1}, \quad \widehat{\boldsymbol{\xi}}_e = \mathbf{T}_{se}^{-1} \dot{\mathbf{T}}_{se}$$

convert the same pose rate into the space twist ξ_s , expressed using the fixed frame $\{s\}$ and its origin, or the body twist ξ_e , expressed using the moving end-effector frame $\{e\}$ and its origin. Both use the project's linear-first component order. The later velocity-kinematics section will combine these relations with $\dot{\mathbf{q}}$ to construct space and body robot Jacobians.

The derivative is local. It predicts first-order output change for a small coordinate increment and gives the exact instantaneous output rate along a differentiable motion; it does not by itself give an exact finite displacement, enforce joint limits, resolve angle-wrap boundaries, or prove that an inverse map exists. Equal input and output dimensions are also insufficient for local invertibility: the derivative must have the required rank, which will be studied in Section 10.4.

Quick test A: coordinates, forward maps, and total derivatives

1. **Recall:** Distinguish $\mathcal{C}$, $\mathcal{Q}$, and one coordinate column $\mathbf{q}$.
2. **Explanation:** Why can two real angle values that differ by 2π describe the same revolute-joint configuration, and why does this make coordinates local?
3. **Task choice:** For one forward-pose map $\mathbf{F}_e$, give two different task-coordinate maps and state what information each discards.
4. **Definition:** Partial derivatives examine one joint-coordinate direction at a time. Why is that not enough to conclude that $\mathbf{f}$ has a total derivative at $\mathbf{q}_0$? State the condition that makes $D\mathbf{f}(\mathbf{q}_0)$ a valid first-order model for every sufficiently small combined increment $\Delta\mathbf{q}$.
5. **Derivation:** Starting from $\mathbf{y}(t) = \mathbf{f}(\mathbf{q}(t))$, write the scalar multivariable chain rule for $\dot{y}_i$ and its matrix form.
6. **Application:** For the planar 2R example with $L_1 = 1$ m, $L_2 = 0.5$ m, $\theta_1 = \pi/2$, $\theta_2 = -\pi/2$, and $\dot{\mathbf{q}} = [0.3 \ -0.1]^\top$ rad s⁻¹, calculate $[\dot{x}_e \ \dot{y}_e]^\top$ and state its frame and units.
7. **Composition:** If actuator coordinates $\mathbf{u}$ produce joint coordinates $\mathbf{q} = \mathbf{g}(\mathbf{u})$ and the task is $\mathbf{y} = \mathbf{f}(\mathbf{q})$, which derivative matrix multiplies an actuator-rate column $\dot{\mathbf{u}}$?
8. **Limitation:** Why does the first-order approximation $\Delta\mathbf{y} \approx D\mathbf{f}(\mathbf{q})\Delta\mathbf{q}$ not provide an exact finite-motion formula or guarantee a local inverse?

Answers and hints

1. $\mathcal{C}$ is the set of complete admissible robot placements. $\mathcal{Q}$ is the declared domain of their generalised-coordinate descriptions. $\mathbf{q}$ is one coordinate column in $\mathcal{Q}$; together with the model it locally specifies one configuration.
2. A complete turn returns a revolute joint to the same relative orientation, so θ and $\theta + 2\pi$ are different numerical representatives of one physical arrangement. Any one-to-one real coordinate must select a local interval or a branch boundary; it cannot unwrap the whole circle globally without duplication.

3. The position map $\mathbf{f}_p = \pi_p \circ \mathbf{F}_e$ returns ${}^s\mathbf{p}_e$ and discards orientation. A declared orientation-coordinate map would retain orientation and discard position. A full local pose-coordinate map can retain both, but its orientation coordinates require a declared representation and may have a representation boundary.
4. Suppose $\mathbf{q} = [q_1 \ q_2]^\top$. The partial derivative with respect to q_1 tests a small change $[h \ 0]^\top$, and the partial derivative with respect to q_2 tests $[0 \ h]^\top$, where in each test h is a small scalar increment with the same unit as the selected coordinate. These tests show what happens when one coordinate changes alone. A general increment $\Delta\mathbf{q} = [\Delta q_1 \ \Delta q_2]^\top$ can change both coordinates together, so the matrix of partial derivatives is a total derivative only if the same matrix predicts the output change for every sufficiently small $\Delta\mathbf{q}$. Specifically, $\mathbf{f}(\mathbf{q}_0 + \Delta\mathbf{q}) - \mathbf{f}(\mathbf{q}_0) = \mathbf{Df}(\mathbf{q}_0)\Delta\mathbf{q} + \mathbf{r}(\Delta\mathbf{q})$ must hold with a remainder $\mathbf{r}(\Delta\mathbf{q})$ that becomes negligible compared with $\Delta\mathbf{q}$ as $\Delta\mathbf{q} \rightarrow \mathbf{0}_2$.
5. For every output index i , $\dot{y}_i = \sum_{j=1}^n (\partial f_i / \partial q_j) \dot{q}_j$. Collecting the components gives $\dot{\mathbf{y}} = \mathbf{Df}(\mathbf{q})\dot{\mathbf{q}}$.
6. At this configuration, $\theta_1 + \theta_2 = 0$ and the derivative matrix is $\begin{bmatrix} -1 & 0 \\ 0.5 & 0.5 \end{bmatrix} \text{ m rad}^{-1}$. Therefore the position rate is $\begin{bmatrix} -0.3 & 0.1 \end{bmatrix}^\top \text{ m s}^{-1}$. It is the velocity of the selected end-effector origin expressed along the x_s and y_s axes.
7. Apply the composition chain rule: $\dot{\mathbf{y}} = \mathbf{Df}(\mathbf{g}(\mathbf{u}))\mathbf{Dg}(\mathbf{u})\dot{\mathbf{u}}$. The rightmost matrix acts first, and the product $\mathbf{Df}\mathbf{Dg}$ is the derivative of the actuator-to-task map.
8. The derivative retains only the linear term at the current configuration; second- and higher-order terms can become significant over a finite coordinate change. A local inverse is a rule that recovers the unique nearby joint-coordinate column $\mathbf{q}$ from a nearby task output $\mathbf{y} = \mathbf{f}(\mathbf{q})$. Such a two-sided local inverse requires equal input and output dimensions, $n = m$, and a full-rank derivative matrix $\mathbf{Df}(\mathbf{q})$ at the selected configuration. Even then, the inverse is valid only in a neighbourhood that does not cross a joint limit, an angle-wrap boundary, or another excluded part of the physical coordinate domain.

10.2 Serial-chain pose through products of exponentials

The forward-pose map $\mathbf{F}_e$ states what the robot model must calculate, but it does not yet supply a formula for that calculation. A fixed-base serial chain needs a rule that combines the joint motions in their physical order while preserving their effects on every outward link. The product-of-exponentials construction answers this question from one declared zero arrangement, one end-effector pose in that arrangement, and one screw axis for each joint. This section first defines those stored geometric data, then constructs the space and body forms, and finally connects the exponential products to the familiar chain of relative link transformations.

10.2.1 Home configuration and joint screw axes

The reference data must describe the whole chain before any nonzero joint value is applied. Let

$$\mathbf{0}_n := [0 \ \cdots \ 0]^\top \in \mathbb{R}^n$$

be the reference column in which every joint coordinate has its declared zero value. The rigid-link geometry assigned to this column is the **zero reference arrangement**, shortened below to the **zero arrangement**. This reference is chosen by the modeller; it need not resemble a person's resting posture, place every link along one straight line, or make the end-effector pose equal to the identity. A physical joint stop or a deliberately tighter operating interval may make $\mathbf{0}_n \notin \mathcal{Q}$. In that case the zero reference arrangement still defines the product-of-exponentials geometry, but it is not an admissible robot command. We write $\mathbf{T}_{se}(\mathbf{0}_n)$ for this reference value even in that case; the forward map $\mathbf{F}_e : \mathcal{Q} \rightarrow SE(3)$ remains restricted to admissible coordinate columns.

The product-of-exponentials model first records the end-effector pose in this zero arrangement. Define the **home end-effector pose**, commonly called the **home configuration** in product-of-exponentials terminology, by

$$\mathbf{M} := \mathbf{T}_{se}(\mathbf{0}_n) = \begin{bmatrix} \mathbf{R}_{se,0} & {}^s\mathbf{p}_{e,0} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3).$$

Here $\mathbf{R}_{se,0} \in SO(3)$ is the orientation of the end-effector frame $\{e\}$ relative to the fixed space frame $\{s\}$ in the zero arrangement, and ${}^s\mathbf{p}_{e,0} \in \mathbb{R}^3$ is the position of the end-effector origin O_e relative to O_s , expressed along the axes of $\{s\}$ and measured in metres. The matrix $\mathbf{M}$ contains only the end-effector pose at the reference column; it does not contain the reference placement of every robot link.

The home pose $\mathbf{M}$ records where the end effector is when $q_1 = 0$, but it does not record the motion permitted by the joint. For a one-joint chain, let ${}^s\mathbf{S}_1$ be the joint's normalised screw axis expressed using the space frame $\{s\}$. This fixed screw-axis column is not an instantaneous twist. If the joint coordinate follows $q_1(t)$, then its rate produces the space twist

$$\xi_{s,1} = {}^s\mathbf{S}_1 \dot{q}_1.$$

Thus ${}^s\mathbf{S}_1$ describes the permitted motion per unit joint motion, whereas $\xi_{s,1}$ describes the instantaneous velocity produced by the current rate $\dot{q}_1$. When the joint stops, $\dot{q}_1 = 0$ and the twist is zero, but the stored screw axis remains unchanged.

Forward kinematics requires the finite displacement produced by the coordinate q_1 , rather than the instantaneous velocity produced by $\dot{q}_1$. Define the corresponding finite exponential-coordinate column by

$${}^s\boldsymbol{\eta}_1 := {}^s\mathbf{S}_1 q_1.$$

The $SE(3)$ exponential from Section 8.3 converts this finite column into the active joint displacement

$$\mathbf{T}_{\Delta,1}(q_1) = \exp(\widehat{{}^s\boldsymbol{\eta}_1}) = \exp(\widehat{{}^s\mathbf{S}_1} q_1).$$

Applying that displacement to the home pose gives

$$\boxed{\mathbf{T}_{se}(q_1) = \mathbf{T}_{\Delta,1}(q_1)\mathbf{M} = \exp(\widehat{{}^s\mathbf{S}_1} q_1)\mathbf{M}.}$$

The displacement appears on the left of $\mathbf{M}$ because the two transformations answer successive questions about a point fixed to the end effector. Let $\bar{\mathbf{p}}_e = [\mathbf{p}_e^\top 1]^\top$ be the homogeneous coordinates of any such point in $\{e\}$. In the zero arrangement, the home pose first places the point in the space frame:

$$\bar{\mathbf{p}}_s(0) = \mathbf{M}\bar{\mathbf{p}}_e.$$

The active displacement $\mathbf{T}_{\Delta,1}(q_1)$ is expressed in the fixed space frame, so it next moves that already placed point:

$$\begin{aligned}\bar{\mathbf{p}}_s(q_1) &= \mathbf{T}_{\Delta,1}(q_1)\bar{\mathbf{p}}_s(0) \\ &= \mathbf{T}_{\Delta,1}(q_1)\mathbf{M}\bar{\mathbf{p}}_e.\end{aligned}$$

Because this relation holds for every point fixed to the end effector, the complete moved pose is $\mathbf{T}_{se}(q_1) = \mathbf{T}_{\Delta,1}(q_1)\mathbf{M}$. Matrix products act on a column from right to left: $\mathbf{M}$ places the home geometry first, and the space-frame joint displacement moves that geometry second. Reversing the factors would apply a different displacement interpretation and would not represent the stated motion about the fixed space screw axis.

Example: Rotating a home position about the space origin

This example shows that changing the multiplication order changes the physical motion. Suppose the home pose has identity orientation and end-effector position ${}^s\mathbf{p}_{e,0} = [2 \ 0 \ 0]^\top$ m, and suppose the joint displacement is a rotation $\mathbf{R}_z(\pi/2)$ about the z -axis through O_s . Left multiplication gives

$$\mathbf{T}_{\Delta,1} = \begin{bmatrix} \mathbf{R}_z(\pi/2) & \mathbf{0}_3 \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \mathbf{M} = \begin{bmatrix} \mathbf{I}_3 & \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

and therefore

$$\mathbf{T}_{\Delta,1}\mathbf{M} = \begin{bmatrix} \mathbf{R}_z(\pi/2) & \mathbf{0}_3 \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I}_3 & \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_z(\pi/2) & \mathbf{R}_z(\pi/2) \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

Since $\mathbf{R}_z(\pi/2)[2 \ 0 \ 0]^\top = [0 \ 2 \ 0]^\top$, the product becomes

$$\mathbf{T}_{\Delta,1}\mathbf{M} = \begin{bmatrix} \mathbf{R}_z(\pi/2) & \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

The end-effector origin therefore moves from $(2, 0, 0)$ m to $(0, 2, 0)$ m as it rotates about the fixed space axis. By contrast, the reversed product $\mathbf{M}\mathbf{T}_{\Delta,1}$ retains the translation $[2 \ 0 \ 0]^\top$ m in this example, so it cannot describe the same motion about O_s .

When $q_1 = 0$, the exponential is $\mathbf{I}_4$ and the result returns to $\mathbf{T}_{se}(0) = \mathbf{M}$. Thus $\mathbf{M}$ answers “where is the end effector when every joint is zero?”, whereas ${}^s\mathbf{S}_1$ answers “which rigid displacement does positive motion of this joint generate from that reference arrangement?”. Neither quantity determines the other; they are complementary model data tied to the same zero arrangement. An n -joint serial chain needs one such screw axis for every joint and must apply the resulting displacements in chain order. Before deriving that ordered product, define the required ${}^s\mathbf{S}_j$ precisely.

The model must next record the motion permitted by each joint. First set the complete joint-coordinate column to $\mathbf{q} = \mathbf{0}_n$. While the robot has this placement, identify the axis line and positive direction of every joint. For each joint index $j \in \{1, \dots, n\}$, express its positive axis direction using the fixed space frame $\{s\}$ and denote the resulting dimensionless unit column by ${}^s\mathbf{s}_j \in \mathbb{R}^3$, so $\|{}^s\mathbf{s}_j\|_2 = 1$. If joint j is revolute, also choose any point on its axis line at $\mathbf{q} = \mathbf{0}_n$ and let ${}^s\mathbf{p}_{\ell,j} \in \mathbb{R}^3$ be that point’s position relative to O_s , expressed in $\{s\}$ and measured in metres. Reusing the normalised screw-axis definitions from Section 8.2 gives the joint’s **space screw axis at the zero arrangement** in the project’s linear-first order:

$${}^s\mathbf{S}_j := \begin{cases} \begin{bmatrix} -{}^s\mathbf{s}_j \times {}^s\mathbf{p}_{\ell,j} \\ {}^s\mathbf{s}_j \end{bmatrix}, & \text{if joint } j \text{ is revolute,} \\ \begin{bmatrix} {}^s\mathbf{s}_j \\ \mathbf{0}_3 \end{bmatrix}, & \text{if joint } j \text{ is prismatic.} \end{cases} \quad (10.7)$$

Every ${}^s\mathbf{S}_j$ in Equation 10.7 is therefore constructed from joint j 's axis geometry when $\mathbf{q} = \mathbf{0}_n$; it is not constructed from an axis measured at an arbitrary nonzero joint-coordinate column.

For a revolute joint, the bottom block gives the positive rotation direction and the top block gives the spatial velocity-field coefficient at O_s per unit angular rate. Multiplying ${}^s\mathbf{S}_j$ by the joint angle q_j produces a linear exponential-coordinate block in metres and an angular block in radians. For a prismatic joint, the top block is the positive translation direction and multiplication by the joint displacement q_j produces a translation in metres; the angular block remains zero. The two joint types therefore use different blockwise units and normalisation rules, so the complete six-entry column must not be normalised with one Euclidean norm.

The revolute-axis point ${}^s\mathbf{p}_{\ell,j}$ is not unique, but the resulting screw-axis column is unique for the declared oriented line. If another selected point is ${}^s\mathbf{p}'_{\ell,j} = {}^s\mathbf{p}_{\ell,j} + \lambda {}^s\mathbf{s}_j$ for some $\lambda \in \mathbb{R}$ measured in metres, then

$$-{}^s\mathbf{s}_j \times {}^s\mathbf{p}'_{\ell,j} = -{}^s\mathbf{s}_j \times {}^s\mathbf{p}_{\ell,j} - \lambda ({}^s\mathbf{s}_j \times {}^s\mathbf{s}_j) = -{}^s\mathbf{s}_j \times {}^s\mathbf{p}_{\ell,j}.$$

Thus any point on the same axis line produces the same ${}^s\mathbf{S}_j$. Before using a numerical model, verify that $\mathbf{M} \in SE(3)$, every supplied component is finite, every declared axis direction has nonzero norm before normalisation, and every revolute axis includes a finite point. Measure every axis direction and axis point when $\mathbf{q} = \mathbf{0}_n$, and express every resulting coordinate column using $\{s\}$. The declared positive direction must agree with the sign convention of q_j : the right-hand rule for a revolute joint and translation along ${}^s\mathbf{s}_j$ for a prismatic joint.

The notation records two separate modelling choices. The superscript s states that the screw-axis coordinates are expressed using the fixed space frame $\{s\}$. To obtain those coordinates, place the robot at $\mathbf{q} = \mathbf{0}_n$, identify each joint's axis line and positive direction at that placement, and express them using $\{s\}$. Thus every ${}^s\mathbf{S}_j$ uses one common coordinate frame and is constructed from the same zero arrangement. The next subsection will show how these fixed model data are combined when the joint coordinates are nonzero.

The home pose and ordered space screw columns together define the **geometric forward-kinematics model**, denoted by $\mathcal{M}$:

$$\mathcal{M} := (\mathbf{M}, ({}^s\mathbf{S}_j)_{j=1}^n). \quad (10.8)$$

- $\mathbf{M} \in SE(3)$ is the home end-effector pose relative to the fixed space frame $\{s\}$.
- $({}^s\mathbf{S}_j)_{j=1}^n$ is the ordered sequence of six-entry space screw columns, from joint 1 at the base to joint n nearest the end effector. Every column describes its joint at the same zero arrangement and is expressed in $\{s\}$.
- n is the number of joints.

The parentheses denote an ordered pair containing a pose and an ordered sequence, not an unordered set: changing the joint order can change the resulting end-effector pose. The current joint coordinates and their allowed domains are specified separately from these fixed geometric data.

Example: Home data for a planar revolute–prismatic chain

This example shows how one zero arrangement supplies $\mathbf{M}$ and both types of space screw axis. Let $n = 2$, place O_s on joint 1, and align the end-effector frame with $\{s\}$ when $\mathbf{q} = \mathbf{0}_2$. In this zero arrangement, joint 1 revolves about positive z_s through O_s , joint 2 translates along positive x_s , and O_e is L metres from O_s along positive x_s , where $L > 0$.

The home end-effector pose is

$$\mathbf{M} = \begin{bmatrix} \mathbf{I}_3 & [L \ 0 \ 0]^\top \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

For joint 1, choose ${}^s\mathbf{s}_1 = [0 \ 0 \ 1]^\top$ and ${}^s\mathbf{p}_{\ell,1} = \mathbf{0}_3$ m. For joint 2, choose ${}^s\mathbf{s}_2 = [1 \ 0 \ 0]^\top$. Equation Equation 10.7 then gives

$${}^s\mathbf{S}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad {}^s\mathbf{S}_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

The model has now recorded the end-effector pose at zero and the positive motion permitted by each joint. It has not yet evaluated the end-effector pose for nonzero joint coordinates; the next subsection will combine these data in the required order.

10.2.2 Space-form product of exponentials

The stored home pose and space screw axes now answer the forward-kinematics question. Number the joints from the fixed base toward the end effector, so joint 1 is the most proximal joint and joint n is the most distal. For each $j \in \{1, \dots, n\}$, define the active rigid displacement generated by joint j from its zero value as

$$\boxed{\mathbf{E}_j(q_j) := \exp(\widehat{{}^s\mathbf{S}_j} q_j) \in SE(3)}. \quad (10.9)$$

The hat operator gives the 4×4 matrix representation of the linear-first screw axis. If ${}^s\mathbf{S}_j = [({}^s\mathbf{s}_{v,j})^\top ({}^s\mathbf{s}_{\omega,j})^\top]^\top$, then

$$\widehat{{}^s\mathbf{S}_j} = \begin{bmatrix} [{}^s\mathbf{s}_{\omega,j}]_\times & {}^s\mathbf{s}_{v,j} \\ \mathbf{0}_3^\top & 0 \end{bmatrix}.$$

For a revolute joint, q_j is the signed angle in radians; for a prismatic joint, q_j is the signed displacement in metres. In both cases, Equation Equation 10.9 is the finite joint displacement established in Section 8.3. In particular, $\mathbf{E}_j(0) = \mathbf{I}_4$.

To derive the serial-chain product without changing any stored screw axis, begin with the robot at $\mathbf{M}$ and first move only the most distal joint n . This joint moves the end effector but cannot move any joint closer to the base, so its axis still has the geometry recorded at $\mathbf{q} = \mathbf{0}_n$. The resulting end-effector pose is

$$\mathbf{T}_{se} = \mathbf{E}_n(q_n)\mathbf{M}.$$

Next move joint $n - 1$. It carries joint n , the final link, and the end effector as one outward assembly, so its active displacement premultiplies the pose already obtained:

$$\mathbf{T}_{se} = \mathbf{E}_{n-1}(q_{n-1})\mathbf{E}_n(q_n)\mathbf{M}.$$

Continue toward the base. Each newly moved joint carries every link and joint outward from it, while none of the previously moved distal joints changes that more proximal joint's axis. After joint 1 is included, the **space-form product of exponentials** is

$$\boxed{\mathbf{F}_e(\mathbf{q}) = \mathbf{T}_{se}(\mathbf{q}) = \mathbf{E}_1(q_1)\mathbf{E}_2(q_2) \cdots \mathbf{E}_n(q_n)\mathbf{M} = \left(\prod_{j=1}^n \exp(\widehat{{}^s\mathbf{S}_j} q_j) \right) \mathbf{M}.} \quad (10.10)$$

In Equation Equation 10.10, the product symbol is defined to preserve the physical joint order:

$$\prod_{j=1}^n \mathbf{E}_j(q_j) := \mathbf{E}_1(q_1)\mathbf{E}_2(q_2) \cdots \mathbf{E}_n(q_n).$$

Matrix multiplication acts on the rightmost factor first. Thus the derivation begins with $\mathbf{M}$, applies joint n , then joint $n - 1$, and finishes with joint 1, even though the written joint

indices increase from left to right. This distal-to-proximal construction is a way to derive and evaluate the final pose from the stored zero-arrangement axes; it is not a claim about the order in which the physical robot moved through time. Because rigid transformations generally do not commute, reversing any two joint factors usually produces a different pose and therefore a different kinematic model.

The formula satisfies two immediate consistency checks. If $\mathbf{q} = \mathbf{0}_n$, every joint exponential is $\mathbf{I}_4$ and $\mathbf{T}_{se}(\mathbf{0}_n) = \mathbf{M}$. If only q_k is nonzero, every other factor is the identity and $\mathbf{T}_{se}(\mathbf{q}) = \mathbf{E}_k(q_k)\mathbf{M}$. Moreover, every factor belongs to $SE(3)$, so closure of the group guarantees that the resulting pose also belongs to $SE(3)$.

Equation 10.10 is evaluated for $\mathbf{q} \in \mathcal{Q}$. Before forming any numerical exponential, an implementation must verify that every q_j is finite, belongs to its declared joint domain $\mathcal{Q}_j$, and follows the model's declared wrapped or unwrapped representation for revolute angles. The matrix exponential converts valid joint coordinates into rigid displacements; it does not make an invalid coordinate column admissible.

Example: Space-form pose of a planar 2R chain

This example shows that the space-form product reproduces the planar 2R position map derived earlier. Let both links point along positive x_s when $\boldsymbol{\theta} = [\theta_1 \ \theta_2]^\top = \mathbf{0}_2$, let their lengths be $L_1, L_2 > 0$ metres, place joint 1 at O_s , and let both positive joint axes point along positive z_s . Attach $\{e\}$ to the distal end of link 2 with the same orientation as $\{s\}$ at zero. The home pose and linear-first space screw axes are

$$\mathbf{M} = \begin{bmatrix} \mathbf{I}_3 & [L_1 + L_2 \ 0 \ 0]^\top \\ \mathbf{0}_3^\top & 1 \end{bmatrix},$$

$${}^s\mathbf{S}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Joint 1 passes through O_s , so its linear block is zero. Joint 2 also rotates about positive z_s , but its axis does not pass through O_s . Its positive unit direction and one point on its axis in the zero arrangement are

$${}^s\mathbf{s}_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad {}^s\mathbf{p}_{\ell,2} = \begin{bmatrix} L_1 \\ 0 \\ 0 \end{bmatrix} \text{ m.}$$

For a revolute joint, Equation 10.7 defines the linear block as $-{}^s\mathbf{s}_2 \times {}^s\mathbf{p}_{\ell,2}$. The required cross product is

$$-{}^s\mathbf{s}_2 \times {}^s\mathbf{p}_{\ell,2} = - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} L_1 \\ 0 \\ 0 \end{bmatrix} = - \begin{bmatrix} 0 \\ L_1 \\ 0 \end{bmatrix}.$$

Combining this linear block with the angular column $[0 \ 0 \ 1]^\top$ gives

$${}^s\mathbf{S}_2 = \begin{bmatrix} 0 \\ -L_1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

The entry $-L_1$ does not mean that joint 2 translates. It records that the revolute axis is displaced L_1 metres from O_s ; together with the angular block, it makes every point on that axis have zero instantaneous velocity.

Let $\mathbf{R}_z(\alpha)$ denote active rotation through α about positive z_s . To calculate the joint displacement using the general $SE(3)$ exponential from Section 8.3, first multiply the stored screw axis by the joint angle and separate the resulting finite linear and rotation-vector blocks:

$${}^s\boldsymbol{\eta}_2 = {}^s\mathbf{S}_2\theta_2 = \begin{bmatrix} {}^s\boldsymbol{\rho}_2 \\ {}^s\boldsymbol{\phi}_2 \end{bmatrix},$$

where

$${}^s\boldsymbol{\rho}_2 = \begin{bmatrix} 0 \\ -L_1\theta_2 \\ 0 \end{bmatrix}, \quad {}^s\boldsymbol{\phi}_2 = \begin{bmatrix} 0 \\ 0 \\ \theta_2 \end{bmatrix}.$$

Equation Equation 8.24 then gives

$$\mathbf{E}_2(\theta_2) = \exp(\widehat{{}^s\boldsymbol{\eta}_2}) = \begin{bmatrix} \mathbf{R}({}^s\boldsymbol{\phi}_2) & \mathbf{J}({}^s\boldsymbol{\phi}_2) {}^s\boldsymbol{\rho}_2 \\ \mathbf{0}_3^\top & 1 \end{bmatrix}. \quad (10.11)$$

Here $\mathbf{J}$ is the $SO(3)$ left Jacobian from the earlier $SE(3)$ exponential derivation, not a robot manipulator Jacobian. It maps the finite linear exponential-coordinate block to the translation accumulated while the rotation develops. For the rotation vector above and $\theta_2 \neq 0$, it is

$$\mathbf{J}({}^s\phi_2) = \begin{bmatrix} \frac{\sin \theta_2}{\theta_2} & -\frac{1 - \cos \theta_2}{\theta_2} & 0 \\ \frac{1 - \cos \theta_2}{\theta_2} & \frac{\sin \theta_2}{\theta_2} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

At $\theta_2 = 0$, the continuous value is $\mathbf{J}(\mathbf{0}_3) = \mathbf{I}_3$. For a nonzero angle, multiplying the displayed matrix by the finite linear block gives

$$\begin{aligned} \mathbf{J}({}^s\phi_2) {}^s\boldsymbol{\rho}_2 &= \begin{bmatrix} \frac{\sin \theta_2}{\theta_2} & -\frac{1 - \cos \theta_2}{\theta_2} & 0 \\ \frac{1 - \cos \theta_2}{\theta_2} & \frac{\sin \theta_2}{\theta_2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -L_1\theta_2 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} L_1(1 - \cos \theta_2) \\ -L_1 \sin \theta_2 \\ 0 \end{bmatrix}. \end{aligned}$$

The same translation has a direct geometric interpretation as rotation about an axis that does not pass through the space origin.²

The rotation block is

$$\mathbf{R}({}^s\phi_2) = \mathbf{R}_z(\theta_2).$$

Therefore

$$\mathbf{E}_2(\theta_2) = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & L_1(1 - \cos \theta_2) \\ \sin \theta_2 & \cos \theta_2 & 0 & -L_1 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & L_1 + L_2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Multiplying these matrices explicitly gives

$$\begin{aligned}
\mathbf{E}_2(\theta_2)\mathbf{M} &= \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & L_1(1 - \cos \theta_2) \\ \sin \theta_2 & \cos \theta_2 & 0 & -L_1 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & L_1 + L_2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & (L_1 + L_2) \cos \theta_2 + L_1(1 - \cos \theta_2) \\ \sin \theta_2 & \cos \theta_2 & 0 & (L_1 + L_2) \sin \theta_2 - L_1 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & L_1 + L_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & L_2 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
\end{aligned}$$

Joint 1 has zero linear screw-axis block because its axis passes through O_s . Its exponential is therefore

$$\mathbf{E}_1(\theta_1) = \begin{bmatrix} \mathbf{R}_z(\theta_1) & \mathbf{0}_3 \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

This displacement rotates the complete outward assembly already represented by $\mathbf{E}_2(\theta_2)\mathbf{M}$. Premultiplication gives

$$\begin{aligned}
\mathbf{T}_{se}(\boldsymbol{\theta}) &= \mathbf{E}_1(\theta_1)\mathbf{E}_2(\theta_2)\mathbf{M} \\
&= \begin{bmatrix} \mathbf{R}_z(\theta_1) & \mathbf{0}_3 \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_z(\theta_2) & \begin{bmatrix} L_1 + L_2 \cos \theta_2 \\ L_2 \sin \theta_2 \\ 0 \\ 1 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{R}_z(\theta_1)\mathbf{R}_z(\theta_2) & \mathbf{R}_z(\theta_1) \begin{bmatrix} L_1 + L_2 \cos \theta_2 \\ L_2 \sin \theta_2 \\ 0 \\ 1 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.
\end{aligned}$$

Both rotation factors act about the same positive z_s -axis, so their angles add:

$$\mathbf{R}_z(\theta_1)\mathbf{R}_z(\theta_2) = \mathbf{R}_z(\theta_1 + \theta_2).$$

The position block is

$$\begin{aligned}
{}^s\mathbf{p}_e(\boldsymbol{\theta}) &= \mathbf{R}_z(\theta_1) \begin{bmatrix} L_1 + L_2 \cos \theta_2 \\ L_2 \sin \theta_2 \\ 0 \end{bmatrix} \\
&= \begin{bmatrix} L_1 \cos \theta_1 + L_2 (\cos \theta_1 \cos \theta_2 - \sin \theta_1 \sin \theta_2) \\ L_1 \sin \theta_1 + L_2 (\sin \theta_1 \cos \theta_2 + \cos \theta_1 \sin \theta_2) \\ 0 \end{bmatrix} \\
&= \begin{bmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \\ 0 \end{bmatrix} \text{ m.}
\end{aligned}$$

Combining the two blocks gives

$$\mathbf{T}_{se}(\boldsymbol{\theta}) = \begin{bmatrix} \mathbf{R}_z(\theta_1 + \theta_2) & {}^s\mathbf{p}_e(\boldsymbol{\theta}) \\ \mathbf{0}_3^T & 1 \end{bmatrix}.$$

The position block is exactly the planar 2R forward-position map already obtained from link geometry: link 1 points at angle θ_1 , while link 2 has the absolute angle $\theta_1 + \theta_2$. The rotation block additionally retains the end-effector orientation.

²**Rotation about a point away from the origin.** The point

$${}^s\mathbf{p}_{\ell,2} = \begin{bmatrix} L_1 \\ 0 \\ 0 \end{bmatrix}$$

lies on joint 2's rotation axis. To rotate an arbitrary point $\mathbf{p}$ about the axis through this point:

1. Subtract the axis point, temporarily moving the rotation centre to the origin:

$$\mathbf{p} - {}^s\mathbf{p}_{\ell,2}.$$

2. Rotate the resulting relative vector:

$$\mathbf{R}_z(\theta_2) (\mathbf{p} - {}^s\mathbf{p}_{\ell,2}).$$

3. Add the axis point to restore the original rotation centre:

$$\mathbf{p}' = {}^s\mathbf{p}_{\ell,2} + \mathbf{R}_z(\theta_2) (\mathbf{p} - {}^s\mathbf{p}_{\ell,2}).$$

Expanding the geometric construction gives

$$\begin{aligned}
\mathbf{p}' &= {}^s\mathbf{p}_{\ell,2} + \mathbf{R}_z(\theta_2)\mathbf{p} - \mathbf{R}_z(\theta_2){}^s\mathbf{p}_{\ell,2} \\
&= \mathbf{R}_z(\theta_2)\mathbf{p} + (\mathbf{I}_3 - \mathbf{R}_z(\theta_2)) {}^s\mathbf{p}_{\ell,2}.
\end{aligned}$$

Now let $\bar{\mathbf{p}} = [\mathbf{p}^T \ 1]^T$ be the homogeneous column for the same point. In the main text, Equation 10.11 makes the joint-2 transformation act as

10.2.3 Body-form product of exponentials

The space form stores every joint screw axis using $\{s\}$. The same physical model can instead store the axes using the end-effector frame $\{e\}$ when the robot is at $\mathbf{q} = \mathbf{0}_n$. This alternative is useful when geometric measurements or calculations are naturally referred to the tool frame, but it must produce the same $\mathbf{T}_{se}(\mathbf{q})$.

The adjoint rule recalled from Section 7.9.4 is stated first for an instantaneous twist. If $\mathbf{T}_{ab}$ maps coordinates from frame $\{b\}$ to frame $\{a\}$, then the two coordinate columns of the same physical instantaneous rigid-body velocity satisfy

$${}^a\boldsymbol{\xi}_j = \text{Ad}_{\mathbf{T}_{ab}} {}^b\boldsymbol{\xi}_j.$$

Here ${}^a\boldsymbol{\xi}_j$ and ${}^b\boldsymbol{\xi}_j$ describe the same joint-induced velocity; only the frame used for their coordinates differs. In each frame, the corresponding screw-axis column maps the same scalar joint rate $\dot{q}_j$ to that twist:

$${}^a\boldsymbol{\xi}_j = {}^a\mathbf{S}_j \dot{q}_j, \quad {}^b\boldsymbol{\xi}_j = {}^b\mathbf{S}_j \dot{q}_j.$$

Substituting these two expressions into the twist transformation gives

$${}^a\mathbf{S}_j \dot{q}_j = \text{Ad}_{\mathbf{T}_{ab}} {}^b\mathbf{S}_j \dot{q}_j.$$

This equality must hold for every possible value of the same joint rate. Therefore, the columns multiplying $\dot{q}_j$ must be equal:

$${}^a\mathbf{S}_j = \text{Ad}_{\mathbf{T}_{ab}} {}^b\mathbf{S}_j.$$

$$\begin{aligned} \bar{\mathbf{p}}' &= \mathbf{E}_2(\theta_2) \bar{\mathbf{p}} \\ &= \begin{bmatrix} \mathbf{R}_z(\theta_2) & \mathbf{J}({}^s\phi_2) {}^s\rho_2 \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix}. \end{aligned}$$

Reading the upper three entries gives

$$\mathbf{p}' = \mathbf{R}_z(\theta_2) \mathbf{p} + \mathbf{J}({}^s\phi_2) {}^s\rho_2.$$

The geometric construction and the homogeneous-matrix calculation describe the same moved point and already contain the same rotation term $\mathbf{R}_z(\theta_2) \mathbf{p}$. Their remaining translation terms must therefore agree:

$$\mathbf{J}({}^s\phi_2) {}^s\rho_2 = (\mathbf{I}_3 - \mathbf{R}_z(\theta_2)) {}^s\mathbf{p}_{\ell,2}.$$

Both sides evaluate to $[L_1(1 - \cos \theta_2) - L_1 \sin \theta_2 \ 0]^\top$, the translation column used by the explicit $\mathbf{E}_2(\theta_2)$ matrix in the main text.

The required conversion here is from the space-frame column ${}^s\mathbf{S}_j$ to a column expressed using the home end-effector frame $\{e\}$. At the zero arrangement, $\mathbf{M} = \mathbf{T}_{se}(\mathbf{0}_n)$ maps coordinates from $\{e\}$ to $\{s\}$, so $\mathbf{T}_{es} = \mathbf{M}^{-1}$ maps them from $\{s\}$ to $\{e\}$. Substituting $a = e$, $b = s$, and $\mathbf{T}_{ab} = \mathbf{T}_{es} = \mathbf{M}^{-1}$ into the screw-axis transformation defines the **body screw axis** for joint j :

$$\boxed{{}^e\mathbf{B}_j := \text{Ad}_{\mathbf{M}^{-1}} {}^s\mathbf{S}_j \in \mathbb{R}^6}. \quad (10.12)$$

The columns ${}^s\mathbf{S}_j$ and ${}^e\mathbf{B}_j$ represent the same joint screw using different frames. When the robot is in the zero arrangement and only that joint moves, they are related by

$$\boxed{\boldsymbol{\xi}_{e,j} = {}^e\mathbf{B}_j \dot{q}_j}.$$

The body screw axis ${}^e\mathbf{B}_j$ is fixed home-configuration model data, whereas $\boldsymbol{\xi}_{e,j}$ is an instantaneous velocity that depends on the current joint rate. Multiplication by the joint coordinate instead gives the finite exponential-coordinate column ${}^e\boldsymbol{\eta}_j = {}^e\mathbf{B}_j q_j$ used inside the body-form joint factor. At a general nonzero configuration, the complete end-effector twist combines all joint rates through the robot Jacobian developed later; it is not obtained by treating every stored home-axis column as an unmodified current twist.

To make the linear-first conversion explicit, write

$$\mathbf{M}^{-1} = \begin{bmatrix} \mathbf{R}_{es} & {}^e\mathbf{p}_s \\ \mathbf{0}_3^T & 1 \end{bmatrix}, \quad {}^s\mathbf{S}_j = \begin{bmatrix} {}^s\mathbf{s}_{v,j} \\ {}^s\mathbf{s}_{\omega,j} \end{bmatrix}.$$

Here $\mathbf{R}_{es}$ maps vector-coordinate columns from $\{s\}$ to $\{e\}$, and ${}^e\mathbf{p}_s$ is the position of O_s relative to O_e , expressed in $\{e\}$ at the zero arrangement. The linear-first adjoint and the resulting body-axis blocks are

$$\text{Ad}_{\mathbf{M}^{-1}} = \begin{bmatrix} \mathbf{R}_{es} & [{}^e\mathbf{p}_s]_{\times} \mathbf{R}_{es} \\ \mathbf{0}_{3 \times 3} & \mathbf{R}_{es} \end{bmatrix}, \quad {}^e\mathbf{B}_j = \begin{bmatrix} \mathbf{R}_{es} {}^s\mathbf{s}_{v,j} + {}^e\mathbf{p}_s \times (\mathbf{R}_{es} {}^s\mathbf{s}_{\omega,j}) \\ \mathbf{R}_{es} {}^s\mathbf{s}_{\omega,j} \end{bmatrix}.$$

The superscript e states that both three-entry blocks are expressed using the home end-effector frame. The rotation $\mathbf{R}_{es}$ re-expresses the linear and angular components along the home end-effector axes, while the cross-product term accounts for changing the reference origin from O_s to O_e . Equation Equation 10.12 changes only the coordinate representation of the joint screw; it does not change the physical joint, its positive direction, its coordinate q_j , or the zero arrangement.

To obtain the body-form pose equation, the finite joint displacement must undergo the same change of coordinates. For any $\mathbf{T} \in SE(3)$ and linear-first six-column $\boldsymbol{\eta} \in \mathbb{R}^6$, the definition of the adjoint gives the hat-matrix relation

$$\widehat{\text{Ad}_{\mathbf{T}^{-1}} \boldsymbol{\eta}} = \mathbf{T}^{-1} \hat{\boldsymbol{\eta}} \mathbf{T}.$$

The matrix exponential preserves this similarity transformation. Indeed, $(\mathbf{T}^{-1} \hat{\boldsymbol{\eta}} \mathbf{T})^k = \mathbf{T}^{-1} \hat{\boldsymbol{\eta}}^k \mathbf{T}$ because the adjacent factors $\mathbf{T} \mathbf{T}^{-1}$ cancel. Applying the power-series definition of the exponential therefore gives

$$\begin{aligned} \exp(\widehat{\text{Ad}_{\mathbf{T}^{-1}} \boldsymbol{\eta}}) &= \sum_{k=0}^{\infty} \frac{(\mathbf{T}^{-1} \hat{\boldsymbol{\eta}} \mathbf{T})^k}{k!} \\ &= \mathbf{T}^{-1} \left(\sum_{k=0}^{\infty} \frac{\hat{\boldsymbol{\eta}}^k}{k!} \right) \mathbf{T} \\ &= \mathbf{T}^{-1} \exp(\hat{\boldsymbol{\eta}}) \mathbf{T}. \end{aligned}$$

Reading this equality in the opposite direction gives the **exponential conjugation identity**

$$\boxed{\mathbf{T}^{-1} \exp(\hat{\boldsymbol{\eta}}) \mathbf{T} = \exp(\widehat{\text{Ad}_{\mathbf{T}^{-1}} \boldsymbol{\eta}})}.$$

The left side first forms the finite displacement $\exp(\hat{\boldsymbol{\eta}})$ and then changes its coordinate representation using $\mathbf{T}^{-1}$ and $\mathbf{T}$. The right side first changes the generator coordinates using the adjoint and then exponentiates them. The identity states that these two orders produce the same rigid displacement matrix; conjugation changes its coordinates, not the physical displacement it represents.

Set $\mathbf{T} = \mathbf{M}$ and $\boldsymbol{\eta} = {}^s\mathbf{S}_j q_j$. Using the definition of $\mathbf{E}_j$, the conjugation identity, and the definition of ${}^e\mathbf{B}_j$ gives the bridge one equality at a time:

$$\begin{aligned} \mathbf{M}^{-1} \mathbf{E}_j(q_j) \mathbf{M} &= \mathbf{M}^{-1} \exp(\widehat{{}^s\mathbf{S}_j q_j}) \mathbf{M} \\ &= \exp\left(\widehat{\text{Ad}_{\mathbf{M}^{-1}} ({}^s\mathbf{S}_j q_j)}\right) \\ &= \exp\left(\widehat{(\text{Ad}_{\mathbf{M}^{-1}} {}^s\mathbf{S}_j) q_j}\right) \\ &= \exp(\widehat{{}^e\mathbf{B}_j q_j}) \\ &= \exp(\widehat{{}^e\mathbf{B}_j} q_j) =: \mathbf{G}_j(q_j). \end{aligned} \tag{10.13}$$

The first equality substitutes $\mathbf{E}_j(q_j) = \exp(\widehat{{}^s\mathbf{S}_j q_j})$. The second applies exponential conjugation. The third uses linearity of the adjoint with respect to the scalar q_j . The fourth substitutes ${}^e\mathbf{B}_j = \text{Ad}_{\mathbf{M}^{-1}} {}^s\mathbf{S}_j$, and the fifth uses linearity of the hat operator, $\widehat{{}^e\mathbf{B}_j q_j} = \widehat{{}^e\mathbf{B}_j} q_j$. The final relation defines the body-represented joint displacement $\mathbf{G}_j(q_j)$.

Equivalently,

$$\boxed{\mathbf{G}_j(q_j) = \mathbf{M}^{-1} \mathbf{E}_j(q_j) \mathbf{M}, \quad \mathbf{E}_j(q_j) = \mathbf{M} \mathbf{G}_j(q_j) \mathbf{M}^{-1}}.$$

These equations separate the physical displacement from its coordinate representation:

1. $\mathbf{G}_j(q_j)$ represents joint j 's displacement using the home end-effector frame.
2. $\mathbf{E}_j(q_j)$ represents the same physical displacement using the space frame.
3. Conjugation by $\mathbf{M}$ and $\mathbf{M}^{-1}$ changes between those two coordinate representations.

Right-multiplying $\mathbf{E}_j(q_j) = \mathbf{M} \mathbf{G}_j(q_j) \mathbf{M}^{-1}$ by $\mathbf{M}$ gives $\mathbf{E}_j(q_j) \mathbf{M} = \mathbf{M} \mathbf{G}_j(q_j)$. Starting with the rightmost joint factor in the space form and applying this identity repeatedly moves $\mathbf{M}$ to the left:

$$\begin{aligned} \mathbf{E}_1 \cdots \mathbf{E}_{n-1} \mathbf{E}_n \mathbf{M} &= \mathbf{E}_1 \cdots \mathbf{E}_{n-1} \mathbf{M} \mathbf{G}_n \\ &= \cdots \\ &= \mathbf{M} \mathbf{G}_1 \mathbf{G}_2 \cdots \mathbf{G}_n. \end{aligned}$$

The arguments q_j have been suppressed only in this intermediate line to keep the algebra readable. Restoring them gives the **body-form product of exponentials**:

$$\boxed{\mathbf{T}_{se}(\mathbf{q}) = \mathbf{M} \exp(\widehat{{}^e\mathbf{B}_1} q_1) \exp(\widehat{{}^e\mathbf{B}_2} q_2) \cdots \exp(\widehat{{}^e\mathbf{B}_n} q_n)}. \quad (10.14)$$

Equation Equation 10.14 should be read as follows:

1. Every exponential uses a screw-axis column ${}^e\mathbf{B}_j$ expressed using the end-effector frame in the zero arrangement; these are fixed home-frame coordinates, not screw axes recomputed at the current $\mathbf{q}$.
2. $\mathbf{M} = \mathbf{T}_{se}(\mathbf{0}_n)$ is the home transformation that maps coordinates from $\{e\}$ to $\{s\}$ at the zero arrangement.
3. The complete product $\mathbf{M} \mathbf{G}_1(q_1) \cdots \mathbf{G}_n(q_n)$, not $\mathbf{M}$ alone, is the current end-effector pose $\mathbf{T}_{se}(\mathbf{q})$.

The joint indices have the same order in the space and body forms; they are not reversed. A convenient matrix-building interpretation differs: the space form may start from $\mathbf{M}$ and premultiply $\mathbf{E}_n, \dots, \mathbf{E}_1$, whereas the body form may start from $\mathbf{M}$ and postmultiply $\mathbf{G}_1, \dots, \mathbf{G}_n$. Both procedures evaluate the same pose because Equation Equation 10.12 and the conjugation identity connect every pair of factors.

Example: Body axes for the planar 2R chain

This example converts the preceding planar 2R space model into body coordinates. Its home orientation is $\mathbf{I}_3$, and its home end-effector origin is $L_1 + L_2$ metres along positive x_s . Therefore, the home pose and its inverse are

$$\mathbf{M} = \begin{bmatrix} \mathbf{I}_3 & \begin{bmatrix} L_1 + L_2 \\ 0 \\ 0 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \mathbf{M}^{-1} = \begin{bmatrix} \mathbf{I}_3 & \begin{bmatrix} -(L_1 + L_2) \\ 0 \\ 0 \end{bmatrix} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

The inverse pose maps coordinates from $\{s\}$ to the home end-effector frame $\{e\}$. Its translation column is the position of O_s relative to O_e , expressed in $\{e\}$:

$${}^e\mathbf{p}_s = \begin{bmatrix} -(L_1 + L_2) \\ 0 \\ 0 \end{bmatrix}.$$

Because $\mathbf{R}_{es} = \mathbf{I}_3$, applying the linear-first adjoint to a screw axis ${}^s\mathbf{S}_j = [({}^s\mathbf{s}_{v,j})^\top ({}^s\mathbf{s}_{\omega,j})^\top]^\top$ gives

$${}^e\mathbf{B}_j = \text{Ad}_{\mathbf{M}^{-1}} {}^s\mathbf{S}_j = \begin{bmatrix} \mathbf{I}_3 {}^s\mathbf{s}_{v,j} + {}^e\mathbf{p}_s \times (\mathbf{I}_3 {}^s\mathbf{s}_{\omega,j}) \\ \mathbf{I}_3 {}^s\mathbf{s}_{\omega,j} \end{bmatrix}.$$

Both joints rotate about positive z_s , so their angular blocks are $[0 \ 0 \ 1]^\top$. The origin shift contributes the same cross product to both transformed linear blocks:

$${}^e\mathbf{p}_s \times \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -(L_1 + L_2) \\ 0 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ L_1 + L_2 \\ 0 \end{bmatrix}.$$

The preceding space model has

$${}^s\mathbf{S}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad {}^s\mathbf{S}_2 = \begin{bmatrix} 0 \\ -L_1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

For joint 1, the space-axis linear block is zero. Adding the origin-shift contribution gives its body-axis linear block, while the identity home rotation leaves its angular block unchanged:

$${}^e\mathbf{b}_{v,1} = \mathbf{I}_3 \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ L_1 + L_2 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ L_1 + L_2 \\ 0 \end{bmatrix},$$

$${}^e\mathbf{b}_{\omega,1} = \mathbf{I}_3 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Stacking the linear block above the angular block in the project's linear-first order gives

$${}^e\mathbf{B}_1 = \begin{bmatrix} {}^e\mathbf{b}_{v,1} \\ {}^e\mathbf{b}_{\omega,1} \end{bmatrix} = \begin{bmatrix} 0 \\ L_1 + L_2 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

For joint 2, the $-L_1$ entry in its space-axis linear block combines with $L_1 + L_2$ from the origin shift, and its angular block is again unchanged:

$${}^e\mathbf{b}_{v,2} = \begin{bmatrix} 0 \\ -L_1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ L_1 + L_2 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ L_2 \\ 0 \end{bmatrix},$$

$${}^e\mathbf{b}_{\omega,2} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Therefore,

$${}^e\mathbf{B}_2 = \begin{bmatrix} {}^e\mathbf{b}_{v,2} \\ {}^e\mathbf{b}_{\omega,2} \end{bmatrix} = \begin{bmatrix} 0 \\ L_2 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

The signs have a direct velocity-field interpretation. Joint 1 is $L_1 + L_2$ metres behind the home end-effector origin along negative x_e , whereas joint 2 is L_2 metres behind it. Define their positive offset magnitudes and axis-point positions by

$$d_1 := L_1 + L_2, \quad d_2 := L_2, \quad {}^e\mathbf{p}_{\ell,j} = \begin{bmatrix} -d_j \\ 0 \\ 0 \end{bmatrix}.$$

For joint j rotating at rate $\dot{\theta}_j$, the velocity of the selected axis point is

$${}^e\mathbf{v}_{\ell,j} = ({}^e\mathbf{b}_{v,j} + {}^e\mathbf{s}_{\omega,j} \times {}^e\mathbf{p}_{\ell,j}) \dot{\theta}_j.$$

The selected point lies on the rotation axis, so it must remain fixed and ${}^e\mathbf{v}_{\ell,j} = \mathbf{0}_3$ for every joint rate. The positive- z_e angular direction produces the rotational velocity-field contribution

$$\begin{aligned} {}^e\mathbf{s}_{\omega,j} \times {}^e\mathbf{p}_{\ell,j} &= \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} -d_j \\ 0 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ -d_j \\ 0 \end{bmatrix}. \end{aligned}$$

This contribution points along negative y_e . To make the total velocity at the axis point zero, the linear coefficient must point along positive y_e with the same magnitude:

$$\mathbf{0}_3 = {}^e\mathbf{b}_{v,j} + \begin{bmatrix} 0 \\ -d_j \\ 0 \end{bmatrix} \implies {}^e\mathbf{b}_{v,j} = \begin{bmatrix} 0 \\ d_j \\ 0 \end{bmatrix}.$$

Substituting $d_1 = L_1 + L_2$ and $d_2 = L_2$ recovers the two linear blocks calculated above. For joint 1, the fixed axis point may be chosen as O_s because its axis passes through the space origin. For joint 2, the fixed point is instead a point on joint 2's axis; O_s does not lie on that axis. Substituting the two body screw axes into Equation 10.14 gives

$$\mathbf{T}_{se}(\boldsymbol{\theta}) = \mathbf{M} \exp(\widehat{{}^e\mathbf{B}_1} \theta_1) \exp(\widehat{{}^e\mathbf{B}_2} \theta_2),$$

which equals the space-form pose for every allowed $\boldsymbol{\theta}$ even though the stored six-columns are different.

10.2.4 Equivalence with a transform chain

A product of exponentials is not a different physical model from a chain of parent-child transformations. The two descriptions store the same joint and link geometry in different forms. This subsection starts with a relative-transform chain and derives its space-form product, thereby showing exactly how their constants correspond.

Let $\{0\} = \{s\}$ be the fixed base frame, let $\{i\}$ be a frame rigidly attached to link i for each $i \in \{1, \dots, n\}$, and let $\{e\}$ be the selected end-effector frame rigidly attached to link n . For each joint i , let

$$\mathbf{T}_{i-1,i}(q_i) \in SE(3)$$

be the pose of link frame $\{i\}$ relative to its parent frame $\{i-1\}$. This matrix maps coordinate columns from $\{i\}$ to $\{i-1\}$. Because joint i is independent and has one degree of freedom, only q_i appears in $\mathbf{T}_{i-1,i}(q_i)$.

Let $\mathbf{T}_{ne} \in SE(3)$ be the pose of the end-effector frame $\{e\}$ relative to the final link frame $\{n\}$. This matrix maps end-effector coordinates to final-link coordinates:

$$\bar{\mathbf{p}}_n = \mathbf{T}_{ne} \bar{\mathbf{p}}_e.$$

Both frames are rigidly attached to link n , and there is no joint between them, so $\mathbf{T}_{ne}$ is constant and does not depend on $\mathbf{q}$. If $\{e\}$ is chosen to coincide with $\{n\}$, then $\mathbf{T}_{ne} = \mathbf{I}_4$.

Let $\mathbf{T}_{sn}(\mathbf{q})$ denote the pose of the final link frame relative to $\{s\}$ at joint-coordinate column $\mathbf{q}$. If $\bar{\mathbf{p}}_e$, $\bar{\mathbf{p}}_n$, and $\bar{\mathbf{p}}_s$ are homogeneous coordinate columns of the same physical point, expressed in $\{e\}$, $\{n\}$, and $\{s\}$, respectively, then

$$\begin{aligned} \bar{\mathbf{p}}_s &= \mathbf{T}_{sn}(\mathbf{q}) \bar{\mathbf{p}}_n \\ &= \mathbf{T}_{sn}(\mathbf{q}) \mathbf{T}_{ne} \bar{\mathbf{p}}_e. \end{aligned}$$

Because this relation holds for every end-effector point,

$$\boxed{\mathbf{T}_{se}(\mathbf{q}) = \mathbf{T}_{sn}(\mathbf{q}) \mathbf{T}_{ne}}.$$

Expanding $\mathbf{T}_{sn}(\mathbf{q})$ through the relative transforms gives the complete chain:

$$\boxed{\mathbf{T}_{se}(\mathbf{q}) = \mathbf{T}_{01}(q_1) \mathbf{T}_{12}(q_2) \cdots \mathbf{T}_{n-1,n}(q_n) \mathbf{T}_{ne}}. \quad (10.15)$$

The rightmost factor first maps coordinates from $\{e\}$ to $\{n\}$; the remaining factors successively map them through the parent frames until the result is expressed in $\{s\}$. This meaning differs from the active-displacement interpretation of each $\mathbf{E}_j$, even though the final homogeneous matrix represents the same end-effector pose.

To expose the one-joint motion inside each relative transform, write the relative pose directly as

$$\mathbf{T}_{i-1,i}(q_i) = \exp\left(\widehat{{}^{i-1}\mathbf{S}_i} q_i\right) \mathbf{T}_{i-1,i}(0).$$

Here ${}^{i-1}\mathbf{S}_i$ is joint i 's normalised linear-first screw axis expressed using its parent frame $\{i-1\}$ in the zero arrangement. The exponential is the active displacement generated by joint i from its zero value, represented using the parent frame; left multiplication applies that displacement to the zero relative pose $\mathbf{T}_{i-1,i}(0)$.

The transform from link frame $\{k\}$ to $\{s\}$ in the zero arrangement is obtained by composing the zero relative poses:

$$\mathbf{T}_{sk}(\mathbf{0}_n) = \mathbf{T}_{01}(0)\mathbf{T}_{12}(0)\cdots\mathbf{T}_{k-1,k}(0), \quad \mathbf{T}_{s0}(\mathbf{0}_n) = \mathbf{I}_4.$$

In particular, $\mathbf{T}_{s,i-1}(\mathbf{0}_n)$ maps coordinates from joint i 's parent frame $\{i-1\}$ to the space frame $\{s\}$. If $\bar{\mathbf{p}}_{i-1}$ and $\bar{\mathbf{p}}_s$ are homogeneous coordinate columns of the same physical point expressed in those two frames, then

$$\bar{\mathbf{p}}_s = \mathbf{T}_{s,i-1}(\mathbf{0}_n)\bar{\mathbf{p}}_{i-1}.$$

The adjoint associated with the same transform converts an instantaneous twist from parent-frame coordinates to space-frame coordinates. If only joint i moves at rate $\dot{q}_i$ while the robot is in the zero arrangement, the same joint-induced physical velocity satisfies

$${}^s\boldsymbol{\xi}_i = \text{Ad}_{\mathbf{T}_{s,i-1}(\mathbf{0}_n)} {}^{i-1}\boldsymbol{\xi}_i.$$

In the two frames, the corresponding screw-axis columns map the same scalar joint rate to that twist:

$${}^s\boldsymbol{\xi}_i = {}^s\mathbf{S}_i\dot{q}_i, \quad {}^{i-1}\boldsymbol{\xi}_i = {}^{i-1}\mathbf{S}_i\dot{q}_i.$$

Substituting these expressions into the twist transformation and using linearity of the adjoint gives

$${}^s\mathbf{S}_i\dot{q}_i = \left(\text{Ad}_{\mathbf{T}_{s,i-1}(\mathbf{0}_n)} {}^{i-1}\mathbf{S}_i\right)\dot{q}_i.$$

This equality must hold for every possible value of the same joint rate $\dot{q}_i$. Therefore, the columns multiplying $\dot{q}_i$ must be equal:

$$\boxed{{}^s\mathbf{S}_i = \text{Ad}_{\mathbf{T}_{s,i-1}(\mathbf{0}_n)} {}^{i-1}\mathbf{S}_i}. \quad (10.16)$$

The exponential conjugation identity now expresses the parent-frame joint displacement using the space frame:

$$\begin{aligned}
& \mathbf{T}_{s,i-1}(\mathbf{0}_n) \exp\left(\widehat{{}^{i-1}\mathbf{S}_i} q_i\right) \left[\mathbf{T}_{s,i-1}(\mathbf{0}_n)\right]^{-1} \\
&= \exp\left(\mathbf{T}_{s,i-1}(\mathbf{0}_n) \widehat{{}^{i-1}\mathbf{S}_i} \left[\mathbf{T}_{s,i-1}(\mathbf{0}_n)\right]^{-1} q_i\right) \\
&= \exp\left(\widehat{\text{Ad}_{\mathbf{T}_{s,i-1}(\mathbf{0}_n)} {}^{i-1}\mathbf{S}_i} q_i\right) \\
&= \exp\left(\widehat{{}^s\mathbf{S}_i} q_i\right) \\
&= \mathbf{E}_i(q_i).
\end{aligned} \tag{10.17}$$

The first equality uses similarity of the matrix exponential. The second uses the hat–adjoint relation, the third substitutes Equation 10.16, and the final equality uses the definition of $\mathbf{E}_i(q_i)$. Thus the exponential represented using $\{i-1\}$ and $\mathbf{E}_i(q_i)$ represented using $\{s\}$ describe the same active joint displacement.

These relations are sufficient to prove that the two complete products agree. For any $k \in \{1, \dots, n\}$, the first k relative transforms satisfy

$$\boxed{\mathbf{T}_{01}(q_1) \mathbf{T}_{12}(q_2) \cdots \mathbf{T}_{k-1,k}(q_k) = \mathbf{E}_1(q_1) \mathbf{E}_2(q_2) \cdots \mathbf{E}_k(q_k) \mathbf{T}_{sk}(\mathbf{0}_n)}. \tag{10.18}$$

For $k = 1$, frame $\{0\}$ is $\{s\}$, so ${}^0\mathbf{S}_1 = {}^s\mathbf{S}_1$. Expanding the direct relative-pose formula gives

$$\begin{aligned}
\mathbf{T}_{01}(q_1) &= \exp\left(\widehat{{}^0\mathbf{S}_1} q_1\right) \mathbf{T}_{01}(0) \\
&= \exp\left(\widehat{{}^s\mathbf{S}_1} q_1\right) \mathbf{T}_{01}(0) \\
&= \mathbf{E}_1(q_1) \mathbf{T}_{01}(0) \\
&= \mathbf{E}_1(q_1) \mathbf{T}_{s1}(\mathbf{0}_n).
\end{aligned}$$

The first equality uses the one-joint relative-pose formula. The second uses $\{0\} = \{s\}$, the third uses the definition of $\mathbf{E}_1(q_1)$, and the fourth uses $\mathbf{T}_{01}(0) = \mathbf{T}_{s1}(\mathbf{0}_n)$. This proves the claim for $k = 1$.

Now suppose Equation 10.18 holds through $k-1$. First separate the final relative transform, then apply the induction assumption to the preceding $k-1$ transforms, and finally substitute the direct relative-pose formula for joint k :

$$\begin{aligned}
& \mathbf{T}_{01}(q_1)\mathbf{T}_{12}(q_2) \cdots \mathbf{T}_{k-1,k}(q_k) \\
&= (\mathbf{T}_{01}(q_1) \cdots \mathbf{T}_{k-2,k-1}(q_{k-1})) \mathbf{T}_{k-1,k}(q_k) \\
&= (\mathbf{E}_1(q_1) \cdots \mathbf{E}_{k-1}(q_{k-1}) \mathbf{T}_{s,k-1}(\mathbf{0}_n)) \mathbf{T}_{k-1,k}(q_k) \\
&= \mathbf{E}_1(q_1) \cdots \mathbf{E}_{k-1}(q_{k-1}) \mathbf{T}_{s,k-1}(\mathbf{0}_n) \left[\exp\left(\widehat{{}^{k-1}\mathbf{S}_k} q_k\right) \mathbf{T}_{k-1,k}(0) \right].
\end{aligned}$$

Insert the identity $[\mathbf{T}_{s,k-1}(\mathbf{0}_n)]^{-1} \mathbf{T}_{s,k-1}(\mathbf{0}_n) = \mathbf{I}_4$ between the local exponential and $\mathbf{T}_{k-1,k}(0)$. Regrouping the factors gives

$$\begin{aligned}
& \mathbf{T}_{01}(q_1)\mathbf{T}_{12}(q_2) \cdots \mathbf{T}_{k-1,k}(q_k) \\
&= \mathbf{E}_1(q_1) \cdots \mathbf{E}_{k-1}(q_{k-1}) \\
&\quad \times \underbrace{\mathbf{T}_{s,k-1}(\mathbf{0}_n) \exp\left(\widehat{{}^{k-1}\mathbf{S}_k} q_k\right) [\mathbf{T}_{s,k-1}(\mathbf{0}_n)]^{-1}}_{\mathbf{E}_k(q_k)} \\
&\quad \times \underbrace{\mathbf{T}_{s,k-1}(\mathbf{0}_n) \mathbf{T}_{k-1,k}(0)}_{\mathbf{T}_{sk}(\mathbf{0}_n)} \\
&= \mathbf{E}_1(q_1) \cdots \mathbf{E}_k(q_k) \mathbf{T}_{sk}(\mathbf{0}_n).
\end{aligned}$$

This proves the claim for every k . Taking $k = n$, multiplying by the fixed tool transform $\mathbf{T}_{ne}$, and using

$$\mathbf{T}_{sn}(\mathbf{0}_n) \mathbf{T}_{ne} = \mathbf{T}_{se}(\mathbf{0}_n) = \mathbf{M}$$

produces

$$\boxed{\underbrace{\mathbf{T}_{01}(q_1)\mathbf{T}_{12}(q_2) \cdots \mathbf{T}_{n-1,n}(q_n) \mathbf{T}_{ne}}_{\text{relative-transform chain}} = \underbrace{\mathbf{E}_1(q_1)\mathbf{E}_2(q_2) \cdots \mathbf{E}_n(q_n)\mathbf{M}}_{\text{space-form product of exponentials}}}. \quad (10.19)$$

The proof above starts with selected intermediate link frames and converts their relative-transform chain into a space-form product of exponentials. Conversion in the opposite direction requires additional choices because the space-form data $\mathbf{M}$ and ${}^s\mathbf{S}_1, \dots, {}^s\mathbf{S}_n$ do not specify where the intermediate link frames are attached. First select those frames and their zero relative transforms so that

$$\boxed{\mathbf{T}_{01}(0)\mathbf{T}_{12}(0) \cdots \mathbf{T}_{n-1,n}(0) \mathbf{T}_{ne} = \mathbf{M}}.$$

For joint i , these zero transforms determine the pose of its parent frame relative to $\{s\}$:

$$\mathbf{T}_{s,i-1}(\mathbf{0}_n) = \mathbf{T}_{01}(0) \cdots \mathbf{T}_{i-2,i-1}(0),$$

where the product is empty and equals $\mathbf{I}_4$ when $i = 1$. Equation Equation 10.16 can then be inverted because $\text{Ad}_{\mathbf{T}}^{-1} = \text{Ad}_{\mathbf{T}^{-1}}$:

$$\boxed{{}^{i-1}\mathbf{S}_i = \text{Ad}_{[\mathbf{T}_{s,i-1}(\mathbf{0}_n)]^{-1}} {}^s\mathbf{S}_i}.$$

This equation re-expresses the same physical joint screw using the selected parent frame. The parent-frame screw axis and the selected zero relative pose then reconstruct joint i 's relative transform:

$$\boxed{\mathbf{T}_{i-1,i}(q_i) = \exp\left(\widehat{{}^{i-1}\mathbf{S}_i} q_i\right) \mathbf{T}_{i-1,i}(0)}.$$

The intermediate link frames are not unique: a modeller may choose different fixed origins or orientations for a frame attached to a link. Such a choice changes the neighbouring zero relative transforms and the numerical parent-frame screw axes, but a consistent set still satisfies the home-pose product above and produces the same end-effector map $\mathbf{T}_{se}(\mathbf{q})$.

The representations therefore answer different implementation questions. A relative-transform chain makes every parent-child frame relationship explicit and naturally supplies intermediate link poses. The space form needs only $\mathbf{M}$ and the home screw axes to calculate the selected end-effector pose. The body form stores those same physical axes in home end-effector coordinates. None is inherently more accurate; disagreement between their evaluated poses indicates inconsistent frames, zero values, signs, joint order, or multiplication direction.

A deterministic cross-check begins by choosing one joint-coordinate column $\mathbf{q}$ and evaluating it with two different calculations. For example, calculation a may use the relative-transform chain, while calculation b uses the space-form product of exponentials. Denote their resulting pose matrices by $\mathbf{T}_a(\mathbf{q})$ and $\mathbf{T}_b(\mathbf{q})$, respectively. Both matrices must describe the pose of the same end-effector frame relative to the same space frame, with the same units, signs, joint order, and active-transformation convention:

$$\mathbf{T}_a(\mathbf{q}) = \begin{bmatrix} \mathbf{R}_a & \mathbf{p}_a \\ \mathbf{0}_3^T & 1 \end{bmatrix}, \quad \mathbf{T}_b(\mathbf{q}) = \begin{bmatrix} \mathbf{R}_b & \mathbf{p}_b \\ \mathbf{0}_3^T & 1 \end{bmatrix}.$$

Compare their translation blocks with the length residual

$$\boxed{e_p := \|\mathbf{p}_a - \mathbf{p}_b\|_2},$$

which is measured in metres. Compare their rotation blocks by forming the relative rotation

$$\mathbf{R}_\Delta := \mathbf{R}_a^\top \mathbf{R}_b \in SO(3).$$

Let ϕ_Δ be the principal rotation vector satisfying $\mathbf{R}_\Delta = \exp([\phi_\Delta]_\times)$ and $\|\phi_\Delta\|_2 \leq \pi$. The angular residual is

$$e_R := \|\phi_\Delta\|_2,$$

measured in radians. Choose a translation tolerance ε_p in metres and an angular tolerance ε_R in radians, and accept agreement only when

$$e_p \leq \varepsilon_p, \quad e_R \leq \varepsilon_R.$$

Do not combine the metre-valued and radian-valued residuals in one unscaled norm. Repeat the comparison at $\mathbf{q} = \mathbf{0}_n$, with each joint moved individually, with several joints moved together, and for mixed revolute–prismatic chains when supported. Also include a negative control in which the joint-factor order is deliberately changed at a configuration where noncommutativity makes the incorrect result detectable.

Quick test B: serial-chain products of exponentials

1. **Recall:** What fixed model data and current input are required by the space-form product of exponentials?
2. **Contrast:** What is the difference between the zero arrangement $\chi(\mathbf{0}_n)$ and the home end-effector pose $\mathbf{M}$?
3. **Application:** At $\mathbf{q} = \mathbf{0}_n$, a revolute joint points along positive z_s and passes through $[2 \ 0 \ 0]^\top$ m, while a prismatic joint translates along negative y_s . Find both linear-first space screw axes.
4. **Explanation:** For a three-joint chain, write the space-form product and explain why its derivation applies the distal joint displacement to $\mathbf{M}$ first even though the written factors are ordered 1, 2, 3 from left to right.
5. **Application:** In the planar 2R example, let $L_1 = 2$ m, $L_2 = 1$ m, $\theta_1 = \pi/2$, and $\theta_2 = -\pi/2$. Find the end-effector orientation and position relative to $\{s\}$.
6. **Derivation:** Starting from the space axes and $\mathbf{M}$, derive ${}^e\mathbf{B}_j$ and the body-form product. Why are the joint indices not reversed?
7. **Proof:** Starting from the relative-transform chain, state the induction claim that leads to Equation Equation 10.19 and identify the conjugation step used in its proof.
8. **Boundary:** What do the space form, body form, and relative-transform chain return at $\mathbf{q} = \mathbf{0}_n$? Why may the joint factors not generally be reordered?

9. **Verification:** How should two independently evaluated pose matrices be compared without mixing translation and rotation units?
10. **Domain:** Which properties of $\mathbf{q}$ must a numerical evaluator check before forming the joint exponentials?

Answers and hints

1. Store the home end-effector pose $\mathbf{M}$ and one normalised space screw axis ${}^s\mathbf{S}_j$ for every joint, all constructed from the same zero arrangement and expressed in $\{s\}$. Supply the current joint-coordinate column $\mathbf{q}$ when evaluating the map.
2. The zero arrangement $\chi(\mathbf{0}_n)$ places every modelled robot point when all joint coordinates are zero. The matrix $\mathbf{M} = \mathbf{T}_{se}(\mathbf{0}_n)$ retains only the pose of the selected end-effector frame at that arrangement.
3. For the revolute joint, $-{}^s\mathbf{s} \times {}^s\mathbf{p}_\ell = [0 \ -2 \ 0]^\top$, so ${}^s\mathbf{S}_R = [0 \ -2 \ 0 \ 0 \ 0 \ 1]^\top$. For the prismatic joint, ${}^s\mathbf{S}_P = [0 \ -1 \ 0 \ 0 \ 0 \ 0]^\top$.
4. The product is $\mathbf{T}_{se} = \mathbf{E}_1(q_1)\mathbf{E}_2(q_2)\mathbf{E}_3(q_3)\mathbf{M}$. Matrix multiplication acts on the rightmost factor first, so joint 3 first moves the home end effector, joint 2 then carries joint 3 and the end effector, and joint 1 finally carries the whole outward assembly. Distal motion does not move a more proximal axis, allowing every factor to use its stored zero-arrangement screw axis. This construction calculates the final pose; it does not prescribe a time history for the physical joints.
5. The end-effector angle relative to $\{s\}$ is the sum of the two joint angles:

$$\theta_e = \theta_1 + \theta_2 = \frac{\pi}{2} - \frac{\pi}{2} = 0.$$

Therefore, the end-effector orientation is

$$\mathbf{R}_{se} = \mathbf{R}_z(\theta_e) = \mathbf{R}_z(0) = \mathbf{I}_3.$$

Substitute the given lengths and angles into the planar 2R position formula:

$$\begin{aligned} x_e &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ &= (2 \text{ m}) \cos\left(\frac{\pi}{2}\right) + (1 \text{ m}) \cos(0) \\ &= 0 \text{ m} + 1 \text{ m} = 1 \text{ m}, \\ y_e &= L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \\ &= (2 \text{ m}) \sin\left(\frac{\pi}{2}\right) + (1 \text{ m}) \sin(0) \\ &= 2 \text{ m} + 0 \text{ m} = 2 \text{ m}. \end{aligned}$$

Hence,

$${}^s\mathbf{p}_e = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \mathbf{m}, \quad \mathbf{T}_{se} = \begin{bmatrix} \mathbf{I}_3 & \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \mathbf{m} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

6. Use ${}^e\mathbf{B}_j = \text{Ad}_{\mathbf{M}^{-1}} {}^s\mathbf{S}_j$ and the identity $\mathbf{M}^{-1}\mathbf{E}_j(q_j)\mathbf{M} = \exp(\widehat{{}^e\mathbf{B}_j} q_j)$. Repeatedly substitute $\mathbf{E}_j\mathbf{M} = \mathbf{M}\mathbf{G}_j$ into the space product to obtain $\mathbf{T}_{se} = \mathbf{M}\mathbf{G}_1 \cdots \mathbf{G}_n$. Conjugation changes each factor's coordinate representation but does not change its joint index, so the index order remains $1, \dots, n$.
7. The claim is $\mathbf{T}_{01}(q_1) \cdots \mathbf{T}_{k-1,k}(q_k) = \mathbf{E}_1(q_1) \cdots \mathbf{E}_k(q_k) \mathbf{T}_{sk}(\mathbf{0}_n)$. In the induction step, substitute $\mathbf{T}_{k-1,k}(q_k) = \exp(\widehat{{}^{k-1}\mathbf{S}_k} q_k) \mathbf{T}_{k-1,k}(0)$, insert $[\mathbf{T}_{s,k-1}(\mathbf{0}_n)]^{-1} \mathbf{T}_{s,k-1}(\mathbf{0}_n) = \mathbf{I}_4$, and use Equation Equation 10.17 to identify the conjugated local exponential as $\mathbf{E}_k(q_k)$.
8. Every exponential becomes $\mathbf{I}_4$, while every relative transform becomes its zero value, so all three forms return $\mathbf{M}$. Joint factors generally cannot be reordered because rigid rotations and translations do not commute; changing their order changes which outward geometry each joint carries.
9. First verify that both matrices describe the same end-effector frame relative to the same space frame and use the same conventions. Compare $e_p = \|\mathbf{p}_a - \mathbf{p}_b\|_2$ with a length tolerance ε_p in metres. Form $\mathbf{R}_\Delta = \mathbf{R}_a^\top \mathbf{R}_b$, recover its principal rotation vector ϕ_Δ , and compare $e_R = \|\phi_\Delta\|_2$ with an angular tolerance ε_R in radians. Accept agreement only when both residual tests pass; do not combine metre-valued and radian-valued residuals in one unscaled Euclidean norm.
10. Verify that every q_j is finite, belongs to its declared domain $\mathcal{Q}_j$, has the correct joint-dependent unit, and follows the declared wrapped or unwrapped angle representation when joint j is revolute.

10.3 Velocity kinematics and robot Jacobians

10.3.1 Task Jacobians

10.3.2 Space Jacobians

10.3.3 Body Jacobians

10.3.4 The body–space adjoint relation

10.4 Rank, singularities, and conditioning

10.4.1 Task-specific rank loss

10.4.2 Singular values and numerical conditioning

10.4.3 Full-twist and reduced-task Jacobians

10.5 Modelling limits and implementation checks

10.5.1 Supported serial-chain model

10.5.2 Explicit exclusions

10.5.3 Analytic and finite-difference checks

Cumulative chapter review

Answers and hints

References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapter 2 on robot configuration, Chapter 4 on relative-transform forward kinematics and the space and body forms of the product of exponentials, and the opening

of Chapter 5 on task-coordinate forward maps, the multivariable chain rule, and the derivative mapping from joint rates to task rates.

- John J. Craig, *Introduction to Robotics: Mechanics and Control*, Chapter 3 on joint space, link-frame assignment, parent–child transform chains, and the end-link pose as a function of the joint variables, and Chapter 5, especially Section 5.7, on the Jacobian as a multidimensional derivative and the velocity form of the chain rule.
- Peter Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, Chapter 7 on configuration coordinates, elementary transform sequences, parent–child link products, and forward kinematics, and Chapter 8, especially Section 8.1, on task-space derivatives and computational Jacobian evaluation.

11 General Robot Kinematics and Jacobians Implementation

Prerequisites

Section 10.1 defines a joint-coordinate column and the allowed set for each coordinate. Recall that revolute coordinates are signed angles in radians and prismatic coordinates are signed displacements in metres. Section 9.2 introduces validated C++ value objects, public factories, private constructors, and exceptions; the same construction pattern is used here.

Section 10.2 establishes the space-form pose calculation from fixed joint axes at the zero arrangement and the home end-effector pose. Recall that screw columns use linear-first ordering and that transformation products act on coordinate columns rightmost first. Section 9.10 implements the conversion from six exponential coordinates to a rigid displacement, and Section 9.4 implements ordered transformation composition.

11.1 Typed joint-coordinate limits

An allowed interval describes which coordinate values a caller may supply for one joint. For joint index j , let $q_j \in \mathbb{R}$ be its coordinate, and let $\mathcal{Q}_j \subseteq \mathbb{R}$ be its allowed set, as in Section 10.1. When both bounds are present, denote the lower bound by ℓ_j and the upper bound by u_j , in the same unit as q_j . The membership rule is

$$q_j \in \mathcal{Q}_j \iff \ell_j \leq q_j \leq u_j.$$

The bounds are inclusive. Either side may be unbounded; in that case its inequality is omitted. The software accepts only finite coordinate values and requires every supplied bound to be finite. Construction checks that $\ell_j \leq u_j$ when both bounds exist. Equality describes an interval containing one value. Zero need not belong to the interval: a joint's geometric zero reference can lie outside its allowed operating range.

11.1.1 Representing an optional bound

The angular and displacement intervals use separate C++ types. Each stores two values that may be absent. The complete class declarations are:

```
#include <optional>

namespace rigid_body_kinematics
{
    class RevoluteLimits
    {
    public:
        static RevoluteLimits from_bounds(
            std::optional<double> lower_bound_radians,
            std::optional<double> upper_bound_radians);

        [[nodiscard]] std::optional<double> lower_bound_radians() const noexcept;
        [[nodiscard]] std::optional<double> upper_bound_radians() const noexcept;

        [[nodiscard]] bool contains(double angle_radians) const noexcept;

    private:
        RevoluteLimits(
            std::optional<double> lower_bound_radians,
            std::optional<double> upper_bound_radians);

        std::optional<double> lower_bound_radians_;
        std::optional<double> upper_bound_radians_;
    };

    class PrismaticLimits
    {
    public:
        static PrismaticLimits from_bounds(
            std::optional<double> lower_bound_metres,
            std::optional<double> upper_bound_metres);

        [[nodiscard]] std::optional<double> lower_bound_metres() const noexcept;
        [[nodiscard]] std::optional<double> upper_bound_metres() const noexcept;

        [[nodiscard]] bool contains(double displacement_metres) const noexcept;

    private:
        PrismaticLimits(
```

```

        std::optional<double> lower_bound_metres,
        std::optional<double> upper_bound_metres);

    std::optional<double> lower_bound_metres_;
    std::optional<double> upper_bound_metres_;
};

} // namespace rigid_body_kinematics

```

`std::optional<double>` owns either one `double` or no value. Here a present value stores ℓ_j or u_j ; absence means that side imposes no bound. These two small declarations illustrate the storage alternatives:

```

const std::optional<double> supplied_bound{0.2};
const std::optional<double> absent_bound = std::nullopt;

```

`supplied_bound` contains the number 0.2. `absent_bound` contains no number: `std::nullopt` is the missing-value marker supplied by `<optional>`. An infinite number would be a present value and would be rejected during limits construction.

`RevoluteLimits` labels its stored numbers in radians, while `PrismaticLimits` labels them in metres. Keeping the classes distinct allows an interface to require the appropriate interval type. The scalar arguments are still ordinary `double` values, so callers must supply the declared units; the compiler cannot identify a number entered in degrees.

11.1.2 Validating before construction

Both interval types use one internal check because finiteness and bound ordering have the same numerical meaning for either unit. These are complete production helper definitions; their containing source includes `<cmath>`, `<optional>`, and the serial-chain exception declaration.

```

namespace rigid_body_kinematics
{
    namespace
    {

        void validate_bounds(
            const std::optional<double> & lower_bound,
            const std::optional<double> & upper_bound)
        {
            if (
                (lower_bound.has_value() && !std::isfinite(*lower_bound)) ||
                (upper_bound.has_value() && !std::isfinite(*upper_bound))) {

```

```

    throw SerialChainException(
        SerialChainError::invalid_joint_limits,
        "Present joint-limit bounds must be finite.");
}

if (
    lower_bound.has_value() && upper_bound.has_value() &&
    *lower_bound > *upper_bound) {
    throw SerialChainException(
        SerialChainError::invalid_joint_limits,
        "The lower joint limit must not exceed the upper joint limit.");
}
}

bool interval_contains(
    const std::optional<double> & lower_bound,
    const std::optional<double> & upper_bound, double value) noexcept
{
    if (!std::isfinite(value)) {
        return false;
    }

    return (!lower_bound.has_value() || *lower_bound <= value) &&
        (!upper_bound.has_value() || value <= *upper_bound);
}

} // namespace
} // namespace rigid_body_kinematics

```

The unnamed namespace confines these helper names to the source file. Each `const std::optional<double> &` parameter borrows an existing optional for the duration of the call; the helper neither copies nor modifies it. `lower_bound` and `upper_bound` represent the optional ℓ_j and u_j . In `interval_contains`, `value` represents the supplied q_j .

`has_value()` tests whether an optional contains a number, and unary `*` retrieves that number. The built-in logical operators evaluate from left to right and short-circuit: `&&` skips its right operand when the left is false, while `||` skips its right operand when the left is true. Consequently, the code never dereferences an absent optional. The same rules make an absent bound automatically satisfy its side of the membership test.

`std::isfinite`, supplied by `<cmath>`, rejects both NaN and infinity. Invalid construction throws `SerialChainException` with the category `SerialChainError::invalid_joint_limits` and a message. The exception leaves the factory before it can return a limits object. An out-of-range or non-finite membership query instead returns `false`: the already constructed interval remains valid even when a supplied coordinate is unsuitable.

Membership uses the stored bounds directly. It adds no tolerance, wraps no angle, and

clamps no displacement. A coordinate just outside a finite bound fails. An unbounded side permits arbitrarily large finite values as far as this interval check is concerned; it does not guarantee that a subsequent geometric calculation can represent their results.

The angular factory, constructor, and membership method connect those helpers to the stored value. These complete function definitions are an excerpt from the implementation inside the `rigid_body_kinematics` namespace; the simple accessor definitions are omitted.

```
RevoluteLimits RevoluteLimits::from_bounds(  
    std::optional<double> lower_bound_radians,  
    std::optional<double> upper_bound_radians)  
{  
    validate_bounds(lower_bound_radians, upper_bound_radians);  
  
    return RevoluteLimits(lower_bound_radians, upper_bound_radians);  
}  
  
RevoluteLimits::RevoluteLimits(  
    std::optional<double> lower_bound_radians,  
    std::optional<double> upper_bound_radians)  
: lower_bound_radians_(lower_bound_radians),  
  upper_bound_radians_(upper_bound_radians)  
{  
}  
  
bool RevoluteLimits::contains(double angle_radians) const noexcept  
{  
    return interval_contains(  
        lower_bound_radians_, upper_bound_radians_, angle_radians);  
}
```

`RevoluteLimits::` identifies an out-of-class definition as belonging to that class. The factory validates its parameters before calling the private constructor. The constructor's member initializer list, introduced by `:`, initializes the two stored optionals directly from those parameters. The object owns those values after the factory returns; it does not retain references to the caller's variables. `contains` forwards the stored interval and the current angle to the common membership helper. The displacement class follows the same sequence with its metre-labelled members and argument.

11.1.3 Using an interval

Example: A joint interval that excludes zero

An angular operating range can permit 0.2 through 1.0 radians while excluding the geometric zero reference. A separate displacement range can have only a lower bound. This usage example constructs both intervals and queries their membership:

```
const auto angular_limits =
    rigid_body_kinematics::RevoluteLimits::from_bounds(0.2, 1.0);

const bool lower_endpoint_allowed = angular_limits.contains(0.2);
const bool zero_allowed = angular_limits.contains(0.0);

const auto displacement_limits =
    rigid_body_kinematics::PrismaticLimits::from_bounds(-0.2, std::nullopt);

const bool extension_allowed = displacement_limits.contains(0.5);
```

`auto` deduces the type returned by each factory: `angular_limits` is a `RevoluteLimits` and `displacement_limits` is a `PrismaticLimits`. Supplying a `double` to an optional parameter constructs a present optional containing that number. `std::nullopt` constructs an absent upper bound. The three Boolean results are respectively `true`, `false`, and `true`: the angular lower endpoint is inclusive, zero is outside that angular range, and 0.5 m exceeds the displacement lower bound of -0.2 m.

11.1.4 Verification meaning

The focused interval tests check an angular range that excludes zero while including both endpoints, displacement intervals with either side absent, and a revolute interval with neither bound present. Separate rejection cases check a NaN angular bound, an infinite displacement bound, and reversed angular bounds. The rejection tests inspect the exception category as well as requiring construction to fail. Together these cases exercise interval semantics and construction failure without depending on a robot pose calculation.

11.2 Revolute and prismatic joints

A joint definition converts axis geometry into a fixed screw column that can be reused for many coordinate queries. For joint index j , let ${}^s\mathbf{s}_j \in \mathbb{R}^3$ be the dimensionless positive unit direction of its axis, expressed in the fixed space frame $\{s\}$ at the robot's zero arrangement. For a revolute joint, let ${}^s\mathbf{p}_{\ell,j} \in \mathbb{R}^3$ be the position of one point on its axis line relative to the

space-frame origin, expressed in $\{s\}$ and measured in metres. The subscript ℓ identifies the axis line. From Section 10.2, the linear-first space screw column ${}^s\mathbf{S}_j \in \mathbb{R}^6$ is

$${}^s\mathbf{S}_j = \begin{cases} \begin{bmatrix} -{}^s\mathbf{s}_j \times {}^s\mathbf{p}_{\ell,j} \\ {}^s\mathbf{s}_j \end{bmatrix}, & \text{revolute,} \\ \begin{bmatrix} {}^s\mathbf{s}_j \\ \mathbf{0}_3 \end{bmatrix}, & \text{prismatic.} \end{cases}$$

Here $\times$ is the three-dimensional cross product, and $\mathbf{0}_3$ is the three-entry zero column. A positive revolute coordinate follows the right-hand rule about the declared direction; a positive prismatic coordinate translates along it. The revolute upper block is a linear displacement coefficient per radian of joint motion, while its lower block is the dimensionless rotation direction. The prismatic upper block is a dimensionless translation direction and its lower block is zero. Only the three-entry direction is normalized; normalizing the entire six-entry screw would change the joint's coordinate scale and mix the blocks' physical meanings.

The screw column is not a transformation and is not a current velocity. It describes the joint's unit motion relative to its geometric zero reference. Multiplication by an angle or displacement, followed by the spatial exponential, produces a finite active displacement. A prismatic joint needs no axis point because translating a rigid body along a direction does not depend on a selected line's offset.

11.2.1 Storing geometry and typed limits

The two joint classes own their geometry, the matching interval type, and the screw derived from that geometry. The complete declarations below omit only header guards; their project headers supply the interval types and the linear-first six-entry column type.

```
#include <Eigen/Core>
#include <variant>

#include "rigid_body_kinematics/joint_limits.hpp"
#include "rigid_body_kinematics/lie_algebra.hpp"

namespace rigid_body_kinematics
{
class RevoluteJoint
{
public:
    [[nodiscard]] static RevoluteJoint from_axis(
        const Eigen::Vector3d & space_axis_direction,
```



```

    const Eigen::Vector3d & space_axis_point_metres, RevoluteLimits limits);

[[nodiscard]] const Eigen::Vector3d & space_axis_direction() const noexcept;
[[nodiscard]] const Eigen::Vector3d & space_axis_point_metres()
    const noexcept;
[[nodiscard]] const Vector6LinearFirst & space_screw_axis() const noexcept;
[[nodiscard]] const RevoluteLimits & limits() const noexcept;

private:
    RevoluteJoint(
        Eigen::Vector3d space_axis_direction,
        Eigen::Vector3d space_axis_point_metres, RevoluteLimits limits,
        Vector6LinearFirst space_screw_axis);

    Eigen::Vector3d space_axis_direction_;
    Eigen::Vector3d space_axis_point_metres_;
    RevoluteLimits limits_;
    Vector6LinearFirst space_screw_axis_;
};

class PrismaticJoint
{
public:
    [[nodiscard]] static PrismaticJoint from_axis(
        const Eigen::Vector3d & space_axis_direction, PrismaticLimits limits);

    [[nodiscard]] const Eigen::Vector3d & space_axis_direction() const noexcept;
    [[nodiscard]] const Vector6LinearFirst & space_screw_axis() const noexcept;
    [[nodiscard]] const PrismaticLimits & limits() const noexcept;

private:
    PrismaticJoint(
        Eigen::Vector3d space_axis_direction, PrismaticLimits limits,
        Vector6LinearFirst space_screw_axis);

    Eigen::Vector3d space_axis_direction_;
    PrismaticLimits limits_;
    Vector6LinearFirst space_screw_axis_;
};

using JointDefinition = std::variant<RevoluteJoint, PrismaticJoint>;

} // namespace rigid_body_kinematics

```

`Eigen::Vector3d` stores a fixed three-entry column of `double` values. `Vector6LinearFirst`,

introduced by the spatial kernel, stores six `double` values with the linear block first. The direction parameter is allowed to be non-unit; the corresponding stored member contains the normalized direction. The revolute point member contains the supplied point in metres. Each `space_screw_axis_` member owns the derived sS_j , so evaluation need not repeat normalization or the cross product.

The factories borrow their geometry inputs through `const` references and take the validated limits by value. Requiring `RevoluteLimits` for a revolute factory prevents accidentally passing a `PrismaticLimits` object, although callers still have to supply correctly labelled scalar units when constructing the limits. The private constructors receive already checked data. As with the interval classes, there are no individual setters, but ordinary copying and assignment can copy or replace complete joint values.

The accessors return read-only references to stored members, not copies. A caller must not retain such a reference beyond the lifetime of its owning joint. Keeping the owner alive and unchanged also keeps the reference's meaning stable.

The `using` declaration gives the name `JointDefinition` to `std::variant<RevoluteJoint, PrismaticJoint>`. A variant stores one of its listed alternatives by value and records which alternative it holds. This permits one ordered collection to contain both joint types without separating the screw from the geometry and limits that give it meaning. The common container does not erase which unit its coordinate requires.

11.2.2 Normalizing the positive direction

Both joint factories need the same operation: divide a finite, nonzero direction by its Euclidean length. The complete internal helper is shown below. It is defined in an unnamed namespace inside the implementation namespace; the source includes `<cmath>` and the serial-chain exception declaration.

```
Eigen::Vector3d normalize_axis_direction(const Eigen::Vector3d & axis_direction)
{
    if (!axis_direction.allFinite()) {
        throw SerialChainException(
            SerialChainError::non_finite, "Joint axis direction must be finite.");
    }

    const double direction_norm = axis_direction.stableNorm();

    if (direction_norm == 0.0) {
        throw SerialChainException(
            SerialChainError::invalid_model, "Joint axis direction must be nonzero.");
    }

    if (!std::isfinite(direction_norm)) {
```

```

    throw SerialChainException(
        SerialChainError::unsupported_magnitude,
        "Joint axis direction cannot be normalized safely.");
}

const Eigen::Vector3d normalized_direction = axis_direction / direction_norm;

if (!normalized_direction.allFinite()) {
    throw SerialChainException(
        SerialChainError::unsupported_magnitude,
        "Normalized joint axis direction is not finite.");
}

return normalized_direction;
}

```

`allFinite()` checks all vector entries. `stableNorm()` computes the Euclidean length using scaling to reduce intermediate overflow and underflow, rather than directly summing unscaled squared entries. A finite input vector can still have a length too large to represent as a `double`; the explicit finiteness check therefore remains necessary. A zero direction has no orientation and cannot be repaired by choosing an arbitrary axis.

`direction_norm` is the length of the supplied direction, and `normalized_direction` is $^s\mathbf{s}_j$. Division by that positive length preserves the declared sign. The final check rejects a non-finite computed direction instead of letting invalid geometry enter a joint object.

11.2.3 Constructing a revolute screw

The revolute factory checks the point, normalizes the direction, and forms the cross-product block. These complete production definitions appear inside the implementation namespace; `<Eigen/Geometry>` supplies the cross-product operation and `<utility>` supplies the move utility.

```

RevoluteJoint RevoluteJoint::from_axis(
    const Eigen::Vector3d & space_axis_direction,
    const Eigen::Vector3d & space_axis_point_metres, RevoluteLimits limits)
{
    if (!space_axis_point_metres.allFinite()) {
        throw SerialChainException(
            SerialChainError::non_finite, "Revolute axis point must be finite.");
    }

    const Eigen::Vector3d normalized_direction =
        normalize_axis_direction(space_axis_direction);
}

```

```

const Eigen::Vector3d linear_screw_block =
    -normalized_direction.cross(space_axis_point_metres);

if (!linear_screw_block.allFinite()) {
    throw SerialChainException(
        SerialChainError::unsupported_magnitude,
        "Revolute screw-axis calculation is not finite.");
}

Vector6LinearFirst space_screw_axis;
space_screw_axis.head<3>() = linear_screw_block;
space_screw_axis.tail<3>() = normalized_direction;

return RevoluteJoint(
    normalized_direction, space_axis_point_metres, std::move(limits),
    space_screw_axis);
}

RevoluteJoint::RevoluteJoint(
    Eigen::Vector3d space_axis_direction, Eigen::Vector3d space_axis_point_metres,
    RevoluteLimits limits, Vector6LinearFirst space_screw_axis)
: space_axis_direction_(std::move(space_axis_direction)),
  space_axis_point_metres_(std::move(space_axis_point_metres)),
  limits_(std::move(limits)),
  space_screw_axis_(std::move(space_screw_axis))
{
}

```

`linear_screw_block` represents $-\mathbf{s}_j \times \mathbf{p}_{\ell,j}$. The leading minus sign applies to the whole cross product; swapping the cross-product operands would also change its sign. Even with finite operands, an intermediate calculation can exceed the representable range, so the factory checks the resulting block before constructing the joint.

`head<3>()` selects the first three entries and `tail<3>()` selects the last three. The angle-bracket argument fixes the block length at compile time. These expressions are writable views into `space_screw_axis`, so the assignments fill entries 0, 1, 2 and 3, 4, 5, respectively. Both blocks are assigned before the six-entry column is used; the default declaration alone does not initialize its numerical entries.

The constructor code stores the checked geometry and limits in the new joint through three parts:

1. **Constructor** (`RevoluteJoint::RevoluteJoint`): This special member function initializes a new joint. The factory (`RevoluteJoint::from_axis`) calls it after checking the inputs and calculating the screw.

2. **Member initializer list (the part after :):** This is not a separate function. It initializes the joint's stored members before the constructor body runs. For example, `space_axis_direction_(std::move(space_axis_direction))` initializes the stored direction from the direction parameter.
3. **Move utility (std::move):** `std::move` marks an existing object as available for move construction: creating a new object that can reuse the original object's data instead of copying it, when the type supports this. In `space_axis_direction_(std::move(space_axis_direction))` the stored member is initialized from the direction parameter. `std::move` allows C++ to use the type's move constructor, but this fixed-size Eigen vector may still copy its three numbers. The joint keeps its own direction, axis point, limits, and screw; these remain available after the factory and constructor finish.

For example, a supplied direction $(0, 0, 5)^T$ and axis point $(2, 0, 0)^T$ m produce the unit direction $(0, 0, 1)^T$ and linear block $(0, -2, 0)^T$ in metres per radian. The stored screw is therefore $(0, -2, 0, 0, 0, 1)^T$. The point is on a line parallel to the space z -axis but displaced from the space origin; the upper block records that offset's effect on the motion.

11.2.4 Constructing a prismatic screw

A prismatic factory uses the same direction normalization but puts the result in the linear block. These are its complete factory and constructor definitions in the implementation namespace.

```
PrismaticJoint PrismaticJoint::from_axis(
    const Eigen::Vector3d & space_axis_direction, PrismaticLimits limits)
{
    const Eigen::Vector3d normalized_direction =
        normalize_axis_direction(space_axis_direction);

    Vector6LinearFirst space_screw_axis;
    space_screw_axis.head<3>() = normalized_direction;
    space_screw_axis.tail<3>().setZero();

    return PrismaticJoint(
        normalized_direction, std::move(limits), space_screw_axis);
}

PrismaticJoint::PrismaticJoint(
    Eigen::Vector3d space_axis_direction, PrismaticLimits limits,
    Vector6LinearFirst space_screw_axis)
: space_axis_direction_(std::move(space_axis_direction)),
  limits_(std::move(limits)),
  space_screw_axis_(std::move(space_screw_axis))
{
}
```

```
}
```

`setZero()` fills the selected angular block with zeros. Thus a supplied direction $(0, -4, 0)^T$ produces the screw $(0, -1, 0, 0, 0, 0)^T$. Multiplying that screw by a displacement of 0.4 m gives a linear exponential-coordinate block $(0, -0.4, 0)^T$ m and zero rotation. Unlike a revolute screw, it has no axis-offset contribution.

The following representative accessor definitions complete the ownership picture. The other declared accessors use the same direct return of their corresponding stored member.

```
const Vector6LinearFirst & RevoluteJoint::space_screw_axis() const noexcept
{
    return space_screw_axis_;
}

const PrismaticLimits & PrismaticJoint::limits() const noexcept
{
    return limits_;
}
```

Each returned reference refers to a member of the joint, not to a temporary created by the accessor. The caller can inspect that member but cannot modify it through the read-only reference.

11.2.5 Verification meaning

The joint tests check normalization together with the derived six entries for a displaced revolute axis and a negative-direction prismatic axis. A separate geometric test selects two points on the same oriented revolute line and checks that both definitions produce the same screw; it protects the distinction between the physical axis line and the arbitrary point chosen to describe it. Rejection tests exercise a zero direction, a non-finite direction, and a non-finite axis point, including their typed error categories.

11.3 Serial-chain model

A fixed-base open serial chain needs one geometric reference pose and an ordered sequence of joint definitions. Let $n \geq 1$ be the number of independent one-degree-of-freedom joints, ordered from the fixed base toward the selected end effector. Let $\{e\}$ be that end-effector frame. As in Section 10.2, the home pose $\mathbf{M} \in SE(3)$ is

$$\mathbf{M} := \mathbf{T}_{se}(\mathbf{0}_n) = \begin{bmatrix} \mathbf{R}_{se,0} & {}^s\mathbf{p}_{e,0} \\ \mathbf{0}_3^T & 1 \end{bmatrix}.$$

Here $SE(3)$ is the set of rigid transformations represented by 4×4 homogeneous matrices, $\mathbf{0}_n$ is the all-zero joint-coordinate column, and $\mathbf{T}_{se}$ is the pose of $\{e\}$ relative to $\{s\}$. The dimensionless 3×3 rotation block $\mathbf{R}_{se,0}$ gives the end-effector frame's orientation relative to the space frame when all joint coordinates are zero. The three-entry column ${}^s\mathbf{p}_{e,0}$ gives the end-effector origin's position relative to the space origin, expressed in $\{s\}$ and measured in metres. The superscript T transposes the zero column into a row. The matrix maps homogeneous end-effector point coordinates to space-frame coordinates by left multiplication.

The **geometric forward-kinematics model** collects the fixed data needed to calculate the selected end effector's pose. As defined in Equation Equation 10.8, it is the ordered pair

$$\mathcal{M} := (\mathbf{M}, ({}^s\mathbf{S}_j)_{j=1}^n),$$

where:

- $\mathbf{M}$ is the home end-effector pose relative to the fixed space frame $\{s\}$.
- $({}^s\mathbf{S}_j)_{j=1}^n$ is the ordered sequence of space screw columns, from joint 1 at the base to joint n nearest the end effector, all expressed in $\{s\}$ at the same zero arrangement.
- n is the number of joints.

The software's **serial-chain model** stores this geometric description together with the information needed to construct and validate it. It contains the home pose and ordered joint definitions; each joint definition retains its type, axis geometry, coordinate limits, and derived screw column. These are fixed reference data, not the current joint coordinates: the same model can be used to calculate poses for different coordinate inputs.

Every axis must describe the same zero arrangement and use the same space frame as $\mathbf{M}$. This agreement is a caller precondition: finite numerical entries alone cannot establish which physical frame or robot arrangement they describe. The zero arrangement defines the geometry even when zero lies outside a joint's operating interval; storing the home pose does not make a zero-coordinate query admissible.

11.3.1 Owning the home pose and ordered joints

The complete class declaration is:

```
#include <cstdint>
#include <vector>

#include "rigid_body_kinematics/joint_definition.hpp"
```

```

#include "rigid_body_kinematics/transform3.hpp"

namespace rigid_body_kinematics
{

class SerialChainModel
{
public:
    [[nodiscard]] static SerialChainModel from_home_and_joints(
        Transform3 home_pose, std::vector<JointDefinition> joint_definitions);

    [[nodiscard]] const Transform3 & home_pose() const noexcept;
    [[nodiscard]] const std::vector<JointDefinition> & joint_definitions()
        const noexcept;
    [[nodiscard]] std::size_t joint_count() const noexcept;

private:
    SerialChainModel(
        Transform3 home_pose, std::vector<JointDefinition> joint_definitions);

    Transform3 home_pose_;
    std::vector<JointDefinition> joint_definitions_;
};

} // namespace rigid_body_kinematics

```

`home_pose_` owns the validated representation of $\mathbf{M}$. `std::vector<JointDefinition>` is a variable-length, ordered container that owns its elements. Its first element is mathematical joint 1, and its last is joint n . Each element retains its joint kind, geometry, limits, and screw together; separate arrays do not have to be kept in matching order. This C++ container is not the numerical joint-coordinate column.

`joint_count()` returns `std::size_t`, the unsigned integer type used for container sizes. Its value is n . The other two accessors return read-only references, allowing repeated evaluation to borrow the stored model instead of copying all its joints. The model contains no current coordinate input or evaluation policy: changing the requested robot configuration does not require rebuilding the geometric reference.

11.3.2 Validating construction without duplicating nested checks

The complete model implementation below is an excerpt inside the implementation namespace. Its source also includes `<utility>` and the serial-chain exception declaration.


```

SerialChainModel SerialChainModel::from_home_and_joints(
    Transform3 home_pose, std::vector<JointDefinition> joint_definitions)
{
    if (joint_definitions.empty()) {
        throw SerialChainException(
            SerialChainError::invalid_model,
            "A serial-chain model must contain at least one joint.");
    }

    return SerialChainModel(std::move(home_pose), std::move(joint_definitions));
}

SerialChainModel::SerialChainModel(
    Transform3 home_pose, std::vector<JointDefinition> joint_definitions)
: home_pose_(std::move(home_pose)),
  joint_definitions_(std::move(joint_definitions))
{
}

const Transform3 & SerialChainModel::home_pose() const noexcept
{
    return home_pose_;
}

const std::vector<JointDefinition> & SerialChainModel::joint_definitions()
    const noexcept
{
    return joint_definitions_;
}

std::size_t SerialChainModel::joint_count() const noexcept
{
    return joint_definitions_.size();
}

```

`empty()` tests whether the supplied joint container has no elements. Rejecting that case enforces $n \geq 1$. The factory does not repeat rotation, axis, or interval checks: those properties were established when its input values were constructed. It neither sorts the list nor recalculates the screw columns, so the caller's base-to-end-effector order is retained exactly.

The factory receives its inputs by value. Passing an existing ordinary variable copies it into the parameter; explicitly moving a suitable variable permits move construction instead. The factory then moves its parameters into the private constructor, which moves them into the stored members. For the joint vector, move construction transfers ownership of its allocated element storage. This is different from keeping a reference to the caller's vector:

subsequently changing the caller's original vector cannot edit the model's stored joints.

The accessors, `joint_definitions()` and `joint_count`, borrow the model's members. Such references should be used while the model remains alive and unchanged; replacing or moving the owner can change or invalidate access to its former contents. The model itself remains an ordinary value that can be copied or replaced as a whole. Its public interface provides no operation to mutate an individual stored joint.

11.3.3 Verification meaning

The model tests check that construction retains a nonidentity home pose and a joint, rejects an empty sequence with the model-error category, and preserves a mixed revolute-prismatic order. The order check matters mathematically: exchanging two stored joints can change the transformation product even if each joint is independently valid. These tests check representation and construction, not the computed pose for a nonzero coordinate query.

11.4 Space-form forward kinematics

Given the fixed model and a current coordinate column $\mathbf{q} = (q_1, \dots, q_n)^\top$, forward kinematics returns $\mathbf{T}_{se}(\mathbf{q}) \in SE(3)$. Its allowed input set is $\mathcal{Q} = \mathcal{Q}_1 \times \dots \times \mathcal{Q}_n \subseteq \mathbb{R}^n$, the Cartesian product of the declared joint intervals. Entry q_j is in radians for a revolute joint or metres for a prismatic joint; the whole column has no single physical unit.

Recall the space-form construction from Section 10.2. For each joint, define its six-entry exponential-coordinate column ${}^s\boldsymbol{\eta}_j$, its active displacement $\mathbf{E}_j(q_j) \in SE(3)$, and the resulting pose by

$${}^s\boldsymbol{\eta}_j := {}^s\mathbf{S}_j q_j, \quad \mathbf{E}_j(q_j) := \exp(\widehat{{}^s\boldsymbol{\eta}_j}), \quad \mathbf{T}_{se}(\mathbf{q}) = \mathbf{E}_1(q_1) \cdots \mathbf{E}_n(q_n) \mathbf{M}.$$

The hat operator maps the linear-first six-entry column to its 4×4 Lie-algebra matrix, and $\exp$ is the matrix exponential implemented by the spatial kernel.

Write ${}^s\boldsymbol{\eta}_j = [\boldsymbol{\rho}_j^\top \ \boldsymbol{\phi}_j^\top]^\top$, where the three-entry blocks $\boldsymbol{\rho}_j$ and $\boldsymbol{\phi}_j$ are linear exponential coordinates in metres and angular exponential coordinates in radians, respectively, expressed in $\{s\}$. Recall that

$$\widehat{{}^s\boldsymbol{\eta}_j} = \begin{bmatrix} \widehat{\boldsymbol{\phi}_j} & \boldsymbol{\rho}_j \\ \mathbf{0}_3^\top & 0 \end{bmatrix} \in \mathfrak{se}(3), \quad \exp(\widehat{{}^s\boldsymbol{\eta}_j}) = \begin{bmatrix} \exp(\widehat{\boldsymbol{\phi}_j}) & \mathbf{t}_j \\ \mathbf{0}_3^\top & 1 \end{bmatrix} \in SE(3).$$

Here $\widehat{\phi}_j$ is the 3×3 skew-symmetric cross-product matrix of ϕ_j , and $\mathbf{0}_3$ is the three-entry zero column. The three-entry translation column $\mathbf{t}_j$, expressed in $\{s\}$ and measured in metres, is calculated by the spatial exponential; it need not equal ρ_j when rotation is present.¹

The factors preserve base-to-end-effector joint order from left to right. They act rightmost first on coordinate columns, and each E_j is an active motion expressed in the fixed space frame. This factor order describes the final pose, not a time sequence for commanding the robot. The stored axes remain the zero-arrangement axes throughout evaluation; the ordered product accounts for the motion of the outward assembly.

11.4.1 Public input and output

The public declaration is:

```
namespace rigid_body_kinematics
{

[[nodiscard]] Transform3 space_form_forward_kinematics(
    const SerialChainModel & model,
    Eigen::Ref<const Eigen::VectorXd> joint_coordinates,
    const NumericalPolicy & policy = NumericalPolicy{});

} // namespace rigid_body_kinematics
```

`space_form_forward_kinematics` borrows `model` and returns an owned `Transform3` representing $T_{se}(\mathbf{q})$. `Eigen::VectorXd` is a dynamically sized column of `double` values. `Eigen::Ref<const Eigen::VectorXd>` provides read-only access to a compatible column: an ordinary compatible vector can be borrowed without copying its entries. Some expression inputs require Eigen to evaluate temporary storage, so this parameter is not a guarantee that every possible caller expression avoids a copy. The evaluator retains no coordinate reference after it returns.

`joint_coordinates` represents $\mathbf{q}$, and the mathematical entry q_j appears at zero-based C++ index $j - 1$. `policy` supplies numerical evaluation thresholds rather than physical robot properties. The default argument `NumericalPolicy{}` constructs the policy with its de-

¹For a revolute motion, let $\mathbf{p} \in \mathbb{R}^3$ be a point on the rotation axis, and let $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^3$ be a point's positions before and after the motion. All positions are measured from the space-frame origin, expressed in $\{s\}$, and given in metres. Let $\mathbf{R} \in SO(3)$ be the dimensionless rotation matrix. Rotation around that offset axis acts as

$$\mathbf{x}' = \mathbf{R}(\mathbf{x} - \mathbf{p}) + \mathbf{p} = \mathbf{R}\mathbf{x} + \underbrace{(\mathbf{p} - \mathbf{R}\mathbf{p})}_{\mathbf{t}}.$$

Thus the translation column is $\mathbf{t} = \mathbf{p} - \mathbf{R}\mathbf{p}$, measured in metres.

clared member defaults when the caller omits the third argument; the temporary remains alive for the call. The return type is a transform rather than an optional partial answer. Invalid queries throw a typed serial-chain exception.

11.4.2 Accessing either joint kind

The following two helpers let the evaluator query either joint type without duplicating the calculation. Their complete definitions belong to an unnamed namespace inside the implementation namespace; the variant operations use `<variant>`. The separate policy-validation helper is omitted.

```
bool joint_contains(
    const JointDefinition & joint_definition, double joint_coordinate)
{
    return std::visit(
        [joint_coordinate](const auto & joint) {
            return joint.limits().contains(joint_coordinate);
        },
        joint_definition);
}

const Vector6LinearFirst & space_screw_axis(
    const JointDefinition & joint_definition)
{
    return std::visit(
        [](const auto & joint) -> const Vector6LinearFirst & {
            return joint.space_screw_axis();
        },
        joint_definition);
}
```

11.4.2.1 What `std::visit` does

For one variant, the call has the following form. This is schematic syntax: the argument names are placeholders, not additional declarations in the implementation.

```
std::visit(visitor, variant_value);
```

1. **What problem it solves:** A variant can hold different types of objects, but holds only one at a time. For example, a joint definition holds either a revolute joint or a prismatic joint. `std::visit` lets you call a function with whichever joint is actually stored there.

2. **First argument — what to do:** `visitor` is the function you want to run on the stored object. A lambda lets you write that function directly inside the call, as in the helpers above.
3. **Second argument — where the object is:** `variant_value` is the variant holding the object. `std::visit` passes the object inside it to your function, not the whole variant.
4. **Which object the function receives:** If the variant holds a revolute joint, your function receives that revolute joint. If it holds a prismatic joint, your function receives that prismatic joint. Only one call happens, using the object currently stored.
5. **What comes back:** The answer from your function becomes the answer from `std::visit`. If your function checks a joint limit and returns `true`, `std::visit` returns `true` too. The function you supply performs the check; `std::visit` only connects it to the stored object and returns its answer.

11.4.2.2 How the joint helpers use it

1. **Variant and contained joint:** `joint_definition` is the variant containing a `RevoluteJoint` or a `PrismaticJoint`. In both helpers, the first argument to `std::visit` is the lambda, and the second is this variant. The lambda's `joint` parameter refers to the actual stored joint.
2. **Coordinate capture:** In `joint_contains`, `[joint_coordinate]` captures a copy of the coordinate to check. This is separate from the parameter supplied by `std::visit`: the capture provides the query, while the parameter provides the joint whose interval is queried.
3. **Joint parameter:** `const auto & joint` becomes `const RevoluteJoint &` or `const PrismaticJoint &` for the selected alternative. It borrows the joint without copying or modifying it. Both classes provide the operations used by the visitor; they do not need a common base class.
4. **Membership result:** `joint.limits().contains(joint_coordinate)` checks the appropriate interval and returns a `bool` for either joint type. The inner `return` passes that Boolean out of the lambda, `std::visit` returns it, and the outer `return` passes it out of `joint_contains`.
5. **Screw result:** In the screw helper, `[]` captures nothing because the supplied joint is the only input needed. The explicit `-> const Vector6LinearFirst &` makes the lambda return a read-only reference to that joint's stored screw. Both alternatives return the same reference type.
6. **Reference lifetime:** The lambda's `-> const Vector6LinearFirst &` makes it return a read-only reference to the screw stored in the joint. The outer helper also returns `const Vector6LinearFirst &`, so the caller receives a reference to that same stored screw, usable while the joint remains alive and unchanged. If the lambda's `-> const Vector6LinearFirst &` were removed, the lambda would instead return a copied `Vector6LinearFirst` value. The outer helper would still return `const Vector6LinearFirst &`, but that reference would point to the temporary copy, not the joint's stored screw. The temporary is destroyed during the helper's return, leaving the caller with a **dangling reference** to an object that no longer exists.

11.4.3 Evaluating the ordered product

The calculation has two stages: establish that the query is admissible, then accumulate the ordered displacements. The pseudocode includes the checks and failure behaviour so their role remains visible without reproducing every rejection branch.

```

Validate the numerical policy and coordinate count.
Reject any non-finite coordinate.

For each joint:
    Check its coordinate against its limits.
    For a revolute joint, check the supported angular magnitude.

Start the accumulated transform at identity.
For each joint in base-to-end-effector order:
    Multiply its stored screw by its coordinate.
    Reject non-finite exponential coordinates.
    Convert those coordinates to a displacement.
    Append the displacement on the right.

Append the home pose on the right and return.
On evaluation failure, report the error without returning a partial pose.

```

Production excerpt — joint-limit validation. After checking the numerical policy, coordinate count, and finiteness, the evaluator checks each coordinate against its joint's limits. Those earlier checks and the function wrapper are omitted here. The revolute angular-magnitude check, which follows the limit check inside the same production loop, is also omitted.

```

for (std::size_t index = 0; index < model.joint_count(); ++index) {
    const double coordinate =
        joint_coordinates(static_cast<Eigen::Index>(index));

    const JointDefinition & joint_definition =
        model.joint_definitions().at(index);

    if (!joint_contains(joint_definition, coordinate)) {
        throw SerialChainException(
            SerialChainError::joint_out_of_domain,
            "Joint " + std::to_string(index + 1) +
                " is outside its declared limits.");
    }
}

```

- The same `index` selects a joint definition and its coordinate.

- `joint_contains` uses `std::visit` to check that coordinate against the selected joint's angular or displacement limits.
- If the coordinate is outside its limits, the exception stops evaluation before any joint exponential is calculated.

Production excerpt — core accumulation, not a complete function. This excerpt uses the inputs from the public signature above after all validation has passed. The function wrapper, preceding validation code, and surrounding exception-translation blocks are omitted from this excerpt. Screw scaling, its finiteness check, exponential evaluation, and multiplication order are unchanged.

```
Transform3 result = Transform3::identity();

for (std::size_t index = 0; index < model.joint_count(); ++index) {
    const double coordinate =
        joint_coordinates(static_cast<Eigen::Index>(index));

    const Vector6LinearFirst exponential_coordinates =
        space_screw_axis(model.joint_definitions().at(index)) * coordinate;

    if (!exponential_coordinates.allFinite()) {
        throw SerialChainException(
            SerialChainError::unsupported_magnitude,
            "Joint " + std::to_string(index + 1) +
            " produced non-finite exponential coordinates.");
    }

    const Transform3 joint_displacement =
        Transform3::from_exponential_coordinates(
            exponential_coordinates, policy);

    result = result.compose(joint_displacement);
}

return result.compose(model.home_pose());
```

The same zero-based `index` selects a coordinate and its joint definition, preserving their base-to-end-effector correspondence. The explicit conversion to `Eigen::Index` connects the container's index type to Eigen's signed indexing type. The earlier coordinate-count check establishes that there is exactly one coordinate for every stored joint. Error messages report `index + 1`, the mathematical joint index.

After validation, `exponential_coordinates` is the owned column ${}^s\eta_j = {}^sS_j q_j$, and `joint_displacement` is $E_j(q_j)$. The multiplication can overflow even when both factors are finite, which explains the check before calling the kernel. `Transform3::from_exponential_coordinates` performs the coupled spatial exponential; the

evaluator does not construct a translation directly from the screw's upper entries.

`result` starts as the identity transform $\mathbf{I}_4$, the 4×4 identity matrix. Define $\mathbf{P}_0 = \mathbf{I}_4$ and let $\mathbf{P}_j$ be the partial product after joint j . The call `result.compose(joint_displacement)` implements

$$\mathbf{P}_j = \mathbf{P}_{j-1} \mathbf{E}_j(q_j), \quad j = 1, \dots, n.$$

Consequently the loop builds $\mathbf{E}_1$, then $\mathbf{E}_1 \mathbf{E}_2$, and finally $\mathbf{E}_1 \cdots \mathbf{E}_n$. The last composition appends $\mathbf{M}$ on the right. Starting from the home pose and using the same loop would instead place $\mathbf{M}$ on the left, which is generally a different pose. The identity initialization and final home-pose composition are therefore part of the equation-to-code mapping, not interchangeable setup choices.

Additional validation checks and error handling are omitted here for clarity; they remain in the production implementation.

Example: A rotating extension

Consider a revolute joint about the positive space z -axis through the origin, followed by a prismatic joint along the positive space x -direction at zero. The home end-effector orientation is the identity and its origin is at $(1, 0, 0)^T$ m. Query a revolute angle of $\pi/2$ radians and a prismatic displacement of 0.5 m. This is a usage example, not part of the production evaluator; the statements below belong inside a caller function after including the public forward-kinematics header and `<numbers>`.


```

namespace rbk = rigid_body_kinematics;

const auto revolute = rbk::RevoluteJoint::from_axis(
    Eigen::Vector3d(0.0, 0.0, 1.0), Eigen::Vector3d::Zero(),
    rbk::RevoluteLimits::from_bounds(-std::numbers::pi, std::numbers::pi));

const auto prismatic = rbk::PrismaticJoint::from_axis(
    Eigen::Vector3d(1.0, 0.0, 0.0),
    rbk::PrismaticLimits::from_bounds(-10.0, 10.0));

Eigen::Matrix4d home_matrix = Eigen::Matrix4d::Identity();
home_matrix(0, 3) = 1.0;

const auto model = rbk::SerialChainModel::from_home_and_joints(
    rbk::Transform3::from_matrix(home_matrix), {revolute, prismatic});

Eigen::VectorXd joint_coordinates(2);
joint_coordinates << std::numbers::pi / 2.0, 0.5;

const rbk::Transform3 pose =
    rbk::space_form_forward_kinematics(model, joint_coordinates);

const Eigen::Matrix4d pose_matrix = pose.matrix();

```

`namespace rbk = rigid_body_kinematics` introduces a shorter namespace alias for this example; it does not create another implementation. `home_matrix(0, 3)` selects the first translation entry using zero-based row and column indices. The braced list `{revolute, prismatic}` initializes the factory's vector parameter with copies of the two joint values in that order. Eigen's `<<` comma initializer fills the two-entry coordinate column in the written order. `pose_matrix` is an owned matrix representation of the returned pose.

An independent geometric calculation first extends the home position along its zero-arrangement x -direction and then rotates the outward assembly. Writing $\mathbf{R}_z(\pi/2)$ for the positive quarter-turn rotation about the space z -axis, the expected position is

$$\mathbf{R}_z(\pi/2) \left(\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 0 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 1.5 \\ 0 \end{bmatrix} \text{ m.}$$

The prismatic joint does not change orientation, so the expected complete matrix is

$$\mathbf{T}_{se}(\mathbf{q}) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 1.5 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The returned numerical entries should agree up to floating-point rounding. The translation lies along the rotated extension direction, not along the unchanged space x -axis. This distinguishes the ordered product from simply adding the prismatic displacement to an already rotated home position in fixed space coordinates.

11.4.4 Worked example: spatial two-revolute-joint motion

This example supplies an independent geometric reference for a two-joint arm whose motion leaves its home plane. It applies the ordered-rotation construction in Section 10.2: rotate the distal link about the elbow, then rotate the resulting arrangement about the base. This is a calculation order, not a requirement to move the joints one at a time; the same construction underlies the space-form product in Lynch and Park, *Modern Robotics*, Chapter 4, Section 4.1.1.

Let the link lengths be $L_1 = 2$ m and $L_2 = 1$ m. Both links initially point along $+x_s$, and the base is the origin of the fixed space frame $\{s\}$, identified with `world` for this example. Joint 1 rotates about $+z_s$ through the base. Joint 2 rotates about an axis that points along $+y_s$ through the elbow at home; joint 1 carries that physical axis with it. The joint angles q_1 and q_2 are in radians, with positive angles following the right-hand rule. Both lie in $[-\pi, \pi]$.

All position and link-vector columns below have three entries expressed along $\{s\}$ axes, with lengths in metres. Positions are measured from the base; the link vector is the difference between the end-effector and elbow positions. The dimensionless 3×3 rotation matrices act actively on columns. The end-effector frame $\{e\}$ is attached to the tip of link 2 and aligned with $\{s\}$ when both angles are zero.

11.4.4.1 Rotate the second link about the elbow

First hold joint 1 at zero. The elbow position is ${}^s\mathbf{p}_{\ell,0}$, where ℓ denotes the elbow and 0 its home position. The vector from the elbow to the end effector at home is ${}^s\mathbf{r}_0$:

$${}^s\mathbf{p}_{\ell,0} = \begin{bmatrix} L_1 \\ 0 \\ 0 \end{bmatrix}, \quad {}^s\mathbf{r}_0 = \begin{bmatrix} L_2 \\ 0 \\ 0 \end{bmatrix}.$$

With joint 1 held at zero, the second joint's axis points along $+y_s$. Its active right-handed rotation matrix is

$$\mathbf{R}_y(q_2) = \begin{bmatrix} \cos q_2 & 0 & \sin q_2 \\ 0 & 1 & 0 \\ -\sin q_2 & 0 & \cos q_2 \end{bmatrix}.$$

Joint 2 rotates the link vector about the elbow:

$$\mathbf{R}_y(q_2)^s \mathbf{r}_0 = \begin{bmatrix} L_2 \cos q_2 \\ 0 \\ -L_2 \sin q_2 \end{bmatrix}.$$

Write ${}^s\mathbf{p}_e(q_1, q_2) \in \mathbb{R}^3$ for the end-effector origin's position relative to the base when joint 1 has angle q_1 and joint 2 has angle q_2 . The subscript e identifies the end effector; the superscript s says that the three position components are expressed along frame $\{s\}$ axes, in metres. The parentheses contain the two input angles, not position components. Thus ${}^s\mathbf{p}_e(0, q_2)$ means the same position function evaluated with $q_1 = 0$ and joint 2 at angle q_2 .

Adding the elbow position to the rotated link vector gives

$${}^s\mathbf{p}_e(0, q_2) = {}^s\mathbf{p}_{\ell,0} + \mathbf{R}_y(q_2)^s \mathbf{r}_0 = \begin{bmatrix} L_1 + L_2 \cos q_2 \\ 0 \\ -L_2 \sin q_2 \end{bmatrix}.$$

The elbow position is added after rotating the link vector because joint 2 rotates about the elbow, not the base. The negative sign in the last entry follows from the right-hand rule: a positive rotation about $+y_s$ carries a vector initially along $+x_s$ toward $-z_s$. A negative second-joint angle therefore lifts the link toward $+z_s$.

11.4.4.2 Rotate the whole arrangement about the base

Joint 1 rotates both the first link and the bent second link about $+z_s$, using

$$\mathbf{R}_z(q_1) = \begin{bmatrix} \cos q_1 & -\sin q_1 & 0 \\ \sin q_1 & \cos q_1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Because the base is the space-frame origin, this rotation needs no additional offset translation. Applying it to the preceding position gives

$$\begin{aligned}
{}^s\mathbf{p}_e(q_1, q_2) &= \mathbf{R}_z(q_1) {}^s\mathbf{p}_e(0, q_2) \\
&= \begin{bmatrix} (L_1 + L_2 \cos q_2) \cos q_1 \\ (L_1 + L_2 \cos q_2) \sin q_1 \\ -L_2 \sin q_2 \end{bmatrix}.
\end{aligned}$$

Substituting the two link lengths yields

$${}^s\mathbf{p}_e(q_1, q_2) = \begin{bmatrix} (2 + \cos q_2) \cos q_1 \\ (2 + \cos q_2) \sin q_1 \\ -\sin q_2 \end{bmatrix} \text{ m.}$$

The same base rotation gives the elbow position:

$${}^s\mathbf{p}_\ell(q_1) = \mathbf{R}_z(q_1) {}^s\mathbf{p}_{\ell,0} = \begin{bmatrix} L_1 \cos q_1 \\ L_1 \sin q_1 \\ 0 \end{bmatrix}.$$

To calculate orientation, let $\mathbf{R}_{se}(q_1, q_2)$ be the dimensionless 3×3 matrix whose columns are the end-effector axes expressed in $\{s\}$. At home, those axes are aligned with $\{s\}$. With joint 1 held at zero, joint 2 rotates them by $\mathbf{R}_y(q_2)$. Joint 1 then rotates each resulting axis by $\mathbf{R}_z(q_1)$. Applying these rotations to each column gives

$$\begin{aligned}
\mathbf{R}_{se}(q_1, q_2) &= \mathbf{R}_z(q_1) \mathbf{R}_y(q_2) \\
&= \begin{bmatrix} \cos q_1 & -\sin q_1 & 0 \\ \sin q_1 & \cos q_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos q_2 & 0 & \sin q_2 \\ 0 & 1 & 0 \\ -\sin q_2 & 0 & \cos q_2 \end{bmatrix} \\
&= \begin{bmatrix} \cos q_1 \cos q_2 & -\sin q_1 \cos q_2 & \sin q_1 \sin q_2 \\ \sin q_1 \cos q_2 & \cos q_1 \cos q_2 & \cos q_1 \sin q_2 \\ -\sin q_2 & 0 & \cos q_2 \end{bmatrix}.
\end{aligned}$$

The multiplication order accounts for joint 1 carrying joint 2's axis; the second joint does not keep rotating about the unchanged world y -axis after the base turns. The elbow offset affects position, but not the orientation of the end-effector axes.

11.4.4.3 A bounded motion and its endpoint

Let the dimensionless scalar $u \in [0, 1]$ select the joint-coordinate column

$$\mathbf{q}(u) = u \begin{bmatrix} \pi/2 \\ -\pi/2 \end{bmatrix} \text{ rad.}$$

At $u = 0$, the elbow is at $(2, 0, 0)$ m, the end effector is at $(3, 0, 0)$ m, and its orientation is the 3×3 identity matrix $\mathbf{I}_3$. At $u = 1$, substituting $q_1 = \pi/2$ and $q_2 = -\pi/2$ gives

$${}^s\mathbf{p}_\ell = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix} \text{ m}, \quad {}^s\mathbf{p}_e = \begin{bmatrix} (2+0) 0 \\ (2+0) 1 \\ -(-1) \end{bmatrix} \text{ m} = \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix} \text{ m}.$$

For orientation, the same angles give $\cos q_1 = 0$, $\sin q_1 = 1$, $\cos q_2 = 0$, and $\sin q_2 = -1$. Substituting into the derived rotation matrix yields

$$\mathbf{R}_{se}(\pi/2, -\pi/2) = \begin{bmatrix} 0 \cdot 0 & -1 & 0 \cdot (-1) \\ 1 \cdot 0 & 0 & 1 \cdot (-1) \\ -(-1) & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}.$$

Choosing $q_2 = +\pi/2$ instead would put the end effector at $(0, 2, -1)$ m. In the stated motion, the arm swings sideways and lifts its second link while the end-effector orientation changes. These closed-form values provide an independent reference for pose and display-coordinate comparisons; they do not replace the general product-of-exponentials calculation.

11.4.5 RViz implementation example: displaying the spatial 2R arm

The arm in Section 11.4.4 can be displayed by converting its calculated poses into drawing messages. The numerical core calculates geometry; a separate ROS 2 program publishes the drawing instructions, and RViz displays them. All displayed points are measured from its origin and expressed along its axes in metres. This is a prescribed kinematic animation, not a simulation of forces or a controller.

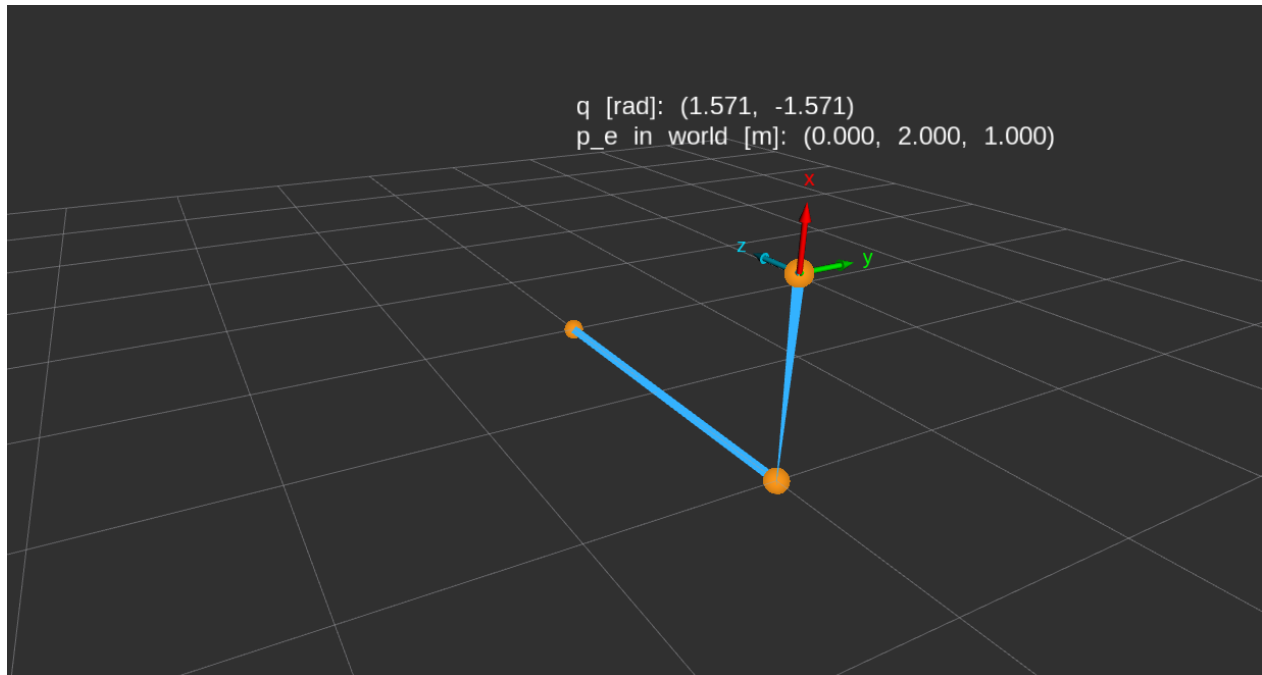


Figure 11.1: Static spatial 2R pose at joint angles $(\pi/2, -\pi/2)$ radians. The elbow is at $(0, 2, 0)$ metres and the end effector at $(0, 2, 1)$ metres in world coordinates.

11.4.5.1 Construct the fixed models

The elbow is carried by joint 1 alone, whereas the end effector is carried by both joints. Their home orientations are identity, with home positions $(2, 0, 0)$ and $(3, 0, 0)$ metres, respectively.

Production excerpt — model construction. The namespace alias, geometry constants, and home matrices are shown. Joint factory calls are summarized at their point of omission; surrounding node setup and logging are omitted.

```
namespace rbk = rigid_body_kinematics;

const double first_link_length_metres = 2.0;
const double second_link_length_metres = 1.0;
// Omitted: construct joint_1 about +z through the origin, and joint_2
// about +y through (2, 0, 0) m at home, both with limits [-pi, pi].

Eigen::Matrix4d elbow_home_matrix = Eigen::Matrix4d::Identity();
elbow_home_matrix(0, 3) = first_link_length_metres;
Eigen::Matrix4d end_home_matrix = Eigen::Matrix4d::Identity();
end_home_matrix(0, 3) =
    first_link_length_metres + second_link_length_metres;

const rbk::SerialChainModel elbow_model =
```

```

rbk::SerialChainModel::from_home_and_joints(
    rbk::Transform3::from_matrix(elbow_home_matrix), {joint_1});
const rbk::SerialChainModel end_model =
    rbk::SerialChainModel::from_home_and_joints(
        rbk::Transform3::from_matrix(end_home_matrix), {joint_1, joint_2});

```

The code specializes the geometric model $\mathcal{M} = (\mathbf{M}, ({}^s\mathbf{S}_j)_{j=1}^n)$ from Section 11.3 to the geometry in Section 11.4.4. Define an auxiliary elbow frame $\{\ell\}$ whose origin is the elbow and whose axes follow link 1, so its orientation is $\mathbf{R}_z(q_1)$. It does not include joint 2's rotation. Let $\mathbf{M}_\ell, \mathbf{M}_e \in SE(3)$ be the home poses of this frame and the end-effector frame relative to $\{s\}$:

$$\mathbf{M}_\ell = \begin{bmatrix} \mathbf{I}_3 & {}^s\mathbf{p}_{\ell,0} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}, \quad \mathbf{M}_e = \begin{bmatrix} \mathbf{I}_3 & {}^s\mathbf{p}_{e,0} \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

Here ${}^s\mathbf{p}_{\ell,0} = (L_1, 0, 0)^\top$ and ${}^s\mathbf{p}_{e,0} = (L_1 + L_2, 0, 0)^\top$ are the home positions in metres from the worked example; $\mathbf{I}_3$ is their identity home orientation. The subscript ℓ selects the elbow and e the end effector. The code-to-mathematics mapping is:

Code value	Mathematical quantity
first_link_length_metres, second_link_length_metres	$L_1 = 2 \text{ m}, L_2 = 1 \text{ m}$
elbow_home_matrix	Matrix representation of $\mathbf{M}_\ell$
end_home_matrix	Matrix representation of $\mathbf{M}_e$, the end-effector home pose $\mathbf{M}$
elbow_model	$\mathcal{M}_\ell = (\mathbf{M}_\ell, ({}^s\mathbf{S}_1))$: one ordered joint
end_model	$\mathcal{M}_e = (\mathbf{M}_e, ({}^s\mathbf{S}_1, {}^s\mathbf{S}_2))$: two ordered joints

The model objects also retain the joints' limits, as explained in Section 11.3. They own fixed reference data and are constructed once; neither stores the current pose. Joint 2's supplied axis remains its home description, not its current world direction after joint 1 rotates.

11.4.5.2 Evaluate one complete drawing update

The dimensionless parameter $u \in [0, 1]$ selects how far the arm has moved along the joint-coordinate path from the straight home configuration to the bent configuration. Using the motion defined in Section 11.4.4, the two-entry joint-coordinate column is

$$\mathbf{q}(u) = \begin{bmatrix} q_1(u) \\ q_2(u) \end{bmatrix} = u \begin{bmatrix} \pi/2 \\ -\pi/2 \end{bmatrix} \text{ rad.}$$

Substituting these angles into the end-effector pose map gives

$$\mathbf{T}_{se}(q_1(u), q_2(u)) = \mathbf{T}_{se}\left(u\frac{\pi}{2}, -u\frac{\pi}{2}\right).$$

The arguments are joint angles in radians. The fixed end-effector model supplies the geometry; evaluating it at these angles returns the complete 4×4 pose, containing both orientation and position. The elbow model uses only the first angle, $q_1 = u\pi/2$ radians. Changing u changes the queries, not either model's stored geometry.

One call receives one value of u and draws one configuration. Animation comes from calling it repeatedly with changing values; u itself is not time and does not specify a speed. The timer supplies the time-dependent choice below. The separate timestamp labels the drawing update but does not enter the pose calculation.

```
Given the fixed models, u, and a timestamp:
  Reject a non-finite u or a value outside [0, 1].
  Form the two joint angles and the elbow's one-angle query.
  Evaluate both poses through the forward-kinematics core.
  Build joint spheres, links, axis arrows, and text from those poses.
  Return the complete marker array.
  If validation or evaluation throws, return no marker array.
```

The function declaration is:

```
visualization_msgs::msg::MarkerArray make_spatial_2r_markers(
  const rigid_body_kinematics::SerialChainModel & elbow_model,
  const rigid_body_kinematics::SerialChainModel & end_model,
  double u,
  const rclcpp::Time & stamp);
```

`make_spatial_2r_markers` borrows both models and the timestamp, and returns an owned `MarkerArray`. A ROS message is a typed data object that can be sent between programs. Here each `Marker` describes a drawable object or group of objects; a `MarkerArray` collects the drawing for one update. The helper constructs messages without creating a node or publishing anything, so a test can call it directly.

Production excerpt — coordinates and poses. The input guard, later marker construction, and final return are omitted; this excerpt assumes finite $u \in [0, 1]$.

```
namespace rbk = rigid_body_kinematics;
```



```

Eigen::VectorXd joint_coordinates(2);
joint_coordinates << u * std::numbers::pi / 2.0,
    -u * std::numbers::pi / 2.0;
Eigen::VectorXd elbow_coordinates(1);
elbow_coordinates(0) = joint_coordinates(0);

const rbk::Transform3 elbow_pose =
    rbk::space_form_forward_kinematics(elbow_model, elbow_coordinates);
const rbk::Transform3 end_pose =
    rbk::space_form_forward_kinematics(end_model, joint_coordinates);

const Eigen::Vector3d elbow_position_metres =
    elbow_pose.transform_point(Eigen::Vector3d::Zero());
const Eigen::Vector3d end_position_metres =
    end_pose.transform_point(Eigen::Vector3d::Zero());

```

Let $\mathbf{T}_{s\ell}(q_1) \in SE(3)$ denote the auxiliary elbow frame's current pose relative to $\{s\}$. The end-effector pose is the previously defined $\mathbf{T}_{se}(q_1, q_2)$. Both map local point coordinates into $\{s\}$ by left multiplication. Applying the space-form product from Section 11.4 to the two models gives the direct mapping:

Code value	Mathematical quantity
<code>joint_coordinates</code>	$\mathbf{q}(u) = u[\pi/2, -\pi/2]^\top$ in radians
<code>elbow_coordinates</code>	The one-entry column $[q_1]$ in radians
<code>elbow_pose</code>	$\mathbf{T}_{s\ell}(q_1) = \exp(\widehat{{}^s\mathbf{S}_1 q_1}) \mathbf{M}_\ell$
<code>end_pose</code>	$\mathbf{T}_{se}(q_1, q_2) = \exp(\widehat{{}^s\mathbf{S}_1 q_1}) \exp(\widehat{{}^s\mathbf{S}_2 q_2}) \mathbf{M}_e$
<code>elbow_position_metres</code>	${}^s\mathbf{p}_\ell(q_1) = \mathbf{R}_z(q_1)[L_1, 0, 0]^\top$
<code>end_position_metres</code>	${}^s\mathbf{p}_e(q_1, q_2)$, the position derived in Section 11.4.4

The hat and exponential are the same linear-first operations used in the forward-kinematics core. Each position is a three-entry column in metres, measured from the world origin and expressed along world axes. The base remains the fixed world origin.

A pose locates an entire frame

At a selected joint configuration, the elbow pose has the block form

$$\mathbf{T}_{s\ell} = \begin{bmatrix} \mathbf{R}_{s\ell} & {}^s\mathbf{p}_\ell \\ \mathbf{0}_3^\top & 1 \end{bmatrix}.$$

- $\mathbf{R}_{s\ell} \in SO(3)$ is the dimensionless 3×3 orientation matrix of the auxiliary elbow frame relative to $\{s\}$.
- ${}^s\mathbf{p}_\ell \in \mathbb{R}^3$ gives that frame's origin position in $\{s\}$, in metres.

For the same physical point, let ${}^\ell\mathbf{x} \in \mathbb{R}^3$ be its coordinates measured from the elbow-frame origin along the elbow-frame axes, and ${}^s\mathbf{x} \in \mathbb{R}^3$ its coordinates measured from the world origin along world axes. Both columns are in metres. The pose converts between them:

$${}^s\mathbf{x} = \mathbf{R}_{s\ell} {}^\ell\mathbf{x} + {}^s\mathbf{p}_\ell.$$

We placed the auxiliary elbow frame's origin at the elbow itself. Therefore, to locate the elbow, choose ${}^\ell\mathbf{x} = \mathbf{0}_3$:

$${}^s\mathbf{x} = \mathbf{R}_{s\ell} \mathbf{0}_3 + {}^s\mathbf{p}_\ell = {}^s\mathbf{p}_\ell.$$

This is exactly the calculation performed by the call from the preceding excerpt:

```
elbow_pose.transform_point(Eigen::Vector3d::Zero());
```

It asks: "Where is the elbow frame's origin in $\{s\}$?" The zero column is a point expressed in the elbow frame, not the world origin.

The end-effector call applies the same reasoning to the origin of $\{e\}$:

```
end_pose.transform_point(Eigen::Vector3d::Zero());
```

Its result is $\mathbf{R}_{se} \mathbf{0}_3 + {}^s\mathbf{p}_e = {}^s\mathbf{p}_e$. The two calls use identical local coordinates but locate different physical points because each pose describes a different frame. The function does not recognize an elbow or a tool; the result is the desired position because we chose that frame's origin at the desired point.

11.4.5.3 Convert positions into spheres and links

This complete helper copies the three coordinates into the ROS representation without changing their units or frame:

```
geometry_msgs::msg::Point to_point_message(
    const Eigen::Vector3d & position_metres)
{
    geometry_msgs::msg::Point point;
    point.x = position_metres.x();
    point.y = position_metres.y();
    point.z = position_metres.z();
}
```

```
    return point;
}
```

`geometry_msgs::msg::Point` stores three coordinates. The surrounding marker supplies their frame name; a point message alone does not identify a coordinate frame.

Production excerpt — joint centres and links. Colour assignments are omitted; production uses opaque orange spheres and cyan links.

```
using Marker = visualization_msgs::msg::Marker;
using MarkerArray = visualization_msgs::msg::MarkerArray;

Marker joints;
joints.header.frame_id = "world";
joints.header.stamp = stamp;
joints.ns = "spatial_2r_fk";
joints.id = 0;
joints.type = Marker::SPHERE_LIST;
joints.action = Marker::ADD;
joints.pose.orientation.w = 1.0;
joints.scale.x = 0.12; // diameter
joints.scale.y = 0.12;
joints.scale.z = 0.12;
// Omitted: orange colour and fully opaque alpha.
joints.lifetime = rclcpp::Duration::from_seconds(0.5);
joints.points = {
    to_point_message(Eigen::Vector3d::Zero()),
    to_point_message(elbow_position_metres),
    to_point_message(end_position_metres),
};

Marker links = joints;
links.id = 1;
links.type = Marker::LINE_STRIP;
links.scale.x = 0.05;
// Omitted: cyan link colour.

MarkerArray markers;
markers.markers = {joints, links};
```

`joints.header.frame_id = "world"` names the reference frame for the marker. In this example, we choose `world` as the ROS name for the fixed mathematical space frame $\{s\}$: its origin is the robot base, and its axes are x_s, y_s, z_s from the worked example. Because the marker pose is identity, its point coordinates are expressed directly in that frame, in metres.

The string "world" is a name we choose, not a special keyword that creates a coordinate frame or transforms coordinates. It tells RViz how to interpret the supplied geometry. Setting RViz's **Fixed Frame** to the same name lets it display these world-frame coordinates without a transform between different frames.

`SPHERE_LIST` draws a sphere of diameter 0.12 m at each point. `LINE_STRIP` joins consecutive points, producing base @ elbow @ end effector; its `scale.x` is the 0.05 m line width. These dimensions describe the illustration, not physical link thickness.

`ns` is a grouping string. Namespace and ID together identify a marker: publishing the same pair updates it instead of accumulating another object. The 0.5-second lifetime lets markers expire if updates stop while the viewer's clock advances. Copies inherit the common timestamp.

The marker pose has zero translation by default and quaternion $(x, y, z, w) = (0, 0, 0, 1)$ after setting `orientation.w`.² This is identity orientation. Point-based markers must retain identity poses because their points already contain world coordinates; another pose would transform them again. Field meanings follow the [ROS 2 Jazzy marker documentation](#).

11.4.5.4 Draw orientation arrows and labels

Let $\mathbf{e}_i \in \mathbb{R}^3$ be the dimensionless Cartesian unit column for local end-effector axis i , where code indices $i = 0, 1, 2$ select x, y, z . Its unit direction in `world` is $\mathbf{R}_{se}\mathbf{e}_i$. For arrow length $\ell_a = 0.3$ m and label gap $d_a = 0.08$ m, define the world-frame tip and label positions, both three-entry columns in metres, by

$$\begin{aligned} {}^s\mathbf{p}_{\text{tip},i} &= {}^s\mathbf{p}_e + \ell_a \mathbf{R}_{se}\mathbf{e}_i, \\ {}^s\mathbf{p}_{\text{label},i} &= {}^s\mathbf{p}_{\text{tip},i} + d_a \mathbf{R}_{se}\mathbf{e}_i. \end{aligned}$$

The gap moves the letter beyond the arrowhead; it does not change the arm geometry.

Production excerpt — axis geometry and text placement. The preceding sphere/link setup is assumed. Arrow dimensions and colour assignments are omitted; production uses red, green, and light blue for the three axes.

```
const Eigen::Matrix3d end_rotation =
    end_pose.matrix().topLeftCorner<3, 3>();
const double axis_length_metres = 0.3;
```

²For a right-handed rotation by angle θ radians about a unit axis $\mathbf{a} = (a_x, a_y, a_z)^T$ expressed in the reference frame, the dimensionless quaternion components in ROS order are $(x, y, z, w) = (a_x \sin(\theta/2), a_y \sin(\theta/2), a_z \sin(\theta/2), \cos(\theta/2))$. These x, y, z are quaternion components, not position coordinates. At $\theta = 0$, $\sin(0) = 0$ and $\cos(0) = 1$, giving $(0, 0, 0, 1)$ regardless of the chosen axis. The marker's quaternion x, y, z fields start at zero; setting `orientation.w = 1.0` therefore gives zero rotation.

```

for (int axis_index = 0; axis_index < 3; ++axis_index) {
    const Eigen::Vector3d tip_position_metres =
        end_position_metres +
        axis_length_metres * end_rotation.col(axis_index);

    Marker axis = joints;
    axis.id = 2 + axis_index;
    axis.type = Marker::ARROW;
    // Omitted: shaft/head dimensions and the axis colour.
    axis.points = {
        to_point_message(end_position_metres),
        to_point_message(tip_position_metres),
    };
    markers.markers.push_back(axis);

    const Eigen::Vector3d axis_label_position_metres =
        tip_position_metres + 0.08 * end_rotation.col(axis_index);
    Marker axis_label = axis;
    axis_label.id = 7 + axis_index;
    axis_label.type = Marker::TEXT_VIEW_FACING;
    axis_label.points.clear();
    axis_label.pose.position =
        to_point_message(axis_label_position_metres);
    axis_label.scale.z = 0.10;
    axis_label.text = "xyz"[axis_index];
    markers.markers.push_back(axis_label);
}

```

`end_rotation` owns the dimensionless 3×3 rotation block. `col(axis_index)` selects an axis direction from the calculated pose; it does not reconstruct orientation using another kinematics implementation. Each arrow replaces the copied sphere list with exactly two points: start and tip.

The text marker copies its arrow's colour, clears the endpoints, and uses `pose.position` for placement. `"xyz"[axis_index]` selects the corresponding letter. `TEXT_VIEW_FACING` keeps the letter facing the viewer as the camera moves, while its position follows the end-effector axis. For text, `scale.z` controls character height rather than displacement along z .

Production excerpt — numerical label contents. The construction and configuration of the text marker are omitted: ID 5, opaque white, cleared points, character height 0.11 m, and placement 0.6 m above the end effector along world $+z$.

```

std::ostringstream pose_text;
pose_text << std::fixed << std::setprecision(3)
    << "q [rad]: (" << joint_coordinates(0) << ", "

```

```

    << joint_coordinates(1) << ")\n"
    << "p_e in world [m]: (" << end_position_metres.x() << ", "
    << end_position_metres.y() << ", "
    << end_position_metres.z() << ")\n";

    // Omitted: construct and configure Marker label as described above.
    label.text = pose_text.str();
    markers.markers.push_back(label);
    return markers;

```

`std::ostringstream`, supplied by `<sstream>`, collects text in memory through `<<` instead of printing it. The `<iomanip>` header supplies `std::setprecision`. With `std::fixed`, precision three means three digits after the decimal point; `str()` returns the assembled string. Only the displayed text is rounded, not the calculated coordinates.

The array contains nine markers: spheres (ID 0), links (ID 1), arrows (IDs 2–4), pose text (ID 5), and axis text (IDs 7–9). Marker identity is not its array index; consumers select by namespace and ID.

11.4.5.5 Publish static poses or a repeating motion

Let $t \geq 0$ be elapsed steady-clock time in seconds, $P = 8\text{ s}$ the cycle period, and $H = P/2 = 4\text{ s}$ its half-period. Define time within the cycle by $\tau = t - P\lfloor t/P \rfloor \in [0, P)$, where $\lfloor \cdot \rfloor$ takes the greatest integer not exceeding its argument. The dimensionless animation input is

$$u(t) = \begin{cases} \tau/H, & 0 \leq \tau < H, \\ (P - \tau)/H, & H \leq \tau < P. \end{cases}$$

The arm moves from home to the bent pose in four seconds and back in four seconds. The reversals change velocity abruptly; this display motion is not a dynamically smooth actuator command.

```

Construct the models and publisher once.
Read the startup animation setting and store a steady-clock start time.
On each requested 50 ms callback:
    Hold u = 1 for a static display.
    Otherwise, derive u from elapsed time within the eight-second cycle.
    Build one complete marker array with the current ROS timestamp.
    Publish only after the helper returns successfully.

```

Production excerpt — node, publisher, and timer. Model construction is the earlier excerpt. The initial diagnostic drawing call, logging, surrounding main/try block, and ex-

ception handler are omitted. Production logs a caught exception and exits with failure; no partial marker array is published.

```
using MarkerArray = visualization_msgs::msg::MarkerArray;
using rigid_body_kinematics_visualization::make_spatial_2r_markers;

rclcpp::init(argc, argv);
const auto node =
    std::make_shared<rclcpp::Node>("spatial_2r_forward_kinematics_demo");
// Omitted here: construct elbow_model and end_model once.

const auto publisher = node->create_publisher<MarkerArray>(
    "~/markers", rclcpp::QoS(1).reliable().durability_volatile());
const bool animate = node->declare_parameter<bool>("animate", false);
const auto start_time = std::chrono::steady_clock::now();

const auto timer = node->create_wall_timer(
    std::chrono::milliseconds(50),
    [node, publisher, elbow_model, end_model, animate, start_time]() {
        double u = 1.0;
        if (animate) {
            const double elapsed_seconds =
                std::chrono::duration<double>(
                    std::chrono::steady_clock::now() - start_time)
                    .count();
            const double cycle_seconds = std::fmod(elapsed_seconds, 8.0);
            u = cycle_seconds < 4.0
                ? cycle_seconds / 4.0
                : (8.0 - cycle_seconds) / 4.0;
        }
        const auto message =
            make_spatial_2r_markers(elbow_model, end_model, u, node->now());
        publisher->publish(message);
    });

rclcpp::spin(node);
rclcpp::shutdown();
```

A ROS node is a named participant in communication. Here `rclcpp::Node` provides the publisher and timer interfaces. `std::make_shared` creates the node with shared ownership; copying its pointer into the callback keeps the same node accessible without copying the node itself. `rclcpp::spin` processes callbacks until shutdown, while the local `timer` variable keeps the timer alive.

The publisher sends `MarkerArray` messages on `~/markers`, resolving to `/spatial_2r_forward_kinematics_d` for this node name. Its second argument sets the message-delivery rules, called QoS

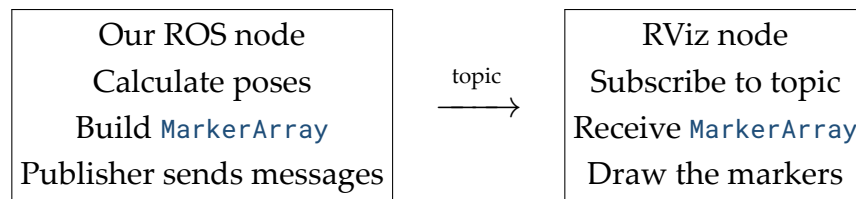
(*quality of service*).³

How the displayed call uses these settings:

- `create_publisher<MarkerArray>` creates a publisher whose message type is `MarkerArray`. Its first argument names the topic; its second argument supplies the QoS configuration.
- `QoS(1)` keeps a history depth of one complete marker-array message. It does not limit the array to one marker: our nine markers are carried together in each message.
- `reliable()` requests reliable delivery of those updates. A delayed update is still possible, so this is not a real-time timing guarantee.
- `durability_volatile()` means RViz does not receive an old drawing merely by subscribing. Our repeated publication supplies a fresh drawing after it connects.

The returned publisher is held through a shared pointer. `auto` deduces that pointer type; `const` prevents reassigning the local pointer, but does not prevent calling `publisher->publish(message)`. Publisher and subscriber QoS settings must also be compatible for messages to flow.

The publisher belongs to our node; it is not a separate node. The topic is the named communication channel, and RViz is a separate consumer that subscribes to it:



Selecting the marker topic in RViz connects its `MarkerArray` display to these messages. Our node computes the geometry; RViz renders it. The publisher does not call RViz directly and can continue publishing when RViz is closed.

How the timer calls our code. `create_wall_timer` creates a repeating timer associated with the node. Its basic call pattern is:

```
// Schematic syntax: period and callback are placeholders.
node->create_wall_timer(period, callback);
```

1. `period` is the requested time between timer callbacks.
2. `callback` is the function ROS should run when the timer is ready. Creating the timer registers that function; it does not execute its body immediately.

³`QoS(n)` keeps the latest `n` messages in history. `reliable()` requests acknowledgements and retries for missing messages, but delivery can still be delayed. `durability_volatile()` does not supply past messages to a newly connected subscriber. Each method returns the same settings object, allowing the calls to be joined with dots; they configure the publisher rather than send data.

3. The result is a shared pointer to the timer. Keeping it in `timer` keeps the timer available while `rclcpp::spin(node)` processes events. The call does not create a separate thread dedicated to running this callback.

In the displayed implementation:

- `std::chrono::milliseconds(50)` requests one callback every 50 ms, or nominally 20 times per second.
- The lambda beginning with `[node, publisher, ...]()` is the callback. Each invocation selects u , calculates the current marker array, and publishes it.
- `rclcpp::spin(node)` runs ready callbacks, including this timer's callback. If the program is busy, execution can be delayed; the period is not an exact timing guarantee.

The timer decides when to request an update; the elapsed-time calculation inside the callback decides which pose to draw.

The timing operations have distinct jobs:

1. `std::chrono::steady_clock::now()` reads a monotonic clock: system-clock corrections do not move its elapsed-time reading backwards.
2. Subtracting `start_time` gives an elapsed duration; `std::chrono::duration<double>(...).count()` expresses it as seconds.
3. `std::fmod(elapsed_seconds, 8.0)`, from `<cmath>`, gives the **floating-point remainder** after dividing elapsed seconds by eight. For nonnegative elapsed time, this implements τ , the time within the current cycle.
4. `node->now()` supplies a ROS timestamp for the drawing; it does not determine the animation phase.

The callback captures owned copies of both models, the startup setting, and the start time. It creates a fresh local message on each call; no mutable lambda is required. `animate` is read once, so later parameter changes do not change the captured setting. The timer requests 20 updates per second, not a real-time guarantee. A delayed callback skips to the pose corresponding to elapsed time rather than stretching the cycle by counting callbacks.

11.4.5.6 Run and inspect the example

Usage commands — a configured ROS workspace terminal. After building the visualization package, source its installed environment and start the static display:

```
source install/setup.zsh
ros2 run rigid_body_kinematics_visualization \
  spatial_2r_forward_kinematics_demo
```

To animate, stop that process and run:

```
ros2 run rigid_body_kinematics_visualization \  
  spatial_2r_forward_kinematics_demo \  
  --ros-args -p animate:=true -p use_sim_time:=false
```

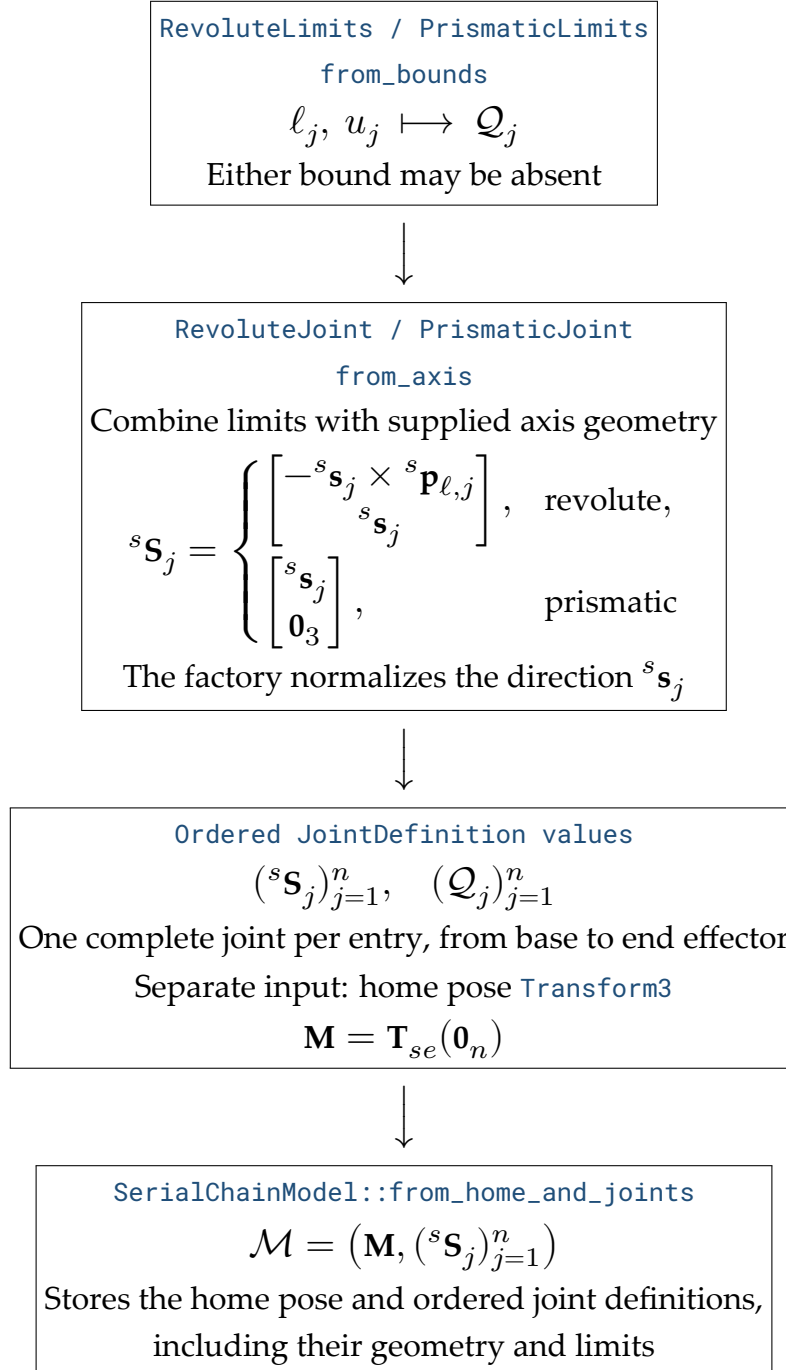
In another sourced terminal, run `rviz2`. Set **Global Options -> Fixed Frame** to `world`, add a **MarkerArray** display, and select `/spatial_2r_forward_kinematics_demo/markers`.

The static image in Figure 11.1 also serves as the PDF illustration; the animation is included in HTML. Visual inspection helps explain motion and orientation, but it does not replace numerical acceptance tests.

11.4.6 How the classes and functions connect

The diagrams pair code names with their mathematical meaning. Arrows show construction or calculation steps, not the order of physical robot motion. Only the relevant contents and checks are shown.

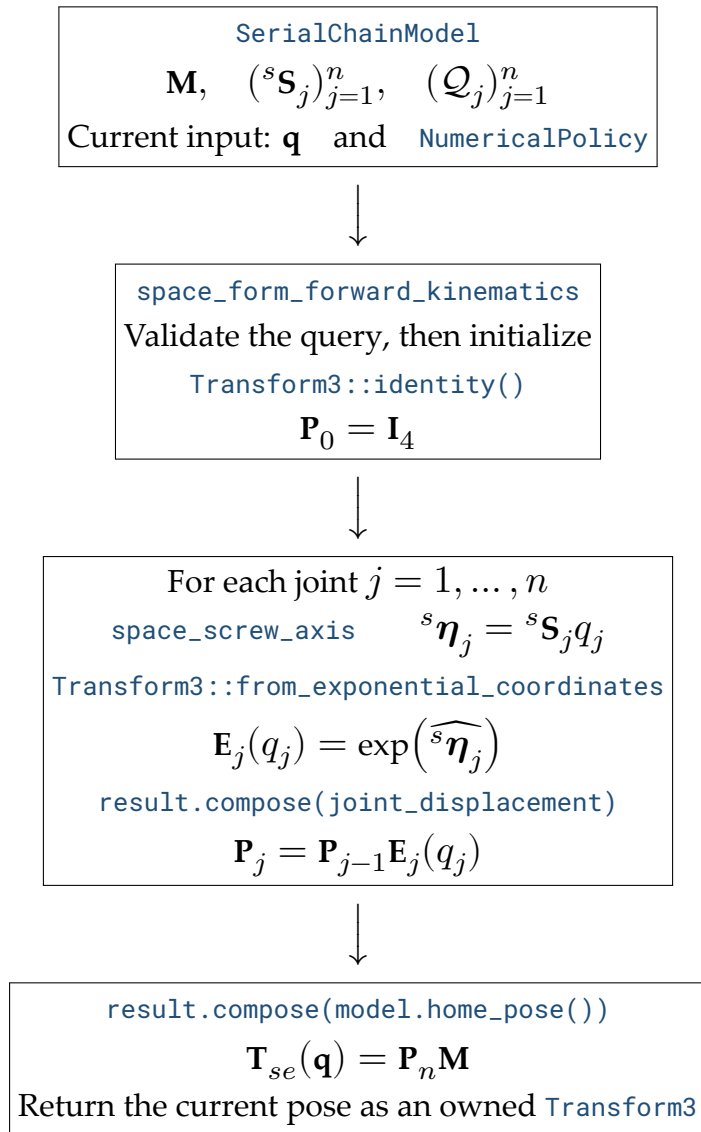
11.4.6.1 Construct the fixed model



The model owns its home pose and joint definitions. The axis geometry and the separately supplied home pose describe the same zero arrangement in $\{s\}$.

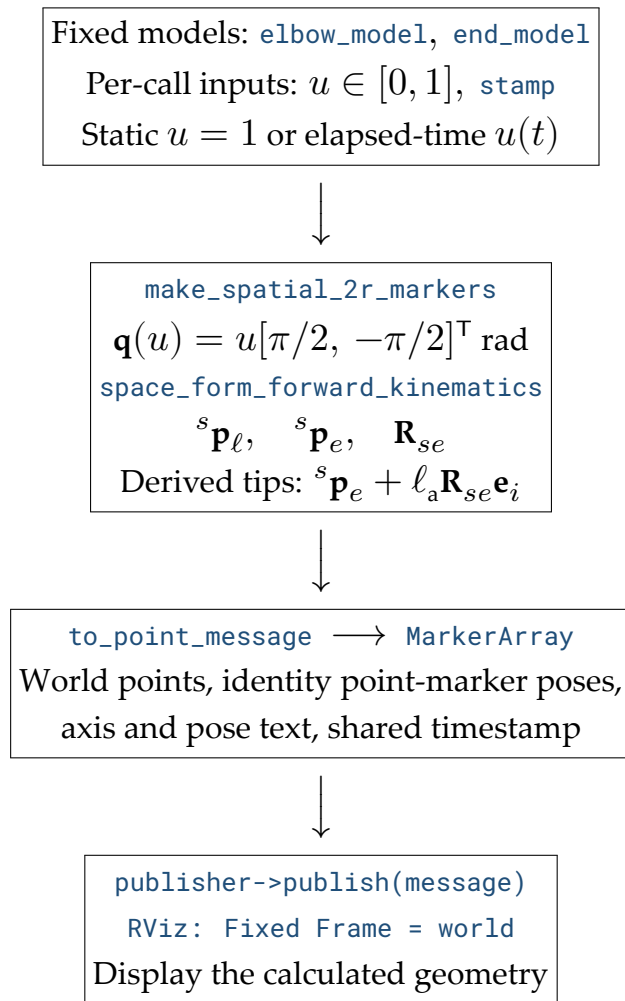
11.4.6.2 Evaluate a current pose

The joint-loop box is repeated in joint order; validation and failure handling are summarized rather than drawn as separate branches.



Evaluation reads the model and coordinates without changing them and returns an owned pose. A failed query throws instead of returning a partial result.

11.4.6.3 Connect the pose calculation to the viewer



The marker helper borrows the models and owns the returned drawing data. ROS carries the published messages to RViz, which renders the supplied coordinates rather than recalculating the robot's kinematics.

Part II

Probability and Estimation

12 Finite Markov Chains

Prerequisites

This chapter assumes familiarity with finite sets, conditional probability, the product rule, the law of total probability, finite sums, and matrix multiplication.

Why model an uncontrolled state sequence?

An intelligence system often needs to answer a prediction question before it chooses an action: if the current situation is known only probabilistically, what situations may occur next, and with what probabilities? A **finite Markov chain** answers this question for an uncontrolled process whose state takes one of finitely many values at each discrete time step.

The known inputs are a finite set of possible states, an initial probability distribution over those states, and one-step transition probabilities. The required outputs are the distribution at the next time step, the probability of a declared state trajectory, and the recognition of states that cannot be left.

The dependency chain for this chapter is

random state sequence $\longrightarrow$ Markov property
 $\longrightarrow$ transition matrix $\longrightarrow$ distribution and trajectory predictions.

The first two objects say what information the model retains. The matrix records the resulting one-step law. Matrix multiplication and the probability product rule then produce predictions over multiple possible paths.

12.1 A finite stochastic state sequence

Consider a process observed at discrete times $t \in \mathbb{N}_0 = \{0, 1, 2, \dots\}$. At time t , the process occupies exactly one state from the finite **state set**

$$\mathcal{S} := \{s_1, s_2, \dots, s_n\},$$

where $n \in \mathbb{N}$ is the number of states and s_i is the state with index $i \in \{1, \dots, n\}$. A state is a modelled summary of the current situation. The modeller chooses what information that summary contains; the Markov property below tests whether the choice is sufficient for the intended prediction.

Let Ω be the sample space whose elements $\omega \in \Omega$ represent complete possible realisations of the process. At each time t , define the mapping

$$S_t : \Omega \rightarrow \mathcal{S}, \quad \omega \mapsto S_t(\omega).$$

Assume that, for every state s_i , the set of realisations mapped to s_i is an event with a defined probability. Under this assumption, S_t is a **discrete random variable** whose possible values belong to the finite state set $\mathcal{S}$. In this model, S_t is called the **random state** at time t : *random variable* names its mathematical type, while *random state* names its role. Before a realisation is observed, S_t may equal any member of $\mathcal{S}$ with a declared probability. The notation $S_t = s_i$ abbreviates the event that the realised state at time t is s_i . The ordered collection

$$(S_0, S_1, S_2, \dots)$$

is a **stochastic state sequence**: a time-ordered sequence of random states. Upper-case S_t denotes the random quantity, while a lower-case symbol denotes one possible state value. To avoid confusing a time position with the fixed enumeration $(s_1, \dots, s_n)$, write a realised finite history through time t as

$$(S_0 = x_0, S_1 = x_1, \dots, S_t = x_t),$$

where each $x_k \in \mathcal{S}$ is the state value realised at time k .

For every state value $s \in \mathcal{S}$, define its probability at time t by

$$p_t : \mathcal{S} \rightarrow [0, 1], \quad s \mapsto p_t(s) := \Pr(S_t = s).$$

Because the states are mutually exclusive and exhaustive, exactly one state occurs at time t . Therefore

$$\sum_{s \in \mathcal{S}} p_t(s) = 1.$$

Indices enter when these probabilities are stored using the declared state order. Define the **state-distribution row vector**

$$\mathbf{p}_t := [p_t(s_1) \ p_t(s_2) \ \cdots \ p_t(s_n)] \in [0, 1]^{1 \times n}, \quad [\mathbf{p}_t]_i = p_t(s_i).$$

The function p_t and vector $\mathbf{p}_t$ describe uncertainty about the current state; neither is itself a state from $\mathcal{S}$. If the current state is known to be s_k , then $p_t(s_k) = 1$ and every other state has probability zero.

12.2 The first-order Markov property

A general stochastic sequence may need its entire history to predict the next state. A first-order Markov model makes a more specific claim: once the current state is known, earlier states provide no additional information about the next-state distribution.

Formally, the sequence $(S_0, S_1, \dots)$ has the **first-order Markov property** when, for every time $t \in \mathbb{N}_0$, realised history $x_0, \dots, x_t \in \mathcal{S}$, and possible next state $s' \in \mathcal{S}$ satisfying

$$\Pr(S_0 = x_0, S_1 = x_1, \dots, S_t = x_t) > 0,$$

the following equality holds:

$$\Pr(S_{t+1} = s' \mid S_t = x_t, S_{t-1} = x_{t-1}, \dots, S_0 = x_0) = \Pr(S_{t+1} = s' \mid S_t = x_t).$$

The displayed positive-probability condition ensures that the conditional probabilities are defined. The left-hand side uses the complete observed history. The right-hand side uses only the current state. Their equality states that the current state retains all history relevant to this one-step prediction.

The word *first-order* refers to dependence on one immediately preceding state. It does not mean that the physical process has no memory. Instead, it means that the modelled state already contains the memory needed for prediction. For example, a position alone is generally not a Markov state for inertial motion because the next position also depends on velocity. The pair consisting of position and velocity can be Markov under a suitable motion model because the missing predictive information has been added to the state.

The Markov property belongs to the chosen state representation and transition model, not to a name assigned to the physical system. If two histories produce the same state label but

imply different next-state distributions, that label is not a Markov state for the model. One remedy is to enlarge the state so that the histories with different predictive consequences no longer collapse into the same state.

12.3 Time-homogeneous transition probabilities

The Markov property removes unnecessary dependence on earlier states. A second assumption removes explicit dependence on time. A Markov chain is **time-homogeneous** when the probability of moving from any current state $s \in \mathcal{S}$ to any next state $s' \in \mathcal{S}$ is the same at every time step. Define the one-step **transition kernel**

$$p(s' \mid s) := \Pr(S_{t+1} = s' \mid S_t = s).$$

The notation $\Pr(\cdot)$ denotes the probability of an event, lowercase p denotes the probability mass assigned to particular state values, and bold $\mathbf{P}$ denotes the matrix representation. The argument s is the supplied current state, while s' is the possible next state. Time homogeneity means that $p(s' \mid s)$ does not depend on t .

For each current state s , the function $p(\cdot \mid s) : \mathcal{S} \rightarrow [0, 1]$ is a probability mass function. Therefore

$$\begin{aligned} p(s' \mid s) &\geq 0 \quad \text{for every } s' \in \mathcal{S}, \\ \sum_{s' \in \mathcal{S}} p(s' \mid s) &= 1. \end{aligned}$$

Indices enter only when the kernel is stored using the declared state order. Define the **transition matrix** by

$$[\mathbf{P}]_{ij} := p(s_j \mid s_i), \quad \mathbf{P} \in [0, 1]^{n \times n}.$$

Row i contains the complete next-state distribution $p(\cdot \mid s_i)$. Thus every entry is non-negative and every row sums to one. A matrix satisfying these two conditions is **row-stochastic**. Row stochasticity is not a numerical convenience; it expresses that, from each current state, the process must enter exactly one of the declared next states. A negative entry cannot be a probability, and a row sum different from one leaves probability missing or assigns probability more than once.

This book deliberately stores state distributions as row vectors and current states along matrix rows. Under this convention, distributions propagate by right multiplication. A column-vector convention would instead use a transposed matrix or reverse the multiplication order. Either convention can be correct, but mixing them changes the model.

12.4 Propagating the state distribution

Suppose the state distribution p_t and transition kernel $p(s' \mid s)$ are known. To find the probability of a possible next state $s' \in \mathcal{S}$, partition the possibilities according to the current state. The finite law of total probability gives

$$p_{t+1}(s') = \sum_{s \in \mathcal{S}} p_t(s) p(s' \mid s).$$

This equation is semantic: it sums over possible current-state values rather than matrix indices. When the states are stored in their declared order, the same calculation for next state s_j becomes

$$[\mathbf{p}_{t+1}]_j = \sum_{i=1}^n [\mathbf{p}_t]_i [\mathbf{P}]_{ij}.$$

This is component j of a row-vector-matrix product. Therefore the complete one-step distribution update is

$$\boxed{\mathbf{p}_{t+1} = \mathbf{p}_t \mathbf{P}.$$

The update preserves the defining properties of a probability distribution. Each term $p_t(s)p(s' \mid s)$ is non-negative, so every value $p_{t+1}(s')$ is non-negative. Summing over all next states gives

$$\begin{aligned} \sum_{s' \in \mathcal{S}} p_{t+1}(s') &= \sum_{s' \in \mathcal{S}} \sum_{s \in \mathcal{S}} p_t(s) p(s' \mid s) \\ &= \sum_{s \in \mathcal{S}} p_t(s) \left(\sum_{s' \in \mathcal{S}} p(s' \mid s) \right) \\ &= \sum_{s \in \mathcal{S}} p_t(s) \\ &= 1. \end{aligned}$$

The reduction to $\sum_{s \in \mathcal{S}} p_t(s)$ uses the normalization of every conditional distribution $p(\cdot \mid s)$. Thus a valid distribution and a valid transition kernel produce another valid distribution.

Applying the same update repeatedly gives a prediction $k \in \mathbb{N}$ steps into the future:

$$\boxed{\mathbf{p}_{t+k} = \mathbf{p}_t \mathbf{P}^k,$$

where $\mathbf{P}^k$ is the product of k copies of $\mathbf{P}$. The formula follows by repeated substitution: $\mathbf{p}_{t+2} = (\mathbf{p}_t \mathbf{P}) \mathbf{P} = \mathbf{p}_t \mathbf{P}^2$, and the same argument continues for later steps.

12.5 Trajectory probabilities and absorbing states

A distribution prediction sums over all paths that can reach a future state. Sometimes the required output is instead the probability of one declared path. Let $T \in \mathbb{N}$ be a finite final time, and choose realised state values $x_0, x_1, \dots, x_T \in \mathcal{S}$. The corresponding trajectory event is

$$S_0 = x_0, S_1 = x_1, \dots, S_T = x_T.$$

The probability product rule first factors the joint probability into an initial probability and successive conditional probabilities. The Markov property then removes all history before the immediately preceding state, and time homogeneity replaces each remaining conditional probability by the transition kernel:

$$\begin{aligned} & \Pr(S_0 = x_0, S_1 = x_1, \dots, S_T = x_T) \\ &= \Pr(S_0 = x_0) \prod_{t=0}^{T-1} \Pr(S_{t+1} = x_{t+1} \mid S_t = x_t, \dots, S_0 = x_0) \\ &= \Pr(S_0 = x_0) \prod_{t=0}^{T-1} \Pr(S_{t+1} = x_{t+1} \mid S_t = x_t) \\ &= \boxed{p_0(x_0) \prod_{t=0}^{T-1} p(x_{t+1} \mid x_t).} \end{aligned}$$

The trajectory probability multiplies probabilities along one path. In contrast, the distribution $\mathbf{p}_T = \mathbf{p}_0 \mathbf{P}^T$ adds the probabilities of every path ending in each state at time T .

A state $s \in \mathcal{S}$ is an **absorbing state** when

$$\boxed{p(s \mid s) = 1.}$$

Because $p(\cdot \mid s)$ must sum to one and every probability is non-negative, this condition forces $p(s' \mid s) = 0$ for every $s' \neq s$. Once the chain enters s , it remains there at every later time with probability one. If $s = s_k$ in the declared state order, the matrix form of the absorbing condition is $[\mathbf{P}]_{kk} = 1$.

Example: A three-state reasoning process

This example demonstrates the state, transition, propagation, trajectory, and absorption definitions in one finite chain. Let an engine be in exactly one mode at each step:

$$\mathcal{S} = \{s_1, s_2, s_3\} = \{\text{searching, clue held, answered}\}.$$

Assume the declared mode contains all information needed for the next-mode prediction, and let

$$\mathbf{P} = \begin{bmatrix} 0.5 & 0.5 & 0 \\ 0.1 & 0.4 & 0.5 \\ 0 & 0 & 1 \end{bmatrix}.$$

For example, $p(s_3 | s_2) = 0.5$ means that the next mode is answered with probability 0.5 when the current mode is clue held. In the declared state order, this probability is stored as $[\mathbf{P}]_{23} = 0.5$. Every row is non-negative and sums to one, so $\mathbf{P}$ is row-stochastic. State s_3 is absorbing because $p(s_3 | s_3) = 1$, equivalently $[\mathbf{P}]_{33} = 1$. Suppose the engine begins in searching with certainty:

$$\mathbf{p}_0 = [1 \ 0 \ 0].$$

One-step propagation gives

$$\mathbf{p}_1 = \mathbf{p}_0 \mathbf{P} = [0.5 \ 0.5 \ 0],$$

and a second update gives

$$\mathbf{p}_2 = \mathbf{p}_1 \mathbf{P} = [0.30 \ 0.45 \ 0.25].$$

The particular two-step trajectory searching $\rightarrow$ clue held $\rightarrow$ answered has probability

$$p_0(s_1)p(s_2 | s_1)p(s_3 | s_2) = 1 \times 0.5 \times 0.5 = 0.25.$$

Here the value 0.25 also equals the total probability of being answered at time 2 because no other two-step path from searching reaches answered. At later times, several different paths may reach answered, so its marginal probability must add all such path probabilities rather than select only one.

Quick test: finite Markov chains

1. **Recall:** What equality defines the first-order Markov property, and why must the conditioning history have positive probability?
2. **Explanation:** Why is the statement “the future ignores the past” less accurate than “the present retains the past information relevant to predicting the next state”?

3. **Representation:** What does $p(s' \mid s)$ mean, and how is it related to matrix entry $[\mathbf{P}]_{ij}$? State the two row-stochastic conditions.
4. **Derivation:** Starting from the law of total probability, derive $p_{t+1}(s') = \sum_{s \in \mathcal{S}} p_t(s)p(s' \mid s)$, then obtain $\mathbf{p}_{t+1} = \mathbf{p}_t \mathbf{P}$.
5. **Application:** For the example chain, calculate the probability of searching $\rightarrow$ searching $\rightarrow$ clue held. Is this probability the same object as $\Pr(S_2 = \text{clue held})$?
6. **Model diagnosis:** Suppose the next-mode probability from a state labelled working differs depending on whether the previous hidden mode was searching or clue held. Does working satisfy the Markov property? State one repair.

Answers and hints

1. Let $H_t := \{S_0 = x_0, \dots, S_t = x_t\}$. When $\Pr(H_t) > 0$, the Markov property requires $\Pr(S_{t+1} = s' \mid H_t) = \Pr(S_{t+1} = s' \mid S_t = x_t)$ for every $s' \in \mathcal{S}$. If $\Pr(H_t) = 0$, then $\Pr(\{S_{t+1} = s'\} \cap H_t) = 0$, so the defining ratio $\Pr(\{S_{t+1} = s'\} \cap H_t) / \Pr(H_t) = 0/0$ is undefined.
2. Earlier events may have affected the current state. The Markov claim is that, after the current state is known, those earlier events add no further one-step predictive information.
3. The value $p(s' \mid s)$ is the probability of next state s' conditional on current state s . Once the states are placed in their declared order, $[\mathbf{P}]_{ij} = p(s_j \mid s_i)$. Every entry is non-negative and every row sums to one.
4. For each $s' \in \mathcal{S}$, the law of total probability gives $p_{t+1}(s') = \sum_{s \in \mathcal{S}} p_t(s)p(s' \mid s)$. Setting $s' = s_j$ and replacing semantic probabilities by their stored entries gives $[\mathbf{p}_{t+1}]_j = \sum_i [\mathbf{p}_t]_i [\mathbf{P}]_{ij}$, which is component j of $\mathbf{p}_t \mathbf{P}$.
5. The path probability is $p_0(s_1)p(s_1 \mid s_1)p(s_2 \mid s_1) = 1 \times 0.5 \times 0.5 = 0.25$. It is only one contribution to $\Pr(S_2 = \text{clue held}) = 0.45$; the other contribution comes from searching $\rightarrow$ clue held $\rightarrow$ clue held.
6. No. The current label working loses history that changes the next-state distribution. Split working into predictive substates or augment it with the missing information.

References

- Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*, second-edition draft, Chapter 3, especially Sections 3.5 and 3.6 on the Markov property and finite Markov decision processes.
- Dimitri P. Bertsekas, *Reinforcement Learning and Optimal Control*, second edition, Chapter 1, especially Sections 1.3 and 1.4 on stochastic state evolution, policies, and time-homogeneous infinite-horizon models.

- Sebastian Thrun, Wolfram Burgard, and Dieter Fox, *Probabilistic Robotics*, Chapter 2, especially Section 2.2 on conditional probability, the probability product rule, and the law of total probability.

Part III

Optimisation, Planning, and Decisions

13 Finite Markov Decision Processes

Prerequisites

This chapter extends the finite Markov-chain model from Chapter 12. That chapter defined the finite state set $\mathcal{S} = \{s_1, \dots, s_n\}$, the random state S_t , the row state-distribution vector $\mathbf{p}_t$, and the row-stochastic transition matrix $\mathbf{P}$.

Why add control to stochastic dynamics?

An uncontrolled Markov chain assigns one next-state distribution to each current state. A decision problem asks a different question: if an agent can choose an action, how does that choice change the distribution of the next state? The mathematical model must therefore distinguish uncertainty in the environment's response from choice by the agent.

The known inputs are the current state, the actions available in that state, and one next-state distribution for each feasible state–action pair. The required output is the probability of every possible next state after a particular action is applied. The dependency chain for this block is

current state $\longrightarrow$ feasible action
 $\longrightarrow$ controlled next-state distribution
 $\longrightarrow$ action-conditioned transition matrix.

The action identifies which transition law applies. The probabilities within that law describe the environment's stochastic response after the choice has been made.

13.1 Actions and feasible action sets

Let the finite **action set** be

$$\mathcal{A} := \{a_1, a_2, \dots, a_m\},$$

where $m \in \mathbb{N}$ is the number of distinct action labels and a_ℓ is the action with index $\ell \in \{1, \dots, m\}$. An action is a control input that the agent can apply to influence the next-state distribution. The set $\mathcal{A}$ is the global vocabulary of action labels; it does not imply that every action can be applied in every state.

For each state $s \in \mathcal{S}$, define the nonempty **feasible action set**

$$\mathcal{A}(s) \subseteq \mathcal{A}, \quad \mathcal{A}(s) \neq \emptyset.$$

The set $\mathcal{A}(s)$ contains exactly the actions admitted when the current state is s . The state-dependent set is needed because a physically or logically meaningful action in one state may be unavailable in another. For example, an action called open may be feasible for a closed container but unavailable for a container that is already open.

Assume that every global action label is feasible in at least one state, so $\mathcal{A} = \bigcup_{s \in \mathcal{S}} \mathcal{A}(s)$. This convention prevents the global set from containing an action that can never be used.

On the sample space Ω introduced in Section 12.1, represent the action at time t by the discrete random variable

$$A_t : \Omega \rightarrow \mathcal{A}.$$

For any $a \in \mathcal{A}$, the event $A_t = a$ says that action a is realised at time t . The random-variable representation allows actions to differ across possible trajectories. The variable records the selected action, while the action-selection mechanism is a separate mathematical object. Once a particular trajectory is considered, the realised value a is the control input supplied to the environment.

A state-action pair (s, a) is **feasible** when $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$. An unavailable pair with $a \notin \mathcal{A}(s)$ is outside the controlled model's domain. Its transition probabilities are therefore undefined; they must not be invented as a row of zeros, because a zero row is not a probability distribution.

13.2 The controlled Markov property

Before action A_t is applied, define $H_0 := (S_0)$. For $t \in \mathbb{N}$, the observed interaction history through state S_t is the tuple

$$H_t := (S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}, S_t).$$

A possible realised value h_t of H_t ends in some current state $s \in \mathcal{S}$. The **controlled first-order Markov property** states that, after the current state and current action are known,

earlier states and actions provide no additional information about the next-state distribution. For every next state $s' \in \mathcal{S}$, every feasible action $a \in \mathcal{A}(s)$, and every joint conditioning event with $\Pr(H_t = h_t, A_t = a) > 0$,

$$\Pr(S_{t+1} = s' \mid H_t = h_t, A_t = a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a).$$

The left-hand side conditions on the complete state–action history and the current action. The right-hand side retains only the current state and current action. Their equality means that (S_t, A_t) contains the information needed for the one-step prediction. As with an uncontrolled Markov chain, this is a modelling claim about the chosen state representation. If two histories ending in the same state–action pair imply different next-state distributions, the state representation has discarded relevant information.

13.3 Controlled transition probabilities

Time homogeneity answers whether the same feasible state–action pair obeys the same one-step law at different times. At each time $t \in \mathbb{N}_0$, let $p_t(s' \mid s, a)$ denote the next-state probability that the controlled model assigns to $s' \in \mathcal{S}$ after feasible pair (s, a) . Whenever $\Pr(S_t = s, A_t = a) > 0$, this model probability agrees with the elementary conditional probability

$$p_t(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a).$$

The controlled dynamics are **time-homogeneous** when, for every $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, $s' \in \mathcal{S}$, and $t, u \in \mathbb{N}_0$,

$$p_t(s' \mid s, a) = p_u(s' \mid s, a).$$

The equality says that changing the time index while holding the state–action pair fixed does not change the next-state distribution. All times therefore share one law. Define its common value as the **controlled transition probability**

$$p(s' \mid s, a) := p_t(s' \mid s, a).$$

The notation $\Pr(\cdot)$ denotes the probability of an event, lowercase p denotes the probability mass assigned to particular values, and bold $\mathbf{P}$ denotes a matrix representation. The arguments show the model directly: s is the current state, a is the applied action, and s' is the possible next state. The vertical bar records that the next-state probability is conditional on the supplied current state and action. For a fixed feasible pair (s, a) , the function

$$p(\cdot \mid s, a) : \mathcal{S} \rightarrow [0, 1], \quad s' \mapsto p(s' \mid s, a),$$

is the probability mass function of the next state.

For a fixed feasible pair (s, a) , the events $S_{t+1} = s'$ over all $s' \in \mathcal{S}$ are mutually exclusive and exhaustive. Therefore

$$\begin{aligned} p(s' \mid s, a) &\geq 0 \quad \text{for every } s' \in \mathcal{S}, \\ \sum_{s' \in \mathcal{S}} p(s' \mid s, a) &= 1. \end{aligned}$$

Indices enter only when these probabilities are stored in a matrix. If one action $a \in \mathcal{A}$ is feasible in every state, define its **action-conditioned transition matrix** by

$$\boxed{[\mathbf{P}^{(a)}]_{ij} := p(s_j \mid s_i, a), \quad \mathbf{P}^{(a)} \in [0, 1]^{n \times n}.}$$

The indices i and j now have a strictly representational role: they select a row and column after the semantic states s_i and s_j have been placed in the declared global state order. Each $\mathbf{P}^{(a)}$ is row-stochastic. If action a is deliberately applied regardless of the current state, the row-vector convention from Section 12.3 gives

$$\mathbf{p}_{t+1} = \mathbf{p}_t \mathbf{P}^{(a)}.$$

If a is unavailable in any state, $\mathbf{P}^{(a)}$ is not a complete transition matrix because the corresponding infeasible row is outside the model's domain. The kernel $p(s' \mid s, a)$ on feasible triples (s, a, s') remains the general representation.

13.4 Uncontrolled and controlled dynamics are different objects

An uncontrolled Markov chain contains one transition matrix $\mathbf{P}$. Once the current state distribution is known, that matrix alone determines the next-state distribution. A controlled transition model instead contains one probability row for every feasible state-action pair. The current state alone does not determine which row will govern the next transition because the action must also be supplied.

The controlled model is not one uncontrolled Markov chain with a single transition matrix. Choosing one fixed action a produces the special matrix $\mathbf{P}^{(a)}$ when that action is feasible in every state. A state-dependent action-selection rule instead combines the controlled rows according to the actions selected in each state. Keeping these objects distinct prevents the environment's stochastic response from being confused with the agent's method for choosing actions.

Example: One state with two possible transition laws

This example demonstrates how the chosen action changes the next-state distribution. Reuse the three states from the Markov-chain example:

$$\mathcal{S} = \{s_1, s_2, s_3\} = \{\text{searching, clue held, answered}\}.$$

Let a_1 mean continue and a_2 mean reset, and suppose both actions are feasible in every state. Their action-conditioned transition matrices are

$$\mathbf{P}^{(a_1)} = \begin{bmatrix} 0.5 & 0.5 & 0 \\ 0.1 & 0.4 & 0.5 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{P}^{(a_2)} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

If the current state is clue held, then applying continue selects row 2 of $\mathbf{P}^{(a_1)}$:

$$[p(s_1 | s_2, a_1) \quad p(s_2 | s_2, a_1) \quad p(s_3 | s_2, a_1)] = [0.1 \quad 0.4 \quad 0.5].$$

Thus continuing reaches answered with probability $p(s_3 | s_2, a_1) = 0.5$. Applying reset instead selects row 2 of $\mathbf{P}^{(a_2)}$, so the next state is searching with probability $p(s_1 | s_2, a_2) = 1$. The current state is identical in both cases; the different action selects a different stochastic law.

For an uncertain current-state distribution $\mathbf{p}_t = [0.2 \ 0.5 \ 0.3]$, applying continue in every state gives

$$\mathbf{p}_{t+1} = \mathbf{p}_t \mathbf{P}^{(a_1)} = [0.15 \ 0.30 \ 0.55].$$

The resulting distribution is a valid row distribution because $\mathbf{P}^{(a_1)}$ is row-stochastic.

Quick test: actions and controlled transitions

1. **Recall:** What is the difference between the global action set $\mathcal{A}$ and the feasible action set $\mathcal{A}(s)$?
2. **Explanation:** Why can A_t be represented as a random variable even though an action is a control choice?
3. **Derivation:** Starting from the definition of $p(s' | s, a)$, show why $\sum_{s' \in \mathcal{S}} p(s' | s, a) = 1$ for every feasible state–action pair.
4. **Representation:** Explain the relationship between the semantic probability $p(s_j | s_i, a)$ and matrix entry $[\mathbf{P}^{(a)}]_{ij}$. When is $\mathbf{P}^{(a)}$ a complete transition matrix?
5. **Application:** In the example, what is the probability of moving from clue held to searching under continue? What is the same probability under reset?
6. **Calculation:** Verify the answered-state entry 0.55 in $\mathbf{p}_t \mathbf{P}^{(a_1)}$.
7. **Validity:** Why must an infeasible state–action pair not be represented by a zero transition row?

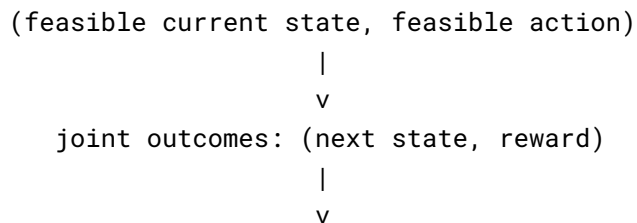
8. **Model distinction:** Why is a controlled transition kernel different from an uncontrolled Markov chain with one transition matrix?

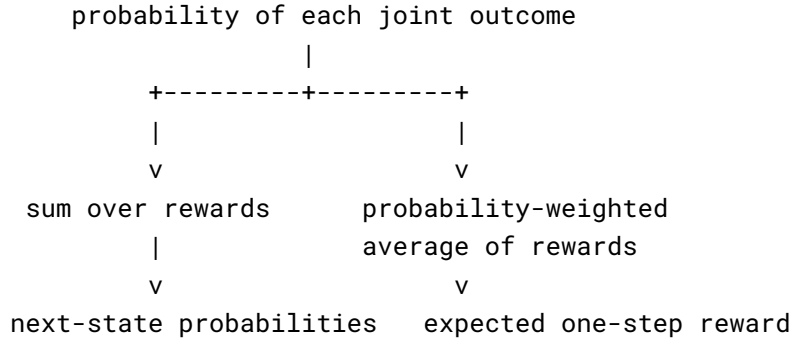
Answers and hints

1. $\mathcal{A}$ contains every action label in the model. $\mathcal{A}(s) \subseteq \mathcal{A}$ contains only the actions admitted when the current state is s .
2. Across possible trajectories, different actions may be realised, so $A_t : \Omega \rightarrow \mathcal{A}$ records which action occurs. The realised value a is still the control input, while a separate action-selection mechanism determines which value is chosen.
3. The possible next-state events partition the next-state sample outcomes. Hence $\sum_{s' \in \mathcal{S}} p(s' \mid s, a) = \Pr(S_{t+1} \in \mathcal{S} \mid S_t = s, A_t = a) = 1$.
4. The kernel value $p(s_j \mid s_i, a)$ describes the transition from semantic state s_i to semantic state s_j under action a . Once the states are placed in their declared order, the same number is stored at row i , column j of $\mathbf{P}^{(a)}$. The matrix is complete only when a is feasible in every state.
5. Under continue, the probability is $p(s_1 \mid s_2, a_1) = 0.1$. Under reset, it is $p(s_1 \mid s_2, a_2) = 1$.
6. The answered-state entry is $0.2(0) + 0.5(0.5) + 0.3(1) = 0.55$.
7. A zero row sums to zero, not one, so it is not a next-state distribution. More importantly, the transition law is undefined outside its feasible domain; inserting zeros would turn unavailability into false numerical data.
8. The kernel supplies several possible next-state distributions for each state, one per feasible action. The current state alone does not select a unique next-state distribution; the action or an action-selection rule is an additional input.

13.5 Rewards and joint one-step dynamics

The controlled transition kernel describes where the process may move after an action, but different transitions can have different consequences. A **reward** is a real-valued scalar produced by the environment to evaluate one step of interaction. The known inputs for the one-step reward model are a feasible current state–action pair and the possible joint outcomes consisting of a next state and a reward. The required outputs are the probability of each joint outcome, the next-state probabilities, and the expected one-step reward.





Let the finite **reward set** be

$$\mathcal{R} := \{r_1, r_2, \dots, r_q\} \subset \mathbb{R},$$

where $q \in \mathbb{N}$ is the number of possible reward values and r_k is the real-valued reward with index $k \in \{1, \dots, q\}$. A reward may be negative, zero, or positive. Its sign and magnitude belong to the model and express how the modeller evaluates the corresponding one-step outcome.

Represent the reward produced after action A_t by the discrete random variable

$$R_{t+1} : \Omega \rightarrow \mathcal{R}.$$

The index $t + 1$ records the interaction order: the environment receives current state S_t and action A_t , then produces reward R_{t+1} together with next state S_{t+1} . Upper-case R_{t+1} denotes the random reward, while lower-case $r \in \mathcal{R}$ denotes one possible realised value.

The Markov claim must cover the complete one-step outcome, not only the next state. Let h_t be a realised history whose final state is $s \in \mathcal{S}$. For every feasible action $a \in \mathcal{A}(s)$, next state $s' \in \mathcal{S}$, reward $r \in \mathcal{R}$, and joint conditioning event satisfying $\Pr(H_t = h_t, A_t = a) > 0$, require

$$\begin{aligned} & \Pr(S_{t+1} = s', R_{t+1} = r \mid H_t = h_t, A_t = a) \\ &= \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a). \end{aligned}$$

This equality states that the current state and action contain the information needed to predict the joint distribution of the next state and immediate reward. At each time $t \in \mathbb{N}_0$, let $p_t(s', r \mid s, a)$ denote the joint probability that the controlled model assigns to outcome (s', r) after feasible pair (s, a) . Whenever $\Pr(S_t = s, A_t = a) > 0$, this model probability agrees with the elementary conditional probability

$$p_t(s', r \mid s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

The complete environment response is **time-homogeneous** when, for every $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, $s' \in \mathcal{S}$, $r \in \mathcal{R}$, and $t, u \in \mathbb{N}_0$,

$$p_t(s', r \mid s, a) = p_u(s', r \mid s, a).$$

Define the common value of these time-independent probabilities as the **joint transition–reward probability**

$$p(s', r \mid s, a) := p_t(s', r \mid s, a).$$

The arguments now state the meaning directly: (s, a) is the supplied feasible state–action pair and (s', r) is the joint environment outcome. For fixed s, a , and s' , several reward values r may have positive probability. The process then reaches the same next state with different immediate rewards. This can happen because:

- the same destination may be reached with different energy, time, or resource costs;
- the environment may provide a stochastic bonus or penalty;
- events occurring during the transition may affect the reward without changing the recorded next state.

For each feasible current state–action pair, the joint events cover every possible one-step outcome. Therefore

$$p(s', r \mid s, a) \geq 0, \quad \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1.$$

The controlled transition probability $p(s' \mid s, a)$ ignores which reward accompanied next state s' . The reward events $R_{t+1} = r$ over all $r \in \mathcal{R}$ partition that next-state event, so the finite law of total probability gives

$$\begin{aligned} p(s' \mid s, a) &= \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} p(s', r \mid s, a). \end{aligned}$$

Thus the transition kernel is the next-state marginal of the joint transition–reward kernel.

Before the next state or reward is observed, the **expected one-step reward** for feasible pair (s, a) is defined by

$$\bar{r}(s, a) := \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a].$$

For a finite reward set, expectation multiplies each possible reward by its conditional probability and sums the results. Marginalising the joint kernel over the next state gives

$$\begin{aligned}\bar{r}(s, a) &= \sum_{r \in \mathcal{R}} r \Pr(R_{t+1} = r \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a) \\ &= \boxed{\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r p(s', r \mid s, a)}.\end{aligned}$$

The scalar $\bar{r}(s, a)$ averages over both the next-state uncertainty and the reward uncertainty associated with the current state–action pair.

Sometimes the next state is included in a transition record. For a next state satisfying $p(s' \mid s, a) > 0$, define the **expected transition reward**

$$\boxed{\bar{r}(s, a, s') := \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s']}.$$

The corresponding conditional reward probability follows from the ratio definition of conditional probability:

$$\Pr(R_{t+1} = r \mid S_t = s, A_t = a, S_{t+1} = s') = \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}.$$

Substitution into the finite expectation gives

$$\boxed{\bar{r}(s, a, s') = \frac{\sum_{r \in \mathcal{R}} r p(s', r \mid s, a)}{p(s' \mid s, a)}, \quad p(s' \mid s, a) > 0}.$$

The positivity condition is essential because conditioning on a next state with zero transition probability is undefined. The joint kernel $p(s', r \mid s, a)$ specifies the complete finite one-step distribution. The expected rewards $\bar{r}(s, a)$ and $\bar{r}(s, a, s')$ are summaries of that distribution and generally do not determine its individual reward probabilities.

Example: Transition-dependent rewards

This example demonstrates how one joint distribution determines transition probabilities and expected rewards. In the three-state process, consider current state $s_2 = \text{clue held}$ and action $a_1 = \text{continue}$. Let $\mathcal{R} = \{-1, 0, 2\}$ and define the possible joint outcomes by

Next state	Reward	Joint probability
$s_1 = \text{searching}$	-1	0.1
$s_2 = \text{clue held}$	0	0.4
$s_3 = \text{answered}$	2	0.5

The joint probabilities sum to $0.1 + 0.4 + 0.5 = 1$. Marginalising over reward reproduces the controlled transition row

$$[p(s_1 | s_2, a_1) \quad p(s_2 | s_2, a_1) \quad p(s_3 | s_2, a_1)] = [0.1 \quad 0.4 \quad 0.5].$$

The expected one-step reward is

$$\begin{aligned} \bar{r}(s_2, a_1) &= (-1)(0.1) + (0)(0.4) + (2)(0.5) \\ &= 0.9. \end{aligned}$$

Each next state has exactly one possible reward in this example, so the expected transition rewards are $\bar{r}(s_2, a_1, s_1) = -1$, $\bar{r}(s_2, a_1, s_2) = 0$, and $\bar{r}(s_2, a_1, s_3) = 2$. The transition row alone does not contain these reward values.

Quick test: rewards and joint dynamics

1. **Recall:** Why is the reward following action A_t indexed by $t + 1$?
2. **Representation:** What does each argument in $p(s', r | s, a)$ identify, and why are (s', r) written to the left of the conditioning bar?
3. **Derivation:** Derive $p(s' | s, a) = \sum_{r \in \mathcal{R}} p(s', r | s, a)$ from the joint transition–reward law.
4. **Derivation:** Derive $\bar{r}(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r p(s', r | s, a)$.
5. **Validity:** Why is $\bar{r}(s, a, s')$ defined by the displayed ratio only when $p(s' | s, a) > 0$?
6. **Model distinction:** Do $p(s' | s, a)$ and $\bar{r}(s, a)$ determine the complete joint transition–reward distribution? Explain.

Answers and hints

1. State S_t and action A_t are the inputs to one environment step; reward R_{t+1} and state S_{t+1} are the outputs of that step.
2. The variables s and a are the supplied current state and feasible action. The variables s' and r are the possible next-state and reward outcome. They appear to the left of the bar because their joint probability is being evaluated conditional on (s, a) .

3. For fixed (s, a, s') , the mutually exclusive reward events partition the event $S_{t+1} = s'$. Summing their joint probabilities therefore gives its marginal probability: $p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$.
4. First sum $p(s', r \mid s, a)$ over s' to obtain the probability of reward r given (s, a) , multiply by r , and then sum over r . Rearranging the finite sums gives the stated formula for $\bar{r}(s, a)$.
5. The value $p(s' \mid s, a)$ is the probability of reaching s' from (s, a) . If it is zero, that transition never occurs, so there are no such outcomes on which to condition the reward distribution. The elementary conditional-probability ratio would divide by zero and is undefined.
6. No. The transition probability removes the reward variable by marginalisation, and the expected reward retains only a weighted average. Different reward distributions can have the same transition probabilities and expected reward.

13.6 Policies and policy-induced dynamics

The controlled environment assigns a next-outcome distribution to every feasible state–action pair. To obtain one distribution for what happens from a state, the agent must also specify the probability with which it selects each feasible action in that state.

A **stationary Markov stochastic policy** assigns a probability mass function to the feasible action set of every state. For each state $s \in \mathcal{S}$, this function is

$$\pi(\cdot \mid s) : \mathcal{A}(s) \rightarrow [0, 1], \quad a \mapsto \pi(a \mid s),$$

where $\pi(a \mid s)$ is the probability of selecting feasible action a when the current state is s . For every state, the policy probabilities satisfy

$$\begin{aligned} \pi(a \mid s) &\geq 0 \quad \text{for every } a \in \mathcal{A}(s), \\ \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) &= 1. \end{aligned}$$

Thus $\pi(\cdot \mid s)$ is defined only on feasible actions and distributes all probability among them. No probability needs to be invented for an infeasible state–action pair.

The three adjectives describe distinct properties. The policy is *Markov* because action selection depends on the complete interaction history only through the current state. It is *stationary* because the same state-to-action mapping π is used at every time t . It is *stochastic* because several feasible actions may have positive probability in one state.

Indices enter when the policy is stored in a matrix using the global state order $(s_1, \dots, s_n)$ and global action order $(a_1, \dots, a_m)$. Define the **policy matrix** by zero-padding its infeasible action columns:

$$[\mathbf{\Pi}]_{i\ell} := \begin{cases} \pi(a_\ell \mid s_i), & a_\ell \in \mathcal{A}(s_i), \\ 0, & a_\ell \notin \mathcal{A}(s_i), \end{cases} \quad \mathbf{\Pi} \in [0, 1]^{n \times m}.$$

Row i of $\mathbf{\Pi}$ represents $\pi(\cdot \mid s_i)$, and column ℓ represents global action label a_ℓ . The zero entries are a matrix-storage convention, not environment transition rows for infeasible actions. Every matrix row sums to one. The policy contains a distribution for every state in $\mathcal{S}$, so it is a complete state-to-action rule rather than a record of one realised trajectory.

The policy and environment kernels specify what happens conditionally after a state is given, but they do not specify the probability of the first state S_0 . Define the **initial state distribution** by

$$\begin{aligned} p_0(s) &:= \Pr(S_0 = s), \quad s \in \mathcal{S}, \\ p_0(s) &\geq 0, \quad \sum_{s \in \mathcal{S}} p_0(s) = 1. \end{aligned}$$

Using the declared state order, the corresponding row vector is

$$\mathbf{p}_0 := [p_0(s_1) \ p_0(s_2) \ \cdots \ p_0(s_n)] \in [0, 1]^{1 \times n}.$$

Fix p_0 , policy π , and the controlled environment kernel $p(s', r \mid s, a)$. At each time step, the policy selects an action and the environment then produces a next-state–reward outcome:

$$\begin{aligned} S_t = s &\xrightarrow[\text{policy}]{\pi(a|s)} A_t = a \\ &\xrightarrow[\text{environment}]{p(s', r|s, a)} (S_{t+1} = s', R_{t+1} = r). \end{aligned}$$

To define trajectory probabilities, choose an integer horizon $T \geq 1$, state values $s_0, \dots, s_T \in \mathcal{S}$, feasible actions $a_t \in \mathcal{A}(s_t)$ for $t \in \{0, \dots, T-1\}$, and reward values $r_1, \dots, r_T \in \mathcal{R}$. Define the corresponding **finite trajectory event**

$$\begin{aligned} E_T &:= \{S_0 = s_0\} \\ &\cap \bigcap_{t=0}^{T-1} \{A_t = a_t, S_{t+1} = s_{t+1}, R_{t+1} = r_{t+1}\}. \end{aligned}$$

The **trajectory probability law under policy** π , denoted by $\Pr_\pi$, is defined on each such event by

$$\Pr_{\pi}(E_T) := p_0(s_0) \prod_{t=0}^{T-1} \pi(a_t | s_t) p(s_{t+1}, r_{t+1} | s_t, a_t).$$

The first factor selects the initial state. For every time t , factor $\pi(a_t | s_t)$ is the probability of selecting action a_t in state s_t , and factor $p(s_{t+1}, r_{t+1} | s_t, a_t)$ is the probability that the environment then produces state s_{t+1} and reward r_{t+1} . Probabilities of events containing several possible trajectories are obtained by summing the probabilities of their mutually exclusive finite trajectory events. The notation $\Pr_{\pi}$ displays the policy dependence; the fixed initial distribution p_0 and environment kernel are also part of this probability law.

For $T = 1$, the definition gives

$$\begin{aligned} \Pr_{\pi}(S_0 = s, A_0 = a, S_1 = s', R_1 = r) \\ = p_0(s) \pi(a | s) p(s', r | s, a). \end{aligned}$$

The initial distribution affects trajectory and state-visitation probabilities, but it does not affect the conditional policy-induced transition kernel. Let H_t denote the complete interaction history available immediately before action A_t is selected, so H_t ends with current state S_t . For every positive-probability history value h_t ending in s and every feasible action $a \in \mathcal{A}(s)$, the Markov and stationary conditions are expressed by

$$\Pr_{\pi}(A_t = a | H_t = h_t) = \pi(a | s), \quad \Pr_{\pi}(H_t = h_t) > 0.$$

The right-hand side contains neither earlier history values nor the time index t . If $\Pr_{\pi}(S_t = s) > 0$, the policy value also has the elementary current-state conditional-probability interpretation

$$\pi(a | s) = \Pr_{\pi}(A_t = a | S_t = s).$$

If $\Pr_{\pi}(S_t = s) = 0$, the elementary ratio for this conditional probability is undefined, but the policy distribution $\pi(\cdot | s)$ remains defined by the state-to-action mapping.

Example: A policy row with an infeasible action

This example demonstrates the difference between the semantic policy and its matrix storage. Suppose $\mathcal{A} = \{a_1, a_2, a_3\}$ and $\mathcal{A}(s_2) = \{a_1, a_3\}$. Define

$$\pi(a_1 | s_2) = 0.4, \quad \pi(a_3 | s_2) = 0.6.$$

These two feasible-action probabilities sum to one. Because the matrix uses the global action order (a_1, a_2, a_3) , its second row is

$$[[\mathbf{\Pi}]_{21} \quad [\mathbf{\Pi}]_{22} \quad [\mathbf{\Pi}]_{23}] = [0.4 \quad 0 \quad 0.6].$$

The middle matrix entry is zero because a_2 is infeasible in s_2 ; $\pi(a_2 \mid s_2)$ itself is outside the semantic policy's domain.

A **deterministic policy** is a mapping

$$\mu : \mathcal{S} \rightarrow \mathcal{A}, \quad \mu(s) \in \mathcal{A}(s) \quad \text{for every } s \in \mathcal{S},$$

that selects exactly one feasible action in each state. It is the special stochastic policy

$$\pi_\mu(a \mid s) = \begin{cases} 1, & a = \mu(s), \\ 0, & a \neq \mu(s), \end{cases} \quad a \in \mathcal{A}(s).$$

Thus a stochastic policy can randomise among several feasible actions, whereas a deterministic policy assigns all probability to one feasible action.

13.6.1 The policy-induced transition matrix

Fix current state $s \in \mathcal{S}$ and possible next state $s' \in \mathcal{S}$. Under policy π , each action $a \in \mathcal{A}(s)$ may be selected with probability $\pi(a \mid s)$, after which the environment reaches s' with probability $p(s' \mid s, a)$. When $\Pr_\pi(S_t = s) > 0$, the events $A_t = a$ over $a \in \mathcal{A}(s)$ partition the possible action outcomes conditional on $S_t = s$. The conditional law of total probability and the product rule therefore give

$$\begin{aligned} \Pr_\pi(S_{t+1} = s' \mid S_t = s) &= \sum_{a \in \mathcal{A}(s)} \Pr_\pi(S_{t+1} = s', A_t = a \mid S_t = s) \\ &= \sum_{a \in \mathcal{A}(s)} \Pr_\pi(S_{t+1} = s' \mid S_t = s, A_t = a) \Pr_\pi(A_t = a \mid S_t = s) \\ &= \sum_{a \in \mathcal{A}(s)} p(s' \mid s, a) \pi(a \mid s). \end{aligned}$$

The last equality uses the environment kernel $\Pr_\pi(S_{t+1} = s' \mid S_t = s, A_t = a) = p(s' \mid s, a)$ and the policy definition $\Pr_\pi(A_t = a \mid S_t = s) = \pi(a \mid s)$.

Define the **policy-induced transition probability**

$$p_\pi(s' \mid s) := \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) p(s' \mid s, a).$$

For fixed current state s , the policy forms a weighted average of the controlled next-state distributions available in that state. Every value is non-negative. Summing over all possible next states gives

$$\begin{aligned}
 \sum_{s' \in \mathcal{S}} p_\pi(s' \mid s) &= \sum_{s' \in \mathcal{S}} \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) p(s' \mid s, a) \\
 &= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \left(\sum_{s' \in \mathcal{S}} p(s' \mid s, a) \right) \\
 &= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \\
 &= 1.
 \end{aligned}$$

In the third line, each controlled next-state distribution sums to one. Thus $p_\pi(\cdot \mid s)$ is a probability mass function. Indices enter when the induced kernel is stored as the row-stochastic **policy-induced transition matrix**

$$[\mathbf{P}^\pi]_{ij} := p_\pi(s_j \mid s_i), \quad \mathbf{P}^\pi \in [0, 1]^{n \times n}.$$

Define the **state distribution under policy** π by

$$d_t^\pi(s) := \Pr_\pi(S_t = s), \quad s \in \mathcal{S}.$$

The letter d labels the state distribution, the subscript t identifies time, and the superscript π identifies the policy that generates the probabilities. Its row-vector representation is

$$\mathbf{d}_t^\pi := [d_t^\pi(s_1) \quad d_t^\pi(s_2) \quad \cdots \quad d_t^\pi(s_n)].$$

The fixed initial distribution supplies the initial condition $d_0^\pi(s) = p_0(s)$ for every state, or equivalently

$$\mathbf{d}_0^\pi = \mathbf{p}_0.$$

For each next state $s' \in \mathcal{S}$, the finite law of total probability gives

$$\begin{aligned}
 d_{t+1}^\pi(s') &= \sum_{s \in \mathcal{S}} \Pr_\pi(S_{t+1} = s' \mid S_t = s) d_t^\pi(s) \\
 &= \sum_{s \in \mathcal{S}} d_t^\pi(s) p_\pi(s' \mid s).
 \end{aligned}$$

These component equations form the row-vector propagation rule

$$\mathbf{d}_{t+1}^\pi = \mathbf{d}_t^\pi \mathbf{P}^\pi.$$

The controlled Markov property removes dependence on the earlier history after the current state and action are known. A stationary Markov policy supplies the same action distribution $\pi(\cdot \mid s)$ at every time and depends only on the current state. Therefore, for every time t , every realised history h_t ending in state s with $\Pr_\pi(H_t = h_t) > 0$, and every next state $s' \in \mathcal{S}$,

$$\begin{aligned} \Pr_\pi(S_{t+1} = s' \mid H_t = h_t) &= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) p(s' \mid s, a) \\ &= p_\pi(s' \mid s). \end{aligned}$$

The right-hand side depends on the history only through its final state s , so the state sequence has the Markov property. The same kernel $p_\pi(s' \mid s)$ applies at every time t , so the state sequence is time-homogeneous. It is therefore a time-homogeneous Markov chain with transition matrix $\mathbf{P}^\pi$.

The controlled transition kernel and policy have different roles. The kernel $p(s' \mid s, a)$ describes how the environment responds after action a is supplied in state s . The policy $\pi(a \mid s)$ describes how the agent selects that action. Their weighted combination $p_\pi(s' \mid s)$ describes the state sequence observed when that policy controls the environment.

For a deterministic policy μ , only action $\mu(s)$ has non-zero policy probability in state s , so

$$p_\mu(s' \mid s) = p(s' \mid s, \mu(s)).$$

A deterministic policy therefore selects one controlled next-state distribution in each state instead of averaging several distributions. Its matrix representation satisfies $[\mathbf{P}^\mu]_{ij} = p_\mu(s_j \mid s_i)$.

13.6.2 Policy-induced joint outcomes and rewards

The same action averaging applies to the joint next-state–reward law. Define the **policy-induced joint outcome probability**

$$p_\pi(s', r \mid s) := \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) p(s', r \mid s, a).$$

Every $p_\pi(s', r \mid s)$ is non-negative. For a fixed current state s , summing over every possible next state and reward gives

$$\begin{aligned}
\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p_{\pi}(s', r \mid s) &= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \left(\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \right) \\
&= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \\
&= 1.
\end{aligned}$$

Thus $p_{\pi}(\cdot, \cdot \mid s)$ is a valid joint probability mass function for each current state s . Marginalising this joint distribution over reward recovers the policy-induced transition probability:

$$\sum_{r \in \mathcal{R}} p_{\pi}(s', r \mid s) = p_{\pi}(s' \mid s).$$

When $\Pr_{\pi}(S_t = s) > 0$, the defined value has the conditional-probability interpretation

$$p_{\pi}(s', r \mid s) = \Pr_{\pi}(S_{t+1} = s', R_{t+1} = r \mid S_t = s).$$

The **policy-induced expected reward** in state s is defined from the policy-induced joint distribution:

$$\bar{r}_{\pi}(s) := \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r p_{\pi}(s', r \mid s).$$

Let $X : \Omega \rightarrow \mathbb{R}$ be a random variable with finite value set $\mathcal{X} := X(\Omega)$, and let $B \subseteq \Omega$ be an event with $\Pr_{\pi}(B) > 0$. Define expectation and conditional expectation under $\Pr_{\pi}$ by

$$\begin{aligned}
\mathbb{E}_{\pi}[X] &:= \sum_{x \in \mathcal{X}} x \Pr_{\pi}(X = x), \\
\mathbb{E}_{\pi}[X \mid B] &:= \sum_{x \in \mathcal{X}} x \Pr_{\pi}(X = x \mid B).
\end{aligned}$$

For $X = R_{t+1}$ and $B = \{S_t = s\}$, the notation $\mathbb{E}_{\pi}[R_{t+1} \mid S_t = s]$ abbreviates the second definition. Whenever $\Pr_{\pi}(S_t = s) > 0$, marginalising the policy-induced joint outcome probability over the next state gives

$$\begin{aligned}
\mathbb{E}_\pi[R_{t+1} \mid S_t = s] &= \sum_{r \in \mathcal{R}} r \Pr_\pi(R_{t+1} = r \mid S_t = s) \\
&= \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p_\pi(s', r \mid s) \\
&= \bar{r}_\pi(s).
\end{aligned}$$

Substituting the policy-induced joint probabilities into $\bar{r}_\pi(s)$ and rearranging the finite sums yields

$$\begin{aligned}
\bar{r}_\pi(s) &= \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r p_\pi(s', r \mid s) \\
&= \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) p(s', r \mid s, a) \\
&= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \left(\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r p(s', r \mid s, a) \right) \\
&= \boxed{\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \bar{r}(s, a)}.
\end{aligned}$$

Using the global state order, define the reward vector

$$\bar{\mathbf{r}}^\pi := [\bar{r}_\pi(s_1) \quad \bar{r}_\pi(s_2) \quad \cdots \quad \bar{r}_\pi(s_n)] \in \mathbb{R}^{1 \times n}.$$

Then $\mathbf{P}^\pi$ and $\bar{\mathbf{r}}^\pi$ form the policy-induced Markov reward model. If rewards are ignored, $\mathbf{P}^\pi$ alone defines the policy-induced Markov chain.

Example: Mixing continue and reset

This example demonstrates how a stochastic policy averages controlled transition rows and expected rewards. In every state, let the policy choose continue with probability 0.75 and reset with probability 0.25:

$$\pi(a_1 \mid s) = 0.75, \quad \pi(a_2 \mid s) = 0.25, \quad s \in \mathcal{S}.$$

Because the global action order is $(a_1, a_2) = (\text{continue}, \text{reset})$, the corresponding policy matrix is

$$\mathbf{\Pi} = \begin{bmatrix} 0.75 & 0.25 \\ 0.75 & 0.25 \\ 0.75 & 0.25 \end{bmatrix}.$$

Using the action-conditioned matrices from the earlier example,

$$\begin{aligned}\mathbf{P}^\pi &= 0.75\mathbf{P}^{(a_1)} + 0.25\mathbf{P}^{(a_2)} \\ &= \begin{bmatrix} 0.625 & 0.375 & 0 \\ 0.325 & 0.300 & 0.375 \\ 0.250 & 0 & 0.750 \end{bmatrix}.\end{aligned}$$

For example, the probability of moving from clue held to answered is

$$p_\pi(s_3 | s_2) = (0.75)(0.5) + (0.25)(0) = 0.375.$$

Suppose the expected reward for continuing from clue held is $\bar{r}(s_2, a_1) = 0.9$, as calculated above, and the expected reward for resetting from clue held is $\bar{r}(s_2, a_2) = -1$. The policy-induced expected reward in clue held is

$$\bar{r}_\pi(s_2) = (0.75)(0.9) + (0.25)(-1) = 0.425.$$

The policy averages probabilities to obtain an induced transition row and averages expected rewards to obtain an induced state reward.

Quick test: policies and induced dynamics

1. **Representation and recall:** What does $\pi(a | s)$ mean? How is it related to matrix entry $[\mathbf{\Pi}]_{i\ell}$ for feasible and infeasible actions?
2. **Contrast:** What is the difference between a stochastic policy and a deterministic policy?
3. **Derivation:** Prove that $p_\pi(\cdot | s)$ is normalized and hence every row of $\mathbf{P}^\pi$ sums to one.
4. **Application:** Verify the clue-held row $[0.325 \ 0.300 \ 0.375]$ in the example.
5. **Model distinction:** Which object describes the environment's response, which object describes the agent's action selection, and which object describes the resulting state sequence?
6. **Joint model:** Prove that $p_\pi(s', r | s)$ is normalized over (s', r) for each state s , and show how it recovers $p_\pi(s' | s)$.

Answers and hints

1. For $a \in \mathcal{A}(s)$, $\pi(a | s)$ is the probability of selecting action a in state s . Matrix entry $[\mathbf{\Pi}]_{i\ell}$ equals $\pi(a_\ell | s_i)$ when a_ℓ is feasible and is the zero-padding value when a_ℓ is infeasible. Feasible-action probabilities are non-negative and sum to one.
2. A stochastic policy may assign positive probability to several feasible actions. A deterministic policy selects exactly one feasible action in each state and is represented by policy probabilities zero and one.

3. Sum the defining equation over $s' \in \mathcal{S}$, interchange the finite sums, use $\sum_{s' \in \mathcal{S}} p(s' | s, a) = 1$ for every feasible action, and then use $\sum_{a \in \mathcal{A}(s)} \pi(a | s) = 1$.
4. The row is $0.75[0.1 \ 0.4 \ 0.5] + 0.25[1 \ 0 \ 0] = [0.325 \ 0.300 \ 0.375]$.
5. The controlled kernel $p(s' | s, a)$ describes the environment's response to a supplied action. The policy $\pi(a | s)$ describes action selection. The induced kernel $p_\pi(s' | s)$ describes the resulting state sequence, and $\mathbf{P}^\pi$ is its matrix representation.
6. Sum the definition of $p_\pi(s', r | s)$ over s' and r , interchange the finite sums, use $\sum_{s'} \sum_r p(s', r | s, a) = 1$ for every feasible action, and then use $\sum_{a \in \mathcal{A}(s)} \pi(a | s) = 1$. Summing only over reward gives $\sum_r p_\pi(s', r | s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) p(s' | s, a) = p_\pi(s' | s)$.

13.7 Episode endings: termination and truncation

A finite interaction record can end because the modelled task has ended or because an external collector has stopped recording. The distinction is useful because later return and value calculations must know whether future task rewards truly cease or merely become unobserved.

The known inputs are the task's state and transition definitions, the realised next state and reward, and any stopping rule imposed outside the task. The required outputs are two separate facts: whether the task terminated and whether collection was truncated. Their relationship is

$$\begin{aligned} \text{task-defined ending} &\longrightarrow \text{termination,} \\ \text{external collection ending} &\longrightarrow \text{truncation.} \end{aligned}$$

13.7.1 A terminal state ends the modelled task

Let $\mathcal{S}_{\text{term}} \subseteq \mathcal{S}$ be the set of **terminal states**, possibly empty, and define the set of nonterminal **decision states** by

$$\mathcal{S}_{\text{dec}} := \mathcal{S} \setminus \mathcal{S}_{\text{term}}.$$

A state-based transition terminates the task when its realised next state satisfies $S_{t+1} \in \mathcal{S}_{\text{term}}$. The reward R_{t+1} belongs to that completed transition and is retained. No action A_{t+1} is selected within the completed episode. A later reset starts a new episode by sampling a new initial state according to p_0 ; reset is not a continuation of the terminated trajectory.

For a state-based finite MDP, the next state must contain enough information to determine whether termination has occurred. If two physically different outcomes share one state label but only one ends the task, that state representation is not sufficient for the declared termination rule and must be enlarged.

13.7.2 An absorbing extension represents the stopped episode

A terminal state and an absorbing state answer different questions. *Terminal* means that the task-defined interaction stops. *Absorbing* means that a transition model keeps returning to the same state. A terminal state therefore has no real post-terminal action, whereas an absorbing state is defined through a transition law.

It is often useful to represent a terminal state $s_{\dagger} \in \mathcal{S}_{\text{term}}$ as an absorbing state so that ordinary state-sequence and matrix equations remain valid after the episode ends. This is a deliberate mathematical extension: introduce one bookkeeping action $a_{\perp} \in \mathcal{A}$, include 0 in $\mathcal{R}$ if necessary, and define a zero-reward self-transition

$$\mathcal{A}(s_{\dagger}) = \{a_{\perp}\}, \quad p(s_{\dagger}, 0 \mid s_{\dagger}, a_{\perp}) = 1.$$

All other next-state–reward outcomes from $(s_{\dagger}, a_{\perp})$ then have probability zero. The extended sequence remains in $s_{\dagger}$ and receives only zero rewards, but an episode interface still stops before requesting $a_{\perp}$. The bookkeeping row is fixed by the absorbing convention; it is not an arbitrary transition row invented for an infeasible physical action.

13.7.3 Truncation stops the record, not the task

A **truncation** occurs when a rule outside the MDP stops the current interaction record even though the modelled task may continue. Examples include a data-collection length cap, a wall-clock budget, or an exhausted external resource. In a pure truncation, the final recorded state is a nonterminal state in $\mathcal{S}_{\text{dec}}$ and the MDP still defines feasible subsequent actions and their outcome probabilities.

Termination and truncation must therefore be recorded separately. A termination flag states a fact about the task model. A truncation flag states a fact about the collection procedure. Stopping the current episode record requires a reset after either flag, but resetting after truncation does not imply that the underlying task had reached a terminal state. The two flags describe different conditions and need not be treated as logical complements.

Ending condition	Part of the MDP?	Could the modelled task continue from the final state?
Termination	Yes	No

Ending condition	Part of the MDP?	Could the modelled task continue from the final state?
Truncation	No	Yes, unless termination also occurred

13.7.4 A time limit can belong inside or outside the task

Let a task-defined finite horizon be $N \in \mathbb{N}$ decision steps, with actions selected at times $t \in \{0, \dots, N-1\}$. The transition to S_N completes the task because the horizon is part of its definition. This is termination, not truncation. If the transition and decision laws can depend on the number of decision steps remaining, a stationary Markov representation must include that information. Define the augmented state space and state by

$$\tilde{\mathcal{S}} := \mathcal{S} \times \{0, \dots, N\}, \quad \tilde{S}_t := (S_t, N - t).$$

The second component records the number of decision steps remaining. The augmented terminal set is

$$\tilde{\mathcal{S}}_{\text{term}} := \mathcal{S} \times \{0\}.$$

Thus reaching time N becomes state-based termination in the augmented model, and two otherwise identical physical states at different stages need not be forced to share one transition law or one stationary policy row. By contrast, if an indefinitely continuing task is recorded for at most L transitions only because the collector cannot run forever, L is external to the MDP and reaching it causes truncation.

Example: Equal record lengths with different endings

This example demonstrates why record length alone cannot identify the kind of ending. Suppose $g \in \mathcal{S}_{\text{term}}$ is a task-defined goal state and $x \in \mathcal{S}_{\text{dec}}$ is an ordinary state. One record receives its seventh reward and enters g on the seventh transition. That record terminates, and the seventh reward is part of the episode. A second record also contains seven transitions but ends in x because the collector has a seven-transition storage limit. The second record is truncated: the MDP still specifies what could happen after x , even though that continuation was not recorded.

Quick test: termination and truncation

1. **Recall:** What mathematical condition makes a realised transition terminate the task?
2. **Contrast:** Why is a terminal state not identical by definition to an absorbing state?

3. **Application:** A trajectory reaches its task-defined goal while receiving reward 5. Is the reward discarded when the episode terminates?
4. **Application:** A collector stops after 200 transitions in an ordinary nonterminal state. Is this termination or truncation?
5. **Validity condition:** A task lasts exactly N decision steps, but the optimal action can depend on the number of decision steps remaining. What must a stationary Markov state representation contain?

Answers and hints

1. A state-based transition terminates when its realised next state satisfies $S_{t+1} \in \mathcal{S}_{\text{term}}$. A task-defined finite horizon is also termination and can be expressed by augmenting the state with the remaining decision-step count.
2. Terminal is a task-ending meaning, whereas absorbing is the transition condition that the next state equals the same state with probability one. A zero-reward absorbing self-loop can represent a terminal state after the task has stopped, but that representation is an added convention.
3. No. Reward $R_{t+1} = 5$ belongs to the transition that enters the terminal state and remains part of the episode.
4. It is truncation because the external collector stopped the record while the MDP still admitted continuation from the final state.
5. The state must include the relevant stage information, such as the remaining decision-step count $N - t$. Otherwise identical physical-state labels can require different transition or decision laws at different stages.

13.8 The finite MDP and its task settings

The preceding sections provide all components needed to separate three questions: what outcomes the controlled environment permits, where an episode starts and ends, and how rewards at different decision steps are weighted. This separation is useful because changing a policy, initial distribution, horizon, discount, or collection limit does not necessarily change the environment dynamics.

13.8.1 The finite MDP specifies controlled one-step outcomes

In this book, the **finite Markov decision process** is the controlled environment model

$$\mathcal{M} := (\mathcal{S}, \mathcal{A}, \{\mathcal{A}(s)\}_{s \in \mathcal{S}}, \mathcal{R}, p).$$

The sets $\mathcal{S}$, $\mathcal{A}$, and $\mathcal{R}$ are finite. The family $\{\mathcal{A}(s)\}_{s \in \mathcal{S}}$ supplies the nonempty feasible action set for each state, and $p(s', r \mid s, a)$ supplies a normalized joint next-state–reward

distribution for each feasible pair (s, a) . Together these components answer the one-step question: after action a is applied in state s , what probabilities are assigned to the possible outcomes (s', r) ?

The word *finite* describes the cardinalities of the state, action, and reward sets. It does not mean that every trajectory has a finite number of decision steps. A finite MDP may define a continuing task with no terminal state and no fixed horizon.

This tuple convention keeps the initial distribution p_0 , policy π , and reward-weighting objective separate from the controlled environment. Some texts include p_0 or a discount factor in the object they call an MDP, so the declared tuple must be checked when comparing notation.

13.8.2 Initial conditions, policies, and task endings add distinct information

The initial distribution p_0 states where trajectories begin. The policy π states how actions are selected. Combining p_0 , π , and $\mathcal{M}$ produces the trajectory law $\Pr_\pi$. Changing p_0 or π generally changes trajectory probabilities without changing the controlled kernel $p(s', r | s, a)$.

The terminal set $\mathcal{S}_{\text{term}}$ and a task-defined horizon specify when task rewards cease. Let

$$N \in \mathbb{N} \cup \{\infty\},$$

where finite N denotes a fixed maximum of N decision steps and $N = \infty$ denotes the absence of a fixed horizon. A trajectory may still terminate before a finite horizon by entering $\mathcal{S}_{\text{term}}$, and it may terminate after a random number of decision steps when $N = \infty$. An external truncation rule is not part of $\mathcal{M}$ or the task horizon because it stops data collection rather than the modelled task.

13.8.3 Cumulative reward gives discounting a purpose

The one-step reward R_{t+1} evaluates only the transition from time t to time $t + 1$, but the chosen action can also affect later rewards. To compare such consequences, combine the reward sequence into one scalar. Choose a dimensionless **discount factor** $\gamma \in [0, 1]$.

Define the **discounted return** from decision epoch t by

$$G_t := \begin{cases} \sum_{k=0}^{N-t-1} \gamma^k R_{t+k+1}, & N < \infty, \\ \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, & N = \infty. \end{cases}$$

For finite N , the sum applies at $t \in \{0, \dots, N-1\}$ and $G_N := 0$. The return G_t is a random variable because its component rewards are random variables. For a finite horizon, the earlier zero-reward absorbing extension supplies zero terms if the episode terminates before time N . An external truncation stops observation of the reward sequence but does not redefine the task horizon or remove the unobserved terms from G_t .

The subscript in G_t identifies the decision epoch from which consequences are evaluated: R_{t+1} is the next reward and the immediate consequence of action A_t , even though the environment produces it after the transition from t to $t+1$.

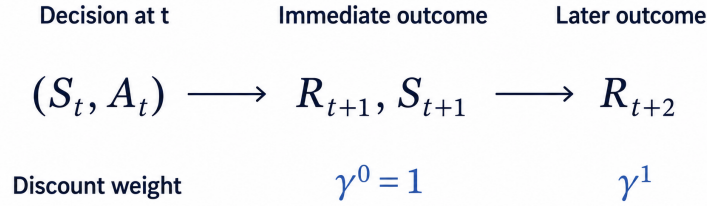


Figure 13.1: Decision timing and discount weights for the return G_t .

The exponent k is the number of additional decision steps before reward R_{t+k+1} is received. The immediate reward R_{t+1} has weight $\gamma^0 = 1$. If $0 < \gamma < 1$, later weights decrease geometrically. If $\gamma = 0$, only the immediate reward has nonzero weight. If $\gamma = 1$, rewards are not discounted.

For a continuing task, discounting also provides a simple convergence guarantee. Because $\mathcal{R}$ is finite, define the finite reward bound $R_{\max} := \max_{r \in \mathcal{R}} |r|$. If $N = \infty$ and $0 \leq \gamma < 1$, then

$$\sum_{k=0}^{\infty} |\gamma^k R_{t+k+1}| \leq R_{\max} \sum_{k=0}^{\infty} \gamma^k = \frac{R_{\max}}{1 - \gamma}.$$

Thus the infinite discounted return converges absolutely. When $N = \infty$ and $\gamma = 1$, this bound is unavailable and the undiscounted sum may diverge. The discount factor is not a transition probability: it does not alter $p(s', r \mid s, a)$ or the trajectory law under a

fixed policy. It changes the scalar assigned to a reward sequence and can therefore change which policy is preferred.

Example: One MDP with two discount factors

This example demonstrates that changing γ can change the preferred reward timing without changing the environment. Suppose one action produces reward 1 immediately and then terminates, while another produces reward 0 immediately and reward 2 one decision step later before terminating. Their returns are

$$1 \quad \text{and} \quad 0 + 2\gamma.$$

For $\gamma = 0.4$, the delayed alternative has return 0.8, so the immediate reward is larger. For $\gamma = 0.9$, the delayed alternative has return 1.8, so the delayed reward is larger. The transition and reward kernel is identical in both comparisons; only the reward-timing objective changes.

Quick test: the finite MDP and task settings

1. **Recall:** What does *finite* mean in a finite MDP?
2. **Contrast:** What is the difference between a finite MDP and a finite-horizon task?
3. **Model identification:** Which object specifies the probability of outcome (s', r) after feasible pair (s, a) ?
4. **Reasoning:** If p_0 changes while $\mathcal{M}$ and π remain fixed, which probability law changes and which kernel remains unchanged?
5. **Return and discounting:** Why is G_t needed, and does changing γ change the trajectory law under a fixed policy?
6. **Classification:** Is an external data-collection limit part of N or an external truncation rule?

Answers and hints

1. The state, action, and reward sets are finite. The trajectory length need not be finite.
2. A finite MDP has finite state, action, and reward sets. A finite-horizon task permits only a fixed finite number N of decision steps; either property can hold without the other.
3. The joint controlled kernel $p(s', r \mid s, a)$.
4. The trajectory law $\Pr_\pi$ changes because the initial-state probabilities change. The controlled kernel $p(s', r \mid s, a)$ remains unchanged.
5. The return G_t combines the sequence of rewards affected by an action into one scalar. The fixed-policy trajectory law does not contain γ , so changing γ does not change that law; it changes the relative weighting of rewards at different decision steps and can therefore change which policy is preferred.

6. It is an external truncation rule. Only a limit belonging to the task definition is represented by the task horizon N .

Cumulative chapter test

Use the following finite MDP for Questions 1–8. Its state, action, terminal-state, and reward sets are

$$\begin{aligned}\mathcal{S} &= \{s_1, s_2, s_\dagger\}, & \mathcal{S}_{\text{term}} &= \{s_\dagger\}, \\ \mathcal{A} &= \{a_1, a_2, a_\perp\}, \\ \mathcal{R} &= \{-1, 0, 1, 2, 4\}.\end{aligned}$$

The feasible action sets are

$$\mathcal{A}(s_1) = \mathcal{A}(s_2) = \{a_1, a_2\}, \quad \mathcal{A}(s_\dagger) = \{a_\perp\}.$$

The table lists every joint outcome with positive probability. Every outcome not listed has probability zero.

Current state	Action	Next state	Reward	$p(s', r \mid s, a)$
s_1	a_1	s_1	0	0.5
s_1	a_1	s_2	1	0.5
s_1	a_2	$s_\dagger$	2	1
s_2	a_1	$s_\dagger$	4	1
s_2	a_2	s_1	-1	1
$s_\dagger$	$a_\perp$	$s_\dagger$	0	1

In state order $(s_1, s_2, s_\dagger)$, let the initial distribution be

$$\mathbf{p}_0 = [1 \ 0 \ 0].$$

Let policy π be

$$\begin{aligned}\pi(a_1 \mid s_1) &= 0.6, & \pi(a_2 \mid s_1) &= 0.4, \\ \pi(a_1 \mid s_2) &= 0.75, & \pi(a_2 \mid s_2) &= 0.25, \\ \pi(a_\perp \mid s_\dagger) &= 1.\end{aligned}$$

1. **Model validation:** Explain why each feasible state–action pair defines a valid joint probability mass function.
2. **Marginalisation and expectation:** Calculate $p(s' \mid s_1, a_1)$ for every $s' \in \mathcal{S}$ and calculate $\bar{r}(s_1, a_1)$.
3. **Policy-induced model:** Derive the policy-induced transition matrix $\mathbf{P}^\pi$ in state order $(s_1, s_2, s_\dagger)$ and the reward vector $\bar{\mathbf{r}}^\pi$.
4. **Trajectory probability:** Calculate the probability under $\Pr_\pi$ of the two-step event

$$S_0 = s_1, A_0 = a_1, R_1 = 1, S_1 = s_2, A_1 = a_2, R_2 = -1, S_2 = s_1.$$

5. **Markov-chain identification:** Explain why the state sequence under π is a time-homogeneous Markov chain and identify its transition matrix.
6. **Termination:** What happens to the episode and its final reward when a transition enters $s_\dagger$? What role does the $a_\perp$ row serve?
7. **Truncation:** Suppose a collector stops a record in state s_2 solely because it has stored its maximum number of transitions. Has the MDP terminated?
8. **Return:** Suppose the event in Question 4 belongs to a task with horizon $N = 2$ and discount factor $\gamma = 0.5$. Calculate G_0 .
9. **Contrast:** Can a finite MDP have an infinite horizon? Can an MDP with infinitely many states have a finite horizon? Explain which use of *finite* each statement concerns.
10. **Validity conditions:** State one condition needed to use an elementary conditional probability and one condition that guarantees convergence of a continuing discounted return in this chapter.

Answers and hints

1. For each feasible pair, every listed probability is non-negative and the probabilities over all (s', r) sum to one. In particular, the two outcomes for (s_1, a_1) sum to $0.5 + 0.5 = 1$, and every other feasible pair has one probability-one outcome.
2. Marginalising over reward gives $p(s_1 \mid s_1, a_1) = 0.5$, $p(s_2 \mid s_1, a_1) = 0.5$, and $p(s_\dagger \mid s_1, a_1) = 0$. Therefore $\bar{r}(s_1, a_1) = (0)(0.5) + (1)(0.5) = 0.5$.
3. Averaging each controlled row and expected reward with the policy gives

$$\mathbf{P}^\pi = \begin{bmatrix} 0.30 & 0.30 & 0.40 \\ 0.25 & 0 & 0.75 \\ 0 & 0 & 1 \end{bmatrix}, \quad \bar{\mathbf{r}}^\pi = [1.10 \quad 2.75 \quad 0].$$

4. The trajectory law multiplies the initial-state, action-selection, and joint-outcome probabilities:

$$(1)(0.6)(0.5)(0.25)(1) = 0.075.$$

5. The controlled kernel is Markov and time-homogeneous, and the stationary Markov policy depends only on the current state. Their action-weighted kernel $p_\pi(s' | s)$ therefore has the same rows at every time and depends on no earlier state. Its matrix is $\mathbf{P}^\pi$ from Answer 3.
6. The transition reward is retained and the episode then stops before another real action is requested. The $a_\perp$ row is the zero-reward absorbing extension used to keep state-sequence and matrix calculations valid after termination.
7. No. This is truncation: $s_2 \notin \mathcal{S}_{\text{term}}$, and the MDP still defines feasible actions and outcome probabilities from s_2 .
8. The two rewards are $R_1 = 1$ and $R_2 = -1$, so

$$G_0 = R_1 + \gamma R_2 = 1 + (0.5)(-1) = 0.5.$$

9. Yes to both. A finite MDP has finite state, action, and reward sets; its horizon may be infinite. A finite-horizon MDP has finitely many decision steps; its state space need not be finite.
10. The elementary conditional probability $\Pr(E | B)$ requires $\Pr(B) > 0$. For the continuing return used here, the finite reward set supplies a bound $R_{\max} < \infty$, and $0 \leq \gamma < 1$ then guarantees absolute convergence.

References

- Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*, second-edition draft, Chapter 3, especially Sections 3.1–3.4, 3.6, and 3.7 on the agent–environment interaction, rewards, episodic and continuing tasks, terminal-state extensions, finite-MDP dynamics, policies, and policy-dependent state evolution.
- Dimitri P. Bertsekas, *Reinforcement Learning and Optimal Control*, second edition, Chapter 1, especially Sections 1.3.1, 1.4, 1.6.2, and 1.6.4 on stochastic controlled models, discounted objectives, termination states, and finite-to-infinite-horizon reformulations.
- Zhiquing Xiao, *Reinforcement Learning: Theory and Python Implementation*, Chapters 1 and 2, especially the agent–environment interface and Sections 2.1.1–2.1.4 on finite-MDP components, policies, terminal states, and discounting.
- Farama Foundation, *Gymnasium Documentation*, “Handling Time Limits,” on the distinction between task-defined termination and externally imposed truncation.

14 Finite-MDP Model Implementation

Prerequisites

The feasible action sets in Section 13.1 distinguish a global action label from an action available in one state. The joint transition–reward kernel in Section 13.5 assigns a probability to each possible next-state–reward event; summing those probabilities gives the next-state marginal, and weighting them by reward gives the expected one-step reward. The absorbing extension in Section 13.7 retains the reward on entering a terminal state and assigns zero reward to subsequent self-transitions.

14.1 Identifying states and actions

A state and an action may both be represented by integers, but interchanging their roles would change the meaning of a model query. We need an index into each finite domain and a way to keep state indices distinct from action indices.

14.1.1 Mathematical domains and ordering

Let $\mathcal{S} = \{s_1, \dots, s_n\}$ be the state set with a fixed order, where $n = |\mathcal{S}| \geq 1$. The bars denote the number of elements in a finite set. Let $\mathcal{A}_{\text{task}} = \{a_1, \dots, a_{m_{\text{task}}}\}$ be the **task-action set**: the labels for real task decisions, excluding artificial actions used only after termination. Its size is $m_{\text{task}} = |\mathcal{A}_{\text{task}}|$; when this size is zero, the set is empty.

We choose zero-based integer indices for storage. State index i represents s_{i+1} for $0 \leq i < n$, and task-action index ℓ represents $a_{\ell+1}$ for $0 \leq \ell < m_{\text{task}}$. These indices identify mathematical objects; they do not encode a state’s physical meaning or an action’s effect. For example, with three states and two task actions, the state indices are 0, 1, 2 and the task-action indices are 0, 1.

14.1.2 Class declarations

```

#include <compare>
#include <cstdint>

class StateIndex
{
private:
    std::size_t value_;

public:
    explicit constexpr StateIndex(std::size_t value) noexcept : value_{value} {}

    [[nodiscard]] constexpr std::size_t value() const noexcept { return value_; }

    constexpr auto operator<=>(const StateIndex &) const noexcept = default;
};

class ActionIndex
{
private:
    std::size_t value_;

public:
    explicit constexpr ActionIndex(std::size_t value) noexcept : value_{value} {}

    [[nodiscard]] constexpr std::size_t value() const noexcept { return value_; }

    constexpr auto operator<=>(const ActionIndex &) const noexcept = default;
};

```

`StateIndex` represents a state index and `ActionIndex` an action index. Both store a `std::size_t` index value. The enclosing classes are distinct **strong types**, so code cannot silently use an action index where a state index is required, or vice versa.

`value_` stores the index, and `value()` returns it. The explicit constructors prevent implicit integer-to-index conversion. The defaulted `operator<=>` compares stored index values, supporting equality and ordering within each index type.

14.1.3 Concrete use and conversion checks

Example: Constructing and reading typed indices

For a three-state, two-action model, the following calls construct two state indices and one action index, then read back an integer index:

```
const StateIndex first_state{0};
const StateIndex last_state{2};
const ActionIndex second_action{1};

const std::size_t state_index = last_state.value();
```

Here `first_state` represents s_1 , `last_state` represents s_3 , and `second_action` represents a_2 . Reading `last_state.value()` gives `state_index` the value 2.

The index classes do not store a model's size. They distinguish state from action, while construction and query operations establish whether an index belongs to a particular model. External labels such as $\{1, 5, 8, 10\}$ therefore need a separate mapping to four internal indices $\{0, 1, 2, 3\}$.

The conversion properties can be checked during compilation:

```
#include <type_traits>

static_assert(std::is_constructible_v<StateIndex, std::size_t>);
static_assert(!std::is_convertible_v<std::size_t, StateIndex>);
static_assert(!std::is_convertible_v<StateIndex, ActionIndex>);
```

These compile-time checks establish the intended conversion contract: integer-to-state construction is available, but implicit integer-to-state and state-to-action conversion are not. `std::is_constructible_v` checks direct construction, whereas `std::is_convertible_v` checks implicit conversion.

14.2 Storing a joint-outcome distribution

For one feasible current pair (s, a) , we need to store the possible next states together with their rewards and probabilities. Keeping the next state and reward in the same record preserves their relationship.

14.2.1 Mathematical records and their representation

Let $K \geq 1$ be the number of positive-probability events for the fixed pair. For record $k \in \{1, \dots, K\}$, let $s'_k \in \mathcal{S}$ be the next state, $r_k \in \mathbb{R}$ the reward, and $q_k = p(s'_k, r_k \mid s, a)$

the joint probability. Distinct records represent distinct pairs (s'_k, r_k) , with

$$0 < q_k \leq 1, \quad \sum_{k=1}^K q_k = 1.$$

Omitted events have probability zero. Probabilities are dimensionless; rewards are scalar task evaluations with no physical unit imposed by this generic model.

```
struct OutcomeProbability
{
    StateIndex next_state;
    double reward;
    double probability;
};
```

The field `next_state` stores the next-state index representing s'_k ; `reward` and `probability` store r_k and q_k respectively. The current state and action are absent because every record in one distribution shares that conditioning pair.

Example: Recording two rewards for the same next state

The same next state may have several different rewards. These records show how to retain both events in a three-state domain:

```
#include <vector>

const std::vector<OutcomeProbability> outcomes{
    {StateIndex{0}, -1.0, 0.2},
    {StateIndex{1}, 4.0, 0.3},
    {StateIndex{1}, 5.0, 0.5},
};
```

The vector stores the events $(s_1, -1)$, $(s_2, 4)$, and $(s_2, 5)$ with probabilities 0.2, 0.3, 0.5. The last two records are different events even though they share a next state. No event reaches s_3 from this pair.

14.2.2 The distribution class

```
#include <span>

class JointOutcomeDistribution
{
private:
```

```

JointOutcomeDistribution(
    std::size_t state_count, std::vector<OutcomeProbability> outcomes);

std::size_t state_count_;
std::vector<OutcomeProbability> outcomes_;

public:
    [[nodiscard]] static JointOutcomeDistribution from_outcomes(
        std::size_t state_count, std::vector<OutcomeProbability> outcomes,
        double probability_sum_tolerance);

    [[nodiscard]] std::size_t state_count() const noexcept;
    [[nodiscard]] std::span<const OutcomeProbability> outcomes() const noexcept;
    [[nodiscard]] std::vector<double> next_state_probabilities() const;
    [[nodiscard]] double expected_reward() const noexcept;
};

```

`JointOutcomeDistribution` owns the outcome vector in `outcomes_` and the size of the full state space in `state_count_`. The domain can contain states absent from the positive-probability records.

`from_outcomes` creates a validated distribution; `outcomes()` exposes its records; `next_state_probabilities` calculates a marginal vector; and `expected_reward()` calculates the expected one-step reward. The private constructor prevents callers from directly storing an unchecked vector.

The returned span borrows the distribution's records, so it depends on their owner's lifetime. The calculated probability vector and expected reward can be used independently of the distribution.

14.2.3 Construction and ownership

The construction operation checks the records and then transfers its local storage into the new object:

```

#include <utility>

JointOutcomeDistribution JointOutcomeDistribution::from_outcomes(
    std::size_t state_count, std::vector<OutcomeProbability> outcomes,
    double probability_sum_tolerance)
{
    validate_distribution(state_count, outcomes, probability_sum_tolerance);

    return JointOutcomeDistribution(state_count, std::move(outcomes));
}

```

```

}

JointOutcomeDistribution::JointOutcomeDistribution(
    std::size_t state_count, std::vector<OutcomeProbability> outcomes)
: state_count_{state_count}, outcomes_{std::move(outcomes)}
{
}

```

The **factory** `from_outcomes` creates a distribution from the state-domain size, the outcome records, and the allowed probability-sum error.

The helper `validate_distribution` establishes the mathematical conditions before the private constructor runs. It checks valid next-state indices, finite rewards, unique events, positive finite probabilities, and the row sum. A failed condition raises an exception and construction produces no distribution; accepted probabilities are not repaired or renormalised.

The vector parameter is passed by value. Passing a named vector copies it into the local parameter; an eligible temporary or moved vector can transfer storage. `std::move` permits the receiving constructor to take ownership of that local vector's storage. It does not itself move elements or validate them. The constructor's initialiser list stores the state count and transfers the records into `outcomes_`.

Example: Constructing a distribution from the records

Using the three records from the preceding example, construction is requested as follows:

```

const auto distribution = JointOutcomeDistribution::from_outcomes(
    3U, outcomes, 1.0e-12);

```

The arguments specify a three-state domain, the outcome records, and a dimensionless sum tolerance of 10^{-12} . The source vector `outcomes` remains a separate value because this call copies the named vector.

The records satisfy the construction conditions, including a probability sum of one, so the call returns a distribution owning its own copy of the three records.

14.2.4 Recognising repeated events

The next state and reward form the event's identity. During construction, the following excerpt records those identities and rejects repetition:

```

#include <set>
#include <utility>

```

```

std::set<std::pair<StateIndex, double>> observed_outcomes;

for (const OutcomeProbability & outcome : outcomes) {
    const bool inserted =
        observed_outcomes.emplace(outcome.next_state, outcome.reward).second;

    if (!inserted) {
        throw FiniteMdpException{
            FiniteMdpError::duplicate_outcome,
            "A joint outcome distribution contains a duplicate event."};
    }
}

```

This excerpt shows the event-identity operation; construction checks reward finiteness before inserting each key. A `std::pair<StateIndex, double>` is a two-field key for (s', r) , holding the next-state index followed by the reward. The surrounding `std::set` holds unique keys in comparison order. The loop attempts to insert each record's event key.

`emplace` constructs a candidate key from the next-state index and reward. It returns another pair: an iterator to the stored key and a Boolean insertion flag. The final `.second` selects that flag, not the event's reward. If the key already exists, the flag is false and the set remains unchanged.

Repeated keys raise `FiniteMdpException` with category `FiniteMdpError::duplicate_outcome`: two records claim to be the same event.

To name both insertion results, the same call can instead be written:

```

const auto [iterator, inserted] =
    observed_outcomes.emplace(outcome.next_state, outcome.reward);

```

The brackets form a **structured binding**, giving separate names to the returned components. An **iterator** identifies a position in a container. Here `iterator` refers to the newly inserted key, or to the existing equivalent key if insertion did not occur. This is an alternative spelling of the earlier insertion line, not another call to execute after it.

Example: Reading the result of an insertion

Inserting the same event twice distinguishes the key's components from the insertion result's components:

```

std::set<std::pair<StateIndex, double>> events;
const auto first = events.emplace(StateIndex{1}, 4.0);
const auto again = events.emplace(StateIndex{1}, 4.0);

const StateIndex next_state_index = first.first->first;
const double reward_value = first.first->second;

```

`first.second` is true and `again.second` is false. Their `.first` members are iterators to the same key. `first.first->first` retrieves the key's next-state index and `first.first->second` retrieves its reward. Thus `next_state_index` contains 1 and `reward_value` contains 4. Inserting a reward of 5 with the same next-state index would create a different event.

The ordering compares next-state indices first and reward second. Two keys are equivalent when neither precedes the other. Finite reward values are compared exactly, not using a tolerance; positive and negative floating-point zero compare equal. Rejecting non-finite rewards before insertion prevents NaN, the floating-point marker for “not a number”, from invalidating the ordering.

14.2.5 Reading records without copying them

```
std::span<const OutcomeProbability> JointOutcomeDistribution::outcomes()
    const noexcept
{
    return std::span<const OutcomeProbability>{outcomes_};
}
```

The returned span provides read-only access to `outcomes_` without copying its records. The owning distribution and its storage must remain valid while the view is used; do not retain it across destruction or replacement of the owner.

14.3 Kahan compensation for floating-point sums

Checking probability normalisation, calculating next-state marginals, and calculating expected reward all require adding terms. The numerical question is how to reduce the rounding loss from repeated addition while leaving the supplied probabilities and rewards unchanged. The code uses **Kahan compensated summation**, whose update is given by Goldberg's treatment of floating-point summation. [Goldberg, “Errors In Summation” and Theorem 8.](#)

14.3.1 Why ordinary addition loses information

For this numerical discussion, assume IEEE 754 binary64 `double` arithmetic with rounding to nearest, ties to even, and without compiler reassociation that treats floating-point addition as associative. Let $\text{fl}(z)$ denote the stored floating-point rounding of the exact real result z .

Given finite summands $\text{term}_1, \dots, \text{term}_N$, where N is their number, the exact desired sum is $\sum_{k=1}^N \text{term}_k$. A straightforward loop starts from $\text{sum}_0 = 0$ and stores

$$\text{sum}_k = \text{fl}(\text{sum}_{k-1} + \text{term}_k).$$

Each addition rounds before the next iteration. A small positive term_k may therefore leave the stored sum unchanged. Rounding can also increase the stored result: it is not always a loss in the downward direction.

Example: Losing two small increments

This example shows how two individually lost increments can have a representable total. Near 1, consecutive binary64 values above and including 1 are separated by 2^{-52} . This follows from the 53-bit significand: one leading bit and 52 fractional binary places at exponent zero. Let $h := 2^{-53}$, half that spacing. The exact value $1 + h$ is halfway between 1 and $1 + 2^{-52}$, and ties-to-even rounds it to 1. Repeating ordinary addition gives

$$\text{fl}(\text{fl}(1 + h) + h) = 1,$$

even though the exact total $1 + 2h = 1 + 2^{-52}$ is representable.

14.3.2 The four operations in the implementation

The following accumulation excerpt shows the initial values and the update for each probability:

```
double probability_sum = 0.0;
double compensation = 0.0;

for (const OutcomeProbability & outcome : outcomes) {
    const double corrected_probability =
        outcome.probability - compensation;
    const double updated_sum =
        probability_sum + corrected_probability;

    compensation =
        (updated_sum - probability_sum) - corrected_probability;
    probability_sum = updated_sum;
}
```

For one iteration, sum_{old} and $\text{correction}_{\text{old}}$ are the values of `probability_sum` and `compensation` before the update; both are initialised to zero before the loop. Let `probability` denote the current `outcome.probability`.

The corrected increment is `corrected_probability`, sum_{new} is `updated_sum`, and $\text{correction}_{\text{new}}$ is the replacement value of `compensation`. The rounded calculations are

$$\begin{aligned}\text{increment} &= \text{fl}(\text{probability} - \text{correction}_{\text{old}}), \\ \text{sum}_{\text{new}} &= \text{fl}(\text{sum}_{\text{old}} + \text{increment}), \\ \text{correction}_{\text{new}} &= \text{fl}(\text{fl}(\text{sum}_{\text{new}} - \text{sum}_{\text{old}}) - \text{increment}).\end{aligned}$$

Finally, `probability_sum = updated_sum` stores sum_{new} for the next iteration; it performs no further addition. The correction must be calculated first, while `probability_sum` still holds sum_{old} . Updating it too soon would destroy the information needed by `updated_sum - probability_sum`.

Why subtract the correction? Let rounding error mean the stored addition result minus the exact addition result. Thus

$$\text{sum}_{\text{new}} = \text{sum}_{\text{old}} + \text{increment} + \text{rounding error}.$$

Temporarily suppose the two subtractions used to recover that error introduce no further rounding. Subtracting the old sum, then the increment, gives

$$\begin{aligned}\text{sum}_{\text{new}} - \text{sum}_{\text{old}} &= \text{increment} + \text{rounding error}, \\ (\text{sum}_{\text{new}} - \text{sum}_{\text{old}}) - \text{increment} &= \text{rounding error}.\end{aligned}$$

Under those simplifying conditions, $\text{correction}_{\text{new}} = \text{rounding error}$. The next iteration subtracts this correction from the next input probability. If the previous addition rounded downward, the correction is negative, so subtracting it increases the next increment. If it rounded upward, the correction is positive, so subtracting it reduces the next increment. Real subtraction also rounds, so the code computes a residual estimate rather than a universally exact error.

This explains why the correction is applied to the **next summand** rather than by repeatedly trying to add a tiny residual directly to the same large sum. The correction can combine with the next input into an increment large enough to affect the stored result. The supplied `outcome.probability` is never assigned to or changed.

14.3.3 A complete binary64 trace

Use the earlier sequence $(1, h, h)$ to see how the small amount is recovered. This is an addition example, not a normalised probability row. Starting with both the sum and correction at zero, the four-operation update gives:

Input	Corrected increment	New sum	New correction
1	1	1	0
h	h	1	$-h$
h	$2h$	$1 + 2h$	0

For the second input, the new sum rounds to 1, so the correction is $(1 - 1) - h = -h$. For the third input, the corrected increment is $h - (-h) = 2h = 2^{-52}$; that increment is representable at the scale of 1. The final stored value is therefore `1.0000000000000002`, whereas the ordinary loop returns 1.

The example also shows why `compensation` should not be described as “the total error so far” or “a positive amount that was rounded up”. It is a signed correction passed between consecutive updates, and its sign is essential.

14.3.4 Validation remains a tolerance test

After summing the records, the implementation checks:

```
if (std::abs(probability_sum - 1.0) > probability_sum_tolerance) {
    throw FiniteMdpException{
        FiniteMdpError::invalid_probability_sum,
        "The outcome probabilities do not sum to one."};
}
```

Here `probability_sum_tolerance` is the dimensionless tolerance ε_p , with $0 < \varepsilon_p \leq 10^{-12}$. The call to `std::abs` calculates the absolute difference from one. If $\hat{q}$ denotes the computed sum, the accepted condition is $|\hat{q} - 1| \leq \varepsilon_p$. The comparison uses `>`, so equality with the tolerance is accepted. This does not divide the probabilities by their sum or force exact normalisation.

Kahan compensation reduces addition error; it does not make every sum exact, remove input or multiplication rounding, prevent overflow, or guarantee the same last bits under every reordering. Compiler options that allow algebraic reassociation, such as `fast-math` modes, can invalidate the residual calculation. Parentheses and the evaluation order matter even though the residual would be zero in exact real arithmetic.

14.4 Marginal probabilities and expected reward

Given a validated joint distribution, the two queries produce the probability of each next state and the expected one-step reward. The record-based forms of the equations from Section 13.5 determine which terms each loop accumulator receives.

14.4.1 One accumulator per next state

For a next state $s' \in \mathcal{S}$, marginalisation sums over all reward values. Since omitted events have zero probability, the sum can be written using the stored records:

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = \sum_{\substack{k \in \{1, \dots, K\} \\ s'_k = s'}} q_k.$$

The output is the row

$$[p(s_1 \mid s, a) \quad \dots \quad p(s_n \mid s, a)] \in \mathbb{R}^{1 \times n}.$$

```
std::vector<double> JointOutcomeDistribution::next_state_probabilities() const
{
    std::vector<double> probabilities(state_count_, 0.0);
    std::vector<double> compensations(state_count_, 0.0);

    for (const OutcomeProbability & outcome : outcomes_) {
        const std::size_t next_state_index = outcome.next_state.value();

        const double corrected_probability =
            outcome.probability - compensations[next_state_index];
        const double updated_probability =
            probabilities[next_state_index] + corrected_probability;

        compensations[next_state_index] =
            (updated_probability - probabilities[next_state_index]) -
            corrected_probability;
        probabilities[next_state_index] = updated_probability;
    }

    return probabilities;
}
```

A `std::vector<double>` has no built-in row/column orientation; its declared interpretation here is a row in the state order. If `next_state_index` has value $j \in \{0, \dots, n - 1\}$, then `probabilities[next_state_index]` accumulates $p(s_{j+1} \mid s, a)$.

Both vectors start with `state_count_` zero entries. The next-state index chooses which next-state accumulator receives q_k .

Each next state needs its own correction because it has its own running sum and rounding history. Applying a correction for next state s_1 to an addition for s_2 would mix two different marginal probabilities. Initial zero entries also represent next states with no stored outcome from this pair.

14.4.2 One accumulator for the expected reward

The expected-reward definition weights each possible reward by its joint probability. Replacing the sum over all events by the positive support gives

$$\bar{r}(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} r p(s', r \mid s, a) = \sum_{k=1}^K r_k q_k.$$

All products contribute to one scalar, so this calculation uses one sum and one correction:

```
double JointOutcomeDistribution::expected_reward() const noexcept
{
    double expected_reward = 0.0;
    double compensation = 0.0;

    for (const OutcomeProbability & outcome : outcomes_) {
        const double weighted_reward = outcome.reward * outcome.probability;
        const double corrected_weighted_reward = weighted_reward - compensation;
        const double updated_expected_reward =
            expected_reward + corrected_weighted_reward;

        compensation =
            (updated_expected_reward - expected_reward) - corrected_weighted_reward;
        expected_reward = updated_expected_reward;
    }

    return expected_reward;
}
```

`weighted_reward` is the floating-point product corresponding to $r_k q_k$. `corrected_weighted_reward` is the Kahan increment, and `expected_reward` is the running approximation to $\bar{r}(s, a)$. Since probabilities are dimensionless, every accumulator here has the same reward scale as r_k . Unlike probability accumulation, the summands can be negative. Compensation is applied to the products after they have been rounded; it does not recover rounding lost in multiplication.

Example: Querying two rewards for the same next state

This example shows how the two queries use the same joint records differently:

```
const auto distribution = JointOutcomeDistribution::from_outcomes(
    3U,
    {{StateIndex{0}, -1.0, 0.2},
     {StateIndex{1}, 4.0, 0.3},
     {StateIndex{1}, 5.0, 0.5}},
    1.0e-12);

const auto probabilities = distribution.next_state_probabilities();
const double reward = distribution.expected_reward();
```

The factory receives three records in a three-state domain. The two calls calculate `probabilities` and `reward` from the accepted distribution. Independently summing the probabilities by next state gives

$$\begin{bmatrix} 0.2 & 0.3 + 0.5 & 0 \end{bmatrix} = \begin{bmatrix} 0.2 & 0.8 & 0 \end{bmatrix}.$$

Weighting each event's reward by its probability gives

$$(-1)(0.2) + (4)(0.3) + (5)(0.5) = -0.2 + 1.2 + 2.5 = 3.5.$$

The returned vector therefore combines both events reaching state index one, while the scalar calculation retains their different reward values.

14.4.3 Finite inputs do not guarantee a finite computed reward

`expected_reward()` does not check the accumulated result for finiteness. Finite input rewards alone do not guarantee a finite computed result.

For exactly normalised nonnegative probabilities and $R_{\max} := \max_k |r_k|$, the mathematical bound is

$$\left| \sum_k r_k q_k \right| \leq \sum_k |r_k| q_k \leq R_{\max} \sum_k q_k = R_{\max}.$$

Example: Overflow despite finite input rewards

Tolerance-based acceptance can allow a stored sum slightly above one. The following call shows why even that small excess matters at the largest finite reward magnitude:

```

#include <limits>

const double maximum = std::numeric_limits<double>::max();
const auto extreme = JointOutcomeDistribution::from_outcomes(
    2U,
    {{StateIndex{0}, maximum, 0.5},
     {StateIndex{1}, maximum, 0.5000000000005}},
    1.0e-12);

const double result = extreme.expected_reward();

```

`std::numeric_limits<double>::max()` supplies the largest finite value of the scalar type, stored here in `maximum`. Both events have finite rewards and valid individual probabilities, and the sum excess is below the supplied tolerance. Their weighted sum nevertheless exceeds the finite range, so `result` is positive infinity.

Kahan compensation does not increase the floating-point range. A reward scale with numerical headroom is necessary, and a caller requiring a finite result must check it rather than infer it from input finiteness.

14.5 Constructing the complete finite MDP

An outcome distribution specifies the possible responses to one decision. To represent the whole MDP, we must attach each distribution to its current state and action, collect the feasible pairs, and supply the absorbing continuation at terminal states.

Collect the environment data into the **finite-MDP tuple**

$$\mathcal{M} := (\mathcal{S}, \mathcal{A}, \{\mathcal{A}(s)\}_{s \in \mathcal{S}}, \mathcal{R}, p, \mathcal{S}_{\text{term}}),$$

where:

- $\mathcal{S}$ is the finite state set.
- $\mathcal{A}$ is the complete finite action set, including the bookkeeping action when terminal states exist.
- $\{\mathcal{A}(s)\}_{s \in \mathcal{S}}$ specifies the nonempty feasible action set $\mathcal{A}(s) \subseteq \mathcal{A}$ for every state.
- $\mathcal{R} \subset \mathbb{R}$ is the finite set of possible reward values, including zero for the absorbing continuation when terminal states exist.
- p is the time-homogeneous joint transition–reward law: $p(s', r \mid s, a)$ assigns a probability to next state s' and reward r for each feasible current pair (s, a) .
- $\mathcal{S}_{\text{term}} \subseteq \mathcal{S}$ is the declared terminal set.

This tuple describes the environment. A policy and discount factor are specified separately for control and value calculations. The feasible sets and joint law are the objects defined in Section 13.1 and Section 13.5.

14.5.1 Attaching a distribution to its current pair

Mathematically, one row associates a feasible pair (s, a) with the conditional distribution $p(s', r \mid s, a)$. The current pair is fixed for that row; the next state and reward vary across its outcomes.

```
struct StateActionDistribution
{
    StateIndex state;
    ActionIndex action;
    JointOutcomeDistribution outcome_distribution;
};
```

`StateActionDistribution` has three fields: `state` stores the current-state index, `action` stores the action index, and `outcome_distribution` owns the possible joint outcomes. This is why an individual `OutcomeProbability` needs only a next-state index, reward, and probability: the enclosing row supplies the conditioning pair.

14.5.2 Task actions and the absorbing extension

Define the **task-action set** $\mathcal{A}_{\text{task}}$ as the finite set of caller-declared action labels for task decisions, excluding bookkeeping after termination. Its size is $m_{\text{task}} := |\mathcal{A}_{\text{task}}|$.

Let $\mathcal{S}_{\text{term}} \subseteq \mathcal{S}$ be the declared terminal set. If it is nonempty, introduce one fresh action label $a_{\perp} \notin \mathcal{A}_{\text{task}}$ for bookkeeping after termination. The symbol $\perp$ marks that artificial role. The complete action space is

$$\mathcal{A} = \begin{cases} \mathcal{A}_{\text{task}}, & \mathcal{S}_{\text{term}} = \emptyset, \\ \mathcal{A}_{\text{task}} \cup \{a_{\perp}\}, & \mathcal{S}_{\text{term}} \neq \emptyset. \end{cases}$$

The fresh label adds exactly one element, regardless of the number of terminal states. Therefore the complete action count $m := |\mathcal{A}|$ satisfies

$$m = \begin{cases} m_{\text{task}}, & \mathcal{S}_{\text{term}} = \emptyset, \\ m_{\text{task}} + 1, & \mathcal{S}_{\text{term}} \neq \emptyset. \end{cases}$$

For each terminal state $s_{\dagger} \in \mathcal{S}_{\text{term}}$, the absorbing extension is

$$\mathcal{A}(s_{\dagger}) = \{a_{\perp}\}, \quad p(s_{\dagger}, 0 \mid s_{\dagger}, a_{\perp}) = 1.$$

For a nonterminal state s , its feasible set is a nonempty subset of $\mathcal{A}_{\text{task}}$. This separates a global action label from a feasible decision in a particular state. With two task actions and two terminal states, for example, the complete space contains three action labels: the two real actions and one shared bookkeeping action. The extension assigns zero reward to the generated outgoing terminal responses without changing any task reward, including rewards on entering a terminal state.

14.5.3 The model class

```
#include <functional>
#include <optional>

class FiniteMdp
{
private:
    FiniteMdp(
        std::size_t state_count, std::size_t task_action_count,
        std::vector<StateActionDistribution> rows,
        std::vector<StateIndex> terminal_states);

    std::size_t state_count_;
    std::size_t task_action_count_;
    std::vector<StateActionDistribution> rows_;
    std::vector<StateIndex> terminal_states_;

public:
    [[nodiscard]] static FiniteMdp from_rows(
        std::size_t state_count, std::size_t task_action_count,
        std::vector<StateActionDistribution> rows,
        std::vector<StateIndex> terminal_states = {});

    [[nodiscard]] std::size_t state_count() const noexcept;

    [[nodiscard]] std::size_t task_action_count() const noexcept;

    [[nodiscard]] std::size_t action_count() const noexcept;

    [[nodiscard]] std::optional<ActionIndex> bookkeeping_action() const noexcept;

    [[nodiscard]] std::span<const StateActionDistribution> rows() const noexcept;
```

```

[[nodiscard]]
std::optional<std::reference_wrapper<const JointOutcomeDistribution>>
find_outcome_distribution(StateIndex state, ActionIndex action) const;

[[nodiscard]] bool is_terminal(StateIndex state) const;
};

```

`FiniteMdp` owns four pieces of data. `state_count_` represents n , `task_action_count_` represents m_{task} , `rows_` stores the feasible state–action distributions, and `terminal_states_` stores terminal indices. No independent complete-action count is stored: it can be derived from the task-action count and whether any terminals exist.

The public factory `from_rows` receives those four inputs. The final parameter’s default `{}` supplies an empty terminal vector when the caller omits it. The count queries return scalar sizes; `rows()` exposes a read-only span; `find_outcome_distribution` returns an optional read-only reference wrapper for one stored distribution; and `is_terminal` returns a Boolean membership answer. The type `std::optional<ActionIndex>` allows `bookkeeping_action()` to return an action index or no value. The distribution lookup instead contains a wrapper that refers to model-owned data; its access and lifetime rules are explained in Section 14.6.

The private constructor is the point where accepted local data becomes owned model storage. *The factory accepts only rows originating at nonterminal states and generates outgoing terminal rows itself.* It checks that supplied distributions use the same state-space size and that each current state–action key is unique. Thus the model establishes relationships among already-checked distributions rather than repeating every probability calculation.

14.5.4 Completing and ordering the model rows

Construction adds the absorbing continuation to the supplied nonterminal rows, then orders the complete collection for lookup. The stages are:

1. Check the state count and terminal indices; sort the terminal indices.
2. Check the task-action count and supplied nonterminal rows.
3. Generate one zero-reward self-transition per terminal state.
4. Check that every state has a feasible row.
5. Sort the completed rows by current-state index, then action index.
6. Return the model owning the rows and terminal indices.
7. If any check fails, raise an exception without returning a model.

The following production excerpt combines terminal-row generation and final row ordering inside the factory signature. Validation branches, per-state feasible-row counting, and production comments are omitted; the comments below are teaching annotations. This is not a complete replacement function.

```

#include <algorithm>

FiniteMdp FiniteMdp::from_rows(
    std::size_t state_count, std::size_t task_action_count,
    std::vector<StateActionDistribution> rows,
    std::vector<StateIndex> terminal_states)
{
    // Omitted: validate the state count and terminal indices.
    std::ranges::sort(terminal_states);

    // Omitted: validate the task-action count and supplied nonterminal rows.

    // Generate the absorbing continuations.
    for (const StateIndex state : terminal_states) {
        rows.push_back(StateActionDistribution{
            state,
            ActionIndex{task_action_count},
            JointOutcomeDistribution::from_outcomes(
                state_count, {{state, 0.0, 1.0}}, 1.0e-12),
        });
    }

    // Omitted: reject the model if any state has no feasible row.

    // Order the completed rows for lookup.
    std::ranges::sort(
        rows, [](
            const StateActionDistribution & left,
            const StateActionDistribution & right) {
            if (left.state != right.state) {
                return left.state < right.state;
            }

            return left.action < right.action;
        });

    return FiniteMdp{
        state_count,
        task_action_count,
        std::move(rows),
        std::move(terminal_states),
    };
}

```


14.5.5 Generating terminal rows

The initial `std::ranges::sort(terminal_states)` uses the comparison already defined for `StateIndex`. Sorting changes the vector's order, not the declared terminal set. Thus different input orders lead to the same stored terminal order.

The range-based loop visits the terminal vector. `rows.push_back` appends a complete row to the row vector; appending there does not disturb traversal of the separate terminal vector. The generated row's current-state and next-state indices both equal `state`, while its reward and probability are exactly zero and one.

The expression `ActionIndex{task_action_count}` assigns the bookkeeping label the first index after all task-action indices. Every terminal uses that same action index. The supplied task-action indices are integers from zero through `task_action_count - 1`. If `task_action_count` is zero, there are no task-action indices. The generated index is not a caller-supplied task action.

After completion, every state needs at least one feasible row. A terminal has its generated row; a nonterminal needs a supplied row. Zero task actions are consequently possible only when all states are terminal.

The action count uses `std::size_t`, an unsigned integer type with a largest representable value. Adding one to that largest representable value produces zero under unsigned integer arithmetic; this is called **wraparound**. The model would then incorrectly report zero actions instead of the task actions plus the bookkeeping action. To prevent this, the factory rejects construction when terminal states are declared and `task_action_count` already equals that largest value, before generating the bookkeeping action.

14.5.6 Establishing the row order

The second sorting operation, `std::ranges::sort(rows, ...)`, runs after terminal completion and the feasible-row check. Its comparator orders rows by current-state index first, then by action index when the states match. This lexicographic ordering puts all rows for a state together and orders the actions within that group.

Sorting happens before the final `return` transfers the local vectors to the model. The receiving constructor stores them as follows:

```
FiniteMdp::FiniteMdp(
    std::size_t state_count, std::size_t task_action_count,
    std::vector<StateActionDistribution> rows,
    std::vector<StateIndex> terminal_states)
: state_count_{state_count},
  task_action_count_{task_action_count},
  rows_{std::move(rows)},
  terminal_states_{std::move(terminal_states)}
```

```
{
}
```

The initialiser list stores the two counts and transfers ownership of the two vectors. The finished model does not borrow those local vectors.

14.5.7 Complete-action and bookkeeping queries

```
std::size_t FiniteMdp::action_count() const noexcept
{
    return task_action_count_ + (terminal_states_.empty() ? 0U : 1U);
}

std::optional<ActionIndex> FiniteMdp::bookkeeping_action() const noexcept
{
    if (terminal_states_.empty()) {
        return std::nullopt;
    }

    return ActionIndex{task_action_count_};
}
```

`action_count()` adds zero for an empty terminal set and one otherwise. This implements the piecewise formula for m without changing the stored task-action count.

In the optional-returning query, `std::nullopt` represents absence of a bookkeeping action. Otherwise, returning `ActionIndex{task_action_count_}` supplies the shared generated index. Presence is derived from the terminal declarations, so there is no extra stored copy of the bookkeeping index.

14.6 Finding a distribution and checking terminal membership

A distribution query receives $s \in \mathcal{S}$ and $a \in \mathcal{A}$. If $a \in \mathcal{A}(s)$, it returns access to the stored joint distribution $p(s', r \mid s, a)$; otherwise, it reports absence without inventing dynamics. Indices outside the state or global action domain are errors, not ordinary absence. Terminal membership receives a state and decides whether it belongs to the declared terminal set. The sorted storage supports both operations.

14.6.1 Searching for a state–action row

The following production excerpt includes the function signature and lookup logic. It omits the initial state- and action-index range checks and the production comments; the teaching comment marks where those checks occur. This excerpt is not a complete replacement function.

```
std::optional<std::reference_wrapper<const JointOutcomeDistribution>>
FiniteMdp::find_outcome_distribution(
    const StateIndex state, const ActionIndex action) const
{
    // Omitted: reject state or action indices outside the model's domains.
    const std::pair<StateIndex, ActionIndex> target{state, action};

    const auto row = std::lower_bound(
        rows_.begin(), rows_.end(), target,
        [](
            const StateActionDistribution & candidate,
            const std::pair<StateIndex, ActionIndex> & key) {
                if (candidate.state != key.first) {
                    return candidate.state < key.first;
                }

                return candidate.action < key.second;
            });

    if (row == rows_.end() || row->state != state || row->action != action) {
        return std::nullopt;
    }

    return std::cref(row->outcome_distribution);
}
```

`std::optional<T>` contains a value of type `T` or is empty; `std::nullopt` represents the empty case. Here the contained value is a `std::reference_wrapper<const JointOutcomeDistribution>`: a small object that refers to an existing distribution without copying it or allowing modification through the wrapper. `std::cref(row->outcome_distribution)` creates that read-only wrapper for the matching row. The function therefore returns either borrowed access to the exact distribution or an empty optional. The next subsection explains how to access the result and keep the borrowed reference valid.

`target` holds the requested state and action as one pair. `std::lower_bound` receives four arguments: the beginning of the range, the excluded end of the range, the requested value, and the comparison operation. In this call:

- `rows_.begin()` identifies the first row.

- `rows_.end()` identifies the position after the last row; it is not a row that may be read.
- `target` is the requested pair.
- The lambda beginning `[]` is the comparator.

The comparator receives a stored `StateActionDistribution` as `candidate` and the requested pair as `key`. Their types differ because the stored row contains a distribution as well as its indices, while the search key needs only the indices.

Within the pair, `key.first` is the state and `key.second` is the action. The first `return` handles different states; equal states reach the action comparison. The result means “this candidate comes before the key”, not “this candidate equals the key”.

Since the rows use this same ordering, comparison results form a run of true values followed by a run of false values. The search returns one iterator to the first false position, or the end iterator if all candidates precede the key. Binary search on this vector uses logarithmically many comparisons and position steps. [C++ standard draft, binary-search algorithms](#).

Example: Searching for a missing state–action pair

This trace shows why the search result still needs an equality check. The target is $(0, 1)$: current-state index 0 and action index 1. We want the stored outcome distribution for that exact pair. The stored rows have these keys:

Row index:	0	1	2
Stored key:	$(0, 0)$	$(0, 2)$	$(1, 0)$
Target:	$(0, 1)$ – not stored		

`lower_bound` finds the **first key that is not smaller than the target**, rather than guaranteeing an exact match. One search trace is:

1. **Compare the middle key $(0, 2)$ with $(0, 1)$.** Their state indices match, so compare actions: $2 < 1$ is false. This row could be the first key that is not smaller, but an earlier row might also qualify. Search left.
2. **Compare $(0, 0)$ with $(0, 1)$.** Their state indices match, and $0 < 1$ is true. This key is smaller than the target, so discard it and search to its right.
3. **The search ends at $(0, 2)$.** It is the first key that is not smaller than $(0, 1)$.

The final equality check then asks: **is $(0, 2)$ exactly $(0, 1)$?** No—the action indices differ. Therefore, the requested state–action pair is absent, and the function returns `std::nullopt`. Both indices are valid; only their combination is infeasible.

The `||` operator stops evaluating as soon as one operand is true. If `row == rows_.end()`, neither member access is evaluated, so the code never dereferences the end iterator. Otherwise, checking both state and action establishes an exact match. A failed match returns an empty optional; an exact match returns a wrapper referring to the selected distribution.

14.6.2 Accessing an optional borrowed distribution

The following access pattern assumes a previously constructed model and current-state and action indices within its domains:

```
const auto result = mdp.find_outcome_distribution(current, action);

if (result.has_value()) {
    const JointOutcomeDistribution & selected = result->get();

    // Alternative spellings for the same borrowed access:
    const JointOutcomeDistribution & also_selected = (*result).get();
    const JointOutcomeDistribution & checked_selected = result.value().get();
}
```

The return type combines two standard-library objects:

- `std::optional<T>` contains a value of type `T` or is empty. `std::nullopt` supplies the empty case. `has_value()` checks presence; `.value()` accesses the contained value and throws `std::bad_optional_access` if empty. Dereferencing with `*` or accessing a member with `->` requires a nonempty optional.
- `std::reference_wrapper<T>` is a copyable object that refers to an existing `T`. Its `.get()` operation returns that object's reference. Copying the wrapper does not copy the referenced object or extend its lifetime.
- `std::cref(object)` creates a read-only reference wrapper. Here it refers to the stored `JointOutcomeDistribution`, and the returned optional contains that wrapper without copying the distribution. `<functional>` declares the wrapper and `std::cref;` `<optional>` declares the optional type and `std::nullopt`.

C++20 does not allow an optional to contain a reference directly. The wrapper supplies an object that the optional can contain while still borrowing the distribution. The model's stored distribution must remain alive and its reference valid throughout use; destroying or replacing the owning storage invalidates the borrowed access.

Example: Reading a distribution when present

This one-state model has one feasible action with a certain reward of two:

```

const auto distribution = JointOutcomeDistribution::from_outcomes(
    1U, {{StateIndex{0}, 2.0, 1.0}}, 1.0e-12);
const auto mdp = FiniteMdp::from_rows(
    1U, 1U, {{StateIndex{0}, ActionIndex{0}, distribution}});

const auto result =
    mdp.find_outcome_distribution(StateIndex{0}, ActionIndex{0});

if (result.has_value()) {
    const double reward = result->get().expected_reward();
}

```

`result` is nonempty because the pair was supplied. In `result->get()`, `->` accesses the wrapper inside the optional, and `get()` returns the referenced distribution. Thus `get()` belongs to the wrapper, not the optional. The expected reward is $1 \times 2 = 2$, which is the value of `reward`. The model remains alive throughout this access.

14.6.3 Reading the terminal declaration

```

bool FiniteMdp::is_terminal(StateIndex state) const
{
    if (state.value() >= state_count_) {
        throw FiniteMdpException{
            FiniteMdpError::invalid_current_state,
            "The query contains an invalid state index."};
    }

    return std::ranges::binary_search(terminal_states_, state);
}

```

The initial `if` separates an invalid index from a valid nonterminal state. The former raises an exception; the latter may correctly return false. `std::ranges::binary_search` then asks whether the index occurs in the sorted terminal vector. It returns a Boolean rather than the candidate iterator used by the distribution lookup.

Membership follows the declaration, not the transition pattern. A self-loop alone does not make a state terminal.

Example: A nonterminal state with a self-loop

Take a model with state indices 0, 1, 2, terminal declarations $\{2, 0\}$, and a supplied zero-reward self-loop at state index 1. Terminal-membership queries in increasing state order return true, false, true. The middle answer remains false because state

index 1 was not declared terminal, even though its transition is a self-loop.

14.7 Model usage

The individual types now have defined roles. A complete example shows how construction combines them and how a query retrieves information without executing an environment transition.

14.7.1 Construction from individual events

Take two states $\mathcal{S} = \{s_1, s_2\}$, one task action $\mathcal{A}_{\text{task}} = \{a_1\}$, and terminal set $\mathcal{S}_{\text{term}} = \{s_2\}$. From s_1 , action a_1 either stays at s_1 with reward -1 and probability $1/4$, or reaches s_2 with reward 4 and probability $3/4$.

```
const StateIndex current{0};
const StateIndex terminal{1};
const ActionIndex action{0};

const OutcomeProbability stay{current, -1.0, 0.25};
const OutcomeProbability arrive{terminal, 4.0, 0.75};

const auto distribution = JointOutcomeDistribution::from_outcomes(
    2U, {stay, arrive}, 1.0e-12);

const StateActionDistribution row{current, action, distribution};

const auto mdp = FiniteMdp::from_rows(
    2U, 1U, {row}, {terminal});
```

`current`, `terminal`, and `action` name indices in the state and task-action domains. `stay` and `arrive` turn the two possible events into records. The brace group `{stay, arrive}` supplies the records for the distribution factory's vector parameter. The resulting `distribution` owns that sequence.

The aggregate `row` adds the conditioning pair: it combines `current`, `action`, and the whole distribution. Finally, `{row}` supplies the model's nonterminal rows and `{terminal}` supplies its terminal declarations. The model factory adds the absorbing row with generated action index one.

These are separate values, not a chain of borrowed references. Constructing `row` from the named `distribution` copies the distribution; supplying `{row}` copies the row into the fact-

ory's input. The factory moves its local vectors into the model. The final `mdp` therefore does not depend on the lifetime of the earlier local records or row.

14.7.2 Queries produce calculated values

```
const auto result = mdp.find_outcome_distribution(current, action);

if (result.has_value()) {
    const JointOutcomeDistribution & selected = result->get();
    const std::vector<double> probabilities =
        selected.next_state_probabilities();
    const double reward = selected.expected_reward();
}
```

The supplied pair makes `result` nonempty. Inside the presence check, `selected` borrows the matching distribution inside `mdp`. The next two calls calculate an owning vector `probabilities` and a scalar `reward`. Applying the marginal and expectation formulas to the two supplied events gives

$$\begin{aligned} [p(s_1 \mid s_1, a_1) \quad p(s_2 \mid s_1, a_1)] &= [0.25 \quad 0.75], \\ \bar{r}(s_1, a_1) &= (-1)(0.25) + (4)(0.75) = 2.75. \end{aligned}$$

The first row corresponds to `probabilities` in state order, and the scalar corresponds to `reward`. The returned vector and scalar are independent of the model's storage even though the selected distribution reference is not.

Example: Querying the generated terminal row

Continuing the two-state example above, the generated row can be selected through the optional action query:

```
const auto bookkeeping = mdp.bookkeeping_action();

if (bookkeeping.has_value()) {
    const ActionIndex generated_action = *bookkeeping;
    const auto result =
        mdp.find_outcome_distribution(terminal, generated_action);
    if (result.has_value()) {
        const auto & absorbing = result->get();
        const double terminal_reward = absorbing.expected_reward();
    }
}
```

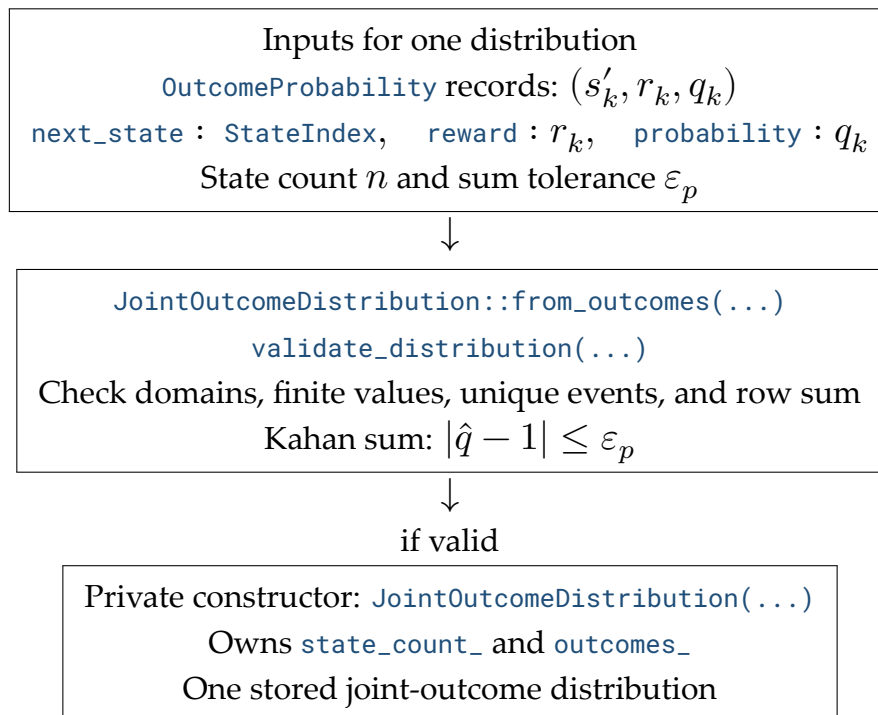

The first `has_value()` checks that the bookkeeping optional contains an index. Once this is true, `*bookkeeping` extracts that index without removing it. The second check establishes that the distribution lookup contains a wrapper before accessing it. Here `generated_action` contains one, `absorbing` refers to the generated terminal distribution, and `terminal_reward` is zero. Its next-state row is $\begin{bmatrix} 0 & 1 \end{bmatrix}$ because the next state remains s_2 .

14.8 How the classes and functions connect

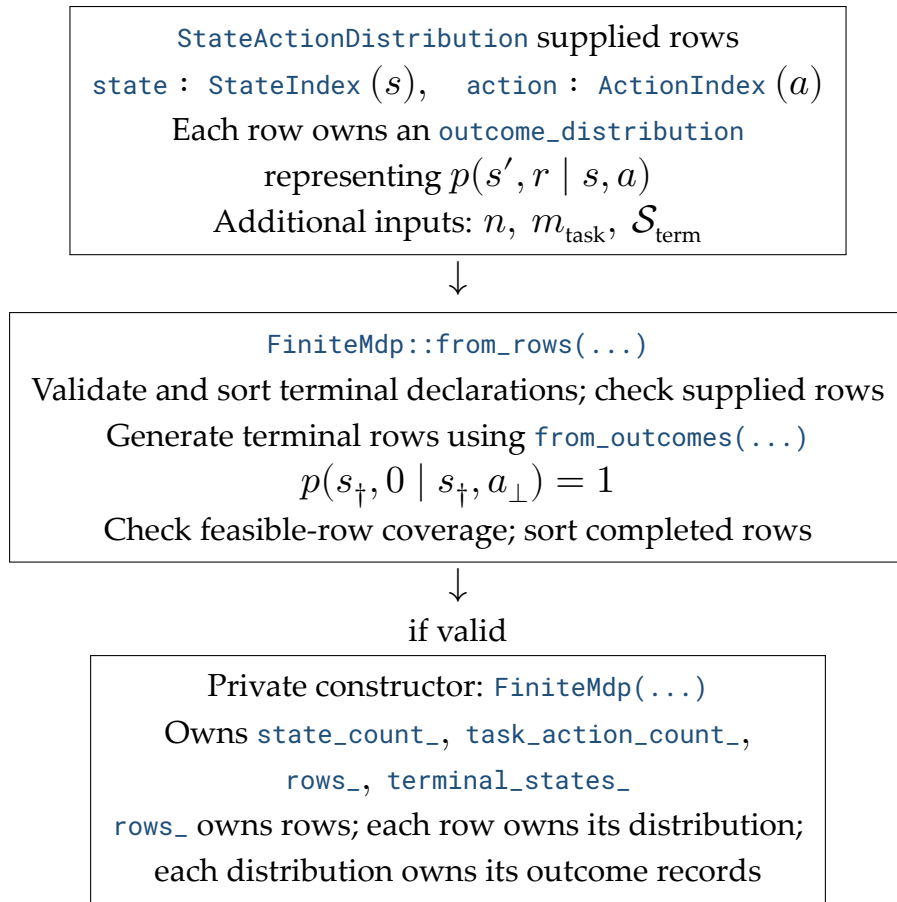
The diagrams separate construction from later queries. Arrows show inputs passed to an operation or results produced by it; the word **owns** identifies stored data, not another calculation. The model remains fixed during every query.

14.8.1 Constructing distributions and model rows

Typed indices identify states and actions. `StateIndex::value()` and `ActionIndex::value()` recover their integer indices; their comparison operators supply the order used for event keys, model rows, and terminal indices. An outcome record uses a next-state index, whereas a model row supplies the current-state and action indices.



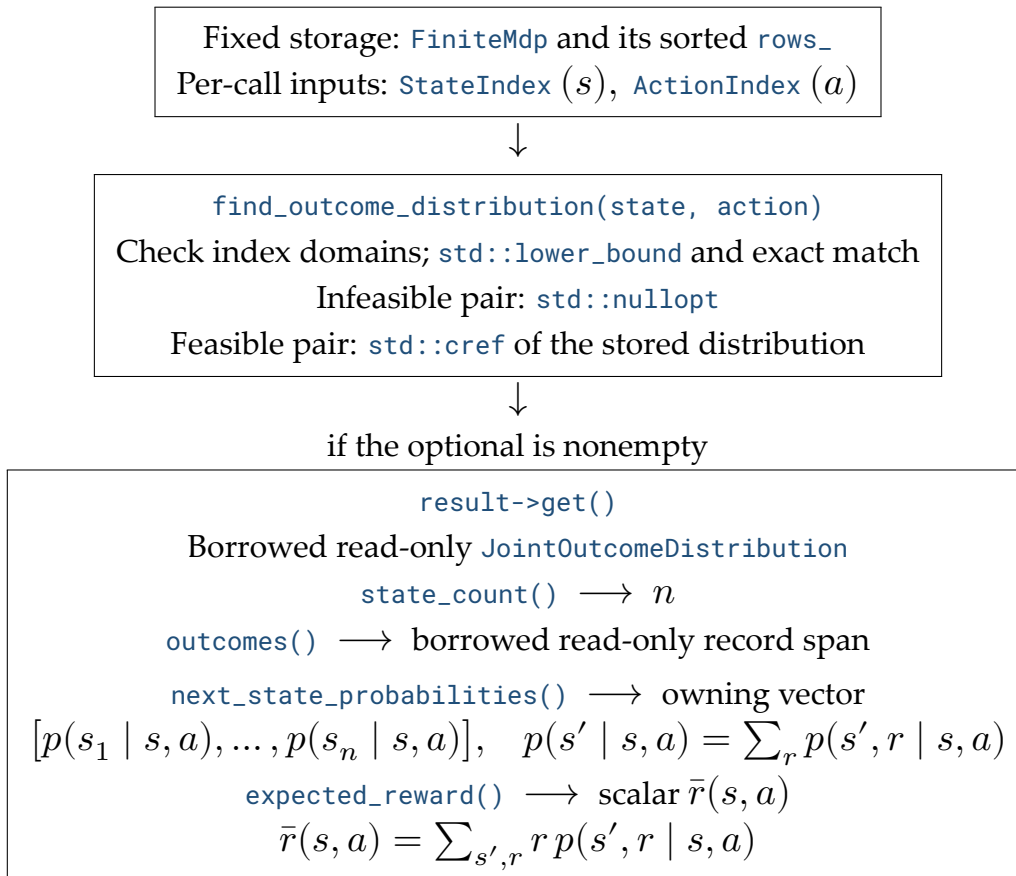
The distribution does not store its conditioning pair. Attaching a current state and action gives that distribution its role in the complete model:



At either factory, a failed validation raises `FiniteMdpException` with a `FiniteMdpError` category instead of producing an object. Construction does not renormalise accepted probabilities or generate missing nonterminal dynamics.

14.8.2 Looking up and evaluating a distribution

The model is fixed input. Each call supplies only a current-state index and an action index; the lookup borrows an existing distribution rather than constructing a new one.



Invalid lookup indices raise a domain exception rather than returning an empty optional. The marginal and reward methods use Kahan accumulation; they return calculated values, not borrowed storage. The wrapper and record span require valid owner storage, while the calculated vector and scalar do not. The expected-reward overflow caution in Section 14.4 still applies.

14.8.3 Inspecting the model itself

These methods read model structure without selecting an outcome distribution. Only terminal membership needs a per-call state index.

Fixed storage: `FiniteMdp`

<code>state_count()</code>	$\longrightarrow$	n
<code>task_action_count()</code>	$\longrightarrow$	m_{task}
<code>action_count()</code>	$\longrightarrow$	$m = \mathcal{A} $
<code>bookkeeping_action()</code>	$\longrightarrow$	optional index of $a_{\perp}$
<code>rows()</code>	$\longrightarrow$	borrowed read-only row span
<code>is_terminal(state)</code>	$\longrightarrow$	$s \in \mathcal{S}_{\text{term}}$

The bookkeeping optional is empty exactly when no terminal states are declared. Terminal membership uses the sorted declaration and rejects an invalid state index. None of these calls samples an outcome, chooses an action, or advances an environment.

14.9 Analytic examples

14.9.1 A task with uncertain progress

Consider three states in the order $\mathcal{S} = \{s_1, s_2, s_3\}$: **start**, **in progress**, and **finished**. Only s_3 is terminal. The task actions are $\mathcal{A}_{\text{task}} = \{a_1, a_2\}$, where a_1 means **attempt** and a_2 means **quit**. Both actions are feasible at the start, but only attempt is feasible in progress.

An attempt from the start can fail or make progress. Two outcomes reach the same in-progress state with different rewards. Quitting ends the task with reward zero; an attempt from the in-progress state completes it with reward six. The diagram uses the mathematical state and action labels, with `a_bot` denoting $a_{\perp}$. In C++, states s_1, s_2, s_3 have indices 0, 1, 2, and actions $a_1, a_2, a_{\perp}$ have indices 0, 1, 2.

Supplied task transitions (p = probability, r = reward):

```
s_1: start -- a_1: attempt --> +-- p=0.25, r=-2 --> s_1: start
                                +-- p=0.25, r= 1 --> s_2: in progress
                                +-- p=0.50, r= 3 --> s_2: in progress
```

```
s_1: start -- a_2: quit --> p=1, r=0 --> s_3: finished
```

```
s_2: in progress -- a_1: attempt --> p=1, r=6 --> s_3: finished
```

Generated terminal continuation:

```
s_3: finished -- a_bot --> p=1, r=0 --> s_3: finished
```

14.9.2 Calculating the expected results

For an attempt from the start, combine the two events reaching the in-progress state to obtain the next-state marginal:

$$\begin{aligned} p(s_1 \mid s_1, a_1) &= 0.25, \\ p(s_2 \mid s_1, a_1) &= 0.25 + 0.50 = 0.75, \\ p(s_3 \mid s_1, a_1) &= 0. \end{aligned}$$

The expected reward must retain the different rewards of the two progress events:

$$\bar{r}(s_1, a_1) = (-2)(0.25) + (1)(0.25) + (3)(0.50) = 1.25.$$

The remaining rows each contain one certain outcome. All expected results are:

Current state	Action	Next-state probability row	Expected reward
Start	Attempt	[0.25 0.75 0]	1.25
Start	Quit	[0 0 1]	0
In progress	Attempt	[0 0 1]	6
Finished	Bookkeeping	[0 0 1]	0

Construction produces three states, two task actions, three complete action labels, and four feasible rows. The reward six belongs to entering the finished state; the generated continuation contributes zero afterward.

14.9.3 Constructing the model

```
const StateIndex start{0};
const StateIndex in_progress{1};
const StateIndex finished{2};
const ActionIndex attempt{0};
const ActionIndex quit{1};

const auto attempt_from_start = JointOutcomeDistribution::from_outcomes(
    3U,
    {{start, -2.0, 0.25},
     {in_progress, 1.0, 0.25},
     {in_progress, 3.0, 0.50}},
    1.0e-12);
```

```

const auto quit_from_start = JointOutcomeDistribution::from_outcomes(
    3U, {{finished, 0.0, 1.0}}, 1.0e-12);

const auto complete_task = JointOutcomeDistribution::from_outcomes(
    3U, {{finished, 6.0, 1.0}}, 1.0e-12);

const auto mdp = FiniteMdp::from_rows(
    3U, 2U,
    {{start, attempt, attempt_from_start},
     {start, quit, quit_from_start},
     {in_progress, attempt, complete_task}},
    {finished});

```

`attempt_from_start` stores all three joint events, including both rewards for `in_progress`. The rows passed to `from_rows` attach each distribution to its current state and action. No terminal row is supplied: the factory generates it with bookkeeping action index two. No row is supplied for quitting from `in_progress`, so that pair remains infeasible.

14.9.4 Reading the model

```

const auto result = mdp.find_outcome_distribution(start, attempt);

if (result.has_value()) {
    const auto probabilities = result->get().next_state_probabilities();
    const double reward = result->get().expected_reward();
}

const auto unavailable =
    mdp.find_outcome_distribution(in_progress, quit);

const auto bookkeeping = mdp.bookkeeping_action();
if (bookkeeping.has_value()) {
    const auto terminal_result =
        mdp.find_outcome_distribution(finished, *bookkeeping);

    if (terminal_result.has_value()) {
        const auto terminal_probabilities =
            terminal_result->get().next_state_probabilities();
        const double terminal_reward =
            terminal_result->get().expected_reward();
    }
}

```

The first lookup is nonempty: `probabilities` is `[0.25 0.75 0]` and `reward` is `1.25`, matching the hand calculation. `unavailable` is empty because quit is a valid global action but is not feasible in the in-progress state. The bookkeeping query contains action index two; its terminal distribution yields `terminal_probabilities` equal to `[0 0 1]` and `terminal_reward` equal to zero.

The two events reaching the in-progress state remain distinct in the stored distribution but combine in the marginal vector. Terminal completion preserves the reward for finishing, and the optional lookup distinguishes an unavailable action from a stored transition.

References

- Sutton, R. S., and Barto, A. G. *Reinforcement Learning: An Introduction*, second-edition draft, 2014–2015. Chapter 3, especially Sections 3.4 and 3.6, supplies the absorbing-episode convention and the joint-kernel, marginal, and reward formulas. The draft's $r(s, a)$ is written $\bar{r}(s, a)$ here to distinguish an expectation from a realised reward.
- Bertsekas, D. P. *A Course in Reinforcement Learning*, second edition, 2026 editorial version. Section 1.3.1, printed pp. 28–29, supplies the state-dependent control-set and random-disturbance viewpoint. Its state x_k and control u_k play the roles of s and a ; its $U_k(x_k)$ is the feasible control set.
- Goldberg, D. “What Every Computer Scientist Should Know About Floating-Point Arithmetic.” *ACM Computing Surveys*, 23(1), 1991. Theorem 8 and “Errors In Summation”; reproduced in [Oracle’s Numerical Computation Guide](#).
- [C++ standard draft: binary-search algorithms](#), for search preconditions and return meanings.
- The companion’s code excerpts follow the [public headers](#), [compiled operations](#), and [focused tests](#) of the C++20 foundations library.